?

mitp

Hinweis des Verlages zum Urheberrecht und Digitalen Rechtemanagement (DRM)

Der Verlag räumt Ihnen mit dem Kauf des ebooks das Recht ein, die Inhalte im Rahmen des geltenden Urheberrechts zu nutzen. Dieses Werk, einschließlich aller seiner Teile, ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Verlages unzulässig und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen und Einspeicherung und Verarbeitung in elektronischen Systemen.

Der Verlag schützt seine ebooks vor Missbrauch des Urheberrechts durch ein digitales Rechtemanagement. Bei Kauf im Webshop des Verlages werden die ebooks mit einem nicht sichtbaren digitalen Wasserzeichen individuell pro Nutzer signiert.

Bei Kauf in anderen ebook-Webshops erfolgt die Signatur durch die Shopbetreiber. Angaben zu diesem DRM finden Sie auf den Seiten der jeweiligen Anbieter.

Neuerscheinungen, Praxistipps, Gratiskapitel,
Einblicke in den Verlagsalltag –
gibt es alles bei uns auf Instagram und Facebook

instagram.com/mitp_verlag

facebook.com/mitp.verlag

Inhaltsverzeichnis

Impressum

Vorwort zur ersten Auflage

Über den Fachkorrektor der deutschen Ausgabe

Einleitung

- Was ist Refactoring?
- Worum geht es in diesem Buch?
- Wer sollte dieses Buch lesen?
- Von anderen geschaffene Grundlagen
- Danksagungen

Kapitel 1: Refactoring: Ein erstes Beispiel

- 1.1 Der Ausgangspunkt
- 1.2 Anmerkungen zum ersten Programm
- 1.3 Der erste Schritt des Refactorings
- 1.4 Aufteilung der statement-Funktion
  - 1.4.1 Entfernen der Variable play
  - 1.4.2 Extrahieren der Berechnung von Treuepunkten
  - 1.4.3 Entfernen der Variable format
  - 1.4.4 Entfernen der Gesamtzahl der Treuepunkte
- 1.5 Status: Eine Menge verschachtelter Funktionen
- 1.6 Die Phasen Berechnung und Formatierung voneinander trennen
- 1.7 Status: Aufgeteilt in zwei Dateien (und Phasen)
- 1.8 Neuorganisation der Berechnungen nach Art
  - 1.8.1 Berechnen der Kosten einer Vorstellung
  - 1.8.2 Funktionen in die Klasse verschieben
  - 1.8.3 Polymorphes Verhalten der Klasse PerformanceCalculator
- 1.9 Status: Erzeugen der Daten mit der polymorphen Klasse PerformanceCalculator
- 1.10 Abschließende Überlegungen

Kapitel 2: Prinzipien des Refactorings

- 2.1 Refactoring: Definition

2.2 Die beiden Hüte
2.3 Gründe für ein Refactoring
2.3.1 Refactoring verbessert das Design der Software
2.3.2 Refactoring macht Software besser verständlich
2.3.3 Refactoring hilft mir, Bugs aufzuspüren
2.3.4 Refactoring hilft mir, schneller zu programmieren
2.4 Der richtige Zeitpunkt für ein Refactoring
2.4.1 Vorbereitendes Refactoring: Hinzufügen eines Features vereinfachen
2.4.2 Refactoring zwecks Verbesserung der Verständlichkeit des Codes
2.4.3 Refactoring als Abfallentsorgung
2.4.4 Geplantes Refactoring und spontanes Refactoring
2.4.5 Langfristiges Refactoring
2.4.6 Refactoring beim Code-Review
2.4.7 Wie sag ich's meinem Chef?
2.4.8 Wann sollte man kein Refactoring durchführen?
2.5 Probleme beim Refactoring
2.5.1 Verlangsamung der Entwicklung neuer Features
2.5.2 Code-Eigentümerschaft
2.5.3 Entwicklungszweige
2.5.4 Testen
2.5.5 Legacy Code
2.5.6 Datenbanken
2.6 Refactoring, Architektur und Yagni
2.7 Refactoring und der Softwareentwicklungsprozess
2.8 Refactoring und Performance
2.9 Ursprünge des Refactorings
2.10 Automatisiertes Refactoring
2.11 Weiterführende Literatur

Kapitel 3: Code-Smells: Schlechte Gerüche im Code

3.1 Rätselhafte Bezeichnung
3.2 Redundanter Code
3.3 Lange Funktion
3.4 Lange Parameterliste
3.5 Globale Daten
3.6 Veränderliche Daten
3.7 Divergierende Änderungen

3.8 Chirurgie mit der Schrotflinte
3.9 Feature-Neid
3.10 Datenklumpen
3.11 Obsession für elementare Datentypen
3.12 Wiederholte switch-Anweisungen
3.13 Schleifen
3.14 Träges Element
3.15 Spekulative Generalisierung
3.16 Temporäres Feld
3.17 Mitteilungsketten
3.18 Vermittler
3.19 Insiderhandel
3.20 Umfangreiche Klasse
3.21 Alternative Klassen mit unterschiedlichen Schnittstellen
3.22 Datenklasse
3.23 Ausgeschlagenes Erbe
3.24 Kommentare

Kapitel 4: Tests erstellen

4.1 Der Nutzen selbsttestenden Codes
4.2 Beispielcode zum Testen
4.3 Ein erster Test
4.4 Hinzufügen eines weiteren Tests
4.5 Ändern des Test-Fixtures
4.6 Austesten der Grenzen
4.7 Und noch viel mehr

Kapitel 5: Der Katalog

5.1 Format der Refactorings
5.2 Die Auswahl der Refactorings

Kapitel 6: Eine erste Zusammenstellung von Refactorings

6.1 Funktion extrahieren (Extract Function)
6.1.1 Motivation
6.1.2 Vorgehen
6.1.3 Beispiel: Keine Variablen außerhalb des Gültigkeitsbereichs
6.1.4 Beispiel: Verwendung lokaler Variablen

6.1.5 Beispiel: Zuweisung zu einer lokalen Variable
6.2 Funktion inline platzieren (Inline Function)
6.2.1 Motivation
6.2.2 Vorgehen
6.2.3 Beispiel
6.3 Variable extrahieren (Extract Variable)
6.3.1 Motivation
6.3.2 Vorgehen
6.3.3 Beispiel
6.3.4 Beispiel mit einer Klasse
6.4 Variable inline platzieren (Inline Variable)
6.4.1 Motivation
6.4.2 Vorgehen
6.5 Funktionsdeklaration ändern (Change Function Declaration)
6.5.1 Motivation
6.5.2 Vorgehen
6.5.3 Beispiel: Umbenennen einer Funktion (einfaches Vorgehen)
6.5.4 Beispiel: Umbenennen einer Funktion (Migrationsansatz)
6.5.5 Beispiel: Hinzufügen eines Parameters
6.5.6 Beispiel: Einen Parameter durch eine seiner Eigenschaften ersetzen
6.6 Variable kapseln (Encapsulate Variable)
6.6.1 Motivation
6.6.2 Vorgehen
6.6.3 Beispiel
6.7 Variable umbenennen (Rename Variable)
6.7.1 Motivation
6.7.2 Vorgehen
6.7.3 Beispiel
6.8 Parameterobjekt einführen (Introduce Parameter Object)
6.8.1 Motivation
6.8.2 Vorgehen
6.8.3 Beispiel
6.9 Funktionen zu einer Klasse vereinen (Combine Functions into Class)
6.9.1 Motivation
6.9.2 Vorgehen
6.9.3 Beispiel
6.10 Funktionen zu einer Transformation vereinen (Combine Functions into Transform)

6.10.1 Motivation
6.10.2 Vorgehen
6.10.3 Beispiel
6.11 Phase aufteilen (Split Phase)
6.11.1 Motivation
6.11.2 Vorgehen
6.11.3 Beispiel

Kapitel 7: Kapselung

7.1 Datensatz kapseln (Encapsulate Record)
7.1.1 Motivation
7.1.2 Vorgehen
7.1.3 Beispiel
7.1.4 Beispiel: Kapselung eines verschachtelten Datensatzes
7.2 Collection kapseln (Encapsulate Collection)
7.2.1 Motivation
7.2.2 Vorgehen
7.2.3 Beispiel
7.3 Elementaren Wert durch Objekt ersetzen (Replace Primitive with Object)
7.3.1 Motivation
7.3.2 Vorgehen
7.3.3 Beispiel
7.4 Temporäre Variable durch Abfrage ersetzen (Replace Temp with Query)
7.4.1 Motivation
7.4.2 Vorgehen
7.4.3 Beispiel
7.5 Klasse extrahieren (Extract Class)
7.5.1 Motivation
7.5.2 Vorgehen
7.5.3 Beispiel
7.6 Klasse inline platzieren (Inline Class)
7.6.1 Motivation
7.6.2 Vorgehen
7.6.3 Beispiel
7.7 Delegation verbergen (Hide Delegate)
7.7.1 Motivation
7.7.2 Vorgehen
7.7.3 Beispiel

7.8 Vermittler entfernen (Remove Middle Man)
7.8.1 Motivation
7.8.2 Vorgehen
7.8.3 Beispiel
7.9 Algorithmus ersetzen (Substitute Algorithm)
7.9.1 Motivation
7.9.2 Vorgehen

Kapitel 8: Verschiebungen

8.1 Funktion verschieben (Move Function)
8.1.1 Motivation
8.1.2 Vorgehen
8.1.3 Beispiel: Verschieben einer verschachtelten Funktion in die oberste Ebene
8.1.4 Beispiel: Verschiebungen zwischen Klassen
8.2 Feld verschieben (Move Field)
8.2.1 Motivation
8.2.2 Vorgehen
8.2.3 Beispiel
8.2.4 Beispiel: Verschiebung in ein gemeinsam genutztes Objekt
8.3 Anweisungen in Funktion verschieben (Move Statements into Function)
8.3.1 Motivation
8.3.2 Vorgehen
8.3.3 Beispiel
8.4 Anweisungen in Aufrufer verschieben (Move Statements to Caller)
8.4.1 Motivation
8.4.2 Vorgehen
8.4.3 Beispiel
8.5 Inline-Code durch Funktionsaufruf ersetzen (Replace Inline Code with Function Call)
8.5.1 Motivation
8.5.2 Vorgehen
8.6 Anweisungen verschieben (Slide Statements)
8.6.1 Motivation
8.6.2 Vorgehen
8.6.3 Beispiel
8.6.4 Beispiel: Verschiebungen und bedingte Anweisungen
8.6.5 Literaturhinweis
8.7 Schleife aufteilen (Split Loop)
8.7.1 Motivation

8.7.2 Vorgehen
8.7.3 Beispiel
8.8 Schleife durch Pipeline ersetzen (Replace Loop with Pipeline)
8.8.1 Motivation
8.8.2 Vorgehen
8.8.3 Beispiel
8.8.4 Literaturhinweis
8.9 Toten Code entfernen (Remove Dead Code)
8.9.1 Motivation
8.9.2 Vorgehen

Kapitel 9: Daten organisieren

9.1 Variable aufteilen (Split Variable)
9.1.1 Motivation
9.1.2 Vorgehen
9.1.3 Beispiel
9.1.4 Beispiel: Zuweisung zu einem Eingabeparameter
9.2 Feld umbenennen (Rename Field)
9.2.1 Motivation
9.2.2 Vorgehen
9.2.3 Beispiel: Umbenennen eines Feldes
9.3 Abgeleitete Variable durch Abfrage ersetzen (Replace Derived Variable with Query)
9.3.1 Motivation
9.3.2 Vorgehen
9.3.3 Beispiel
9.3.4 Beispiel: Mehr als eine Quelle
9.4 Referenz durch Wert ersetzen (Change Reference to Value)
9.4.1 Motivation
9.4.2 Vorgehen
9.4.3 Beispiel
9.5 Wert durch Referenz ersetzen (Change Value to Reference)
9.5.1 Motivation
9.5.2 Vorgehen
9.5.3 Beispiel

Kapitel 10: Bedingungen vereinfachen

10.1 Bedingung zerlegen (Decompose Conditional)
10.1.1 Motivation
10.1.2 Vorgehen
10.1.3 Beispiel

10.2 Bedingten Ausdruck zusammenfassen (Consolidate Conditional Expression)
10.2.1 Motivation
10.2.2 Vorgehen
10.2.3 Beispiel
10.2.4 Beispiel: Verwendung von »and«
10.3 Verschachtelte Bedingung durch Wächterbedingung ersetzen (Replace Nested Conditional with Guard Clauses)
10.3.1 Motivation
10.3.2 Vorgehen
10.3.3 Beispiel
10.3.4 Beispiel: Umkehrung der Bedingungen
10.4 Bedingung durch Polymorphie ersetzen (Replace Conditional with Polymorphism)
10.4.1 Motivation
10.4.2 Vorgehen
10.4.3 Beispiel
10.4.4 Beispiel: Polymorphie für Variationen verwenden
10.5 Sonderfall einführen (Introduce Special Case)
10.5.1 Motivation
10.5.2 Vorgehen
10.5.3 Beispiel
10.5.4 Beispiel: Verwendung eines Objekts mit Literalen
10.5.5 Beispiel: Verwenden einer Transformation
10.6 Assertion einführen (Introduce Assertion)
10.6.1 Motivation
10.6.2 Vorgehen
10.6.3 Beispiel

Kapitel 11: Refactoring von APIs

11.1 Abfrage von Veränderung trennen (Separate Query from Modifier)
11.1.1 Motivation
11.1.2 Vorgehen
11.1.3 Beispiel
11.2 Funktion parametrisieren (Parameterize Function)
11.2.1 Motivation
11.2.2 Vorgehen
11.2.3 Beispiel
11.3 Steuerungs-Flag entfernen (Remove Flag Argument)
11.3.1 Motivation
11.3.2 Vorgehen

11.3.3 Beispiel
11.4 Vollständiges Objekt erhalten (Preserve Whole Object)
11.4.1 Motivation
11.4.2 Vorgehen
11.4.3 Beispiel
11.4.4 Beispiel: Eine Variante zum Erstellen der neuen Funktion
11.5 Parameter durch Abfrage ersetzen (Replace Parameter with Query)
11.5.1 Motivation
11.5.2 Vorgehen
11.5.3 Beispiel
11.6 Abfrage durch Parameter ersetzen (Replace Query with Parameter)
11.6.1 Motivation
11.6.2 Vorgehen
11.6.3 Beispiel
11.7 Setter entfernen (Remove Setting Method)
11.7.1 Motivation
11.7.2 Vorgehen
11.7.3 Beispiel
11.8 Konstruktor durch Fabrikfunktion ersetzen (Replace Constructor with Factory Function)
11.8.1 Motivation
11.8.2 Vorgehen
11.8.3 Beispiel
11.9 Funktion durch Befehl ersetzen (Replace Function with Command)
11.9.1 Motivation
11.9.2 Vorgehen
11.9.3 Beispiel
11.10 Befehl durch Funktion ersetzen (Replace Command with Function)
11.10.1 Motivation
11.10.2 Vorgehen
11.10.3 Beispiel

Kapitel 12: Der Umgang mit Vererbung

12.1 Methode nach oben verschieben (Pull Up Method)
12.1.1 Motivation
12.1.2 Vorgehen
12.1.3 Beispiel
12.2 Feld nach oben verschieben (Pull Up Field)
12.2.1 Motivation

12.2.2 Vorgehen
12.3 Konstruktorrumpf nach oben verschieben (Pull Up Constructor Body)
12.3.1 Motivation
12.3.2 Vorgehen
12.3.3 Beispiel
12.4 Methode nach unten verschieben (Push Down Method)
12.4.1 Motivation
12.4.2 Vorgehen
12.5 Feld nach unten verschieben (Push Down Field)
12.5.1 Motivation
12.5.2 Vorgehen
12.6 Typenschlüssel durch Unterklassen ersetzen (Replace Type Code with Subclasses)
12.6.1 Motivation
12.6.2 Vorgehen
12.6.3 Beispiel
12.6.4 Beispiel: Indirekte Vererbung
12.7 Unterklasse entfernen (Remove Subclass)
12.7.1 Motivation
12.7.2 Vorgehen
12.7.3 Beispiel
12.8 Basisklasse extrahieren (Extract Superclass)
12.8.1 Motivation
12.8.2 Vorgehen
12.8.3 Beispiel
12.9 Hierarchie abbauen (Collapse Hierarchy)
12.9.1 Motivation
12.9.2 Vorgehen
12.10 Unterklasse durch Delegation ersetzen (Replace Subclass with Delegate)
12.10.1 Motivation
12.10.2 Vorgehen
12.10.3 Beispiel
12.10.4 Beispiel: Ersetzen einer Hierarchie
12.11 Basisklasse durch Delegation ersetzen (Replace Superclass with Delegate)
12.11.1 Motivation
12.11.2 Vorgehen
12.11.3 Beispiel

Anhang A: Bibliografie

Anhang B: Liste der Refactorings

Anhang C: Code-Smells

Für Cindy

–Martin

Martin Fowler

Refactoring

Wie Sie das Design bestehender Software verbessern

Mit Beiträgen von Kent Beck

Übersetzung aus dem Englischen
von Dr. Jan Linxweiler und Knut Lorenzen

Impressum

Bibliografische Information der Deutschen Nationalbibliothek
Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.d-nb.de> abrufbar.

ISBN 978-3-95845-943-4
2. Auflage 2020

www.mitp.de
E-Mail: mitp-verlag@sigloch.de
Telefon: +49 7953 / 7189 - 079
Telefax: +49 7953 / 7189 - 082

Authorized translation from the English language edition, REFACTORING: IMPROVING THE DESIGN OF EXISTING CODE, 2nd Edition by MARTIN FOWLER, published by Pearson Education, Inc, publishing as Addison-Wesley Professional, Copyright © 2019 Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means,

electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

GERMAN language edition published by MITP VERLAGS GMBH & CO. KG

© 2020 mitp Verlags GmbH & Co. KG

Dieses Werk, einschließlich aller seiner Teile, ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Verlages unzulässig und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.

Die Wiedergabe von Gebrauchsnamen, Handelsnamen, Warenbezeichnungen usw. in diesem Werk berechtigt auch ohne besondere Kennzeichnung nicht zu der Annahme, dass solche Namen im Sinne der Warenzeichen- und Markenschutz-Gesetzgebung als frei zu betrachten wären und daher von jedermann benutzt werden dürften.

Lektorat: Sabine Schulz, Janina Bahlmann
Fachkorrektorat: Dr. Jan Linxweiler
Sprachkorrektorat: Doris Weißgerber
Covergestaltung: Christian Kalkert

electronic **pub**lication: III-satz, Husby, www.drei-satz.de
Bildnachweis: © liz west / https://www.flickr.com/photos/calliope/3834075344/

Dieses Ebook verwendet das ePub-Format und ist optimiert für die Nutzung mit dem iBooks-reader auf dem iPad von Apple. Bei der Verwendung anderer Reader kann es zu Darstellungsproblemen kommen.

Der Verlag räumt Ihnen mit dem Kauf des ebooks das Recht ein, die Inhalte im Rahmen des geltenden Urheberrechts zu nutzen. Dieses Werk, einschließlich aller seiner Teile, ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheherrechtsgesetzes ist ohne Zustimmung des Verlages unzulässig und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen und Einspeicherung und Verarbeitung in elektronischen Systemen.

Der Verlag schützt seine ebooks vor Missbrauch des Urheberrechts durch ein digitales Rechtemanagement. Bei Kauf im Webshop des Verlages werden die ebooks mit einem nicht sichtbaren digitalen Wasserzeichen individuell pro Nutzer signiert.

Bei Kauf in anderen ebook-Webshops erfolgt die Signatur durch die Shopbetreiber. Angaben zu diesem

DRM finden Sie auf den Seiten der jeweiligen Anbieter.

Vorwort zur ersten Auflage

»Refactoring« wurde ursprünglich in Smalltalk-Kreisen konzipiert. Es dauerte allerdings nicht lange, bis es auch den Weg in die Lager anderer Programmiersprachen fand. Da Refactoring ein integraler Bestandteil bei der Entwicklung von Frameworks ist, kommt dieser Ausdruck häufig zur Sprache, wenn sich »Frameworker« über ihr Handwerk unterhalten. Er wird verwendet, wenn sie ihre Klassenhierarchien überarbeiten und davon schwärmen, wie viele Codezeilen sie löschen konnten. Frameworker wissen, dass ein Framework nicht gleich beim ersten Versuch perfekt ist – vielmehr muss es sich aus ihrer stetig anwachsenden Erfahrung entwickeln. Sie wissen auch, dass Code häufiger gelesen und modifiziert als geschrieben wird. Beim Refactoring handelt es sich um den Schlüssel zu lesbarem und modifizierbarem Code. Das gilt insbesondere für Frameworks, trifft aber auch auf Software im Allgemeinen zu.

Wo also liegt das Problem? Ganz einfach: Refactoring ist riskant. Es erfordert Änderungen an funktionsfähigem Code, die zu schwer aufspürbaren Bugs führen können. Wird ein Refactoring nicht sorgfältig durchgeführt, kann es Sie um Tage oder sogar Wochen zurückwerfen. Und

Refactoring wird noch riskanter, wenn es außerhalb eines formellen Rahmens oder gar spontan durchgeführt wird. Zunächst beginnt man an der Oberfläche des Codes zu schürfen. Dabei findet man schnell Verbesserungsmöglichkeiten und beginnt, tiefer zu graben. Je tiefer man gräbt, desto mehr Verbesserungsmöglichkeiten werden aufgedeckt ... und umso mehr Änderungen nimmt man vor. Unter Umständen gräbt man sich dabei ein sehr tiefes Loch, aus dem man nicht wieder herauskommt. Um zu verhindern, dass man sich sein eigenes Grab schaufelt, muss ein Refactoring systematisch durchgeführt werden. Als meine Co-Autoren und ich das Buch *Design Patterns* verfassten, wiesen wir darauf hin, dass Design Patterns gut geeignete Ziele für Refactorings darstellen. Allerdings ist das Identifizieren des Ziels nur ein Teil des Problems – den Code dementsprechend umzustrukturieren, stellt eine weitere Herausforderung dar.

Martin Fowler und die weiteren mitwirkenden Autoren leisten zur Entwicklung objektorientierter Software einen Beitrag von unschätzbarem Wert, indem sie den Prozess des Refactorings ausführlich darstellen. Dieses Buch erklärt Prinzipien und bewährte Praktiken des Refactorings und zeigt auf, wann und wo Sie beginnen sollten, Ihren Code auf Verbesserungsmöglichkeiten hin zu untersuchen. In der Mitte des Buchs ist eine umfangreiche Liste an Refactorings enthalten. Jedes

dieser Refactorings beschreibt die Motivation sowie die jeweilige Vorgehensweise einer bewährten Transformation des Codes. Einige der Refactorings, wie beispielsweise *Funktion extrahieren* (Extract Function, Abschnitt 6.1) oder *Feld verschieben* (Move Field, Abschnitt 8.2), scheinen auf der Hand zu liegen.

Aber lassen Sie sich nicht täuschen. Die Vorgehensweisen solcher Refactorings zu verstehen, ist von entscheidender Bedeutung, um ein Refactoring auf disziplinierte Weise auszuführen. Die Refactorings in diesem Buch werden Ihnen dabei helfen, Ihren Code Schritt für Schritt zu modifizieren und auf diese Weise das Risiko bei der Weiterentwicklung Ihres Designs zu verringern. Als Entwickler werden Sie schon bald die Refactorings und ihre Bezeichnungen in Ihren Wortschatz aufnehmen.

Meine ersten Erfahrungen mit diszipliniertem »Schritt-für-Schritt«-Refactoring machte ich während des Pair Programmings mit Kent Beck 30.000 Fuß über dem Erdboden. Er stellte sicher, dass wir die Refactorings aus diesem Buch schrittweise anwendeten. Ich war erstaunt, wie gut dieses Verfahren funktionierte. Ich hatte nicht nur größeres Vertrauen in den geschriebenen Code, ich fühlte mich auch weniger gestresst. Ich empfehle Ihnen nachdrücklich, diese Refactorings auszuprobieren: Ihnen und Ihrem Code wird es damit viel besser ergehen.

–Erich Gamma, Object Technology International, Inc.

Januar 1999

Über den Fachkorrektor der deutschen Ausgabe

Dr. Jan Linxweiler ist Geschäftsführer des *Center for Mechanics, Uncertainty and Simulation in Engineering* an der TU Braunschweig. Er verfügt über mehr als 15 Jahre Erfahrung in Forschung und Lehre im Bereich der Softwareentwicklung und war als Softwarearchitekt im Automotive-Sektor tätig. Darüber hinaus entwickelt er erfolgreich Apps für iOS und macOS. Seine Leidenschaft für die Entwicklung hochqualitativer Software vermittelt Linxweiler auch den Studierenden in seinen Vorlesungen.

Einleitung

Es war einmal ein Berater, der sich zu einem Softwareprojekt aufmachte, um sich den Code anzusehen, der im Laufe des Projekts entstanden war. Als er sich die Klassenhierarchie im Kern des Systems ansah, stellte er fest, dass es ein ziemliches Durcheinander war. Klassen auf höherem Abstraktionsniveau gingen von bestimmten Annahmen bezüglich der Funktionsweise anderer Klassen aus – Annahmen, die sie in Form von Code an ihre Unterklassen vererbten. Der Code war allerdings nicht für alle Unterklassen geeignet, daher wurde er ziemlich häufig überschrieben. Schon geringfügige Änderungen an der Basisklasse hätten die Notwendigkeit, den Code zu überschreiben, weitestgehend beseitig. An anderen Stellen hatte man die Intentionen der Basisklasse offenbar nicht richtig verstanden, denn Teile des in der Basisklasse bereits vorhandenen Verhaltens wurden in Unterklassen erneut implementiert. An wieder anderen Stellen erledigten mehrere Unterklassen die gleichen Aufgaben durch Code, der in der Klassenhierarchie eindeutig nach oben verschoben werden konnte.

Der Berater empfahl dem Projektmanagement, den Code zu untersuchen und zu bereinigen – was allerdings nicht

gerade für Begeisterung sorgte. Der Code funktionierte offenbar, und es gab erheblichen Termindruck. Das Management meinte, man werde sich später darum kümmern.

Den Programmierern, die an der Klassenhierarchie arbeiteten, hatte der Berater ebenfalls gezeigt, was da tatsächlich vor sich ging. Sie waren eifrig und erkannten das Problem. Ihnen war klar, dass es nicht wirklich ihr Fehler war. Manchmal benötigt es eben ein weiteres Paar an Augen, um ein Problem zu erkennen. Die Programmierer befassten sich also ein paar Tage lang damit, die Hierarchie aufzuräumen. Als sie fertig waren, hatten sie die Hälfte des Codes aus der Hierarchie entfernt, ohne dadurch die Funktionalität einzuschränken. Sie waren mit dem Ergebnis sehr zufrieden und stellten fest, dass sowohl das Hinzufügen neuer Klassen als auch die Verwendung der Klassen des übrigen Systems schneller und einfacher von der Hand gingen.

Das Projektmanagement war nicht erfreut. Der Terminplan war eng, und es gab eine Menge Arbeit zu erledigen. Die beiden Programmierer hatten zwei Tage mit Arbeiten verbracht, die nichts zu den vielen Features beitrugen, die das System in wenigen Monaten besitzen musste. Der alte Code hatte doch tadellos funktioniert. Ja, zugegeben, das Design war jetzt etwas »reiner« und etwas »sauberer«, aber das Projekt musste Code liefern, der funktioniert, keinen Code, der Akademikern gefällt.

Der Berater hingegen schlug vor, weitere zentrale Teile des Systems auf ähnliche Weise aufzuräumen, wodurch das Projekt womöglich ein bis zwei Wochen zum Stillstand käme. Und das Ganze, um den Code schöner zu machen, nicht, damit er etwas kann, was er nicht jetzt schon könnte.

Was halten Sie von dieser Geschichte? Glauben Sie, dass der Berater zu Recht vorschlug, weiter aufzuräumen? Oder neigen Sie eher zur alten Ingenieur-Weisheit »Was nicht kaputt ist, muss man auch nicht reparieren«?

Ich bin hier zugegebenermaßen etwas voreingenommen, denn der Berater war ich. Das Projekt scheiterte sechs Monate später, vor allem, weil der Code zu komplex war, um ihn zu debuggen oder ihn so anzupassen, dass er eine akzeptable Leistung lieferte.

Dann wurde Kent Beck als Berater engagiert, der das Projekt wieder auf die Beine stellen sollte – eine Aufgabe, die es erforderlich machte, fast das gesamte System von Grund auf neu zu schreiben. Er machte einiges anders, aber die wichtigste Änderung war, dass er darauf bestand, den Code durch Refactorings kontinuierlich aufzuräumen. Die erhöhte Effektivität des Teams und die Rolle, die das Refactoring dabei einnahm, inspirierten mich dazu, die erste Ausgabe dieses Buchs zu verfassen – um das von Kent und anderen erworbene Wissen weiterzugeben, das sie sich durch die Anwendung von Refactorings zur Verbesserung der

Softwarequalität angeeignet hatten.

Seitdem gehört »Refactoring« in der Programmierung zum gängigen Wortschatz. Tatsächlich hat sich das ursprüngliche Buch auch ziemlich gut behaupten können. Allerdings sind achtzehn Jahre ein hohes Alter für ein Buch über Programmierung. Daher dachte ich mir, es sei an der Zeit, es zu überarbeiten. Dabei habe ich praktisch jede Seite des Buchs neu geschrieben. In gewisser Hinsicht hat sich jedoch nur sehr wenig geändert. Die Essenz des Buches ist unverändert; die meisten der wichtigsten Refactorings sind grundsätzlich die gleichen. Ich hoffe jedoch, dass die Neuformulierungen mehr Lesern dabei helfen, zu erlernen, wie das Refactoring effektiv durchgeführt wird.

Was ist Refactoring?

Refactoring ist der Prozess, ein Softwaresystem so zu modifizieren, dass sich das externe Verhalten des Codes nicht ändert, aber dennoch die interne Struktur des Codes zu verbessern. Es handelt sich um eine Vorgehensweise zum Aufräumen von Code, die Disziplin erfordert und die Wahrscheinlichkeit, Bugs zu verursachen, minimiert. Beim Refactoring verbessern Sie im Wesentlichen das Design des Codes, nachdem er geschrieben wurde.

»Verbessern des Designs des Codes, nachdem er geschrieben wurde.« Das ist schon eine seltsame

Ausdrucksweise. Seit es Softwareentwicklung gibt, dachten die meisten Leute, dass zunächst das Design entwickelt wird, und dass die Programmierung erst dann erfolgt, wenn es abgeschlossen ist. Der Code wird im Laufe der Zeit modifiziert, und die Integrität des Systems – die dem ursprünglichen Design entsprechende Struktur – geht allmählich verloren. Der Code wird nicht mehr sorgfältig entwickelt, und die Programmierung wird allmählich zum »Hacken«.

Refactoring ist das Gegenteil dieser Vorgehensweise. Mit Refactoring können wir ein schlechtes oder sogar chaotisches Design in sinnvoll strukturierten Code umwandeln. Die einzelnen Schritte sind einfach – geradezu simpel. Ich verschiebe ein Attribut von einer Klasse in eine andere, entnehme einer Methode etwas Code, um daraus eine eigene Methode zu machen oder verschiebe Teile des Codes in der Hierarchie nach oben oder unten. Dennoch kann die Gesamtheit dieser kleinen Änderungen das Design drastisch verbessern. Hierbei handelt es sich um die genaue Umkehrung dessen, was man als Softwarezerfall bezeichnen könnte.

Beim Refactoring verschiebt sich die Gewichtung der Tätigkeiten. Das Design wird nicht nur ganz am Anfang festgelegt, sondern entsteht kontinuierlich während der Entwicklung. Während sich das System entwickelt, finde ich heraus, wie sich das Design verbessern lässt. Diese Interaktion führt zu einem Programm, dessen Design auch bei fortschreitender Entwicklung gut bleibt.

Worum geht es in diesem Buch?

Dieses Buch ist ein Leitfaden für das Refactoring und wendet sich an den fortgeschrittenen Entwickler. Ich möchte Ihnen zeigen, wie Sie Refactorings auf kontrollierte und effiziente Weise durchführen können. Sie erfahren, wie Sie ein Refactoring vornehmen, ohne Bugs im Code zu verursachen, und gleichzeitig die Struktur des Codes systematisch verbessern.

Traditionell beginnt ein Buch mit einer Einführung. Das halte ich im Prinzip für richtig, finde es jedoch schwierig, Refactoring anhand allgemeiner Erklärungen oder Definitionen einzuführen, deshalb folgt zunächst ein Beispiel. In Kapitel 1 wird ein kleines Programm mit einigen typischen Designfehlern durch Refactorings so umstrukturiert, dass es anschließend besser verständlich und leichter modifizierbar ist. Sie lernen dabei den generellen Refactoring-Prozess sowie eine Reihe nützlicher Refactorings kennen. Hierbei handelt es sich um das wichtigste Kapitel, das Sie lesen sollten, wenn Sie verstehen möchten, worum es beim Refactoring eigentlich geht.

In Kapitel 2 erläutere ich weitere allgemeine Prinzipien des Refactorings und stelle einige Definitionen sowie die Gründe für ein Refactoring vor. Anschließend umreiße ich einige der Herausforderungen, die das Refactoring mit sich bringt. In Kapitel 3 unterstützt mich Kent Beck dabei, zu beschreiben, wie man »Code-Smells«

identifiziert und wie man sie durch Refactorings bereinigen kann. Auch das Testen spielt beim Refactoring eine sehr wichtige Rolle, deshalb beschreibt Kapitel 4, wie Sie Tests in Ihren Code integrieren können.

Der Hauptgegenstand dieses Buchs – der Refactoring-Katalog – nimmt die verbleibenden Seiten ein. Der Katalog ist zwar alles andere als vollständig, enthält jedoch die wichtigsten Refactorings, die wahrscheinlich von den meisten Entwicklern benötigt werden. Der Katalog ist aus den Notizen entstanden, die ich mir gemacht habe, als ich mich Ende der 1990er Jahre mit Refactoring befasste. Ich verwende diese Notizen auch heute noch, weil ich sie mir nicht alle merken kann. Wenn ich ein Refactoring durchführen möchte, wie etwa *Phase aufteilen* (Split Phase, Abschnitt 6.11), kann ich im Katalog nachsehen, wie ich das auf sichere Weise Schritt für Schritt tun kann. Ich kann mir nur wünschen, dass Sie diesen Teil des Buchs regelmäßig aufschlagen werden.

SCHWERPUNKT INTERNET

Das Internet hat enorme Auswirkungen auf unsere Gesellschaft, insbesondere auf die Art und Weise, wie wir uns Informationen beschaffen. Als ich die erste Ausgabe dieses Buchs verfasste, wurde das Wissen über Softwareentwicklung hauptsächlich durch Druckerzeugnisse vermittelt. Inzwischen beschaffe ich

mir die meisten Informationen online. Das stellt für Autoren wie mich eine Herausforderung dar: Haben Bücher noch eine Daseinsberechtigung, und wie sollten sie gestaltet sein?

Ich glaube, Bücher wie dieses haben noch immer ihren Platz – sie müssen sich jedoch ändern. Der Wert eines Buchs besteht in einer großen zusammenhängenden Wissensmenge. Beim Schreiben dieses Buchs habe ich versucht, eine Vielzahl verschiedener Refactorings abzuhandeln und sie in sinnvoller und gegliederter Weise zu organisieren.

Aber im Großen und Ganzen handelt es sich dabei um ein abstraktes literarisches Werk, herkömmlicherweise in Form eines auf Papier gedruckten Buchs, das zukünftig nicht mehr nötig sein wird. Die meisten Verlage bieten noch immer vornehmlich auf Papier gedruckte Bücher an. Zwar wurden eBooks mit Begeisterung angenommen, diese stellen jedoch nur elektronische Versionen der ursprünglichen Werke dar, die auf der Struktur eines auf Papier gedruckten Buchs beruhen.

Mit diesem Buch möchte ich einen anderen Ansatz verfolgen. Die Standardform dieses Buchs ist die dazugehörige englischsprachige Website, die ich als *Webbook* bezeichne (`https://refactoring.com/catalog`). Das auf Papier gedruckte Buch stellt eine Auswahl des auf der Website enthaltenen Materials dar, die so

zusammengestellt wurde, dass es sinnvoll ist, sie auf Papier zu drucken. Das Buch soll auch gar nicht alle auf der Website vorhandenen Refactorings enthalten, zumal ich zukünftig vermutlich weitere Refactorings zu dem Webbook hinzufügen werde. Auf ähnliche Weise stellt auch das eBook eine bestimmte Auswahl des Webbooks dar.

Während ich das hier schreibe, weiß ich natürlich nicht, ob Sie diese Zeilen auf der Website, in einem eBook, in einem auf Papier gedruckten Buch oder in einer anderen Form lesen, die ich mir noch gar nicht vorstellen kann. Jedenfalls bemühe ich mich, ein nützliches Werk zu erstellen, unabhängig davon, in welcher Form es Ihnen vorliegt.

CODEBEISPIELE IN JAVASCRIPT

Wie in den meisten technisch orientierten Bereichen der Softwareentwicklung sind Codebeispiele zur Veranschaulichung der Konzepte sehr wichtig. Die Refactorings sind sich in verschiedenen Programmiersprachen allerdings sehr ähnlich. Hin und wieder gibt es gewisse Dinge, die in einer bestimmten Programmiersprache zu beachten sind, die grundlegenden Elemente der Refactorings ändern sich jedoch nicht.

Zur Veranschaulichung der Refactorings habe ich mich für JavaScript entschieden, weil ich den Eindruck habe,

dass die meisten Leser diese Programmiersprache verstehen dürften. Es sollte Ihnen aber auch nicht schwerfallen, die Refactorings auf beliebige andere Programmiersprachen zu übertragen. Ich habe mich bemüht, keine der komplizierteren Features der Sprache zu verwenden, damit Sie die Refactorings auch mit nur oberflächlichen JavaScript-Kenntnissen verstehen können. Dass ich JavaScript verwende, ist jedenfalls sicher nicht als uneingeschränkte Befürwortung der Sprache zu verstehen.

Ich verwende für meine Beispiele zwar JavaScript, das heißt jedoch nicht, dass die im Buch vorgestellten Verfahren auf JavaScript beschränkt sind. In der ersten Ausgabe dieses Buchs wurde Java verwendet, und viele Programmierer fanden es nützlich, obwohl sie noch nie zuvor auch nur eine einzige Java-Klasse geschrieben hatten. Ich habe mit dem Gedanken gespielt, diese Allgemeingültigkeit durch die Verwendung von einem Dutzend verschiedener Programmiersprachen für die Beispiele zu verdeutlichen, hatte jedoch den Eindruck, dass dieses Vorgehen für den Leser zu verwirrend wäre. Dessen ungeachtet richtet sich das Buch an Programmierer, die beliebige Sprachen verwenden. Außerhalb der Abschnitte mit Codebeispielen treffe ich keine Annahmen über die verwendete Programmiersprache. Ich gehe davon aus, dass die Leser meine allgemeinen Kommentare zur Kenntnis nehmen und sie auf die von ihnen verwendete

Programmiersprache übertragen. Tatsächlich erwarte ich sogar, dass Leser die JavaScript-Beispiele an die von ihnen verwendete Programmiersprache anpassen.

Das bedeutet, dass ich, abseits der Diskussion von bestimmten Beispielen, Begriffe wie »Klasse«, »Modul«, »Funktion« usw. im allgemeinen Kontext der Programmierung verwende und nicht im Kontext des Sprachmodells von JavaScript.

Dass ich für die Codebeispiele JavaScript verwende, bedeutet darüber hinaus auch, dass ich mich bemühe, einen für JavaScript typischen Programmierstil zu vermeiden, der Programmierern, die normalerweise kein JavaScript verwenden, weniger vertraut ist. Es geht in diesem Buch nicht um »Refactoring in JavaScript«, sondern um Refactoring im Allgemeinen, wobei zufälligerweise JavaScript für die Beispiele verwendet wird. Es gibt eine Vielzahl interessanter Refactorings, die spezifisch für JavaScript sind (wie etwa das Refactoring von Callbacks zu Promises oder zu async/await). Diese Refactorings gehen jedoch über den Rahmen dieses Buchs hinaus.

Wer sollte dieses Buch lesen?

Ich richte mich mit diesem Buch an fortgeschrittene Programmierer – an Menschen, die mit dem Schreiben von Software ihren Lebensunterhalt verdienen. Die Beispiele und die Erläuterungen umfassen eine Menge

Code, den es zu lesen und zu verstehen gilt. Die Beispiele sind in JavaScript verfasst, sollten aber auf die meisten Programmiersprachen übertragen werden können. Ich erwarte, dass Leser genügend Erfahrung besitzen, um zu verstehen, was in diesem Buch beschrieben wird, umfassende Kenntnisse werden jedoch nicht vorausgesetzt.

Das Buch wendet sich zwar insbesondere an Entwickler, die das Refactoring erlernen möchten, ist aber auch nützlich für jemanden, der bereits Erfahrungen mit dem Refactoring gesammelt hat – es kann also auch als Lernhilfe verwendet werden. Ich habe große Anstrengungen unternommen, um die Funktionsweise der verschiedenen Refactorings zu erklären, damit erfahrene Entwickler dieses Material zur Unterstützung und Beratung ihrer Kollegen nutzen können.

Das Refactoring konzentriert sich zwar vornehmlich auf den Code, hat aber auch erhebliche Auswirkungen auf das Design eines Systems. Für leitende Designer und Softwarearchitekten ist von entscheidender Bedeutung, die Grundlagen des Refactorings zu verstehen und sie in ihren Projekten zu verwenden. Ein Refactoring wird idealerweise durch einen angesehenen und erfahrenen Entwickler eingeleitet. Ein solcher Entwickler kann die dem Refactoring zugrunde liegenden Prinzipien am besten verstehen und sie auf eine bestimmte Situation übertragen. Das trifft insbesondere dann zu, wenn Sie eine andere Sprache als JavaScript verwenden, weil die

von mir gezeigten Beispiele an diese andere Programmiersprache angepasst werden müssen.

So können Sie das Buch am besten ausschöpfen, ohne es vollständig zu lesen:

- **Wenn Sie wissen möchten, was Refactoring eigentlich ist,** sollten Sie Kapitel 1 lesen – das Beispiel sollte die Vorgehensweise verdeutlichen.
- **Wenn Sie wissen möchten, welche Gründe für ein Refactoring sprechen,** sollten Sie die ersten beiden Kapitel lesen. Sie erläutern, was Refactoring eigentlich ist und warum es durchgeführt werden sollte.
- **Wenn Sie wissen möchten, wo ein Refactoring angebracht ist,** sollten Sie Kapitel 3 lesen. Darin werden die Anzeichen dafür erläutert, dass ein Refactoring notwendig wäre.
- **Wenn Sie das Refactoring tatsächlich durchführen möchten,** sollten Sie die ersten vier Kapitel vollständig lesen und anschließend den Katalog überfliegen. Lesen Sie so viel, dass Sie in groben Zügen wissen, was der Katalog enthält. Sie brauchen die ganzen Details nicht zu kennen. Lesen Sie ausführlich nach, wenn Sie das Refactoring tatsächlich durchführen, und lassen Sie sich von dem Beispiel unterstützen. Der Katalog ist zum Nachschlagen gedacht, deshalb werden Sie ihn vermutlich nicht in einem Zug durchlesen wollen.

Beim Schreiben des Buchs war die Benennung der verschiedenen Refactorings ein wichtiger Teil. Die Terminologie erleichtert uns die Kommunikation: Wenn ein Entwickler einen anderen darum bittet, einen Codeabschnitt in eine Funktion auszulagern oder eine Berechnung in separate Phasen aufzuteilen, ist beiden klar, was mit den Bezeichnungen *Funktion extrahieren*

(Extract Function, Abschnitt 6.1) und *Phase aufteilen* (Split Phase, Abschnitt 6.11) gemeint ist. Diese Bezeichnungen sind auch bei der Auswahl automatisierter Refactorings hilfreich.

Von anderen geschaffene Grundlagen

Ich möchte gleich am Anfang klarstellen, dass ich mit diesem Buch eine große Verpflichtung eingegangen bin – eine Verpflichtung denen gegenüber, die in den 1990er Jahren das Fachgebiet Refactoring entwickelt haben. Ich habe aus ihren Erfahrungen gelernt, die es mir ermöglichten und mich dazu inspirierten, die erste Ausgabe dieses Buchs zu verfassen. Auch wenn seitdem viele Jahre vergangen sind, halte ich es für wichtig, die Grundlagen zu würdigen, die hier geschaffen wurden. Idealerweise hätte einer der Schöpfer des Refactorings diese erste Ausgabe verfasst, aber letzten Endes war ich es, der die Zeit und die Kraft fand, diese Aufgabe zu bewältigen.

Ward Cunningham und **Kent Beck** gehörten zu den führenden ersten Verfechtern des Refactorings. Sie nutzten es schon früh als Grundlage für ihre Projekte und passten ihren Entwicklungsprozess so an, dass sie vom Refactoring profitieren konnten. Insbesondere die Zusammenarbeit mit Kent hat mir immer wieder die große Bedeutung des Refactorings verdeutlicht – eine Erkenntnis, die unmittelbar zum Entstehen dieses Buchs

führte.

Ralph Johnson leitet an der University of Illinois in Urbana-Champaign eine Forschungsgruppe, die für ihre praktischen Beiträge zu objektorientierten Technologien bekannt ist. Ralph war lange ein Meister des Refactorings, und einige seiner Studenten haben auf diesem Gebiet unverzichtbare Beiträge geleistet. **Bill Opdyke** hat mit seiner Doktorarbeit das erste ausführliche schriftliche Werk über Refactoring geschaffen. **John Brant** und **Don Roberts** beließen es nicht beim Schreiben von Wörtern – sie schufen das erste automatisierte Werkzeug für Refactoring, den Refactoring-Browser zur Durchführung des Refactorings von Smalltalk-Programmen.

Seit der Veröffentlichung der ersten Ausgabe dieses Buchs haben viele Leute das Fachgebiet Refactoring fortentwickelt. Insbesondere die Arbeit derer, die das automatisierte Refactoring bei der Entwicklung ermöglichten, hat enorm dazu beigetragen, das Leben der Programmierer zu erleichtern. Für mich ist es praktisch selbstverständlich, dass ich eine häufig verwendete Funktion durch eine einfache Tastenkombination umbenennen kann – aber diese Einfachheit beruht auf den Anstrengungen der Programmierer integrierter Entwicklungsumgebungen, deren Arbeit für uns alle von Nutzen ist.

Danksagungen

Auch wenn ich auf all diesen Grundlagen aufbauen konnte, benötigte ich beim Schreiben des Buchs dennoch eine Menge Hilfe. Die erste Ausgabe beruhte in hohem Maß auf der Erfahrung und Unterstützung von **Kent Beck**. Durch ihn kam ich erstmals mit Refactoring in Berührung. Er inspirierte mich dazu, Notizen zu den Refactorings zu erstellen, und half dabei, sie in lesbare Form zu bringen. Er kam auf die Idee, Refactorings in Form von Gerüchen (»Code-Smells«) zu beschreiben. Ich denke manchmal, er hätte eine bessere Version der ersten Ausgabe geschrieben als ich – wenn er stattdessen nicht damit beschäftigt gewesen wäre, das wegweisende Buch zum Thema Extreme Programming zu verfassen.

Alle mir bekannten Buchautoren weisen darauf hin, wie viel sie ihren technischen Gutachtern verdanken. Wir alle haben Arbeiten mit groben Fehlern verfasst, die nur erkannt wurden, weil unsere Kollegen als Gutachter tätig waren. Ich persönlich beschäftige mich nur wenig mit der Begutachtung technischer Arbeiten, nicht zuletzt, weil ich denke, dass ich das nicht besonders gut kann, deshalb bewundere ich die Kollegen, die sich dieser Aufgabe annehmen. Mit der Begutachtung von Büchern ist noch nicht einmal ein Blumentopf zu gewinnen, deshalb betrachte ich das als einen Akt wahrer Großzügigkeit.

Als ich damit anfing, ernsthaft am Buch zu arbeiten,

habe ich eine Mailingliste mit Beratern erstellt, um Feedback zu erhalten. Wenn ich Fortschritte gemacht hatte, stellte ich der Gruppe Entwürfe der neuen Inhalte zur Verfügung und bat um Rückmeldung. Ich möchte mich bei den folgenden Personen für ihr Feedback per Mailingliste bedanken: **Arlo Belshee**, **Avdi Grimm**, **Beth Anders-Beck**, **Bill Wake**, **Brian Guthrie**, **Brian Marick**, **Chad Wathington**, **Dave Farley**, **David Rice**, **Don Roberts**, **Fred George**, **Giles Alexander**, **Greg Doench**, **Hugo Corbucci**, **Ivan Moore**, **James Shore**, **Jay Fields**, **Jessica Kerr**, **Joshua Kerievsky**, **Kevlin Henney**, **Luciano Ramalho**, **Marcos Brizeno**, **Michael Feathers**, **Patrick Kua**, **Pete Hodgson**, **Rebecca Parsons** und **Trisha Gee**.

Aus dieser Gruppe gebührt **Beth Anders-Beck**, **James Shore** und **Pete Hodgson** für ihre Hilfe bezüglich JavaScript besonderer Dank.

Nachdem ich einen mehr oder weniger vollständigen Entwurf fertiggestellt hatte, bat ich weitere Personen um Rückmeldung, weil ich gerne wissen wollte, wie das Buch als Ganzes beim Leser ankommt. **William Chargin** und **Michael Hunger** haben mir unglaublich detailliertes Feedback gegeben, und auch **Bob Martin** und **Scott Davis** lieferten viele nützliche Kommentare. **Bill Wake** hat neben seinen Beiträgen zur Mailingliste zudem den ersten kompletten Entwurf vollständig begutachtet.

Meine Kollegen bei ThoughtWorks liefern mir kontinuierlich Ideen und Feedback zu meiner Arbeit. In das Buch sind unzählige Fragen, Kommentare und Anmerkungen eingeflossen. Dass ich als Angestellter von ThoughtWorks einen beträchtlichen Teil meiner Arbeitszeit mit dem Schreiben verbringen kann, ist wirklich toll. Ich schätze insbesondere die regelmäßigen Gespräche mit **Rebecca Parsons**, unserer technischen Direktorin, und ihre Ideen.

Der Redakteur **Greg Doench** ist bei Pearson dafür verantwortlich, die vielen bei der Veröffentlichung eines Buchs auftretenden Probleme zu lösen. **Julie Nahil** ist für die Produktion zuständig. Und es war mir wieder ein Vergnügen, mit **Dmitry Kirsanov** (Redigieren) und **Alina Kirsanov** (Zusammenstellung und Verschlagwortung) zusammenzuarbeiten.

Kapitel 1

Refactoring: Ein erstes Beispiel

Wie soll ich anfangen, über das Refactoring zu sprechen? Üblicherweise werden die Geschichte des Themas oder die grundlegenden Prinzipien oder Ähnliches erörtert. Wenn dies auf Konferenzen geschieht, werde ich ein wenig schläfrig. Meine Gedanken schweifen ab, und ein Hintergrundprozess meines Gehirns achtet darauf, ob der Redner ein Beispiel ankündigt, um mich dann zu informieren.

Beispiele dagegen beleben meine Aufmerksamkeit, weil ich nachvollziehen kann, was beschrieben wird. Bei den Prinzipien werden viel zu oft Verallgemeinerungen vorgenommen, sodass es schwierig ist, sich vorzustellen, wie etwas praktisch angewendet wird. Ein Beispiel hingegen ist hilfreich, eine konkrete Anwendung zu verdeutlichen.

Ich werde also dieses Buch mit einem Beispiel für Refactoring beginnen. Anhand dessen werde ich die Funktionsweise des Refactorings erläutern und versuchen, Ihnen ein Gespür für den Refactoring-Prozess zu vermitteln. Im sich daran anschließenden Kapitel

werde ich wie üblich mit den Grundlagen fortfahren.

Die Wahl eines einführenden Beispiels stellt mich jedoch vor ein Problem. Wenn ich ein umfangreiches Programm auswähle, es beschreibe und Ihnen erkläre, wie das Refactoring vorzunehmen ist, dann ist das für die meisten zu kompliziert, es zu verstehen. (Ich habe das bei der ersten Ausgabe des vorliegenden Buches ausprobiert – und letztlich zwei Beispiele verwerfen müssen, die zwar ziemlich überschaubar waren, deren Beschreibung aber jeweils mehr als hundert Seiten lang war.) Wähle ich hingegen ein Beispiel aus, das einfach genug ist, um gut nachvollziehbar zu sein, könnte das den Eindruck erwecken, dass sich Refactoring überhaupt nicht lohnt.

Ich bin also mit dem klassischen Problem konfrontiert, mit dem man zu kämpfen hat, wenn man Verfahren beschreiben möchte, die für Programme in der Praxis nützlich sind. Offen gestanden ist es tatsächlich nicht der Mühe wert, alle Refactorings, die ich Ihnen zeigen werde, bei dem kleinen Programm anzuwenden, das ich als Beispiel verwende. Wenn der Code jedoch Teil eines größeren Systems ist, dann wird das Refactoring wichtig. Stellen Sie sich beim Lesen meines Beispiels einfach vor, dass es Bestandteil eines sehr viel größeren Systems ist.

1.1 Der Ausgangspunkt

In der ersten Ausgabe dieses Buchs gab das erste

Programm die Rechnung einer Videothek aus, was heute dazu führen dürfte, dass sich einige Leser fragen: »Was ist eine Videothek?« Anstatt diese Frage zu beantworten, habe ich das Beispiel überarbeitet, das jetzt auf etwas noch Älterem beruht, das jedoch auch heutzutage noch aktuell ist.

Stellen Sie sich ein Unternehmen vor, das eine Schauspieltruppe beschäftigt, die auf verschiedenen Veranstaltung Theatervorstellungen gibt. Typischerweise beauftragt ein Kunde einige Vorstellungen. Daraufhin stellt das Unternehmen eine Rechnung aus, deren Betrag von der Größe des Publikums und der Art des aufgeführten Theaterstücks abhängt. Derzeit bietet das Unternehmen zwei Arten von Theaterstücken an: Tragödien und Komödien. Das Unternehmen schreibt aber nicht nur Rechnungen, sondern vergibt auch »Treuepunkte« (`volumeCredits`) an seine Kunden, für die sie bei zukünftigen Vorstellungen einen Preisnachlass erhalten. Sie können sich das als eine Maßnahme zur Kundenbindung vorstellen.

Die Darsteller speichern Informationen über die aufgeführten Theaterstücke in einer einfachen JSON-Datei, die in etwa so aussieht:

```
plays.json...
{
"hamlet": {"name": "Hamlet", "type": "tragedy"},
"as-like": {"name": "As You Like It", "type":
```

```
"comedy"},
"othello": {"name": "Othello", "type": "tragedy"}
}
```

Die Daten für die Rechnungen werden ebenfalls in Form von JSON-Dateien gespeichert:

```
invoices.json...
[
  {
    "customer": "BigCo",
    "performances": [
      {
        "playID": "hamlet",
        "audience": 55
      },
      {
        "playID": "as-like",
        "audience": 35
      },
      {
        "playID": "othello",
        "audience": 40
      }
    ]
  }
]
```

Der Code zur Ausgabe der Rechnung ist in dieser einfachen Funktion enthalten:

```
function statement (invoice, plays) {
  let totalAmount = 0;
```

```
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    const play = plays[perf.playID];
    let thisAmount = 0;

    switch (play.type) {
    case "tragedy":
      thisAmount = 40000;
      if (perf.audience > 30) {
        thisAmount += 1000 * (perf.audience - 30);
      }
      break;
    case "comedy":
      thisAmount = 30000;
      if (perf.audience > 20) {
      thisAmount += 10000 + 500 * (perf.audience -
20);
      }
      thisAmount += 300 * perf.audience;
      break;
    default:
      throw new Error(`Unbekannter Typ:
${play.type}`);
    }

    // Treuepunkte hinzufügen
    volumeCredits += Math.max(perf.audience - 30, 0);
    // Zusatzpunkte für jeweils 10 Komödienbesucher
    if ("comedy" === play.type) volumeCredits +=
                              Math.floor(perf.audience
/ 5);
    // Ausgabezeile für diesen Auftrag
    result += ` ${play.name}:
```

```
${format(thisAmount/100)}
                        (${perf.audience} Plätze)\n`;
    totalAmount += thisAmount;
  }
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
}
```

Die Ausführung dieses Codes mit den obigen Testdaten erzeugt die folgende Ausgabe:

```
Abrechnung für BigCo
  Hamlet: $650.00 (55 Plätze)
  As You Like It: $580.00 (35 Plätze)
  Othello: $500.00 (40 Plätze)
Rechnungsbetrag $1,730.00
Sie erhalten 47 Treuepunkte
```

1.2 Anmerkungen zum ersten Programm

Was halten Sie von dem Design dieses Programms? Ich würde zunächst einmal sagen, dass es in dieser Form akzeptabel ist – ein so kurzes Programm benötigt keine tiefgehende Struktur, um verständlich zu sein. Denken Sie jedoch an meinen früheren Hinweis, dass ich kurze Beispiele verwenden muss. Stellen Sie sich dieses Programm deutlich umfangreicher vor – vielleicht

mehrere hundert Zeilen lang. Bei einer derartigen Größe ist eine einzelne Funktion schwer zu verstehen.

Ist eine Aussage über die Struktur des Programms in Anbetracht der Tatsache, dass es funktioniert, nicht lediglich ein ästhetisches Urteil? Die Abneigung gegenüber »hässlichem« Code? Dem Compiler ist es schließlich völlig gleichgültig, ob der Code hässlich oder schön ist. Aber wenn ich Änderungen am System vornehme, sind Menschen beteiligt – und Menschen ist es nicht egal, wie der Code aussieht. Es ist schwierig, ein System mit schlechtem Design zu verändern – weil es schwierig ist, herauszufinden, was geändert werden muss und wie diese Änderungen mit dem vorhandenen Code zusammenwirken, um das gewünschte Verhalten zu erzielen. Und wenn es schwierig ist herauszufinden, was geändert werden muss, dann ist die Wahrscheinlichkeit groß, dass mir Fehler unterlaufen und ich Bugs verursache.

Wenn ich Änderungen an einem Programm vornehmen muss, das aus mehreren Hundert Codezeilen besteht, ist es mir daher lieber, wenn es aus einer Reihe von Funktionen und anderen Programmelementen aufgebaut ist, die es mir ermöglichen, die Funktionsweise des Programms leichter zu verstehen. Wenn das Programm keine Struktur aufweist, fällt es mir für gewöhnlich leichter, das Programm zunächst einmal zu strukturieren und erst dann die erforderlichen Änderungen vorzunehmen.

Tipp

Wenn Sie ein Programm um ein bestimmtes Feature erweitern möchten, der Code jedoch nicht in geeigneter Weise strukturiert ist, sollten Sie zunächst ein Refactoring des Programms vornehmen, um es zu vereinfachen, und anschließend das Feature hinzufügen.

In diesem Fall wünschen die Anwender eine Reihe von Änderungen. Zunächst einmal soll die Abrechnung auch in HTML ausgegeben werden. Überlegen Sie sich, welche Auswirkungen diese Änderung haben würde. Ich müsste bei jedem Befehl, der einen String hinzufügt, eine bedingte Anweisung verwenden. In Anbetracht der Tatsache, dass die Komplexität der Funktion dadurch erheblich zunimmt, ziehen es die meisten vor, die Funktion zu kopieren und so zu ändern, dass sie HTML ausgibt. Eine Kopie zu erstellen ist zwar nicht besonders mühsam, kann zukünftig aber für eine Vielzahl an Schwierigkeiten sorgen. Jede Änderung des Abrechnungsverfahrens würde mich dazu zwingen, beide Methoden zu aktualisieren – und zu gewährleisten, dass beide Methoden in übereinstimmender Weise geändert werden. Wenn ich ein Programm schreibe, das sich niemals ändern wird, ist diese Art von Copy & Paste unproblematisch. Schreibe ich hingegen ein langlebiges

Programm, kann diese Redundanz gefährlich werden.

Das führt mich zu einer zweiten Änderung. Die Schauspieler möchten weitere Arten von Theaterstücken aufführen. Sie planen, ihrem Repertoire Schäferspiele, Pastoralen, Tragikomödien, Einakter usw. hinzuzufügen. Sie haben noch nicht entschieden, was genau sie eigentlich wollen und wann sie es umsetzen werden. Die Änderungen werden sich jedoch auf das Abrechnungsverfahren und die Berechnung der Treuepunkte auswirken. Im Grunde genommen spielt es keine Rolle, welchen Spielplan sie vorlegen, denn als erfahrener Entwickler kann ich davon ausgehen, dass er in den kommenden sechs Monaten ein weiteres Mal geändert wird. Und wenn neue Features implementiert werden sollen, geht es nicht um einige wenige, sondern um raue Mengen.

Wieder müssen Änderungen an der Klassifikation und dem Abrechnungsverfahren in der Methode `statement` vorgenommen werden. Wenn ich den Code von `statement` nach `htmlStatement` kopiere, muss ich gewährleisten, dass alle Änderungen an den beiden Methoden in übereinstimmender Weise vorgenommen werden. Wenn die Komplexität des Abrechnungsverfahrens zunimmt, wird es darüber hinaus schwieriger, herauszufinden, wo die Änderungen vorgenommen werden müssen, und sie fehlerfrei durchzuführen.

Ich möchte an dieser Stelle betonen, dass es genau diese Änderungen sind, die ein Refactoring erforderlich machen. Wenn der Code funktioniert und nie geändert werden muss, ist es absolut in Ordnung, ihn nicht anzutasten. Es wäre zwar schön, ihn zu verbessern, aber wenn niemand den Code verstehen muss, verursacht er auch keine Probleme. Sobald jedoch jemand die Funktionsweise des Codes verstehen muss und Schwierigkeiten hat, diese nachzuvollziehen, müssen Sie etwas unternehmen.

1.3 Der erste Schritt des Refactorings

Wenn ich ein Refactoring vornehme, ist der erste Schritt stets der gleiche. Ich muss mich vergewissern, dass für den betreffenden Codeabschnitt verlässliche Tests existieren. Die Tests sind unverzichtbar, denn obwohl ich Refactorings auf eine strukturierte Weise durchführe, die fast alle Möglichkeiten dafür ausschließt, dass ich Bugs verursache, bin ich auch nur ein Mensch, und mir unterlaufen Fehler. Je umfangreicher ein Programm ist, desto wahrscheinlicher wird es, dass meine Änderungen versehentlich dazu führen, dass irgendetwas nicht mehr funktioniert. Im digitalen Zeitalter ist Fragilität gleichbedeutend mit Software.

Die Methode `statement` liefert einen `String` zurück, daher erstelle ich einige Abrechnungen, indem ich sie mit verschiedenen Arten von Theaterstücken aufrufe und

so den String für die Abrechnung erzeuge. Anschließend vergleiche ich die Ergebnisse mit verschiedenen, von Hand erzeugten Referenzstrings. Zur Einrichtung dieser Tests verwende ich ein Test-Framework, sodass ich sie durch einen einfachen Tastendruck in meiner Entwicklungsumgebung ausführen kann. Die Ausführung der Tests dauert nur ein paar Sekunden, und wie Sie sehen werden, führe ich sie häufig aus.

Ein wichtiger Bestandteil der Tests ist die Art und Weise, wie die Ergebnisse dargestellt werden. Sie sind entweder grün, was bedeutet, dass alle Strings mit den Referenzstrings übereinstimmen, oder rot, in Form einer Liste von aufgetretenen Fehlern – die Zeilen, die ein abweichendes Ergebnis enthalten. Die Tests sind also selbsttestend, und das ist unverzichtbar. Anderenfalls müsste ich die Testergebnisse am Schreibtisch von Hand mit den Referenzwerten vergleichen, und das würde mich stark verlangsamen. Aktuelle Test-Frameworks verfügen über alle notwendigen Features, um selbsttestende Tests zu schreiben.

Tipp

Vergewissern Sie sich, dass Ihnen verlässliche Tests zur Verfügung stehen, bevor Sie das Refactoring durchführen. Diese Tests müssen selbsttestend sein.

Bei der Durchführung des Refactorings verlasse ich mich auf die Tests. Ich stelle sie mir als Bug-Detektoren vor, die mich vor meinen eigenen Fehlern schützen. Um mein Ziel zu erreichen, muss ich es zweimal formulieren, sowohl im Code als auch im Test. Ich müsste einen Fehler an beiden Stellen und auf übereinstimmende Weise begehen, um den Detektor zu umgehen. Durch diese doppelte Überprüfung verringere ich die Wahrscheinlichkeit, etwas falsch zu machen. Es kostet zwar etwas Zeit, die Tests zu erstellen, das macht die beim Debugging eingesparte Zeit jedoch mehr als wett. Dieser Teil ist bei der Durchführung eines Refactorings von so großer Bedeutung, dass ich ihm ein eigenes Kapitel gewidmet habe (Kapitel 4, *Tests erstellen*).

1.4 Aufteilung der statement-Funktion

Beim Refactoring einer umfangreichen Funktion wie dieser versuche ich im Geiste bestimmte Punkte zu identifizieren, die verschiedene Teile des Gesamtverhaltens voneinander trennen. Der erste Codeabschnitt, der mir ins Auge fällt, ist die in der Mitte befindliche `switch`-Anweisung.

```
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
```

```
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    const play = plays[perf.playID];
    let thisAmount = 0;

    switch (play.type) {
    case "tragedy":
      thisAmount = 40000;
      if (perf.audience > 30) {
        thisAmount += 1000 * (perf.audience - 30);
      }
      break;
    case "comedy":
      thisAmount = 30000;
      if (perf.audience > 20) {
      thisAmount += 10000 + 500 * (perf.audience -
20);
      }
      thisAmount += 300 * perf.audience;
      break;
    default:
      throw new Error(`Unbekannnter Typ:
${play.type}`);
    }

    // Treuepunkte hinzufügen
    volumeCredits += Math.max(perf.audience - 30, 0);
    // Zusatzpunkte für jeweils 10 Komödienbesucher
    if ("comedy" === play.type) volumeCredits +=
                              Math.floor(perf.audience
/ 5);
    // Ausgabezeile für diesen Auftrag
    result += ` ${play.name}:
${format(thisAmount/100)}
                        (${perf.audience} Plätze)\n`;
    totalAmount += thisAmount;
  }
```

```
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
}
```

Wenn ich diesen Abschnitt betrachte, komme ich zu dem Ergebnis, dass er die Kosten für eine Vorstellung berechnet. Diese Schlussfolgerung ist eine Erkenntnis über den Code. Ward Cunningham aber würde sagen, dass sich diese Erkenntnis in meinem Kopf befindet – eine bekanntermaßen sehr flüchtige Form eines Speichers. Ich muss diese Erkenntnis dauerhaft speichern, indem ich sie von meinem Kopf zurück in den Code selbst übertrage. Auf diese Weise verrät mir der Code seine Funktionsweise, wenn ich später an diese Stelle zurückkehre – ich muss sie nicht erneut herausfinden.

Diese Erkenntnis wird in den Code übertragen, indem der zugehörige Abschnitt zu einer eigenen Funktion umgewandelt wird, die entsprechend ihrer Aufgabe benannt wird – also etwa `amountFor(aPerformance)`. Wenn ich wie hier einen Codeabschnitt in eine Funktion umwandeln möchte, verwende ich dafür ein Verfahren, das die Wahrscheinlichkeit minimiert, dass ich diese Umwandlung falsch angehe. Ich habe dieses Verfahren dokumentiert und *Funktion extrahieren* (Abschnitt 6.1)

genannt, damit man leicht darauf verweisen kann.

Als Erstes muss ich in dem Codeabschnitt nach Variablen suchen, die nicht mehr zum Gültigkeitsbereich gehören, sobald sich der Code in einer eigenen Funktion befindet. In diesem Fall gibt es davon drei: `perf`, `play` und `thisAmount`. Die ersten beiden werden vom extrahierten Code zwar verwendet, aber nicht verändert, also kann ich sie als Parameter übergeben. Bei Variablen, die verändert werden, muss ich ein wenig mehr Aufwand betreiben. Hier gibt es nur eine, also kann ich sie zurückgeben. Zudem kann ich ihre Initialisierung im extrahierten Code vornehmen. Damit ergibt sich Folgendes:

```
function statement...
function amountFor(perf, play) {
  let thisAmount = 0;
  switch (play.type) {
  case "tragedy":
    thisAmount = 40000;
    if (perf.audience > 30) {
      thisAmount += 1000 * (perf.audience - 30);
    }
    break;
  case "comedy":
    thisAmount = 30000;
    if (perf.audience > 20) {
      thisAmount += 10000 + 500 * (perf.audience -
20);
    }
    thisAmount += 300 * perf.audience;
    break;
```

```
  default:
      throw new Error(`Unbekannter Typ:
${play.type}`);
  }
  return thisAmount;
}
```

Wenn ich eine kursiv gedruckte Überschrift wie

```
function einName...
```

verwende, bedeutet dies, dass sich der nachfolgende Code innerhalb des Gültigkeitsbereichs der angegebenen Funktion, Datei oder Klasse befindet. Für gewöhnlich gibt es noch weiteren Code, der zu diesem Gültigkeitsbereich gehört, der allerdings nicht abgedruckt wird, weil er für das aktuelle Thema nicht von Bedeutung ist.

Die Funktion `statement` ruft ihrerseits nun diese Funktion auf, um `thisAmount` einen Wert zuzuweisen:

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
```

```
}).format;
  for (let perf of invoice.performances) {
    const play = plays[perf.playID];
    let thisAmount = amountFor(perf, play);

  // Treuepunkte hinzufügen
  volumeCredits += Math.max(perf.audience - 30, 0);
  // Zusatzpunkte für jeweils 10 Komödienbesucher
  if ("comedy" === play.type) volumeCredits +=
                              Math.floor(perf.audience
/ 5);
  // Ausgabezeile für diesen Auftrag
  result += ` ${play.name}: ${format(thisAmount/100)}
                                 (${perf.audience}
Plätze)\n`;
  totalAmount += thisAmount;
}
result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
return result;
```

Sofort nach Durchführen dieser Änderung kompiliere ich den Code und führe die Tests aus, um zu prüfen, ob irgendetwas nicht mehr funktioniert. Es ist eine wichtige Angewohnheit, nach jedem Refactoring, mag es auch noch so einfach sein, zu testen. Man macht sehr leicht Fehler – jedenfalls geht es mir so. Sollte mir also ein Fehler unterlaufen, muss ich nur eine kleine Änderung berücksichtigen, um ihn zu finden, wenn ich gewissenhaft nach jeder Änderung die Tests durchführe. Die Fehlersuche und -behebung wird dadurch sehr vereinfacht. Das ist das Entscheidende beim Refactoring:

kleine Änderungen vornehmen und nach jeder Änderung testen. Wenn ich versuche, zu große Änderungen umzusetzen, zwingt mich ein Fehler zu einem kniffligen und langwierigen Debugging, das viel Zeit in Anspruch nehmen kann. Kleine Änderungen, die schnelles Feedback ermöglichen, sind der Schlüssel dazu, ein solches Durcheinander zu vermeiden.

Hinweis

Mit dem Begriff »kompilieren« ist hier gemeint, die notwendigen Schritte zu unternehmen, um den JavaScript-Code auszuführen. Da JavaScript direkt ausführbar ist, brauchen ggf. keine weiteren Schritte zu erfolgen. In manchen Fällen ist es jedoch erforderlich, den Code in ein Ausgabeverzeichnis zu verschieben oder einen sprachspezifischen Compiler wie Babel (`https://babeljs.io`) anzuwenden.

Tipp

Das Refactoring ändert Programme in kleinen Schritten, sodass es leichtfällt, einen Bug im Code aufzuspüren, wenn Sie einen Fehler begehen.

Da es sich hier um JavaScript handelt, kann ich `amountFor` als verschachtelte Funktion in `statement` unterbringen. Das hat den Vorteil, dass ich keine Daten aus dem Gültigkeitsbereich der umgebenden Funktion an die neu extrahierte Funktion übergeben muss. In diesem Fall ergibt sich dadurch kein Unterschied, aber immerhin gibt es ein Problem weniger, das gehandhabt werden muss.

Die Tests werden bestanden, also besteht der nächste Schritt darin, die Änderungen in meine lokale Versionsverwaltung einzuchecken. Ich verwende eine Versionsverwaltung wie git oder Mercurial, die es mir ermöglicht, private Commits vorzunehmen. Nach jeder erfolgreichen Durchführung eines Refactorings checke ich die Änderungen ein, sodass ich mühelos zu einer funktionierenden Version zurückkehren kann, falls ich später etwas durcheinanderbringen sollte. Die gesammelten Änderungen fasse ich dann zu umfassenderen Commits zusammen, bevor ich sie in einem gemeinsam genutzten Repository einchecke.

Funktion extrahieren (Abschnitt 6.1) ist ein Refactoring, das häufig automatisiert wird. Wenn ich in Java programmieren würde, hätte ich instinktiv die Tastenkombination in meiner IDE gedrückt, die dieses Refactoring durchführt. Für JavaScript gibt es aktuell noch keine so ausgereifte Unterstützung für dieses Refactoring, daher führe ich es von Hand durch. Das ist zwar nicht schwer, allerdings muss ich bei den Variablen

mit lokalem Gültigkeitsbereich sehr aufmerksam sein.

Nach der Verwendung von *Funktion extrahieren* (Abschnitt 6.1) sehe ich mir den Code an, um zu überprüfen, ob es irgendeine schnelle und einfache Möglichkeit gibt, die extrahierte Funktion verständlicher zu machen. Zunächst einmal benenne ich einige Variablen um, damit die Bezeichnungen aussagekräftiger sind. Aus `thisAmount` wird beispielsweise `result`.

```
function statement...
function amountFor(perf, play) {
  let result = 0;
  switch (play.type) {
  case "tragedy":
    result = 40000;
    if (perf.audience > 30) {
      result += 1000 * (perf.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (perf.audience > 20) {
      result += 10000 + 500 * (perf.audience - 20);
    }
    result += 300 * perf.audience;
    break;
  default:
      throw new Error(`Unbekannter Typ:
${play.type}`);
  }
  return result;
}
```

Es gehört zu meinem Programmierstil, als Bezeichnung für den Rückgabewert einer Funktion immer »result« zu verwenden. Auf diese Weise ist mir jederzeit klar, welche Rolle die Variable einnimmt. Und wieder kompiliere ich, führe die Tests aus und checke den Code ein. Anschließend geht es mit dem ersten Argument weiter.

```
function statement...
function amountFor(aPerformance, play) {
  let result = 0;
  switch (play.type) {
  case "tragedy":
    result = 40000;
    if (aPerformance.audience > 30) {
      result += 1000 * (aPerformance.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (aPerformance.audience > 20) {
      result += 10000 + 500 * (aPerformance.audience -
20);
    }
    result += 300 * aPerformance.audience;
    break;
  default:
      throw new Error(`Unbekannter Typ:
${play.type}`);
  }
  return result;
}
```

Auch diese Bearbeitung folgt wieder meinem eigenen Programmierstil. Bei einer dynamisch typisierten

Sprache wie JavaScript ist es sinnvoll, die Datentypen im Auge zu behalten – deshalb enthält meine Standardbezeichnung für einen Parameter den Namen des Typs. Außerdem verwende ich einen unbestimmten Artikel, sofern es keine speziellen Informationen über die Rolle des Parameters gibt, die über die Bezeichnung ausgedrückt werden sollte. Ich habe diese Konvention von Kent Beck (Kent Beck: *Smalltalk Best Practice Patterns*. Addison-Wesley, 1997, ISBN 013476904X) übernommen und halte sie für sehr hilfreich.

Tipp

Code schreiben, den ein Computer versteht, kann jeder. Gute Programmierer schreiben Code, den Menschen verstehen.

Ist diese Umbenennung die Mühe wert? Unbedingt. Guter Code sollte deutlich ausdrücken, was er bewirkt, und Variablennamen tragen ganz entscheidend zu verständlichem Code bei. Zögern Sie nie, Bezeichnungen zu ändern, um die Verständlichkeit zu verbessern. Mit geeigneten Suchen-und-Ersetzen-Funktionen ist das für gewöhnlich ganz einfach. Tests und statisch typisierte Programmiersprachen heben die Stellen im Code hervor, die Sie womöglich übersehen haben. Mit automatisierten Refactoring-Tools ist es dann geradezu trivial, selbst sehr

häufig verwendete Funktionen umzubenennen.

Als Nächstes sollte bei der Umbenennung der Bezeichnungen der Parameter `play` berücksichtigt werden, dem allerdings ein anderes Schicksal bevorsteht.

1.4.1 ENTFERNEN DER VARIABLE PLAY

Da ich mir Gedanken über die Parameter für `amountFor` mache, betrachte ich deren Ursprung. `aPerformance` gehört zur Schleife und ändert sich dementsprechend bei jedem Durchlauf. `play` wird dagegen anhand der jeweiligen Vorstellung berechnet, daher ist es überhaupt nicht nötig, diesen Wert als Parameter zu übergeben – ich kann ihn innerhalb von `amountFor` einfach neu berechnen. Beim Aufteilen einer langen Funktion versuche ich, Variablen wie `play` zu eliminieren, weil temporäre Variablen zahlreiche Bezeichnungen mit lokalem Gültigkeitsbereich erzeugen, die das Extrahieren von Code verkomplizieren. Das hier verwendete Refactoring heißt *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4).

Ich extrahiere zunächst die rechte Seite der Zuweisung und mache daraus eine eigene Funktion.

```
function statement...
function playFor(aPerformance) {
  return plays[aPerformance.playID];
}
```

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    const play = playFor(perf);
    let thisAmount = amountFor(perf, play);

  // Treuepunkte hinzufügen
  volumeCredits += Math.max(perf.audience - 30, 0);
  // Zusatzpunkte für jeweils 10 Komödienbesucher
  if ("comedy" === play.type) volumeCredits +=
                              Math.floor(perf.audience
/ 5);
  // Ausgabezeile für diesen Auftrag
  result += ` ${play.name}: ${format(thisAmount/100)}
                                ($${perf.audience}
Plätze)\n`;
  totalAmount += thisAmount;
}
result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
return result;
```

Ich kompiliere, teste, checke ein und verwende anschließend das Refactoring *Variable inline platzieren* (Abschnitt 6.4).

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    const play = plays[perf.playID];
    let thisAmount = amountFor(perf, playFor(perf));

  // Treuepunkte hinzufügen
  volumeCredits += Math.max(perf.audience - 30, 0);
  // Zusatzpunkte für jeweils 10 Komödienbesucher
  if ("comedy" === playFor(perf).type) volumeCredits
+=
                              Math.floor(perf.audience
/ 5);
  // Ausgabezeile für diesen Auftrag
  result += `${playFor(perf).name}:
${format(thisAmount/100)}
                                ($${perf.audience}
Plätze)\n`;
  totalAmount += thisAmount;
}
result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
return result;
```

Ich kompiliere, teste, checke ein. Dank der inline platzierten Variablen kann ich *Funktionsdeklaration*

ändern (Abschnitt 6.5) auf `amountFor` anwenden, um den Parameter `play` zu entfernen. Ich erledige diese Aufgabe in zwei Schritten. Zunächst verwende ich innerhalb von `amountFor` die neue Funktion:

```
function statement...
function amountFor(aPerformance, play) {
  let result = 0;
  switch (playFor(aPerformance).type) {
  case "tragedy":
    result = 40000;
    if (aPerformance.audience > 30) {
      result += 1000 * (aPerformance.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (aPerformance.audience > 20) {
      result += 10000 + 500 * (aPerformance.audience -
20);
    }
    result += 300 * aPerformance.audience;
    break;
  default:
      throw new Error(`Unbekannnter Typ:
                        ${playFor(aPerformance).type}`);
  }
  return result;
}
```

Ich kompiliere, teste, checke ein und lösche den Parameter.

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    let thisAmount = amountFor(perf, playFor(perf));

  // Treuepunkte hinzufügen
  volumeCredits += Math.max(perf.audience - 30, 0);
  // Zusatzpunkte für jeweils 10 Komödienbesucher
  if ("comedy" === playFor(perf).type) volumeCredits
+=
                              Math.floor(perf.audience
/ 5);
  // Ausgabezeile für diesen Auftrag
  result += `${playFor(perf).name}:
                             ${format(thisAmount/100)}
                            (${perf.audience}
Plätze)\n`;
  totalAmount += thisAmount;
}
result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
return result;

function statement...
function amountFor(aPerformance, play) {
  let result = 0;
  switch (playFor(aPerformance).type) {
  case "tragedy":
```

```
    result = 40000;
    if (aPerformance.audience > 30) {
      result += 1000 * (aPerformance.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (aPerformance.audience > 20) {
      result += 10000 + 500 * (aPerformance.audience -
20);
    }
    result += 300 * aPerformance.audience;
    break;
  default:
      throw new Error(`Unbekannnter Typ:
                      ${playFor(aPerformance).type}`);
  }
  return result;
}
```

Und wieder kompilieren, testen, einchecken.

Dieses Refactoring beunruhigt einige Programmierer. Vorher wurde der Code zum Abrufen der Vorstellung (`play`) einmal pro Schleifendurchlauf aufgerufen, jetzt erfolgt der Aufruf dreimal. Auf die Wechselwirkungen zwischen Refactoring und Performance komme ich später noch zu sprechen. Fürs Erste stelle ich hier nur fest, dass diese Änderung höchstwahrscheinlich keine merkliche Auswirkung auf die Performance haben wird. Und selbst wenn es so wäre, ist es viel einfacher, die Performance einer gut strukturierten Codebasis zu verbessern.

Der größte Nutzen des Entfernens lokaler Variablen besteht darin, dass das Extrahieren von Code viel einfacher wird, weil man sich weniger um den lokalen Gültigkeitsbereich kümmern muss. Tatsächlich entferne ich lokale Variablen für gewöhnlich, bevor ich Code extrahiere.

Nun bin ich mit den Argumenten der Funktion `amountFor` fertig und sehe mir an, wo die Funktion aufgerufen wird. Sie wird dazu verwendet, einer temporären Variable einen Wert zuzuweisen, die nie wieder aktualisiert wird, deshalb platziere ich sie inline.

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {

    // Treuepunkte hinzufügen
    volumeCredits += Math.max(perf.audience - 30, 0);
    // Zusatzpunkte für jeweils 10 Komödienbesucher
    if ("comedy" === playFor(perf).type) volumeCredits
+=
                              Math.floor(perf.audience
/ 5);
    // Ausgabezeile für diesen Auftrag
    result += `${playFor(perf).name}:
```

```
${format(amountFor(perf)/100)}
                              (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

1.4.2 EXTRAHIEREN DER BERECHNUNG VON TREUEPUNKTEN

Die `statement`-Funktion sieht derzeit folgendermaßen aus:

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {

    // Treuepunkte hinzufügen
    volumeCredits += Math.max(perf.audience - 30, 0);
    // Zusatzpunkte für jeweils 10 Komödienbesucher
    if ("comedy" === playFor(perf).type) volumeCredits
+=
                              Math.floor(perf.audience
```

```
/ 5);
    // Ausgabezeile für diesen Auftrag
    result += `${playFor(perf).name}:

${format(amountFor(perf)/100)}
                              (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Jetzt kann ich den Nutzen daraus ziehen, dass ich die Variable `play` entfernt habe, denn das Entfernen einer der Variablen mit lokalem Gültigkeitsbereich vereinfacht das Extrahieren der Berechnung der Treuepunkte erheblich.

Ich muss mich allerdings immer noch um die beiden anderen Variablen kümmern. Auch hier lässt sich `perf` problemlos übergeben, aber bei `volumeCredits` wird es etwas kniffliger, weil es sich um eine Variable handelt, die bei jedem Durchlauf der Schleife aktualisiert wird. Es dürfte also das Beste sein, innerhalb der extrahierten Funktion eine Kopie zu initialisieren und diese zurückzugeben.

```
function statement...
function volumeCreditsFor(perf) {
```

```
  let volumeCredits = 0;
  volumeCredits += Math.max(perf.audience - 30, 0);
  if ("comedy" === playFor(perf).type) volumeCredits
+=
                            Math.floor(perf.audience /
5);
  return volumeCredits;
}

Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung  für
${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:

${format(amountFor(perf)/100)}
                              (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Ich entferne den unnötigen (und hier geradezu

irreführenden) Kommentar. Ich kompiliere, teste, checke ein und benenne anschließend die Variablen in der neuen Funktion um.

```
function statement...
function volumeCreditsFor(aPerformance) {
  let result = 0;
  result += Math.max(aPerformance.audience - 30, 0);
  if ("comedy" === playFor(aPerformance).type) result
+=
                    Math.floor(aPerformance.audience /
5);
  return result;
}
```

Hier ist alles in einem Schritt dargestellt, aber tatsächlich habe ich die Variablen der Reihe nach einzeln umbenannt und jeweils kompiliert, getestet und eingecheckt.

1.4.3 ENTFERNEN DER VARIABLE FORMAT

Sehen wir uns noch einmal die Methode `statement` an:

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  const format = new Intl.NumberFormat("en-US",
                        { style: "currency", currency:
```

```
"USD",
                          minimumFractionDigits: 2
}).format;
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
    // Ausgabezeile für diesen Auftrag
    result += `${playFor(perf).name}:

${format(amountFor(perf)/100)}
                              (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Wie bereits erwähnt, können temporäre Variablen Schwierigkeiten bereiten. Sie sind nur in ihrer eigenen Programmroutine von Nutzen und verleiten daher dazu, lange und komplexe Routinen zu programmieren. Mein nächster Schritt besteht also darin, einige der temporären Variablen zu ersetzen. Am einfachsten ist das bei `format`. In diesem Fall wird eine Funktion einer temporären Variable zugewiesen. Ich ziehe es vor, die Zuweisung durch eine deklarierte Funktion zu ersetzen.

```
function statement...
function format(aNumber) {
return new Intl.NumberFormat("en-US",
               { style: "currency", currency: "USD",
                 minimumFractionDigits: 2
```

```
}).format(aNumber);
}
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;

  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:

${format(amountFor(perf)/100)}
                              (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  result += `Rechnungsbetrag
${format(totalAmount/100)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Obwohl das Ändern einer Funktionsvariablen zu einer deklarierten Funktion ein Refactoring ist, habe ich es dennoch nicht benannt und auch nicht in den Katalog aufgenommen. Es gibt eine ganze Reihe von Refactorings, die nach meinem Eindruck dafür zu unbedeutend sind. Dieses hier ist sowohl einfach als auch relativ selten, daher denke ich, dass sich die Aufnahme in den Katalog nicht lohnt.

Die Bezeichnung gefällt mir nicht – »format« sagt zu

wenig darüber aus, welche Aufgabe die Funktion eigentlich erfüllt. »formatAsUSD« wiederum wäre etwas umständlich, zumal die Bezeichnung in einem Template-String verwendet wird, insbesondere in diesem kleinen Geltungsbereich. Meiner Ansicht nach sollte hervorgehoben werden, dass die Funktion einen Währungsbetrag formatiert. Deshalb wähle ich eine entsprechende Bezeichnung aus und verwende das Refactoring *Funktionsdeklaration ändern* (Abschnitt 6.5).

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;

  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  result += `Rechnungsbetrag ${usd(totalAmount)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;

function statement...
function usd(aNumber) {
  return new Intl.NumberFormat("en-US",
           { style: "currency", currency: "USD",
```

```
                minimumFractionDigits: 2
}).format(aNumber/100);
```

Die Namensgebung ist sowohl wichtig als auch knifflig. Die Aufteilung einer großen Funktion in mehrere kleinere ist nur dann sinnvoll, wenn gute Bezeichnungen gewählt werden. Wenn ich aussagekräftige Bezeichnungen verwende, muss ich mir nicht die gesamte Funktion ansehen, um zu verstehen, welche Aufgabe sie erledigt. Es ist allerdings nicht ganz einfach, auf Anhieb wirklich gut geeignete Bezeichnungen zu finden, deshalb verwende ich anfangs den besten Namen, der mir einfällt, zögere aber nicht, ihn später zu ändern. Häufig muss man den Code ein zweites Mal durchgehen, um zu erkennen, welche Bezeichnung tatsächlich am besten geeignet ist.

Beim Ändern des Namens verschiebe ich auch die vorher doppelt vorhandene Division durch 100 in die Funktion. Geldbeträge als Cent-Integers zu speichern ist ein gebräuchlicher Ansatz, weil sich so vermeiden lässt, Cent-Beträge als Floats zu speichern, ohne darauf verzichten zu müssen, arithmetische Operatoren zu verwenden. Bei der Darstellung solcher Beträge benötige ich jedoch eine Dezimalzahl, deshalb soll sich meine Formatierungsfunktion um die Division kümmern.

1.4.4 ENTFERNEN DER GESAMTZAHL DER TREUEPUNKTE

Jetzt nehme ich die Variable `volumeCredits` ins Visier, in der die Treuepunkte gespeichert werden. Hier wird es etwas kniffliger, weil sie sich bei jedem Durchlauf der Schleife ändert. Mein erster Schritt ist also die Verwendung von *Schleife aufteilen* (Abschnitt 8.7), um das Aufsummieren der Treuepunkte auszulagern.

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let volumeCredits = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;

  for (let perf of invoice.performances) {
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
  }

  result += `Rechnungsbetrag ${usd(totalAmount)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Nachdem dieser Arbeitsschritt erledigt ist, kann ich *Anweisungen verschieben* (Abschnitt 8.6) verwenden, um die Variablendeklaration direkt vor der

dazugehörigen Schleife zu platzieren.

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  let volumeCredits = 0;
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
  }

  result += `Rechnungsbetrag ${usd(totalAmount)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Den Code zu vereinen, der die Variable `volumeCredits` aktualisiert, vereinfacht die Anwendung von *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4). Der erste Schritt besteht wieder darin, *Funktion extrahieren* (Abschnitt 6.1) auf die gesamte Berechnung der Variable anzuwenden.

```
function statement...
function totalVolumeCredits() {
```

```
  let volumeCredits = 0;
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
  }
  return volumeCredits;
}

Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  let volumeCredits = totalVolumeCredits();
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
  }

  result += `Rechnungsbetrag ${usd(totalAmount)}\n`;
  result += `Sie erhalten ${volumeCredits}
Treuepunkte\n`;
  return result;
```

Jetzt ist alles extrahiert und ich kann *Variable inline platzieren* (Abschnitt 6.4) anwenden:

```
Oberste Ebene...
function statement (invoice, plays) {
  let totalAmount = 0;
  let result = `Abrechnung für ${invoice.customer}\n`;
```

```
  for (let perf of invoice.performances) {
    // Ausgabezeile für diesen Auftrag
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
    totalAmount += amountFor(perf);
  }
  let volumeCredits = totalVolumeCredits();
  for (let perf of invoice.performances) {
    volumeCredits += volumeCreditsFor(perf);
  }

  result += `Rechnungsbetrag ${usd(totalAmount)}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;
```

An dieser Stelle möchte ich kurz resümieren, was ich hier gerade getan habe. Zunächst einmal ist mir klar, dass sich die Leser aufgrund dieser Änderung wieder Gedanken wegen der Performance machen, denn die Wiederholung von Schleifen mögen viele als kritisch ansehen. In den meisten Fällen hat die Wiederholung einer Schleife wie dieser jedoch eine vernachlässigbare Auswirkung auf die Performance. Wenn Sie die Ausführungszeit vor und nach diesem Refactoring messen, werden Sie vermutlich keinen nennenswerten Unterschied bemerken – und das ist fast immer so. Die meisten Programmierer, selbst erfahrene, können schwer einschätzen, wie schnell der Code tatsächlich ausgeführt wird. Häufig wird unser Gespür dafür durch

clevere Compiler, moderne Caching-Verfahren und Ähnliches in die Irre geführt. Die Performance von Software hängt meistens nur von einigen wenigen Codebestandteilen ab. Änderungen an den übrigen Stellen führen zu keinem merklichen Unterschied.

Aber »meistens« ist natürlich nicht dasselbe wie »immer«. Manchmal wirkt sich ein Refactoring ganz erheblich auf die Performance aus. Dessen ungeachtet führe ich es normalerweise trotzdem aus, denn es ist viel einfacher, die Performance des Codes zu verbessern, wenn er gut strukturiert ist. Sollte ich beim Durchführen des Refactorings spürbare Probleme bei der Performance verursachen, widme ich mich erst im zweiten Schritt der Verbesserung der Geschwindigkeit. Unter Umständen führt das dazu, dass ich einige der zuvor durchgeführten Refactorings wieder rückgängig machen muss. In den meisten Fällen jedoch kann ich dank des Refactorings viel effektivere Maßnahmen zur Verbesserung der Performance ergreifen. Letzen Endes ist der Code sowohl besser verständlich als auch schneller.

Mein allgemeiner Rat hinsichtlich der Performance beim Refactoring lautet also: In den meisten Fällen brauchen Sie sich darum nicht zu kümmern. Falls Ihre Refactorings Verschlechterungen der Performance verursachen, sollten Sie das Refactoring zunächst abschließen und sich erst anschließend mit der Verbesserung der Geschwindigkeit befassen.

Der zweite Aspekt, auf den ich Sie aufmerksam machen möchte, ist, wie klein die Schritte zum Entfernen von `volumeCredits` tatsächlich waren. Hier sind die vier Schritte aufgeführt, denen jeweils Kompilieren, Testen und Einchecken in das lokale Repository folgte:

- *Schleife aufteilen* (Abschnitt 8.7)
- *Anweisungen verschieben* (Abschnitt 8.6)
- *Funktion extrahieren* (Abschnitt 6.1)
- *Variable inline platzieren* (Abschnitt 6.4)

Ich muss gestehen, dass ich nicht immer so kleine Schritte wie hier dargestellt ausführe – wenn ich allerdings auf Schwierigkeiten stoße, ist meine erste Reaktion darauf, zu kleineren Schritten überzugehen. Wenn während des Refactorings ein Test fehlschlägt und ich die Ursache nicht sofort finden und beheben kann, kehre ich zurück zu der zuletzt eingecheckten funktionierenden Version und wiederhole das, was ich zuvor durchgeführt habe, mit kleineren Schritten. Das funktioniert, weil ich meinen Code so häufig einchecke und weil kleine Schritte der Schlüssel dazu sind, schnell voranzukommen, insbesondere beim Umgang mit schwierigem Code.

Zurück zum Beispiel. Ich wiederhole die genannten Schritte, um `totalAmount` zu entfernen. Zuerst teile ich die Schleife auf (Kompilieren, Testen, Einchecken), dann verschiebe ich die Initialisierung der Variable

(Kompilieren, Testen, Einchecken) und extrahiere anschließend die Funktion. Hier gibt es ein kleines Problem: Der beste Name für die Funktion wäre »totalAmount«, das ist jedoch der Name der Variable, und beides gleichzeitig geht nicht. Deshalb vergebe ich einen beliebigen Namen, wenn ich die Funktion extrahiere (und kompiliere, teste und einchecke).

```
function statement...
function appleSauce() {
  let totalAmount = 0;
  for (let perf of invoice.performances) {
    totalAmount += amountFor(perf);
  }
  return totalAmount;
}

Oberste Ebene...
function statement (invoice, plays) {
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
  }
  totalAmount = appleSauce();

  result += `Rechnungsbetrag ${usd(totalAmount)}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;
```

Jetzt platziere ich die Variable inline (Kompilieren, Testen, Einchecken) und gebe der Funktion einen sinnvolleren Namen (Kompilieren, Testen, Einchecken).

```
Oberste Ebene...
function statement (invoice, plays) {
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
  }
  result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;

function statement...
function totalAmount() {
  let totalAmount = 0;
  for (let perf of invoice.performances) {
    totalAmount += amountFor(perf);
  }
  return totalAmount;
}
```

Ich nehme außerdem die Gelegenheit wahr, die Bezeichnungen innerhalb der extrahierten Funktionen zu ändern, damit sie meiner Konvention entsprechen.

```
function statement...
function totalAmount() {
```

```
  let result = 0;
  for (let perf of invoice.performances) {
    result += amountFor(perf);
  }
  return result;
}
function totalVolumeCredits() {
  let result = 0;
  for (let perf of invoice.performances) {
    result += volumeCreditsFor(perf);
  }
  return result;
}
```

1.5 Status: Eine Menge verschachtelter Funktionen

Jetzt ist ein guter Zeitpunkt gekommen, um den Zustand des Codes als Ganzes zu betrachten.

```
function statement (invoice, plays) {
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
  }
  result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;

  function totalAmount() {
```

```
    let result = 0;
    for (let perf of invoice.performances) {
      result += amountFor(perf);
    }
  return result;
  }
  function totalVolumeCredits() {
    let result = 0;
    for (let perf of invoice.performances) {
      result += volumeCreditsFor(perf);
    }
    return result;
  }
  function usd(aNumber) {
    return new Intl.NumberFormat("en-US",
          { style: "currency", currency: "USD",
            minimumFractionDigits: 2
}).format(aNumber/100);
  }
  function volumeCreditsFor(aPerformance) {
    let result = 0;
    result += Math.max(aPerformance.audience - 30, 0);
    if ("comedy" === playFor(aPerformance).type)
result +=
                      Math.floor(aPerformance.audience
/ 5);
    return result;
  }
  function playFor(aPerformance) {
    return plays[aPerformance.playID];
  }
  function amountFor(aPerformance) {
    let result = 0;
    switch (playFor(aPerformance).type) {
    case "tragedy":
      result = 40000;
      if (aPerformance.audience > 30) {
        result += 1000 * (aPerformance.audience - 30);
      }
```

```
      break;
    case "comedy":
      result = 30000;
      if (aPerformance.audience > 20) {
        result += 10000 + 500 * (aPerformance.audience
- 20);
      }
      result += 300 * aPerformance.audience;
      break;
    default:
        throw new Error(`Unbekannter Typ:

${playFor(aPerformance).type}`);
    }
    return result;
  }
}
```

Die Struktur des Codes ist jetzt viel besser. Die `statement`-Funktion auf der obersten Ebene besteht (ohne Zeilenumbrüche) nur noch aus sieben Codezeilen, und ihre einzige Aufgabe besteht darin, für die Ausgabe der Rechnung zu sorgen. Alle Berechnungen wurden in eine Handvoll Hilfsfunktionen verlagert. Auf diese Weise sind die einzelnen Berechnungen und der allgemeine Ablauf verständlicher.

1.6 Die Phasen Berechnung und Formatierung voneinander trennen

Bislang habe ich mich beim Refactoring darauf konzentriert, die Funktion so weit zu strukturieren, dass ich sie verstehe und ihren logischen Aufbau erkenne. Das

ist am Anfang des Refactorings häufig der Fall. Komplizierte Abschnitte in kleinere aufzuteilen ist ebenso wichtig wie die Verwendung aussagekräftiger Bezeichnungen. Jetzt kann ich damit beginnen, mich stärker auf die Änderungen der Funktionalität zu konzentrieren, die ich vornehmen möchte – nämlich eine HTML-Version zu erstellen. Das ist nach den bisher beschriebenen Änderungen in vielerlei Hinsicht einfacher. Der gesamte Code für die Berechnungen ist nun ausgelagert, und ich muss nur noch eine HTML-Version der sieben Codezeilen am Anfang schreiben. Das Problem besteht hier darin, dass die ausgelagerten Funktionen und die Methode zur Textausgabe ineinander verschachtelt sind. Ich möchte aber auf jeden Fall vermeiden, sie einfach in eine neue Funktion zu kopieren, egal, wie gut diese auch organisiert sein mag. Vielmehr möchte ich erreichen, dass die Text- und die HTML-Version der Rechnung dieselben Berechnungsfunktionen verwenden.

Hierfür gibt es mehrere Möglichkeiten, allerdings verwende ich zu diesem Zweck am liebsten das Refactoring *Phase aufteilen* (Abschnitt 6.11). Ich habe hier zum Ziel, die Programmlogik in zwei Phasen aufzuteilen: Die eine berechnet die für die Rechnung erforderlichen Daten, die andere gibt eine Text- oder HTML-Version der Rechnung aus. Die erste Phase erzeugt eine temporäre Datenstruktur, die an die zweite Phase übergeben wird.

Ich führe *Phase aufteilen* (Abschnitt 6.11) durch, indem ich zunächst *Funktion extrahieren* (Abschnitt 6.1) auf den Code der zweiten Phase anwende. Hierbei handelt es sich um den Code zur Ausgabe der Rechnung – tatsächlich ist das auch schon der gesamte Inhalt von `statement`. Dieser Code und die verschachtelten Funktionen bilden eine eigene Funktion auf oberster Ebene, die ich `renderPlainText` nenne.

```
function statement (invoice, plays) {
  return renderPlainText(invoice, plays);
}

function renderPlainText(invoice, plays) {
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
  }
  result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;

function totalAmount() {...}
function totalVolumeCredits() {...}
function usd(aNumber) {...}
function volumeCreditsFor(aPerformance) {...}
function playFor(aPerformance) {...}
function amountFor(aPerformance) {...}
```

Ich kompiliere, teste und checke wie üblich ein und erzeuge ein Objekt, das als temporäre Datenstruktur zwischen den beiden Phasen fungiert. Dieses Objekt übergebe ich als Argument an `renderPlainText`. (Kompilieren, Testen, Einchecken.)

```
function statement (invoice, plays) {
  const statementData = {};
  return renderPlainText(statementData, invoice,
plays);
}

function renderPlainText(data, invoice, plays) {
  let result = `Abrechnung für ${invoice.customer}\n`;
  for (let perf of invoice.performances) {
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
  }
  result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;

function totalAmount() {...}
function totalVolumeCredits() {...}
function usd(aNumber) {...}
function volumeCreditsFor(aPerformance) {...}
function playFor(aPerformance) {...}
function amountFor(aPerformance) {...}
```

Jetzt untersuche ich die übrigen Argumente, die

`renderPlainText` verwendet. Ich möchte die Daten, die sie liefern, in die temporäre Datenstruktur verschieben, sodass sich der gesamte Code für die Berechnung in der `statement`-Funktion befindet und `renderPlainText` ausschließlich die Daten verwendet, die mit dem Parameter `data` übergeben werden.

Zunächst einmal füge ich dem temporären Objekt den Kunden (`customer`) hinzu. (Kompilieren, Testen, Einchecken.)

```
function statement (invoice, plays) {
  const statementData = {};
  statementData.customer = invoice.customer;
  return renderPlainText(statementData, invoice,
plays);
}

function renderPlainText(data, invoice, plays) {
  let result = `Abrechnung für ${data.customer}\n`;
  for (let perf of invoice.performances) {
    result += ` ${playFor(perf).name}:
                              ${usd(amountFor(perf))}
                             (${perf.audience}
Plätze)\n`;
  }
  result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
  result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
  return result;
```

Auf die gleiche Weise füge ich die Vorstellungen

(`performances`) hinzu. Das ermöglicht mir, den an `renderPlainText` übergebenen Parameter `invoice` zu entfernen. (Kompilieren, Testen, Einchecken.)

```
function statement (invoice, plays) {
  const statementData = {};
  statementData.customer = invoice.customer;
  statementData.performances = invoice.performances;
  return renderPlainText(statementData, invoice, plays);
}

function renderPlainText(data, invoice, plays) {
  let result = `Abrechnung für ${data.customer}\n`;
  for (let perf of data.performances) {
    result += ` ${playFor(perf).name}: ${usd(amountFor(perf))} (${perf.audience} Plätze)\n`;
  }
  result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
  result += `Sie erhalten ${totalVolumeCredits()} Treuepunkte\n`;
  return result;
```

Nun möchte ich erreichen, dass auch der Name des Theaterstücks (`play`) von den temporären Daten geliefert wird. Zu diesem Zweck muss ich den Datensatz für die Vorstellung um die Daten des Theaterstücks ergänzen. (Kompilieren, Testen, Einchecken.)

```
function statement (invoice, plays) {
```

```
  const statementData = {};
  statementData.customer = invoice.customer;
  statementData.performances =

invoice.performances.map(enrichPerformance);
  return renderPlainText(statementData, invoice,
plays);
}

function enrichPerformance(aPerformance) {
  const result = Object.assign({}, aPerformance);
  return result;
}
```

Fürs Erste erstelle ich lediglich eine Kopie des Objekts, ich füge dem neuen Datensatz jedoch sogleich Daten hinzu. Ich verwende eine Kopie, weil ich die an die Funktion übergebenen Daten nicht modifizieren möchte. Ich ziehe es vor, Daten möglichst als unveränderlich zu betrachten – ein veränderlicher Zustand kann schnell zu einem furchtbaren Durcheinander führen.

Das Idiom

```
result = Object.assign({}, aPerformance)
```

sieht ziemlich merkwürdig aus, wenn man mit JavaScript nicht vertraut ist. Es erstellt eine »flache« Kopie (*shallow copy*). Ich würde es eigentlich bevorzugen, hierfür eine Funktion zu verwenden, aber die Verwendung dieses Idioms ist in JavaScript so

gebräuchlich, dass es JavaScript-Programmierern unverhältnismäßig erscheinen würde, wenn ich dafür eine eigene Funktion schreiben würde.

Jetzt habe ich einen Platz für das Theaterstück gefunden und kann es hinzufügen. Zu diesem Zweck wende ich *Funktion verschieben* (Abschnitt 8.1) auf `playFor` und `statement` an. (Kompilieren, Testen, Einchecken.)

```
function statement...
function enrichPerformance(aPerformance) {
  const result = Object.assign({}, aPerformance);
  result.play = playFor(aPerformance);
  return result;
}

function playFor(aPerformance) {
  return plays[aPerformance.playID];
}
```

Anschließend ersetze ich in `renderPlainText` alle Verweise auf `playFor`, sodass stattdessen die Daten verwendet werden. (Kompilieren, Testen, Einchecken.)

```
function renderPlainText...
let result = `Abrechnnung für ${data.customer}\n`;
for (let perf of data.performances) {
  result += ` ${perf.play.name}:
${usd(amountFor(perf))}
                                ($${perf.audience}
Plätze)\n`;
}
```

```
result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
return result;

function volumeCreditsFor(aPerformance) {
  let result = 0;
  result += Math.max(aPerformance.audience - 30, 0);
  if ("comedy" === aPerformance.play.type) result +=

Math.floor(aPerformance.audience / 5);
  return result;
}

function amountFor(aPerformance) {
  let result = 0;
  switch (aPerformance.play.type) {
  case "tragedy":
    result = 40000;
    if (aPerformance.audience > 30) {
      result += 1000 * (aPerformance.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (aPerformance.audience > 20) {
      result += 10000 + 500 * (aPerformance.audience -
20);
    }
    result += 300 * aPerformance.audience;
    break;
  default:
      throw new Error(`Unbekannter Typ:
                          ${aPerformance.play.type}`);
  }
  return result;
}
```

Mit `amountFor` verfahre ich auf ähnliche Weise. (Kompilieren, Testen, Einchecken.)

```
function statement...
function enrichPerformance(aPerformance) {
  const result = Object.assign({}, aPerformance);
  result.play = playFor(result);
  result.amount = amountFor(result);
  return result;
}

function amountFor(aPerformance) {...}

function renderPlainText...
let result = `Abrechnung für ${data.customer}\n`;
for (let perf of data.performances) {
  result += ` ${perf.play.name}: ${usd(perf.amount)}
                                ($${perf.audience}
Plätze)\n`;
}
result += `Rechnungsbetrag ${usd(totalAmount())}\n`;
result +=
        `Sie erhalten ${totalVolumeCredits()}
Treuepunkte\n`;
return result;

function totalAmount() {
  let result = 0;
  for (let perf of data.performances) {
    result += perf.amount;
  }
  return result;
}
```

Als Nächstes verschiebe ich die Berechnung der

Treuepunkte. (Kompilieren, Testen, Einchecken.)

```
function statement...
function enrichPerformance(aPerformance) {
  const result = Object.assign({}, aPerformance);
  result.play = playFor(result);
  result.amount = amountFor(result);
  result.volumeCredits = volumeCreditsFor(result);
  return result;
}

function volumeCreditsFor(aPerformance) {...}

function renderPlainText...
function totalVolumeCredits() {
  let result = 0;
  for (let perf of data.performances) {
    result += perf.volumeCredits;
  }
  return result;
}
```

Und schließlich verschiebe ich die beiden Berechnungen der Gesamtsummen.

```
function statement...
const statementData = {};
statementData.customer = invoice.customer;
statementData.performances =

invoice.performances.map(enrichPerformance);
statementData.totalAmount =
totalAmount(statementData);
statementData.totalVolumeCredits =
```

```
totalVolumeCredits(statementData);
return renderPlainText(statementData, plays);

function totalAmount(data) {...
function totalVolumeCredits(data) {...}

function renderPlainText...
let result = `Abrechnung für ${data.customer}\n`;
for (let perf of data.performances) {
  result += ` ${perf.play.name}: ${usd(perf.amount)}
                                ($${perf.audience}
Plätze)\n`;
}
result += `Rechnungsbetrag
${usd(data.totalAmount)}\n`;
result +=
     `Sie erhalten ${data.totalVolumeCredits}
Treuepunkte\n`;
return result;
```

Ich hätte die beiden Funktionen zur Berechnung der Gesamtsummen auch direkt modifizieren und die Variable `statementData` verwenden können (da sie sich im Gültigkeitsbereich befindet), ich ziehe es jedoch vor, sie als expliziten Parameter zu übergeben.

Nachdem ich das Kompilieren, Testen und Einchecken für diese Änderung erledigt habe, kann ich nicht widerstehen, einige kurze Versuche mit *Schleife durch Pipeline ersetzen* (Abschnitt 8.8) anzustellen.

```
function renderPlainText...
function totalAmount(data) {
  return data.performances
```

```
    .reduce((total, p) => total + p.amount, 0);
}
function totalVolumeCredits(data) {
  return data.performances
    .reduce((total, p) => total + p.volumeCredits, 0);
}
```

Nun extrahiere ich den kompletten Code der ersten Phase und mache daraus eine eigene Funktion. (Kompilieren, Testen, Einchecken.)

```
Oberste Ebene...
function statement (invoice, plays) {
  return
renderPlainText(createStatementData(invoice,plays));
}

function createStatementData(invoice, plays) {
  const statementData = {};
  statementData.customer = invoice.customer;
  statementData.performances =

invoice.performances.map(enrichPerformance);
  statementData.totalAmount =
totalAmount(statementData);
  statementData.totalVolumeCredits =

totalVolumeCredits(statementData);
  return statementData;
```

Da der Code jetzt deutlich abgegrenzt ist, verlagere ich ihn in eine eigene Datei (und ändere die Bezeichnung des Rückgabewerts, damit sie meiner üblichen Konvention

entspricht).

```
statement.js...
import createStatementData from
'./createStatementData.js';

createStatementData.js...
export default function createStatementData(invoice,
plays) {
  const result = {};
  result.customer = invoice.customer;
  result.performances =

invoice.performances.map(enrichPerformance);
  result.totalAmount = totalAmount(result);
  result.totalVolumeCredits =
totalVolumeCredits(result);
  return result;

function enrichPerformance(aPerformance) {...}
function playFor(aPerformance) {...}
function amountFor(aPerformance) {...}
function volumeCreditsFor(aPerformance) {...}
function totalAmount(data) {...}
function totalVolumeCredits(data) {...}
```

Eine letzte Runde Kompilieren, Testen und Einchecken – nun ist es ganz einfach, eine HTML-Version zu schreiben. (Die Funktion `usd` habe ich in die oberste Ebene verschoben, damit `renderHTML` sie verwenden kann.)

```
statement.js...
function htmlStatement (invoice, plays) {
```

```
  return renderHtml(createStatementData(invoice,
plays));
}
function renderHtml (data) {
  let result = `<h1>Abrechnung für ${data.customer}
</h1>\n`;
  result += "<table>\n";
  result += "<tr><th>Stück</th><th>Plätze</th><th>
                                          Kosten</th>
</tr>";
  for (let perf of data.performances) {
    result += ` <tr><td>${perf.play.name}</td><td>
                                     ${perf.audience}
</td>`;
    result += `<td>${usd(perf.amount)}</td></tr>\n`; }
    result += "</table>\n";
    result += `<p>Rechnungsbetrag <em>
                      ${usd(data.totalAmount)}</em>
</p>\n`;
    result += `<p>Sie erhalten
<em>${data.totalVolumeCredits}
                                     </em>
Treuepunkte</p>\n`;
    return result;
}

function usd(aNumber) {...}
```

1.7 Status: Aufgeteilt in zwei Dateien (und Phasen)

Jetzt ist wieder der Zeitpunkt für eine Bestandsaufnahme gekommen und dafür, darüber nachzudenken, in welchem Zustand sich der Code befindet. Ich habe zwei Quelltextdateien erstellt.

```
statement.js
import createStatementData from
'./createStatementData.js';
function statement (invoice, plays) {
  return
renderPlainText(createStatementData(invoice,plays));
}
function renderPlainText(data, plays) {
  let result = `Abrechnung für ${data.customer}\n`;
  for (let perf of data.performances) {
    result += ` ${perf.play.name}: ${usd(perf.amount)}
                                ($${perf.audience}
Plätze)\n`;
  }
  result += `Rechnungsbetrag
${usd(data.totalAmount)}\n`;
  result +=
     `Sie erhalten ${data.totalVolumeCredits}
Treuepunkte\n`;
  return result;
}
function htmlStatement (invoice, plays) {
  return renderHtml(createStatementData(invoice,
plays));
}
function renderHtml (data) {
  let result = `<h1>Abrechnung für ${data.customer}
</h1>\n`;
  result += "<table>\n";
  result +=
     "<tr><th>Stück</th><th>Plätze</th><th>Kosten</th>
</tr>";
  for (let perf of data.performances) {
    result += ` <tr><td>${perf.play.name}</td><td>
                                    ${perf.audience}
</td>`;
    result += `<td>${usd(perf.amount)}</td></tr>\n`;
  }
```

```
  result += "</table>\n";
  result += `<p>Rechnungsbetrag <em>
                        ${usd(data.totalAmount)}</em>
</p>\n`;
  result += `<p>Sie erhalten
<em>${data.totalVolumeCredits}
                                    </em>
Treuepunkte</p>\n`;
  return result;
}
function usd(aNumber) {
  return new Intl.NumberFormat("en-US",
           { style: "currency", currency: "USD",
             minimumFractionDigits: 2
}).format(aNumber/100);
}
```

createStatementData.js

```
export default function createStatementData(invoice,
plays) {
  const result = {};
  result.customer = invoice.customer;
  result.performances =

invoice.performances.map(enrichPerformance);
  result.totalAmount = totalAmount(result);
  result.totalVolumeCredits =
totalVolumeCredits(result);
  return result;

function enrichPerformance(aPerformance) {
  const result = Object.assign({}, aPerformance);
  result.play = playFor(result);
  result.amount = amountFor(result);
  result.volumeCredits = volumeCreditsFor(result);
  return result;
}
function playFor(aPerformance) {
  return plays[aPerformance.playID]
```

```
}
function amountFor(aPerformance) {
  let result = 0;
  switch (aPerformance.play.type) {
  case "tragedy":
    result = 40000;
    if (aPerformance.audience > 30) {
      result += 1000 * (aPerformance.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (aPerformance.audience > 20) {
      result += 10000 + 500 * (aPerformance.audience -
20);
    }
    result += 300 * aPerformance.audience;
    break;
  default:
      throw new Error(
               `Unbekannter Typ:
${aPerformance.play.type}`);
  }
  return result;
}
function volumeCreditsFor(aPerformance) {
  let result = 0;
  result += Math.max(aPerformance.audience - 30, 0);
  if ("comedy" === aPerformance.play.type) result +=

Math.floor(aPerformance.audience / 5);
  return result;
}
function totalAmount(data) {
  return data.performances
    .reduce((total, p) => total + p.amount, 0);
}
function totalVolumeCredits(data) {
  return data.performances
```

```
    .reduce((total, p) => total + p.volumeCredits, 0);
}
```

Der Code ist jetzt umfangreicher als zu Beginn: 70 Zeilen (ohne `htmlStatement` und ohne Zeilenumbrüche) gegenüber 44 Zeilen. Das liegt hauptsächlich an dem zusätzlichen Aufwand, der durch das »Verpacken« in Funktionen entsteht. Wenn alles andere gleich bleibt, ist mehr Code schlecht – aber in den seltensten Fällen bleibt alles gleich. Der zusätzliche Code unterteilt die Programmlogik in erkennbare Abschnitte, indem er die Berechnungen von der Ausgabeformatierung trennt. Diese Modularität erleichtert es mir, die Codebestandteile zu verstehen und zu begreifen, wie sie zusammenwirken. In der Kürze liegt bekanntlich die Würze, aber Klarheit und Verständlichkeit ist die Grundlage für erweiterbare Software. Die hergestellte Modularität ermöglicht mir, eine HTML-Version des Codes zu unterstützen, ohne irgendwelche Berechnungen doppelt auszuführen.

Tipp

Folgen Sie beim Programmieren der alten Pfadfinderregel: Hinterlassen Sie die Codebasis immer ordentlicher, als sie vorgefunden wurde.

Es gibt noch weitere Möglichkeiten, die Ausgabelogik zu vereinfachen, aber fürs Erste ist dies ausreichend. Man muss immer einen Mittelweg zwischen all den möglichen Refactorings und dem Hinzufügen neuer Features finden. Die meisten Leute schenken dem Refactoring zu wenig Beachtung – einen gangbaren Mittelweg gibt es aber dennoch. Meine Regel ist eine Variante der alten Pfadfinderregel für Zeltplätze: Hinterlassen Sie die Codebasis immer ordentlicher, als Sie sie vorgefunden haben. Sie wird zwar nie perfekt sein, aber zumindest besser als vorher.

1.8 Neuorganisation der Berechnungen nach Art

Nun widme ich meine Aufmerksamkeit der nächsten Änderung: der Unterstützung zusätzlicher Arten von Theaterstücken, mit eigenen Abrechnungsregeln und Vorschriften zur Berechnung der Treuepunkte. Momentan muss ich die Berechnungsfunktionen bearbeiten und die Regeln dort anpassen, wenn ich Änderungen vornehmen möchte. Die Funktion `amountFor` unterstreicht, welche zentrale Rolle die Art des Theaterstücks dabei für die Wahl der Berechnungsmethode einnimmt. Eine solche Fallunterscheidung hat jedoch die Tendenz, in Folge von Änderungen mehr und mehr zu »verwahrlosen«, insofern sie nicht durch zusätzliche strukturelle Elemente der Programmiersprache davor geschützt wird.

Es gibt verschiedene Möglichkeiten, eine Struktur einzuführen und den Code expliziter zu gestalten. In diesem Fall ist es allerdings naheliegend, Polymorphie anzuwenden – ein wesentliches Prinzip der klassischen Objektorientierung. In der JavaScript-Welt war die klassische Objektorientierung lange umstritten, aber die ECMAScript-2015-Version bietet eine geeignete Syntax und Struktur dafür. Deshalb ist es sinnvoll, sie in Fällen wie diesem einzusetzen.

Mein Plan sieht vor, eine Vererbungshierarchie mit Komödie- und Tragödie-Unterklassen einzurichten, die die Programmlogik der jeweiligen Berechnungen enthalten. Die Clients rufen eine polymorphe Funktion auf, und die Programmiersprache sorgt dafür, dass der für die verschiedenen Berechnungsverfahren von Komödien und Tragödien vorgesehene Code ausgeführt wird. Für die Berechnung der Treuepunkte werde ich eine vergleichbare Struktur einrichten. Zu diesem Zweck werde ich eine Reihe Refactorings verwenden. Das wichtigste ist hier *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4), das eine Reihe von Fallunterscheidungen durch Polymorphie ersetzt. Doch bevor ich es verwenden kann, muss ich eine Vererbungshierarchie einrichten. Ich benötige eine Klasse, die Funktionen zum Berechnen des Rechnungsbetrags und der Treuepunkte bereitstellt.

Zunächst einmal betrachte ich erneut den Code der Berechnungen. (Zu den angenehmen Folgen des

vorangegangenen Refactorings gehört, dass ich mich nicht mehr um den Code zur Formatierung kümmern muss, sofern ich bei der Ausgabe die gleiche Datenstruktur erzeuge. Zur weiteren Unterstützung kann ich Tests zur Überprüfung der temporären Datenstruktur hinzufügen.)

```
createStatementData.js
export default function createStatementData(invoice,
plays) {
  const result = {};
  result.customer = invoice.customer;
  result.performances =

invoice.performances.map(enrichPerformance);
  result.totalAmount = totalAmount(result);
  result.totalVolumeCredits =
totalVolumeCredits(result);
  return result;

function enrichPerformance(aPerformance) {
  const result = Object.assign({}, aPerformance);
  result.play = playFor(result);
  result.amount = amountFor(result);
  result.volumeCredits = volumeCreditsFor(result);
  return result;
}
function playFor(aPerformance) {
  return plays[aPerformance.playID]
}
function amountFor(aPerformance) {
  let result = 0;
  switch (aPerformance.play.type) {
  case "tragedy":
    result = 40000;
    if (aPerformance.audience > 30) {
```

```
      result += 1000 * (aPerformance.audience - 30);
    }
    break;
  case "comedy":
    result = 30000;
    if (aPerformance.audience > 20) {
      result += 10000 + 500 * (aPerformance.audience -
20);
    }
    result += 300 * aPerformance.audience;
    break;
  default:
      throw new Error(
                `Unbekannter Typ:
${aPerformance.play.type}`);
  }
  return result;
}
function volumeCreditsFor(aPerformance) {
  let result = 0;
  result += Math.max(aPerformance.audience - 30, 0);
  if ("comedy" === aPerformance.play.type) result +=

Math.floor(aPerformance.audience / 5);
  return result;
}
function totalAmount(data) {
  return data.performances
    .reduce((total, p) => total + p.amount, 0);
}
function totalVolumeCredits(data) {
  return data.performances
    .reduce((total, p) => total + p.volumeCredits, 0);
}
```

1.8.1 BERECHNEN DER KOSTEN EINER VORSTELLUNG

Die Funktion `enrichPerformance` ist der Schlüssel, denn sie weist der temporären Datenstruktur die Daten der verschiedenen Vorstellungen zu. Aktuell ruft sie die bedingten Funktionen zur Berechnung des Rechnungsbetrags und der Treuepunkte auf. Ich möchte jedoch, dass sie diese Aufrufe an eine Klasse richtet, die ihrerseits Funktionen zur Berechnung der Daten bezüglich der Vorstellungen bereitstellt. Deshalb bezeichne ich sie als `PerformanceCalculator`.

```
function createStatementData...
function enrichPerformance(aPerformance) {
  const calculator = new
PerformanceCalculator(aPerformance);
  const result = Object.assign({}, aPerformance);
  result.play = playFor(result);
  result.amount = amountFor(result);
  result.volumeCredits = volumeCreditsFor(result);
  return result;
}

Oberste Ebene...
class PerformanceCalculator {
  constructor(aPerformance) {
    this.performance = aPerformance;
  }
}
```

Bis jetzt erledigt dieses neue Objekt noch gar keine Aufgabe. Ich möchte Verhalten in dieses Objekt verlagern – und mit dem anfangen, das sich am einfachsten verlagern lässt, nämlich dem Datensatz für

das Theaterstück (`play`). Genau genommen wäre das eigentlich gar nicht nötig, denn er verhält sich nicht polymorph, aber auf diese Weise erfolgt die gesamte Aufbereitung der Daten an einer einzigen Stelle, und diese Konsistenz sorgt für besser verständlichen Code.

Ich verwende dabei *Funktionsdeklaration ändern* (Abschnitt 6.5), um das Theaterstück der Vorstellung an die Berechnungsfunktion zu übergeben.

```
function createStatementData...
function enrichPerformance(aPerformance) {
  const calculator = new
PerformanceCalculator(aPerformance,

playFor(aPerformance));
  const result = Object.assign({}, aPerformance);
  result.play = calculator.play;
  result.amount = amountFor(result);
  result.volumeCredits = volumeCreditsFor(result);
  return result;
}

class PerformanceCalculator...
class PerformanceCalculator {
  constructor(aPerformance, aPlay) {
    this.performance = aPerformance;
    this.play = aPlay;
  }
}
```

(Ich werde ab jetzt weitgehend darauf verzichten, ständig zu erwähnen, dass ich kompiliere, teste und einchecke,

obwohl ich das bei jeder sich bietenden Gelegenheit tue, weil ich annehme, dass Sie es zur Genüge gelesen haben. Gelegentlich werde ich es müde, es auch wirklich auszuführen – und ermögliche es so, dass sich Fehler einschleichen. Das lehrt mich, zu meinem gewohnten Rhythmus zurückzukehren.)

1.8.2 FUNKTIONEN IN DIE KLASSE VERSCHIEBEN

Der nächste Teil der Programmlogik, den ich verschiebe, ist für die Berechnung der Kosten einer Vorstellung von weitaus größerer Bedeutung. Ich habe ganz beiläufig Funktionen verschoben und dabei die verschachtelten Funktionen umgeordnet – aber das hier ist eine tiefgreifendere Änderung am Kontext der Funktion, deshalb werde ich das Refactoring *Funktion verschieben* (Abschnitt 8.1) schrittweise durchführen. Der erste Teil dieses Refactorings ist das Kopieren der Programmlogik in den neuen Kontext – die Klasse `PerformanceCalculator`. Anschließend passe ich den Code an die neue Umgebung an, indem ich `aPerformance` durch `this.performance` und `playFor(aPerformance)` durch `this.play` ersetze.

```
class PerformanceCalculator...
get amount() {
  let result = 0;
  switch (this.play.type) {
    case "tragedy":
    result = 40000;
```

```
    if (this.performance.audience > 30) {
      result += 1000 * (this.performance.audience -
30);
    }
    break;
  case "comedy":
    result = 30000;
    if (this.performance.audience > 20) {
      result += 10000+500 * (this.performance.audience
- 20);
    }
    result += 300 * this.performance.audience;
    break;
  default:
      throw new Error(`Unbekannter Typ:
${this.play.type}`);
  }
  return result;
}
```

Zu diesem Zeitpunkt kann ich kompilieren, um den Code auf Fehler während des Kompilierens zu überprüfen. Das »Kompilieren« findet in meiner Entwicklungsumgebung statt, wenn ich den Code ausführe, denn ich verwende Babel (`https://babeljs.io`). Das genügt, um alle Syntaxfehler in der neuen Funktion zu finden – aber mehr auch nicht. Dieser Schritt kann sich dennoch als nützlich erweisen.

Sobald die neue Funktion an ihr Umfeld angepasst ist, wandle ich die ursprüngliche Funktion so um, dass sie den Aufruf an die neue Funktion delegiert.

```
function createStatementData...
function amountFor(aPerformance) {
  return new PerformanceCalculator(aPerformance,

playFor(aPerformance)).amount;
}
```

Nun kann ich kompilieren, testen und einchecken, um mich zu vergewissern, dass der Code im neuen Umfeld richtig funktioniert. Nachdem das erledigt ist, verwende ich *Funktion inline platzieren* (Abschnitt 6.2), um die neue Funktion direkt aufzurufen.

```
function createStatementData...
function enrichPerformance(aPerformance) {
  const calculator = new
PerformanceCalculator(aPerformance,

playFor(aPerformance));
  const result = Object.assign({}, aPerformance);
  result.play = calculator.play;
  result.amount = calculator.amount;
  result.volumeCredits = volumeCreditsFor(result);
  return result;
}
```

Das gleiche Verfahren wende ich an, um die Berechnung der Treuepunkte zu verschieben.

```
function createStatementData...
function enrichPerformance(aPerformance) {
```

```
  const calculator = new
PerformanceCalculator(aPerformance,

playFor(aPerformance));
  const result = Object.assign({}, aPerformance);
  result.play = calculator.play;
  result.amount = calculator.amount;
  result.volumeCredits = calculator.volumeCredits;
  return result;
}

class PerformanceCalculator...
get volumeCredits() {
  let result = 0;
  result += Math.max(this.performance.audience - 30,
0);
  if ("comedy" === this.play.type) result +=

Math.floor(this.performance.audience / 5);
  return result;
}
```

1.8.3 POLYMORPHES VERHALTEN DER KLASSE PERFORMANCECALCULATOR

Jetzt enthält die Klasse die Programmlogik, und es ist an der Zeit, dass die Polymorphie zur Anwendung kommt. Der erste Schritt ist das Refactoring *Typenschlüssel durch Unterklassen ersetzen* (Abschnitt 12.6), das anstelle eines Typenschlüssels Unterklassen verwendet. Ich muss also Unterklassen der Klasse `PerformanceCalculator` erstellen und in `createPerformanceData` die entsprechenden Unterklassen verwenden. Um an die richtige Unterklasse

zu gelangen, muss ich den Aufruf des Konstruktors durch eine Funktion ersetzen, denn JavaScript-Konstruktoren können keine Unterklassen zurückgeben. Deshalb verwende ich *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8).

```
function createStatementData...
function enrichPerformance(aPerformance) {
  const calculator =

createPerformanceCalculator(aPerformance,

playFor(aPerformance));
  const result = Object.assign({}, aPerformance);
  result.play = calculator.play;
  result.amount = calculator.amount;
  result.volumeCredits = calculator.volumeCredits;
  return result;
}

Oberste Ebene...
function createPerformanceCalculator(aPerformance,
aPlay) {
  return new PerformanceCalculator(aPerformance,
aPlay);
}
```

Da es sich jetzt um eine Funktion handelt, kann ich Unterklassen von `PerformanceCalculator` erstellen und die erzeugende Funktion auswählen lassen, welche zurückgegeben wird.

```
Oberste Ebene...
```

```
function createPerformanceCalculator(aPerformance,
aPlay) {
  switch(aPlay.type) {
    case "tragedy":
           return new TragedyCalculator(aPerformance,
aPlay);
    case "comedy" :
            return new ComedyCalculator(aPerformance,
aPlay);
    default:
        throw new Error(`Unbekannter Typ:
${aPlay.type}`);
  }
}
class TragedyCalculator extends PerformanceCalculator
{
}
class ComedyCalculator extends PerformanceCalculator {
}
```

Damit ist die Struktur für die Polymorphie eingerichtet, und ich kann mit *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) fortfahren.

Zunächst berechne ich den Betrag für eine Tragödie.

```
class TragedyCalculator...
get amount() {
  let result = 40000;
  if (this.performance.audience > 30) {
    result += 1000 * (this.performance.audience - 30);
  }
  return result;
}
```

Das Vorhandensein dieser Methode in der Unterklasse reicht aus, um die Fallunterscheidung in der Basisklasse zu überschreiben. Aber falls Sie so paranoid sind wie ich, könnten Sie Folgendes tun:

```
class PerformanceCalculator...
get amount() {
  let result = 0;
  switch (this.play.type) {
    case "tragedy":
      throw 'Achtung!';
    case "comedy":
      result = 30000;
      if (this.performance.audience > 20) {
        result += 10000 + 500 *
                            (this.performance.audience -
20);
      }
      result += 300 * this.performance.audience;
      break;
    default:
      throw new Error(`Unbekannter Typ:
${this.play.type}`);
  }
  return result;
}
```

Ich hätte auch die `case`-Anweisung für die Tragödie entfernen und im `default`-Zweig einen Ausnahmefehler auslösen können. Aber die explizite Fehlermeldung gefällt mir – und sie wird ohnehin nur noch ein paar Minuten vorhanden sein. (Deshalb habe ich auch statt eines sinnvolleren Objekts einen String für

die Fehlermeldung verwendet.)

Nach dem Kompilieren, Testen und Einchecken verfahre ich für den Fall, dass es sich um eine Komödie handelt, auf die gleiche Weise.

```
class ComedyCalculator...
get amount() {
  let result = 30000;
  if (this.performance.audience > 20) {
    result += 10000 + 500 * (this.performance.audience
- 20);
  }
  result += 300 * this.performance.audience;
  return result;
}
```

Jetzt kann ich die Methode `amount` aus der Basisklasse entfernen, denn sie sollte eigentlich niemals aufgerufen werden. Um nett zu meinem zukünftigen Ich zu sein, lasse ich jedoch ein Andenken zurück.

```
class PerformanceCalculator...
get amount() {
throw new Error('subclass responsibility');
}
```

Die nächste zu entfernende Fallunterscheidung ist die Berechnung der Treuepunkte. Bei der Diskussion um zukünftige Arten von Theaterstücken stellte sich heraus,

dass meistens überprüft werden soll, ob es mehr als 30 Besucher gibt, und nur in einigen wenigen Fällen mit Abweichungen davon zu rechnen ist. Deshalb ist es sinnvoll, den häufigeren Fall als Standard in der Basisklasse zu belassen und diese bei Abweichungen nach Bedarf zu überschreiben. Im Falle von Komödien sieht das folgendermaßen aus:

```
class PerformanceCalculator...
get volumeCredits() {
  return Math.max(this.performance.audience - 30, 0);
}

class ComedyCalculator...
  get volumeCredits() {
  return super.volumeCredits +

Math.floor(this.performance.audience / 5);
}
```

1.9 Status: Erzeugen der Daten mit der polymorphen Klasse PerformanceCalculator

Es ist an der Zeit, darüber nachzudenken, welche Auswirkungen eine polymorphe Berechnungsklasse auf den Code hat.

```
createStatementData.js
export default function createStatementData(invoice,
plays) {
```

```
  const result = {};
  result.customer = invoice.customer;
  result.performances =

invoice.performances.map(enrichPerformance);
  result.totalAmount = totalAmount(result);
  result.totalVolumeCredits =
totalVolumeCredits(result);
  return result;

  function enrichPerformance(aPerformance) {
  const calculator =

createPerformanceCalculator(aPerformance,

playFor(aPerformance));
  const result = Object.assign({}, aPerformance);
  result.play = calculator.play;
  result.amount = calculator.amount;
  result.volumeCredits = calculator.volumeCredits;
  return result;
  }
  function playFor(aPerformance) {
    return plays[aPerformance.playID]
  }
  function totalAmount(data) {
    return data.performances
      .reduce((total, p) => total + p.amount, 0);
  }
  function totalVolumeCredits(data) {
    return data.performances
      .reduce((total, p) => total + p.volumeCredits,
0);
  }
}
function createPerformanceCalculator(aPerformance,
aPlay) {
switch(aPlay.type) {
  case "tragedy": return new
```

```
TragedyCalculator(aPerformance,
                                                  aPlay);
  case "comedy" : return new
ComedyCalculator(aPerformance,
                                                 aPlay);
  default:
      throw new Error(`Unbekannnter Typ:
${aPlay.type}`);
  }
}
class PerformanceCalculator {
  constructor(aPerformance, aPlay) {
    this.performance = aPerformance;
    this.play = aPlay;
  }
  get amount() {
    throw new Error('subclass responsibility');
  }
  get volumeCredits() {
    return Math.max(this.performance.audience - 30,
0);
  }
}
class TragedyCalculator extends PerformanceCalculator
{
  get amount() {
    let result = 40000;
    if (this.performance.audience > 30) {
      result += 1000 * (this.performance.audience -
30);
    }
    return result;
  }
}
class ComedyCalculator extends PerformanceCalculator {
  get amount() {
    let result = 30000;
    if (this.performance.audience > 20) {
      result += 10000+500 * (this.performance.audience
```

```
- 20);
    }
    result += 300 * this.performance.audience;
    return result;
  }
  get volumeCredits() {
    return super.volumeCredits +
                  Math.floor(this.performance.audience
/ 5);
  }
}
```

Der Umfang des Codes hat durch das Einführen von zusätzlicher Struktur erneut zugenommen. Es ist jedoch von Vorteil, dass die Berechnungen für die verschiedenen Arten von Theaterstücken jeweils zusammengefasst sind. Insofern die meisten Änderungen diesen Code betreffen, ist es hilfreich, wenn er auf diese Weise angeordnet ist. Um eine neue Art von Theaterstück hinzuzufügen, muss eine neue Unterklasse erstellt und der erzeugenden Funktion hinzugefügt werden.

Dieses Beispiel macht deutlich, unter welchen Umständen es sich als nützlich erweist, Unterklassen zu verwenden. Ich habe hier die beiden Fallunterscheidungen aus zwei Funktionen (`amountFor` und `volumeCreditsFor`) in eine einzige Erzeugungsfunktion `createPerformanceCalculator` verschoben. Je mehr Funktionen vorhanden sind, die auf derselben

Form von Polymorphie basieren, desto nützlicher wird dieser Ansatz.

Eine Alternative zu dem von mir hier verwendeten Verfahren wäre eine Funktion `createPerformanceData`, die den `PerformanceCalaculator` selbst zurückgibt, anstatt der temporären Datenstruktur Werte zuzuweisen. JavaScripts Klassensystem hat die angenehme Eigenschaft, dass die Verwendung von Getter-Funktionen wie ein regulärer Datenzugriff aussieht. Meine Entscheidung, ob ich die Instanz zurückgebe oder separate Rückgabedaten berechne, hängt davon ab, wie die Datenstruktur anschließend verwendet wird. In diesem Fall habe ich es vorgezogen, zu zeigen, wie man die temporäre Datenstruktur einsetzt, um zu verbergen, dass eine polymorphe Klasse verwendet wird.

1.10 Abschließende Überlegungen

Hierbei handelt es sich um ein einfaches Beispiel, dennoch hoffe ich, dass Sie ein Gespür dafür entwickelt haben, wie das Refactoring abläuft. Ich habe eine Reihe Refactorings verwendet, wie etwa *Funktion extrahieren* (Abschnitt 6.1), *Variable inline platzieren* (Abschnitt 6.4), *Funktion verschieben* (Abschnitt 8.1) und *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4).

Bei diesem Refactoring gab es drei Stufen: Die Zerlegung der ursprünglichen Funktion in mehrere verschachtelte Funktionen, die Unterteilung des Codes in Berechnungen und Ausgabe durch *Phase aufteilen* (Abschnitt 6.11) und schließlich der Einsatz einer polymorphen Klasse für die Programmlogik der Berechnungen. Jede Stufe hat dem Code Struktur hinzugefügt, die es mir ermöglicht, besser zu vermitteln, welche Aufgabe er erfüllt.

Beim Refactoring kommt es häufig vor, dass die ersten Stufen hauptsächlich dadurch geprägt sind, zu verstehen, was eigentlich ausgeführt wird. Meistens läuft es folgendermaßen ab: Sie lesen den Code, gewinnen einige Erkenntnisse und führen ein Refactoring durch, um die gewonnenen Erkenntnisse aus Ihrem Kopf zurück in den Code zu übertragen. Der klarere Code erleichtert es anschließend, ihn besser zu verstehen, was zu weitergehenden Erkenntnissen und einer positiven Feedback-Schleife führt. Es gibt noch immer Verbesserungen, die ich vornehmen könnte, aber ich habe den Eindruck, dass ich genug getan habe, um den Test zu bestehen, den Code in einem deutlich besseren Zustand zu hinterlassen als dem, in dem ich ihn vorgefunden habe.

Tipp

> Der wahre Test für guten Code ist, wie leicht er sich ändern lässt.

Ich spreche über das Verbessern des Codes – Programmierer hingegen lieben es, darüber zu diskutieren, wie guter Code aussieht. Mir ist klar, dass einige Leute Einspruch gegen meine Vorliebe für kurze, aussagekräftige Funktionsnamen erheben. Wenn wir das Ganze als eine Frage der Ästhetik betrachten, also nichts an sich gut oder schlecht ist, sondern es erst durch die Sichtweise wird, haben wir – außer dem persönlichen Geschmack – keinen Leitfaden mehr, an den wir uns halten können. Ich glaube allerdings, dass wir Geschmacksfragen hinter uns lassen und feststellen sollten, dass der wahre Test für guten Code ist, wie leicht er sich ändern lässt. Code sollte eindeutig sein: Wenn jemand eine Änderung vornehmen muss, sollte der zu ändernde Code schnell und einfach zu finden und zu ändern sein, ohne dass dies irgendwelche Fehler nach sich zieht. Eine gute Codebasis maximiert unsere Produktivität und ermöglicht uns so, zusätzliche Features für unsere Anwender sowohl schneller als auch kostengünstiger zu entwickeln. Achten Sie darauf, welche Hindernisse sich zwischen dem Programmierteam und diesem Ideal aufbauen, um den Code gut in Form zu halten. Führen Sie dann ein Refactoring durch, um diesem Ideal näher zu kommen.

Das Wichtigste, was man aus diesem Beispiel lernen kann, ist der »Rhythmus« des Refactorings. Immer wenn ich Interessierten zeige, wie ich beim Refactoring vorgehe, sind sie überrascht, wie klein die einzelnen Schritte sind, wobei jeder Schritt den Code in einem funktionierenden Zustand belässt, in dem er kompiliert werden kann und seine Tests besteht. Ich selbst war ebenso überrascht, als mir Kent Beck vor zwei Jahrzehnten in einem Hotelzimmer in Detroit zeigte, wie man dabei vorgeht. Für ein effektives Refactoring ist von entscheidender Bedeutung, zu erkennen, dass man mit kleinen Schritten schneller vorankommt. Der Code ist stets funktionsfähig, und Sie können problemlos mehrere kleine Schritte zu umfangreichen Änderungen zusammenfassen. Das sollten Sie sich merken – der Rest ist Schweigen.

Kapitel 2

Prinzipien des Refactorings

Das Beispiel aus dem letzten Kapitel sollte Ihnen einen angemessenen Eindruck davon vermittelt haben, wie das Refactoring abläuft. Nachdem Sie diesen Eindruck gewonnen haben, ist jetzt der richtige Zeitpunkt gekommen, um einige allgemeinere Prinzipien des Refactorings zu erläutern.

2.1 Refactoring: Definition

In der Softwareentwicklung werden in der Praxis viele Begriffe häufig sehr zwanglos verwendet. Das gilt auch für »Refactoring«. Ich lege allerdings großen Wert auf eine präzisere Verwendung des Begriffs und halte es für sinnvoll, diese präzisere Form zu benutzen. (Die nachfolgenden Definitionen sind die gleichen wie in der ersten Ausgabe dieses Buchs.) Ich unterscheide zwischen dem Substantiv »Refactoring« und der Tätigkeit.

Refactoring (als Substantiv): Eine Änderung an der internen Struktur von Software, um sie verständlicher zu machen und Modifikationen zu erleichtern, ohne dass sich das sichtbare Verhalten verändert.

Diese Definition entspricht derjenigen der benannten Refactorings, die ich in den vorangegangenen Beispielen erwähnt habe, wie etwa *Funktion extrahieren* (Abschnitt 6.1) oder *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4).

Refactoring (als Tätigkeit): Software durch die Anwendung einer Reihe von Refactorings umzustrukturieren, ohne dass sich das sichtbare Verhalten verändert.

Ich könnte mich also einige Stunden mit der Tätigkeit Refactoring befassen und währenddessen ein paar Dutzend verschiedene Refactorings anwenden.

Im Laufe der Jahre sind viele in der Branche dazu übergegangen, jegliche Art des Aufräumens von Code als »Refactoring« zu bezeichnen – die obige Definition kennzeichnet jedoch einen bestimmten Ansatz beim Aufräumen des Codes. Beim Refactoring geht es darum, kleine verhaltensbewahrende Schritte durchzuführen und größere Veränderungen durch die Aneinanderreihung dieser verhaltensbewahrenden Schritte vorzunehmen. Die einzelnen Refactorings sind entweder selbst ein kleiner Schritt oder eine Kombination kleiner Schritte. Als Resultat ist der Code während des Refactorings nur selten in einem nicht funktionsfähigen Zustand, was es mir erlaubt, jederzeit aufzuhören, selbst wenn ich noch gar nicht fertig bin.

Tipp

Wenn Ihnen jemand erzählt, dass sein Code während des Refactorings einige Tage nicht funktionsfähig war, können Sie sich ziemlich sicher sein, dass es sich nicht um Refactoring im eigentlichen Sinne gehandelt hat.

Ich verwende »Umstrukturieren« (»restructuring«) als allgemeinen Begriff, der jegliche Art der Neuorganisation oder des Aufräumens einer Codebasis bezeichnet, und betrachte Refactoring als eine ganz bestimmte Art der Umstrukturierung. Manchen Leuten mag das Refactoring ineffizient erscheinen, wenn sie ihm zum ersten Mal begegnen und beobachten, wie ich eine Vielzahl winziger Schritte ausführe, obwohl ein einzelner größerer Schritt ausreichen würde. Die winzigen Schritte ermöglichen mir jedoch ein schnelleres Vorankommen, weil sie sich so gut miteinander kombinieren lassen – und, sehr wichtig, weil ich keine Zeit mit dem Debugging verbringe.

In meinen Definitionen verwende ich den Ausdruck »sichtbares Verhalten«. Dieser Begriff ist bewusst weit gefasst und weist darauf hin, dass der Code nach dem Refactoring im Großen und Ganzen die gleichen Aufgaben erfüllt wie vorher. Das heißt aber nicht, dass er auf exakt die gleiche Weise funktioniert. *Funktion extrahieren* (Abschnitt 6.1) beispielsweise modifiziert

den Call-Stack (Aufrufstapel), was sich auf die Performance auswirken könnte – es sollte sich jedoch nichts ändern, um das sich der Anwender kümmern müsste. Durch Refactorings wie *Funktionsdeklaration ändern* (Abschnitt 6.5) oder *Funktion verschieben* (Abschnitt 8.1) ändern sich insbesondere die Schnittstellen zu Modulen. Wenn ich beim Refactoring Bugs bemerke, sollten diese nach dem Refactoring immer noch vorhanden sein (latente Bugs, die bislang noch niemand bemerkt hat, kann ich allerdings beheben).

Refactoring hat große Ähnlichkeit mit der Performance-Optimierung, denn beide Verfahren modifizieren den Code, ohne die Funktionalität des Programms zu verändern. Der Unterschied besteht in dem Ziel: Ein Refactoring wird durchgeführt, um den Code »verständlicher zu machen und Modifikationen zu erleichtern«. Das Programm kann dadurch schneller oder langsamer werden. Bei der Performance-Optimierung bin ich nur daran interessiert, das Programm zu beschleunigen, und muss damit rechnen, dass der Code letztendlich unhandlicher wird, wenn die Leistungssteigerung wirklich unverzichtbar ist.

2.2 Die beiden Hüte

Die Metapher der beiden Hüte hat sich Kent Beck ausgedacht. Wenn ich bei der Softwareentwicklung ein

Refactoring durchführe, widme ich meine Zeit zwei verschiedenen Aktivitäten: dem Hinzufügen von Funktionalität (das ist der erste Hut, den ich trage) und dem Refactoring (das ist der zweite Hut). Beim Erweitern der Funktionalität sollte ich den vorhandenen Code nicht verändern – ich füge lediglich neue Fähigkeiten hinzu. Meinen Fortschritt messe ich dabei, indem ich Tests hinzufüge und sie zum Laufen bringe. Beim Refactoring habe ich es mir zu eigen gemacht, keine Funktionalität hinzuzufügen – ich strukturiere den Code lediglich um. Ich füge auch keine Tests hinzu (es sei denn, ich habe vorher einen übersehen). Ich ändere die Tests nur dann, wenn sie wegen Änderungen an einer Schnittstelle angepasst werden müssen.

Ich habe festgestellt, dass ich bei der Entwicklung von Software häufig den Hut wechsle. Ich versuche zunächst, eine neue Fähigkeit hinzuzufügen, und stelle dann fest, dass es viel einfacher wäre, wenn der Code anders strukturiert wäre. Ich wechsle also den Hut und befasse mich eine Weile mit dem Refactoring. Sobald der Code besser strukturiert ist, wechsle ich erneut den Hut und füge die neue Fähigkeit hinzu. Nachdem ich die neue Funktionalität zum Laufen gebracht habe, wird mir klar, dass ich sie auf kaum verständliche Weise programmiert habe, also wechsle ich wieder den Hut und führe ein Refactoring durch. Das Ganze dauert vielleicht nur zehn Minuten, aber während dieser Zeitspanne ist mir immer bewusst, welchen Hut ich gerade trage – und welchen

feinen Unterschied das für meine Art und Weise zu programmieren macht.

2.3 Gründe für ein Refactoring

Ich möchte nicht behaupten, dass Refactoring ein Allheilmittel für alle Softwareprobleme ist. Es ist keine Wunderwaffe und keine Patentlösung, aber doch ein nützliches Werkzeug – eine stabile Zange, die Ihnen dabei hilft, Ihren Code in den Griff zu bekommen. Refactoring ist ein Werkzeug, das für verschiedene Zwecke eingesetzt werden kann – und auch sollte.

2.3.1 REFACTORING VERBESSERT DAS DESIGN DER SOFTWARE

Ohne Refactoring droht das interne Design von Software – die Architektur – zu zerfallen. Durch Änderungen des Codes, die vorgenommen werden, um kurzfristige Ziele zu erreichen, oftmals ohne vollständiges Verständnis der Architektur, verliert der Code seine Struktur. Es wird schwieriger, beim Lesen des Codes das Design zu erkennen. Der Verlust der Struktur des Codes hat einen kumulativen Effekt: Je schwieriger es ist, das Design des Codes zu erkennen, desto schwerer fällt es mir, es zu bewahren, und umso schneller zerfällt es. Regelmäßiges Refactoring kann dabei helfen, den Code in Form zu halten.

Bei schlechtem Design ist für gewöhnlich mehr Code

erforderlich, um die gleichen Aufgaben zu erledigen. Das liegt häufig daran, dass Code an verschiedenen Stellen buchstäblich dieselben Dinge tut. Das Entfernen von redundantem Code ist deshalb ein wichtiger Aspekt bei der Verbesserung des Designs. Das bedeutet nicht, dass weniger Code ein System in irgendeiner Form beschleunigt – der Einfluss auf den Speicherbedarf des Programms ist nur selten spürbar. Ein verringerter Umfang des Codes ist bei Änderungen allerdings von großer Bedeutung. Je mehr Code vorhanden ist, desto schwieriger ist es, ihn korrekt zu modifizieren, weil ich einfach mehr Code verstehen muss. Ich ändere den Code an einer Stelle, aber das System verhält sich nicht so, wie ich es erwarte, weil ich den Code an einer anderen Stelle, der praktisch die gleiche Aufgabe in einem anderen Kontext erledigt, nicht ebenfalls geändert habe. Durch das Entfernen von redundantem Code kann ich gewährleisten, dass im Code alles wirklich nur einmal ausformuliert ist – und das ist das Wesentliche eines guten Designs.

2.3.2 REFACTORING MACHT SOFTWARE BESSER VERSTÄNDLICH

Programmieren ist in vielerlei Hinsicht eine Art Unterhaltung mit einem Computer. Ich schreibe Code, der den Computer anweist, welche Aufgaben er erledigen soll, und er reagiert darauf, indem er genau das tut, was ich ihm sage. Mit der Zeit schließe ich die Lücke zwischen dem, was ich erreichen möchte, und dem, was

meine Anweisungen tatsächlich bewirken. Beim Programmieren geht es darum, exakt zu formulieren, was Sie eigentlich wollen. Wahrscheinlich gibt es jedoch noch andere, die meinen Quellcode verwenden. In ein paar Monaten wird ein anderer Mensch meinen Code lesen, um einige Änderungen vorzunehmen. Diese Person vergessen wir häufig. Tatsächlich ist sie aber die wichtigste. Wen interessiert es schon, ob der Computer ein paar zusätzliche Taktzyklen benötigt, um etwas zu kompilieren? Es spielt jedoch durchaus eine Rolle, ob ein Programmierer eine Woche benötigt, um eine Änderung vorzunehmen, die mit dem richtigen Verständnis meines Codes in einer halben Stunde erledigt gewesen wäre.

Hier zeigt sich die folgende Problematik: Wenn ich versuche, ein Programm zum Laufen zu bekommen, mache ich mir keine Gedanken über diesen zukünftigen Entwickler. Um den Code besser verständlich zu machen, ist ein Wechsel des Arbeitsrhythmus erforderlich. Refactoring hilft mir dabei, meinen Code besser verständlich zu machen. Vor dem Refactoring funktioniert mein Code zwar, er ist jedoch nicht bestmöglich strukturiert. Ein geringfügiger Zeitaufwand für das Refactoring kann dafür sorgen, dass dem Code viel besser anzusehen ist, welchem Zweck er dient – also deutlicher vermittelt, was ich eigentlich erreichen möchte.

Ich bin hier zugegebenermaßen nicht ganz uneigennützig, denn in vielen Fällen bin ich selbst dieser

zukünftige Entwickler. Das Refactoring ist deswegen sogar von noch größerer Bedeutung. Ich bin ein äußerst fauler Programmierer. Das ist beispielsweise daran erkennbar, dass ich mir niemals irgendetwas über den von mir geschriebenen Code merke. Tatsächlich versuche ich ganz bewusst, mir nichts zu merken, was ich nachsehen kann, weil ich befürchte, dass mein Gehirn überläuft. Ich lege großen Wert darauf, zu versuchen, alles Erinnerungswerte im Code unterzubringen, damit ich mir nichts merken muss. Auf diese Weise muss ich mir weniger Sorgen machen, dass meine Gehirnzellen Maudite[1] zum Opfer fallen.

2.3.3 REFACTORING HILFT MIR, BUGS AUFZUSPÜREN

Hilfe beim Verständnis des Codes bedeutet auch Hilfe beim Aufspüren von Bugs. Ich muss zugeben, dass ich im Finden von Bugs nicht besonders gut bin. Manche Leute lesen einen Codeabschnitt und erkennen dabei sofort die Fehler; ich kann das nicht. Ich habe allerdings festgestellt, dass ich mich beim Refactoring bemühe, die Funktionsweise des Codes detailliert zu verstehen. Dieses Verständnis lasse ich anschließend wieder in den Code einfließen. Durch die deutlichere Struktur des Programms sind somit auch die von mir getroffenen Annahmen klar ersichtlich. Das geht so weit, dass sogar ich nicht darum herumkomme, die Bugs zu erkennen.

Das erinnert mich an eine Aussage von Kent Beck über

sich selbst: »Ich bin kein großartiger Programmierer; ich bin nur ein guter Programmierer mit großartigen Gewohnheiten.« Das Refactoring hilft mir, beim Schreiben von robustem Code sehr viel effektiver zu sein.

2.3.4 REFACTORING HILFT MIR, SCHNELLER ZU PROGRAMMIEREN

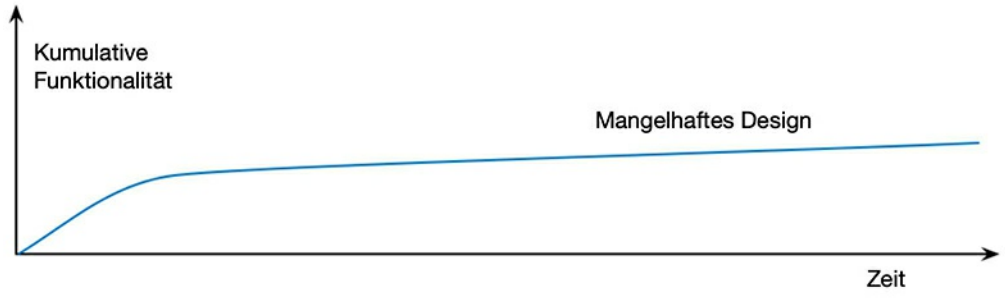
Letzen Endes lassen sich alle schon genannten Punkte folgendermaßen zusammenfassen: Refactoring hilft mir, Code schneller zu entwickeln.

Das klingt zunächst einmal nicht sehr einleuchtend. Dass Refactoring die Qualität des Codes verbessert, ist leicht ersichtlich. Besseres internes Design, Lesbarkeit, weniger Bugs – all das verbessert die Qualität. Aber verringert die Zeit, die ich mit dem Refactoring verbringe, nicht das Tempo der Entwicklung?

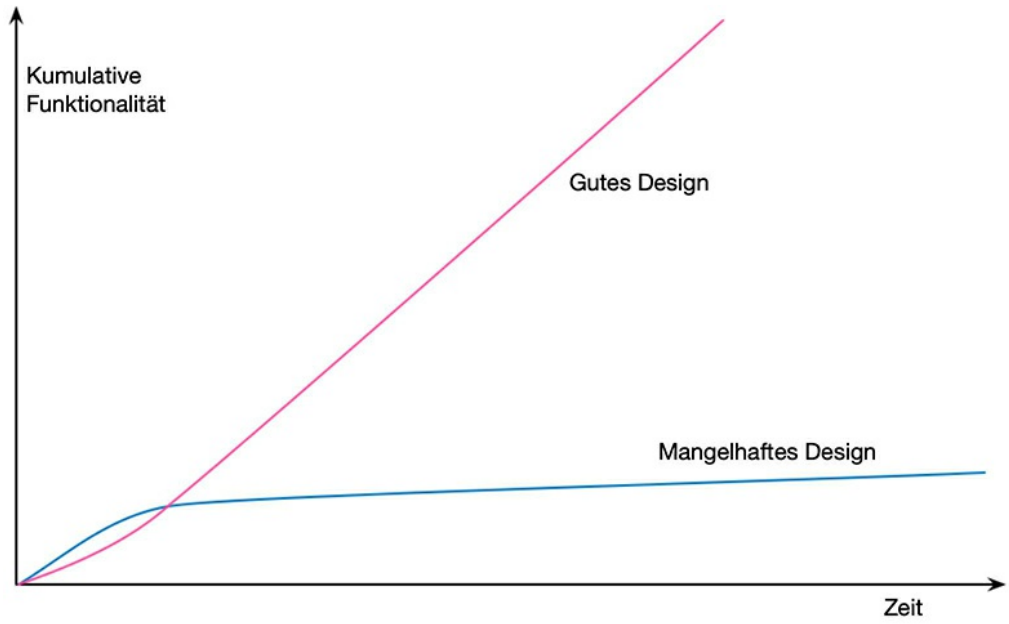
Wenn ich mich mit Softwareentwicklern unterhalte, die seit einer Weile an einem System gearbeitet haben, bekomme ich oft zu hören, dass sie anfangs schnell Fortschritte erzielt haben, aber jetzt viel mehr Zeit benötigen, um neue Features hinzuzufügen. Bei einem neuen Feature dauert es immer länger, zu verstehen, wie es in die bestehende Codebasis intergiert werden kann. Wenn es dann integriert ist, tauchen oft Bugs auf, die zu beheben sogar noch länger dauert. Die Codebasis sieht allmählich wie ein Flickenteppich aus, und es kommt einer archäologischen Expedition gleich, herausfinden zu

wollen, wie das Ganze funktioniert. Diese Last verlangsamt das Hinzufügen neuer Features so sehr, dass sich die Entwickler irgendwann wünschen, sie könnten reinen Tisch machen und noch einmal ganz von vorn anfangen.

Das folgende Diagramm visualisiert diesen Stand der Dinge:



Einige Teams haben jedoch andere Erfahrungen gemacht. Sie haben festgestellt, dass sie neue Features *schneller* hinzufügen können, weil sie sich Bestehendes zunutze machen und auf Vorhandenem aufbauen können.



Der Unterschied zwischen gutem und schlechtem Design besteht in der internen Softwarequalität. Software mit gutem internem Design ermöglicht mir, schnell und einfach herauszufinden, wie und wo ich Änderungen vornehmen muss, um ein neues Feature hinzuzufügen. Dank guter Modularität muss ich nur eine kleine Teilmenge der Codebasis verstehen, um Änderungen vorzunehmen. Wenn der Code verständlich ist, sinkt die Wahrscheinlichkeit, dass ich Fehler mache und Bugs verursache. Sollte es mir dennoch passieren, ist der Aufwand für das Debugging viel geringer. Wenn ich alles richtig mache, wird meine Codebasis zu einer Plattform für die Entwicklung neuer Features.

Ich bezeichne diesen Effekt als *Design-Stamina-Hypothesis* (etwa: Hypothese der Belastbarkeit von Design, `https://martinfowler.com/bliki/DesignStaminaHypothes`

`is.html`): Wenn wir einen gewissen Aufwand in ein gutes internes Design investieren, erhöhen wir die »Belastbarkeit« der Software, was uns erlaubt, länger schneller voranzukommen. Ich kann nicht beweisen, dass es sich tatsächlich so verhält, deshalb spreche ich von einer Hypothese. Sie erklärt jedenfalls die von mir gemachten Erfahrungen, zusammen mit hunderten von großartigen Programmierern, die ich während meiner Karriere kennengelernt habe.

Vor zwanzig Jahren war man der allgemeinen Ansicht, dass es zum Erreichen eines guten Designs notwendig ist, dieses abzuschließen, bevor die eigentliche Programmierung erfolgt – denn sobald wir den Code einmal geschrieben haben, können wir ihm nur noch beim Zerfall zusehen. Diese Ansicht hat sich durch Refactoring geändert. Wir wissen jetzt, dass wir das Design vorhandenen Codes verbessern können. Wir können das Design im Laufe der Zeit formen und verbessern, selbst wenn sich die Anforderungen eines Programms ändern. Da es äußerst schwierig ist, im Vorhinein ein gutes Design zu entwickeln, ist das Refactoring unverzichtbar geworden, um die positive Dynamik zu erreichen, die für schnelle Änderungen der Funktionalität nötig ist.

2.4 Der richtige Zeitpunkt für ein Refactoring

Immer wenn ich programmiere, führe ich praktisch stündlich ein Refactoring durch. Dabei habe ich einige Möglichkeiten entdeckt, wie ich es an meinen Arbeitsablauf anpassen kann.

Die Dreierregel

Die folgende Richtlinie hat mir Don Roberts mit auf den Weg gegeben: Wenn Sie irgendeine Aufgabe zum ersten Mal erledigen, dann tun Sie es einfach. Wenn Sie das zweite Mal eine sehr ähnliche Aufgabe erledigen, zucken Sie in Anbetracht des doppelten Codes zwar zusammen, belassen es jedoch trotzdem bei dem redundanten Code. Wenn Sie das dritte Mal eine ähnliche Aufgabe erledigen, führen Sie ein Refactoring durch.

Oder wenn Sie Fußball mögen: Führen Sie nach dem dritten Eigentor ein Refactoring durch.

2.4.1 VORBEREITENDES REFACTORING: HINZUFÜGEN EINES FEATURES VEREINFACHEN

Wenn ich vorhabe, der Codebasis ein neues Feature hinzuzufügen, ist das der beste Zeitpunkt, um ein Refactoring durchzuführen. Ich betrachte den vorhandenen Code, und häufig fällt mir auf, dass es

meine Arbeit sehr erleichtern würde, wenn er etwas anders strukturiert wäre. Vielleicht gibt es eine Funktion, die fast alles kann, was ich brauche, aber einige Literale enthält, die im Konflikt mit meinen Anforderungen stehen. Ohne Refactoring könnte ich die Funktion kopieren und die entsprechenden Werte ändern. Allerdings führt das zu redundantem Code – wenn ich in Zukunft Änderungen vornehmen möchte, muss ich beide Stellen anpassen (und noch schlimmer: erst einmal wiederfinden), zumal mir simples Kopieren auch nicht weiterhilft, wenn ich demnächst eine ähnliche Variante für ein neues Feature erstellen muss. Ich trage gerade den Refactoring-Hut und verwende dementsprechend *Funktion parametrisieren* (Abschnitt 11.2). Wenn das erledigt ist, muss ich die Funktion nur noch mit den von mir benötigten Parametern aufrufen.

»Es ist etwa so, also ob ich 100 Meilen nach Osten fahren möchte, aber anstatt einfach die direkte Route durch den Wald zu nehmen, fahre ich 20 Meilen nach Norden zur Autobahn und reise dann mit der dreifachen Geschwindigkeit derer, die ich auf direktem Weg erreicht hätte, 100 Meilen nach Osten. Wenn Sie dazu gedrängt werden, den direkten Weg zu nehmen, sollten Sie manchmal sagen: ›Moment, ich muss erst einmal die Karte überprüfen, um den schnellsten Weg zu finden.‹ Das erledigt das vorbereitende Refactoring für mich.«

–Jessica Kerr

Beim Beheben von Bugs verhält es sich genauso. Nachdem ich die Ursache gefunden habe, stelle ich fest, dass sich der Fehler viel einfacher beheben ließe, wenn ich die drei Stellen redundanten Codes, die für den Fehler verantwortlich sind, an einer Stelle vereine. Es könnte auch sein, dass ich eine Logik zur Aktualisierung von der Abfragelogik trennen muss, um Verwicklungen zu vermeiden, die den Fehler verursachen. Wenn ich zur Verbesserung des Codes ein Refactoring durchführe, erhöht sich außerdem die Wahrscheinlichkeit, dass der Bug dauerhaft behoben ist, während die Wahrscheinlichkeit, dass in den Abgründen des betreffenden Codes weitere Fehler auftauchen, sinkt.

2.4.2 REFACTORING ZWECKS VERBESSERUNG DER VERSTÄNDLICHKEIT DES CODES

Um Teile des Codes ändern zu können, muss ich zunächst einmal verstehen, welche Aufgabe er erledigt. Der Code könnte von mir selbst oder von jemand anderem geschrieben worden sein. Wenn ich nachdenken muss, um zu verstehen, welche Aufgabe der Code erledigt, frage ich mich stets, ob ich ein Refactoring durchführen könnte, um diese Erkenntnis offensichtlicher zu machen. Vielleicht betrachte ich eine Fallunterscheidung, die ungeschickt strukturiert ist. Womöglich hätte ich bereits vorhandene Funktionen verwenden wollen, muss jedoch mehrere Minuten damit verbringen, ihre Funktionsweise zu verstehen, weil sie

nicht aussagekräftig benannt worden sind.

Zu diesem Zeitpunkt habe ich eine gewisse Erkenntnis in meinem Kopf, der allerdings alles andere als ein geeigneter Speicherort für solche Details ist. Ward Cunningham würde sagen, dass ich durch das Refactoring diese Erkenntnis von meinem Kopf zurück in den Code selbst übertrage. Dann teste ich diese Erkenntnis, indem ich die Software ausführe und prüfe, ob sie noch funktioniert. Wenn ich die so gewonnene Erkenntnis in den Code übertrage, bleibt sie erhalten und ist auch für meine Kollegen sichtbar.

Das wird sich nicht nur zukünftig als nützlich erweisen – in vielen Fällen hilft es mir unmittelbar. Ich nehme schon frühzeitig ein Refactoring kleiner Details zur Verbesserung der Verständlichkeit des Codes vor. Ich benenne einige Variablen um, weil ich jetzt ihre Bedeutung kenne, oder ich zerlege eine umfangreiche Funktion in mehrere kleinere Teile. Wenn der Code dann allmählich aussagekräftiger wird, kann ich die Designaspekte erkennen, die vorher nicht ersichtlich waren. Wenn ich den Code nicht geändert hätte, wären mir diese Dinge vermutlich verborgen geblieben, weil ich einfach nicht clever genug bin, mir all diese Änderungen in meinem Kopf auszumalen. Ralph Johnson beschreibt diese frühzeitigen Refactorings als das Reinigen eines verschmutzten Fensters, das anschließend klare Sicht gewährt. Beim Studieren des Codes erreiche ich dank des Refactorings ein Verständnisniveau, das mir andernfalls

verwehrt bliebe. Wer das Refactoring zwecks Verbesserung der Verständlichkeit als nutzloses Herumdoktern am Code abtut, nimmt nicht zur Kenntnis, dass ihm durch den Verzicht darauf die hinter all dem Durcheinander verborgenen Chancen verborgen bleiben.

2.4.3 REFACTORING ALS ABFALLENTSORGUNG

Eine Variante des Refactorings zwecks Verbesserung der Verständlichkeit liegt vor, wenn ich die Funktionsweise des Codes zwar verstehe, aber feststellen muss, dass sie mangelhaft ist. Die Programmlogik ist unnötig kompliziert, oder ich stoße auf fast identische Funktionen, die durch eine einzige parametrisierte Funktion ersetzt werden können. Hier muss häufig ein Kompromiss eingegangen werden. Einerseits möchte ich nicht längere Zeit von meiner eigentlichen Aufgabe abgelenkt werden, andererseits möchte ich aber auch nicht den ganzen Abfall herumliegen lassen, der mich bei zukünftigen Änderungen behindert. Wenn sich die Änderung leicht vornehmen lässt, führe ich sie sofort durch. Sollte es etwas aufwendiger sein, das Problem zu beheben, mache ich mir eine Notiz und erledige das, wenn ich meine aktuelle Aufgabe abgeschlossen habe.

Es kann natürlich unter Umständen mehrere Stunden dauern, das Problem zu beheben, und ich habe dringendere Aufgaben zu erledigen. Aber selbst dann

lohnt es sich meistens, kleine Verbesserungen vorzunehmen. Denken Sie an die Pfadfinderregel, den Campingplatz immer ordentlicher zu hinterlassen, als er vorgefunden wurde. Wenn ich jedes Mal, wenn ich den Code durchgehe, eine kleine Verbesserung vornehme, ist das Problem irgendwann behoben. Refactoring hat ja die angenehme Eigenschaft, dass der Code durch die kleinen Schritte nicht funktionsunfähig wird. In manchen Fällen kann es daher Monate dauern, das Problem vollständig zu beheben, aber wie bereits erwähnt, ist der Code zu keinem Zeitpunkt funktionsunfähig, selbst wenn ich ihn erst teilweise durchgearbeitet habe.

2.4.4 GEPLANTES REFACTORING UND SPONTANES REFACTORING

Vorbereitendes Refactoring, Refactoring zwecks Verbesserung der Verständlichkeit und Refactoring als Abfallentsorgung finden jeweils spontan statt. Ich sehe anfangs keine Zeit für das Durchführen von Refactorings vor. Die Refactorings finden vielmehr bei Hinzufügen von Features oder dem Beheben von Bugs statt. Sie sind Teil meines individuellen Arbeitsablaufs beim Programmieren. Beim Hinzufügen eines Features oder beim Beheben eines Bugs hilft mir das Refactoring dabei, meine aktuelle Aufgabe zu erledigen, und ermöglicht mir obendrein, die zukünftige Arbeit zu vereinfachen. Dies ist ein wichtiger Aspekt, der häufig übersehen wird. Refactoring ist keine Tätigkeit, die von der Programmierung getrennt werden kann. Sie reservieren

ja auch keine Zeit für das Schreiben von `if`-Anweisungen. Ich erstelle keinen Zeitplan für die Durchführung von Refactorings. Die meisten Refactorings finden statt, während ich andere Aufgaben erledige.

Wenn Sie auf mangelhaften Code stoßen, müssen Sie ein Refactoring vornehmen – aber auch bei ausgezeichnetem Code sind viele Refactorings erforderlich.

Ein weiterer gängiger Irrtum ist, dass Refactoring nur durchgeführt wird, um früher begangene Fehler zu beheben oder mangelhaften Code zu bereinigen. Wenn Ihnen mangelhafter Code begegnet, müssen Sie natürlich ein Refactoring vornehmen, aber auch bei ausgezeichnetem Code sind viele Refactorings erforderlich. Ich gehe beim Schreiben von Code stets Kompromisse ein – wie umfangreich sollte ich parametrisieren oder wo sollte ich die Grenze zwischen zwei Funktionen ziehen? Die für die gestern hinzugefügten Features zu Recht eingegangenen Kompromisse sind für die neuen Features, die ich heute hinzufüge, möglicherweise ungeeignet. Wenn ich die eingegangenen Kompromisse an die neuen Gegebenheiten anpassen muss, ist das Refactoring von verständlichem Code allerdings viel einfacher.

»Für alle gewünschten Änderungen gilt: Machen Sie die Änderung möglichst einfach (Achtung: das kann

schwierig werden) und führen Sie dann die einfache Änderung durch.«

–Kent Beck

Man hat sich das Schreiben von Software lange als einen Wachstumsprozess vorgestellt: Beim Hinzufügen neuer Features sollten wir vor allem neuen Code hinzufügen. Erfahrene Entwickler wissen jedoch, dass das Hinzufügen eines neuen Features häufig am schnellsten gelingt, wenn der Code zunächst dahingehend geändert wird, dass das Hinzufügen des Features möglichst einfach ist. Software sollte deshalb niemals als »fertig« angesehen werden. Wenn neue Fähigkeiten benötigt werden, muss die Software das widerspiegeln. Die dafür erforderlichen Änderungen am vorhandenen Code sind oft umfangreicher als der neue Code.

All dies bedeutet aber nicht, dass ein geplantes Refactoring grundsätzlich falsch ist. Falls ein Team das Refactoring vernachlässigt hat, muss häufig eigens dafür vorgesehene Zeit bereitgestellt werden, um die Codebasis in einen Zustand zu versetzen, der besser für die neuen Features geeignet ist. Wenn Sie heute dem Refactoring eine Woche widmen, kann diese Zeit sich über Monate hinweg auszahlen. Gelegentlich erkenne ich auch bei regelmäßig stattfindendem Refactoring Probleme, die ein solches Ausmaß annehmen können, dass eine konzertierte Aktion erforderlich ist, um sie zu beheben. Derartige geplante Refactorings sollten aber selten sein.

In den meisten Fällen sollte ein Refactoring ungeplant erfolgen und mit geringem Aufwand verbunden sein. Mir ist der Rat zu Ohren gekommen, Refactorings und neue Features voneinander getrennt in die Versionsverwaltung einzuchecken. Das hat den großen Vorteil, dass sie unabhängig voneinander überprüft und freigegeben werden können. Vollständig überzeugt bin ich davon jedoch nicht. Die Refactorings sind zu oft eng mit den neuen Features verwoben, und der Zeitaufwand, sie voneinander zu trennen, lohnt sich nicht. Darüber hinaus ist möglicherweise nicht mehr erkennbar, in welchem Zusammenhang ein Refactoring stattfand, und dadurch wird es schwierig, solche Commits zu begründen. Jedes Team sollte experimentieren, um herauszufinden, was für es am besten funktioniert. Denken Sie jedoch daran, dass getrennten Refactoring-Commits kein notwendiges Prinzip zugrunde liegt – sie lohnen sich nur, wenn sie Ihnen das Leben erleichtern.

2.4.5 LANGFRISTIGES REFACTORING

Die meisten Refactorings können innerhalb von wenigen Minuten oder höchstens Stunden durchgeführt werden. Es gibt allerdings auch umfangreichere Refactorings, für deren Durchführung ein Team mehrere Wochen benötigt. Vielleicht muss eine vorhandene Bibliothek durch eine neue ersetzt werden. Oder ein Codeabschnitt wird in eine Komponente verschoben, die gemeinsam mit einem anderen Team genutzt werden kann. Oder ein unerfreuliches Durcheinander von Abhängigkeiten, das

sich im Laufe der Zeit entwickelt hat, soll aufgeräumt werden.

Selbst in solchen Fällen zögere ich, ein Team damit zu beauftragen, sich ausschließlich mit dem Refactoring zu beschäftigen. Häufig erweist es sich als nützliche Strategie, zu vereinbaren, die Aufgabe im Laufe der nächsten Wochen allmählich in Angriff zu nehmen. Sobald sich jemand mit Code befasst, der zum Bereich des Refactorings gehört, wird dieser ein wenig in Richtung der beabsichtigten Verbesserung angepasst. Hier wird Nutzen aus der Tatsache gezogen, dass der Code durch das Refactoring nicht funktionsunfähig wird – jede kleine Änderung belässt das System in einem funktionsfähigen Zustand. Beim Wechsel von einer Bibliothek zu einer anderen sollte zunächst eine neue Abstraktion eingeführt werden, die als Schnittstelle für beide Bibliotheken fungieren kann. Wenn der aufrufende Code diese Abstraktion verwendet, ist es viel einfacher, von der einen Bibliothek zur anderen zu wechseln. (Diese Vorgehensweise wird als *Branch By Abstraction* bezeichnet, siehe `https://martinfowler.com/bliki/BranchByAbstraction.html`).

2.4.6 REFACTORING BEIM CODE-REVIEW

In manchen Organisationen finden regelmäßige Code-Reviews statt; diejenigen, die darauf verzichten, wären besser dran, wenn sie es ihnen gleichtäten. Durch Code-

Reviews verbreitet sich Wissen in einem Entwicklerteam. Die Reviews ermöglichen es den erfahreneren Entwicklern, ihr Wissen an die weniger erfahrenen Entwickler weiterzugeben. Auf diese Weise können Entwickler großer Softwaresysteme weitere Aspekte ihrer Systeme kennenlernen und verstehen. Darüber hinaus sind Code-Reviews auch relevant für das Schreiben von verständlichem Code. Mein Code mag mir selbst verständlich erscheinen, meinem Team aber vielleicht nicht. Das ist unvermeidbar – es ist schwierig, sich in jemanden hineinzuversetzen, der mit den Themen, mit denen man sich selbst gerade befasst, nicht vertraut ist. Die Reviews bieten zudem weiteren Personen die Möglichkeit, nützliche Ideen einzubringen. Ich habe in einer Woche eine sehr begrenzte Anzahl guter Ideen. Die Beiträge anderer Leute erleichtern mir das Leben, deshalb nehme ich jede Gelegenheit für Code-Reviews wahr.

Ich habe festgestellt, dass mir Refactoring auch ein Code-Review von fremdem Code erleichtert. Bevor ich damit anfing, Refactoring zu verwenden, konnte ich den Code lesen, ihn in gewissem Maße verstehen und Vorschläge unterbreiten. Wenn ich heutzutage Ideen habe, erwäge ich zunächst, ob sie sich mit Hilfe eines Refactorings auf der Stelle implementieren lassen. Wenn dem so ist, führe ich das Refactoring durch. Nachdem ich das einige Male wiederholt habe, kann ich deutlicher erkennen, wie der Code mit den vorgeschlagenen

Änderungen aussieht. Ich muss mir nicht vorstellen, wie er aussehen würde, sondern sehe ihn direkt vor mir. Letztendlich fallen mir so noch weitere Ideen ein, auf die ich niemals gekommen wäre, wenn ich kein Refactoring durchgeführt hätte.

Refactoring kann außerdem dafür sorgen, dass Code-Reviews konkretere Ergebnisse liefern. Es werden nicht nur reine Vorschläge gemacht – viele davon werden an Ort und Stelle implementiert. Letztendlich hat man ein viel stärkeres Gefühl, durch das Code-Review etwas erreicht zu haben.

Wie ich das Refactoring bei einem Code-Review einbringe, hängt vom Einzelfall ab. Das gängige Pull-Request-Vorgehen, bei dem der Code in Abwesenheit des Autors begutachtet wird, funktioniert nicht besonders gut. Es ist besser, wenn der ursprüngliche Autor des Codes anwesend ist, weil er zusätzlichen Kontext zum Code beisteuern kann und nur so die Intentionen der Änderungswünsche der Review-Partner vollständig nachvollziehen kann. Ich habe die besten Erfahrungen damit gemacht, den Code zusammen in einer Eins-zu-eins-Sitzung mit dem Autor durchzuarbeiten und ein Refactoring gegebenenfalls direkt durchzuführen. Der folgerichtige Schluss dieser Vorgehensweise ist das Pair Programming: kontinuierliches Code-Review während der Programmierung.

2.4.7 WIE SAG ICH'S MEINEM CHEF?

Eine der häufigsten Fragen, die mir im Zusammenhang mit Refactoring gestellt werden, lautet: »Wie sag ich's meinem Chef?« Ich bin schon an so manchem Ort gewesen, an dem Refactoring zu einem unbeliebten Wort geworden ist. Dort glauben die Führungskräfte (und Kunden), dass es beim Refactoring darum geht, früher begangene Fehler zu korrigieren oder alternativ um eine Tätigkeit, die keine verwertbaren Features hervorbringt. Diese Situation wird weiter verschärft durch Teams, die wochenlanges reines Refactoring einplanen – insbesondere dann, wenn sie in Wahrheit kein Refactoring vornehmen, sondern eine unachtsame Umstrukturierung, bei der die Codebasis Schaden nimmt.

Gegenüber einer Führungskraft, die sich wirklich mit Technologie auskennt und die Design-Stamina-Hypothesis verstanden hat, lässt sich ein Refactoring leicht begründen. Solche Führungskräfte sollten die regelmäßige Durchführung von Refactorings fördern und nach Hinweisen dafür suchen, dass ein Team zu wenige vornimmt. Es kommt zwar vor, dass Teams zu viele Refactorings durchführen, sie sind aber viel seltener als Teams, die zu wenige vornehmen.

Natürlich verfügen viele Führungskräfte und Kunden nicht über die technischen Kenntnisse, um beurteilen zu können, wie der Zustand der Codebasis die Produktivität beeinflusst. In solchen Fällen lautet mein eher umstrittener Rat: *Informieren Sie Ihren Chef nicht!*

Sie halten das für rebellisch? Das sehe ich anders. Softwareentwickler sind professionelle Fachkräfte. Unsere Aufgabe ist es, so schnell wie möglich effektive Software zu entwickeln. Meiner Erfahrung nach ist das Refactoring bei der schnellen Entwicklung von Software äußerst hilfreich. Wenn ich eine neue Funktion hinzufügen muss und das Design für diese Änderung ungeeignet ist, geht es meiner Meinung nach schneller, wenn ich erst ein Refactoring vornehme und anschließend die Funktion hinzufüge. Wenn ich einen Bug beheben möchte, muss ich die Funktionsweise der Software verstehen – und ich meine, dass ein Refactoring die schnellste Möglichkeit ist, diese Aufgabe zu erledigen. Ein Manager unter Termindruck verlangt von mir, meine Aufgaben schnellstmöglich zu erledigen; wie ich das anstelle, liegt in meiner Verantwortung. Ich werde für meine Expertise bezahlt, neue Funktionalitäten schnell zu implementieren, und Refactoring bietet die schnellste Möglichkeit – deshalb verwende ich Refactorings.

2.4.8 WANN SOLLTE MAN KEIN REFACTORING DURCHFÜHREN?

Es hört sich vielleicht so an, als ob ich immer zu einem Refactoring rate – es gibt jedoch auch Fälle, in denen es sich nicht lohnt.

Wenn mir Code über den Weg läuft, der zwar ein einziges Chaos ist, den ich aber nicht ändern muss, dann

muss ich auch kein Refactoring durchführen. Mangelhafter Code, den ich als API ansehen kann, kann mangelhaft bleiben. Ein Refactoring bringt mir nur dann Vorteile, wenn ich die Funktionsweise des Codes verstehen muss.

Darüber hinaus gibt es noch den Fall, dass es einfacher ist, den Code neu zu schreiben, als ein Refactoring vorzunehmen. Das ist allerdings eine schwierige Entscheidung. Oftmals kann ich nicht beurteilen, wie aufwendig es wäre, ein Refactoring vorzunehmen, wenn ich nicht eine gewisse Zeit lang versucht habe, ein Gespür dafür zu entwickeln, wie groß der Aufwand tatsächlich ist. Die Entscheidung, ein Refactoring durchzuführen oder den Code neu zu schreiben, erfordert ein gutes Urteilsvermögen und Erfahrung. Daher ist es mir nicht möglich, das Ganze in Form eines einfachen Ratschlags zusammenzufassen.

2.5 Probleme beim Refactoring

Wann immer jemand für ein bestimmtes Verfahren, ein Werkzeug oder eine Architektur plädiert, suche ich nach Problemen. Die wenigsten Dinge im Leben bieten ausschließlich Vorteile. Sie sollten sich der Kompromisse bewusst sein, um zu entscheiden, wann und wo Sie etwas anwenden. Ich halte das Refactoring für ein wirklich nützliches Verfahren – eines, das von den meisten Teams häufiger eingesetzt werden sollte. Allerdings sind

damit auch einige Probleme verbunden, und es ist wichtig, zu verstehen, wie sich diese äußern und wie wir darauf reagieren können.

2.5.1 VERLANGSAMUNG DER ENTWICKLUNG NEUER FEATURES

Nach der Lektüre des letzten Abschnitts sollten Sie meine Ansicht bereits kennen. Obgleich viele Leute der Meinung sind, dass die mit Refactoring verbrachte Zeit die Entwicklung neuer Features verlangsamt, besteht der eigentliche Sinn und Zweck des Refactorings darin, die Entwicklung zu beschleunigen. Das stimmt zwar, aber ebenso richtig ist, dass Refactoring häufig noch immer so wahrgenommen wird, als ob es die Entwicklung verlangsamen würde. Das ist vielleicht die größte Hürde, die verhindert, dass mehr Refactorings durchgeführt werden.

Sinn und Zweck des Refactorings ist, uns schnelleres Programmieren zu ermöglichen, sodass wir mit weniger Aufwand einen größeren Mehrwert schaffen können.

Hier liegt ein echter Zielkonflikt vor. Ich gerate hin und wieder in die Situation, dass ich die Notwendigkeit eines (umfangreichen) Refactorings erkenne, aber das neue Feature, das ich hinzufügen möchte, so klein ist, dass ich es einfach hinzufüge, auf das umfassende Refactoring aber lieber verzichte. Die Entscheidung darüber obliegt

meinem persönlichen Urteilsvermögen, das zu meinen Fähigkeiten als professioneller Entwickler gehört. Mit einigen wenigen Worten lässt sich nicht beschreiben, geschweige denn quantifizieren, wie ich zu dieser Entscheidung gelange.

Mir ist durchaus bewusst, dass ein vorbereitendes Refactoring das Durchführen einer Änderung häufig vereinfacht, deshalb werde ich es sicherlich vornehmen, wenn ich absehen kann, dass es die Implementierung meines neuen Features erleichtert. Darüber hinaus bin ich eher geneigt, ein Refactoring vorzunehmen, wenn es sich um ein mir schon bekanntes Problem handelt – manchmal muss ich erst einige Male über etwas besonders Unschönes stolpern, bevor ich mich entschließe, es durch ein Refactoring zu entfernen. Umgekehrt sinkt die Wahrscheinlichkeit dafür, dass ich ein Refactoring vornehme, wenn es sich um einen Teil des Codes handelt, mit dem ich nur selten in Berührung komme und der mit dem schlechten Code verbundene Mehraufwand für mich erträglich ist. Gelegentlich schiebe ich ein Refactoring auf, weil ich mir nicht sicher bin, welche Verbesserung ich vornehmen sollte, obwohl ich manchmal auch experimentiere und etwas ausprobiere, um zu überprüfen, ob sich dadurch eine Verbesserung ergibt.

Nach allem, was mir meine Kollegen in der Branche berichten, ist zu wenig Refactoring jedoch noch immer deutlich verbreiteter als zu viel. Die meisten Entwickler

sollten also versuchen, häufiger Refactorings durchzuführen. Möglicherweise werden Sie Schwierigkeiten haben, den Unterschied in der Produktivität zwischen einer ordentlichen, gut strukturierten und einer mangelhaften Codebasis zu erkennen, weil Sie nicht genügend Erfahrung mit einer ordentlichen Codebasis sammeln konnten – mit der Stärke, die sich durch das einfache Kombinieren der vorhandenen Bestandteile zu neuen Konfigurationen ergibt, um so selbst komplizierte neue Features schnell umzusetzen.

Häufig werden die Führungskräfte für die kontraproduktive Gepflogenheit kritisiert, Refactorings zu unterbinden, damit die Geschwindigkeit nicht beeinträchtigt wird. Allerdings bin ich auch schon des Öfteren Entwicklern begegnet, die sich selbst so verhalten. Manchmal sind sie der Ansicht, dass ein Refactoring nicht angebracht sei, obwohl ihre Führungskraft es befürwortet. Als technischer Leiter eines Teams müssen Sie Ihren Teammitgliedern zeigen, dass Sie Verbesserungen an der Codebasis wertschätzen. Für das vorhin erwähnte Urteilsvermögen, um zu entscheiden, ob ein Refactoring durchgeführt wird oder nicht, ist jahrelange Erfahrung erforderlich. Diejenigen, die weniger erfahren mit der Anwendung von Refactorings sind, benötigen dabei umfassende Betreuung und Beratung.

Ich bin der Ansicht, dass es am gefährlichsten ist, das

Durchführen von Refactorings durch »sauberen Code«, »gute Ingenieurpraktik« oder ähnliche moralische Gründe zu rechtfertigen. Der Grund für ein Refactoring besteht nicht darin, hervorzuheben, wie schick die Codebasis ist, sondern ist rein ökonomischer Natur. Wir nehmen Refactorings vor, weil wir dadurch schneller vorankommen – beim Hinzufügen von Features, beim Beheben von Bugs. Es ist wichtig, sich das immer wieder zu vergegenwärtigen und auch in der Kommunikation mit anderen hervorzuheben. Die wirtschaftlichen Vorteile, die das Refactoring bietet, sollten stets die treibende Kraft sein. Je besser Entwickler, Führungskräfte und Kunden das verstehen, desto mehr »gutes Design« wird entstehen.

2.5.2 CODE-EIGENTÜMERSCHAFT

Bei vielen Refactorings werden Änderungen vorgenommen, die sich nicht nur auf die Interna eines Moduls, sondern auch auf die Beziehungen zu anderen Teilen des Systems auswirken. Wenn ich eine Funktion umbenennen möchte und alle Aufrufer dieser Funktion finden kann, wende ich einfach *Funktionsdeklaration ändern* (Abschnitt 6.5) an und modifiziere die Deklaration und die Aufrufer in einer einzelnen Änderung. Manchmal ist es jedoch nicht möglich, dieses einfache Refactoring vorzunehmen. Möglicherweise wurde der aufrufende Code von einem anderen Team erstellt und ich besitze keine Schreibrechte für das Repository. Vielleicht ist die Funktion auch Bestandteil

einer deklarierten API, die von Kunden verwendet wird – und ich kann noch nicht einmal feststellen, ob sie überhaupt verwendet wird, geschweige denn, von wem und wie häufig. Solche Funktionen sind Teil einer *öffentlichen Schnittstelle*, die von Clients genutzt wird, welche von den Entwicklern, die diese Schnittstelle deklarieren, unabhängig sind.

Die Eigentümerschaft des Codes behindert das Refactoring, weil ich die Art von Änderungen, die ich vornehmen möchte, nicht ausführen kann, denn dann ist der Client-Code nicht mehr funktionsfähig. Vollkommen unterbunden wird das Refactoring dadurch nicht – viele Aufgaben kann ich trotzdem erledigen. Mir sind jedoch Grenzen gesetzt. Wenn ich eine Funktion umbenenne, muss ich *Funktionsdeklaration ändern* (Abschnitt 6.5) verwenden und die alte Deklaration in Form einer Weiterleitung an die neue Funktion erhalten. Das verkompliziert zwar die Schnittstelle – aber das ist der Preis, den ich zahlen muss, um zu verhindern, dass die Clients nicht mehr funktionieren. Ich könnte die alte Schnittstelle als »veraltet« (deprecated) kennzeichnen und sie irgendwann ausmustern, in manchen Fällen werde ich eine solche Schnittstelle jedoch niemals los.

Aufgrund dieser Komplikationen rate ich von einer feingranularen Code-Eigentümerschaft ab. In manchen Organisationen muss jedem Codeschnipsel ein einzelner Programmierer als Eigentümer zugeordnet sein, und nur dieser ist berechtigt, den Code zu ändern. Ich bin einem

aus drei Personen bestehenden Team begegnet, dessen Vorgehen darin bestand, dass jedes Teammitglied den anderen beiden öffentliche Schnittstellen bereitstellte. Das führte zu allen möglichen Verwerfungen bei der Wartung der Schnittstellen, obwohl es viel einfacher gewesen wäre, die Änderungen direkt an der Codebasis vorzunehmen. Ich ziehe es vor, dass ein Team als Ganzes die Eigentümerschaft übernimmt, sodass jedes Mitglied eines Teams den Code ändern kann, auch wenn er ursprünglich von jemand anderem geschrieben wurde. Allerdings können einzelne Programmierer für bestimmte Bereiche eines Systems verantwortlich sein. Das sollte damit einhergehen, dass sie Änderungen in ihrem Verantwortlichkeitsbereich stets im Auge haben, aber nicht, dass sie diese grundsätzlich blockieren.

Ein solches offeneres Verfahren für die Vergabe der Eigentümerschaft kann sogar teamübergreifend funktionieren. Manche Teams fördern ein Open-Source-ähnliches Modell. Die Mitglieder anderer Teams dürfen einen Zweig ihres Codes ändern, der Commit muss jedoch vom Eigentümer-Team freigegeben werden. Dies ermöglicht, dass ein Team die Clients seiner Funktionen ändert – es kann die alten Deklarationen löschen, sobald die Commits für den Client-Code freigegeben worden sind. Diese Vorgehensweise stellt in großen Systemen häufig einen vernünftigen Kompromiss zwischen strenger Code-Eigentümerschaft und chaotischen Änderungen dar.

2.5.3 ENTWICKLUNGSZWEIGE

In vielen Teams ist es üblich, dass die Teammitglieder mithilfe einer Versionsverwaltung an einem Entwicklungszweig (Branch) der Codebasis arbeiten. Auf diesem Zweig wird die wesentliche Arbeit geleistet, bevor sie in den Hauptentwicklungszweig (der meist als Master oder Trunk bezeichnet wird) des Teams übertragen wird. Dabei wird ein vollständiges Feature nicht selten nur innerhalb des Zweigs entwickelt. Erst dann, wenn das neue Feature abgeschlossen ist und veröffentlicht werden kann, wird es in den Hauptentwicklungszweig integriert. Anhänger dieses Ansatzes behaupten, dass der Hauptentwicklungszweig auf diese Weise von Code verschont bleibt, der noch in Bearbeitung ist, dass der Versionsverlauf der hinzugefügten Features verständlicher ist und dass Features leicht zurückgezogen werden können, falls sie Probleme bereiten sollten.

Solche Feature-Branches weisen allerdings auch Nachteile auf. Je länger ich an einem isolierten Zweig arbeite, desto schwieriger wird es, meine Arbeit in den Hauptentwicklungszweig zu integrieren, wenn ich fertig bin. Die meisten Entwickler versuchen, die damit verbundenen Schwierigkeiten zu entschärfen, indem sie häufig ein Merge oder Rebase ihres Zweiges mit dem Hauptentwicklungszweig durchführen. Das löst allerdings nicht das Problem, wenn mehrere Entwickler zeitglich an Zweigen mit individuellen Features arbeiten.

Ich unterscheide hier zwischen Merge und Integration. Wenn ich den Hauptentwicklungszweig in meinen Code merge, handelt es sich um eine einseitige Änderung – mein Zweig ändert sich, der Hauptentwicklungszweig hingegen nicht. Mit »Integration« bezeichne ich einen beidseitigen Vorgang, der die Änderungen im Hauptentwicklungszweig in meinen Zweig übernimmt und das Ergebnis in den Hauptentwicklungszweig einfließen lässt, sodass sich beide ändern. Ein Beispiel: Wenn Rachel an ihrem Zweig arbeitet, kann ich ihre Änderungen erst dann sehen, nachdem sie ihren Zweig in den Hauptentwicklungszweig integriert hat. Jetzt muss ich den Hauptentwicklungszweig in meinen Zweig mergen, was mit beträchtlichem Aufwand verbunden sein kann. Der schwierige Teil dieser Arbeit ist der Umgang mit semantischen Änderungen. Moderne Versionsverwaltungen können beim Mergen komplexer Änderungen des Programmtexts wahre Wunder vollbringen, sind aber für die Semantik des Codes blind. Wenn ich den Namen einer Funktion geändert habe, wird meine Versionsverwaltung meine und Rachels Änderungen problemlos integrieren. Wenn sie in ihrem Zweig jedoch den Aufruf einer Funktion hinzugefügt hat, die ich in meinem Zweig umbenannt habe, wird der Code nicht mehr funktionieren.

Die durch komplizierte Merges verursachten Probleme wachsen exponentiell an, wenn die Länge der Zweige zunimmt. Die Integration vier Wochen alter Zweige ist

mehr als doppelt so schwierig wie die Integration von Zweigen, die nur zwei Wochen alt sind. Deshalb wird gefordert, dass die Entwicklungszweige kurz gehalten werden sollte – nicht länger als einige wenige Tage. Andere, wie ich, plädieren dafür, sie noch kürzer zu halten. Dieser Ansatz wird als Continuous Integration (kontinuierliche Integration, kurz CI) oder auch als Trunk-based Development bezeichnet. Bei der kontinuierlichen Integration muss jedes Teammitglied seinen Code mindestens einmal täglich in den Hauptentwicklungszweig integrieren. Dadurch wird verhindert, dass die verschiedenen Entwicklungszweige zu sehr voneinander abweichen, was die Komplexität von Merges erheblich verringert. Kostenlos ist Continuous Integration allerdings nicht: Sie müssen Verfahren einsetzen, die gewährleisten, dass der Hauptentwicklungszweig ordentlich strukturiert ist, müssen erlernen, umfangreiche Features in kleinere Bestandteile zu unterteilen, und Schalter für die Features (sogenannte Feature-Flags) einsetzen, mit denen sich in Bearbeitung befindliche Features, die nicht unterteilt werden können, deaktivieren lassen.

Den Anhängern von CI gefällt es natürlich, dass es die Komplexität der Merges reduziert, aber der entscheidende Grund dafür, es zu verwenden, besteht in der viel besseren Kompatibilität mit dem Refactoring. Beim Refactoring finden oft viele kleine, in der gesamten Codebasis verstreute Änderungen statt – die besonders

anfällig für semantische Konflikte beim Mergen sind (wie etwa die Umbenennung einer weithin verwendeten Funktion). Vielen von uns sind Teams begegnet, die Entwicklungszweige für einzelne Features verwenden und der Meinung sind, Refactorings würden die Probleme beim Mergen derart verschärfen, dass sie darauf verzichten. CI und Refactoring arbeiten hingegen gut zusammen – deshalb hat Kent Beck die beiden Verfahren in Form des Extreme Programmings vereint.

Das soll nicht heißen, dass Sie niemals Feature-Branches verwenden sollten. Wenn die Zweige hinreichend kurz sind, treten erheblich weniger Probleme auf. (Tatsächlich verwenden die meisten Anwender von Continuous Integration für gewöhnlich auch Entwicklungszweige, integrieren diese jedoch täglich in den Hauptentwicklungszweig.) Für Open-Source-Projekte mit sporadischen Commits von Programmierern, die Sie kaum kennen (und denen Sie deswegen auch nicht vertrauen), dürften Feature-Branches eine geeignete Technik darstellen. Für ein in Vollzeit beschäftigtes Entwicklerteam sind die Grenzen, die dem Refactoring durch Feature-Branches gesetzt werden, jedoch untragbar. Selbst wenn Sie nicht beabsichtigen, vollständig auf Continuous Integration umzustellen, kann ich nur dringend dazu raten, die Integration so oft wie möglich durchzuführen. Ich möchte Sie zudem auf einen objektiven Nachweis dafür aufmerksam machen, dass Teams bei der Auslieferung ihrer Software deutlich

effektiver sind, wenn sie CI verwenden (Nicole Forsgren, Jez Humble und Gene Kim: *Accelerate: The Science of Lean Software and Devops*, ISBN 1942788339).

2.5.4 TESTEN

Als eine der wichtigsten Eigenschaften des Refactorings gilt, dass es das sichtbare Verhalten des Programms nicht verändert. Wenn ich das Refactoring sorgfältig durchführe, sollte alles wie gehabt funktionieren – aber wenn mir ein Fehler unterläuft, was dann? (Wie ich mich kenne, stellt sich hier nicht die Frage, *ob*, sondern *wann* ich einen Fehler begehe.) Fehler passieren jedem, stellen jedoch kein Problem dar, sofern ich sie umgehend bemerke. Da jedes Refactoring lediglich eine kleine Änderung vornimmt, muss ich auch nur nach einer kleinen Änderung suchen, um den Fehler aufzuspüren, wenn irgendetwas nicht mehr funktioniert. Sollte ich ihn trotzdem nicht finden können, kann ich dank meiner Versionsverwaltung zur letzten funktionierenden Version zurückkehren.

An dieser Stelle ist von entscheidender Bedeutung, einen Fehler schnell finden zu können. Zu diesem Zweck muss ich realistischerweise die Möglichkeit haben, eine umfassende Testsuite für den Code auszuführen – und das muss schnell gehen, damit mich nichts davon abhält, sie möglichst oft auszuführen. Das bedeutet in den meisten Fällen, dass ich selbsttestenden Code benötige (`https://martinfowler.com/bliki/SelfTestingCode.html`)

.

Für manche Leser klingt selbsttestender Code nach einer derart hohen Anforderung, dass sie praktisch nicht zu erfüllen ist. Im Laufe der letzten Jahrzehnte sind mir aber viele Teams begegnet, die Software auf diese Weise erstellen. Für das Testen sind außerordentliche Aufmerksamkeit und großes Engagement erforderlich, was sich aber durch die erzielbaren Vorteile bezahlt macht. Selbsttestender Code ermöglicht nicht nur Refactoring – er macht auch das Hinzufügen neuer Features viel einfacher, denn ich kann die von mir verursachten Bugs sehr schnell finden und beheben. Wenn ein Test fehlschlägt, kann ich mir die Änderungen ansehen, die ich seit dem letzten erfolgreichen Durchlauf der Tests vorgenommen habe – das ist der entscheidende Punkt. Werden die Testläufe häufig genug ausgeführt, sind das nur einige wenige Zeilen Code. Weil ich weiß, dass diese wenigen Codezeilen das Fehlschlagen verursacht haben, kann ich den Bug sehr viel leichter finden.

Wer sich Sorgen macht, dass Refactoring ein zu großes Risiko birgt, Bugs zu verursachen, kann ebenfalls beruhigt sein. Ohne selbsttestenden Code sind das durchaus berechtigte Bedenken – deshalb lege ich so großen Wert darauf, dass verlässliche Tests verfügbar sind.

Es gibt eine weitere Möglichkeit, wie man die

Notwendigkeit zu testen umgehen kann. Wenn ich eine Entwicklungsumgebung verwende, die gute automatisiertes Refactorings bietet, kann ich diesen Refactorings vertrauen – auch ohne Tests durchzuführen. Ich kann also ein Refactoring vornehmen, sofern ich ausschließlich die automatisierbaren Refactorings verwende. Dadurch stehen mir zwar viele sinnvolle Refactorings nicht mehr zur Verfügung, mir verbleiben aber genügend, um mit einigen Vorteilen aufwarten zu können. Ich hätte zwar trotzdem lieber selbsttestenden Code, aber das ist eine Option, die sich als durchaus nützlich erweisen kann.

Es führt auch zu einem Refactoring-Stil, der lediglich eine begrenzte Anzahl von Refactorings verwendet, die nachweislich zuverlässig sind. Refactorings dieser Art sind sprachabhängig, und ihre einzelnen Schritte müssen mit äußerster Sorgfalt durchgeführt werden. Die Teams, die sie eingesetzt haben, sind allerdings zu dem Schluss gelangt, dass auf diese Weise hilfreiche Refactorings großer Codebasen mit geringer Testabdeckung möglich werden. Ich werde in diesem Buch nicht weiter darauf eingehen, weil es sich um ein neues, kaum beschriebenes oder verstandenes Verfahren handelt, das ein sehr spezielles sprachspezifisches Vorgehen erfordert. (Ich hoffe allerdings, auf meiner Website zukünftig etwas mehr zu diesem Thema sagen zu können. Einen kleinen Vorgeschmack erhalten Sie, wenn Sie sich Jay Bazuzis Beschreibung einer sichereren Version von *Funktion*

extrahieren (Abschnitt 6.1) in C++ ansehen (`http://jay.bazuzi.com/Safely-extract-a-method-in-any-C++-code/`).

Es ist nicht weiter überraschend, dass selbsttestender Code eng mit Continuous Integration verknüpft ist – dabei handelt es sich um den Mechanismus, den wir nutzen, um semantische Konflikte bei der Integration aufzuspüren. Solche Testverfahren gehören ebenfalls zum Extreme Programming und sind wesentlicher Bestandteil von Continuous Delivery (CD).

2.5.5 LEGACY CODE

Die meisten Menschen würden sagen, dass ein umfangreiches Vermächtnis eine feine Sache ist – Programmierer sehen das allerdings anders. Legacy Code (veralteter Code) ist oft kompliziert, bietet keine oder kaum zuverlässige Tests und wurde darüber hinaus *von jemand anderem* geschrieben (schauderhaft!).

Refactoring kann sich als ein fantastisches Werkzeug erweisen, um ein veraltetes System besser zu verstehen. Funktionen mit irreführenden Namen können umbenannt werden, damit die Bezeichnungen wieder einen Sinn ergeben, ungeschickte Programmkonstrukte lassen sich abrunden, und schon ist das Programm kein grober Klunker mehr, sondern ein geschliffenes Juwel. Leider hat diese schöne Geschichte einen Haken, nämlich die meist fehlenden Tests. Wenn Sie ein

umfangreiches Altsystem ohne Tests vor sich haben, ist es nicht möglich, ein zuverlässiges Refactoring vorzunehmen, um den Code zu bereinigen.

Die Lösung dieses Problems liegt auf der Hand: Fügen Sie Tests hinzu. Das scheint zwar ein einfaches, wenn auch mühsames Vorhaben zu sein, ist in der Praxis jedoch oft ziemlich knifflig. In den meisten Fällen lässt sich ein System nur dann auf einfache Art mit einer guten Testabdeckung versehen, wenn dem Testen schon beim Design Beachtung geschenkt wurde – dann gäbe es aber schon Tests, und ich müsste mir keine Gedanken darüber machen.

Eine einfache Patentlösung für dieses Problem gibt es nicht. Der beste Rat, den ich geben kann, ist Michael Feathers *Effektives Arbeiten mit Legacy Code* (mitp-Verlag 2010, ISBN 9783826690211) zu lesen und den dortigen Anleitungen zu folgen. Wundern Sie sich nicht über das Alter des Buchs – die Ratschläge sind auch fast ein Jahrzehnt nach seinem Erscheinen noch gültig. Eine grobe Zusammenfassung: Das Buch hilft Ihnen, eine gute Testabdeckung für ihr System zu erzielen, indem Sie Nahtstellen im Programm identifizieren, an denen Sie Tests hinzufügen können. Diese Nahtstellen herauszuarbeiten erfordert wiederum Refactorings, die risikoreich sind, weil sie ohne Tests ausgeführt werden. Dieses Risiko muss man jedoch in Kauf nehmen, um Fortschritte zu erzielen. Hier können sich zuverlässige automatisierte Refactorings als wahrer Segen erweisen.

Wenn sich das Ganze schwierig anhört, liegt das daran, dass es das tatsächlich auch ist. Es gibt leider keine Abkürzung, um aus einer so verfahrenen Situation zu entkommen – deshalb bin ich ein so nachdrücklicher Befürworter davon, von Anfang an selbsttestenden Code zu schreiben.

Aber selbst wenn Tests zur Verfügung stehen, rate ich davon ab, zu versuchen, ein veraltetes Durcheinander mit Hilfe von Refactorings in einem Rutsch in ordentlichen Code zu verwandeln. Ich ziehe es vor, die Aufgabe in angemessenen Schritten in Angriff zu nehmen. Jedes Mal, wenn ich einen Codeabschnitt durchgehe, versuche ich, ihn ein wenig zu verbessern – hier kommt wieder die Pfadfinderregel ins Spiel. Sollte es sich um ein umfangreiches System handeln, führe ich in den Bereichen, die ich häufig in Augenschein nehme, mehr Refactorings durch. Das ist durchaus sinnvoll, denn wenn ich bestimmte Bereiche des Codes häufiger aufsuchen muss, lohnt es sich umso mehr, wenn sie leicht verständlich sind.

2.5.6 DATENBANKEN

In der ersten Ausgabe dieses Buchs ist zu lesen, dass das Refactoring von Datenbanken problematisch ist. Innerhalb eines Jahres nach Veröffentlichung des Buchs war das jedoch nicht mehr der Fall. Mein Kollege Pramod Sadalage entwickelte einen Ansatz für ein evolutionäres Datenbankdesign

(https://martinfowler.com/articles/evodb.html) und ein Refactoring für Datenbanken (https://martinfowler.com/books/refactoringDatabases.html), der inzwischen weit verbreitet ist. Das Verfahren beruht im Wesentlichen darauf, dass es die strukturellen Änderungen an einem Datenbankschema miteinander kombiniert und mit Datenmigrationsskripten auf den Code zugreift, die leicht aneinandergereiht werden können, um größere Änderungen zu handhaben.

Stellen Sie sich als einfaches Beispiel das Umbenennen eines Datenfelds (einer Spalte) vor. Wie beim Refactoring *Funktionsdeklaration ändern* (Abschnitt 6.5) muss ich die ursprüngliche Deklaration der Struktur und alle Aufrufer dieser Struktur finden und sie in einer einzelnen Aktion ändern. Das Komplizierte daran ist, dass ich auch sämtliche Einträge in der Datenbank, die auf dem alten Datenfeld basieren, so anpassen muss, dass sie das neue Datenfeld verwenden. Ich schreibe etwas Code, der diese Transformation ausführt, und speichere ihn zusammen mit dem Code, der die deklarierte Struktur und die Zugriffsroutinen modifiziert, in der Versionsverwaltung. Wenn ich jetzt eine Migration von einer Version der Datenbank zu einer anderen vornehmen muss, führe ich alle vorhandenen Skripte aus, die die aktuelle Version der Datenbank zu der gewünschten Version migrieren.

Wie beim regulären Refactoring ist hier von

entscheidender Bedeutung, dass die einzelnen Modifikationen zwar klein sind, aber dennoch eine jeweils vollständige Änderung abbilden, damit das System nach der Migration weiterhin lauffähig ist. Kleine Änderungen zu verwenden bedeutet, dass sie leicht zu schreiben sind, ich kann jedoch viele davon aneinanderreihen, um größere Änderungen an der Struktur der Datenbank und den darin gespeicherten Daten vorzunehmen.

Ein Unterschied zum regulären Refactoring besteht darin, dass Änderungen an Datenbanken häufig am besten auf mehrere Releases verteilt werden. Das erleichtert es, Änderungen zu revidieren, wenn sie in der Praxis Probleme bereiten. Wenn ich also ein Datenfeld umbenenne, fügt mein erster Commit das Datenfeld hinzu, verwendet es jedoch noch nicht. Anschließend könnte ich die Aktualisierung einrichten, die das neue und das alte Datenfeld gleichzeitig auf den neuesten Stand bringt. Dann kann ich nach und nach alle Zugriffe ändern, sodass sie das neue Datenfeld verwenden. Wenn dieser Vorgang abgeschlossen ist, warte ich eine Weile, damit sich gegebenenfalls vorhandene Bugs zeigen können. Erst dann entferne ich das nun nicht mehr verwendete Datenfeld. Diese Art und Weise, Änderungen an Datenbanken vorzunehmen, ist ein Beispiel für einen allgemeineren Ansatz von Parallel Change (gleichzeitige Änderungen) (`https://martinfowler.com/bliki/ParallelChange.html`),

der mitunter auch als *Expand and Contract* bezeichnet wird.

2.6 Refactoring, Architektur und Yagni

Refactoring hat die Denkweise über Softwarearchitektur nachhaltig verändert. Am Anfang meiner Berufslaufbahn hieß es, Softwaredesign und Architektur seien Dinge, mit denen man sich befassen muss und die weitestgehend abgeschlossen sein sollten, bevor irgendjemand damit anfängt, Code zu schreiben. Nachdem der Code erst einmal geschrieben war, stand die Architektur fest und hätte aufgrund von Nachlässigkeit allmählich verfallen können.

Refactoring hat diese Sichtweise verändert. Es erlaubt mir, die Architektur von Software, die seit Jahren produktiv eingesetzt wird, in beträchtlichem Maße zu ändern. Wie bereits dem Untertitel des Buchs zu entnehmen ist, kann das Refactoring das Design vorhandenen Codes verbessern. Aber wie bereits erwähnt, kann das Ändern von Legacy Code oft eine Herausforderung darstellen, insbesondere, wenn geeignete Tests fehlen.

Die tatsächliche Auswirkung von Refactorings auf die Architektur wird deutlich, wenn man betrachtet, inwieweit es verwendet werden kann, um eine gut strukturierte Codebasis zu bilden, die es ermöglicht, auf veränderte Anforderungen zu reagieren. Wenn die

Architektur abgeschlossen ist, bevor die Programmierung beginnt, besteht das größte Problem darin, dass dieser Ansatz davon ausgeht, dass die Anforderungen an die Software schon von vornherein klar sind. Die Erfahrung zeigt jedoch, dass dieses Ziel oft oder sogar meistens unerreichbar ist. Ich habe wiederholt erlebt, dass Anwender erst erkannten, was sie von der Software tatsächlich erwarten, als die Möglichkeit bestand, sie zu verwenden und über diesen Weg festzustellen, wie sie sich auf die eigene Tätigkeit auswirkt.

Eine Möglichkeit, auf zukünftige Änderungen vorbereitet zu sein, besteht darin, die Software mit entsprechenden Mechanismen auszustatten. Beim Programmieren einer Funktion stelle ich fest, dass sie allgemein anwendbar ist. Um alle vorhersehbaren Anwendungsfälle der Funktion zu handhaben, könnte ich der Funktion ein Dutzend zusätzlicher Parameter hinzufügen. Diese Parameter stellen Mechanismen für eine bessere Flexibilität dar, die wie alle Mechanismen aber nicht umsonst zu haben sind. Das Hinzufügen all dieser Parameter verkompliziert die Funktion für diesen einen Fall, für den sie aktuell verwendet wird. Falls ich einen Parameter übersehe, erschwert die schon vorhandene Parametrisierung das Hinzufügen weiterer Parameter. Ich stelle nicht selten fest, dass meine Flexibilitätsmechanismen nicht richtig funktionieren – entweder, weil die veränderten Anforderungen nicht meinen Erwartungen entsprechen,

oder weil das Design meines Mechanismus fehlerhaft war. Wenn ich das alles berücksichtige, komme ich zu dem Schluss, dass meine Flexibilitätsmechanismen meine Fähigkeit, auf Änderungen zu reagieren, tatsächlich sogar *hemmen*.

Beim Refactoring kann ich eine andere Strategie verfolgen. Anstatt zu spekulieren, welche Flexibilität ich zukünftig benötige und welche Mechanismen dafür am besten geeignet sind, erstelle ich Software, die nur die gegenwärtig bekannten Anforderungen erfüllt, sorge aber für eine hohe Qualität des Softwaredesigns. Sobald sich mein Verständnis darüber ändert, was der Anwender von der Software erwartet, nehme ich ein Refactoring vor, um die Architektur an die neuen Anforderungen anzupassen. Ich kann zudem problemlos Mechanismen einarbeiten, durch die sich die Komplexität nicht erhöht (wie etwa kurze, aussagekräftige Funktionsnamen), aber jegliche Flexibilität, durch die sich die Software verkompliziert, muss sich erst bewähren, bevor ich sie integriere. Wenn sich der Wert eines Parameters bei verschiedenen Aufrufern nicht unterscheidet, füge ich ihn auch nicht zur Parameterliste hinzu. Sollte ich ihn früher oder später doch benötigen, kann ich einfach das Refactoring *Funktion parametrisieren* (Abschnitt 11.2) anwenden. Ich finde es oft nützlich, abzuschätzen, wie schwierig es wäre, später ein Refactoring durchzuführen, um eine vorhersehbare Änderung vorzunehmen. Das Hinzufügen

eines Flexibilitätsmechanismus ziehe ich nur dann in Betracht, wenn erkennbar ist, dass ein späteres Refactoring erheblich schwieriger wäre.

Für diesen Designansatz gibt es mehrere Bezeichnungen: einfaches Design, inkrementelles Design, oder Yagni (ursprünglich ein englisches Akronym für »*You aren't gonna need it*« – »Du wirst es nicht brauchen«, siehe `https://martinfowler.com/bliki/Yagni.html`). Yagni bedeutet nicht, dass die architektonische Denkweise wegfällt, wenngleich es manchmal naiverweise so interpretiert wird. Ich betrachte Yagni als einen anderen Stil, Architektur und Design in den Entwicklungsprozess einzubeziehen – es handelt sich um einen Stil, der sich nur dann bezahlt macht, wenn ihm das Refactoring zugrunde liegt.

Yagni zu akzeptieren heißt nicht, dass ich im Vorhinein stattfindende architektonische Überlegungen komplett vernachlässige. Es gibt noch immer Fälle, in denen Änderungen durch Refactoring schwierig sind und vorbereitende Überlegungen können Zeit sparen. Dennoch hat sich das Gleichgewicht erheblich verschoben – ich neige viel eher dazu, Probleme später, wenn ich sie besser verstehe, in Angriff zu nehmen. Das Ganze hat zu einem wachsenden Fachgebiet der evolutionären Architektur (Neal Ford und Rebecca Parsons: *Building Evolutionary Architectures: Support Constant Change*, ISBN 1491986360) geführt, in dem

Softwarearchitekten die Muster und Praktiken untersuchen, die einen Vorteil aus unserer Fähigkeit ziehen, architektonische Entscheidungen zu überdenken.

2.7 Refactoring und der Softwareentwicklungsprozess

Nach der Lektüre des Abschnitts über Probleme beim Refactoring sind Sie womöglich zu dem Schluss gekommen, dass die Effektivität des Refactorings davon abhängt, welche anderen Verfahren ein Team bei der Softwareentwicklung einsetzt. Tatsächlich wurde Refactoring anfänglich als Teil des Extreme Programming (XP, Kent Beck und Cynthia Andres: *Extreme Programming Explained: Embrace Change*) eingesetzt, einer Methode, die sich dadurch auszeichnet, dass sie eine Reihe vergleichsweise unüblicher ineinandergreifender Verfahren miteinander kombiniert, wie etwa Continuous Integration, selbsttestender Code und Refactoring (die beiden Letzteren gehören auch zur testgetriebenen Entwicklung).

Extreme Programming war eine der ersten Methoden zur agilen Softwareentwicklung und führte über mehrere Jahre hinweg zum Aufkommen agiler Techniken. Inzwischen verwenden so viele Projekte agile Methoden, dass die agile Denkweise allgemein als Mainstream angesehen wird – in Wahrheit verwenden die meisten »agilen« Projekte allerdings nur die Bezeichnung. Um

wirklich auf agile Weise vorzugehen, muss ein Team aus fähigen und begeisterten Anhängern des Refactorings bestehen. Um das zu gewährleisten, müssen viele Aspekte des Entwicklungsprozesses daran ausgerichtet werden, dass Refactorings zu den regulären Aufgaben des Teams gehören.

Die erste Grundvoraussetzung für Refactoring ist selbsttestender Code. Damit ist gemeint, dass eine automatisierte Testsuite zur Verfügung steht und ich mich darauf verlassen kann, dass ein Test fehlschlägt, wenn mir bei der Programmierung ein Fehler unterlaufen ist. Diese Grundvoraussetzung ist von so großer Bedeutung für das Refactoring, dass ich ihr ein eigenes Kapitel gewidmet habe.

Wenn das Refactoring im Team stattfindet, ist es wichtig, dass jedes Teammitglied bei Bedarf ein Refactoring vornehmen kann, ohne die Arbeit der anderen Teammitglieder zu beeinträchtigen. Aus diesem Grund rate ich dazu, Continuous Integration (CI) einzusetzen. Beim Einsatz von CI sind die Refactorings der einzelnen Teammitglieder allen Kollegen schnell zugänglich. So kann es nicht dazu kommen, dass jemand seine Entwicklung auf Schnittstellen aufbaut, die später entfernt werden, und wenn ein Refactoring Probleme mit dem Code anderer Teammitglieder verursacht, lässt sich das schnell feststellen. Selbsttestender Code ist ebenfalls wesentlicher Bestandteil von CI, daher entstehen beim Einsatz von selbsttestendem Code, CI und Refactoring

ausgeprägte Synergieeffekte.

Nachdem wir mit diesen drei Verfahren den Grundstein gelegt haben, kommt der im letzten Abschnitt erwähnte Yagni-Designansatz zur Anwendung. Refactoring und Yagni unterstützen sich gegenseitig: Refactoring (und die damit einhergehenden Voraussetzungen) bildet nicht nur die Grundlage für Yagni – Yagni erleichtert es auch, ein Refactoring vorzunehmen. Das liegt daran, dass es leichter ist, ein einfaches System zu ändern, als eins, das eine Menge spekulativer Flexibilität umfasst. Wenn Sie das richtige Gleichgewicht zwischen beiden Verfahren finden, entsteht eine positive Rückkopplung mit einer Codebasis, die schnell auf wechselnde Anforderungen reagieren kann und verlässlich ist.

Mit diesen zentralen Praktiken haben wir die Grundlage geschaffen, um uns weitere Elemente der agilen Denkweise zunutze zu machen. Dank Continuous Delivery ist unsere Software stets in einem Zustand, in dem wir sie veröffentlichen können. Auf diese Weise gelingt es vielen Internet-Organisationen, mehrmals täglich Updates zu veröffentlichen. Aber auch wenn wir das gar nicht benötigen, reduziert es doch das Risiko und ermöglicht es uns, den Zeitpunkt der Veröffentlichung mehr auf unsere unternehmerischen Anforderungen auszurichten als auf die technischen Rahmenbedingungen. Dank eines soliden technischen Fundaments können wir die erforderliche Zeit, um eine gute Idee in Produktionscode umzusetzen, drastisch

verkürzen und damit den Kundenwünschen leichter nachkommen. Darüber hinaus erhöhen diese Verfahren die Zuverlässigkeit unserer Software, die weniger Bugs enthält, deren Behebung Zeit kostet.

Wenn man den Sachverhalt so darstellt, hört sich das alles ganz einfach an – in der Praxis ist dies allerdings nicht der Fall. Softwareentwicklung ist unabhängig vom verwendeten Ansatz eine knifflige Angelegenheit, die komplexe Interaktionen zwischen Mensch und Maschine beinhaltet. Der Ansatz, den ich hier beschreibe, ist ein bewährtes Verfahren, um diese Komplexität zu handhaben, sind aber wie bei jedem anderen Ansatz auch Übung und Kompetenz erforderlich.

2.8 Refactoring und Performance

Es werden häufig Bedenken geäußert, dass Refactoring sich negativ auf die Performance eines Programms auswirkt. Ich nehme häufig Änderungen vor, die das Programm verlangsamen, um die Software besser verständlich zu machen. Ich bin kein Anhänger der Denkweise, zugunsten der Reinheit des Designs oder in der Hoffnung auf schnellere Hardware die Performance außer Acht zu lassen. Zu langsame Software kann auf Ablehnung stoßen, und schnellere Rechner ändern zwar die Bedingungen, lösen aber das eigentliche Problem nicht. Ein Refactoring kann zweifelsohne eine Verlangsamung der Software verursachen – aber es

macht die Software auch offener für Performanceoptimierungen. Wenn man von zeitkritischen Echtzeitanwendungen absieht, besteht das Geheimnis schneller Software darin, die Software zunächst so zu schreiben, dass die Performance leicht angepasst werden kann, und anschließend die Performance so lange zu optimieren, bis sie ausreichend ist. Mir sind drei allgemeine Ansätze für das Schreiben schneller Software begegnet. Am weitesten geht die Zeitbudgetierung, die häufig in anspruchsvollen Echtzeitsystemen zum Einsatz kommt. Wenn Sie den Entwurf unterteilen, weisen Sie jeder Komponente ein bestimmtes Budget für die Ressourcen Zeit und Speicherplatz zu. Eine Komponente darf ihr Budget nicht überschreiten, allerdings ist ein Mechanismus zum Austauschen der budgetierten Ressourcen gestattet. Bei der Zeitbudgetierung wird dabei besonderer Fokus auf die harten Laufzeitanforderungen gesetzt. Das ist bei Systemen wie etwa Herzschrittmachern, bei denen nicht rechtzeitig eintreffende Daten verhängnisvoll sein können, unverzichtbar. Diese Technik schießt allerdings über das Ziel hinaus, wenn es um die Informationssysteme für Firmen geht, mit denen ich in der Regel zu tun habe.

Der zweite Ansatz besteht in ständiger Aufmerksamkeit. Alle Programmierer sind unablässig darauf bedacht, eine hohe Performance zu gewährleisten. Das ist ein gebräuchlicher Ansatz, der auf den ersten Blick

einleuchtend erscheint – aber nicht besonders gut funktioniert. Veränderungen zur Verbesserung der Performance führen für gewöhnlich dazu, dass die Bedienung des Programms schwieriger wird, was wiederum die Entwicklung verlangsamt. Das könnte man in Kauf nehmen, wenn die Software insgesamt schneller würde – das ist aber meistens nicht der Fall. Die Performanceverbesserungen verteilen sich über das gesamte Programm; jede dieser Verbesserungen erfolgt aus einer zielgerichteten Perspektive hinsichtlich eines bestimmten Programmverhaltens heraus und geht häufig einher mit einer falschen Auffassung davon, wie Compiler, Laufzeitumgebung und Hardware sich verhalten.

Es dauert eine Weile, nichts zu produzieren

Der Zahlungsvorgang im Chrysler Comprehensive Compensation System lief viel zu langsam. Wir befanden uns zwar noch in der Entwicklungsphase, aber es störte uns trotzdem, weil die Tests dadurch verlangsamt wurden.

Kent Beck, Martin Fowler und ich beschlossen, das Problem zu beheben. Während ich auf unser Zusammentreffen wartete, machte ich mir, ausgehend von meiner umfangreichen Kenntnis des Systems, Gedanken darüber, wodurch es wohl so verlangsamt

werden könnte. Ich konnte mir verschiedene Möglichkeiten vorstellen und unterhielt mich mit den Mitarbeitern über die möglicherweise notwendigen Änderungen. Wir hatten einige wirklich gute Ideen, um das System zu beschleunigen.

Dann maßen wir mit Kents Profiler die Performance. Keine der Möglichkeiten, an die ich gedacht hatte, war auch nur ansatzweise für das Problem verantwortlich. Wir stellten vielmehr fest, dass das System während der Hälfte der Zeit damit beschäftigt war, Datumsinstanzen zu erzeugen. Noch interessanter war, dass sämtliche Instanzen die gleichen Werte hatten.

Als wir die Logik zum Erzeugen der Daten untersuchten, fanden wir einige Möglichkeiten, um die Erzeugung dieser Daten zu optimieren. Alle Daten durchliefen eine Stringkonvertierung, obwohl gar keine externen Eingaben erfolgten. Der Code verwendete die Stringkonvertierung lediglich als eine bequeme Möglichkeit der Typisierung. Das würden wir vielleicht verbessern können.

Dann sahen wir uns an, wie diese Daten verwendet wurden. Es stellte sich heraus, dass größtenteils Objekte zur Repräsentation von Datumsbereichen erstellt wurden, also Objekte mit einem Anfangs- und einem Enddatum. Und als wir noch genauer hinsahen, bemerkten wir, dass fast alle Objekte gar keine Werte enthielten!

Bei der Arbeit mit Datumsbereichen verwendeten wir die Konvention, dass ein Datumsbereich, der endet, bevor er anfängt, leer sein soll. Diese Konvention ist durchaus sinnvoll und passt gut zur Funktionsweise dieser Klasse. Kurz nachdem wir diese Konvention eingeführt hatten, fiel uns auf, dass es schwer verständlich ist, einen Zeitraum zu erzeugen, der beginnt, nachdem er endet. Deshalb extrahierten wir dieses Verhalten in eine Fabrikmethode für leere Datumsbereiche.

Wir hatten diese Änderung vorgenommen, um den Code besser verständlich zu machen, erzielten damit jedoch einen unerwarteten Effekt. Wir erzeugten einen leeren Datumsbereich als Konstante und passten die Fabrikmethode so an, dass sie dieses Objekt zurückgibt, anstatt es jedes Mal neu zu erzeugen. Diese Änderung verdoppelte die Geschwindigkeit des Systems, was ausreichte, um die Dauer der Tests erträglich zu machen. Wir benötigten dafür vielleicht fünf Minuten.

Zusammen mit anderen Teammitgliedern hatte ich spekuliert (Kent und Martin bestreiten, an diesen Spekulationen beteiligt gewesen zu sein), was wohl an dem Code, den wir so gut kannten, nicht stimmte. Wir hatten sogar schon einige Designs für Verbesserungen skizziert, ohne vorher gemessen zu haben, was da eigentlich vor sich ging.

> Wir lagen völlig falsch. Abgesehen von einer wirklich spannenden Diskussion hatten wir überhaupt nichts zustande gebracht.
>
> Die Lektion lautet: Auch wenn Sie ganz genau wissen, was in Ihrem System vor sich geht, messen sie die Performance und spekulieren Sie nicht. Sie werden etwas daraus lernen und feststellen, dass Sie mit Ihren Spekulationen in neun von zehn Fällen falsch liegen!
>
> –Ron Jeffries

Das Interessante an der Performance ist, dass die meisten Programme den größten Teil ihrer Zeit in einem kleinen Teil des Codes verbringen. Wenn ich den gesamten Code auf die gleiche Weise optimiere, werde ich letztendlich 90 Prozent meiner Zeit damit verschwenden, Code zu optimieren, der nur selten ausgeführt wird. Die Zeit, die Sie investieren, um das Programm zu beschleunigen – die Zeit, die Sie mangels Klarheit verlieren – ist vollständig verschwendete Zeit.

Der dritte Ansatz zur Performanceoptimierung macht sich diese 90%-Statistik zunutze. Bei diesem Ansatz entwickle ich mein Programm in einer gut strukturierten Weise, ohne der Performance Beachtung zu schenken, bis ich mit einer gezielten Performanceoptimierung beginne. Dabei folge ich einem bestimmten Verfahren.

Ich beginne damit, das Programm zusammen mit einem Profiler auszuführen, der es überwacht und mir anzeigt, wo es Zeit und Speicherplatz verbraucht. Auf diese Weise kann ich die kleinen Teile im Programm identifizieren, die für die Performance entscheidend sind. Anschließend konzentriere ich mich auf diese Stellen und verwende die gleichen Optimierungen, die ich auch beim Ansatz der ständigen Aufmerksamkeit durchführen würde. Da ich mich jedoch auf die performance-kritischen Stellen konzentriere, erziele ich mit weniger Arbeit einen größeren Effekt. Ich bleibe aber trotzdem vorsichtig. Wie beim Refactoring nehme ich Änderungen nur in kleinen Schritten vor. Nach jeder Änderung kompiliere ich, führe die Tests durch und setze erneut den Profiler ein. Wenn sich die Performance nicht verbessert hat, revidiere ich die Änderung. Diese Suche und Korrektur der performance-kritischen Stellen wiederhole ich, bis ich eine Performance erziele, mit der meine Anwender zufrieden sind.

Bei dieser Art der Optimierung ist ein gut strukturiertes Programm auf zweierlei Weise hilfreich. Erstens gibt es mir Zeit für die Performanceoptimierungen, da ich durch gut strukturierten Code die Features schneller hinzufügen kann. So gewinne ich Zeit, um mich auf die Performanceoptimierung zu konzentrieren. (Das Profiling gewährleistet, dass ich diese Zeit an den richtigen Stellen investiere.) Zweitens ermöglicht mir gut strukturierter Code eine feingranulare

Performanceanalyse. Die Teile des Codes, auf die mich der Profiler hinweist, sind kürzer und lassen sich somit leichter verbessern. Wenn der Code gut verständlich ist, habe ich auch einen besseren Überblick über meine Optionen und darüber, welche Optimierungen sinnvoll sein werden.

Ich bin zu dem Schluss gelangt, dass Refactoring mir dabei hilft, schnelle Software zu schreiben. Während des Refactorings wird die Software zwar kurzfristig verlangsamt, lässt sich anschließend aber einfacher optimieren. Am Ende liege ich mit Refactoring deutlich vorn.

2.9 Ursprünge des Refactorings

Es ist mir nicht gelungen, die tatsächliche Geburtsstunde des Begriffs »Refactoring« herauszufinden. Gute Programmierer haben schon immer zumindest etwas Zeit in das Aufräumen ihres Codes investiert. Sie machen das, weil sie aus Erfahrung wissen, dass verständlicher Code sich leichter ändern lässt als komplexer und chaotischer Code. Guten Programmierern ist darüber hinaus klar, dass es ihnen nur selten gelingt, beim ersten Versuch verständlichen Code zu schreiben.

Refactoring geht noch einen Schritt weiter. In diesem Buch propagiere ich, Refactoring als ein Schlüsselelement des gesamten Softwareentwicklungsprozesses anzusehen. Zwei der

Ersten, die erkannt haben, von welcher Bedeutung Refactoring ist, sind Ward Cunningham und Kent Beck, die seit den 1980er-Jahren mit Smalltalk arbeiteten. Smalltalk ist eine Umgebung, die schon damals besonders gut für Refactoring geeignet war. Es ist eine sehr dynamische Umgebung, die es Ihnen ermöglicht, schnell hochfunktionelle Software zu schreiben. Für die damaligen Verhältnisse, in denen es nicht unüblich war, dass Kompiliervorgänge über Nacht laufen, hatte Smalltalk einen sehr kurzen Kompilieren-Linken-Ausführen-Zyklus, wodurch es leicht wurde, Dinge schnell zu ändern. Smalltalk ist zudem objektorientiert und bietet dementsprechend leistungsfähige Werkzeuge, um die Auswirkungen von Änderungen hinter wohldefinierten Schnittstellen zu minimieren. Ward und Kent entwickelten Softwareentwicklungsansätze, die auf eine solche Umgebung zugeschnitten waren. Aus dieser Arbeit ging schließlich das Extreme Programming hervor. Sie erkannten, dass Refactoring wichtig war, um ihre Produktivität zu erhöhen, und sie haben seither immer damit gearbeitet, es auf anspruchsvolle Softwareprojekte angewendet und es verfeinert.

Wards und Kents Ideen hatten großen Einfluss auf die Smalltalk-Community, und der Begriff Refactoring wurde somit zu einem wichtigen Bestandteil der Smalltalk-Kultur. Eine weitere Leitfigur in der Smalltalk-Community ist Ralph Johnson, Professor an der Universität von Illinois in Urbane-Champaign, der als

einer der »Gang of Four« berühmt wurde, den vier Autoren des Buchs über Design Patterns (Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides: *Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software*, mitp-Verlag 2015, ISBN 9783826697005). Eines von Ralphs besonderen Interessengebieten ist die Entwicklung von Software-Frameworks, und er hat untersucht, wie Refactoring dabei helfen kann, effiziente und flexible Frameworks zu entwickeln.

Bill Opdyke war einer von Ralphs Doktoranden und ebenso sehr an Frameworks interessiert. Er erkannte den potenziellen Nutzen des Refactorings und dass es bei weitem nicht nur auf Smalltalk anwendbar war. Er hatte Erfahrung mit der Entwicklung von Telefonvermittlungsanlagen, in denen sich im Laufe der Zeit immer mehr Komplexität aufbaut, wodurch Änderungen erschwert werden. In Bills Doktorarbeit ging es um Refactoring aus der Perspektive eines Programmierers, der Tools für andere Entwickler erstellt. Bill interessierte sich für Refactorings, die für die Entwicklung eines C++-Framework nützlich wären. Er untersuchte die notwendigen semantik-erhaltenden Refactorings und zeigte, wie man beweisen kann, dass sie die Semantik tatsächlich erhalten und wie ein Werkzeug diese Ideen tatsächlich implementieren kann. Bills Dissertation war die erste namhafte Arbeit zum Thema Refactoring.

Ich kann mich noch gut an ein Treffen mit Bill 1992 auf der OOPSLA-Konferenz (Object-Oriented Programming, Systems, Languages, and Applications) erinnern. Wir saßen in einem Café und er erzählte mir von seiner Forschungsarbeit. Ich erinnere mich, wie ich dachte: »Interessant, aber so wichtig nun auch wieder nicht.« Mensch, lag ich falsch!

John Brant und Don Roberts trieben die Idee, ein Werkzeug zu entwickeln, weiter voran und entwickelten schließlich den Refactoring-Browser, das erste Refactoring-Tool für die Smalltalk-Umgebung.

Und ich? Ich hatte schon immer einen Hang zu verständlichem Code, hielt es aber nie für so wichtig. Bis ich zusammen mit Kent an einem Projekt arbeitete und beobachtete, wie er das Refactoring einsetzte. Ich erkannte, welchen Einfluss es auf die Produktivität und die Qualität hatte. Diese Erfahrung überzeugte mich davon, dass Refactoring ein sehr wichtiges Verfahren ist. Allerdings war ich auch frustriert, weil es zu diesem Thema kein Buch gab, das ich einem Programmierer in die Hand hätte drücken können, und keiner der genannten Experten hatte die Absicht, ein solches Buch zu schreiben. Deshalb habe ich es – mit ihrer Hilfe – selbst verfasst. Auf diese Weise ist die erste Ausgabe dieses Buchs entstanden.

Erfreulicherweise konnte sich das Konzept des Refactorings in der Branche etablieren. Das Buch

verkaufte sich gut, und inzwischen gehört Refactoring zum Vokabular der meisten Programmierer. Zudem entstanden weitere Tools, insbesondere für Java. Ein Nachteil dieser Popularität ist allerdings, dass der Begriff »Refactoring« mehr oder weniger zwanglos für jede Art von Umstrukturierung verwendet wird. Trotz alledem hat sich Refactoring als gängiges Verfahren etabliert.

2.10 Automatisiertes Refactoring

Die größte Änderung, die sich in den letzten zehn Jahren ergeben hat, ist die Verfügbarkeit von Werkzeugen, die ein automatisiertes Refactoring unterstützen. Wenn ich eine Methode in Java umbenennen möchte und IntelliJ IDEA (`https://www.jetbrains.com/idea/`) oder Eclipse (`http://www.eclipse.org`) verwende (um nur zwei zu nennen), steht mir dafür ein Menüpunkt zur Verfügung. Das Tool vervollständigt das Refactoring für mich – und ich vertraue für gewöhnlich darauf, dass es funktioniert und erspare es mir, die Testsuite auszuführen.

Das erste dazu fähige Werkzeug war der von John Brant und Don Roberts geschriebene Smalltalk Refactoring Browser. Diese Idee verbreitete sich Anfang des Jahrhunderts wie ein Lauffeuer in der Java-Community. Als JetBrains IntelliJ IDEA veröffentlichte, gehörte das automatisierte Refactoring zu den spannendsten Features. IBM ließ nicht lange auf sich warten und veröffentlichte kurz danach Java-Refactoring-Tools für

Visual Age. Die Visual-Age-Entwicklungsumgebung hatte zwar keinen großen Erfolg, aber ein Großteil ihres Funktionsumfangs wurde auch in Eclipse implementiert, auch die Unterstützung für Refactoring.

Refactoring hielt auch in C# Einzug, ursprünglich mittels JetBrains Resharper, einem Plugin für Visual Studio. Später fügte auch das Visual-Studio-Team einige Refactoring-Fähigkeiten hinzu.

Heutzutage ist es üblich, dass Editoren oder Tools eine Unterstützung für Refactoring bieten, wenngleich sie sich hinsichtlich der tatsächlichen Fähigkeiten ziemlich deutlich unterschieden. Das liegt zum einen an den Werkzeugen selbst, zum anderen aber auch an den Einschränkungen, denen automatisierte Refactorings in verschiedenen Programmiersprachen unterliegen. Ich werde an dieser Stelle nicht näher auf die Fähigkeiten der verschiedenen Tools eingehen, halte es aber für lohnenswert, die zugrunde liegenden Prinzipien kurz zu erläutern.

Eine vergleichsweise primitive Art, ein Refactoring zu automatisieren, besteht darin, Texte zu manipulieren, wie etwa durch Suchen und Ersetzen zum Umbenennen eines Bezeichners oder durch eine Neuorganisation des Codes für das Refactoring *Variable extrahieren* (Abschnitt 6.3). Diesem Ansatz sollte man sicherlich nicht vertrauen, ohne anschließend zu testen. Als ein erster Schritt kann es aber durchaus praktisch sein. Ich

verwende zu diesem Zweck Makros in Emacs, wenn mir keine fortgeschritteneren Refactorings zur Verfügung stehen.

Das Tool muss den Syntaxbaum des Codes bearbeiten, nicht den eigentlichen Text, damit das Refactoring korrekt ausgeführt wird, denn das Bearbeiten des Syntaxbaums gewährleistet sehr viel zuverlässiger die Funktionalität des Codes. Das ist der Grund dafür, dass sich die Möglichkeiten zum automatisierten Refactoring derzeit auf leistungsstarke IDEs beschränken – sie verwenden den Syntaxbaum nicht nur für Refactoring, sondern auch für die Navigation im Code, Codeanalyse und dergleichen. Durch dieses Zusammenwirken von Text und Syntaxbaum heben sich IDEs von einfachen Texteditoren ab.

Refactoring bedeutet nicht nur, den Syntaxbaum zu analysieren und zu aktualisieren. Das Tool muss auch fähig sein, den Code wieder als Text in der Editoransicht darzustellen. Alles in allem ist die Implementierung eines korrekten Refactorings eine ziemlich anspruchsvolle Aufgabe – der ich mir kaum bewusst bin, wenn ich die Tools einfach so verwende.

Viele Refactorings sind deutlich unproblematischer, wenn sie in statisch typisierten Programmiersprachen angewendet werden. Betrachten Sie beispielsweise das einfache Refactoring *Funktionsdeklaration ändern* (Abschnitt 6.5). Nehmen wir an, die Klassen `Salesman`

und `Server` besitzen beide eine `addClient`-Methode. Ich möchte die Methode der `Salesman`-Klasse umbenennen, die eine etwas andere Aufgabe erledigt als diejenige der `Server`-Klasse, deren Namen ich so belassen möchte. Ohne statische Typisierung wird es dem Tool schwerfallen, zu erkennen, ob sich ein Aufruf von `addClient` auf die `Salesman`-Klasse bezieht. Der Refactoring-Browser würde eine Liste aller Methodenaufrufe erstellen und ich könnte selbst entscheiden, welche geändert werden sollen. Dadurch ist das Refactoring nicht mehr zweifelsfrei fehlerlos, sodass anschließend zwingend die Tests ausgeführt werden müssen. Dennoch ist ein solches Tool hilfreich – wenngleich derselbe Vorgang in Java völlig unproblematisch und automatisch möglich ist. Da das Tool dank der statischen Typisierung erkennen kann, welche der beiden Klassenmethoden gemeint ist, kann ich mich darauf verlassen, dass nur diejenigen Methoden umbenannt werden, die tatsächlich umbenannt werden sollen.

Manche Werkzeuge gehen sogar noch weiter. Beim Umbenennen einer Variable kann ich mich auffordern lassen, Änderungen an den Kommentaren vorzunehmen, die den Variablennamen enthalten. Wenn ich *Funktion extrahieren* (Abschnitt 6.1) anwende, findet das Tool Codestellen, die dem Inhalt der neuen Funktion entsprechen, und bietet mir an, den Code durch einen Funktionsaufruf zu ersetzen. Beim Programmieren auf

so leistungsstarke Refactorings wie diese zurückgreifen zu können, ist ein überzeugender Grund dafür, anstelle eines herkömmlichen Texteditors eine IDE zu verwenden. Ich persönlich bin ein großer Freund von Emacs, aber wenn ich in Java programmiere, ziehe es vor, IntelliJ IDEA oder Eclipse zu verwenden – vor allem wegen der Unterstützung von Refactorings.

Die Fähigkeit der komplexen Werkzeuge, zuverlässige Refactorings vorzunehmen, erscheint manchmal fast wie Zauberei, allerdings gibt es auch einige Sonderfälle, die ihnen zu schaffen machen. Manche weniger ausgereifte Tools haben Schwierigkeiten, wenn Reflektion verwendet wird, wie etwa bei `Method.invoke` in Java (ausgereiftere Werkzeuge kommen damit hingegen ganz gut zurecht). Es empfiehlt sich also, auch bei überwiegend zuverlässigen Refactorings hin und wieder die Testsuite auszuführen, um zu gewährleisten, dass nichts aus dem Ruder läuft.

Ich verwende zumeist eine Mischung von automatisierten und manuellen Refactorings und führe die Testsuite dementsprechend häufig genug aus.

Die Leistungsstärke, die eine IDE dank der Verwendung des Syntaxbaums zur Analyse und zum Durchführen von Refactorings bietet, ist ein handfester Vorteil gegenüber einfachen Texteditoren. Dennoch bevorzugen viele Programmierer die Flexibilität ihres Lieblingseditors und würden gerne beide verwenden. Derzeit wird eine

Technologie, die als *Language Server* (`https://langserver.org`) bezeichnet wird, immer populärer: Software, die einen Syntaxbaum erzeugt und Texteditoren eine API bereitstellt. Language Server können viele verschiedene Texteditoren unterstützen und stellen Befehle zur Verfügung, die fortgeschrittene Codeanalysen und Refactoring-Funktionen ermöglichen.

2.11 Weiterführende Literatur

Es mag ein wenig merkwürdig erscheinen, schon am Ende des zweiten Kapitels auf weiterführende Literatur zu verweisen, aber im Grunde genommen ist es egal, wann der Hinweis erfolgt, dass andernorts viele weitere Materialien zum Thema Refactoring verfügbar sind, die über die in diesem Buch erörterten Grundlagen hinausgehen.

Mithilfe dieses Buchs haben bereits viele das Refactoring erlernt, allerdings habe ich mich eher darauf konzentriert, ein Nachschlagewerk für Refactorings zu verfassen und weniger darauf, den Leser beim Lernen zu begleiten. Wenn Sie an einem solchen Buch interessiert sind, empfehle ich Bill Wakes *Refactoring Workbook* (William C. Wake, Addison Wesley 2003, ISBN 0321109295), das viele Aufgaben zum Üben von Refactorings enthält.

Viele der Vorreiter des Refactorings waren auch auf dem

Gebiet der Design Patterns tätig. Josh Kerievsky hat diese beiden Welten mit seinem Buch *Refactorings to Patterns* (https://martinfowler.com/books/r2p.html) eng miteinander verknüpft. In seinem Buch werden die wichtigsten Patterns aus dem einflussreichen Buch der Gang of Four (Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides: *Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software*, mitp-Verlag 2015, ISBN 9783826697005) betrachtet, und es wird aufgezeigt, wie man Refactoring einsetzen kann, um den Code in Richtung der Entwurfsmuster weiterzuentwickeln.

Dieses Buch konzentriert sich zwar auf Refactoring bei der Programmierung im Allgemeinen, allerdings lässt sich Refactoring auch in sehr spezialisierten Bereichen einsetzen, wie beispielsweise im Kontext von Datenbanken (Scott Ambler und Pramod Sadalage: *Refactoring Databases*, https://martinfowler.com/books/refactoringDatabases.html) oder von HTML (Elliotte Rusty Harold: *Refactoring HTML*, https://martinfowler.com/books/refactoringHtml.html).

Auch wenn der Titel das Wort »Refactoring« nicht enthält, soll hier auch Michael Feathers Buch *Effektives Arbeiten mit Legacy Code* (mitp-Verlag 2010, ISBN 9783826690211) nicht unerwähnt bleiben, in dem es vorrangig um das Refactoring veralteter Codebasen geht,

für die kaum Tests vorhanden sind.

Dieses Buch ist zwar (wie schon die erste Ausgabe) für Anwender beliebiger Programmiersprachen gedacht, aber auch Bücher über sprachspezifische Refactorings haben ihren Platz. Zwei meiner früheren Kollegen, Jay Fields und Shane Harvey, haben ein solches Buch für die Programmiersprache Ruby geschrieben (`https://martinfowler.com/books/refactoringRubyEd.html`).

Aktuelle Informationen zu diesen Themen finden Sie auf der Refactoring-Website `https://refactoring.com`.

[1] Ein Ale der Brauerei Unibroue mit 8% Alkoholgehalt

Kapitel 3

Code-Smells: Schlechte Gerüche im Code

von Kent Beck und Martin Fowler

»Wenn es stinkt, muss es gewechselt werden.«

–Oma Beck zur Philosophie der Kindererziehung

Inzwischen dürften Sie eine ziemlich klare Vorstellung davon haben, wie Refactoring funktioniert. Aber nur weil Sie jetzt wissen, wie es anzuwenden ist, heißt das noch lange nicht, dass Sie auch wissen, wann Sie es einsetzten sollten. Die Entscheidung, wann man mit einem Refactoring anfängt – und wann man wieder aufhört –, ist von ebenso großer Bedeutung wie die versierte technische Durchführung.

Und das ist ein Dilemma: Es ist ganz leicht zu erklären, wie eine Instanzvariable gelöscht oder eine Hierarchie erzeugt wird. Das sind simple Angelegenheiten. Die Antwort auf die Frage, *wann* diese Aufgaben erledigt werden sollten, fällt hingegen nicht so klar und deutlich aus. Ich möchte mich dabei nicht auf irgendeine vage

Vorstellung der Ästhetik des Programmierens berufen (was wir Berater zugegebenermaßen ja meistens tun), sondern stattdessen eine handfestere Antwort geben.

Während ich die erste Ausgabe dieses Buchs schrieb und bei Kent Beck in Zürich zu Besuch war, grübelte ich über genau diese Fragestellung nach. Vielleicht stand er unter dem Einfluss des olfaktorischen Eindrucks, den seine damals gerade geborene Tochter bei ihm hinterließ, jedenfalls hatte er sich ausgedacht, das »Wann« eines Refactorings in Form von Gerüchen zu beschreiben.

Nun denken Sie sich vielleicht: »Gerüche? Und das soll besser sein als eine vage Vorstellung der Ästhetik des Programmierens?« Ja, tatsächlich. Wir hatten bis dato wirklich eine Menge Code gesehen, der für Projekte geschrieben wurde, die auf der gesamten Skala von äußerst erfolgreich bis praktisch tot zu finden sind. Dabei haben wir gelernt, auf bestimmte Strukturen im Code zu achten, die es nahelegen – oder uns manchmal auch mit der Nase darauf stoßen –, dass ein Refactoring angebracht ist. (Wir wechseln in diesem Kapitel zur 1. Person Plural, um die Tatsache widerzuspiegeln, dass Kent und ich dieses Kapitel gemeinsam verfasst haben. Sie können den jeweiligen Autor daran erkennen, dass die guten Witze von mir stammen und die anderen von ihm.)

Wir werden nicht den Versuch unternehmen, Ihnen genaue Kriterien dafür zu vermitteln, wann ein

Refactoring überfällig ist. Nach unserer Erfahrung gibt es schlicht und einfach keine Kriterien, die mit dem Urteilsvermögen eines sachkundigen Menschen mithalten können. Wir werden jedoch auf Indikatoren zurückgreifen, die Hinweise darauf geben, dass es möglicherweise Probleme gibt, die durch ein Refactoring zu lösen sind. Sie werden Ihr ganz eigenes Gespür dafür entwickeln müssen, wie viele Instanzvariablen oder wie viele Codezeilen in einer Methode angebracht sind.

Verwenden Sie dieses Kapitel und die Tabelle am Ende des Buchs, wenn Sie sich nicht sicher sind, welche Refactorings angebracht sind. Lesen Sie hier nach (oder überfliegen Sie die Tabelle), um festzustellen, welchen Code-Smell Sie wahrgenommen haben. Schlagen Sie anschließend die von uns empfohlenen Refactorings nach, um festzustellen, ob Sie damit weiterkommen. Sie werden womöglich nicht exakt das wiederfinden, was Sie wahrgenommen haben, kommen aber hoffentlich einen Schritt in der richtigen Richtung voran.

3.1 Rätselhafte Bezeichnung

Beim Lesen eines Krimis kann es durchaus Spaß machen, über den Text nachzugrübeln, um zu verstehen, was passiert – aber nicht beim Lesen von Code. In unserer Fantasie können wir Autoren faszinierender Mitratekrimis sein, aber unser Code muss nüchtern und klar gestaltet sein. Aussagekräftige Namen gehören zu

den wichtigsten Bestandteilen gut verständlichen Codes, deshalb haben wir uns eine Menge Gedanken über die Benennung von Funktionen, Modulen, Variablen und Klassen gemacht, damit sie unmissverständlich aussagen, welchen Zweck sie erfüllen und wie sie zu verwenden sind.

Leider gehört die Auswahl geeigneter Bezeichnungen bei der Programmierung zu den beiden schwierigsten Aufgaben (`https://martinfowler.com/bliki/TwoHardThings.html`). Aus diesem Grund sind Umbenennungen vermutlich die am häufigsten vorgenommenen Refactorings: *Funktionsdeklaration ändern* (Abschnitt 6.5, hier zum Umbenennen einer Funktion), *Variable umbenennen* (Abschnitt 6.7) und *Feld umbenennen* (Abschnitt 9.2). Oft zögern Entwickler, Dinge umzubenennen, weil sie denken, dass sich der Aufwand nicht lohnt. Aber ein aussagekräftiger Name kann Ihnen zukünftiges, stundenlanges Herumrätseln ersparen.

Bei Umbenennungen geht es nicht nur darum, die Bezeichnungen zu ändern. Wenn Ihnen nicht unverzüglich bessere Namen einfallen, ist das oft ein Hinweis auf einen tiefer liegenden Designfehler. Das Rätselraten über irreführende Namen hat schon häufig zu beträchtlichen Vereinfachungen unseres Codes geführt.

3.2 Redundanter Code

Wenn es mehrere Stellen mit gleichartiger Codestruktur gibt (sog. Redundanzen), können Sie sicher sein, dass sich Ihr Programm verbessern lässt, wenn Sie eine Möglichkeit finden, diese Stellen zu vereinen. Redundanter Code bedeutet, dass Sie bei jedem Zugriff darauf sorgfältig überprüfen müssen, ob es irgendeinen Unterschied zwischen den verschiedenen Duplikaten gibt. Wenn Sie Änderungen an redundantem Code vornehmen möchten, müssen Sie sämtliche Duplikate finden und modifizieren.

Das einfachste Problem mit mehrfach vorhandenem Code tritt auf, wenn zwei Methoden derselben Klasse den gleichen Ausdruck enthalten. Sie müssen dann lediglich *Funktion extrahieren* (Abschnitt 6.1) anwenden und den Code an beiden Stellen aufrufen. Wenn zwar ähnlicher, aber nicht vollständig identischer Code vorliegt, sollten Sie sich an dem Refactoring *Anweisungen verschieben* (Abschnitt 8.6) versuchen. Es sieht vor, den Code so anzuordnen, dass ähnliche Teile zusammenhängen, um diese leicht extrahieren zu können. Sollten die mehrfach vorhandenen Codeabschnitte zu Unterklassen derselben Basisklasse gehören, können Sie *Methode nach oben verschieben* (Abschnitt 12.1) verwenden, um so ein gegenseitiges Aufrufen zu verhindern.

3.3 Lange Funktion

Unserer Erfahrung nach sind Programme mit kurzen Funktionen die nachhaltigsten und langlebigsten. Programmierer, die sich zum ersten Mal mit einer solchen Codebasis beschäftigen, haben oft den Eindruck, dass überhaupt keine Berechnungen stattfinden, sondern dass das Programm aus einer endlosen Verkettung von Funktionsaufrufen besteht. Wenn Sie einige Jahre mit solch einem Programm gearbeitet haben, wird Ihnen allerdings klar, wie nützlich diese kurzen Funktionen tatsächlich sind. Kurze Funktionen bieten alle Vorteile der Indirektion: Sie sind oft selbsterklärend, wiederverwendbar und leicht selektierbar.

Seit den Anfangstagen der Programmierung war klar: Je länger eine Funktion ist, desto schwieriger ist es, sie zu verstehen. In älteren Programmiersprachen war der Aufruf von Subroutinen mit einem gewissen Aufwand verbunden, und das hielt die Programmierer davon ab, kurze Funktionen einzusetzen. Bei modernen Sprachen entfällt dieser Aufwand bei prozessinternen Aufrufen weitgehend. Für den Betrachter des Codes ist es zwar immer noch etwas aufwendiger, weil man den Kontext wechseln muss, um zu erkennen, welche Aufgabe eine Funktion erledigt. Es gibt jedoch Entwicklungsumgebungen, die es ermöglichen, schnell zwischen dem Aufruf einer Funktion und ihrer Deklaration hin- und herzuwechseln oder beide gleichzeitig anzuzeigen und diesen Schritt somit zu erleichtern. Wirklich entscheidend für das bessere

Verständnis kurzer Funktionen sind aber aussagekräftige Bezeichnungen. Wenn eine Funktion einen aussagekräftigen Namen hat, müssen Sie sich ihre Deklaration meistens gar nicht ansehen.

Daraus ergibt sich letztendlich, dass Sie bei der Aufteilung von Funktionen aggressiver vorgehen sollten. Wir folgen einer Heuristik, nämlich, dass wir, wenn wir das Bedürfnis verspüren, etwas zu kommentieren, stattdessen eine neue Funktion schreiben. Die Funktion enthält zwar den Code, den wir kommentieren wollten, sie wird aber *entsprechend ihrer Intention* benannt anstatt nach ihrer tatsächlichen Funktionsweise. Wir können diese Vorgehensweise auf mehrere Codezeilen oder auch nur auf eine einzelne Zeile anwenden. Wir gehen sogar auf diese Weise vor, wenn der Aufruf der Methode länger ist als der Code, der dadurch ersetzt wird – vorausgesetzt, die Bezeichnung der Methode zielt auf den Zweck des Codes ab. Hier ist nicht die Länge der Funktion entscheidend, sondern der *semantische Abstand*, also der inhaltliche Unterschied zwischen der Aufgabe, die der Code erledigt, und der Art und Weise, wie er sie erledigt.

In 99% der Fälle müssen Sie lediglich *Funktion extrahieren* (Abschnitt 6.1) anwenden, um eine Funktion zu verkürzen. Suchen Sie nach Teilen der Funktion, die gut zusammenpassen, und erstellen Sie eine neue Funktion.

Wenn eine Funktion viele Parameter und temporäre Variablen verwendet, sind diese beim Extrahieren hinderlich. Wenn Sie versuchen, *Funktion extrahieren* (Abschnitt 6.1) anzuwenden, müssen Sie so viele Parameter an die extrahierte Methode übergeben, dass das Ergebnis kaum besser verständlich ist als der ursprüngliche Code. Häufig ist *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4) anwendbar, um einige der temporären Variablen zu eliminieren. Lange Parameterlisten lassen sich durch *Parameterobjekt einführen* (Abschnitt 6.8) und *Vollständiges Objekt erhalten* (Abschnitt 11.4) verkürzen.

Wenn Sie das ausprobiert haben und noch immer zu viele temporäre Variablen und Parameter vorhanden sind, ist es an der Zeit, schwereres Geschütz aufzufahren: *Funktion durch Befehl ersetzen* (Abschnitt 11.9).

Woran genau erkennt man den zu extrahierenden Code? Halten Sie Ausschau nach Kommentaren, denn sie weisen oft auf diese Art des semantischen Abstands hin. Ein Codeblock mit einem Kommentar, der die Anweisungen erläutert, kann durch eine Methode ersetzt werden, deren Bezeichnung auf dem Kommentar beruht. Es kann sich schon lohnen, eine einzige Zeile zu extrahieren, wenn sie einer Erklärung bedarf.

Bedingte Anweisungen, Fallunterscheidungen und Schleifen sind weitere Hinweise darauf, dass eine Extraktion angebracht sein kann. Verwenden Sie für

bedingte Ausdrücke *Bedingung zerlegen* (Abschnitt 10.1). Eine umfangreiche `switch`-Anweisung können Sie mit *Funktion extrahieren* (Abschnitt 6.1) in eine einzige Funktion umwandeln. Wenn mehrere `switch`-Anweisungen die gleiche Bedingung verwenden, sollten Sie *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) verwenden.

Extrahieren Sie eine Schleife und den Code innerhalb der Schleife und erstellen Sie eine eigene Methode dafür. Wenn es Ihnen schwerfällt, die Methode der extrahierten Schleife zu benennen, könnte das daran liegen, dass sie mehrere unterschiedliche Aufgaben erledigt. Zögern Sie in diesem Fall nicht, *Schleife aufteilen* (Abschnitt 8.7) zu verwenden, um die verschiedenen Aufgaben auszulagern.

3.4 Lange Parameterliste

In den Anfangstagen der Programmierung wurde gelehrt, einer Funktion alle benötigten Werte als Parameter zu übergeben. Das ist auch durchaus verständlich, denn die Alternative war die Verwendung globaler Daten – diese sind allerdings schwer beherrschbar und können schnell Unheil mit sich bringen. Zudem sorgen lange Parameterlisten oft für Verwirrung.

Wenn Sie einen Parameter erhalten können, indem Sie

einen anderen Parameter danach fragen, können Sie durch *Parameter durch Abfrage ersetzen* (Abschnitt 11.5) den zweiten Parameter entfernen. Anstatt lauter Daten aus einer vorhandenen Datenstruktur abzurufen, können Sie *Vollständiges Objekt erhalten* (Abschnitt 11.4) verwenden, um die Datenstruktur selbst zu übergeben. Wenn bestimmte Parameter stets zusammen auftreten, sollten Sie diese mit *Parameterobjekt einführen* (Abschnitt 6.8) zu einer Datenstruktur kombinieren. Wird ein Parameter als Flag (Schalter) verwendet, der unterschiedliches Verhalten auslöst, wenden Sie *Steuerungs-Flag entfernen* (Abschnitt 11.3) an.

Klassen sind bestens dafür geeignet, die Länge von Parameterlisten zu verringern. Sie erweisen sich als besonders nützlich, wenn mehrere Funktionen die gleichen Parameter verwenden. In diesem Fall können Sie *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) verwenden, um die gemeinsamen Werte als Felder zu erfassen. Aus der Perspektive der funktionalen Programmierung handelt es sich hier um eine partiell angewandte Funktion (engl. *partially applied function*).

3.5 Globale Daten

Seitdem Software entwickelt wird, werden wir vor den Gefahren gewarnt, die mit globalen Daten einhergehen. Sie wurden von bösen Dämonen erfunden, die eine

Ebene der Hölle bevölkern, die Ruhestätte derjenigen Programmierer ist, die es wagen, globale Daten zu verwenden. Hinsichtlich Feuer und Schwefel sind wir zwar einigermaßen skeptisch, aber globale Daten verbreiten sozusagen einen stechenden Geruch, dem wir regelmäßig begegnen. Globale Daten sind so problematisch, weil sie überall in der Codebasis modifiziert werden können, und es gibt keine Möglichkeit, festzustellen, welcher Codeabschnitt Änderungen an ihnen vorgenommen hat. Das führt immer wieder zu Bugs, die einer spukhaften Erscheinung aus dem Jenseits gleichen – es ist nämlich sehr schwierig, herauszufinden, welcher Teil des Programms fehlerhaft ist. Dass globale Variablen zu den globalen Daten gehören, liegt auf der Hand, wir zählen jedoch auch Klassenvariablen und Singletons dazu.

Unser wichtigstes Verteidigungsmittel ist hier *Variable kapseln* (Abschnitt 6.6). Das ist immer der erste Schritt, wenn wir auf Daten stoßen, die dafür anfällig sind, von einem beliebigen Programmteil verändert zu werden. Wenn die Daten von einer Funktion gekapselt werden, können Sie zumindest ungefähr verfolgen, wo Änderungen vorgenommen werden, und den Zugriff auf die Daten allmählich kontrollieren. Dann ist es sinnvoll, den Gültigkeitsbereich so weit wie möglich zu begrenzen, indem die Funktion in eine Klasse oder ein Modul verlagert wird, sodass nur der Code der Klasse bzw. des Moduls Zugriffsrechte besitzt.

Globale Daten sind besonders tückisch, wenn sie veränderlich sind. Wenn Sie garantieren können, dass sich die globalen Daten nach dem Start des Programms niemals ändern, sind Sie auf der sicheren Seite – sofern Sie eine Programmiersprache verwenden, die eine solche Garantie ermöglicht.

Globale Daten sind ein schönes Beispiel für Paracelsus' Weisheit: »Alle Dinge sind Gift, und nichts ist ohne Gift; allein die Dosis machts, dass ein Ding kein Gift sei.« Mit einer kleinen Dosis globaler Daten werden Sie vielleicht ungeschoren davonkommen, es wird jedoch exponentiell schwieriger, sie zu handhaben, wenn ihr Umfang zunimmt. Auch wenn es nur sehr wenige Daten sind, ziehen wir es vor, sie zu kapseln – denn das ist der Schlüssel zur Realisierung von Änderungen, wenn die Software weiterentwickelt wird.

3.6 Veränderliche Daten

Änderungen an den Daten haben häufig unerwartete Folgen und führen zu kniffligen Fehlern. So kann ich etwa an einer Stelle Daten verändern, ohne zu bemerken, dass ein weiterer Teil der Software etwas ganz anderes erwartet und nicht mehr funktioniert. Ein solches Fehlschlagen ist besonders schwierig zu entdecken, wenn es nur unter selten vorkommenden Umständen auftritt. Aus diesem Grund beruht eine ganze Denkrichtung der Softwareentwicklung, nämlich die funktionale

Programmierung, auf der Vorstellung, dass Daten sich niemals ändern sollten und dass beim Aktualisieren einer Datenstruktur stets eine Kopie der Datenstruktur mit den Änderungen zurückgegeben werden sollte, damit die ursprüngliche Datenstruktur unangetastet bleibt.

Programmiersprachen dieser Art kommen allerdings noch immer vergleichsweise selten zum Einsatz. Viele von uns verwenden Sprachen, in denen Variablen sich ändern dürfen. Das heißt aber nicht, dass wir die Vorteile der Unveränderlichkeit ignorieren sollten – es gibt noch immer viele Möglichkeiten, die mit uneingeschränkten Aktualisierungen der Daten einhergehenden Risiken zu begrenzen.

Sie können *Variable kapseln* (Abschnitt 6.6) verwenden, um sicherzustellen, dass Aktualisierungen durch schlanke Funktionen erfolgen, die sich einfacher überwachen und weiterentwickeln lassen. Wenn eine Variable aktualisiert wird, die unterschiedliche Dinge speichert, sollten Sie *Variable aufteilen* (Abschnitt 9.1) verwenden. Auf diese Weise trennen Sie die unterschiedlichen Dinge voneinander und vermeiden eine risikoreiche Aktualisierung. Versuchen Sie, so viel Programmlogik wie möglich aus dem Code, der die Aktualisierung vornimmt, zu verlagern, indem Sie *Anweisungen verschieben* (Abschnitt 8.6) und *Funktion extrahieren* (Abschnitt 6.1) verwenden, um den Code, der frei von Seiteneffekten ist, von allem zu trennen, was an der Aktualisierung beteiligt ist. Verwenden Sie für

APIs *Abfrage von Veränderung trennen* (Abschnitt 11.1), um zu gewährleisten, dass Aufrufer keinen Code mit Seiteneffekten aufrufen müssen, wenn es nicht unbedingt erforderlich ist. Wir legen Wert darauf, baldmöglichst *Setter entfernen* (Abschnitt 11.7) anzuwenden – manchmal genügt es schon, nach Aufrufen eines Setters zu suchen, um Möglichkeiten zu finden, den Gültigkeitsbereich einer Variable zu verkleinern.

Veränderliche Daten, die an völlig anderen Stellen berechnet werden können, sind besonders tückisch und verbreiten diesen bereits beschriebenen stechenden Geruch. Sie sorgen nicht nur ständig für Verwirrung, Bugs und verpasste heimische Abendessen – sie sind auch gar nicht notwendig. Wir besprühen sie also mit einer hochkonzentrierten Essiglösung und verwenden *Abgeleitete Variable durch Abfrage ersetzen* (Abschnitt 9.3).

Veränderliche Daten stellen kein großes Problem dar, wenn es sich um eine Variable handelt, deren Gültigkeitsbereich nur einige wenige Zeilen umfasst – das Risiko nimmt jedoch zu, wenn der Gültigkeitsbereich größer wird. Verwenden Sie eines der beiden Refactorings *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) oder *Funktionen zu einer Transformation vereinen* (Abschnitt 6.10), um den Code für die Aktualisierung einer Variable einzugrenzen. Wenn eine Variable Daten mit interner Struktur enthält,

ist es für gewöhnlich besser, die gesamte Struktur zu ersetzen, indem Sie *Referenz durch Wert ersetzen* (Abschnitt 9.4) anwenden, anstatt ihre einzelnen Werte zu modifizieren.

3.7 Divergierende Änderungen

Wir strukturieren unsere Software, um Änderungen zu erleichtern. Der Begriff enthält den Wortbestandteil »Soft«, also »weich« oder »nachgebend«, und so sollte Software auch sein: leicht zu ändern. Wenn wir eine Änderung vornehmen, möchten wir, dass es möglich ist, diese Änderung an einer einzelnen, klar erkennbaren Stelle im System vorzunehmen. Wenn das nicht möglich ist, nehmen Sie einen von zwei eng verwandten Code-Smells wahr.

Divergierende Änderungen treten auf, wenn ein Modul häufig auf unterschiedliche Weise und aus verschiedenen Gründen geändert wird. Nehmen wir an, Sie untersuchen ein Modul und denken: »Ich werde diese drei Funktionen jedes Mal ändern müssen, wenn ich eine neue Datenbank bekomme. Und diese vier Funktionen werde ich ändern müssen, wenn es ein neues Finanzinstrument gibt.« Hierbei handelt es sich um einen Hinweis auf divergierende Änderungen. Fragen der Datenbankzugriffe und der Finanzverwaltung gehören zu verschiedenen Kontexten, und wir können uns das Programmiererleben sehr erleichtern, wenn wir

dafür separate Module verwenden. Wenn wir eine Änderung in einem dieser Bereiche vornehmen, müssen wir auch nur diesen Bereich verstehen und können den anderen außer Acht lassen. Wir hielten das schon immer für wichtig, aber jetzt, da unsere Hirne mit zunehmendem Alter schrumpfen, ist es absolut unverzichtbar. Natürlich bemerkt man die Schwierigkeiten manchmal erst, nachdem schon einige Datenbanken oder Finanzinstrumente hinzufügt wurden. Die Kontextgrenzen eines Programms sind in der Regel zu Beginn undeutlich und verschieben sich meist, wenn die Fähigkeiten eines Softwaresystems ausgebaut werden.

Wenn die beiden Aspekte auf natürliche Weise eine Sequenz bilden – Sie erhalten beispielsweise Daten aus der Datenbank und wenden anschließend die Finanzverfahren darauf an –, dann können Sie die beiden Bereiche mit *Phase aufteilen* (Abschnitt 6.11) durch eine eindeutige Datenstruktur voneinander trennen. Falls die Aufrufe hingegen eher hin- und herspringen, sollten Sie entsprechende Module erstellen und *Funktion verschieben* (Abschnitt 8.1) verwenden, um die Verarbeitungsschritte voneinander zu trennen. Wenn Funktionen beide Arten der Verarbeitungsschritte in sich vereinen, können Sie *Funktion extrahieren* (Abschnitt 6.1) verwenden, um sie vor dem Verschieben voneinander zu trennen. Falls die Module Klassen sind, kann *Klasse extrahieren* (Abschnitt 7.5) bei der

formellen Unterteilung hilfreich sein.

3.8 Chirurgie mit der Schrotflinte

»Chirurgie mit der Schrotflinte« (engl. *Shotgun Surgery*) ist eng verwandt mit divergierenden Änderungen, ist aber das genaue Gegenteil davon. Sie nehmen diesen Code-Smell wahr, wenn Sie bei jeder Änderung, die Sie vornehmen, viele kleine Modifikationen an vielen verschiedenen Klassen durchführen müssen. Wenn die Stellen für notwendige Änderungen weit verstreut sind, ist es schwierig, erstere zu finden, und es passiert nur allzu leicht, dass eine wichtige Änderung übersehen wird.

In diesem Fall sollten Sie die Refactorings *Funktion verschieben* (Abschnitt 8.1) und *Feld verschieben* (Abschnitt 8.2) verwenden und alle Änderungen in einem einzigen Modul zusammenfassen. Falls eine ganze Reihe von Funktionen ähnliche Daten verarbeitet, sollten Sie *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) anwenden. Wenn Funktionen vorhanden sind, die eine Datenstruktur transformieren oder erweitern, sollten Sie *Funktionen zu einer Transformation vereinen* (Abschnitt 6.10) verwenden. Hier erweist sich *Phase aufteilen* (Abschnitt 6.11) oft als nützlich, wenn sich die Ergebnisse der betreffenden Funktionen für eine anschließende Programmphase kombinieren lassen.

Als nützliche Taktik gegen die »Chirurgie mit der

Schrotflinte« erweist sich die Verwendung von Inline-Refactorings wie z.B. *Funktion inline platzieren* (Abschnitt 6.2) oder *Klasse inline platzieren* (Abschnitt 7.6), um schlecht voneinander getrennte Programmlogik zu bündeln. Das führt zwar zu langen Methoden oder großen Klassen, die Sie aber anschließend durch Extraktionen in sinnvollere Teile aufgliedern können. Auch wenn uns kleine Funktionen und Klassen in unserem Code unheimlich lieb sind, scheuen wir nicht davor zurück, bei der Umstrukturierung als Zwischenschritt auch mal größere zu erstellen.

3.9 Feature-Neid

Bei der Modularisierung eines Programms versuchen wir, den Code derart in verschiedene Bereiche aufzuteilen, dass die Interaktion innerhalb eines Bereichs maximiert und die Interaktion zwischen den Bereichen minimiert wird. Ein klassischer Fall von Feature-Neid (engl. *Feature Envy*) liegt vor, wenn eine Funktion in einem Modul den Funktionen oder Daten aus einem anderen Modul mehr Zeit widmet als denen im eigenen Modul. Wir können schon gar nicht mehr zählen, wie oft wir darauf gestoßen sind, dass eine Funktion ein halbes Dutzend Getter-Methoden eines anderen Objekts aufruft, um irgendeinen Wert zu berechnen. Erfreulicherweise gibt es hierfür eine offensichtliche Lösung: Die Funktion will eindeutig bei den Daten sein, verwenden Sie also *Funktion*

verschieben (Abschnitt 8.1), um sie dorthin zu bewegen. Manchmal leidet nur ein Teil einer Funktion unter dem Phänomen des Neids. Verwenden Sie in diesem Fall zunächst *Funktion extrahieren* (Abschnitt 6.1) und anschließend *Funktion verschieben* (Abschnitt 8.1), um ihr ein passendes Zuhause zu bieten.

Natürlich sind nicht alle Fälle so eindeutig. Häufig verwendet eine Funktion Funktionalität mehrerer Module – wohin also soll man sie verschieben? Die Heuristik, die wir verwenden, beruht darauf, zu ermitteln, welches Modul die meisten der betreffenden Daten enthält, und die Funktion mit diesen Daten zu vereinen. Dieser Schritt wird oft vereinfacht, wenn Sie zunächst *Funktion extrahieren* (Abschnitt 6.1) verwenden, um die Funktion in mehrere Teile aufzugliedern, die dann an verschiedenen Stellen platziert werden.

Es gibt natürlich auch einige ausgeklügelte Muster, die gegen diese Regel verstoßen. Von den Entwurfsmustern der »Gang of Four« (Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides: *Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software*, mitp-Verlag 2015, ISBN 9783826697005) kommen mir spontan *Strategie* (Strategy) und *Besucher* (Visitor) in den Sinn. Kent Becks *Selbstdelegation* (Kent Beck: *SmallTalk Best Practice Patterns*, Prentice Hall 1996, ISBN 013476904X) ist ein weiteres. Verwenden Sie diese

Patterns, um divergierende Änderungen zu verhindern. Die Faustregel lautet: Platzieren Sie Dinge, die sich gleichzeitig ändern, am selben Ort. Daten und das auf die Daten bezogene Verhalten ändern sich typischerweise gemeinsam – es gibt jedoch Ausnahmen. Wenn solche Ausnahmen auftreten, verschieben wir das Verhalten, um Änderungen nur auf eine Stelle zu beschränken. Mithilfe der Muster Strategie und Besucher lässt sich das Verhalten leicht ändern, weil sie das zu ändernde Verhalten isolieren – auf Kosten weiterer Indirektion.

3.10 Datenklumpen

Datenelemente sind tendenziell wie Kinder: Es macht ihnen Spaß, gemeinsam miteinander rumzuhängen. Ihnen werden an vielen Stellen die gleichen drei oder vier Datenelemente begegnen: als Felder in einigen Klassen oder als Parameter vieler Methoden. Datenbündel, die miteinander herumhängen, sollten wirklich ein gemeinsames Zuhause finden. Zunächst einmal müssen wir feststellen, wo diese Datenklumpen als Felder auftreten. Wenden Sie *Klasse extrahieren* (Abschnitt 7.5) auf die Felder an, um aus den Datenklumpen ein Objekt zu machen. Richten Sie Ihre Aufmerksamkeit anschließend auf Methodensignaturen und verwenden Sie *Parameterobjekt einführen* (Abschnitt 6.8) oder *Vollständiges Objekt erhalten* (Abschnitt 11.4), um sie zu verschlanken. Der unmittelbare Vorteil der gerade beschriebenen Schritte

besteht darin, dass Sie viele Parameterlisten verkürzen und die Aufrufe der Methoden vereinfachen können. Machen Sie sich keine Gedanken über Datenklumpen, die nur einige der Felder des neuen Objekts verwenden. Wenn Sie mindestens zwei Felder durch das neue Objekt ersetzen können, sind Sie schon einen Schritt vorwärtsgekommen.

Ein guter Test ist es, darüber nachzudenken, einen der Datenwerte zu löschen. Wären die anderen dann noch sinnvoll? Wenn nicht, ist das ein sicheres Zeichen dafür, dass Sie ein Objekt vor sich haben, das es kaum erwarten kann, das Licht der Welt zu erblicken.

Sie werden bemerkt haben, dass wir hier für das Erstellen einer Klasse plädieren, nicht einer einfachen Datensatzstruktur. Das hat den Grund, dass eine Klasse Ihnen die Möglichkeit bietet, einen angenehmen Duft zu verbreiten (um im Bild zu bleiben). Sie können jetzt nach weiteren Fällen *Feature-Neid* (Abschnitt 3.9) suchen, bei denen es angebracht wäre, das Verhalten in neue Klassen auszulagern. Wir haben schon häufig festgestellt, dass durch diese Vorgehensweise nützliche Klassen entstehen, die eine Menge Redundanzen überflüssig machen und die weitere Entwicklung beschleunigen, denn den Daten wird sozusagen ermöglicht, produktive Mitglieder der Gesellschaft zu werden.

3.11 Obsession für elementare

Datentypen

Die meisten Programmierumgebungen beruhen auf einer Reihe gebräuchlicher elementarer Datentypen: Integer, Fließkommazahlen und Strings. Manche Bibliotheken fügen weitere kleine Objekte hinzu, wie etwa Kalenderdaten. Wir haben festgestellt, dass seltsamerweise viele Programmierer zögern, eigene grundlegende Datentypen zu erstellen, die für ihre Aufgabenbereiche nützlich wären – beispielsweise für Geldbeträge, für Koordinaten oder für Datumsbereiche. Uns begegnen also Berechnungen, die Geldbeträge wie einfache Zahlen handhaben, oder Berechnungen physikalischer Größen, die Einheiten ignorieren (Addieren von Millimetern zu Zentimetern), oder jede Menge Code, in dem `if (a < upper && a > lower)` vorkommt.

Strings sind eine besonders häufige Brutstätte für Code-Smells dieser Art: Eine Telefonnummer ist nicht nur eine schlichte Verkettung von Zeichen. Ein angemessener Typ kann zumindest für eine einheitliche Darstellung sorgen, wenn in der Benutzeroberfläche Werte angezeigt werden. Solche Werte in Form von Strings zu repräsentieren, ist ein Code-Smell, der so häufig auftritt, dass hier scherzhaft von »string-typisierten« Variablen gesprochen wird.

Sie können die steinzeitliche Höhle der elementaren Typen verlassen und es sich in der zentralbeheizten Welt

aussagekräftiger Typen gemütlich machen, indem Sie *Elementaren Wert durch Objekt ersetzen* (Abschnitt 7.3) verwenden. Falls es sich bei dem elementaren Wert um einen Typenschlüssel handelt, der bedingtes Verhalten steuert, können Sie *Typenschlüssel durch Unterklassen ersetzen* (Abschnitt 12.6) mit nachfolgendem *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) verwenden.

Ansammlungen elementarer Werte, die häufig zusammen vorkommen, sind Datenklumpen und sollten mit *Klasse extrahieren* (Abschnitt 7.5) sowie *Parameterobjekt einführen* (Abschnitt 6.8) zivilisiert werden.

3.12 Wiederholte switch-Anweisungen

Wenn Sie sich mit überzeugten Verfechtern der objektorientierten Programmierung unterhalten, werden Ihre Gesprächspartner schon bald auf die mit der `switch`-Anweisung verbundenen Übel zu sprechen kommen. Sie argumentieren, dass eine `switch`-Anweisung geradezu nach *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) schreit. Einige Leute sind sogar der Ansicht, dass sämtliche bedingte Programmlogik durch Polymorphie ersetzt werde sollte. Ein Großteil aller `if`-Anweisungen wäre dann Geschichte.

Selbst in unseren wildesten Zeiten haben wir Fallunterscheidungen oder bedingte Anweisungen nie uneingeschränkt abgelehnt. Allerdings enthielt die erste Ausgabe dieses Buchs einen Code-Smell namens »`switch`-Anweisungen«, weil wir Ende der 1990er-Jahre der Meinung waren, dass Polymorphie leider stark unterschätzt wurde, und es für richtig hielten, ihre Verwendung zu fördern.

Heutzutage kommt Polymorphie häufiger zum Einsatz, und `switch`-Anweisungen sind nicht mehr das einfache Warnsignal, das sie vor fünfzehn Jahren oft waren. Darüber hinaus unterstützen viele Programmiersprachen ausgeklügelte `switch`-Anweisungen, die nicht nur elementare Typen zur Grundlage haben. Wir konzentrieren uns im Folgenden deshalb auf die wiederholte Verwendung der gleichen Steuerungslogik (entweder in einer `switch/case`-Anweisung oder in einer Abfolge von `if/else`-Anweisungen), die an mehreren Stellen auftaucht. Die redundante Verwendung solcher Fallunterscheidungen ist problematisch, weil Sie beim Hinzufügen eines weiteren Falls nach sämtlichen `switch`- bzw. `if`-Anweisungen suchen und sie aktualisieren müssen. Polymorphie bietet der Codebasis einen eleganten Schutz vor den dunklen Mächten solcher Redundanzen.

3.13 Schleifen

Seitdem es die ersten Programmiersprachen gab, sind Schleifen Kernbestandteil der Programmierung. Wir sind jedoch der Ansicht, dass sie heutzutage nicht bedeutsamer sind als Schlaghosen oder Samttapeten. Wir haben sie bei der ersten Ausgabe geringgeschätzt – allerdings bot Java damals, wie auch die meisten anderen Sprachen, keine bessere Alternative. Heute jedoch werden First-Class-Funktionen umfassend unterstützt, sodass wir *Schleife durch Pipeline ersetzen* (Abschnitt 8.8) verwenden können, um diese Anachronismen in den Ruhestand zu versetzen. Wir haben festgestellt, dass Pipeline-Operationen, wie etwa Filtern (`filter`) oder Zuordnen (`map`), dabei helfen können, schnell zu erkennen, welche Elemente verarbeitet werden und wie mit ihnen zu verfahren ist.

3.14 Träges Element

Wir verwenden zum Ergänzen von Struktur gern Programmelemente – sie bieten Möglichkeiten, Varianten zu erstellen, sie wiederzuverwenden oder einfach nur aussagekräftigere Bezeichnungen zu vergeben. In manchen Fällen ist diese Struktur jedoch überflüssig. Denken Sie beispielsweise an eine Funktion, deren Bezeichnung dem enthaltenen Code entspricht, oder an eine Klasse, die im Wesentlichen nur aus einer einfachen Funktion besteht. Manchmal handelt es sich dabei um eine Funktion, von der erwartet wurde, dass sie später wachsen und häufig verwendet werden würde,

aber dazu kam es nicht. Es könnte sich auch um eine Klasse handeln, die ihre Aufgabe eigentlich gut erledigte, aber durch ein Refactoring verkleinert wurde. Wie dem auch sei, solche Programmelemente sollten in Würde sterben. Meistens bedeutet das, *Funktion inline platzieren* (Abschnitt 6.2) oder *Klasse inline platzieren* (Abschnitt 7.6) einzusetzen. Wenn Vererbung im Spiel ist, können Sie *Hierarchie abbauen* (Abschnitt 12.9) verwenden.

3.15 Spekulative Generalisierung

Die Bezeichnung »Spekulative Generalisierung« (engl. *Speculative Generality*) für einen Code-Smell, bei dem wir besonders hellhörig werden, wurde von Brian Foote vorgeschlagen. Sie tritt beispielsweise auf, wenn jemand sagt: »Früher oder später werden wir das wohl benötigen« und daraufhin alle möglichen Erweiterungen hinzugefügt und Spezialfälle berücksichtigt werden, die momentan tatsächlich überflüssig sind. Das führt häufig zu Code, der schwerer zu verstehen ist und sich schlechter warten lässt. Das Vorgehen könnte sinnvoll sein, wenn all diese Erweiterungen tatsächlich zum Einsatz kämen. Ist das aber nicht der Fall, war die Arbeit umsonst. Sie werden durch all die ungenutzte Funktionalität nur behindert, also weg damit.

Falls Sie abstrakte Klassen verwenden, die nur wenige Aufgaben erledigen, sollten Sie *Hierarchie abbauen*

(Abschnitt 12.9) verwenden. Unnötige Delegationen lassen sich durch *Funktion inline platzieren* (Abschnitt 6.2) oder *Klasse inline platzieren* (Abschnitt 7.6) entfernen. Funktionen mit ungenutzten Parametern können Sie mithilfe von *Funktionsdeklaration ändern* (Abschnitt 6.5) von den Parametern befreien. Dieses Refactoring sollten Sie ebenfalls anwenden, um nicht benötigte Parameter zu entfernen, die häufig aufgrund geplanter, aber später nie zustande kommender Erweiterungen angelegt wurden.

Spekulative Generalisierung können Sie daran erkennen, dass eine Funktion oder eine Klasse (wenn überhaupt) ausschließlich innerhalb von Testfällen verwendet wird. Falls Ihnen ein solches Getier über den Weg läuft, sollten Sie den Testfall löschen und *Toten Code entfernen* (Abschnitt 8.9) anwenden.

3.16 Temporäres Feld

Gelegentlich begegnen Sie einer Klasse, in der einem Feld nur unter bestimmten Umständen ein Wert zugewiesen wird. Code dieser Art ist schwer verständlich, weil Sie eigentlich erwarten, dass ein Objekt alle seine Felder benötigt. Es kann Sie in den Wahnsinn treiben, wenn Sie versuchen zu verstehen, warum ein Feld vorhanden ist, das offenbar nicht verwendet wird.

Verwenden Sie *Klasse extrahieren* (Abschnitt 7.5), um den verwaisten Variablen ein neues Zuhause zu geben,

und *Funktion verschieben* (Abschnitt 8.1), um sämtlichen Code, der das Feld betrifft, in die neue Klasse zu verlagern. Möglicherweise können Sie auch bedingten Code entfernen, indem Sie *Sonderfall einführen* (Abschnitt 10.5) verwenden, um für den Fall, dass die Variablen ungültig sind, eine alternative Klasse zu erstellen.

3.17 Mitteilungsketten

Mitteilungsketten treten auf, wenn ein Client ein Objekt nach einem anderen Objekt fragt, das der Client wiederum nach einem weiteren Objekt fragt, das der Client um noch ein weiteres Objekt fragt und so weiter. Dieser Vorgang könnte als eine Aneinanderreihung von `getThis`-Methoden oder als Sequenz temporärer Variablen in Erscheinung treten. Diese Art der Navigation bedeutet, dass der Client an die Struktur der Navigation gekoppelt ist. Jede Änderung an den Beziehungen der beteiligten Objekte untereinander erfordert es, den Client entsprechend anzupassen.

Hier sollte *Delegation verbergen* (Abschnitt 7.7) zum Einsatz kommen. Sie können das an mehreren Stellen der Mitteilungskette durchführen, prinzipiell sogar für alle zur Kette gehörigen Objekte, allerdings können die beteiligten Objekte dadurch zu Vermittlern werden. Zu überprüfen, wofür das resultierende Objekt verwendet wird, ist vielfach eine bessere Alternative. Prüfen Sie, ob

Sie *Funktion extrahieren* (Abschnitt 6.1) anwenden können, um einen Teil des Codes, der das resultierende Objekt verwendet, zu extrahieren, und anschließend *Funktion verschieben* (Abschnitt 8.1), um ihn in der Kette nach unten zu verschieben.

Manche halten die Verkettung von Methoden für Teufelswerk. Wir hingegen sind für unsere differenzierte Betrachtung der Dinge bekannt. Zumindest in diesem Fall.

3.18 Vermittler

Zu den wichtigsten Merkmalen eines Objekts gehört die Kapselung – das Verbergen interner Details vor dem Rest der Welt. Kapselung und Delegation gehen oft Hand in Hand. Sie fragen Ihre Chefin, ob sie Zeit für ein Treffen hat; sie leitet Ihre Mitteilung an ihren Kalender weiter (»sie delegiert sie«) und gibt Ihnen eine Antwort. Alles schön und gut. Es ist nicht notwendig, zu wissen, ob sie einen Kalender einsetzt, einen digitalen Assistenten verwendet oder ob ihre Sekretärin die Termine im Auge behält.

Delegation kann aber auch zu weit gehen. Sie betrachten die Schnittstelle einer Klasse und stellen fest, dass die Hälfte der Methodenaufrufe an eine andere Klasse delegiert wird. Früher oder später ist es an der Zeit, *Vermittler entfernen* (Abschnitt 7.8) anzuwenden und

sich direkt an das Objekt zu wenden, dem wirklich bekannt ist, was ausgeführt werden soll. Wenn nur ein paar Methoden vorhanden sind, die zudem wenige Aufgaben erledigen, sollten Sie *Funktion inline platzieren* (Abschnitt 6.2) anwenden, um sie in den Aufrufer zu verschieben. Wenn zusätzliches Verhalten vorhanden ist, können Sie *Basisklasse durch Delegation ersetzen* (Abschnitt 12.11) oder *Unterklasse durch Delegation ersetzen* (Abschnitt 12.10) verwenden, um den Vermittler zum Bestandteil des wirklich wichtigen Objekts zu machen. Auf diese Weise können Sie das Verhalten erweitern, ohne sämtliche Delegationen nachverfolgen zu müssen.

3.19 Insiderhandel

Softwareentwickler errichten gern hohe Mauern, um ihre Module voneinander zu trennen, und beklagen sich bitterlich, dass der übertriebene Austausch von Daten die Kopplung zu sehr verstärkt. Nun ist ein gewisser Austausch der Daten zwar unverzichtbar, wir sollten ihn jedoch auf ein Minimum beschränken und dabei sorgfältig vorgehen.

Module, die hinter unserem Rücken miteinander tuscheln, sollten durch *Funktion verschieben* (Abschnitt 8.1) und *Feld verschieben* (Abschnitt 8.2) voneinander getrennt werden, um die notwendige Kommunikation zu verringern. Wenn Module gemeinsame Interessen

haben, sollten Sie versuchen, ein drittes Modul zu erstellen, um diese Gemeinsamkeiten zu vereinen, oder *Delegation verbergen* (Abschnitt 7.7) verwenden, um ein weiteres Modul zu erstellen, das als Vermittler fungiert.

Die Vererbung führt oft zu heimlichen Absprachen, und die Unterklassen wissen oft mehr über ihre Basisklassen, als Ihnen lieb ist. Wenn die Zeit gekommen ist, sie voneinander zu trennen, können Sie *Unterklasse durch Delegation ersetzen* (Abschnitt 12.7) oder *Basisklasse durch Delegation ersetzen* (Abschnitt 12.11) verwenden.

3.20 Umfangreiche Klasse

Wenn eine Klasse versucht, zu viele Aufgaben selbst zu erledigen, ist das oft daran zu erkennen, dass sie zu viele Felder besitzt. Und wenn das der Fall ist, geht damit meist auch redundanter Code (Abschnitt 3.2) einher.

Verwenden Sie *Klasse extrahieren* (Abschnitt 7.5), um mehrere Variablen zu bündeln. Identifizieren Sie Variablen, die sinnvollerweise zusammengehören. So werden beispielsweise die beiden Variablen `depositAmount` und `depositCurrency` für einen Geldbetrag und dessen Währung höchstwahrscheinlich in eine gemeinsame Komponente gehören. Im Allgemeinen weisen gemeinsame Präfixe oder Suffixe der Bezeichnungen bei mehreren Variablen einer Klasse auf die Möglichkeit für eine eigene Komponente hin. Wenn

für diese Komponente Vererbung angebracht ist, ist es oft einfacher, *Basisklasse extrahieren* (Abschnitt 12.8) oder *Typenschlüssel durch Unterklassen ersetzen* (Abschnitt 12.6) zu verwenden. (Letzteres ist im Wesentlichen die Extraktion einer Unterklasse.)

In manchen Fällen verwendet eine Klasse gar nicht durchgängig alle ihre Felder. In diesem Fall können Sie womöglich mehrere Extraktionen vornehmen.

Ebenso wie eine Klasse mit zu vielen Instanzvariablen ist auch eine Klasse, die zu viel Code enthält, eine Brutstätte für redundanten Code, Chaos und Leid. Die einfachste Lösung (hatten wir schon erwähnt, dass wir einfache Lösungen mögen?) besteht darin, Redundanzen in der Klasse selbst zu beseitigen. Wenn es fünf hundertzeilige Methoden mit jeweils einigen redundanten Code-Abschnitten gibt, können Sie aus dem ursprünglichen Code vielleicht fünf zehnzeilige Methoden und zehn weitere zweizeilige Methoden machen.

Häufig bieten die Clients einer solchen Klasse die besten Hinweise für das Aufteilen der Klasse. Prüfen Sie, ob die Clients nur eine Teilmenge der Funktionalität der Klasse nutzen. Jede dieser Teilmengen ist potenziell für eine separate Klasse geeignet. Wenn Sie eine solche Teilmenge identifiziert haben, können Sie *Klasse extrahieren* (Abschnitt 7.5), *Basisklasse extrahieren* (Abschnitt 12.8) oder *Typenschlüssel durch Unterklassen ersetzen* (Abschnitt 12.6) für die Zerlegung

verwenden.

3.21 Alternative Klassen mit unterschiedlichen Schnittstellen

Die Verwendung von Klassen bietet den großen Vorteil der Substituierbarkeit, die es bei Bedarf ermöglicht, eine Klasse durch eine andere auszutauschen. Das funktioniert allerdings nur, wenn die Schnittstellen übereinstimmen. Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um Funktionen einander anzugleichen. In vielen Fällen ist das nicht ausreichend; wenden Sie zum Verschieben des Verhaltens in Klassen gegebenenfalls so lange *Funktion verschieben* (Abschnitt 8.1) an, bis die Schnittstellen übereinstimmen. Sollte dies zu Redundanzen führen, können Sie zu deren Beseitigung das Refactoring *Basisklasse extrahieren* (Abschnitt 12.8) einsetzen.

3.22 Datenklasse

Hierbei handelt es sich um Klassen mit Feldern, die außerdem Getter- und Setter-Methoden für die Felder besitzen – und sonst nichts. Solche Klassen sind schlichte Datencontainer und werden oft von anderen Klassen in viel zu großem Ausmaß verändert. Unter Umständen besitzen diese Klassen öffentlich zugängliche Felder. Wenn das der Fall ist, sollten Sie umgehend *Datensatz kapseln* (Abschnitt 7.1) anwenden, bevor das

irgendjemand bemerkt. Wenden Sie *Setter entfernen* (Abschnitt 11.7) auf jedes Feld an, das nicht geändert werden soll.

Halten Sie danach Ausschau, an welchen Stellen diese Getter- und Setter-Methoden von anderen Klassen verwendet werden. Versuchen Sie, das Verhalten mit *Funktion verschieben* (Abschnitt 8.1) in die Datenklasse zu verlagern. Wenn es nicht möglich ist, die gesamte Funktion zu verschieben, können Sie mit *Funktion extrahieren* (Abschnitt 6.1) eine Funktion erstellen, die sich verschieben lässt.

Datenklassen sind oft ein Hinweis darauf, dass sich Verhalten an einer falschen Stelle befindet. Sie können eine große Verbesserung erzielen, indem Sie das Verhalten vom Client in die Datenklasse selbst verlagern. Hier gibt es jedoch Ausnahmen. Das beste Beispiel ist ein Datensatz, der von einem gesonderten Funktionsaufruf als Ergebnisdatensatz verwendet wird, wie etwa die temporäre Datenstruktur nach der Anwendung von *Phase aufteilen* (Abschnitt 6.11). Für einen solchen Ergebnisdatensatz ist charakteristisch, dass er unveränderlich ist (zumindest in der Praxis). Unveränderliche Felder müssen nicht gekapselt werden, und die aus unveränderlichen Daten abgeleiteten Informationen können als Felder anstatt als Getter-Methoden repräsentiert werden.

3.23 Ausgeschlagenes Erbe

Unterklassen erben die Methoden und Daten ihrer Eltern. Aber was ist, wenn sie dieses Erbe überhaupt nicht wollen oder benötigen? Sie erhalten ein großes Vermächtnis und wählen doch nur einige wenige Geschenke aus.

Das bedeutet klassischerweise, dass die Hierarchie falsch aufgebaut ist. Sie müssen eine neue Geschwisterklasse erstellen und *Methode nach unten verschieben* (Abschnitt 12.4) und *Feld nach unten verschieben* (Abschnitt 12.5) verwenden, um den überflüssigen Code in die Geschwisterklasse zu verlagern. Auf diese Weise enthält die Basisklasse nur noch die Gemeinsamkeiten. Hier wird oft der Rat gegeben, dass alle Basisklassen abstrakt sein sollten.

Aufgrund des despektierlichen Gebrauchs von »klassischerweise« ahnen Sie vielleicht schon, dass wir dem nicht zustimmen – zumindest nicht immer. Wir verwenden ständig Unterklassen, um ein bestimmtes Verhalten wiederzuverwenden, und betrachten das als völlig unproblematisch. Es liegt etwas in der Luft – das können wir nicht abstreiten – aber dieser Code-Smell ist für gewöhnlich erträglich. Wenn das ausgeschlagene Erbe Verwirrung oder Probleme verursacht, können Sie dem klassischen Rat folgen. Sie sollten jedoch nicht den Eindruck bekommen, dass dies in jedem Fall erforderlich ist. In neun von zehn Fällen ist dieser Code-Smell so

harmlos, dass es nicht lohnt, ihn zu beseitigen.

Allerdings ist der Code-Smell des ausgeschlagenen Erbes nicht mehr so harmlos, wenn die Unterklasse Verhalten wiederverwendet, aber die Schnittstelle der Basisklasse nicht mehr unterstützen möchte. Die Implementierung ist uns egal, aber wenn die Schnittstelle nicht unterstützt wird, dann sitzen wir auf dem sprichwörtlichen hohen Ross fest im Sattel. In diesem Fall sollten Sie nicht an der Hierarchie herumtüfteln; verwenden Sie stattdessen *Unterklasse durch Delegation ersetzen* (Abschnitt 12.10) oder *Basisklasse durch Delegation ersetzen* (Abschnitt 12.11).

3.24 Kommentare

Keine Sorge, wir behaupten nicht, dass keine Kommentare verwendet werden sollten. Gemäß unserer olfaktorischen Analogie sind Kommentare kein unangenehmer Geruch, sondern vielmehr ein süßer Duft. Dass wir hier Kommentare erwähnen, hat folgenden Grund: Sie werden oft als Deodorant missbraucht. Es ist wirklich erstaunlich, wie oft man auf ausgiebig kommentierten Code stößt und feststellen muss, dass die Kommentare vorhanden sind, weil der Code zu wünschen übrig lässt.

Kommentare führen uns zu mangelhaftem Code, der sämtliche üblen Geruchsnuancen in sich trägt, die wir in

diesem Kapitel erläutert haben. Als Erstes entfernen wir die Code-Smells durch ein Refactoring. Wenn das erledigt ist, stellen wir häufig fest, dass die Kommentare überflüssig sind.

Wenn Sie einen Kommentar für erforderlich halten, um die Aufgabe eines Codeblocks zu erläutern, sollten Sie *Funktion extrahieren* (Abschnitt 6.1) ausprobieren. Wenn Sie die Methode bereits extrahiert haben und immer noch denken, dass Sie einen Kommentar benötigen, sollten Sie *Funktionsdeklaration ändern* (Abschnitt 6.5) verwenden, um sie umzubenennen. Wenn Sie einige Hinweise darlegen möchten, die den erforderlichen Zustand des Systems betreffen, können Sie dafür *Assertion einführen* (Abschnitt 10.6) nutzen.

Tipp

Wenn Sie das Bedürfnis haben, einen Kommentar zu verfassen, versuchen Sie zunächst, ein Refactoring vorzunehmen, sodass jeglicher Kommentar überflüssig wird.

Ein guter Moment, einen Kommentar zu verwenden, ist, wenn Sie nicht mehr wissen, was Sie tun sollen. Kommentare beschreiben nicht nur, was vor sich geht, sie können auch Hinweise auf Bereiche geben, in denen

Sie noch unsicher sind. Ein Kommentar kann zudem darüber Aufschluss geben, warum Sie etwas getan haben. Diese Art von Information ist für zukünftige Bearbeiter hilfreich, vor allem, wenn sie vergesslich sind.

Kapitel 4

Tests erstellen

Refactoring ist ein wertvolles Werkzeug, sollte jedoch nicht das einzige sein. Zur ordnungsgemäßen Durchführung eines Refactorings benötige ich eine zuverlässige Testsuite, um die von mir unweigerlich begangenen Fehler zu finden. Selbst wenn ich automatisierte Refactoring-Tools einsetze, müssen viele meiner Refactorings durch eine Testsuite überprüft werden.

Ich halte das nicht für einen Nachteil. Auch ohne Refactoring erhöht das Schreiben guter Tests meine Effektivität als Programmierer. Das war eine Überraschung für mich und erscheint den meisten Programmierern unlogisch – deshalb lohnt es sich, zu erklären, warum dem so ist.

4.1 Der Nutzen selbsttestenden Codes

Wenn Sie sich ansehen, wie die meisten Programmierer ihre Zeit verbringen, werden Sie feststellen, dass das Schreiben von Code nur einen Bruchteil der Zeit in Anspruch nimmt. Darüber nachzudenken, was das Programm leisten soll, kostet Zeit. Das Design zu

entwickeln, kostet ebenfalls Zeit. Den Löwenanteil der Zeit nimmt jedoch das Debugging in Anspruch. Ich bin mir sicher, dass alle Leser schon einmal Überstunden mit dem Debugging verbracht haben – oft bis spät in die Nacht. Jeder Programmierer kann Ihnen eine Geschichte von einem Bug erzählen, bei dem es einen ganzen Tag (oder länger) gedauert hat, ihn zu finden. Das Beheben des Bugs geht für gewöhnlich ziemlich schnell, aber ihn zu finden ist ein Albtraum. Wenn Sie den Bug dann endlich behoben haben, gibt es immer die Möglichkeit, dass ein weiterer auftaucht, den Sie aber erst viel später bemerken. Und wieder brauchen Sie eine kleine Ewigkeit, um ihn zu finden.

Ich kam zum ersten Mal mit selbsttestendem Code in Berührung, als ich 1992 einen Vortrag auf der OOPSLA-Konferenz besuchte. Irgendjemand (ich glaube, es war »Bedarra« Dave Thomas) sagte unverblümt: »Klassen sollten ihre eigenen Tests enthalten.« Ich fasste also den Entschluss, zusammen mit dem Produktionscode Tests in die Codebasis zu integrieren. Ich entwickelte gemäß einem inkrementellen Vorgehensmodell und versuchte, nach Abschluss jeder Iteration Tests hinzuzufügen. Das Projekt, an dem ich damals arbeitete, war ziemlich klein, deshalb konnten wir ungefähr wöchentlich eine Iteration veröffentlichen. Die Ausführung der Tests war ziemlich unkompliziert. Es war zwar einfach, aber auch ziemlich langweilig, weil jeder Test Ausgaben auf der Konsole erzeugte, die ich überprüfen musste. Nun bin ich ein

ziemlich fauler Mensch und durchaus bereit, mich ins Zeug zu legen, um mir weitere Arbeit zu ersparen. Schon bald fiel mir auf, dass ich gar nicht auf den Bildschirm sehen musste, um zu überprüfen, ob bestimmte Informationen über das Modell angezeigt wurden – das könnte auch der Computer erledigen. Ich musste lediglich die zu erwartende Ausgabe im Testcode hinterlegen und einen Vergleich durchführen. Jetzt konnte ich die Tests ausführen, die ein schlichtes »OK« auf dem Bildschirm ausgaben, wenn alles in Ordnung war. Die Software war damit selbsttestend.

Tipp

Stellen Sie sicher, dass alle Tests vollständig automatisiert ablaufen und dass sie ihre eigenen Ergebnisse überprüfen.

Nun war die Ausführung der Tests ganz einfach – genauso einfach wie das Kompilieren. Ich führte also jedes Mal, wenn ich kompilierte, Tests durch. Schon bald bemerkte ich, dass meine Produktivität stark angestiegen war, weil ich weniger Zeit mit dem Debugging verbrachte. Wenn ich versehentlich einen Bug verursachte, der durch einen früheren Test abgefangen wurde, würde er sofort auftauchen, wenn ich den Test erneut ausführte. Der Test wurde vorher bestanden,

deshalb war klar, dass der Bug in den Änderungen stecken musste, die ich seit dem letzten Testdurchlauf vorgenommen hatte. Ich habe die Tests sehr häufig ausgeführt – und das bedeutete, dass seit dem letzten Test immer jeweils nur ein paar Minuten vergangen waren. Ich wusste also, dass der Grund für den Bug der Code war, den ich gerade geschrieben hatte. Da es sich um einige wenige Codezeilen handelte, die ich noch frisch in Erinnerung hatte, war der Bug leicht zu finden. Bugs, nach denen ich früher stundenlang gesuchte hatte, konnte ich in nur ein paar Minuten beheben. Meine Software war nicht nur selbsttestend, dank der häufigen Ausführung der Tests verfügte ich auch über einen leistungsfähigen Bug-Detektor.

Nachdem ich all das bemerkt hatte, ging ich bei den Tests etwas aggressiver vor. Anstatt das Ende einer Iteration abzuwarten, fügte ich die Tests unmittelbar nach dem Schreiben einer Funktion hinzu. Ich ergänzte täglich ein paar neue Features und die dazugehörigen Tests. Dabei habe ich fast nie mehr als ein paar Minuten damit verbracht, einen Regressionsbug aufzuspüren.

Tipp

Eine Testsuite ist ein leistungsfähiger Bug-Detektor, der die erforderliche Zeit zum Aufspüren von Bugs drastisch verkürzt.

Seit meinen Experimenten wurde eine Vielzahl von Hilfsmitteln zum Schreiben und Organisieren dieser Tests entwickelt. Während des Flugs von der Schweiz nach Atlanta zur OOPSLA-Konferenz 1997 taten sich Kent Beck und Erich Gamma zusammen, um Kents Framework für Unittests von Smalltalk nach Java zu portieren. Das Ergebnis namens JUnit beeinflusste das Testen von Programmen enorm und inspirierte eine Vielzahl ähnlicher Werkzeuge in unterschiedlichsten Programmiersprachen (https://www.martinfowler.com/bliki/Xunit.html).

Es ist zugegebenermaßen nicht ganz einfach, andere davon zu überzeugen, den gleichen Weg einzuschlagen. Das Implementieren der Tests bedeutet, eine Menge zusätzlichen Code schreiben zu müssen. Insofern Sie es noch nicht selbst erlebt haben, wie sehr die Programmierung dadurch beschleunigt wird, scheint das Selbsttesten jedoch keinen Sinn zu ergeben. Die Tatsache, dass viele Programmierer nie gelernt haben, Tests zu schreiben oder auch nur über Tests nachzudenken, ist der Sache auch nicht gerade zuträglich. Manuell ausgeführte Tests sind so langweilig, dass sich einem der Magen umdreht. Das Schreiben automatisierter Tests hingegen kann durchaus Spaß machen.

Einer der sinnvollsten Zeitpunkte, um Tests zu schreiben, ist tatsächlich der, bevor ich mit dem

eigentlichen Programmieren beginne. Wenn ich ein Feature hinzufügen muss, schreibe ich als Erstes den Test. Das klingt, als ob das Pferd von hinten aufgezäumt würde, aber das stimmt nicht. Beim Schreiben des Tests frage ich mich, welche Aufgaben erledigt werden müssen, um die Funktionalität hinzuzufügen. Zudem konzentriere ich mich beim Schreiben des Tests nicht auf die Implementierung, sondern auf die Schnittstelle (das ist immer sinnvoll). Darüber hinaus weiß ich genau, wann die Implementierung erledigt ist – nämlich wenn der Test funktioniert.

Kent Beck hat diese Gewohnheit, zuerst die Tests zu schreiben, in eine Form gegossen, die als Testgetriebene Entwicklung (engl. *Test-Driven Development*, TDD) bezeichnet wird. Der TDD-Ansatz beruht auf kurzen Zyklen, die aus dem Schreiben eines (fehlschlagenden) Tests, dem Schreiben des Codes, der den Test besteht, und einem Refactoring bestehen. Durch das Refactoring wird gewährleistet, dass die Qualität des Ergebnisses so gut wie möglich ist. Dieser Testen-Schreiben-Refactoring-Zyklus sollte mehrmals pro Stunde durchlaufen werden und kann eine sehr produktive und beruhigende Methode für das Schreiben von Code sein. Ich werde an dieser Stelle nicht weiter auf diesen Ansatz eingehen, aber ich verwende ihn selbst und kann ihn nur wärmstens empfehlen.

Aber genug der Polemik. Ich bin zwar der Ansicht, dass alle Programmierer davon profitieren würden,

selbsttestenden Code zu schreiben, das ist jedoch nicht Gegenstand dieses Buchs. In diesem Buch geht es um Refactoring. Refactoring erfordert Tests. Wenn Sie ein Refactoring vornehmen möchten, müssen Sie Tests schreiben. Dieses Kapitel weist Ihnen den Weg und zeigt, wie das in JavaScript erfolgt. In diesem Buch geht es auch nicht um Tests, deshalb werde ich diesbezüglich nicht allzu sehr ins Detail gehen. Ich habe allerdings festgestellt, dass bei der Verwendung von Tests durch sehr wenig Arbeit erstaunlich große Vorteile erzielt werden können.

Wie bei allem anderen in diesem Buch beschreibe ich den Ansatz des Testens anhand von Beispielen. Wenn ich Code entwickle, schreibe ich währenddessen Tests. Manchmal muss ich jedoch Refactorings vornehmen, ohne dass für den betreffenden Code Tests existieren – in diesem Fall muss ich Tests ergänzen, bevor ich mit dem Refactoring beginnen kann.

4.2 Beispielcode zum Testen

Im Folgenden finden Sie Code, den Sie sich ansehen und testen können. Der Code gehört zu einer einfachen Software, die es dem Anwender ermöglicht, einen Produktionsplan zu untersuchen und zu bearbeiten. Die (grob gestaltete) Benutzeroberfläche sieht folgendermaßen aus:

Region: **Asien**

Bedarf: 30 Preis: 20

3 Produzenten:

Byzantium:	Kosten: 10	Produktion: 9	Umsatz: 90
Attalia:	Kosten: 12	Produktion: 10	Umsatz: 120
Sinope:	Kosten: 10	Produktion: 6	Umsatz: 60

Fehlbestand: **5** Gewinn: **230**

Der Produktionsplan enthält die Nachfrage und den Preis der verschiedenen Regionen. Zu jeder Region gehören verschiedene Produzenten, die jeweils eine gewisse Anzahl von Einheiten zu einem bestimmten Preis liefern können. Die Benutzeroberfläche zeigt zudem an, welchen Umsatz ein Produzent erzielt, wenn er seine gesamte Produktion verkauft. Im unteren Bereich werden der Fehlbestand der Produktion (der Bedarf abzüglich der Gesamtproduktion) und der Gewinn für diesen Produktionsplan angezeigt. Die Benutzeroberfläche ermöglicht dem Benutzer, den Bedarf, den Preis sowie die Produktion und die Kosten der jeweiligen Produzenten zu ändern, um die Auswirkungen auf den Fehlbestand und den Gewinn zu ermitteln. Wann immer ein Benutzer irgendeine der angezeigten Zahlen ändert, werden die anderen Werte sofort aktualisiert.

Ich zeige hier die Benutzeroberfläche, damit Sie einen Eindruck davon bekommen, wie die Software verwendet wird. Ich werde mich aber ausschließlich auf die Geschäftslogik konzentrieren, also auf die Klassen zur Berechnung des Gewinns und des Fehlbestands, nicht auf den Code zur HTML-Erzeugung und zum Zugriff auf die Inhalte der angezeigten Felder. Dieses Kapitel ist lediglich eine Einführung in die Welt des selbsttestenden Codes, deshalb erscheint es mir sinnvoll, mit dem einfachsten Fall anzufangen – und das ist eben Code, der die Benutzeroberfläche, das Speichern der Werte oder Interaktionen mit externen Diensten außer Acht lässt. Diese Trennung ist in jedem Fall eine gute Idee: Sobald diese Art der Geschäftslogik komplizierter wird, würde ich sie ohnehin von den Mechanismen zur Steuerung der Benutzeroberfläche trennen, damit ich sie leichter nachvollziehen und testen kann.

Der Code der Geschäftslogik besteht aus zwei Klassen: Die eine repräsentiert einen einzelnen Produzenten (`Producer`), die andere eine Region (`Province`). Der Konstruktor der Klasse `Province` nimmt ein JavaScript-Objekt entgegen, das beispielsweise in Form eines JSON-Dokuments bereitgestellt werden könnte.

Hier sehen Sie den Code, der aus den JSON-Daten die Regionen einliest:

```
class Province...
```

```
constructor(doc) {
  this._name = doc.name;
  this._producers = [];
  this._totalProduction = 0;
  this._demand = doc.demand;
  this._price = doc.price;
  doc.producers.forEach(d =>
                    this.addProducer(new Producer(this,
d)));
}
addProducer(arg) {
  this._producers.push(arg);
  this._totalProduction += arg.production;
}
```

Diese Funktion liefert die dazugehörenden JSON-Daten. Zu Testzwecken kann ich anhand des Ergebnisses dieser Funktion eine Region erstellen.

```
Oberste Ebene...
function sampleProvinceData() {
  return {
    name: "Asia",
    producers: [
      {name: "Byzantium", cost: 10, production: 9},
      {name: "Attalia",   cost: 12, production: 10},
      {name: "Sinope",    cost: 10, production: 6},
    ],
    demand: 30,
    price: 20
  };
}
```

Die Klasse besitzt Zugriffsfunktionen für die

verschiedenen Datenwerte:

```
class Province...
get name() {return this._name;}
get producers() {return this._producers.slice();}
get totalProduction() {return this._totalProduction;}
set totalProduction(arg) {this._totalProduction =
arg;}
get demand() {return this._demand;}
set demand(arg) {this._demand = parseInt(arg);}
get price() {return this._price;}
set price(arg) {this._price = parseInt(arg);}
```

Die Setter werden mit Strings der Benutzeroberfläche aufgerufen. Diese enthalten Zahlenwerte, die ich parsen muss, um Berechnungen mit ihnen ausführen zu können.

Die Klasse `Producer` ist im Wesentlichen ein einfacher Datencontainer:

```
class Producer...
constructor(aProvince, data) { this._province =
aProvince;
  this._cost = data.cost;
  this._name = data.name;
  this._production = data.production || 0;
}
get name() {return this._name;}
get cost() {return this._cost;}
set cost(arg) {this._cost = parseInt(arg);}

get production() {return this._production;}
```

```
set production(amountStr) {
  const amount = parseInt(amountStr);
  const newProduction = Number.isNaN(amount) ? 0 :
amount;
  this._province.totalProduction +=
                          newProduction -
this._production;
  this._production = newProduction;
}
```

Die Art und Weise, wie `set production` die hergeleiteten Daten für die Region aktualisiert, gefällt mir überhaupt nicht. Immer wenn ich so etwas sehe, möchte ich ein Refactoring vornehmen. Ich muss jedoch zunächst einmal Tests schreiben, bevor ich ein Refactoring in Angriff nehmen kann.

Die Berechnung des Fehlbestands ist einfach:

```
class Province...
get shortfall() {
  return this._demand - this.totalProduction;
}
```

Die Berechnung des Gewinns gestaltet sich ein wenig komplizierter:

```
class Province...
get profit() {
  return this.demandValue - this.demandCost;
}
```

```
get demandCost() {
  let remainingDemand = this.demand;
  let result = 0;
  this.producers
    .sort((a,b) => a.cost - b.cost)
    .forEach(p => {
      const contribution =
                    Math.min(remainingDemand,
p.production);
        remainingDemand -= contribution;
        result += contribution * p.cost;
    });
  return result;
}
get demandValue() {
  return this.satisfiedDemand * this.price;
}
get satisfiedDemand() {
  return Math.min(this._demand, this.totalProduction);
}
```

4.3 Ein erster Test

Zum Testen dieses Codes benötige ich eine Art von Test-Framework. Davon gibt es eine ganze Reihe, gerade auch für JavaScript. Ich werde Mocha (`https://mochajs.org`) verwenden, das recht gebräuchlich ist und ein gutes Ansehen genießt. Auf die Verwendungsweise des Frameworks werde ich nicht näher eingehen, sondern lediglich einige Beispieltests zeigen. Sie sollten vergleichbare Tests mit einem anderen Framework relativ leicht nachvollziehen können.

Hier ist ein einfacher Test für die Berechnung des

Fehlbestands `shortfall`:

```
describe('province', function() {
  it('shortfall', function() {
    const asia = new Province(sampleProvinceData());
    assert.equal(asia.shortfall, 5);
  });
});
```

Das Mocha-Framework teilt den Testcode in Blöcke auf, die jeweils eine Testsuite darstellen. Jeder Test tritt in Form eines `it`-Blocks in Erscheinung. Bei diesem einfachen Beispiel besteht der Test aus zwei Schritten. Der erste Schritt legt ein Test-Fixture fest – Daten und Objekte, die für den Test benötigt werden, in diesem Fall ein eingelesenes `province`-Objekt. Die nachfolgende Zeile überprüft bestimmte Eigenschaften des Fixtures – in diesem Fall, dass der Betrag des Fehlbestands (`shortfall`) dem Wert entspricht, der in Anbetracht der Ausgangsdaten zu erwarten ist.

Die Erläuterungstexte in den `describe`- und `it`-Blöcken werden von Entwicklern für unterschiedliche Zwecke genutzt. Manche schreiben einen Satz, der erklärt, was genau getestet wird, andere ziehen es vor, ihn leer zu lassen, weil sie der Meinung sind, dass der beschreibende Text, ebenso wie ein Kommentar, nur das Gleiche beinhaltet wie der eigentliche Code. Ich gebe hier gerade so viele Informationen an, dass ich bei Fehlschlägen erkennen kann, um welchen Test es sich

handelt.

Wenn ich diesen Test in der NodeJS-Konsole ausführe, erhalte ich folgende Ausgabe:

```
,,,,,,,,,,,,,
1 passing (61ms)
```

Beachten Sie, wie schlicht die Ausgabe ist – lediglich die Anzahl der ausgeführten und der bestandenen Tests.

Tipp

Vergewissern Sie sich, dass ein Test auch tatsächlich fehlschlägt, wenn er es sollte.

Wenn ich Tests für bereits existierenden Code wie diesen schreibe, ist es erfreulich, feststellen zu können, dass alles in Ordnung ist – ich bin allerdings von Natur aus skeptisch. Insbesondere wenn ich viele Tests ausführe, werde ich etwas nervös, weil möglicherweise ein Test den Code nicht so ausführt, wie ich mir das vorgestellt habe, und ein Bug deswegen unentdeckt bleibt. Deshalb achte ich beim Schreiben von Tests akribisch darauf, dass jeder Test wenigstens einmal fehlschlägt. Zu diesem Zweck füge ich dem Code meistens wissentlich einen vorübergehenden Fehler hinzu, so wie hier:

```
class Province...
get shortfall() {
  return this._demand - this.totalProduction * 2;
}
```

Die Konsole zeigt jetzt Folgendes an:

```
!

0 passing (72ms)
1 failing

1) province shortfall:
   AssertionError: expected -20 to equal 5
   at Context.<anonymous> (src/tester.js:10:12)
```

Das Framework zeigt an, welcher Test fehlgeschlagen ist, und liefert Informationen über die Art des Fehlers – in diesem Fall, welcher Wert erwartet wurde und welcher Wert tatsächlich auftrat. Ich bemerke also sofort, dass etwas nicht funktioniert, und kann auf einen Blick erkennen, welcher Test warum fehlgeschlagen ist (und erhalte hier die Bestätigung, dass sich der Fehler dort befindet, wo ich ihn eingebaut habe).

Tipp: Führen Sie die Tests regelmäßig aus

Führen Sie die Tests für den Code, an dem Sie gerade arbeiten, mindestens alle paar Minuten aus. Führen

Sie alle Tests mindestens einmal täglich aus.

In der Praxis können mehrere Tausend Tests zum Einsatz kommen. Ein gutes Test-Framework ermöglicht es mir, sie schnell und einfach auszuführen und sofort zu erkennen, ob irgendwelche Tests fehlgeschlagen sind. Dieses unmittelbare Feedback ist für selbsttestenden Code von entscheidender Bedeutung. Ich führe sehr häufig Tests durch – zur Begutachtung der Fortschritte mit neuem Code oder zwecks Fehlersuche beim Refactoring.

Das Mocha-Framework kann verschiedene sogenannte Assertion-Bibliotheken verwenden, um das Fixture eines Tests zu überprüfen. Für JavaScript sind Myriaden solcher Bibliotheken verfügbar, von denen viele noch aktuell sein dürften, wenn Sie dieses Buch lesen. Ich verwende derzeit Chai (`https://www.chaijs.com`). Diese Bibliothek ermöglicht es mir, für die Überprüfungen entweder einen »assert«- oder einen »expect«-Stil zu verwenden:

```
describe('province', function() {
  it('shortfall', function() {
    const asia = new Province(sampleProvinceData());
    assert.equal(asia.shortfall, 5);
  });
});

describe('province', function() {
```

```
  it('shortfall', function() {
    const asia = new Province(sampleProvinceData());
    expect(asia.shortfall).equal(5);
  });
});
```

Für gewöhnlich bevorzuge ich den assert-Stil, aber derzeit verwende ich in JavaScript meistens den expect-Stil.

Die verschiedenen Entwicklungsumgebungen bieten unterschiedliche Möglichkeiten, die Tests auszuführen. Bei der Programmierung in Java verwende ich eine IDE, die eine grafische Oberfläche zum Ausführen der Tests bereitstellt. Der Fortschrittsbalken wird grün dargestellt, solange alle Tests bestanden werden. Er wird rot, falls einer der Tests fehlschlägt. Meine Kollegen verwenden die Ausdrücke »grüner Balken« und »roter Balken«, um den Status von Tests zu beschreiben. Wenn ich sage: »Nehmen Sie kein Refactoring vor, wenn ein roter Fortschrittsbalken angezeigt wird,« heißt das, dass Sie kein Refactoring durchführen sollten, wenn einer der Tests Ihrer Testsuite nicht bestanden wird. Und wenn ich sage: »Zurück zu grün«, meine ich damit, dass Sie kürzlich durchgeführte Änderungen revidieren sollten, um zum letzten Zustand zurückzukehren, in dem das System alle Tests bestanden hat (normalerweise durch Wiederherstellen eines früheren Standes mit Hilfe Ihrer Versionsverwaltung).

Grafische Oberflächen sind nett, aber nicht unbedingt notwendig. Ich führe meine Tests meistens durch einen einzigen Tastendruck in Emacs aus und beobachte die Textausgaben während des Kompilierens. Der springende Punkt ist, dass ich auf einen Blick erkennen kann, ob alle Tests bestanden werden.

4.4 Hinzufügen eines weiteren Tests

Anschließend fahre ich damit fort, weitere Tests hinzuzufügen. Dabei gehe ich wie folgt vor: Ich betrachte dabei alle Aufgaben, die von der Klasse erledigt werden sollen, und überprüfe jede einzelne von ihnen auf Bedingungen, die zum Versagen der Klasse führen könnten. Das ist nicht dasselbe wie das Testen aller öffentlichen Methoden, für das manche Programmierer plädieren. Tests dienen dazu, Risiken einzuschränken; ich versuche, Bugs zu finden, jetzt sofort und zukünftig. Deshalb teste ich auch keine Zugriffsfunktionen, die nur lesend und schreibend auf Felder zugreifen: Sie sind so einfach, dass ich höchstwahrscheinlich keine Bugs finden werde.

Das ist wichtig, weil der Versuch, zu viele Tests zu schreiben, meist dazu führt, dass nicht genügend geschrieben werden. Das Testen bietet mir viele Vorteile, auch wenn ich nur wenig teste. Ich konzentriere mich auf die Bereiche, bei denen ich mir die größten Sorgen mache, dass etwas schiefgehen könnte. Auf diese Weise

ziehe ich den größten Nutzen aus meinen Tests.

Tipp

Es ist besser, unvollständige Tests zu schreiben und auszuführen, als vollständige Tests gar nicht auszuführen.

Ich fahre mit der zweiten entscheidenden Aufgabe dieses Codes fort – der Gewinnermittlung. Auf Basis meines anfänglichen Fixtures führe ich einen Test für den Gewinn durch.

```
describe('province', function() {
  it('shortfall', function() {
    const asia = new Province(sampleProvinceData());
    assert.equal(asia.shortfall, 5);
  });
  it('profit', function() {
    const asia = new Province(sampleProvinceData());
    expect(asia.profit).equal(230);
  });
});
```

Das Listing zeigt bereits das Endergebnis. Ich habe für den erwarteten Wert zunächst einen Platzhalter verwendet und ihn anschließend durch die Ausgabe des Programms (230) ersetzt. Ich hätte ihn auch selbst berechnen können, aber da der Code vermeintlich

korrekt funktioniert, verlasse ich mich vorerst darauf. Wenn ich den neuen Test zum Laufen gebracht habe, bringe ich ihn auch gleich zum Scheitern, indem ich der Berechnung des Gewinns ein fehlerhaftes `* 2` hinzufüge. Ich vergewissere mich, dass der Test erwartungsgemäß fehlschlägt, und entferne den von mir eingebauten Fehler anschließend wieder. Auf diese Weise – Platzhalter für den erwarteten Wert verwenden, diesen durch den vom Code gelieferten Wert ersetzen, Fehler einbauen und wieder entfernen – gehe ich beim Hinzufügen von Tests zu vorhandenem Code häufig vor.

Die Tests enthalten einigen redundanten Code – beide verwenden den gleichen Code zum Einrichten des Fixtures. Redundanzen gegenüber bin ich nicht nur bei regulärem Code argwöhnisch, sondern auch bei Testcode, deshalb entferne ich sie, indem ich ein Refactoring vornehme. Eine Möglichkeit bestünde darin, die Konstante in den äußeren Geltungsbereich zu verschieben.

```
describe('province', function() {
  const asia = new Province(sampleProvinceData());
  // Tun Sie das nicht!
  it('shortfall', function() {
    expect(asia.shortfall).equal(5);
  });
  it('profit', function() {
    expect(asia.profit).equal(230);
  });
});
```

Dem Kommentar können Sie entnehmen, dass ich niemals so vorgehen würde. Diese Variante funktioniert zwar kurzfristig, kann aber zu einem der lästigsten Bugs beim Testen führen: gemeinsam genutzte Fixtures (shared fixtures), durch die sich verschiedene Tests gegenseitig beeinflussen. Das Schlüsselwort `const` besagt in JavaScript lediglich, dass der Verweis auf `asia` konstant ist, nicht aber der Inhalt dieses Objekts. Falls ein zukünftiger Test dieses gemeinsam genutzte Objekt verändert, werden Tests sporadisch fehlschlagen, weil sie sich durch das gemeinsam genutzte Test-Fixture gegenseitig beeinflussen. Die Tests liefern unterschiedliche Ergebnisse, die davon abhängen, in welcher Reihenfolge die Tests ausgeführt werden. Dabei handelt es sich um ein nicht-deterministisches Verhalten der Tests, das bestenfalls zu langwierigem und schwierigem Debugging führt und schlimmstenfalls das Vertrauen in die Tests zerstört. Ich gehe stattdessen folgendermaßen vor:

```
describe('province', function() {
  let asia;
  beforeEach(function() {
    asia = new Province(sampleProvinceData());
  });
  it('shortfall', function() {
    expect(asia.shortfall).equal(5);
  });
  it('profit', function() {
    expect(asia.profit).equal(230);
  });
});
```

Die `beforeEach`-Anweisung wird vor jedem einzelnen Test ausgeführt, wodurch die Variable `asia` jedes Mal neu initialisiert wird. Auf diese Weise wird vor der Ausführung jedes Tests ein neues Fixture (fresh fixture) eingerichtet, das die Tests voneinander isoliert und somit Probleme durch nicht-deterministisches Verhalten verhindert.

Wenn ich zu dieser Vorgehensweise rate, haben manche Leute Bedenken, dass die wiederholte Initialisierung des Fixtures die Ausführung der Tests verlangsamt. In den meisten Fällen ist die Verlangsamung unmerklich. Sollten sich tatsächlich Probleme ergeben, würde ich ein gemeinsam genutztes Fixture in Betracht ziehen, müsste dann aber peinlich genau darauf achten, dass keiner der Tests die Werte ändert. Ich könnte auch dann ein gemeinsam genutztes Fixture verwenden, wenn ich mir sicher bin, dass es wirklich unveränderlich ist. Spontan neige ich allerdings dazu, dieses Test-Fixture jeweils neu zu initialisieren, weil mir der Aufwand beim Debugging von Fehlern im Zusammenhang mit gemeinsam genutzten Fixtures in der Vergangenheit zu oft Probleme bereitet hat.

Wenn ich den vorbereitenden Code in `beforeEach` bei jedem Test ausführe, warum sollte er dann nicht in den einzelnen `it`-Blöcken verbleiben? Ich möchte, dass alle meine Tests ein einheitliches Fixture verwenden, sodass

mir dieses Fixture vertraut wird und damit ich die verschiedenen Merkmale erkennen kann, die anhand dessen überprüft werden. Das Vorhandensein des `beforeEach`-Blocks signalisiert dem Betrachter, dass ich ein Standard-Fixture verwende. Sie können sich dann alle Tests innerhalb des Gültigkeitsbereichs des `describe`-Blocks ansehen und sind sich darüber im Klaren, dass sie allesamt die gleichen Ausgangsdaten verwenden.

4.5 Ändern des Test-Fixtures

Bisher zeigen die von mir geschriebenen Tests, wie ich die Eigenschaften des Test-Fixtures überprüfe, nachdem ich sie eingelesen habe. Bei der Verwendung des Programms wird dieses Fixture jedoch regelmäßig aktualisiert, wenn die Anwender die anderen Werte ändern.

Bei den meisten Aktualisierungen werden einfache Setter aufgerufen, die ich normalerweise nicht teste, weil es sehr unwahrscheinlich ist, dass sie für einen Bug verantwortlich sind. Das Verhalten des `production`-Setters der `Producer`-Klasse ist allerdings etwas komplizierter, sodass ich einen Test für angebracht halte.

```
describe('province'...
it('change production', function() {
  asia.producers[0].production = 20;
  expect(asia.shortfall).equal(-6);
```

```
  expect(asia.profit).equal(292);
});
```

Hierbei handelt es sich um ein gängiges Muster. Ich verwende das initiale Standard-Fixture, das im `beforeEach`-Block eingerichtet wurde (Setup), bereite das Fixture für den Test vor (Exercise), und überprüfe anschließend, ob das Resultat meinen Erwartungen entspricht (Verify). Diese drei Phasen werden auch als *Setup-Exercise-Verify, Given-When-Then* oder *Arrange-Act-Assert* beschrieben. Manchmal sind alle drei Schritte Bestandteil des Tests, in anderen Fällen wird die gemeinsame erste Phase in Standard-Routinen zur Initialisierung wie `beforeEach` verlagert.

(Es gibt eine weitere implizite Phase, die meist nicht erwähnt wird: das Teardown. Dabei wird das Test-Fixture nach jedem einzelnen Test entfernt, damit sich verschiedene Tests nicht gegenseitig beeinflussen. Wenn ich für das Einrichten `beforeEach` verwende, gestatte ich dem Test-Framework implizit, mein Fixture zwischen verschiedenen Tests zu entfernen, sodass ich davon ausgehen kann, dass die Teardown-Phase stattfindet. Die meisten Autoren, die über Tests schreiben, verschweigen diese Phase – und das zu Recht, denn meistens lassen wir sie außer Acht. Hin und wieder kann es jedoch von Bedeutung sein, dass es eine explizite Teardown-Operation gibt, wenn wir ein Fixture haben, das von verschiedenen Tests gemeinsam genutzt werden muss,

weil eine Neuinitialisierung zu langsam ist.)

In diesem Test verifiziere ich zwei unterschiedliche Merkmale in einem einzigen it-Block. Als allgemeine Regel ist es ratsam, nur eine einzige verify-Anweisung in jeder it-Klausel zu verwenden. Das ist damit zu begründen, dass der Test beim ersten Verifizierungsfehler abbricht – wodurch nützliche Informationen verborgen bleiben können und es erschwert wird, herauszufinden, warum ein Test fehlschlägt. In diesem Fall habe ich das Gefühl, dass die beiden Merkmale eng genug miteinander verbunden sind, um sich im selben Test zu befinden. Sollte ich sie in getrennte it-Blöcke verschieben wollen, kann ich das auch später noch tun.

4.6 Austesten der Grenzen

Bislang haben sich meine Tests auf die reguläre Nutzung bezogen, oft auch als »Happy Path«-Bedingungen bezeichnet: Alles funktioniert und alle Objekte werden den Erwartungen entsprechend genutzt. Es ist jedoch auch sinnvoll, Tests für die Grenzfälle dieser Bedingungen auszuführen – um zu beobachten, was geschieht, wenn etwas schiefgeht.

Sobald ich eine Sammlung von Objekten vor mir habe, wie in diesem Beispiel `producers`, sehe ich mir an, was passiert, wenn diese Datenstruktur leer ist.

```
describe('no producers', function() {
  let noProducers;
  beforeEach(function() {
    const data = {
      name: "No producers",
      producers: [],
      demand: 30,
      price: 20
  };
  noProducers = new Province(data);
  });
  it('shortfall', function() {
    expect(noProducers.shortfall).equal(30);
  });
  it('profit', function() {
    expect(noProducers.profit).equal(0);
});
```

Zum Überprüfen von Zahlen sind Nullen gut geeignet:

```
describe(’province’...
it('zero demand', function() {
  asia.demand = 0;
  expect(asia.shortfall).equal(-25);
  expect(asia.profit).equal(0);
});
```

Ebenso wie negative Werte:

```
describe(’province’...
it('negative demand', function() {
  asia.demand = -1;
  expect(asia.shortfall).equal(-26);
```

```
  expect(asia.profit).equal(-10);
});
```

An dieser Stelle frage ich mich, ob ein negativer Bedarf (`demand`), der zu einem negativen Gewinn (`profit`) führt, überhaupt einen Sinn ergibt. Sollte der minimale Bedarf nicht null sein? Sollte der Setter in diesem Fall auf ein negatives Argument vielleicht anders reagieren – beispielsweise eine Fehlermeldung ausgeben oder den Wert auf null setzen? Das sind berechtigte Fragen, die man sich als Entwickler stellen sollte. Das Schreiben von Tests wie diesem hilft mir, darüber nachzudenken, wie der Code auf solche Grenzfälle reagieren sollte.

Tipp

Überlegen Sie sich die Bedingungen für Grenzfälle, bei denen etwas schiefgehen könnte, und konzentrieren Sie sich beim Testen auf diese Grenzfälle.

Die Setter verwenden Strings aus den Eingabefeldern in der grafischen Benutzeroberfläche, die als Eingaben nur Zahlen akzeptieren – sie können aber auch leer sein, deshalb benötige ich Tests, die gewährleisten, dass der Code auch auf leere Eingaben sinnvoll reagiert.

```
describe('province'...
```

```
it('empty string demand', function() {
  asia.demand = "";
  expect(asia.shortfall).NaN;
  expect(asia.profit).NaN;
});
```

Beachten Sie, dass ich mich an dieser Stelle meinem Code gegenüber wie ein Feind verhalte. Ich denke darüber nach, wie ich ihn aushebeln könnte. Ich empfinde diese Geisteshaltung nicht nur als produktiv, sondern auch als unterhaltsam. Sie verwöhnt den boshaften Teil meiner Psyche.

Folgendes ist interessant:

```
describe('string for producers', function() {
  it('', function() {
    const data = {
      name: "String producers",
      producers: "",
      demand: 30,
      price: 20
    };
    const prov = new Province(data);
    expect(prov.shortfall).equal(0);
  });
```

Der Code erzeugt nicht einfach nur eine simple Fehlermeldung, die besagt, dass der Fehlbestand (`shortfall`) nicht null ist. Stattdessen wird auf der Konsole Folgendes ausgegeben:

```
''''''''!
9 passing (74ms)
1 failing

1) string for producers :
   TypeError: doc.producers.forEach is not a function
    at new Province (src/main.js:22:19)
    at Context.<anonymous> (src/tester.js:86:18)
```

Mocha behandelt das als Fehlschlagen des Tests – allerdings unterscheiden viele Test-Frameworks zwischen dieser Situation, die als Error bezeichnet wird, und dem regulären Fehlschlagen eines Tests. Das Fehlschlagen bezieht sich auf die Überprüfung von Annahmen, bei denen der tatsächliche Wert außerhalb der vorgegebenen Grenzen liegt. Hier handelt es sich jedoch um einen anderen Fall – einen Ausnahmefehler (Exception), der während einer früheren Phase ausgelöst wurde (in diesem Fall in der Setup-Phase). Offenbar handelt es sich um eine Exception, die von den Autoren des Codes nicht erwartet wurde, daher erhalten wir eine Fehlermeldung, die JavaScript-Programmierern nur allzu vertraut ist (»… ist keine Funktion«).

Wie sollte der Code in diesem Fall reagieren? Eine Möglichkeit wäre es, die Behandlung von Fehlern zu verbessern – entweder durch eine aussagekräftigere Fehlermeldung, oder indem `producers` ein leeres Array zugewiesen wird (vielleicht zusammen mit einem Log-Eintrag). Es mag aber auch triftige Gründe dafür geben,

alles beim Alten zu belassen. Vielleicht entstammt das Eingabeobjekt einer vertrauenswürdigen Quelle – wie etwa einer anderen Komponente derselben Codebasis. Wenn die Module derselben Codebasis viele Gültigkeitsüberprüfungen vornehmen, kann das zu überflüssigen doppelten Überprüfungen führen, die mehr Schaden verursachen als Nutzen erzeugen, insbesondere, wenn Dinge erneut überprüft werden, die schon andernorts überprüft wurden. Wenn das Eingabeobjekt hingegen einer externen Quelle entstammt, wie etwa Antwortdaten eines Webservices im JSON-Format, dann sind Gültigkeitsprüfungen erforderlich und diese sollten getestet werden. In jedem Fall wirft das Schreiben von Tests Fragen wie diese auf.

Wenn ich Tests wie die hier gezeigten geschrieben hätte, bevor ich ein Refactoring vornehme, würde ich sie vermutlich verwerfen. Refactoring sollte das sichtbare Verhalten nicht verändern. Ein Fehler wie dieser liegt allerdings jenseits der Grenze des Sichtbaren, deshalb muss ich mir keine Sorgen machen, wenn mein Refactoring die Verhaltensweise des Codes unter diesen Bedingungen ändert.

Wenn dieser Fehler dazu führen kann, dass sich inkorrekte Daten im Programm ausbreiten, die Fehlfunktionen verursachen, die sich nur schwer debuggen lassen, könnte ich *Assertion einführen* (Abschnitt 10.6) verwenden, um ein »schnelles Scheitern« zu erreichen. Ich füge keine Tests hinzu, um

fehlgeschlagene Assertions aufzuspüren, denn diese stellen selbst eine Art Test dar.

Tipp

Lassen Sie sich von der Befürchtung, dass durch Tests nicht sämtliche Bugs entdeckt werden, nicht davon abhalten, generell Tests zu schreiben, mit denen die meisten Bugs gefunden werden.

Wann sollte man aufhören? Ich bin mir sicher, dass Sie schon oft gehört haben, dass man durch Tests nicht beweisen kann, dass ein Programm fehlerfrei ist. Das stimmt zwar, hat aber keinen Einfluss auf die Möglichkeit, die Programmierung durch Tests zu beschleunigen. Ich habe schon so einige Vorschläge für Regeln gesehen, die gewährleisten sollen, dass Sie jede mögliche Kombination getestet haben. Sie sind durchaus einen Blick wert – aber lassen Sie sich nicht davon beeindrucken. Immer umfangreichere Tests lohnen sich zunehmend weniger, und wenn Sie versuchen, zu viele Tests zu schreiben, laufen Sie Gefahr, demotiviert zu werden und letzten Endes gar keine Tests zu schreiben. Konzentrieren Sie sich auf die risikobehafteten Bereiche. Sehen Sie sich den Code an und stellen Sie fest, wo er komplex und zunehmend unverständlich wird. Sehen Sie sich eine Funktion an und identifizieren Sie die

fehleranfälligen Bereiche. Ihre Tests werden nicht sämtliche Bugs aufdecken, aber durch das Anwenden von Refactorings werden Sie das Programm besser verstehen und deshalb mehr Bugs finden. Ich verwende bei einem Refactoring immer von Beginn an eine Testsuite, die ich im weiteren Verlauf kontinuierlich erweitere.

4.7 Und noch viel mehr

Das soll für dieses Kapitel reichen – schließlich geht es in diesem Buch um Refactoring, nicht ums Testen. Dessen ungeachtet sind Tests ein wichtiges Thema, zum einen, weil sie die Grundlage für Refactorings bilden, und zum anderen, weil Tests an sich ein nützliches Werkzeug sind. Ich habe mich zwar über den zunehmenden Gebrauch von Refactorings in der Programmierung seit der Veröffentlichung meines Buchs gefreut, aber dass sich die Haltung gegenüber Tests geändert hat, freut mich noch mehr. Die frühere Einstellung, dass eine eigene (und untergeordnete) Personengruppe für Tests verantwortlich sei, hat sich gewandelt. Tests sind inzwischen für jeden ernstzunehmenden Softwareentwickler von großer Bedeutung. Architekturen werden häufig zu Recht anhand ihrer Testbarkeit beurteilt.

Bei den hier vorgestellten Tests handelt es sich um Unittests, die auf kleine Codeabschnitte ausgelegt und

schnell ausführbar sind. Sie bilden das Fundament für selbsttestenden Code; die meisten Tests eines solchen Systems sind Unittests. Es gibt noch weitere Arten von Tests, die sich auf die Integration von Softwarekomponenten, die gleichzeitige Ausführung mehrere Softwareschichten, Performanceprobleme usw. konzentrieren. (Noch verschiedenartiger als die Arten der Tests sind nur die Debatten, die über die Klassifizierung der Tests geführt werden.)

Wie so viele Aspekte der Programmierung ist das Testen eine iterative Tätigkeit. Wenn Sie nicht sehr erfahren sind oder einfach nur Glück haben, wird Ihnen das Testen beim ersten Anlauf nicht unbedingt einfach fallen. Ich arbeite kontinuierlich an meiner Testsuite – ich verbringe damit praktisch genauso viel Zeit wie mit der Arbeit am eigentlichen Code. Wenn ich neue Features hinzufüge, muss ich natürlich neue Tests erstellen, aber auch die vorhandenen Tests berücksichtigen. Sind sie einleuchtend? Muss ich vielleicht ein Refactoring vornehmen, damit ich besser verstehen kann, welche Aufgabe sie erledigen? Setze ich die richtigen Tests ein? Sie sollten sich angewöhnen, als Reaktion auf einen Bug als Erstes einen Test zu schreiben, der den Bug klar und deutlich zum Vorschein bringt. Den Bug behebe ich erst, wenn mir ein Test zur Verfügung steht. Das Vorhandensein des Tests gewährleistet, dass sich der Bug nicht wieder einschleichen kann. Darüber hinaus schenke ich einem

Bug und dem dazugehörigen Test weitere Beachtung: Gibt er vielleicht Hinweise auf andere Lücken in der Testsuite?

Tipp

Wenn Ihnen ein Bug gemeldet wird, sollten Sie als Erstes einen Unittest implementieren, der den Bug aufdeckt.

»Wie viel Testen ist ausreichend?« ist eine häufig gestellte Frage. Eine wirklich gute Bewertungskennzahl gibt es hierfür nicht. Manche Leute plädieren dafür, die Testabdeckung (`https://martinfowler.com/bliki/TestCoverage.html`) als Kennzahl zu verwenden. Eine Analyse der Testabdeckung ist jedoch nur dazu geeignet, noch nicht getestete Bereiche des Codes zu identifizieren, nicht aber für die Beurteilung der Qualität einer Testsuite.

Die beste Einschätzung, ob der Umfang einer Testsuite ausreicht, ist rein subjektiv: Wie sicher sind Sie sich, dass einer Ihrer Tests fehlschlägt, wenn jemand einen willkürlichen Fehler im Code verursacht? Diese Fragestellung lässt sich nicht objektiv beurteilen und liefert keine berechtigte Grundlage für Vertrauen, obwohl die eigentliche Aufgabe selbsttestenden Codes ja

gerade darin besteht, dieses Vertrauen zu schaffen. Wenn ich ein Refactoring vornehme und mir ziemlich sicher bin, dass ich keinen Bug verursacht habe, weil meine Tests grün anzeigen, dann kann ich mich glücklich schätzen, ausreichend gute Tests zu haben.

Es ist durchaus möglich, zu viele Tests zu schreiben. Wenn ich mit dem Anpassen der Tests mehr Zeit verbringe als mit dem Ändern des zu testenden Codes und den Eindruck habe, dass die Tests mein Vorankommen verlangsamen, ist das ein deutliches Zeichen hierfür. Dass zu viele Tests durchgeführt werden, kommt zwar durchaus vor, spielt aber im Vergleich mit zu wenig ausgeführten Tests nur eine verschwindend kleine Rolle.

Kapitel 5

Der Katalog

Der verbleibende Teil dieses Buchs ist ein Katalog von Refactorings. Er entstand aus meinen persönlichen Notizen, die ich mir ursprünglich gemacht hatte, um mich daran zu erinnern, wie man Refactorings zuverlässig und effizient vornehmen kann. Seither habe ich den Katalog verbessert und um vieles erweitert, das ich aus der intensiven Auseinandersetzung mit bestimmten Refactoringschritten gelernt habe. Ich greife selbst darauf zurück, wenn ich ein Refactoring vornehme, das ich längere Zeit nicht durchgeführt habe.

5.1 Format der Refactorings

Zur Beschreibung der Refactorings verwende ich in diesem und anderen Kapiteln ein Standardformat. Die Refactorings bestehen aus den folgenden fünf Teilen:

- Ich beginne mit dem **Namen**. Der Name ist von Bedeutung, um ein Vokabular der Refactorings aufzubauen. Den angegebenen Namen verwende ich überall im Buch. Für viele Refactorings kursieren verschiedene Bezeichnungen, die ich aufliste, sofern sie gebräuchlich sind.
- Dem Namen folgt eine kleine **Skizze** des Refactorings. Sie soll Ihnen dabei helfen, ein gesuchtes Refactoring schneller zu finden.

- Die **Motivation** beschreibt, warum das Refactoring ausgeführt werden sollte, und nennt Umstände, unter denen das besser unterbleibt.
- Die Beschreibung des **Vorgehens** bezeichnet eine kompakte, schrittweise Anleitung, wie das Refactoring durchgeführt wird.
- Das **Beispiel** zeigt eine sehr einfache Anwendung des Refactorings, die veranschaulicht, wie es funktioniert.

Die Skizze zeigt ein Codebeispiel für die Transformation, die durch das Refactoring vorgenommen wird. Sie ist nicht dazu gedacht, zu erklären, was das Refactoring leistet, geschweige denn, wie man es durchführt. Sie soll Sie vielmehr daran erinnern, welchem Zweck das Refactoring dient, falls Sie ihm zuvor schon begegnet sind. Ist das nicht der Fall, sollten Sie am besten das Beispiel durcharbeiten, um eine bessere Vorstellung von dem Refactoring zu bekommen. Zudem finden Sie eine kleine Abbildung, die ebenfalls nicht der Erklärung dient, sondern eher eine Art visuelle Auffrischung des Gedächtnisses darstellt.

Die Beschreibung der Vorgehensweise entstammt meinen eigenen Notizen, die mich daran erinnern sollen, wie das Refactoring durchgeführt wird, wenn ich es eine Weile nicht verwendet habe. Aus diesem Grund ist sie oft ziemlich knapp gehalten und gibt für gewöhnlich keine Auskunft darüber, warum die Schritte auf die angegebene Weise ausgeführt werden. Im Beispiel liefere ich eine ausführlichere Erklärung. Die Beschreibungen der Vorgehensweise sind also kurze Notizen, auf die Sie

zurückgreifen können, wenn Ihnen ein Refactoring zwar bekannt ist, Sie aber die erforderlichen Schritte nachschlagen müssen (jedenfalls verwende ich sie auf diese Weise). Wenn Sie ein Refactoring zum ersten Mal durchführen, sollten Sie sich am besten das Beispiel durchlesen.

Ich habe die Vorgehen derart beschrieben, dass die verschiedenen Schritte eines Refactorings so klein wie möglich sind. Dabei betone ich die sichere Art und Weise, das Refactoring durchzuführen – also in sehr kleinen Schritten vorzugehen und nach jedem einzelnen Schritt zu testen. In der Praxis führe ich meistens etwas größere als die beschriebenen Mini-Schritte aus, aber sobald ich auf einen Bug stoße, nehme ich den letzten Schritt zurück und gehe in kleineren Schritten vor. Die Beschreibungen der Schritte enthalten nicht nur eine Vielzahl von Verweisen auf Sonderfälle, sie können auch als Checkliste fungieren – ich vergesse selbst häufig den einen oder anderen Schritt.

Ich gebe zwar (von wenigen Ausnahmen abgesehen) nur eine Beschreibung der Vorgehensweise an, allerdings stellt dies nicht die einzige Möglichkeit dar, das Refactoring durchzuführen. Ich habe die in diesem Buch beschriebenen Vorgehen ausgewählt, weil sie in den meisten Fällen zuverlässig funktionieren. Wenn Sie ein wenig praktische Erfahrung mit Refactorings gesammelt haben, werden Sie vermutlich Varianten einsetzen wollen, und das ist auch gut so. Denken Sie aber daran,

dass es von entscheidender Bedeutung ist, in kleinen Schritten vorzugehen – und je komplizierter die Situation ist, desto kleiner sollten die Schritte sein.

Die Beispiele sind lehrbuch-typisch oft lächerlich einfach. Mein Gedanke dabei ist es, das elementare Refactoring möglichst ohne Umschweife zu erklären, deshalb bitte ich um Ihr Verständnis dafür, dass sie so einfach sind. (Es sind jedenfalls keine Beispiele für eine sinnvolle Modellierung von Geschäftsprozessen.) Ich bin sicher, dass Sie in der Lage sein werden, sie auf Ihre vermutlich viel komplexeren Anwendungsfälle in der Praxis zu übertragen. Bei einigen sehr einfachen Refactorings ist kein Beispiel angegeben, weil meiner Ansicht nach ein Beispiel keinen nennenswerten Beitrag zum Verständnis leisten würde.

Halten Sie sich insbesondere immer vor Augen, dass die Beispiele lediglich das eine Refactoring veranschaulichen sollen, um das es gerade geht. In vielen Fällen sind im Code am Ende noch Probleme vorhanden, deren Behebung jedoch weitere Refactorings erfordern würde. In einigen Fällen kommen bestimmte Refactorings gemeinsam zum Einsatz. Hier habe ich das Beispiel des einen Refactorings im anderen übernommen. In den meisten Fällen lasse ich den Code nach Durchführung eines einzelnen Refactorings unverändert, damit die verschiedenen Refactorings in sich abgeschlossen sind, denn der Katalog soll ja in erster Linie als Nachschlagewerk dienen.

Geänderter Code ist fett gedruckt, um ihn hervorzuheben, wenn es schwierig ist, ihn im unveränderten Code zu erkennen. Nicht alle Änderungen sind fett gedruckt, denn das würde dem Sinn der Hervorhebung zuwiderlaufen.

5.2 Die Auswahl der Refactorings

Der Katalog der Refactorings ist natürlich alles andere als vollständig. Es handelt sich vielmehr um eine Sammlung derjenigen Refactorings, bei denen es am nützlichsten ist, wenn sie in schriftlicher Form vorliegen. Mit »am nützlichsten« meine ich die Refactorings, die häufig verwendet werden und bei denen es sich lohnt, einen Namen zu vergeben und sie zu beschreiben. Meiner Meinung nach lohnt es sich aus mehreren Gründen, ein Refactoring zu beschreiben: Einige der Refactorings weisen eine besonders interessante Vorgehensweise auf, deren Kenntnis die allgemeinen Refactoring-Fähigkeiten fördern. Andere wirken sich stark auf die Verbesserung des Codedesigns aus.

Vereinzelt sind Refactorings nicht aufgeführt, weil sie so einfach und unkompliziert sind, dass ich es nicht für notwendig hielt, sie aufzuschreiben. Ein Beispiel aus der ersten Ausgabe war *Anweisungen verschieben* (Abschnitt 8.6) – ein Refactoring, das ich häufig anwende, aber nicht für wert hielt, in den Katalog aufgenommen zu werden (offensichtlich habe ich in

dieser Ausgabe meine Meinung dazu geändert.) Andere Refactorings dieser Art könnten im Laufe der Zeit ebenfalls Eingang in den Katalog finden. Das hängt davon ab, wie viel Aufwand ich zukünftig in neue Refactorings stecke.

Eine weitere Kategorie bilden Refactorings, die zwar grundsätzlich existieren, aber entweder kaum verwendet werden oder eine deutliche Ähnlichkeit zu anderen Refactorings aufweisen. Zu jedem in diesem Buch genannten Refactoring gibt es ein inverses Refactoring, das eine logische Umkehrung der durchgeführten Änderungen vornimmt. Aber ich habe nicht alle Refactorings dieser Art aufgeführt, weil ich viele dieser Umkehrungen für uninteressant halte. So ist beispielsweise *Variable kapseln* (Abschnitt 6.6) ein gebräuchliches und leistungsstarkes Refactoring, dessen Umkehrung (die sich darüber hinaus auch sehr leicht durchführen lässt) ich aber praktisch nie benötige. Deshalb denke ich, dass dafür kein Katalogeintrag nötig ist.

Kapitel 6

Eine erste Zusammenstellung von Refactorings

Am Anfang des Katalogs befindet sich eine Reihe besonders nützlicher Refactorings, die meiner Ansicht nach zuerst erlernt werden sollten.

Das Refactoring, das ich wohl am häufigsten vornehme, ist das Extrahieren von Code in eine neue Funktion (*Funktion extrahieren*, Abschnitt 6.1) oder Variable (*Variable extrahieren*, Abschnitt 6.3). Da es beim Refactoring immer um Änderungen geht, überrascht es nicht, dass ich auch die Umkehrungen der beiden (*Funktion inline platzieren*, Abschnitt 6.2, und *Variable inline platzieren*, Abschnitt 6.4) häufig verwende.

Beim Extrahieren müssen stets Namen vergeben werden, und ich muss die Bezeichnungen häufig ändern, wenn ich etwas dazugelernt habe. *Funktionsdeklaration ändern* (Abschnitt 6.5) ändert die Namen von Funktionen, ich verwende dieses Refactoring aber auch zum Hinzufügen oder Entfernen von Funktionsargumenten. Für Variablen verwende ich

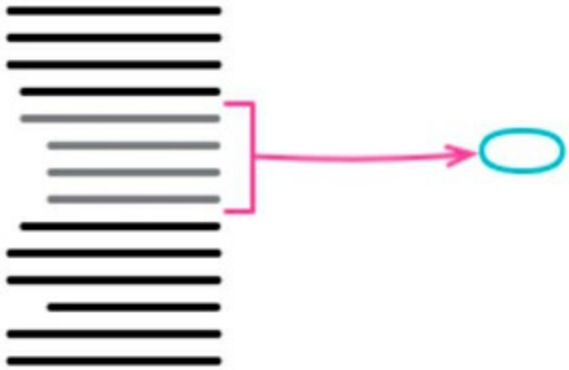
Variable umbenennen (Abschnitt 6.7), das wiederum auf *Variable kapseln* (Abschnitt 6.6) aufbaut. Beim Ändern von Funktionsargumenten erweist es sich oft als nützlich, mehrere zusammengehörige Argumente mit *Parameterobjekt einführen* (Abschnitt 6.8) zu einem eigenen Objekt zu vereinen.

Das Erstellen und das Benennen von Funktionen sind grundlegende Low-Level-Refactorings, aber sobald sie erfolgt sind, ist es notwendig, die Funktionen in übergeordnete Module zu gruppieren. Ich verwende *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9), um Funktionen zusammen mit den Daten, die sie verarbeiten, in eine Klasse zu verlagern. Manchmal kombiniere ich sie auch mit *Funktionen zu einer Transformation vereinen* (Abschnitt 6.10) zu einer Transformation, was besonders bei schreibgeschützten Daten praktisch ist. Auf einer höheren Ebene kann ich aus diesen Modulen mit *Phase aufteilen* (Abschnitt 6.11) dann häufig eigene Verarbeitungsphasen bilden.

6.1 Funktion extrahieren (Extract Function)

Ehemalige Bezeichnung: *Methode extrahieren* (Extract Method)

Umkehrung von: *Funktion inline platzieren* (Inline Function, Abschnitt 6.2)



```
function printOwing(invoice) {
  printBanner();
  let outstanding = calculateOutstanding();

  //Details ausgeben
  console.log(`name: ${invoice.customer}`);
  console.log(`amount: ${outstanding}`);
}
```

```
function printOwing(invoice) {
  printBanner();
  let outstanding = calculateOutstanding();
  printDetails(outstanding);

  function printDetails(outstanding) {
    console.log(`name: ${invoice.customer}`);
   console.log(`amount: ${outstanding}`);
  }
}
```

6.1.1 MOTIVATION

Funktion extrahieren (Abschnitt 6.1) gehört zu den Refactorings, die ich am häufigsten vornehme. (Ich verwende hier den Begriff »Funktion«, das Gleiche gilt jedoch auch für eine Methode in einer objektorientierten Programmiersprache oder für beliebige Arten von Prozeduren oder Subroutinen.) Ich betrachte einen Codeabschnitt, verstehe, welche Aufgabe er erledigt, extrahiere den Code und erstelle damit eine eigene Funktion, die der Aufgabe entsprechend benannt wird.

Ich habe während meiner Laufbahn schon viele Debatten gehört, die zum Gegenstand hatten, unter welchen Umständen Code in eine eigene Funktion ausgelagert werden sollte. Einige dieser Richtlinien zogen die Länge des Codes als Kriterium heran: Funktionen sollten auf eine Bildschirmseite passen. Andere beruhten auf der Wiederverwendbarkeit: Jeglicher Code, der mehr als einmal verwendet wird, sollte in eine eigene Funktion ausgelagert werden, nur einmalig verwendeter Code hingegen sollte inline verbleiben. Das Argument, das in meinen Augen am meisten Sinn ergibt, ist jedoch die Trennung von Zweck und Implementierung. Wenn Sie beim Betrachten eines Codeabschnitts Aufwand treiben müssen, um herauszufinden, welche Aufgabe er erledigt, sollten Sie den Code extrahieren, eine neue Funktion anlegen und sie der Aufgabe entsprechend benennen. Wenn Sie den Codeabschnitt das nächste Mal betrachten, ist Ihnen der Zweck dieser Funktion sofort ersichtlich, und in den meisten Fällen brauchen Sie nicht

mehr darüber nachzudenken, wie die Funktion ihre Aufgabe erledigt (also über den eigentlichen Inhalt der Funktion).

Nachdem ich diese Vorgehensweise zum Prinzip erhoben hatte, entwickelte ich die Angewohnheit, sehr kurze Funktionen zu schreiben – typischerweise nur einige wenige Zeilen lang. Nach meinem Dafürhalten macht sich jede Funktion mit mehr als einem halben Dutzend Zeilen eines Code-Smells verdächtig. Meine Funktionen bestehen nicht selten aus nur einer einzigen Codezeile. Die Tatsache, dass die Länge unwichtig ist, wurde mir durch ein Beispiel klar, das Kent Beck mir anhand des ursprünglichen Smalltalk-Systems vorführte. Smalltalk lief damals auf Systemen mit monochromen Monitoren. Wenn Sie einen Text oder eine Grafik hervorheben wollten, mussten Sie die Bildschirmanzeige invertieren. Die Grafikklasse von Smalltalk stellte zu diesem Zweck eine Methode namens `highlight` (hervorheben) bereit, deren Implementierung lediglich aus dem Aufruf der Methode `reverse` (invertieren) bestand. Der Name der Methode war länger als ihre Implementierung – aber das spielte keine Rolle, weil zwischen dem Zweck des Codes und der Implementierung ein großer semantischer Abstand lag.

Manche Leute haben Bedenken, kurze Funktionen zu verwenden, weil sie sich Sorgen machen, dass die Performance unter den vielen Funktionsaufrufen leiden könnte. Als ich noch jung war, spielte das tatsächlich hin

und wieder eine Rolle, ist heutzutage aber selten. Optimierende Compiler liefern bei kürzeren Funktionen oft bessere Ergebnisse, weil das Caching erleichtert wird. Folgen Sie ansonsten den allgemeinen Richtlinien zur Verbesserung der Performance.

Kurze Funktionen sollten am besten gut gewählte Namen tragen, deshalb müssen Sie der Namensgebung große Aufmerksamkeit schenken. Dazu ist Übung erforderlich – aber sobald Sie damit Erfahrung gesammelt haben, führt dieser Ansatz zu erstaunlich selbstdokumentierendem Code.

In längeren Funktionen stoße ich oft auf Codeabschnitte, an deren Anfang ein Kommentar steht, der erklärt, welche Aufgabe sie erledigen. Der Kommentar ist nicht selten ein guter Hinweis für die Benennung der Funktion, wenn ich den Abschnitt extrahiere.

6.1.2 VORGEHEN

- Erstellen Sie eine neue Funktion und benennen Sie diese ihrer Aufgabe entsprechend. Benennen Sie sie nach dem, *was* sie tut und nicht danach, *wie* sie es tut.

  Wenn der zu extrahierende Code sehr einfach ist, wie etwa ein einziger Funktionsaufruf, extrahiere ich ihn trotzdem, falls der Name der neuen Funktion den Zweck des Codes besser verdeutlicht. Wenn mir kein aussagekräftigerer Name einfällt, ist das ein Hinweis darauf, dass ich den Code besser nicht extrahieren sollte. Mir muss aber auch nicht sofort der beste Name einfallen. Manchmal komme ich erst auf einen guten Namen, wenn ich mit der extrahierten Funktion arbeite. Eine Funktion zu extrahieren, sie

auszuprobieren und festzustellen, dass sie nicht weiterhilft, ist auch ein mögliches Vorgehen. Wenn ich dabei etwas dazugelernt habe, war es auch keine Zeitverschwendung.

Wenn die Programmiersprache verschachtelte Funktionen unterstützt, betten Sie die extrahierte Funktion in deren Ursprungsfunktion ein. Das verringert die Anzahl an Variablen, um deren Gültigkeitsbereich Sie sich nach den nächsten Schritten kümmern müssen. Ich kann *Funktion verschieben* (Abschnitt 8.1) später immer noch auf die verschachtelte Funktion anwenden.

- Kopieren Sie den extrahierten Code von der Quellfunktion in eine neue Funktion.
- Suchen Sie im extrahierten Code nach Verweisen auf Variablen, die sich innerhalb des Gültigkeitsbereichs der Quellfunktion befinden, aber nicht in den Gültigkeitsbereich der Zielfunktion fallen. Übergeben Sie diese Variablen als Parameter.

  Wenn die extrahierte Funktion eine verschachtelte Funktion der Quellfunktion ist, entfällt dieses Problem.

  Die meisten Funktionen verwenden lokale Variablen und Parameter. Der generelle Ansatz lautet, alle Parameter dieser Art als Argumente zu übergeben. Variablen, die zwar verwendet werden, denen aber im Vorfeld keine Werte zugewiesen werden, bereiten normalerweise keine Schwierigkeiten.

  Wird eine Variable ausschließlich innerhalb des extrahierten Codes verwendet, aber außerhalb des Codes deklariert, verschieben Sie die Deklaration in den extrahierten Code.

  Alle Variablen, denen Werte zugewiesen werden, bedürfen weiterer Aufmerksamkeit, wenn sie als Wertparameter (pass-by-value) übergeben werden. Wenn es nur eine davon gibt, versuche ich, den extrahierten Code als Abfrage zu behandeln, und weise das Ergebnis der betroffenen Variablen zu.

  Hin und wieder kommt es vor, dass der extrahierte Code zu vielen lokalen Variablen Werte zuweist. In diesem Fall ist es besser, die Extraktion abzubrechen. Ich ziehe dann andere Refactorings in Betracht, wie etwa *Variable aufteilen* (Abschnitt 9.1) oder *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4), um

die Verwendung von Variablen zu vereinfachen, und erwäge die Extraktion später erneut.

- Kompilieren Sie den Code, nachdem Sie sich um alle Variablen gekümmert haben.

  Nachdem Sie allen Variablen Ihre Aufmerksamkeit geschenkt haben, kann es sich als nützlich erweisen, den Code zu kompilieren, wenn Ihre Sprachumgebung Überprüfungen zur Kompilierzeit unterstützt. Häufig lassen sich dadurch Variablen aufdecken, die nicht korrekt behandelt wurden.

- Ersetzen Sie den extrahierten Code in der Quellfunktion durch einen Aufruf der Zielfunktion.

- Testen Sie.

- Suchen Sie nach weiterem Code, der mit dem soeben extrahierten Code identisch ist oder ihm ähnelt, und ziehen Sie in Betracht, *Inline-Code durch Funktionsaufruf ersetzen* (Abschnitt 8.5) anzuwenden, um die neue Funktion aufzurufen.

  Einige Refactoring-Tools bieten dafür direkte Unterstützung. Ansonsten kann es sich lohnen, einige Suchläufe durchzuführen, um weiteren redundanten Code aufzuspüren.

6.1.3 BEISPIEL: KEINE VARIABLEN AUSSERHALB DES GÜLTIGKEITSBEREICHS

Im besten Fall ist *Funktion extrahieren* (Abschnitt 6.1) ein Kinderspiel.

```
function printOwing(invoice) {
  let outstanding = 0;
  console.log("***************************");
  console.log("* Forderung an den Kunden *");
  console.log("***************************");
```

```
  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  // Fälligkeitsdatum erfassen
  const today = Clock.today;
  invoice.dueDate = new Date(today.getFullYear(),
                     today.getMonth(), today.getDate()
+ 30);
  //Details ausgeben
  console.log(`name: ${invoice.customer}`);
  console.log(`amount: ${outstanding}`);
  console.log(`due:
${invoice.dueDate.toLocaleDateString()}`);
}
```

Vielleicht fragen Sie sich, welchen Zweck `Clock.today` *erfüllt. Dabei handelt es sich um einen Clock Wrapper (https://martinfowler.com/bliki/ClockWrapper.html), also um ein Objekt, das Ihnen die aktuelle Uhrzeit oder das aktuelle Datum bereitstellt. Ich verzichte in meinem Code möglichst auf direkte Aufrufe wie* `Date.now()`*, weil sie zu nicht-deterministischen Tests führen und es bei der Diagnose erschweren, die Rahmenbedingungen eines Fehlers zu reproduzieren.*

Den Code zu extrahieren, der das Banner ausgibt, ist nicht weiter schwer. Ich schneide ihn aus, füge ihn ein und rufe die Funktion auf:

```
function printOwing(invoice) {
  let outstanding = 0;
```

```
  printBanner();

  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  // Fälligkeitsdatum erfassen
  const today = Clock.today;
  invoice.dueDate = new Date(today.getFullYear(),
                     today.getMonth(), today.getDate()
+ 30);
  //Details ausgeben
  console.log(`name: ${invoice.customer}`);
  console.log(`amount: ${outstanding}`);
  console.log(`due:
${invoice.dueDate.toLocaleDateString()}`);
}
function printBanner() {
  console.log("***************************");
  console.log("* Forderung an den Kunden *");
  console.log("***************************");
}
```

Auf ähnliche Weise kann ich auch die Ausgabe der Details extrahieren:

```
function printOwing(invoice) {
  let outstanding = 0;

  printBanner();

  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }
```

```
  // Fälligkeitsdatum erfassen
  const today = Clock.today;
  invoice.dueDate = new Date(today.getFullYear(),
                     today.getMonth(), today.getDate()
+ 30);
  printDetails();

  function printDetails() {
  console.log(`name: ${invoice.customer}`);
  console.log(`amount: ${outstanding}`);
  console.log(`due:
${invoice.dueDate.toLocaleDateString()}`);
  }
```

Auf den ersten Blick sieht es so aus, als ob das Refactoring *Funktion extrahieren* (Abschnitt 6.1) völlig trivial wäre. In vielen Situationen kann es sich aber auch als erheblich kniffliger erweisen.

Ich habe im obigen Fall `printDetails` so definiert, dass die Funktion in `printOwing` verschachtelt ist. Auf diese Weise kann sie auf alle Variablen zugreifen, die in `printOwing` definiert sind. Diese Möglichkeit steht mir allerdings nicht zur Verfügung, wenn ich in einer Sprache programmiere, die keine verschachtelten Funktionen unterstützt. Ich sehe mich dann im Wesentlichen vor die Aufgabe gestellt, die Funktion zu extrahieren und auf der obersten Ebene zu definieren, was bedeutet, dass ich alle Variablen berücksichtigen muss, die nur im Gültigkeitsbereich der Quellfunktion vorhanden sind. Dabei handelt es sich um die Argumente der ursprünglichen Funktion und die in ihr definierten

temporären Variablen.

6.1.4 BEISPIEL: VERWENDUNG LOKALER VARIABLEN

Im einfachsten Fall werden lokale Variablen nur verwendet, es werden ihnen aber keine neuen Werte zugewiesen. Ich kann sie in diesem Fall als Parameter übergeben. Wenn ich also die folgende Funktion vorliegen habe...

```
function printOwing(invoice) {
  let outstanding = 0;

  printBanner();

  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  // Fälligkeitsdatum erfassen
  const today = Clock.today;
  invoice.dueDate = new Date(today.getFullYear(),
                     today.getMonth(), today.getDate()
+ 30);

  //Details ausgeben
  console.log(`name: ${invoice.customer}`);
  console.log(`amount: ${outstanding}`);
  console.log(`due:
${invoice.dueDate.toLocaleDateString()}`);
}
```

... kann ich die Ausgabe der Details extrahieren, indem

ich zwei Parameter übergebe:

```
function printOwing(invoice) {
  let outstanding = 0;

  printBanner();

  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  // Fälligkeitsdatum erfassen
  const today = Clock.today;
  invoice.dueDate = new Date(today.getFullYear(),
                     today.getMonth(), today.getDate()
+ 30);

  printDetails(invoice, outstanding);
}
function printDetails(invoice, outstanding) {
  console.log(`name: ${invoice.customer}`);
  console.log(`amount: ${outstanding}`);
  console.log(`due:
${invoice.dueDate.toLocaleDateString()}`);
}
```

Das funktioniert auch, wenn die lokale Variable eine Datenstruktur ist (wie etwa ein Array, ein Datensatz oder ein Objekt). So kann ich beispielsweise die Erfassung des Fälligkeitsdatums (`recordDueDate`) extrahieren:

```
function printOwing(invoice) {
  let outstanding = 0;
```

```
  printBanner();

  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  recordDueDate(invoice);
  printDetails(invoice, outstanding);
}
function recordDueDate(invoice) {
  const today = Clock.today;
  invoice.dueDate = new Date(today.getFullYear(),
                     today.getMonth(), today.getDate()
+ 30);
}
```

6.1.5 BEISPIEL: ZUWEISUNG ZU EINER LOKALEN VARIABLE

Die Zuweisung zu lokalen Variablen wird komplizierter. In diesem Fall betrachten wir ausschließlich temporäre Variablen. Wenn ich auf eine Stelle stoße, an der einem Parameter etwas zugewiesen wird, wende ich umgehend *Variable aufteilen* (Abschnitt 9.1) an, um daraus eine temporäre Variable zu machen.

Bei Zuweisungen zu temporären Variablen sind zwei Fälle zu unterscheiden. Wenn die temporäre Variable nur innerhalb des extrahierten Codes verwendet wird, ist das der einfachere Fall. Die Variable existiert dann nur innerhalb des extrahierten Codes. Gelegentlich kann es praktisch sein, *Anweisungen verschieben* (Abschnitt 8.6) anzuwenden, um alle Änderungen an den Variablen an

einer Stelle zu vereinen, insbesondere wenn die Initialisierung der Variablen weit vor ihrer eigentlichen Verwendung stattfindet.

Wenn die Variable auch außerhalb des extrahierten Codes verwendet wird, liegt ein etwas umständlicherer Fall vor. Dann muss ich den neuen Wert zurückgeben. Ich kann das mit der Ihnen bereits vertrauten folgenden Funktion veranschaulichen:

```
function printOwing(invoice) {
  let outstanding = 0;

  printBanner();

  // Offene Forderung berechnen
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  recordDueDate(invoice);
  printDetails(invoice, outstanding);
}
```

Ich habe die letzten Refactorings immer in einem Schritt vorgenommen, weil sie ganz unkompliziert waren. Aber im vorliegenden Fall werde ich schrittweise vorgehen.

Als Erstes verschiebe ich die Deklaration der Variable dorthin, wo sie verwendet wird.

```
function printOwing(invoice) {
```

```
  printBanner();

  // Offene Forderung berechnen
  let outstanding = 0;
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  recordDueDate(invoice);
  printDetails(invoice, outstanding);
}
```

Anschließend kopiere ich den zu extrahierenden Code für die neue Funktion.

```
function printOwing(invoice) {
  printBanner();

  // Offene Forderung berechnen
  let outstanding = 0;
  for (const o of invoice.orders) {
    outstanding += o.amount; }

  recordDueDate(invoice);
  printDetails(invoice, outstanding);
}
function calculateOutstanding(invoice) {
  let outstanding = 0;
  for (const o of invoice.orders) {
    outstanding += o.amount;
  }
  return outstanding;
}
```

Da ich die Deklaration von `outstanding` in den

extrahierten Code verschoben habe, muss ich den Wert nicht mehr als Parameter übergeben. `outstanding` ist die einzige Variable, der im extrahierten Code ein neuer Wert zugewiesen wird, deshalb gebe ich sie zurück.

Meine JavaScript-Umgebung liefert beim Kompilieren keine nennenswerten Informationen – tatsächlich sogar weniger als die Syntaxanalyse meines Editors, hier ist also kein weiterer Schritt notwendig. Als Nächstes ersetze ich den ursprünglichen Code durch einen Aufruf der neuen Funktion. Da ich den Wert zurückgebe, muss ich ihn in der ursprünglichen Variable speichern.

```
function printOwing(invoice) {
  printBanner();
  let outstanding = calculateOutstanding(invoice);
  recordDueDate(invoice);
  printDetails(invoice, outstanding);
}
function calculateOutstanding(invoice) {
  let outstanding = 0;
  for (const o of invoice.orders) {
    outstanding += o.amount;
  }
  return outstanding;
}
```

Bevor ich die Aufgabe als erledigt betrachte, vergebe ich noch einen neuen Namen für den Rückgabewert, um meiner Konvention treu zu bleiben.

Bei dieser Gelegenheit mache ich außerdem aus

`outstanding` eine Konstante.

```
function printOwing(invoice) {
  printBanner();
  const outstanding = calculateOutstanding(invoice);
  recordDueDate(invoice);
  printDetails(invoice, outstanding);
}
function calculateOutstanding(invoice) {
  let result = 0;
  for (const o of invoice.orders) {
    result += o.amount;
  }
  return result;
}
```

An dieser Stelle fragen Sie sich vielleicht, wie mehrere Variablen zurückgegeben werden können.

Hier gibt es verschiedene Möglichkeiten. Meistens ziehe ich es vor, anderen Code zu extrahieren. Ich möchte, dass jede Funktion möglichst nur einen Wert zurückliefert. Deshalb würde ich versuchen, den Code so umzugestalten, dass die verschiedenen Werte von unterschiedlichen Funktionen zurückgeben werden. Wenn ich tatsächlich gezwungen bin, mehrere Werte zurückzugeben, könnte ich einen Datensatz erstellen und diesen zurückgeben – aber für gewöhnlich halte ich es für besser, stattdessen die temporären Variablen zu überarbeiten. Hier sind *Variable aufteilen* (Abschnitt 9.1) und *Temporäre Variable durch Abfrage ersetzen*

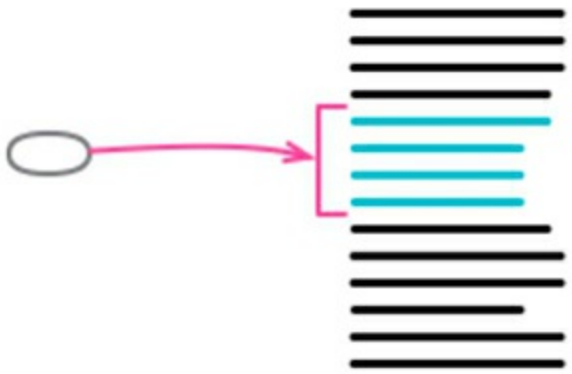
(Abschnitt 7.4) gut anwendbar.

Das wirft eine interessante Frage auf, wenn ich Funktionen extrahiere, von denen ich erwarte, dass sie später in einen anderen Kontext verschoben werden, z.B. auf die oberste Ebene. Ich bevorzuge kleine Schritte, deshalb erstelle ich aus dem extrahierten Code zunächst verschachtelte Funktionen und verschiebe diese dann in ihren neuen Kontext. Das Trickreiche dabei ist jedoch die Handhabung der Variablen, und diese Schwierigkeit tritt erst dann auf, wenn ich die Funktion verschiebe. Obwohl ich aus dem extrahierten Code eine verschachtelte Funktion machen kann, wäre es daher sinnvoll, diese zunächst auf der Geschwisterebene der Quellfunktion anzuordnen, damit ich sofort feststellen kann, ob der extrahierte Code einen Sinn ergibt.

6.2 Funktion inline platzieren (Inline Function)

Ehemalige Bezeichnung: *Methode integrieren* (Inline Method)

Umkehrung von: *Funktion extrahieren* (Extract Function, Abschnitt 6.1)



```
function getRating(driver) {
  return moreThanFiveLateDeliveries(driver) ? 2 : 1;
}

function moreThanFiveLateDeliveries(driver) {
  return driver.numberOfLateDeliveries > 5;
}
```

```
function getRating(driver) {
  return (driver.numberOfLateDeliveries > 5) ? 2 : 1;
}
```

6.2.1 MOTIVATION

Zu den Themen dieses Buchs gehört die Verwendung kurzer Funktionen, die nach ihrem Zweck benannt sind, weil solche Funktionen zu klarerem und verständlicherem Code führen. Gelegentlich treffe ich jedoch auf eine Funktion, deren Inhalt ebenso klar und

deutlich ist wie ihr Name. Oder ich nehme ein Refactoring des Inhaltes einer Funktion vor, der plötzlich ebenso klar und deutlich wie ihr Name wird. In solchen Fällen entferne ich die Funktion. Indirektion kann hilfreich sein, aber unnötige Indirektion ist irritierend.

Ich verwende *Funktion inline platzieren* auch dann, wenn eine Gruppe von offenbar ungünstig unterteilten Funktionen vorliegt. Diese kann ich alle in einer großen Funktion zusammenfassen und sie bei Bedarf wieder so extrahieren, wie es mir zusagt.

Ich setze *Funktion inline platzieren* regelmäßig ein, wenn ich auf Code treffe, der zu viele Indirektionen verwendet – wenn ich den Eindruck habe, dass praktisch jede Funktion Aufgaben an andere Funktionen delegiert und ich bei all den Delegationen den Überblick verliere. Einige dieser Indirektionen mögen sinnvoll sein, aber nicht alle. Durch das Inline-Platzieren kann ich mir die nützlichen herauspicken und die übrigen entfernen.

6.2.2 VORGEHEN

- Vergewissern Sie sich, dass es sich nicht um eine polymorphe Funktion handelt.

  Wenn es um eine Methode in einer Klasse geht, deren Unterklassen sie überschreiben, kann ich die Funktion nicht inline platzieren.

- Suchen Sie nach allen Aufrufern der Funktion.
- Ersetzen Sie alle Aufrufe durch den Inhalt der Funktion.
- Führen Sie nach jedem Ersetzen Tests durch.

Sie brauchen das Inline-Platzieren nicht in einem Rutsch durchzuführen. Wenn Teile davon schwierig sind, kann es den Umständen entsprechend auch schrittweise durchgeführt werden.

- Löschen Sie die Funktionsdefinition.

So wie hier dargestellt, scheint *Funktion inline platzieren* ganz einfach zu sein. Im Allgemeinen trifft das aber nicht zu. Ich könnte viele Seiten damit füllen, wie mit Rekursion, mehreren `return`-Anweisungen, dem Inline-Platzieren einer Methode in einem anderen Objekt ohne Zugriffsfunktionen und dergleichen mehr zu verfahren ist. Der Grund dafür, dass ich es unterlasse, ist einfach: Wenn Sie auf derartige Schwierigkeiten stoßen, sollten Sie auf dieses Refactoring schlichtweg verzichten.

6.2.3 BEISPIEL

Im einfachsten Fall ist dieses Refactoring so simpel, dass man es als trivial bezeichnen kann. Ich beginne folgendermaßen:

```
function getRating(aDriver) {
  return moreThanFiveLateDeliveries(aDriver) ? 2 : 1;
}

function moreThanFiveLateDeliveries(aDriver) {
  return aDriver.numberOfLateDeliveries > 5;
}
```

Jetzt muss ich nur noch den Funktionsaufruf durch den Ausdruck nach der `return`-Anweisung in der

aufgerufenen Funktion ersetzen.

```
function getRating(aDriver) {
  return aDriver.numberOfLateDeliveries > 5 ? 2 : 1;
}
```

Es kann aber auch durchaus etwas komplizierter sein, den Code an sein neues Umfeld anzupassen. Nehmen Sie den folgenden Fall, in dem ich mit einer geringfügigen Änderung zum ursprünglichen Code beginne:

```
function rating(aDriver) {
  return moreThanFiveLateDeliveries(aDriver) ? 2 : 1;
}
function moreThanFiveLateDeliveries(dvr) {
  return dvr.numberOfLateDeliveries > 5;
}
```

Fast das Gleiche wie weiter oben, aber nun weicht der Name des deklarierten Arguments von `moreThanFiveLateDeliveries` von dem des übergebenen Arguments ab. Ich muss den Code also ein wenig anpassen, wenn ich ihn inline platziere.

```
function rating(aDriver) {
  return aDriver.numberOfLateDeliveries > 5 ? 2 : 1;
}
```

Es kann sogar noch verworrener sein. Betrachten Sie

folgenden Code:

```
function reportLines(aCustomer) {
  const lines = [];
  gatherCustomerData(lines,
  aCustomer);
  return lines;
}
function gatherCustomerData(out, aCustomer) {
  out.push(["name", aCustomer.name]);
  out.push(["location", aCustomer.location]);
}
```

Um `gatherCustomerData` in `reportLines` inline zu platzieren, ist mehr als ein einfaches Ausschneiden und Einsetzen erforderlich. Es ist zwar nicht allzu kompliziert, und in den meisten Fällen würde ich die Aufgabe in einem Rutsch erledigen, aber vorsichtshalber könnte es sinnvoll sein, die Zeilen einzeln zu verschieben. Ich wende also zunächst *Anweisungen in Aufrufer verschieben* (Abschnitt 8.4) auf die erste Zeile an. (Ich erledige das auf die einfache Art durch Ausschneiden, Einsetzen und Anpassen.)

```
function reportLines(aCustomer) {
  const lines = [];
  lines.push(["name", aCustomer.name]);
  gatherCustomerData(lines, aCustomer);
 return lines;
}
function gatherCustomerData(out, aCustomer) {
  out.push(["name", aCustomer.name]);
```

```
  out.push(["location", aCustomer.location]);
}
```

Anschließend fahre ich mit den anderen Zeilen fort, bis alle abgearbeitet sind.

```
function reportLines(aCustomer) {
  const lines = [];
  lines.push(["name", aCustomer.name]);
  lines.push(["location", aCustomer.location]);
 return lines;
}
```

Der springende Punkt ist hier, dass Sie sich jederzeit offen halten sollten, kleinere Schritte zu machen. Dank der kurzen Funktionen, die ich üblicherweise schreibe, kann ich *Funktion inline platzieren* meist in einem einzigen Schritt erledigen, selbst wenn einige Anpassungen vorzunehmen sind. Sollte es jedoch zu Komplikationen kommen, verschiebe ich die Zeilen einzeln. Aber sogar beim zeilenweisen Verschieben kann es manchmal kompliziert werden. Dann verwende ich die ausgefeiltere Vorgehensweise von *Anweisungen in Aufrufer verschieben* (Abschnitt 8.4), um das Ganze weiter aufzuteilen. Und wenn ich mir meiner Sache zu sicher bin, irgendwelche Aufgaben zu hastig erledige und die Tests fehlschlagen, kehre ich zu meinem letzten »grünen« Code zurück und wiederhole das Refactoring in kleineren Schritten und mit ein wenig Verdruss.

6.3 Variable extrahieren (Extract Variable)

Ehemalige Bezeichnung: *Erklärende Variable einführen* (Introduce Explaining Variable)

Umkehrung von: *Variable inline platzieren* (Inline Variable, Abschnitt 6.4)

```
return order.quantity * order.itemPrice -
  Math.max(0, order.quantity - 500) * order.itemPrice
* 0.05
  + Math.min(order.quantity * order.itemPrice * 0.1,
100);
```

```
const basePrice = order.quantity * order.itemPrice;
const quantityDiscount = Math.max(0, order.quantity -
500) *
                          order.itemPrice * 0.05;
const shipping = Math.min(basePrice * 0.1, 100);
return basePrice - quantityDiscount + shipping;
```

6.3.1 MOTIVATION

Ausdrücke können sehr kompliziert und schwer verständlich werden. In solchen Situationen können sich lokale Variablen als nützlich erweisen, um den Ausdruck so anzupassen, dass er sich besser handhaben lässt. Sie bieten mir insbesondere die Möglichkeit, die Bestandteile einer komplexen Programmlogik zu benennen. Ich kann so besser verstehen, was vor sich geht.

Zudem sind solche Variablen beim Debugging praktisch, weil sie sich im Debugger anzeigen lassen oder für die Ausgabe durch `print`-Anweisungen geeignet sind.

Das Refactoring *Variable extrahieren* gibt mir die Möglichkeit, einem Ausdruck in meinem Code einen Namen zu geben. Dabei muss ich mir über den Kontext dieses Namens Gedanken machen. Wenn der Name nur in der Funktion, an der ich gerade arbeite, von Bedeutung ist, dann ist *Variable extrahieren* eine gute Wahl. Ist er hingegen auch in einem breiteren Kontext sinnvoll, ziehe ich in Betracht, ihn in diesem größeren Umfeld bereitzustellen, für gewöhnlich in Form einer Funktion. Eine breite Verfügbarkeit des Namens bedeutet, dass anderer Code auf den Wert des Ausdrucks zugreifen kann, ohne den Ausdruck wiederholen zu müssen, was zu weniger Redundanzen führt und meine Absicht besser verdeutlicht.

Die Verfügbarkeit des Namens in einem breiteren Kontext hat den Nachteil, dass damit zusätzlicher Aufwand verbunden ist. Wenn dieser Aufwand beträchtlich ist, werde ich mich wahrscheinlich erst später damit befassen, wenn ich *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4) anwenden kann. Ist der Aufwand hingegen gering, erledige ich die Aufgabe umgehend, damit der Name sofort im Code verfügbar ist. Ein schönes Beispiel hierfür: Wenn ich an einer Klasse arbeite, lässt sich *Funktion extrahieren* ganz einfach umsetzen.

6.3.2 VORGEHEN

- Vergewissern Sie sich, dass der zu extrahierende Ausdruck keine Seiteneffekte verursacht.
- Deklarieren Sie eine unveränderliche Variable. Weisen Sie ihr eine Kopie des Ausdrucks zu, den Sie benennen möchten.
- Ersetzen Sie den ursprünglichen Ausdruck durch die neue Variable.
- Testen Sie.

Ersetzen Sie, wenn er mehrfach vorhanden ist, alle Vorkommen des Ausdrucks durch die Variable. Führen Sie nach jedem Ersetzen Tests durch.

6.3.3 BEISPIEL

Am Anfang steht eine einfache Berechnung:

```
function price(order) {
```

```
  //Der Preis ergibt sich aus dem Grundpreis abzüglich
  //des Mengenrabatts und zuzüglich der Versandkosten.
  return order.quantity * order.itemPrice -
    Math.max(0, order.quantity - 500)
                            * order.itemPrice * 0.05 +
    Math.min(order.quantity * order.itemPrice * 0.1,
100);
}
```

Auch, wenn das sehr simpel ist, kann ich den Code wie folgt noch weiter vereinfachen: Als Erstes fällt mir auf, dass sich der Grundpreis (`basePrice`) aus dem Produkt aus der Anzahl (`quantity`) und einem Einzelpreis (`itemPrice`) zusammensetzt.

```
function price(order) {
  //Der Preis ergibt sich aus dem Grundpreis abzüglich
  //des Mengenrabatts und zuzüglich der Versandkosten.
  return order.quantity * order.itemPrice -
    Math.max(0, order.quantity - 500)
                            * order.itemPrice * 0.05 +
    Math.min(order.quantity * order.itemPrice * 0.1,
100);
}
```

Sobald ich diese Erkenntnis in meinem Kopf habe, übertrage ich sie in den Code, indem ich dafür eine Variable erstelle und benenne.

```
function price(order) {
  //Der Preis ergibt sich aus dem Grundpreis abzüglich
  //des Mengenrabatts und zuzüglich der Versandkosten.
```

```
  const basePrice = order.quantity * order.itemPrice;
  return order.quantity * order.itemPrice -
    Math.max(0, order.quantity - 500)
                            * order.itemPrice * 0.05 +
    Math.min(order.quantity * order.itemPrice * 0.1,
100);
}
```

Lediglich eine Variable zu deklarieren und zu initialisieren bewirkt natürlich noch gar nichts. Ich muss sie auch verwenden, also ersetze ich den Ausdruck, der mir als Quelle der Variablendefinition diente.

```
function price(order) {
  //Der Preis ergibt sich aus dem Grundpreis abzüglich
  //des Mengenrabatts und zuzüglich der Versandkosten.
  const basePrice = order.quantity * order.itemPrice;
  return basePrice -
    Math.max(0, order.quantity - 500)
                            * order.itemPrice * 0.05 +
    Math.min(order.quantity * order.itemPrice * 0.1,
100);
}
```

Der gleiche Ausdruck wird später erneut verwendet, also kann ich ihn dort ebenfalls durch die Variable ersetzen.

```
function price(order) {
  //Der Preis ergibt sich aus dem Grundpreis abzüglich
  //des Mengenrabatts und zuzüglich der Versandkosten.
  const basePrice = order.quantity * order.itemPrice;
  return basePrice -
    Math.max(0, order.quantity - 500)
```

```
                                * order.itemPrice * 0.05 +
    Math.min(basePrice * 0.1, 100);
}
```

Die nächste Zeile ist der Mengenrabatt (`quantityDiscount`). Also extrahiere ich den Code ebenfalls.

```
function price(order) {
  //Der Preis ergibt sich aus dem Grundpreis abzüglich
  //des Mengenrabatts und zuzüglich der Versandkosten.
  const basePrice = order.quantity * order.itemPrice;
  const quantityDiscount = Math.max(0, order.quantity
- 500)
                                    * order.itemPrice *
0.05;
  return basePrice -
    quantityDiscount +
    Math.min(basePrice * 0.1, 100);
}
```

Zum Schluss sind die Versandkosten (`shipping`) an der Reihe. Jetzt kann ich auch den inzwischen überflüssigen Kommentar entfernen, der nichts mehr aussagt, was nicht auch der Code erkennen lässt.

```
function price(order) {
  const basePrice = order.quantity * order.itemPrice;
  const quantityDiscount = Math.max(0, order.quantity
- 500)
                                    * order.itemPrice *
0.05;
```

```
  const shipping = Math.min(basePrice * 0.1, 100);
  return basePrice - quantityDiscount + shipping;
}
```

6.3.4 BEISPIEL MIT EINER KLASSE

Nachstehend sehen Sie den gleichen Code, hier aber im Kontext einer Klasse.

```
class Order {
  constructor(aRecord) {
    this._data = aRecord;
  }
  get quantity() {return this._data.quantity;}
  get itemPrice() {return this._data.itemPrice;}

  get price() {
    return this.quantity * this.itemPrice -
      Math.max(0, this.quantity - 500) *
                  this.itemPrice * 0.05 +
      Math.min(this.quantity * this.itemPrice * 0.1,
100);
  }
}
```

In diesem Fall möchte ich die gleichen Namen extrahieren, bemerke jedoch, dass sich die Namen hier auf die Bestellung (`Order`) als Ganzes beziehen, nicht nur auf die Berechnung des Preises. Weil sie sich auf die gesamte Bestellung beziehen, erstelle ich für die extrahierten Namen keine Variablen, sondern Methoden.

```
class Order {
  constructor(aRecord) {
    this._data = aRecord;
  }
  get quantity() {return this._data.quantity;}
  get itemPrice() {return this._data.itemPrice;}

  get price() {
    return this.basePrice - this.quantityDiscount
                                            +
this.shipping;
  }
  get basePrice() {return this.quantity *
this.itemPrice;}
  get quantityDiscount() {
    return Math.max(0, this.quantity - 500) *
                       this.itemPrice * 0.05;
  }
  get shipping() {
    return Math.min(this.basePrice * 0.1, 100);
  }
}
```

Hier zeigt sich einer der großen Vorteile von Objekten – sie bieten einen sinnvollen Kontext, um Teile der Programmlogik und Daten gemeinsam zu nutzen. Bei diesem einfachen Beispiel macht das keinen großen Unterschied, aber bei einer größeren Klasse erweist es sich als äußerst nützlich, gemeinsames Verhalten als eigenständige Abstraktionen mit eigenem Namen bereitzustellen und bei der Verwendung eines Objekts jederzeit darauf zugreifen zu können.

6.4 Variable inline platzieren (Inline Variable)

Ehemalige Bezeichnung: *Temporäre Variable integrieren* (Inline Temp)

Umkehrung von: *Variable extrahieren* (Extract Variable, Abschnitt 6.3)

```
let basePrice = anOrder.basePrice;
return (basePrice > 1000);
```

```
return anOrder.basePrice > 1000;
```

6.4.1 MOTIVATION

Variablen ermöglichen es, Ausdrücke innerhalb einer Funktion zu benennen, und das ist für gewöhnlich eine feine Sache. Manchmal ist der Name allerdings gar nicht aussagekräftiger als der Ausdruck selbst. Mitunter

behindert eine Variable sogar das Refactoring des benachbarten Codes. In diesen Fällen kann es sinnvoll sein, die Variable inline zu platzieren.

6.4.2 VORGEHEN

- Vergewissern Sie sich, dass die rechte Seite der Zuweisung keine Seiteneffekte verursacht.
- Sollte die Variable nicht schon als unveränderlich deklariert sein, holen Sie das nach und führen Sie einen Test durch. Dadurch ist gewährleistet, dass eine Wertzuweisung nur einmalig stattfindet.
- Suchen Sie nach dem ersten Verweis auf die Variable und ersetzen sie ihn durch die rechte Seite der Zuweisung.
- Testen Sie.
- Wiederholen Sie das Ersetzen der Verweise, bis alle ersetzt worden sind.
- Löschen Sie die Deklaration und die Zuweisung der Variable.
- Testen Sie.

6.5 Funktionsdeklaration ändern (Change Function Declaration)

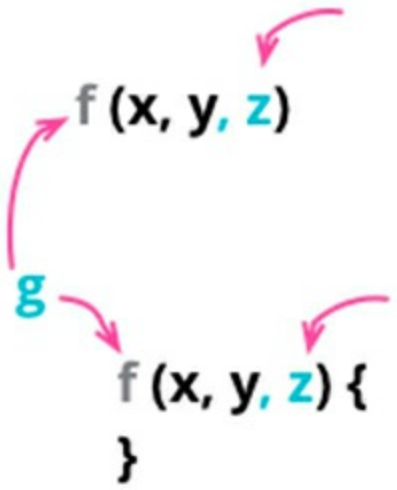
Weitere Bezeichnung: *Funktion umbenennen* (Rename Function)

Ehemalige Bezeichnung: *Methode umbenennen* (Rename Method)

Ehemalige Bezeichnung: *Parameter ergänzen* (Add Parameter)

Ehemalige Bezeichnung: *Parameter entfernen* (Remove Parameter)

Weitere Bezeichnung: *Signatur ändern* (Change Signature)




```
function circum(radius) {...}
```

```
function circumference(radius) {...}
```

6.5.1 MOTIVATION

Zur Aufgliederung eines Programms in mehrere Bestandteile verwenden wir vornehmlich Funktionen. Funktionsdeklarationen geben an, wie diese Teile

zusammenwirken – sie repräsentieren faktisch die Verbindungsstellen unserer Softwaresysteme. Wie auch bei allen anderen Konstruktionen hängt viel von diesen Verbindungen ab. Sinnvolle Verbindungsstellen erlauben mir, dem System mühelos neue Teile hinzuzufügen, während mangelhafte fortwährend Ursache für Probleme sind, die es erschweren, herauszufinden, welche Aufgaben die Software erledigt und welche Änderungen notwendig sind, um letztere an wechselnde Bedürfnisse anzupassen. Glücklicherweise handelt es sich um *Soft*ware, die veränderlich ist, sodass ich die Verbindungen ändern kann, sofern ich dabei sorgfältig vorgehe.

Das wichtigste Element einer solchen Verbindungsstelle ist der Name der Funktion. Ein aussagekräftiger Name ermöglicht es mir, zu verstehen, welche Aufgabe die Funktion erfüllt, ohne dass ich den Code ihrer Implementierung in Augenschein nehmen muss. Es ist allerdings nicht ganz einfach, sich gute Namen auszudenken – mir jedenfalls gelingt das beim ersten Anlauf nur selten. Wenn ich feststelle, dass eine Bezeichnung mich verwirrt, muss ich der Versuchung widerstehen, es trotzdem dabei zu belassen – ist ja schließlich nur ein Name. Das ist das Werk des bösen Dämons Obfuscatis (*Verschleierung*). Der Seele meines Programms zuliebe darf ich ihm niemals nachgeben. Wenn ich auf eine Funktion mit falschem Namen treffe, ist es unabdingbar, dass ich ihn sofort ändere, sobald mir

ein besserer Name einfällt. Dadurch ist gewährleistet, dass ich nicht *schon wieder* herausfinden muss, was der Code bewirkt, wenn ich ihn das nächste Mal betrachte. (Meistens funktioniertes es ganz gut, einen Kommentar zu verfassen, der den Zweck der Funktion beschreibt, und aus dem Kommentar einen besseren Funktionsnamen abzuleiten.)

Das Gleiche gilt auch für die Parameter einer Funktion. Sie legen fest, wie sich eine Funktion in den Rest des Programms einfügt. Die Parameter bestimmen den Kontext, in dem ich eine Funktion verwenden kann. Wenn eine Funktion zum Formatieren der Telefonnummer einer Person als Argument eine Person entgegennimmt, kann ich sie nicht dafür verwenden, die Telefonnummer einer Firma zu formatieren. Wenn ich diesen Parameter durch die Telefonnummer selbst ersetze, ist der Formatierungscode in einem größeren Kontext nützlich.

Ich kann aber nicht nur den möglichen Anwendungsbereich der Funktion erweitern, sondern auch die Kopplung verringern, indem ich ändere, was Module benötigen, um mit anderen Modulen zu kommunizieren. Die Programmlogik zum Formatieren der Telefonnummer könnte sich in einem Modul befinden, das keine Kenntnis von Personen hat. Wenn verschiedene Module weniger voneinander wissen, muss sich mein Gehirn auch weniger Dinge merken, wenn ich Änderungen vornehme – und mein Gehirn ist nicht

mehr so aufnahmefähig wie früher.

Für die Auswahl der richtigen Parameter gibt es keine einfachen Regeln. Ich könnte beispielsweise eine einfache Funktion schreiben, die feststellt, ob eine Rechnung überfällig ist, indem ich überprüfe, ob sie älter als 30 Tage ist. Sollte ich für diese Funktion als Parameter das Rechnungs-Objekt selbst oder das Fälligkeitsdatum der Rechnung wählen? Die Verwendung der Rechnung koppelt die Funktion an die Schnittstelle des Rechnungs-Objekts. Aber wenn ich die Rechnung verwende, kann ich auch leicht auf ihre weiteren Eigenschaften zugreifen, falls die Programmlogik weiterentwickelt wird, ohne dass ich sämtlichen Code anpassen muss, der diese Funktion aufruft – wodurch im Wesentlichen die Kapselung der Funktion verstärkt wird.

Die einzig richtige Antwort auf die obige Frage ist, dass es keine richtige Antwort gibt, schon gar nicht langfristig betrachtet. Ich halte es deshalb für wichtig, mit *Funktionsdeklaration ändern* vertraut zu sein, damit der Code weiterentwickelt werden und ich erkennen kann, welche die besten Verbindungsstellen innerhalb des Codes sind.

Normalerweise verwende ich nur die Hauptbezeichnung eines Refactorings, wenn ich an anderer Stelle im Buch darauf verweise. Die Umbenennung ist jedoch ein so essenzieller Anwendungsfall von *Funktionsdeklaration*

ändern, dass ich mich mit der Bezeichnung *Funktion umbenennen* auf dieses Refactoring beziehe, um zu verdeutlichen, wenn ich etwas umbenenne. Das gilt auch, wenn ich lediglich die Parameter umbenenne oder ändere.

6.5.2 VORGEHEN

Bei den meisten Refactorings in diesem Buch zeige ich nur ein Vorgehen. Das liegt nicht daran, dass nur eine Vorgehensweise geeignet ist, sondern dass das genannte Vorgehen in den meisten Fällen gut funktioniert. *Funktionsdeklaration ändern* stellt hier jedoch eine Ausnahme dar. Einfache Vorgehen sind häufig effektiv. Es gibt jedoch viele Fälle, in denen eine sukzessive stattfindende Migration sinnvoller ist. Bei diesem Refactoring betrachte ich die Änderung und überlege, ob ich das Anpassen der Deklaration und das aller Aufrufer problemlos in einem Rutsch durchführen kann. Ist das der Fall, folge ich der einfachen Vorgehensweise. Das Vorgehen nach dem Migrationsansatz ermöglicht mir, die Aufrufer schrittweise zu ändern, was wichtig ist, wenn es viele davon gibt, wenn sie nur umständlich erreichbar sind, die Funktion eine polymorphe Methode ist oder ich eine kompliziertere Änderung der Deklaration vornehmen muss.

Einfaches Vorgehen

- Vergewissern Sie sich vor dem Löschen eines Parameters, dass die Funktion keinen Verweis mehr darauf enthält.

- Ändern Sie die Deklaration der Methode entsprechend Ihren Bedürfnissen.
- Suchen Sie nach Verweisen auf die alte Deklaration der Methode und passen Sie diese an die neue an.
- Testen Sie.

Nicht selten ist es vorteilhaft, Änderungen einzeln vorzunehmen. Wenn Sie also sowohl den Namen ändern als auch einen Parameter hinzufügen möchten, gehen Sie in separaten Schritten vor. (Wenn Sie dabei auf Schwierigkeiten stoßen, sollten Sie die Änderungen rückgängig machen und den Migrationsansatz wählen.)

Migrationsansatz

- Nehmen Sie, falls erforderlich, ein Refactoring des Funktionsrumpfes vor, um die nachfolgende Extraktion zu vereinfachen.
- Wenden Sie *Funktion extrahieren* (Abschnitt 6.1) auf den Funktionsrumpf an.

  Geben Sie der neuen Funktionen einen temporären Namen, nach dem Sie leicht suchen können, wenn der Name der neuen Funktion anschließend dem der alten entsprechen soll.
- Wenn die neue Funktion zusätzliche Parameter benötigt, verwenden Sie zum Hinzufügen das einfache Vorgehen.
- Testen Sie.
- Wenden Sie *Funktion inline platzieren* (Abschnitt 6.2) auf die alte Funktion an.
- Wenn Sie der neuen Funktion einen temporären Namen gegeben haben, wenden Sie *Funktionsdeklaration ändern* erneut an, um den ursprünglichen Namen wiederherzustellen.
- Testen Sie.

Wenn Sie eine polymorphe Methode einer Klasse ändern, müssen Sie für jede ihrer Bindungen eine Indirektion hinzufügen. Wenn sich das polymorphe Verhalten der Methode auf eine einzelne Klassenhierarchie bezieht, benötigen Sie die weiterleitende Methode nur in der Basisklasse. Wenn die Basisklasse für die Polymorphie keinen Anknüpfungspunkt bietet, ist in jeder implementierenden Klasse eine weiterleitende Methode notwendig.

Wenn Sie ein Refactoring einer öffentlichen API vornehmen, können Sie das Refactoring unterbrechen, nachdem Sie die neue Funktion erstellt haben. Erklären Sie die ursprüngliche Funktion als veraltet und warten Sie, bis die Aufrufer die neue Funktion verwenden. Die ursprüngliche Funktionsdeklaration kann gelöscht werden, wenn Sie sicher sind, dass alle Aufrufer von der alten Funktion zur neuen migriert haben.

6.5.3 BEISPIEL: UMBENENNEN EINER FUNKTION (EINFACHES VORGEHEN)

Betrachten Sie die folgende Funktion, deren Name allzu sehr verkürzt wurde:

```
function circum(radius) {
  return 2 * Math.PI * radius;
}
```

Ich möchte daraus etwas Sinnvolleres machen. Zunächst einmal ändere ich die Deklaration:

```
function circumference(radius) {
  return 2 * Math.PI * radius;
}
```

Anschließend suche ich nach allen Aufrufern von `circum` und ändere den Namen auf `circumference`.

Wie einfach es ist, alle Verweise auf die alte Funktion zu finden, hängt von der Sprachumgebung ab. Statische Typisierung und eine gute IDE bieten die besten Voraussetzungen und ermöglichen mir normalerweise, Funktionen automatisch umzubenennen, wobei die Chance, dass ein Fehler auftritt, sehr gering ist. Ohne statische Typisierung ist das Ganze etwas umständlicher; selbst ausgereifte Suchwerkzeuge werden viele falsch positive Ergebnisse liefern.

Beim Hinzufügen oder Entfernen von Parametern verfolge ich den gleichen Ansatz: alle Aufrufer finden, Deklaration ändern und die Aufrufer ändern. Meistens ist es besser, die Schritte separat auszuführen. Wenn ich also eine Funktion umbenennen und außerdem um einen Parameter erweitern möchte, führe ich zunächst die Umbenennung durch, teste, füge den Parameter hinzu und teste erneut.

Diese einfache Methode, das Refactoring vorzunehmen, hat den Nachteil, dass ich alle Aufrufer und die Deklaration (oder im Fall von Polymorphie alle Deklarationen gleichzeitig) ändern muss. Wenn es nur einige wenige gibt oder wenn mir ein geeignetes automatisiertes Refactoring-Tool zur Verfügung steht, ist das vertretbar. Gibt es hingegen viele, kann es knifflig werden. Ein weiteres Problem ergibt sich, wenn die Namen nicht einmalig sind – beispielsweise wenn ich die `changeAddress`-Methode einer Personen-Klasse ändern möchte, eine Methode gleichen Namens aber schon in der Klasse für Versicherungsverträge vorhanden ist. Je komplexer die Änderung ist, desto weniger möchte ich sie in einem Rutsch vornehmen müssen. Wenn dieses Problem auftaucht, verwende ich den Migrationsansatz. Das tue ich auch, wenn bei der einfachen Vorgehensweise etwas schiefgeht. Dann nehme ich die Änderungen zurück und versuche es mit dem Migrationsansatz erneut.

6.5.4 BEISPIEL: UMBENENNEN EINER FUNKTION (MIGRATIONSANSATZ)

Hier ist wieder die Funktion mit dem verkürzten Namen:

```
function circum(radius) {
  return 2 * Math.PI * radius;
}
```

Damit ich dieses Refactoring als Migration durchführen kann, wende ich zunächst *Funktion extrahieren* (Abschnitt 6.1) auf den gesamten Funktionsrumpf an.

```
function circum(radius) {
  return circumference(radius);
}
function circumference(radius) {
  return 2 * Math.PI * radius;
}
```

Ich teste und wende anschließend *Funktion inline platzieren* (Abschnitt 6.2) auf die alte Funktion an. Ich suche nach allen Aufrufen der alten Funktion und ersetze jeden einzelnen durch einen Aufruf der neuen. Nach jeder dieser Änderungen kann ich testen, sodass ich alle Änderungen einzeln vornehmen kann. Nachdem das erledigt ist, lösche ich die alte Funktion.

Bei den meisten Refactorings ändere ich Code, auf den ich selbst Zugriff habe. Das vorliegende Refactoring erweist sich allerdings als äußerst praktisch, wenn der Code Teil einer öffentlichen API ist, ich den Code also nicht selbst ändern kann. Ich unterbreche das Refactoring, nachdem ich `circumference` erstellt habe, und kennzeichne `circum` als veraltet. Dann warte ich darauf, dass die Aufrufer zu `circumference` wechseln. Sobald das geschehen ist, kann ich `circum` löschen. Selbst wenn ich das Ziel, `circum` zu löschen, womöglich nie erreiche, steht mir beim Schreiben von

neuem Code doch immerhin ein besserer Name zur Verfügung.

6.5.5 BEISPIEL: HINZUFÜGEN EINES PARAMETERS

Eine Software für die Verwaltung des Buchbestands einer Bibliothek enthält eine Klasse namens `Book`, die eine Reservierung für einen Kunden vornehmen kann.

```
class Book...
addReservation(customer) {
  this._reservations.push(customer);
}
```

Ich muss eine Warteschlange zur Priorisierung von Reservierungen bereitstellen. Deshalb benötige ich für `addReservation` einen zusätzlichen Parameter, der angibt, ob die Reservierung in die normale Warteschlange oder in die Warteschlange mit hoher Priorität eingereiht werden soll. Falls sich alle Aufrufer schnell und einfach finden und ändern lassen, kann ich die Änderung problemlos durchführen. Ist das nicht der Fall, verwende ich den Migrationsansatz, den ich im Folgenden zeige.

Als Erstes wende ich *Funktion extrahieren* (Abschnitt 6.1) auf den gesamten Funktionsrumpf von `addReservation` an, um die neue Funktion zu erstellen. Die neue Funktion wird letztendlich zwar

ebenfalls den Namen `addReservation` tragen, die alte und die neue Funktion können jedoch nicht gleichzeitig denselben Namen verwenden. Ich verwende also einen temporären Namen, nach dem sich später leicht suchen lässt.

```
class Book...
addReservation(customer) {
  this.zz_addReservation(customer);
}
zz_addReservation(customer) {
  this._reservations.push(customer);
}
```

Anschließend füge ich den Parameter der neuen Deklaration und dem Aufruf hinzu (dabei verwende ich faktisch das einfache Vorgehen).

```
class Book...
addReservation(customer) {
  this.zz_addReservation(customer, false);
}
zz_addReservation(customer) {
  this._reservations.push(customer, isPriority);
}
```

Wenn ich in JavaScript programmiere, verwende ich *Assertion einführen* (Abschnitt 10.6), um zu überprüfen, ob der Aufrufer den neuen Parameter übergibt, bevor ich irgendeinen der Aufrufer ändere.

```
class Book...
zz_addReservation(customer, isPriority) {
  assert(isPriority === true || isPriority === false);
  this._reservations.push(customer);
}
```

Wenn ich jetzt beim Ändern der Aufrufer einen Fehler begehe und den neuen Parameter weglasse, werde ich durch die Assertion darauf hingewiesen. Und ich weiß aus jahrelanger Erfahrung, dass es nur sehr wenige Programmierer gibt, die noch mehr Fehler begehen als ich.

Nun kann ich damit anfangen, die Aufrufer zu ändern, indem ich *Funktion inline platzieren* (Abschnitt 6.2) auf die ursprüngliche Funktion anwende. Das erlaubt es mir wieder, alle Aufrufer einzeln zu ändern.

Dann verleihe ich der neuen Funktion den eigentlich vorgesehenen Namen. Meistens ist die einfache Vorgehensweise hierfür gut geeignet, ich könnte aber auch den Migrationsansatz verfolgen, wenn es denn notwendig wäre.

6.5.6 BEISPIEL: EINEN PARAMETER DURCH EINE SEINER EIGENSCHAFTEN ERSETZEN

Bei den Beispielen handelte es sich bisher um das Ändern von Namen oder das einfache Hinzufügen von

Parametern. Mit dem Migrationsansatz lassen sich jedoch auch kompliziertere Fälle sehr gut handhaben. Hier ist ein etwas komplexeres Beispiel.

Die folgende Funktion ermittelt, ob ein Kunde in New England ansässig ist:

```
function inNewEngland(aCustomer) {
return ["MA", "CT", "ME",
        "VT", "NH",
"RI"].includes(aCustomer.address.state);
}
```

Und hier ist einer ihrer Aufrufer:

```
Aufrufer...
const newEnglanders =
                  someCustomers.filter(c =>
inNewEngland(c));
```

`inNewEngland` verwendet lediglich den Bundesstaat des Kunden, um zu ermitteln, ob er in New England ansässig ist. Ich dagegen würde ein Refactoring von `inNewEngland` vornehmen, sodass die Funktion das Kürzel eines Bundesstaats als Parameter entgegennimmt. Wenn die Abhängigkeit vom Kunden entfernt wird, kann sie in einem größeren Kontext verwendet werden.

Wenn ich *Funktionsdeklaration ändern* (Abschnitt 6.5) anwende, besteht mein erster Schritt normalerweise darin, *Funktion extrahieren* (Abschnitt 6.1) zu verwenden, aber in diesem Fall kann ich mir die Arbeit erleichtern, indem ich zunächst ein Refactoring des Funktionsrumpfs vornehme. Ich wende also *Variable extrahieren* (Abschnitt 6.3) auf den gewünschten neuen Parameter an.

```
function inNewEngland(aCustomer) {
  const stateCode = aCustomer.address.state;
  return ["MA", "CT", "ME",
          "VT", "NH", "RI"].includes(stateCode);
}
```

Jetzt verwende ich *Funktion extrahieren* (Abschnitt 6.1), um die neue Funktion zu erstellen.

```
function inNewEngland(aCustomer) {
  const stateCode = aCustomer.address.state;
  return xxinNewEngland(stateCode);
}
function xxinNewEngland(stateCode) {
  return ["MA", "CT", "ME",
          "VT", "NH", "RI"].includes(stateCode);
}
```

Ich gebe der Funktion einen Namen, der sich durch einen Suchlauf leicht automatisch durch den ursprünglichen Namen ersetzen lässt. (Wie Sie sehen,

verwende ich für die temporären Namen keine Standardwerte.)

Jetzt wende ich *Variable inline platzieren* (Abschnitt 6.4) auf den Eingabeparameter der ursprünglichen Funktion an.

```
function inNewEngland(aCustomer) {
  return xxinNewEngland(aCustomer.address.state);
}
```

Mit *Funktion inline platzieren* (Abschnitt 6.2) sorge ich dafür, dass die alte Funktion in ihren Aufrufern durch den Aufruf der neuen Funktion ersetzt wird. Diese Vorgänge lassen sich wieder einzeln durchführen.

```
Aufrufer...
const newEnglanders =
  someCustomers.filter(c =>
xxinNewEngland(c.address.state));
```

Nachdem ich die alte Funktion in allen Aufrufern durch *Funktion inline platzieren* (Abschnitt 6.2) ersetzt habe, verwende ich erneut *Funktionsdeklaration ändern* (Abschnitt 6.5), um den Namen der neuen Funktion auf den ursprünglichen Namen zu ändern.

```
Aufrufer...
const newEnglanders =
```

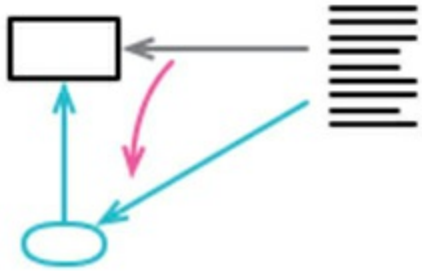
```
   someCustomers.filter(c =>
inNewEngland(c.address.state));

Oberste Ebene...
function inNewEngland(stateCode) {
  return ["MA", "CT", "ME",
          "VT", "NH", "RI"].includes(stateCode);
}
```

Automatische Refactoring-Tools sorgen dafür, dass der Migrationsansatz einerseits weniger nützlich, anderseits aber effektiver wird. Der Ansatz wird weniger nützlich, weil diese Hilfsmittel auch komplizierte Umbenennungen und Änderungen an den Parametern zuverlässig handhaben. Da automatisierte Hilfsmittel verfügbar sind, muss ich den Migrationsansatz nicht mehr so häufig verfolgen. In Fällen wie diesem Beispiel, in denen die automatischen Tools das Refactoring nicht vollständig vornehmen können, stellen sie dennoch eine Erleichterung dar, weil das Extrahieren und das Inline-Platzieren schneller und zuverlässiger erledigt werden können.

6.6 Variable kapseln (Encapsulate Variable)

Ehemalige Bezeichnung: *Feld kapseln* (Encapsulate Field)

Ehemalige Bezeichnung: *Eigenes Feld kapseln* (Self-Encapsulate Field)



```
let defaultOwner = {firstName: "Martin", lastName:
"Fowler"};
```

```
let defaultOwnerData =
                    {firstName: "Martin", lastName:
"Fowler"};
export function defaultOwner()     {return
defaultOwnerData;}
export function setDefaultOwner(arg){defaultOwnerData
= arg;}
```

6.6.1 MOTIVATION

Beim Refactoring geht es immer um die Änderung der Bestandteile von Programmen. Das Ändern von Daten ist umständlicher als das Ändern von Funktionen. Da die Verwendung einer Funktion für gewöhnlich bedeutet, sie aufzurufen, kann ich eine Funktion problemlos umbenennen oder verschieben, solange die alte Funktion vorhanden bleibt und als weiterleitende Funktion agiert.

(Mein alter Code ruft also die alte Funktion auf, die ihrerseits die neue Funktion aufruft.) Ich entferne diese weiterleitende Funktion für gewöhnlich nach kurzer Zeit, sie vereinfacht jedoch das Refactoring.

Das Ändern von Daten ist umständlicher, weil ich nicht auf die gleiche Art und Weise vorgehen kann. Wenn ich Daten verschiebe, muss ich sämtliche Verweise auf die Daten in einem einzigen Schritt anpassen, damit der Code weiterhin funktioniert. Bei Daten mit einem sehr kleinen Gültigkeitsbereich, wie etwa einer temporären Variable in einer kurzen Funktion, ist das unproblematisch. Mit zunehmendem Umfang des Gültigkeitsbereichs steigt auch der Schwierigkeitsgrad an – das ist der Grund dafür, dass globale Daten eine solche Qual sind.

Wenn ich Daten verschieben möchte, auf die von überall zugegriffen wird, besteht der beste Ansatz häufig darin, sie zu kapseln, indem alle Zugriffe durch Funktionen erfolgen müssen. Auf diese Weise wird aus der schwierigen Aufgabe, die Daten neu zu organisieren, die einfachere Aufgabe, Funktionen neu zu organisieren.

Das Kapseln von Daten ist zudem auch anderen Aspekten zuträglich. Es ermöglicht, Änderungen und Gebrauch der Daten zu überwachen. Bei einer Aktualisierung der Daten kann ich problemlos deren Gültigkeit überprüfen oder andere Aktionen ausführen. Ich habe mir zur Gewohnheit gemacht, alle

veränderlichen Daten zu kapseln und einen Zugriff durch Funktionen nur dann zuzulassen, wenn ihr Gültigkeitsbereich größer ist als eine einzelne Funktion. Je größer der Gültigkeitsbereich der Daten ist, desto wichtiger ist die Kapselung. Wenn ich Code überarbeite und einen Verweis auf eine solche Variable ändern oder hinzufügen muss, nutze ich die Gelegenheit, sie zu kapseln. Auf diese Weise kann ich eine zunehmende Kopplung durch gemeinsam genutzte Daten verhindern.

Dieses Prinzip ist der Grund dafür, dass beim objektorientierten Ansatz so großer Wert darauf gelegt wird, dass die Daten des Objekts privat bleiben. Wenn ich auf ein öffentliches Feld treffe, ziehe ich immer in Betracht, *Variable kapseln* (Abschnitt 6.6) zu verwenden, um seine Sichtbarkeit zu verringern. Manche Programmierer gehen sogar noch einen Schritt weiter und fordern, dass auch interne Verweise auf Datenfelder innerhalb einer Klasse durch Zugriffsfunktionen erfolgen sollten – ein Ansatz, der als »Selbstkapselung« (engl. *self-encapsulation*) bezeichnet wird. Im Großen und Ganzen halte ich die Selbstkapselung für übertrieben. Wenn eine Klasse so umfangreich ist, dass die eigenen Felder gekapselt werden müssen, sollte sie ohnehin aufgespalten werden. Die Selbstkapselung eines Feldes kann aber durchaus einen sinnvollen Schritt vor der Aufteilung einer Klasse darstellen.

Das Kapseln ist bei unveränderlichen Daten weit weniger bedeutsam als bei veränderlichen Daten. Wenn sich die

Daten nicht ändern, muss ich auch keine Überprüfungen oder andere Aktionen durchführen, die durch bestimmte Inhalte ausgelöst werden sollen. Darüber hinaus kann ich die Daten nach Belieben kopieren, anstatt sie zu verschieben – dann muss ich weder die Verweise an veralteten Stellen ändern, noch mir Gedanken über Codeabschnitte machen, die veraltete Daten erhalten. Unveränderlichkeit ist ein wirkungsvolles Konservierungsmittel.

6.6.2 VORGEHEN

- Erstellen Sie kapselnde Funktionen zum Lesen und Schreiben der Variable.
- Führen Sie statische Überprüfungen durch (engl. *static checks*, dabei werden automatisch Bugs entdeckt, ohne dass der Code ausgeführt wird).
- Ersetzen Sie alle Verweise auf die Variable durch Aufrufe der entsprechenden kapselnden Funktion. Testen Sie nach jedem Austausch.
- Beschränken Sie die Sichtbarkeit der Variable.

  Unter bestimmten Umständen kann der Zugriff auf die Variable nicht verhindert werden. In diesen Fällen kann es sich als nützlich erweisen, die noch vorhandenen Verweise aufzuspüren, indem Sie die Variable umbenennen und Tests durchführen.
- Testen Sie.
- Wenn die Variable einen Datensatz enthält, sollten Sie in Betracht ziehen, *Datensatz kapseln* (Abschnitt 7.1) zu verwenden.

6.6.3 BEISPIEL

Betrachten Sie folgende Daten, die in einer globalen Variable gespeichert sind.

```
let defaultOwner = {firstName: "Martin", lastName:
"Fowler"};
```

Der Verweis auf die Daten erfolgt wie üblich:

```
spaceship.owner = defaultOwner;
```

Aktualisierungen werden folgendermaßen ausgeführt:

```
defaultOwner = {firstName: "Rebecca", lastName:
"Parsons"};
```

Um eine einfache Kapselung der Daten zu erzielen, definiere ich als Erstes Funktionen zum Lesen und Schreiben der Daten:

```
function getDefaultOwner() {return defaultOwner;}
function setDefaultOwner(arg) {defaultOwner = arg;}
```

Anschließend bearbeite ich Verweise auf `defaultOwner`, die ich durch einen Aufruf der Zugriffsfunktion ersetze:

```
spaceship.owner = getDefaultOwner();
```

Zuweisungen ersetze ich ebenfalls durch eine entsprechende Zugriffsfunktion:

```
setDefaultOwner({firstName: "Rebecca", lastName:
"Parsons"});
```

Nach jedem Ersetzten führe ich Tests durch.

Sobald alle Verweise ersetzt sind, schränke ich die Sichtbarkeit der Variable ein. Auf diese Weise wird zum einen geprüft, ob ich womöglich Verweise auf die Variable übersehen habe, zum anderen ist damit gewährleistet, dass bei zukünftigen Änderungen am Code kein direkter Zugriff auf die Variable erfolgt. In JavaScript kann ich diese Aufgabe erledigen, indem ich sowohl die Variable als auch die Zugriffsmethode in eine eigene Datei verschiebe und nur die Zugriffsmethode exportiere.

```
defaultOwner.js...
let defaultOwner = {firstName: "Martin", lastName:
"Fowler"};
export function getDefaultOwner() {return
defaultOwner;}
export function setDefaultOwner(arg) {defaultOwner =
arg;}
```

Wenn ich den Zugriff auf die Variable aus irgendwelchen Gründen nicht einschränken kann, erweist es sich manchmal als nützlich, die Variable umzubenennen (und Tests auszuführen). Dadurch wird zwar der direkte Zugriff nicht verhindert, aber eine ausdrucksstarke und ungewöhnliche Bezeichnung wie `__privateOnly_defaultOwner` ist vielleicht hilfreich.

Die Verwendung der Vorsilbe `get` für Getter gefällt mir nicht, deshalb benenne ich die Funktion um.

```
defaultOwner.js...
let defaultOwnerData = {firstName: "Martin",
                        lastName: "Fowler"};
export function getDefaultOwner() {return
defaultOwnerData;}
export function setDefaultOwner(arg){defaultOwnerData
= arg;}
```

In JavaScript gibt es die Konvention, die Getter- und die Setter-Funktion gleich zu benennen und sie durch das Vorhandensein eines Arguments zu unterscheiden. Ich bezeichne diesen Ansatz als »überladene Getter/Setter« (`https://martinfowler.com/bliki/OverloadedGetterSetter.html`) – und ich mag ihn gar nicht. Also verwende ich weiterhin die Vorsilbe `set`, verzichte aber auf die Vorsilbe `get`.

Kapselung des Werts

Das hier beschriebene Refactoring kapselt den Verweis auf eine Datenstruktur und ermöglicht mir, den Zugriff darauf und dessen Neuzuweisungen zu kontrollieren. Es kann aber keine Änderung der Datenstruktur selbst verhindern.

```
const owner1 = defaultOwner();
assert.equal("Fowler", owner1.lastName, "when set");
const owner2 = defaultOwner();
owner2.lastName = "Parsons";
assert.equal("Parsons", owner1.lastName,
             "after change owner2"); // Ist das in
Ordnung?
```

Das Refactoring kapselt den Verweis auf das Datenelement. In vielen Fällen ist das fürs Erste ausreichend. Häufig möchte ich mit der Kapselung jedoch nicht nur den Zugriff auf die Variable kontrollieren, sondern auch die Änderung ihres Inhalts.

Hierfür stehen mir mehrere Möglichkeiten zur Verfügung. Am einfachsten ist es, sämtliche Änderungen des Werts zu verhindern. Zu diesem Zweck modifiziere ich vorzugsweise die Getter-Funktion so, dass sie eine Kopie der Daten zurückgibt.

```
defaultOwner.js...
let defaultOwnerData = {firstName: "Martin",
                        lastName: "Fowler"};
export function defaultOwner() {return
Object.assign({},
```

```
defaultOwnerData);}
export function setDefaultOwner(arg){defaultOwnerData
= arg;}
```

Besonders häufig verwende ich diesen Ansatz für Listen. Wenn ich eine Kopie der Daten zurückgebe, kann jeder Client sie ändern, die gemeinsam genutzten Daten bleiben jedoch unangetastet. Ich muss bei der Verwendung von Kopien dennoch vorsichtig sein, weil anderer Code womöglich davon ausgeht, gemeinsam genutzte Daten ändern zu können. Wenn das der Fall ist, muss ich mich auf meine Tests verlassen, um Probleme aufzudecken. Alternativ könnte ich Änderungen untersagen – und dazu ist *Datensatz kapseln* (Abschnitt 7.1) gut geeignet.

```
let defaultOwnerData = {firstName: "Martin",
                          lastName: "Fowler"};
export function defaultOwner() {
                        return new
Person(defaultOwnerData);}
export function setDefaultOwner(arg){defaultOwnerData
= arg;}

  class Person {
    constructor(data) {
      this._lastName = data.lastName;
      this._firstName = data.firstName
    }
    get lastName() {return this._lastName;}
    get firstName() {return this._firstName;}
    // und so weiter für andere Eigenschaften
```

Jetzt verursacht jeder Versuch, die Eigenschaften von `defaultOwner` zu ändern, eine Fehlermeldung. Die verschiedenen Programmiersprachen bieten unterschiedliche Möglichkeiten, Änderungen wie diese zu verhindern. Je nach Sprache würde ich also andere Optionen in Betracht ziehen.

Änderungen auf diese Weise zu erkennen und zu verhindern lohnt sich oft als temporäre Maßnahme. Ich kann die Anpassungen dann entweder rückgängig machen oder aber entsprechende Funktionen zum Ändern bereitstellen. Wenn das erledigt ist, kann ich die Getter-Funktion dahingehend anpassen, dass sie eine Kopie zurückgibt.

Bislang ging es um das Kopieren beim Abfragen von Daten, es könnte jedoch sinnvoll sein, auch in der Setter-Funktion eine Kopie zu erstellen. Das hängt davon ab, woher die Daten stammen und ob ich eine Verknüpfung vorhalten muss, um Änderungen an den ursprünglichen Daten widerzuspiegeln. Wenn ich eine solche Verknüpfung nicht benötige, verhindert eine Kopie versehentliche Änderungen an den Quelldaten. Eine Kopie zu verwenden dürfte in den meisten Fällen überflüssig sein. Das Erstellen von Kopien hat für gewöhnlich allerdings vernachlässigbare Auswirkungen auf die Performance. Wenn ich hingegen auf Kopien verzichte, laufe ich Gefahr, dass mir eine lange und

schwierige Runde Debugging bevorsteht.

Beachten Sie hier, dass sowohl das Erstellen von Kopien als auch das Kapseln in einer Klasse nur dann funktioniert, wenn die Datensatzstruktur nicht tiefer als eine Ebene ist. Weitere Ebenen erfordern zusätzliche Kopien oder weiteres Kapseln.

Wie Sie sehen, ist das Kapseln von Daten erstrebenswert, aber oft nicht ganz unkompliziert. Was genau gekapselt werden sollte – und wie dabei vorzugehen ist –, hängt davon ab, wie die Daten genutzt werden und welche Änderungen ich vorzunehmen gedenke. Je weitreichender die Daten verwendet werden, desto lohnenswerter ist es für mich, richtig zu kapseln.

6.7 Variable umbenennen (Rename Variable)

```
let a = height * width;
```

```
let area = height * width;
```

6.7.1 MOTIVATION

Die Namensgebung ist das Herzstück verständlicher Programmierung. Variablen können viel leisten, wenn es darum geht, deutlich zu machen, was ich vorhabe – sofern ich sie sinnvoll benenne. Dennoch verwende ich immer mal wieder ungeeignete Bezeichnungen – manchmal, weil ich nicht gründlich darüber nachgedacht habe, manchmal, weil ich dazulerne und die Aufgabe besser verstehe, oder manchmal, weil sich durch veränderte Anforderungen der Verwendungszweck des Programms ändert.

Die Bedeutung eines Namens hängt in noch höherem Ausmaß als bei anderen Programmelementen davon ab, wie häufig er verwendet wird. Eine Variable in einem einzeiligen Lambda-Ausdruck ist für gewöhnlich leicht verständlich – ich verwende dafür oft einen einzelnen Buchstaben, weil der Zweck der Variable aus dem Kontext hervorgeht. Aus dem gleichen Grund können die Parameter kurzer Funktionen ebenfalls kurz und knapp sein, wenngleich ich in einer dynamisch typisierten Sprache wie JavaScript bevorzugt den Typ zum Bestandteil des Namens mache (so kommen Parameternamen wie `aCustomer` zustande).

Bei persistenten Feldern, die über einen einzelnen

Funktionsaufruf hinaus bestehen, muss man sich bei der Namensgebung schon etwas mehr Mühe geben. Dem widme ich den größten Teil meiner Aufmerksamkeit.

6.7.2 VORGEHEN

- Wenn die Variable häufig verwendet wird, sollten Sie *Variable kapseln* (Abschnitt 6.6) in Betracht ziehen.
- Suchen Sie nach allen Verweisen auf die Variable und nehmen Sie die Namensänderung vor.

  Wenn es Verweise in anderen Codebasen gibt, handelt es sich um eine öffentliche Variable. Die Anwendung des Refactorings ist in diesem Fall nicht möglich.

  Wenn sich der Wert der Variable nicht ändert, können Sie einer Variable mit dem neuen Namen eine Kopie zuweisen. Gehen Sie schrittweise vor und testen Sie nach jeder Änderung.
- Testen Sie.

6.7.3 BEISPIEL

Im einfachsten Fall handelt es sich um eine lokale Variable in einer einzelnen Funktion: eine temporäre Variable oder ein Argument. Das ist selbst für ein Beispiel zu trivial: Ich suche einfach nach allen Verweisen und ändere sie. Anschließend führe ich die Tests aus, um mich zu vergewissern, dass mir kein Fehler unterlaufen ist. Die Probleme tauchen auf, wenn der Gültigkeitsbereich der Variable mehr als nur eine einzelne Funktion umfasst. Dann gibt es möglicherweise eine ganze Reihe Verweise, die in der gesamten Codebasis verstreut sind:

```
let tpHd = "untitled";
```

Einige der Verweise greifen auf die Variable zu:

```
result += `<h1>${tpHd}</h1>`;
```

Andere aktualisieren sie:

```
tpHd = obj['articleTitle'];
```

Ich rate hier generell dazu, *Variable kapseln* (Abschnitt 6.6) anzuwenden.

```
result += `<h1>${title()}</h1>`;
setTitle(obj['articleTitle']);
  function title() {return tpHd;}
  function setTitle(arg) {tpHd = arg;}
```

Jetzt kann ich die Variable umbenennen.

```
let _title = "untitled";
function title() {return _title;}
function setTitle(arg) {_title = arg;}
```

Ich könnte jetzt damit fortfahren, die Wrapper-Funktionen inline zu platzieren, sodass alle Aufrufer

direkt auf die Variable zugreifen. Das möchte ich jedoch nur selten. Wenn die Variable so weitreichend verwendet wird, dass ich den Eindruck habe, dass ich sie kapseln sollte, um ihren Namen zu ändern, dann lohnt es sich vermutlich, die Kapselung beizubehalten.

In Fällen, in denen ich das Inline-Platzieren vornähme, würde ich bei der Umbenennung keinen Unterstrich verwenden, sondern der Getter-Funktion den Namen `getTitle` geben.

Umbenennen einer Konstanten

Wenn ich eine Konstante (oder eine Variable, die sich gegenüber den Clients wie eine Konstante verhält) umbenenne, kann ich die Kapselung umgehen und dennoch eine schrittweise Umbenennung vornehmen, indem ich sie kopiere. Nehmen wir an, die ursprüngliche Deklaration sieht so aus:

```
const cpyNm = "Acme Gooseberries";
```

Die Umbenennung kann ich mit Erstellung einer Kopie beginnen:

```
const companyName = "Acme Gooseberries";
const cpyNm = companyName;
```

Mit der Kopie kann ich die Verweise allmählich vom

alten auf den neuen Namen ändern. Wenn das erledigt ist, lösche ich die Kopie. Ich ziehe es vor, den neuen Namen zu deklarieren und den alten zu kopieren, wenn es dadurch ein wenig einfacher wird, den alten Namen zu entfernen. Gegebenenfalls füge ich ihn wieder ein, falls ein Test fehlschlägt.

Diese Vorgehensweise ist sowohl auf Konstanten als auch auf schreibgeschützte Variablen anwendbar (wie etwa eine exportierte Variable in JavaScript).

6.8 Parameterobjekt einführen (Introduce Parameter Object)

```
function amountInvoiced(startDate, endDate) {...}
function amountReceived(startDate, endDate) {...}
function amountOverdue(startDate, endDate) {...}
```

```
function amountInvoiced(aDateRange) {...}
function amountReceived(aDateRange) {...}
```

```
function amountOverdue(aDateRange) {...}
```

6.8.1 MOTIVATION

Mir begegnen häufig Gruppierungen von Datenelementen, die regelmäßig gemeinsam auf Reisen gehen und in einer Funktion nach der anderen zusammen auftauchen. Eine solche Gruppe bildet einen Datenklumpen, den ich durch eine einzige Datenstruktur ersetzen möchte.

Daten zu einer Struktur zu gruppieren ist sinnvoll, weil es die Beziehungen zwischen den verschiedenen Datenelementen verdeutlicht. Zudem wird die Parameterliste einer Funktion verkürzt, wenn sie die neue Datenstruktur verwendet. Darüber hinaus trägt die Struktur zur Konsistenz bei, denn alle Funktionen, die sie verwenden, benutzen die gleichen Bezeichner, um an die Elemente zu gelangen.

Die wahre Stärke dieses Refactorings zeigt sich jedoch daran, wie es tiefgreifende Änderungen am Code ermöglicht. Wenn ich derartige neue Strukturen erkenne, kann ich das Verhalten des Programms darauf ausrichten, sie zu verwenden. Ich kann Funktionen erstellen, die das gemeinsame Verhalten bezüglich dieser Daten erfassen – entweder als eine Gruppe zusammenhängender Funktionen oder als eine Klasse, die diese Datenstruktur mit den Funktionen vereint. Dieser Vorgang kann das konzeptionelle Gesamtbild des

Codes verändern, indem diese Strukturen als Abstraktionen hervorgehoben werden, die mir das Verständnis der fachlichen Domäne erheblich erleichtern können. Wenn das gelingt, können sich überraschend weitreichende Effekte ergeben. Allerding muss ich, um dieses Ziel zu erreichen, den Vorgang zunächst einmal mit *Parameterobjekt einführen* anstoßen.

6.8.2 VORGEHEN

- Falls noch keine passende Struktur vorhanden ist: Erstellen Sie eine.

  Ich verwende bevorzugt eine Klasse, weil dadurch die später stattfindende Gruppierung des Verhaltens vereinfacht wird. Für gewöhnlich vergewissere ich mich, dass es sich bei diesen Strukturen um *Value Objects* (Wertobjekte, siehe `https://martinfowler.com/bliki/ValueObject.html`) handelt.

- Testen Sie.
- Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um einen Parameter für die neue Struktur hinzuzufügen.
- Testen Sie.
- Ändern Sie die Aufrufer, sodass sie die richtige Instanz der neuen Struktur übergeben. Testen Sie nach jeder Änderung.
- Ersetzen Sie die ursprünglich verwendeten Parameter der Reihe nach durch die Elemente der neuen Struktur. Löschen Sie die Parameter anschließend und führen Sie die Tests aus.

6.8.3 BEISPIEL

Ich beginne mit einem Code, der eine Reihe von Temperaturmessungen auswertet und feststellt, ob einer

der Werte außerhalb des zulässigen Betriebsbereichs liegt. Die Messwerte sehen folgendermaßen aus:

```
const station = { name: "ZB1",
                  readings: [
                    {temp: 47, time: "2016-11-10
09:10"},
                    {temp: 53, time: "2016-11-10
09:20"},
                    {temp: 58, time: "2016-11-10
09:30"},
                    {temp: 53, time: "2016-11-10
09:40"},
                    {temp: 51, time: "2016-11-10
09:50"},
                  ]
                };
```

Hier ist die Funktion, die nach Messwerten außerhalb des zulässigen Betriebsbereichs sucht:

```
function readingsOutsideRange(station, min, max) {
  return station.readings
    .filter(r => r.temp < min || r.temp > max);
}
```

Sie könnte von Code wie diesem aufgerufen werden:

```
Aufrufer
alerts = readingsOutsideRange(station,

operatingPlan.temperatureFloor,
```

```
operatingPlan.temperatureCeiling);
```

Beachten Sie, wie der aufrufende Code die beiden Datenelemente einem anderen Objekt als Paar entnimmt und dieses Paar an `readingsOutsideRange` übergibt. `operatingPlan` verwendet für die Grenzwerte des zulässigen Temperaturbereichs andere Bezeichnungen als `readingsOutsideRange`. Ein Wertebereich wie dieser ist ein typisches Beispiel für den Fall, dass zwei getrennte Datenelemente besser in einem einzelnen Objekt vereint werden. Ich deklariere als Erstes eine Klasse für die kombinierten Daten:

```
class NumberRange {
  constructor(min, max) {
    this._data = {min: min, max: max};
  }
  get min() {return this._data.min;}
  get max() {return this._data.max;}
}
```

Anstatt ein einfaches JavaScript-Objekt zu verwenden, deklariere ich eine Klasse, weil dieses Refactoring meist den ersten Schritt darstellt, Verhalten in das neu erstellte Objekt zu verlagern. Da es sinnvoll ist, hierfür eine Klasse zu verwenden, erstelle ich diese unmittelbar. Zudem erhält die Klasse keine Methoden zum Aktualisieren des Zustands, weil ich es wahrscheinlich zu einem Value Object machen werde

(https://martinfowler.com/bliki/ValueObject.html). Wenn ich dieses Refactoring verwende, erstelle ich meistens auch ein Value Object.

Jetzt verwende ich *Funktionsdeklaration ändern* (Abschnitt 6.5), um das neue Objekt `readingsOutsideRange` als Parameter hinzuzufügen.

```
function readingsOutsideRange(station, min, max,
range) {
  return station.readings
    .filter(r => r.temp < min || r.temp > max);
}
```

In JavaScript kann ich den Aufrufer unverändert lassen, in anderen Programmiersprachen müsste ich für den neuen Parameter hingegen `null` hinzufügen, was dann etwa so aussieht:

```
Aufrufer
alerts = readingsOutsideRange(station,

operatingPlan.temperatureFloor,

operatingPlan.temperatureCeiling,
                                null);
```

Bislang habe ich noch kein Verhalten geändert, und die Tests sollten bestanden werden. Jetzt ändere ich die Aufrufer, damit sie den richtigen Wertebereich

übergeben:

```
Aufrufer
const range = new
NumberRange(operatingPlan.temperatureFloor,

operatingPlan.temperatureCeiling);
alerts = readingsOutsideRange(station,

operatingPlan.temperatureFloor,

operatingPlan.temperatureCeiling,
                              range);
```

Ich habe noch immer kein Verhalten geändert, denn der Parameter wird ja nicht verwendet. Die Tests sollten ebenfalls noch bestanden werden.

Jetzt kann ich damit beginnen, die Parameter zu ersetzen. Zunächst das Maximum:

```
function readingsOutsideRange(station, min, max,
range) {
  return station.readings
    .filter(r => r.temp < min || r.temp > range.max);
}

Aufrufer
const range = new
NumberRange(operatingPlan.temperatureFloor,

operatingPlan.temperatureCeiling);
alerts = readingsOutsideRange(station,
```

```
operatingPlan.temperatureFloor,

operatingPlan.temperatureCeiling,
                                range);
```

Ich teste und entferne anschließend den anderen Parameter.

```
function readingsOutsideRange(station, min, range) {
  return station.readings
  .filter(r => r.temp < range.min || r.temp >
range.max);
}

Aufrufer
const range = new
NumberRange(operatingPlan.temperatureFloor,

operatingPlan.temperatureCeiling);
alerts = readingsOutsideRange(station,

operatingPlan.temperatureFloor,
                                range);
```

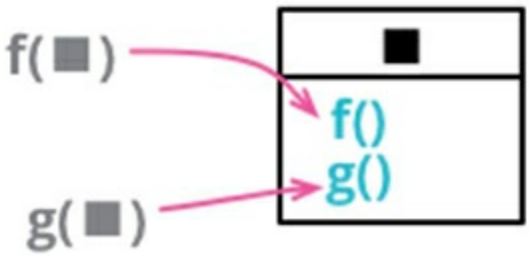
Damit ist dieses Refactoring abgeschlossen. Das Ersetzen einer Reihe von Parametern durch ein Objekt ist jedoch erst der Auftakt für die wirklich interessanten Ausführungen. Eine Klasse wie diese zu verwenden hat den großen Vorteil, dass ich Verhalten in sie auslagern kann. In diesem Fall füge ich eine Methode hinzu, die prüft, ob der Wert innerhalb des gewünschten Bereichs

liegt.

```
function readingsOutsideRange(station, range) {
  return station.readings
    .filter(r => !range.contains(r.temp));
}

class NumberRange...
contains(arg) {return (arg >= this.min && arg <=
this.max);}
```

Dies ist ein erster Schritt für das Erstellen einer Wertebereich-Klasse (`https://martinfowler.com/eaaDev/Range.html`), die eine Menge nützlichen Verhaltens in sich vereinen kann. Seitdem ich die Notwendigkeit für Wertebereiche in meinem Code erkannt habe, halte ich ständig nach weiteren Max/Min-Wertepaaren Ausschau und ersetze sie durch einen Wertebereich. (Eine Möglichkeit wäre beispielsweise, `temperatureFloor` und `temperatureCeiling` in `operatingPlan` durch einen Temperaturbereich `temperatureRange` zu ersetzen.) Wenn ich mir ansehe, wie diese Wertepaare verwendet werden, kann ich ggf. weiteres nützliches Verhalten in die `range`-Klasse verlagern, um so ihre Verwendung in der Codebasis zu vereinfachen. Als Erstes würde ich vermutlich eine Vergleichsmethode hinzufügen, die auf den Werten beruht, nicht auf den Verweisen, um ein echtes Value Object daraus zu machen.

6.9 Funktionen zu einer Klasse vereinen (Combine Functions into Class)




```
function base(aReading) {...}
function taxableCharge(aReading) {...}
function calculateBaseCharge(aReading) {...}
```

```
class Reading {
  base() {...}
  taxableCharge() {...}
  calculateBaseCharge() {...}
}
```

6.9.1 MOTIVATION

Klassen sind in den meisten modernen Programmiersprachen ein fundamentales Konstrukt. Sie verbinden Daten und Funktionen in einer gemeinsam genutzten Umgebung und stellen Teile der Daten und

Funktionen anderen Programmelementen zwecks Zusammenarbeit zur Verfügung. Sie sind in objektorientierten Sprachen das primäre Konstrukt, aber auch für andere Ansätze nützlich.

Wenn mir eine Gruppe eng zusammenarbeitender Funktionen begegnet, die Daten gemeinsam nutzen (meist, indem sie als Argumente übergeben werden), ist das eine Gelegenheit, eine Klasse zu erstellen. Das Verwenden einer Klasse verdeutlicht, dass diese Funktionen eine gemeinsame Umgebung nutzen. Darüber hinaus ermöglicht mir dieses Refactoring, Funktionsaufrufe innerhalb des Objekts zu vereinfachen, indem ich viele der Argumente entferne, und es stellt mir einen Verweis bereit, um ein solches Objekt an andere Teile des Systems zu übergeben.

Das Refactoring organisiert nicht nur die bereits vorhandenen Funktionen neu, sondern bietet auch eine gute Gelegenheit, weitere Berechnungen zu identifizieren und diese in Methoden der neuen Klasse zu verlagern.

Funktionen zu einer Transformation vereinen (Abschnitt 6.10) stellt eine weitere Möglichkeit dar, zusammengehörige Funktionen zu organisieren. Welchen Ansatz man verwenden sollte, hängt vom erweiterten Kontext des Programms ab. Die Verwendung einer Klasse hat den entscheidenden Vorteil, dass die Clients zum Objekt gehörige Daten verändern können und abgeleitete Klassen konsistent bleiben.

Wie bei einer Klasse können Funktionen wie die oben genannten zu einer verschachtelten Funktion kombiniert werden. In den meisten Fällen ziehe ich eine Klasse einer verschachtelten Funktion vor, weil es schwierig sein kann, ineinander verschachtelte Funktionen zu testen. Zudem sind Klassen erforderlich, wenn ich den Clients mehr als eine der Funktionen zur Verfügung stellen möchte.

Sprachen, die keine Klassen als First-Class-Elemente besitzen, wohl aber über First-Class-Funktionen verfügen, verwenden oft *Funktion als Objekt* (Function As Object, siehe `https://martinfowler.com/bliki/FunctionAsObject.html`), um diese Fähigkeit bereitzustellen.

6.9.2 VORGEHEN

- Wenden Sie *Datensatz kapseln* (Abschnitt 7.1) auf den Datensatz an, den die Funktionen gemeinsam nutzen.

  Wenn die Daten, die den Funktionen gemeinsam sind, noch nicht in einer Datensatzstruktur gruppiert sind, können Sie *Parameterobjekt einführen* (Abschnitt 6.8) anwenden, um einen solchen Datensatz zu erstellen.

- Verschieben Sie alle Funktionen, die den gemeinsam genutzten Datensatz verwenden, mit *Funktion verschieben* (Abschnitt 8.1) in die neue Klasse.

  Argumente des Funktionsaufrufs, die Membervariablen sind, können aus der Argumentliste entfernt werden.

- Die gesamte Logik zur Verarbeitung der Daten kann mit *Funktion extrahieren* (Abschnitt 6.1) extrahiert und in die neue Klasse

verlagert werden.

6.9.3 BEISPIEL

Ich bin in England aufgewachsen, einem Land, das für seine Liebe zum Tee bekannt ist. (Ich persönlich mag die meisten Tees, die in England getrunken werden, nicht besonders, habe inzwischen aber Gefallen an chinesischen und japanisches Tees gefunden.) Das wird wohl der Grund dafür sein, dass meine Fantasie einen Staatsbetrieb für die Versorgung der Bevölkerung mit Tee heraufbeschwört. Der Teebestand wird monatlich abgelesen, und dabei ergeben sich Datensätze wie dieser:

```
reading = {customer: "ivan", quantity: 10,
           month: 5, year: 2017};
```

Ich sehe mir den Code zur Verarbeitung dieser Datensätze an und stelle fest, dass an vielen Stellen ähnliche Berechnungen mit den Daten ausgeführt werden. Beispielsweise dieser Code, der den Grundpreis (`baseCharge`) berechnet:

```
Client 1...
const aReading = acquireReading();
const baseCharge = baseRate(aReading.month,
aReading.year) *
                             aReading.quantity;
```

Da es hier um England geht, müssen natürlich alle wichtigen Dinge versteuert werden, so auch der Tee. Das Regelwerk sieht jedoch vor, dass eine Grundmenge Tee steuerfrei ist:

```
Client 2...
const aReading = acquireReading();
const base = (baseRate(aReading.month, aReading.year)
*
                        aReading.quantity);
const taxableCharge =
             Math.max(0, base -
taxThreshold(aReading.year));
```

Ihnen ist sicherlich, ebenso wie mir, sofort aufgefallen, dass die Formel für den Grundpreis in beiden Codeabschnitten vorhanden ist. Wenn es Ihnen wie mir geht, möchten Sie sofort *Funktion extrahieren* (Abschnitt 6.1) anwenden. Interessanterweise sieht es jedoch ganz danach aus, als ob diese Aufgabe bereits an anderer Stelle erledigt wurde:

```
Client 3...
const aReading = acquireReading();
const basicChargeAmount =
calculateBaseCharge(aReading);
function calculateBaseCharge(aReading) {
  return baseRate(aReading.month, aReading.year) *
                  aReading.quantity;
}
```

In Anbetracht dieses Umstands möchte ich am liebsten die anderen beiden Abschnitte des Client-Codes so ändern, dass sie diese Funktion verwenden. Das Problem bei solchen Funktionen auf oberster Ebene ist jedoch, dass sie leicht übersehen werden. Besser sollte der Code dahingehend geändert werden, dass die Funktion in engerer Beziehung zu den von ihr verarbeiteten Daten steht. Das lässt sich auf sinnvolle Weise erreichen, indem aus den Daten eine Klasse erzeugt wird.

Zu diesem Zweck verwende ich *Datensatz kapseln* (Abschnitt 7.1).

```
class Reading {
  constructor(data) {
    this._customer = data.customer;
    this._quantity = data.quantity;
    this._month = data.month;
    this._year = data.year;
  }
  get customer() {return this._customer;}
  get quantity() {return this._quantity;}
  get month()    {return this._month;}
  get year()     {return this._year;}
}
```

Beim Verschieben des Verhaltens fange ich mit der bereits vorhandenen Funktion an: `calculateBaseCharge`.

Wenn ich die neue Klasse verwenden möchte, muss ich

sie zum Einsatz bringen, sobald mir neue Daten vorliegen.

```
Client 3...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const basicChargeAmount =
calculateBaseCharge(aReading);
```

Anschließend verwende ich *Funktion verschieben* (Abschnitt 8.1), um `calculateBaseCharge` in die neue Klasse zu verschieben:

```
class Reading...
get calculateBaseCharge() {
  return baseRate(this.month, this.year) *
this.quantity;
}

Client 3...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const basicChargeAmount =
aReading.calculateBaseCharge;
```

Und wenn ich schon dabei bin, verwende ich *Funktionsdeklaration ändern* (Abschnitt 6.5), um alles an meinen persönlichen Geschmack anzupassen:

```
get baseCharge() {
return baseRate(this.month, this.year) *
```

```
this.quantity;
}

Client 3...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const basicChargeAmount = aReading.baseCharge;
```

Dank dieser Namensgebung kann der Client der `reading`-Klasse nicht unterscheiden, ob der Grundpreis ein Feld oder ein abgeleiteter Wert ist. Und das ist eine feine Sache – nämlich das *Prinzip des einheitlichen Zugriffs* (Uniform Access Principle, siehe https://martinfowler.com/bliki/UniformAccessPrinciple.html).

Ich ändere nun den ersten Client dahingehend, dass er die Methode aufruft, anstatt die Berechnung zu wiederholen.

```
Client 1...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const baseCharge = aReading.baseCharge;
```

Die Chancen stehen nicht schlecht, dass ich *Variable inline platzieren* (Abschnitt 6.4) auf die Variable `baseCharge` anwenden kann, bevor die Sonne untergeht. Von größerer Bedeutung für dieses Refactoring ist jedoch der Client, der den zu

versteuernden Betrag berechnet. Hier besteht mein erster Schritt darin, die neue Eigenschaft `baseCharge` zu verwenden.

```
Client 2...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const taxableCharge = Math.max(0, aReading.baseCharge
-

taxThreshold(aReading.year));
```

Auf die Berechnung des zu versteuernden Betrags wende ich *Funktion extrahieren* (Abschnitt 6.1) an.

```
function taxableChargeFn(aReading) {
  return Math.max(0, aReading.baseCharge -
                  taxThreshold(aReading.year));
}

Client 3...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const taxableCharge = taxableChargeFn(aReading);
```

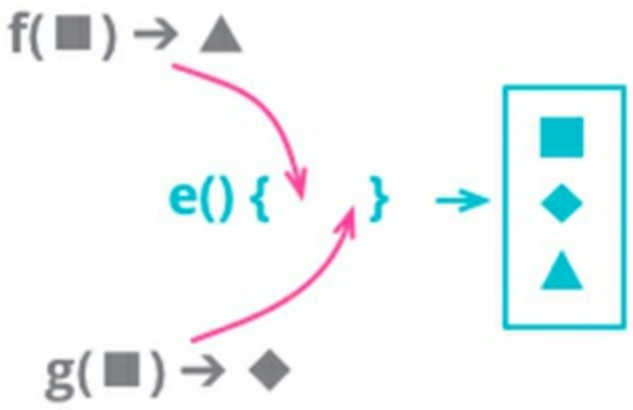
Anschließend verwende ich *Funktion verschieben* (Abschnitt 8.1):

```
class Reading...
get taxableCharge() {
return Math.max(0, this.baseCharge -
```

```
                  taxThreshold(this.year)); }

Client 3...
const rawReading = acquireReading();
const aReading = new Reading(rawReading);
const taxableCharge = aReading.taxableCharge;
```

Da alle abgeleiteten Daten bei Bedarf berechnet werden, entstehen auch keine Probleme, falls ich die gespeicherten Daten aktualisieren muss. Im Allgemeinen bevorzuge ich unveränderliche Daten, aber häufig erfordern die Umstände veränderliche Daten. (Beispielsweise ist JavaScript ein Sprach-Ökosystem, bei dessen Entwicklung kein besonderer Wert auf unveränderliche Daten gelegt wurde.) Wenn es einigermaßen wahrscheinlich ist, dass die Daten irgendwo im Programm aktualisiert werden, dann ist eine Klasse äußerst hilfreich.

6.10 Funktionen zu einer Transformation vereinen (Combine Functions into Transform)




```
function base(aReading) {...}
function taxableCharge(aReading) {...}
```

```
function enrichReading(argReading) {
  const aReading = _.cloneDeep(argReading);
  aReading.baseCharge = base(aReading);
  aReading.taxableCharge = taxableCharge(aReading);
  return aReading;
}
```

6.10.1 MOTIVATION

Programme werden häufig mit Daten gefüttert, aus denen sie dann verschiedene abgeleitete Werte berechnen. Diese abgeleiteten Werte werden möglicherweise an mehreren Stellen benötigt, deshalb erfolgen die dafür notwendigen Berechnungen oft mehrfach, nämlich jeweils dort, wo sie gerade gebraucht

werden. Ich ziehe es vor, diese Berechnungen an einem Ort zu bündeln. So weiß ich, wo sie zu finden sind, wenn ich sie aktualisieren möchte, und vermeide zudem redundante Logik.

Zu diesem Zweck kann man Datentransformationsfunktionen verwenden, die die Ausgangsdaten als Eingabe entgegennehmen und alle abgeleiteten Werte berechnen, die dann in Feldern der Ausgabedaten abgelegt werden. Wenn ich die Berechnungen in Augenschein nehmen möchte, muss ich lediglich einen Blick auf die Transformationsfunktion werfen.

Als Alternative zu *Funktionen zu einer Transformation vereinen* bietet sich *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) an, das die Programmlogik in die Methoden einer Klasse verlagert, die anhand der Ausgangsdaten erstellt wird. Beide Refactorings sind nützlich, und welches ich wähle, hängt oft von dem in der Software vorherrschenden Programmierstil ab. Es gibt jedoch einen wichtigen Unterschied: Wenn der Code die Ausgangsdaten aktualisiert, ist die Verwendung einer Klasse sehr viel besser geeignet. Bei der Verwendung einer Transformation werden abgeleitete Daten im neuen Datensatz abgelegt. Wenn sich die Ausgangsdaten anschließend ändern, kommt es folglich zu Inkonsistenzen.

Funktionen zu vereinen bietet den Vorteil, dass dadurch

Redundanzen in der Logik zur Berechnung der abgeleiteten Werte vermieden werden. Ich muss lediglich *Funktion extrahieren* (Abschnitt 6.1) auf die Programmlogik anwenden. Allerdings ist es oft schwierig, die betreffenden Funktionen zu finden, wenn sie sich nicht in der Nähe der Datenstrukturen befinden, die sie verarbeiten. Das Verwenden einer Transformation (oder einer Klasse) erleichtert es, sie aufzuspüren und zu nutzen.

6.10.2 VORGEHEN

- Erstellen Sie eine Transformationsfunktion, die den zu transformierenden Datensatz entgegennimmt und diese Daten auch zurückgibt.

  Dazu ist im Allgemeinen eine »tiefe« Kopie des Datensatzes erforderlich. Oft ist es sinnvoll, einen Test zu schreiben, der gewährleistet, dass der ursprüngliche Datensatz nicht geändert wird.

- Wählen Sie die Programmlogik aus und verschieben Sie diese in die Transformation, um ein neues Feld im Datensatz anzulegen. Ändern Sie den Client-Code so, dass er auf das neue Feld zugreift.

  Verwenden Sie zunächst *Funktion extrahieren* (Abschnitt 6.1), falls die Programmlogik komplex sein sollte.

- Testen Sie.

- Wiederholen Sie den Vorgang gegebenenfalls für weitere relevante Funktionen.

6.10.3 BEISPIEL

Dort, wo ich aufgewachsen bin, spielt Tee eine bedeutende Rolle – so bedeutend, dass eine eigene

Institution, die der Bevölkerung Tee bereitstellt und wie ein Versorgungsbetrieb für Wasser, Gas oder Strom geführt wird, gar nicht so abwegig erscheint. Der Teeverbrauch jedes Kunden wird monatlich abgelesen und dem Versorgungsbetrieb übermittelt.

```
reading = {customer: "ivan", quantity: 10,
           month: 5, year: 2017};
```

An verschiedenen Stellen berechnet der Code diverse Positionen, die sich aus diesem Teeverbrauch ergeben. Eine dieser Berechnungen ermittelt den Grundbetrag, der bei der Rechnungsstellung zugrunde gelegt wird:

```
Client 1...
const aReading = acquireReading();
const baseCharge = baseRate(aReading.month,
aReading.year) *
                             aReading.quantity;
```

Eine andere Berechnung ermittelt den zu versteuernden Betrag – der geringer ist als der Grundbetrag, weil die Regierung wohlweislich berücksichtigt, dass jeder Bürger eine gewisse Menge Tee steuerfrei erhält:

```
Client 2...
const aReading = acquireReading();
const base = (baseRate(aReading.month, aReading.year)
*
```

```
                        aReading.quantity);
const taxableCharge =
             Math.max(0, base -
taxThreshold(aReading.year));
```

Beim Betrachten des Codes stelle ich fest, dass an mehreren Stellen identische Berechnungen ausgeführt werden. Redundanzen dieser Art sorgen immer wieder für Probleme, wenn der Code geändert werden muss (und ich meine »wenn«, nicht »falls«). Ich kann die Redundanz auflösen, indem ich *Funktion extrahieren* (Abschnitt 6.1) auf diese Berechnungen anwende. Allerdings sind die extrahierten Funktionen anschließend oft weit über das Programm verstreut und erschweren es zukünftigen Entwicklern, ihr Vorhandensein zu bemerken. Und tatsächlich entdecke ich eine solche Funktion in einem anderen Codeabschnitt:

```
Client 3...
const aReading = acquireReading();
const basicChargeAmount =
calculateBaseCharge(aReading);
function calculateBaseCharge(aReading) {
  return baseRate(aReading.month, aReading.year) *
                  aReading.quantity;
}
```

Eine Möglichkeit, damit umzugehen, besteht darin, alle diese Berechnungen in einen Transformationsschritt zu

verlagern, der die Rohdaten einer Ablesung entgegennimmt und einen Datensatz mit allen abgeleiteten Informationen zurückliefert.

Ich erstelle zunächst eine Transformationsfunktion, die das Eingabeobjekt lediglich kopiert:

```
function enrichReading(original) {
  const result = _.cloneDeep(original);
  return result;
}
```

Ich verwende zum Erstellen einer »tiefen« Kopie `cloneDeep` *aus der JavaScript-Bibliothek Lodash.*

Eine Transformation, die im Wesentlichen die Eingabe reproduziert und um zusätzliche Informationen erweitert, benenne ich mit »enrich«. Hätte ich den Eindruck, dass sie etwas Andersartiges erzeugt, würde ich »transform« verwenden.

Ich wähle anschließend eine der Berechnungen aus, die ich ändern möchte. Als Erstes rufe ich `enrichReading` mit den Rohdaten auf, wodurch aktuell noch nichts geschieht:

```
Client 3...
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const basicChargeAmount =
```

```
calculateBaseCharge(aReading);
```

Ich wende *Funktion verschieben* (Abschnitt 8.1) auf `calculateBaseCharge` an, um sie in `enrichReading` zu verschieben.

```
function enrichReading(original) {
  const result = _.cloneDeep(original);
  result.baseCharge = calculateBaseCharge(result);
  return result;
}
```

Innerhalb der Transformationsfunktion kann ich das `result`-Objekt ganz unbekümmert manipulieren, anstatt es jedes Mal erst kopieren zu müssen. Ich bin ja eigentlich ein Freund unveränderlicher Daten, aber die meisten gängigen Programmiersprachen erschweren es, sie zu verwenden. Ich bin bereit, den zusätzlichen Aufwand in Kauf zu nehmen, wenn beispielsweise Modulgrenzen überschritten werden, aber in kleineren Gültigkeitsbereichen werde ich Daten durchaus verändern. Außerdem wähle ich die Namen so, dass die Verschiebung des Codes in die Transformationsfunktion vereinfacht wird (wie hier `aReading` als die Ergänzungen akkumulierende Variable).

Ich ändere den Client, der die Funktion verwendet, sodass er stattdessen das ergänzte Feld verwendet:

```
Client 3...
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const basicChargeAmount = aReading.baseCharge;
```

Nachdem ich alle Aufrufe von `calculateBaseCharge` verschoben habe, kann ich die Funktion in `enrichReading` verschachteln. Dadurch wird verdeutlicht, dass Clients, die diesen Wert benötigen, den erweiterten Datensatz verwenden sollten.

Hier gibt es einen Stolperstein: Wenn ich `enrichReading` so wie hier programmiere, nämlich dass die Funktion den erweiterten Datensatz zurückgibt, impliziere ich damit, dass der ursprüngliche Datensatz nicht verändert wird. Es wäre also sinnvoll, dafür einen Test hinzuzufügen.

```
it('check reading unchanged', function() {
  const baseReading = {customer: "ivan", quantity: 15,
                       month: 5, year: 2017};
  const oracle = _.cloneDeep(baseReading);
  enrichReading(baseReading);
  assert.deepEqual(baseReading, oracle);
});
```

Anschließend kann ich Client 1 ändern, sodass er ebenfalls das Feld verwendet:

```
Client 1...
```

```
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const baseCharge = aReading.baseCharge;
```

Die Chancen stehen nicht schlecht, dass ich anschließend *Variable inline platzieren* (Abschnitt 6.4) auch auf `baseCharge` anwenden kann.

Jetzt wende ich mich der Berechnung des zu versteuernden Betrags zu. Als Erstes füge ich die Transformationsfunktion hinzu:

```
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const base = (baseRate(aReading.month, aReading.year)
*
                        aReading.quantity);
const taxableCharge =
             Math.max(0, base -
taxThreshold(aReading.year));
```

Ich kann die Berechnung des Grundbetrags sofort durch das neue Feld ersetzen. Wenn die Berechnung komplex wäre, könnte ich zunächst *Funktion extrahieren* (Abschnitt 6.1) anwenden, in diesem Fall ist sie jedoch so trivial, dass sie in einem Schritt durchführbar ist.

```
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const base = aReading.baseCharge;
const taxableCharge =
```

```
              Math.max(0, base -
taxThreshold(aReading.year));
```

Nachdem ich getestet habe, ob alles funktioniert, wende ich *Variable inline platzieren* (Abschnitt 6.4) an:

```
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const taxableCharge = Math.max(0, aReading.baseCharge
-

taxThreshold(aReading.year));
```

Diese Berechnung verschiebe ich in die Transformation:

```
function enrichReading(original) {
  const result = _.cloneDeep(original);
  result.baseCharge = calculateBaseCharge(result);
  result.taxableCharge = Math.max(0, result.baseCharge
-

taxThreshold(result.year));
  return result;
```

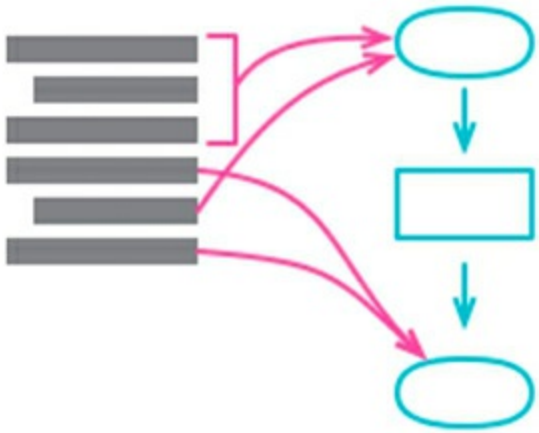
Den ursprünglichen Code passe ich jetzt so an, dass er das neue Feld verwendet:

```
const rawReading = acquireReading();
const aReading = enrichReading(rawReading);
const taxableCharge = aReading.taxableCharge;
```

Nachdem ich das getestet habe, könnte ich vermutlich auch noch *Variable inline platzieren* (Abschnitt 6.4) auf `taxableCharge` anwenden.

Ein Problem besteht allerdings mit einem erweiterten Datensatz wie diesem: Was geschieht, wenn ein Client einen Wert ändert? Wenn er beispielsweise das Feld für die Menge (`quantity`) änderte, würde das zu inkonsistenten Daten führen. Um das zu vermeiden, wäre in JavaScript wohl die beste Wahl, stattdessen auf *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) zurückzugreifen. Wenn ich in einer Sprache mit unveränderlichen Datenstrukturen programmiere, entfällt dieses Problem, deshalb sind Transformationen in derartigen Sprachen gebräuchlicher. Ich kann Transformationen aber auch in Sprachen ohne unveränderliche Datenstrukturen verwenden, wenn die Daten ausschließlich in einem schreibgeschützten Umfeld verwendet werden, beispielsweise bei der Anzeige abgeleiteter Informationen auf einer Webseite.

6.11 Phase aufteilen (Split Phase)



```
const orderData = orderString.split(/\s+/);
const productPrice = priceList[orderData[0].split("-")
[1]];
const orderPrice = parseInt(orderData[1]) *
productPrice;
```

```
const orderRecord = parseOrder(order);
const orderPrice = price(orderRecord, priceList);

function parseOrder(aString) {
  const values = aString.split(/\s+/);
  return ({
    productID: values[0].split("-")[1],
    quantity: parseInt(values[1]), });
}
function price(order, priceList) {
  return order.quantity * priceList[order.productID];
}
```

6.11.1 MOTIVATION

SmWenn mir Code begegnet, der zwei verschiedene Aufgaben erledigt, suche ich nach einer Möglichkeit, ihn in zwei getrennte Module zu unterteilen. Ich versuche, diese Aufteilung vorzunehmen, weil ich mich anschließend mit den unterschiedlichen Themengebieten einzeln befassen kann und nicht beide gleichzeitig im Kopf haben muss, wenn ich eine Änderung durchführe. Wenn ich Glück habe, muss ich nur an einem der Module Änderungen vornehmen und brauche mir um das andere gar keine Gedanken zu machen.

Eine solche Aufteilung lässt sich am besten erreichen, indem das Verhalten in zwei aufeinanderfolgende Phasen aufgeteilt wird. Ein schönes Beispiel hierfür ist ein Verarbeitungsvorgang, bei dem die Eingaben nicht dem Modell entsprechen, das für die Ausführung der Programmlogik benötigt wird. Bevor Sie richtig loslegen, können Sie die Eingabewerte so aufbereiten, dass sie in einer für die eigentliche Weiterverarbeitung geeigneten Form vorliegen.

Sie können die erforderliche Programmlogik auch in zwei aufeinanderfolgende Schritte aufteilen, die sich hinsichtlich ihrer Aufgabe maßgeblich unterscheiden.

Das nächstliegende Beispiel hierfür ist ein Compiler. Er hat grundsätzlich die Aufgabe, einen Text (Code in einer Programmiersprache) in eine ausführbare Form

umzuwandeln (wie etwa Objektcode für eine spezifische Hardware). Inzwischen wissen wir, dass sich dieser Vorgang auf sinnvolle Weise in eine Reihe verschiedener Phasen aufteilen lässt: Tokenisierung des Texts, Aufbau eines Syntaxbaums anhand der Tokens, verschiedene Schritte zur Transformation des Syntaxbaums (z.B. zur Optimierung) und schließlich das Erzeugen des Objektcodes. Jeder dieser Schritte umfasst nur einen begrenzten Umfang, und ich kann über einen der Schritte nachdenken, ohne die Details der anderen verstehen zu müssen.

Die Aufteilung in Phasen ist bei umfangreicher Software gängige Praxis; die verschiedenen Phasen eines Compilers können jeweils sehr viele Funktionen und Klassen enthalten. Das Refactoring *Phase aufteilen* kann ich hingegen auf beliebige Codeabschnitte anwenden – nämlich jedes Mal, wenn ich eine Möglichkeit erkenne, den Code auf sinnvolle Weise in verschiedene Phasen aufzuteilen. Wenn verschiedene Teile des Codeabschnitts auf völlig verschiedene Daten und Funktionen zugreifen, ist das der deutlichste Hinweis darauf, dass ein Refactoring angebracht ist. Wenn ich daraus separate Module erstelle, werden diese Unterschiede offensichtlich, weil der Code sie zu erkennen gibt.

6.11.2 VORGEHEN

- Extrahieren Sie den Code der zweiten Phase in eine eigene Funktion.

- Testen Sie.
- Erstellen Sie eine temporäre Datenstruktur als zusätzlichen Parameter für die extrahierte Funktion.
- Testen Sie.
- Untersuchen Sie die Parameter der extrahierten zweiten Phase. Verschieben Sie Parameter, die von der ersten Phase verwendet werden, in die temporäre Datenstruktur. Führen Sie nach jedem Verschieben Tests durch.

  Unter bestimmten Umständen sollte ein Parameter nicht von der zweiten Phase verwendet werden. Extrahieren Sie in diesem Fall die Ergebnisse der Verwendung des Parameters in ein Feld der temporären Datenstruktur und wenden Sie *Anweisungen in Aufrufer verschieben* (Abschnitt 8.4) auf die Codezeile an, die dieses Feld befüllt.
- Wenden Sie *Funktion extrahieren* (Abschnitt 6.1) auf den Code der ersten Phase an und geben Sie in der Funktion die temporäre Datenstruktur zurück.

  Es kann auch durchaus sinnvoll sein, aus dem Code der ersten Phase ein Transformationsobjekt zu erstellen.

6.11.3 BEISPIEL

Hier ist zunächst einmal der Code, der den Preis für eine Bestellung verschiedener unkonkreter und unbedeutender Produkte ermittelt:

```
function priceOrder(product, quantity, shippingMethod)
{
  const basePrice = product.basePrice * quantity;
  const discount =
         Math.max(quantity -
product.discountThreshold, 0) *
                  product.basePrice *
```

```
product.discountRate;
  const shippingPerCase =
              (basePrice >
shippingMethod.discountThreshold)
              ? shippingMethod.discountedFee :
                shippingMethod.feePerCase;
  const shippingCost = quantity * shippingPerCase;
  const price = basePrice - discount + shippingCost;
  return price;
}
```

Hierbei handelt es sich zwar um die übliche Art eines trivialen Beispiels, aber dennoch kann man sich nicht des Eindrucks erwehren, dass hier zwei Phasen ablaufen. Die ersten Codezeilen verwenden die Produktinformationen, um den produktbezogenen Preis der Bestellung zu berechnen. Die nachfolgenden Zeilen hingegen verwenden Informationen über die Versandmethode zur Berechnung der Versandkosten. Wenn bevorstehende Änderungen die Berechnungen von Produktpreis und Versandkosten verkomplizieren, diese aber voneinander unabhängig sind, dann ist eine Aufteilung des Codes in zwei Phasen sinnvoll.

Ich beginne damit, das Refactoring *Funktion extrahieren* (Abschnitt 6.1) auf die Berechnung der Versandkosten anzuwenden.

```
function priceOrder(product, quantity, shippingMethod)
{
  const basePrice = product.basePrice * quantity;
  const discount =
```

```
         Math.max(quantity -
product.discountThreshold, 0) *
                  product.basePrice *
product.discountRate;
  const price = applyShipping(basePrice,
shippingMethod,
                              quantity, discount);
  return price;
}
function applyShipping(basePrice, shippingMethod,
                       quantity, discount) {
  const shippingPerCase =
              (basePrice >
shippingMethod.discountThreshold)
              ? shippingMethod.discountedFee :
                shippingMethod.feePerCase;
  const shippingCost = quantity * shippingPerCase;
  const price = basePrice - discount + shippingCost;
  return price;
}
```

Ich übergebe alle von der zweiten Phase benötigten Daten als einzelne Parameter. In einem realistischeren Fall kann es davon eine ganze Reihe geben, darüber zerbreche ich mir aber nicht den Kopf, denn ich werde ihre Anzahl nach und nach reduzieren.

Jetzt kommt die temporäre Datenstruktur ins Spiel, die zur Kommunikation zwischen den beiden Phasen dient:

```
function priceOrder(product, quantity, shippingMethod)
{
  const basePrice = product.basePrice * quantity;
  const discount =
```

```
         Math.max(quantity -
product.discountThreshold, 0) *
                  product.basePrice *
product.discountRate;
  const priceData = {};
  const price = applyShipping(priceData, basePrice,
                        shippingMethod, quantity,
discount);
  return price;
}
function applyShipping(priceData, basePrice,
shippingMethod,
                       quantity, discount) {
  const shippingPerCase =
              (basePrice >
shippingMethod.discountThreshold)
              ? shippingMethod.discountedFee :
                shippingMethod.feePerCase;
  const shippingCost = quantity * shippingPerCase;
  const price = basePrice - discount + shippingCost;
  return price;
}
```

Anschließend betrachte ich die verschiedenen Parameter von `applyShipping`. Der erste ist der Grundpreis (`basePrice`) und wird im Code der ersten Phase erstellt. Ich verlagere ihn also in die temporäre Datenstruktur und entferne ihn aus der Parameterliste:

```
function priceOrder(product, quantity, shippingMethod)
{
  const basePrice = product.basePrice * quantity;
  const discount =
         Math.max(quantity -
product.discountThreshold, 0) *
```

```
                    product.basePrice *
product.discountRate;
  const priceData = {basePrice: basePrice};
  const price = applyShipping(priceData, basePrice,
                         shippingMethod, quantity,
discount);
  return price;
}
function applyShipping(priceData, basePrice,
shippingMethod,
                        quantity, discount) {
  const shippingPerCase = (priceData.basePrice >

shippingMethod.discountThreshold)
                            ?
shippingMethod.discountedFee :

shippingMethod.feePerCase;
  const shippingCost = quantity * shippingPerCase;
  const price = priceData.basePrice
                - discount + shippingCost;
  return price;
}
```

Der nächste Parameter in der Liste ist die Versandmethode (`shippingMethod`). Diesen Parameter lasse ich unangetastet, denn er wird in der ersten Phase nicht verwendet.

Jetzt ist die Anzahl (`quantity`) an der Reihe. Der Parameter wird in der ersten Phase zwar verwendet, aber nicht dort erstellt. Ich könnte ihn also eigentlich in der Parameterliste belassen. Ich ziehe es jedoch für gewöhnlich vor, so viel wie möglich in die temporäre

Datenstruktur auszulagern.

```
function priceOrder(product, quantity, shippingMethod)
{
  const basePrice = product.basePrice * quantity;
  const discount =
         Math.max(quantity -
product.discountThreshold, 0) *
                  product.basePrice *
product.discountRate;
  const priceData = {basePrice: basePrice,
                     quantity: quantity};
  const price = applyShipping(priceData,
shippingMethod,
                              quantity, discount);
  return price;
}
function applyShipping(priceData, shippingMethod,
                       quantity, discount) {
  const shippingPerCase = (priceData.basePrice >

shippingMethod.discountThreshold)
                            ?
shippingMethod.discountedFee :

shippingMethod.feePerCase;
  const shippingCost = priceData.quantity *
shippingPerCase;
  const price = priceData.basePrice
                - discount + shippingCost;
  return price;
}
```

Mit dem Mengenrabatt (`discount`) verfahre ich auf die gleiche Weise:

```
function priceOrder(product, quantity, shippingMethod)
{
  const basePrice = product.basePrice * quantity;
  const discount =
         Math.max(quantity -
product.discountThreshold, 0) *
                  product.basePrice *
product.discountRate;
  const priceData = {basePrice: basePrice,
                     quantity: quantity,
                     discount: discount};
  const price = applyShipping(priceData,
shippingMethod,
                              discount);
  return price;
}
function applyShipping(priceData, shippingMethod,
discount) {
  const shippingPerCase = (priceData.basePrice >

shippingMethod.discountThreshold)
                           ?
shippingMethod.discountedFee :

shippingMethod.feePerCase;
  const shippingCost = priceData.quantity *
shippingPerCase;
  const price = priceData.basePrice
                - priceData.discount + shippingCost;
  return price;
}
```

Nachdem ich alle Funktionsparameter abgearbeitet habe, ist die temporäre Datenstruktur vollständig. Jetzt kann ich den Code der ersten Phase extrahieren und eine eigene Funktion erstellen, die diese Daten zurückgibt.

```
function priceOrder(product, quantity, shippingMethod)
{
  const priceData = calculatePricingData(product,
quantity);
  const price = applyShipping(priceData,
shippingMethod);
  return price;
}
function calculatePricingData(product, quantity) {
  const basePrice = product.basePrice * quantity;
  const discount =
           Math.max(quantity -
product.discountThreshold, 0)
                 * product.basePrice *
product.discountRate;
  return {basePrice: basePrice,
          quantity: quantity,
          discount:discount};
}
function applyShipping(priceData, shippingMethod) {
  const shippingPerCase = (priceData.basePrice >

shippingMethod.discountThreshold)
                            ?
shippingMethod.discountedFee :

shippingMethod.feePerCase;
  const shippingCost = priceData.quantity *
shippingPerCase;
  const price = priceData.basePrice -
priceData.discount
                + shippingCost;
  return price;
}
```

Ich kann nicht widerstehen, auch noch die Konstanten

aufzuräumen:

```
function priceOrder(product, quantity, shippingMethod)
{
  const priceData = calculatePricingData(product,
quantity);
  return applyShipping(priceData, shippingMethod);
}
function calculatePricingData(product, quantity) {
  const basePrice = product.basePrice * quantity;
  const discount =
           Math.max(quantity -
product.discountThreshold, 0)
                  * product.basePrice *
product.discountRate;
  return {basePrice: basePrice,
          quantity: quantity,
          discount:discount};
}
function applyShipping(priceData, shippingMethod) {
  const shippingPerCase = (priceData.basePrice >

shippingMethod.discountThreshold)
                          ?
shippingMethod.discountedFee :
                            shippingMethod.feePerCase;
  const shippingCost = priceData.quantity *
shippingPerCase;
  return priceData.basePrice - priceData.discount
          + shippingCost;
}
```

Kapitel 7

Kapselung

Die vielleicht wichtigsten Kriterien bei der Aufteilung in Module sind die »Geheimnisse«, die ein Modul vor dem übrigen System verbergen sollte (`https://dl.acm.org/citation.cfm?id=361623`). Dabei handelt es sich meistens um Datenstrukturen, die ich durch Kapselung mit *Datensatz kapseln* (Abschnitt 7.1) oder *Collection kapseln* (Abschnitt 7.2) verbergen kann. Selbst elementare Daten lassen sich mit *Elementarten Wert durch Objekt ersetzen* (Abschnitt 7.3) kapseln – Entwickler sind oft überrascht, wie viele indirekte Vorteile sich daraus ergeben. Temporäre Variablen sind beim Refactoring oft hinderlich, denn ich muss gewährleisten, dass sie in der richtigen Reihenfolge berechnet werden und dass die Werte anderem Code, der sie benötigt, zur Verfügung stehen. *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4) ist hier äußerst hilfreich, insbesondere bei der Aufteilung einer viel zu langen Funktion.

Klassen wurden entwickelt, um Informationen zu verbergen. Im letzten Kapitel habe ich beschrieben, wie Klassen mit *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) erstellt werden können. Die üblichen

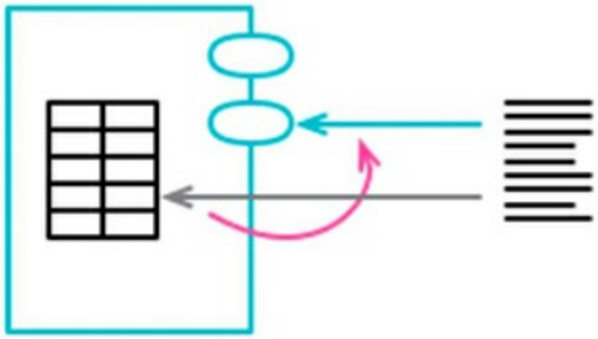
Extraktionen und Inline-Platzierungen sind mit *Klasse extrahieren* (Abschnitt 7.5) und *Klasse inline platzieren* (Abschnitt 7.6) auch auf Klassen anwendbar.

Neben dem Verbergen der Interna von Klassen ist es oft auch ratsam, die Verbindungen zwischen Klassen zu verbergen, beispielsweise mit *Delegation verbergen* (Abschnitt 7.7). Wenn man das Verbergen jedoch übertreibt, führt dies zu überfrachteten Schnittstellen, deshalb benötige ich auch die Umkehrung, nämlich *Vermittler entfernen* (Abschnitt 7.8).

Klassen und Module sind die umfangreichsten Formen von Kapselungen, aber auch Funktionen kapseln etwas, nämlich ihre Implementierung. Gelegentlich muss ich eine grundlegende Änderung an einem Algorithmus vornehmen. Zu diesem Zweck verwende ich *Funktion extrahieren* (Abschnitt 6.1), um ihn in einer Funktion zu »verpacken«, und nutze anschließend *Algorithmus ersetzen* (Abschnitt 7.9).

7.1 Datensatz kapseln (Encapsulate Record)

Ehemalige Bezeichnung: *Datensatz durch Datenklasse ersetzen* (Replace Record with Data Class)



```
organization = {name: "Acme Gooseberries", country:
"GB"};
```

```
class Organization {
  constructor(data) {
    this._name = data.name;
    this._country = data.country;
  }
  get name()    {return this._name;}
  set name(arg) {this._name = arg;}
  get country() {return this._country;}
  set country(arg) {this._country = arg;}
}
```

7.1.1 MOTIVATION

Das obige Beispiel verdeutlicht den Grund, warum ich lieber Objekte anstatt veränderliche Datensätze verwende. Bei Objekten kann ich verbergen, was gespeichert wird, und Methoden für die Werte

bereitstellen. Der Benutzer des Objekts braucht nicht zu wissen, ob ein Wert gespeichert oder berechnet wird. Diese Kapselung erleichtert auch die Umbenennung: Ich kann ein Feld umbenennen, während ich Zugriffsmethoden für den alten und den neuen Namen bereitstelle, und so nach und nach alle Aufrufer aktualisieren.

Ich habe gerade angemerkt, dass ich lieber Objekte als *veränderliche* Datensätze verwende. Wenn der Wert unveränderlich ist, kann ich alle drei Werte einfach im Datensatz ablegen und, falls erforderlich, einen Schritt zum Ergänzen berechneter Werte durchführen. Darüber hinaus lässt sich ein Feld beim Umbenennen auch leicht kopieren.

Ich kann zwei Arten von Datensatzstrukturen verwenden: Bei der ersten Art muss ich die Namen der verfügbaren Felder deklarieren oder ich kann im zweiten Fall beliebige Namen verwenden. Letztere sind meist durch eine Bibliothek in Form einer Klasse implementiert, die Bezeichnungen wie Hash, Map, Hashmap, Dictionary oder assoziatives Array tragen. Viele Sprachen bieten eine komfortable Syntax zum Erstellen von Hashmaps, die für die verschiedensten Programmieraufgaben nützlich sind. Ihre Verwendung hat jedoch den Nachteil, dass sie nicht über explizite Felder verfügen. Um herauszufinden, ob sie `start/end` oder `start/length` verwenden, gibt es nur eine Möglichkeit: Ich muss an den Stellen nachsehen, an

denen sie erzeugt und verwendet werden. Das stellt kein Problem dar, sofern sie lediglich in einem kurzen Codeabschnitt verwendet werden. Je größer aber der Gültigkeitsbereich ist, umso größer sind die Schwierigkeiten, die mir ihre implizite Struktur bereitet. Ich könnte ein Refactoring vornehmen, um die Struktur zu ändern – aber wenn das notwendig ist, mache ich lieber Klassen daraus.

Es ist üblich, verschachtelte Datenstrukturen aus Listen und Hashmaps zu übergeben, die meist serialisiert sind und in Formaten wie JSON oder XML vorliegen. Solche Strukturen können ebenfalls gekapselt werden, was sich als praktisch erweist, wenn sich das Format irgendwann ändert oder wenn ich mir Gedanken über Aktualisierungen der Daten machen muss, die nur schwer nachvollziehbar sind.

7.1.2 VORGEHEN

- Wenden Sie *Variable kapseln* (Abschnitt 6.6) auf die Variable an, die den Datensatz enthält.

  Vergeben Sie leicht auffindbare Namen für die Funktionen zum Kapseln des Datensatzes.

- Ersetzen Sie den Inhalt der Variable durch eine einfache Klasse, die den Datensatz umgibt. Definieren Sie innerhalb der Klasse eine Zugriffsmethode, die den rohen Datensatz zurückgibt. Modifizieren Sie die Funktionen zum Kapseln der Variable so, dass sie die Zugriffsmethode verwenden.

- Testen Sie.

- Stellen Sie neue Funktionen bereit, die anstelle des Datensatzes das

Objekt zurückgeben.

- Ersetzen Sie überall dort, wo der Datensatz verwendet wird, den Aufruf der Funktion, die den Datensatz zurückgibt, durch eine Funktion, die das Objekt zurückgibt. Verwenden Sie eine Zugriffsfunktion des Objekts, um an den Inhalt der Felder zu gelangen. Erstellen Sie diese Zugriffsfunktion bei Bedarf. Führen Sie nach jeder Änderung Tests durch.

  Falls der Datensatz komplex ist, also beispielsweise eine verschachtelte Struktur beinhaltet, konzentrieren Sie sich zuerst auf die Clients, die Daten aktualisieren. Ziehen Sie in Betracht, eine Kopie oder eine schreibgeschützte Version der Daten, einen sogenannten *Proxy* (Stellvertreter) zurückzugeben, wenn Clients die Daten nur lesen.

- Löschen Sie die Zugriffsfunktion der Klasse für den rohen Datensatz und die Funktionen, nach denen sich leicht suchen lässt, die den Datensatz kapseln

- Testen Sie.

- Wenn die Felder des Datensatzes selbst Datenstrukturen sind, sollten Sie in Betracht ziehen, *Datensatz kapseln* und *Collection kapseln* (Abschnitt 7.2) rekursiv anzuwenden.

7.1.3 BEISPIEL

Als Erstes betrachte ich eine Konstante, die überall im Programm verwendet wird:

```
const organization = {name: "Acme Gooseberries",
                      country: "GB"};
```

Dabei handelt es sich um ein JavaScript-Objekt, das als Datensatzstruktur in verschiedenen Teilen des Programms wie folgt verwendet wird:

```
result += `<h1>${organization.name}</h1>`;
```

und

```
organization.name = newName;
```

Der erste Schritt ist ein einfaches *Variable kapseln* (Abschnitt 6.6):

```
function getRawDataOfOrganization() {return
organization;}

Beispiel für eine Lesefunktion...
result += `<h1>${getRawDataOfOrganization().name}
</h1>`;

Beispiel für eine Schreibfunktion...
getRawDataOfOrganization().name = newName;
```

Hierbei handelt es sich um kein ganz gewöhnliches *Variable kapseln* (Abschnitt 6.6), denn ich habe dem Getter absichtlich einen Namen gegeben, der nicht nur hässlich ist, sondern sich auch leicht wiederfinden lässt. Das hat seinen Grund: Es wird ihn nicht mehr lange geben.

Einen Datensatz zu kapseln, bedeutet, tiefer zu gehen als bis zur Ebene der Variable: Ich möchte steuern können, wie der Datensatz verändert werden kann. Das erreiche

ich, indem ich den Datensatz durch eine Klasse ersetze:

```
class Organization...
class Organization {
  constructor(data) {
    this._data = data;
  }
}

Oberste Ebene...
const organization =
                new Organization({name: "Acme
Gooseberries",
                                  country: "GB"});
function getRawDataOfOrganization() {
  return organization._data;
}
function getOrganization() {return organization;}
```

Jetzt, da ich ein Objekt an Ort und Stelle habe, beginne ich, mir die Verwendung des Datensatzes anzusehen. Sämtliche Aktualisierungen des Datensatzes werden durch einen Setter ersetzt:

```
class Organization...
set name(aString) {this._data.name = aString;}

Client...
getOrganization().name = newName;
```

Ebenso ersetze ich alle Lesezugriffe durch den entsprechenden Getter:

```
class Organization...
get name() {return this._data.name;}

Client...
result += `<h1>${getOrganization().name}</h1>`;
```

Nachdem das erledigt ist, kann ich meine Ankündigung, dass es den hässlichen Funktionsnamen nicht mehr lange geben wird, in die Tat umsetzen:

```
function getRawDataOfOrganization() {
  return organization._data;
}
function getOrganization() {return organization;}
```

Zudem verlagere ich das Feld `_data` direkt in das Objekt:

```
class Organization {
  constructor(data) {
    this._name = data.name;
    this._country = data.country;
  }
  get name() {return this._name;}
  set name(aString) {this._name = aString;}
  get country() {return this._country;}
  set country(aCountryCode) {this._country =
aCountryCode;}
}
```

Hier ergibt sich der Vorteil, dass die Kopplung zum

Eingangsdatensatz aufgelöst wird. Das könnte sich als nützlich erweisen, falls einmal ein Verweis darauf herumgereicht werden sollte, was die Kapselung aushebeln würde. Falls ich die Daten nicht in die einzelnen Felder verlagere, sollte ich bei der Zuweisung von `_data` eine Kopie erstellen.

7.1.4 BEISPIEL: KAPSELUNG EINES VERSCHACHTELTEN DATENSATZES

Das letzte Beispiel verwendet einen »flachen« Datensatz, aber wie verfahre ich mit tief verschachtelten Daten, die beispielsweise einer JSON-Datei entstammen? Die grundlegenden Schritte dieses Refactorings bleiben anwendbar, ich muss bei Aktualisierungen ebenso sehr achtsam sein, aber beim Lesen stehen mir einige alternative Möglichkeiten offen.

Als Beispiel dienen hier einige etwas tiefer verschachtelte Daten: eine Hashmap mit Kundendaten, die anhand der Kunden-ID indiziert sind:

```
"1920": {
  name: "martin",
  id: "1920",
  usages: {
    "2016": {
      "1": 50,
      "2": 55,
      // verbleibende Monate des Jahres
    },
    "2015": {
```

```
      "1": 70,
      "2": 63,
      // verbleibende Monate des Jahres
    }
  }
},
"38673": {
  name: "neal",
  id: "38673",
  // weitere Kunden im gleichem Format
```

Mit Schreib- und Lesefunktionen kann auf die tieferen Teile der Datenstruktur zugegriffen werden:

```
Beispiel für eine Aktualisierung...
customerData[customerID].usages[year][month] = amount;

Beispiel für eine Lesefunktion...
function compareUsage (customerID, laterYear, month) {
  const later =
          customerData[customerID].usages[laterYear]
[month];
  const earlier =
       customerData[customerID].usages[laterYear - 1]
[month];
  return {laterAmount: later, change: later -
earlier};
}
```

Ich verwende zunächst wieder *Variable kapseln* (Abschnitt 6.6), um die Kapselung dieser Daten vorzunehmen:

```
function getRawDataOfCustomers() {return
customerData;}
function setRawDataOfCustomers(arg) {customerData =
arg;}

Beispiel für eine Aktualisierung...
getRawDataOfCustomers() \
                  [customerID].usages[year][month] =
amount;

Beispiel für eine Lesefunktion...
function compareUsage (customerID, laterYear, month) {
  const later = getRawDataOfCustomers() \
                      [customerID].usages[laterYear]
[month];
  const earlier = getRawDataOfCustomers() \
                  [customerID].usages[laterYear - 1]
[month];
  return {laterAmount: later, change: later -
earlier};
}
```

Anschließend erstelle ich eine Klasse für die gesamte Datenstruktur:

```
class CustomerData {
  constructor(data) {
    this._data = data;
  }
}

Oberste Ebene...
function getCustomerData() {return customerData;}
function getRawDataOfCustomers() {return
customerData._data;}
```

```
function setRawDataOfCustomers(arg) {
  customerData = new CustomerData(arg);
}
```

Hier ist es am wichtigsten, die Aktualisierungen zu handhaben. Wenn ich mir die Aufrufe von `getRawDataOfCustomers` ansehe, konzentriere ich mich also auf die Aufrufe, die Daten ändern. Zur Erinnerung ist hier noch einmal die Aktualisierung abgedruckt:

```
Beispiel für eine Aktualisierung ...
getRawDataOfCustomers()[customerID].usages[year]
[month] =

amount;
```

Die allgemeine Vorgehensweise besagt, den vollständigen Kundendatensatz zurückzugeben und eine Zugriffsfunktion zu verwenden, die bei Bedarf erstellt wird. Für diese Aktualisierung des Kundendatensatzes steht mir kein Setter zur Verfügung, sie greift jedoch auf tiefere Teile der Datenstruktur zu. Um diesen Setter zu erstellen, wende ich zunächst *Funktion extrahieren* (Abschnitt 6.1) auf den Code an, der auf die tieferen Teile der Datenstruktur zugreift.

```
Beispiel für eine Aktualisierung...
setUsage(customerID, year, month, amount);
```

```
Oberste Ebene...
function setUsage(customerID, year, month, amount) {
  getRawDataOfCustomers()[customerID].usages[year]
[month] =

amount;
}
```

Jetzt verwende ich *Funktion verschieben* (Abschnitt 8.1), um den Code in die neue Datenklasse `CustomerData` zu verschieben.

```
Beispiel für eine Aktualisierung...
getCustomerData().setUsage(customerID, year, month,
amount);

class CustomerData...
setUsage(customerID, year, month, amount) {
  this._data[customerID].usages[year][month] = amount;
}
```

Bei der Arbeit an großen Datenstrukturen konzentriere ich mich auf die Aktualisierungen. Sie deutlich zu machen und an einer Stelle zu bündeln, ist der wichtigste Teil der Kapselung.

Irgendwann werde ich meinen, alle Aktualisierungen berücksichtigt zu haben – aber wie kann ich mir dessen sicher sein? Es gibt verschiedene Möglichkeiten, das zu überprüfen. Ich könnte beispielsweise `getRawDataOfCustomers` so modifizieren, dass die

Funktion eine tiefe Kopie der Daten zurückgibt. Wenn meine Testabdeckung hinreichend gut ist, sollte einer meiner Tests fehlschlagen, falls ich eine Aktualisierung übersehen haben sollte.

```
Oberste Ebene...
function getCustomerData() {return customerData;}
function getRawDataOfCustomers() {
  return customerData.rawData;
}
function setRawDataOfCustomers(arg) {
  customerData = new CustomerData(arg);
}

class CustomerData...
get rawData() {
  return _.cloneDeep(this._data);
}
```

Zum Erstellen der tiefen Kopie verwende ich wieder die JavaScript-Bibliothek Lodash.

Ein anderer Ansatz ist die Rückgabe eines schreibgeschützten Proxys der Datenstruktur, der einen Ausnahmefehler auslöst, wenn der Client-Code versucht, das zugrunde liegende Objekt zu ändern. In manchen Sprachen ist das ganz einfach, in JavaScript hingegen eine Qual, deshalb überlasse ich es dem Leser, das als Übung durchzuführen. Ich könnte auch eine Kopie der Daten verwenden und diese rekursiv einfrieren (vgl. `Object.freeze()`), um jedwede Versuche,

Änderungen vorzunehmen, aufzuspüren.

Es ist natürlich sinnvoll, sich um die Aktualisierungen zu kümmern, aber was ist mit den Lesefunktionen? Hier stehen mehrere Möglichkeiten zur Verfügung.

Die erste Möglichkeit ist, so vorzugehen, wie ich mit den Settern verfahren bin. Extrahieren Sie alle Lesezugriffe, erstellen Sie eigene Funktionen daraus und verschieben Sie diese in die Datenklasse.

```
class CustomerData...
usage(customerID, year, month) {
  return this._data[customerID].usages[year][month];
}

Oberste Ebene...
function compareUsage (customerID, laterYear, month) {
  const later =
       getCustomerData().usage(customerID, laterYear,
month);
  const earlier =
   getCustomerData().usage(customerID, laterYear - 1,
month);
  return {laterAmount: later, change: later -
earlier};
}
```

Das Beste an diesem Ansatz ist, dass er `customerData` eine explizite API bereitstellt, die alle Verwendungen erfasst. Ich werfe einen Blick auf die Klasse und kann alle Verwendungen der Daten erkennen. Allerdings kann hier

viel Code für zahlreiche Sonderfälle zusammenkommen. Moderne Programmiersprachen bieten die Möglichkeit, sogenannte List-and-hash-Datenstrukturen (https://www.martinfowler.com/bliki/ListAndHash.html) zu verwenden, und es erweist sich als nützlich, den Clients nur solche Datenstrukturen bereitzustellen.

Wenn der Client nach einer Datenstruktur verlangt, kann ich ihm die tatsächlichen Daten einfach zur Verfügung stellen. Aber das bringt das Problem mit sich, dass es dann keine Möglichkeit gibt, zu verhindern, dass Clients die Daten direkt verändern, wodurch der eigentliche Sinn der Kapselung aller Aktualisierungen in den Funktionen zunichte gemacht würde. Konsequenterweise ist es daher am einfachsten, mit der zuvor implementierten `rawData`-Methode eine Kopie der zugrundeliegenden Daten bereitzustellen.

```
class CustomerData...
get rawData() {
  return _.cloneDeep(this._data);
}

Oberste Ebene...
function compareUsage (customerID, laterYear, month) {
  const later = getCustomerData().rawData[customerID].
                              usages[laterYear][month];
  const earlier =
getCustomerData().rawData[customerID].
                            usages[laterYear - 1]
[month];
  return {laterAmount: later, change: later -
```

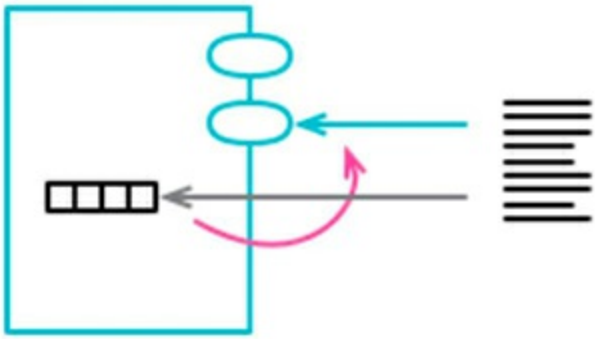
```
earlier};
}
```

Das ist zwar einfach, hat aber auch Nachteile. Das offensichtlichste Problem ist der mit dem Kopieren einer großen Datenstruktur verbundene Aufwand, der zu Performanceeinbußen führen könnte. Wie so oft in vergleichbaren Fällen ist der Performanceverlust möglicherweise akzeptabel – ich würde zunächst Messungen vornehmen, bevor ich mir Sorgen mache. Es kann auch zur Verwirrung bei den Clients führen, die davon ausgehen, dass sie nicht eine Kopie, sondern die ursprünglichen Daten ändern. In diesen Fällen kann Abhilfe schaffen, einen schreibgeschützten Proxy oder eine unveränderliche Kopie der Daten bereitzustellen, weil es dann zu einer Fehlermeldung kommt, wenn ein Client versucht, Daten zu ändern.

Eine weitere Option bedeutet zwar mehr Arbeit, bietet aber die beste Kontrolle: die rekursive Anwendung von *Datensatz kapseln*. Hier mache ich aus dem Kundendatensatz eine eigene Klasse und wende *Collection kapseln* (Abschnitt 7.2) an, um eine `Usage`-Klasse zu erstellen. Ich kann dann die Kontrolle über Aktualisierungen ausüben, indem ich Zugriffsfunktionen verwende und möglicherweise *Referenz durch Wert ersetzen* (Abschnitt 9.4) auf die `Usage`-Objekte anwende. Das kann bei einer großen Datenstruktur eine Menge Arbeit bedeuten – die eigentlich gar nicht

notwendig ist, wenn ich nicht auf weite Teile der Datenstruktur zugreife. Manchmal funktioniert schon eine intelligente Mischung aus Gettern und neuen Klassen, wenn Getter-Funktionen auf tief liegende Teile der Datenstruktur zugreifen, aber ein Objekt zurückliefern, das die Datenstruktur und nicht die ungekapselten Daten enthält. Ich habe über dieses Thema einen Artikel geschrieben: *Refactoring Code to Load a Document* (`https://martinfowler.com/articles/refactoring-document-load.html`).

7.2 Collection kapseln (Encapsulate Collection)



```
class Person {
  get courses() {return this._courses;}
  set courses(aList) {this._courses = aList;}
```

```
class Person {
  get courses() {return this._courses.slice();}
  addCourse(aCourse) { ... }
  removeCourse(aCourse) { ... }
```

7.2.1 MOTIVATION

Ich pflege alle veränderlichen Daten in meinen Programmen zu kapseln. Auf diese Weise ist leichter erkennbar, wann und wie Datenstrukturen modifiziert werden, was es wiederum erleichtert, diese Datenstrukturen bei Bedarf zu ändern. Das Kapseln wird gemeinhin befürwortet, insbesondere von objektorientierten Entwicklern. Bei der Verwendung von Objektsammlungen (Collections) tritt jedoch ein weitverbreiteter Fehler auf. Der Zugriff auf eine Variable der Collection mag gekapselt sein, aber wenn ein Getter die Collection selbst zurückgibt, kann die Zusammensetzung dieser Collection geändert werden, ohne dass die umgebende Klasse etwas dagegen unternehmen könnte.

Um das zu verhindern, stelle ich Methoden zum Ändern der Sammlung – für gewöhnlich `add` und `remove` – durch die Klasse selbst bereit. Auf diese Weise erfolgen Änderungen an der Collection durch die umgebende Klasse, was mir die Möglichkeit bietet, auf solche Änderungen Einfluss zu nehmen, wenn sich das Programm weiterentwickelt. Wenn das Team daran gewöhnt ist, Collections nicht außerhalb des

ursprünglichen Moduls zu modifizieren, reicht es oft schon aus, diese Methoden bereitzustellen. Es ist für gewöhnlich allerdings unklug, sich auf solche Gewohnheiten zu verlassen. Wenn hier ein Fehler gemacht wird, kann das zu Bugs führen, die später schwer zu finden sind. Zu gewährleisten, dass der Getter für die Collection nicht die komplette Sammlung zurückgibt, damit Clients sie nicht versehentlich ändern können, ist ein weitaus besserer Ansatz.

Eine Möglichkeit, Änderungen an der zugrunde liegenden Collection zu verhindern, besteht darin, grundsätzlich nie die Collections selbst zurückgegeben. Bei diesem Ansatz muss jede Verwendung eines Feldes der Sammlung durch spezielle Methoden der umgebenden Klasse erfolgen. Aus `aCustomer.orders.size` wird dann beispielsweise `aCustomer.numberOfOrders`. Ich bin mit diesem Ansatz nicht einverstanden. Moderne Programmiersprachen bieten umfangreiche Klassen für alle erdenklichen Sammlungen mit standardisierten Schnittstellen, die sich auf sinnvolle Weise miteinander kombinieren lassen, z.B. als Collection-Pipelines (`https://martinfowler.com/articles/collection-pipeline/`). Die Verwendung spezieller Methoden zur Handhabung dieser Funktionalität erzeugt Unmengen zusätzlichen Codes und lähmt die einfache Kombinierbarkeit der Operationen für Sammlungen.

Eine andere Möglichkeit besteht darin, lediglich den schreibgeschützten Zugriff auf eine Sammlung zu erlauben. In Java ist es beispielsweise ganz einfach, einen schreibgeschützten Proxy einer Collection zurückzuliefern. Lesezugriffe werden vom Proxy an die zugrunde liegende Sammlung weitergeleitet, Schreibzugriffe hingegen werden blockiert – in Java wird bei diesem Versuch ein Ausnahmefehler ausgelöst. Auf ähnliche Weise gehen Bibliotheken vor, deren Zusammenstellung einer Sammlung auf einem Iterator oder einem aufzählbaren Objekt beruht – vorausgesetzt, dass der Iterator die zugrunde liegende Sammlung nicht ändern kann.

Eine Getter-Methode bereitzustellen, die eine Kopie der Collection zurückgibt, dürfte wohl der häufigste Ansatz sein. Die an der Kopie vorgenommenen Änderungen haben keinerlei Auswirkung auf die gekapselte Sammlung. Das wiederum sorgt nicht selten für Verwirrung, wenn Programmierer davon ausgehen, dass die zurückgegebene Collection die ursprünglichen Daten ändert. Inzwischen haben sich die Programmierer aber daran gewöhnt, dass in vielen Codebasen die Getter Kopien der Sammlung liefern. Bei einer großen Sammlung kann das zu Performanceeinbußen führen – die meisten Listen sind aber gar nicht so groß, deshalb sollten die allgemeinen Regeln für die Performance weiterhin gültig bleiben (siehe auch *Refactoring und Performance,* Abschnitt 2.8).

Ein weiterer Unterschied zwischen der Verwendung eines Proxys und einer Kopie besteht darin, dass eine Änderung in den Ausgangsdaten in einem Proxy sichtbar ist, nicht aber in einer Kopie. Das stellt normalerweise kein Problem dar, weil Listen, auf die solcherart zugriffen wird, für gewöhnlich nur für kurze Zeit existieren.

Wichtig ist hier vor allem die Einheitlichkeit der Codebasis. Verwenden Sie nur einen der beschriebenen Mechanismen, damit sich alle Beteiligten an das Verhalten gewöhnen können, wenn Zugriffsfunktionen für Sammlungen zum Einsatz kommen.

7.2.2 VORGEHEN

- Verwenden Sie *Variable kapseln* (Abschnitt 6.6), wenn der Verweis auf die Variable noch nicht gekapselt ist.
- Erstellen Sie Funktionen zum Hinzufügen und Entfernen von Elementen der Sammlung.

  Verwenden Sie *Setter entfernen* (Abschnitt 11.7), falls eine Setter-Methode für die Sammlung vorhanden ist. Sollte das nicht möglich sein: Sorgen Sie dafür, dass sie eine Kopie der bereitgestellten Sammlung verwendet.
- Führen Sie statische Überprüfungen durch.
- Suchen Sie nach allen Verweisen auf die Collection. Wenn irgendwo Methoden zum Ändern der Sammlung aufgerufen werden, sollten Sie die Aufrufe durch die neuen `add/remove`-Funktionen ersetzen. Führen Sie nach jeder Änderung Tests durch.
- Modifizieren Sie den Getter der Sammlung so, dass er eine geschützte Version zurückgibt, also einen schreibgeschützten Proxy

oder eine Kopie der Daten.

- Testen Sie.

7.2.3 BEISPIEL

Ich verwende die Klasse `Person`, die ein Feld für eine Liste von Lehrgängen besitzt:

```
class Person...
constructor (name) {
  this._name = name;
  this._courses = [];
}
get name() {return this._name;}
get courses() {return this._courses;}
set courses(aList) {this._courses = aList;}

class Course...
constructor(name, isAdvanced) {
  this._name = name;
  this._isAdvanced = isAdvanced;
}
get name() {return this._name;}
get isAdvanced() {return this._isAdvanced;}
```

Die Clients verwenden die Sammlung, um Informationen über die Lehrgänge zusammenzutragen:

```
numAdvancedCourses = aPerson.courses
  .filter(c => c.isAdvanced)
  .length
;
```

Ein naiver Entwickler würde sagen, dass die Daten der Klasse ordnungsgemäß gekapselt sind, schließlich ist jedes Feld durch Zugriffsmethoden geschützt. Ich hingegen behaupte, dass die Liste der Lehrgänge nicht richtig gekapselt ist. Einzelne Kurse können leicht mit Hilfe der Setter aktualisiert werden:

```
Client-Code...
const basicCourseNames =
readBasicCourseNames(filename);
aPerson.courses =
       basicCourseNames.map(name => new Course(name,
false));
```

Ein Client könnte die Lehrgangsliste jedoch auch direkt aktualisieren:

```
Client-Code...
for(const name of readBasicCourseNames(filename)) {
  aPerson.courses.push(new Course(name, false));
}
```

Die Kapselung wird dadurch ausgehebelt, weil die Klasse `Person` keine Möglichkeit hat, Kontrolle auszuüben, wenn die Liste auf diese Weise aktualisiert wird. Der Verweis auf das Feld ist zwar gekapselt, der Inhalt des Feldes aber nicht.

Um eine korrekte Kapselung zu erreichen, füge ich der

Klasse `Person` Methoden hinzu, die es einem Client ermöglichen, einzelne Lehrgänge zu entfernen oder hinzuzufügen.

```
class Person...
addCourse(aCourse) {
  this._courses.push(aCourse);
}
removeCourse(aCourse, fnIfAbsent = () =>
                                 {throw new
RangeError();}) {
  const index = this._courses.indexOf(aCourse);
  if (index === -1) fnIfAbsent();
  else this._courses.splice(index, 1);
}
```

Beim Entfernen muss ich entscheiden, wie ich vorgehe, wenn ein Client ein Element löschen möchte, das gar nicht zur Sammlung gehört. Ich kann das achselzuckend ignorieren oder eine Fehlermeldung erzeugen. Mein Code erzeugt standardmäßig eine Fehlermeldung, gibt dem Aufrufer jedoch die Möglichkeit, den Fall bei Bedarf anders zu behandeln.

Ich ändere jetzt sämtlichen Code, der Methoden zum direkten Modifizieren der Collection aufruft, sodass nur noch die neuen Methoden verwendet werden.

```
Client-Code...
for(const name of readBasicCourseNames(filename)) {
  aPerson.addCourse(new Course(name, false));
```

```
}
```

Wenn individuelle Methoden zum Hinzufügen und Löschen verfügbar sind, wird `setCourses` für gewöhnlich nicht mehr benötigt. In diesem Fall wende ich *Setter entfernen* (Abschnitt 11.7) darauf an. Wenn für die API aus irgendeinem Grund ein Setter erforderlich ist, sorge ich dafür, dass eine Kopie der Sammlung verfügbar ist.

```
class Person...
set courses(aList) {this._courses = aList.slice();}
```

All dies ermöglicht den Clients, die richtigen Methoden zum Ändern zu verwenden. Ich möchte aber auch gewährleisten, dass niemand die Listen modifiziert, ohne sie zu verwenden. Das kann ich erreichen, indem ich auch hier nur eine Kopie bereitstelle.

```
class Person...
get courses() {return this._courses.slice();}
```

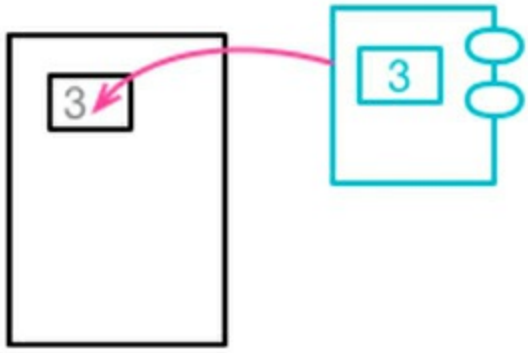
Ich halte es im Allgemeinen für klug, beim Umgang mit Sammlungen einigermaßen paranoid zu sein. Mir ist es lieber, sie überflüssigerweise zu kopieren, als durch unerwartete Veränderungen verursachte Fehler debuggen zu müssen. Veränderungen sind auch nicht immer offensichtlich. Das Sortieren eines Arrays in

JavaScript beispielsweise modifiziert das Original, viele andere Sprachen hingegen verwenden für Operationen, die Collections modifizieren, standardmäßig eine Kopie. Eine Klasse, die für die Verwaltung einer Sammlung verantwortlich ist, sollte stets Kopien zurückliefern – ich habe es mir aber auch zur Gewohnheit gemacht, eine Kopie zu erstellen, wann immer ich etwas tue, was die Sammlung verändert.

7.3 Elementaren Wert durch Objekt ersetzen (Replace Primitive with Object)

Ehemalige Bezeichnung: *Wert durch Objekt ersetzen* (Replace Data Value with Object)

Ehemalige Bezeichnung: *Typenschlüssel durch Klasse ersetzen* (Replace Type Code Value with Class)




```
orders.filter(o => "high" === o.priority
                 || "rush" === o.priority);
```

```
orders.filter(o =>
              o.priority.higherThan(new
Priority("normal")))
```

7.3.1 MOTIVATION

In der Anfangsphase der Entwicklung trifft man oft Entscheidungen darüber, auf welche Weise einfache Dinge, beispielsweise elementare Datenelemente wie Zahlen oder Strings, repräsentiert werden sollen. Wenn die Entwicklung fortschreitet, sind diese einfachen Elemente plötzlich gar nicht mehr so einfach. Eine Telefonnummer kann vielleicht eine Weile als String repräsentiert werden, aber früher oder später wird spezielles Verhalten erforderlich, wie die Formatierung, das Herausfiltern der Vorwahl und dergleichen. Diese Art von Programmlogik kann sich schnell in der gesamten Codebasis ausbreiten, und das mehrfache Vorhandensein solcher Logik kann ihre Verwendung erschweren.

Wenn ich feststelle, dass ich mehr als eine einfache Textausgabe benötige, erstelle ich eine neue Klasse für die betreffenden Daten. Anfangs ist eine solche Klasse wenig mehr als ein »Wrapper« für die elementaren Daten – aber sobald es diese Klasse gibt, steht mir ein

Ort zur Verfügung, an dem ich speziell auf diese Daten zugeschnittenes Verhalten unterbringen kann. Zunächst sind diese Erweiterungen ganz bescheiden, sie können sich im Laufe der Zeit jedoch zu wirklich nützlichen Tools entwickeln. Auf den ersten Blick sind sie nicht besonders beeindruckend, ich habe aber festgestellt, dass sie überraschend große Auswirkungen auf eine Codebasis haben können. Tatsächlich halten viele erfahrene Entwickler dieses Refactoring für eines der nützlichsten – während Programmierneulinge oft denken, es widerspreche der Intuition.

7.3.2 VORGEHEN

- Wenden Sie *Variable kapseln* (Abschnitt 6.6) auf die Variable an, sofern sie noch nicht gekapselt ist.
- Erstellen Sie eine einfache Datenklasse für den Wert. Sie sollte beim Konstruktoraufruf den vorhandenen Wert erhalten und eine Getter-Methode für den Wert bereitstellen.
- Führen Sie statische Überprüfungen durch.
- Die Setter-Methode sollte eine neue Instanz der Datenklasse erstellen und sie im Feld speichern. Dabei wird gegebenenfalls der Typ des Feldes geändert.
- Der Getter sollte das Ergebnis des Aufrufs des Getters der neuen Klasse zurückgeben.
- Testen Sie.
- Ziehen Sie in Betracht, *Funktionsdeklaration ändern* (Abschnitt 6.5) auf die ursprünglichen Zugriffsfunktionen anzuwenden, um besser zu dokumentieren, welche Aufgaben sie erledigen.
- Ziehen Sie in Betracht, die Rolle des neuen Objektes zu verdeutlichen (Wert oder Verweis), indem Sie *Referenz durch Wert*

ersetzen (Abschnitt 9.4) oder *Wert durch Referenz ersetzen* (Abschnitt 9.5) anwenden.

7.3.3 BEISPIEL

Ich verwende eine einfache Klasse für Bestellungen (`Order`), die Daten aus einer Datensatzstruktur einliest. Eine der Eigenschaften repräsentiert eine Priorität (`priority`), die einen einfachen String einliest.

```
class Order...
constructor(data) {
  this.priority = data.priority;
  // weitere Initialisierungen
```

Die Clients verwenden Code wie diesen:

```
Client...
highPriorityCount = orders.filter(o => "high" ===
o.priority
                                      || "rush" ===
o.priority)
                           .length;
```

Immer wenn ich es mit einem Datenwert zu tun habe, wende ich als Erstes *Variable kapseln* (Abschnitt 6.6) auf ihn an:

```
class Order...
get priority() {return this._priority;}
```

```
set priority(aString) {this._priority = aString;}
```

Die Zeile im Konstruktor, die `priority` initialisiert, verwendet jetzt den hier definierten Setter.

Das Feld ist dadurch selbstgekapselt, sodass ich dessen aktuelle Verwendung beibehalten kann, während ich die Daten selbst ändere.

Für die Priorität erstelle ich eine einfache Datenklasse. Sie besitzt einen Konstruktor für den Wert und eine Konvertierungsfunktion zur Rückgabe eines Strings.

```
class Priority {
  constructor(value) {this._value = value;}
  toString() {return this._value;}
}
```

Ich ziehe es hier vor, statt eines Getters (`value`) eine Konvertierungsfunktion (`toString`) zu verwenden. Die Abfrage der Stringrepräsentation sollte sich eher nach einer Konvertierung als nach einer Abfrage der Eigenschaft anfühlen.

Jetzt ändere ich die Zugriffsfunktionen, damit sie die neue Klasse verwenden:

```
class Order...
get priority() {return this._priority.toString();}
```

```
set priority(aString) {
  this._priority = new Priority(aString);
}
```

Jetzt ist die Klasse `Priority` verfügbar, und ich denke, dass der Getter in der Klasse `Order` irreführend ist. Er gibt keine Priorität zurück, sondern einen String, der die Priorität beschreibt. Deshalb wende ich *Funktionsdeklaration ändern* (Abschnitt 6.5) an.

```
class Order...
get priorityString() {return
this._priority.toString();}
set priority(aString) {
  this._priority = new Priority(aString);
}

Client...
highPriorityCount =
              orders.filter(o => "high" ===
o.priorityString
                              || "rush" ===
o.priorityString)
                     .length;
```

In diesem Fall bin ich damit zufrieden, den Namen des Setters beizubehalten. Anhand der Bezeichnung des Arguments ist ersichtlich, was er erwartet.

Jetzt ist das Refactoring grundsätzlich erledigt. Wenn ich mir allerdings ansehe, wo die Priorität verwendet wird, frage ich mich, ob dort nicht auch die Klasse `Priority`

verwendet werden sollte. Also stelle ich in der Klasse `Order` einen Getter bereit, der das neue Prioritäts-Objekt direkt zur Verfügung stellt.

```
class Order...
get priority() {return this._priority;}
get priorityString() {return
this._priority.toString();}
set priority(aString) {
  this._priority = new Priority(aString);
}

Client...
highPriorityCount =
             orders.filter(o => "high" ===
o.priorityString
                              || "rush" ===
o.priorityString)
                    .length;
```

Wenn sich die Klasse `Priority` auch an anderer Stelle als nützlich erweisen sollte, würde ich den Clients der `Order`-Klasse ermöglichen, den Setter und eine `Priority`-Instanz zu verwenden, indem ich den Konstruktor anpasse:

```
class Priority...
constructor(value) {
  if (value instanceof Priority) return value;
  this._value = value;
}
```

Das Entscheidende ist hier, dass meine neue `Priority`-Klasse jetzt als Ort für zusätzliches Verhalten dienen kann – entweder völlig neues oder von anderen Stellen verlagertes Verhalten. Der nachstehende Code fügt eine Überprüfung der Prioritätswerte und Programmlogik für Vergleiche hinzu:

```
class Priority...
constructor(value) {
  if (value instanceof Priority) return value;
  if (Priority.legalValues().includes(value))
    this._value = value;
  else
    throw new Error(`<${value}> is invalid for
Priority`);
}
toString() {return this._value;}
get _index() {
  return Priority.legalValues().findIndex(
                                s => s ===
this._value);
}
static legalValues() {
  return ['low', 'normal', 'high', 'rush'];
}
equals(other) {return this._index === other._index;}
higherThan(other) {return this._index > other._index;}
lowerThan(other) {return this._index < other._index;}
```

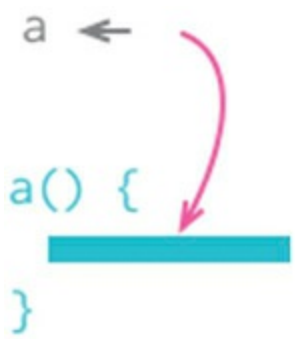
Während ich diese Änderungen vornehme, fasse ich den Entschluss, dass eine Priorität ein Value Object sein sollte. Ich stelle also eine `equals`-Methode bereit und gewährleiste, dass ein Prioritäts-Objekt unveränderlich

ist.

Nachdem ich dieses Verhalten hinzugefügt habe, ist der Client-Code viel aussagekräftiger:

```
Client...
highPriorityCount = orders.filter(o =>
o.priority.higherThan(
                                    new
Priority("normal")))
                            .length;
```

7.4 Temporäre Variable durch Abfrage ersetzen (Replace Temp with Query)




```
const basePrice = this._quantity * this._itemPrice;
if (basePrice > 1000)
  return basePrice * 0.95;
else
  return basePrice * 0.98;
```

```
get basePrice() {this._quantity * this._itemPrice;}

...

if (this.basePrice > 1000)
  return this.basePrice * 0.95;
else
  return this.basePrice * 0.98;
```

7.4.1 MOTIVATION

Temporäre Variablen dienen unter anderem dazu, errechnete Werte zwischenzuspeichern, um im späteren Verlauf der Funktion darauf Bezug nehmen zu können. Die Verwendung einer temporären Variable ermöglicht es mir, auf den Wert zu verweisen und gleichzeitig seine Bedeutung zu erklären, ohne den Code zur Berechnung wiederholen zu müssen. Variablen sind zwar praktisch, es kann aber oft auch sinnvoll sein, einen Schritt weiter zu gehen und stattdessen eine Funktion zu verwenden.

Wenn ich daran arbeite, eine lange Funktion aufzuteilen, erleichtert das Umwandeln von Variablen in eigene Funktionen die Extraktion von Teilen der Funktion, weil ich den extrahierten Funktionen keine Variablen mehr übergeben muss. Die Programmlogik in Funktionen auszulagern sorgt häufig auch für eine strengere Abgrenzung zwischen der extrahierten Programmlogik

und der ursprünglichen Funktion. Das hilft mir dabei, unschöne Abhängigkeiten und ungewollte Nebeneffekte zu vermeiden.

Funktionen statt Variablen zu verwenden hilft mir zudem, redundante Berechnungslogik in ähnlichen Funktionen zu vermeiden. Sobald mir klar wird, dass eine Variable an mehreren Stellen auf dieselbe Weise berechnet wird, versuche ich, dafür eine einzige Funktion zu erstellen.

Dieses Refactoring funktioniert am besten innerhalb einer Klasse, denn die Klasse bietet ein gemeinsames Umfeld für die Methoden, die ich extrahiere. Außerhalb einer Klasse kann es sein, dass eine Funktion auf oberster Ebene zu viele Parameter besitzt, was viele der Vorteile der Verwendung einer Funktion zunichte macht. Durch verschachtelte Funktionen lässt sich das vermeiden. Sie beschränken jedoch meine Möglichkeiten, die Logik in zusammengehörigen Funktionen gemeinsam zu nutzen.

Das Refactoring *Temporäre Variable durch Abfrage ersetzen* ist nur für bestimmte temporäre Variablen geeignet. Die Variable darf nur einmalig berechnet und nachfolgend nur noch gelesen werden. Im einfachsten Fall bedeutet das eine einmalige Zuweisung. Es können jedoch in einem komplizierteren Codeabschnitt auch mehrere Zuweisungen erfolgen, die in der Abfrage berücksichtigt werden müssen. Darüber hinaus muss die

Programmlogik zur Berechnung der Variable das gleiche Ergebnis liefern, wenn die Variable später verwendet wird – was Variablen ausschließt, die als eine Art Momentaufnahme dienen und häufig Namen wie `oldAddress` tragen.

7.4.2 VORGEHEN

- Vergewissern Sie sich, dass die Berechnung der Variable vollständig abgeschlossen ist, bevor sie verwendet wird, und dass der Code zur Berechnung keine unterschiedlichen Ergebnisse liefert, wenn er mehrmals ausgeführt wird.
- Wandeln Sie eine nicht schreibgeschützte Variable in eine schreibgeschützte um, falls möglich.
- Testen Sie.
- Extrahieren Sie die Zuweisung zur Variable in eine eigene Funktion.

  Verwenden Sie einen temporären Namen für die Funktion, wenn die Variable und die Funktion nicht denselben Namen tragen dürfen.

  Vergewissern Sie sich, dass die extrahierte Funktion keine Nebeneffekte verursacht. Verwenden Sie andernfalls *Abfrage von Veränderung trennen* (Abschnitt 11.1)
- Testen Sie.
- Verwenden Sie *Variable inline platzieren* (Abschnitt 6.4), um die temporäre Variable zu entfernen.

7.4.3 BEISPIEL

Hier ist eine einfache Klasse:

```
class Order...
```

```
constructor(quantity, item) {
  this._quantity = quantity;
  this._item = item;
}
get price() {
  var basePrice = this._quantity * this._item.price;
  var discountFactor = 0.98;
  if (basePrice > 1000) discountFactor -= 0.03;
  return basePrice * discountFactor;
}
```

Ich möchte die temporären Variablen `basePrice` und `discountFactor` durch Methoden ersetzen.

Zunächst wandle ich `basePrice` in eine Konstante um und führe Tests durch. Auf diese Weise lässt sich feststellen, ob ich vielleicht eine Neuzuweisung übersehen habe. Das ist bei einer so kurzen Funktion zwar unwahrscheinlich, kommt bei längeren Funktionen aber durchaus vor.

```
class Order...
constructor(quantity, item) {
  this._quantity = quantity;
  this._item = item;
}
get price() {
  const basePrice = this._quantity * this._item.price;
  var discountFactor = 0.98;
  if (basePrice > 1000) discountFactor -= 0.03;
  return basePrice * discountFactor;
}
```

Anschließend extrahiere ich die rechte Seite der Zuweisung und erstelle dafür eine Getter-Methode:

```
class Order...
get price() {
  const basePrice = this.basePrice;
  var discountFactor = 0.98;
  if (basePrice > 1000) discountFactor -= 0.03;
  return basePrice * discountFactor;
}

get basePrice() {
  return this._quantity * this._item.price;
}
```

Ich teste und wende *Variable inline platzieren* (Abschnitt 6.4) an:

```
class Order...
get price() {
  const basePrice = this.basePrice;
  var discountFactor = 0.98;
  if (this.basePrice > 1000) discountFactor -= 0.03;
  return this.basePrice * discountFactor;
}
```

Ich führe dann die gleichen Schritte mit `discountFactor` durch, angefangen mit *Funktion extrahieren* (Abschnitt 6.1):

```
class Order...
```

```
get price() {
  const discountFactor = this.discountFactor;
  return this.basePrice * discountFactor;
}
get discountFactor() {
  var discountFactor = 0.98;
  if (this.basePrice > 1000) discountFactor -= 0.03;
  return discountFactor;
}
```

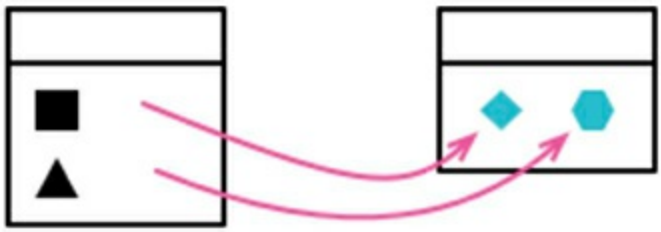
In diesem Fall muss die extrahierte Funktion beide Zuweisungen zu `discountFactor` berücksichtigen. Aus der ursprünglichen Variable mache ich jedoch eine Konstante.

Anschließend nehme eine Inline-Platzierung vor:

```
get price() {
  return this.basePrice * this.discountFactor;
}
```

7.5 Klasse extrahieren (Extract Class)

Umkehrung von: *Klasse inline platzieren* (Inline Class, Abschnitt 7.6)



```
class Person {
  get officeAreaCode() {return this._officeAreaCode;}
  get officeNumber() {return this._officeNumber;}
```

```
class Person {
  get officeAreaCode() {
    return this._telephoneNumber.areaCode;
  }
  get officeNumber() {return
this._telephoneNumber.number;}
}
class TelephoneNumber {
  get areaCode() {return this._areaCode;}
  get number() {return this._number;}
}
```

7.5.1 MOTIVATION

Sie haben vermutlich in Lehrbüchern gelesen, dass eine Klasse eine knackige Abstraktion sein sollte, dass sie nur einige wenige klare Verantwortlichkeiten übernimmt und so weiter. In der Praxis ist es aber so, dass Klassen wachsen. Sie fügen mal hier ein paar Operationen hinzu, mal da ein paar Daten und bürden der Klasse eine zusätzliche Verantwortlichkeit auf, weil Sie denken, diese sei keine eigene Klasse wert – doch wenn die Verantwortlichkeiten zunehmen, wird die Klasse irgendwann zu komplex. Schon bald ist ihre Klasse so

knackig wie eine in der Mikrowelle zubereitete Ente.

Stellen sie sich eine Klasse mit vielen Methoden und einer großen Menge Daten vor. Eine Klasse, die zu groß ist, um leicht verständlich zu sein. Sie müssen überlegen, wo sie aufgeteilt werden kann – und eine Unterteilung vornehmen. Wenn eine Teilmenge der Daten und eine Teilmenge der Methoden offenbar zusammengehören, ist das ein gutes Zeichen. Ein weiteres gutes Zeichen sind Teilmengen der Daten, die sich für gewöhnlich gleichzeitig ändern oder besonders abhängig voneinander sind. Ein nützlicher Test ist, sich zu fragen, was passieren würde, wenn Sie einen Teil der Daten oder eine Methode entfernten. Welche anderen Felder und Methoden würden dadurch unsinnig werden?

Ein Hinweis, der sich bei der Entwicklung oft erst später zeigt, ist die Form, in der die Vererbung erfolgt. Sie werden vielleicht feststellen, dass nur einige wenige Merkmale von der Vererbung betroffen sind oder dass einige Merkmale auf unterschiedliche Weise vererbt werden.

7.5.2 VORGEHEN

- Entscheiden Sie, wie die Verantwortlichkeiten der Klasse aufgeteilt werden.
- Erstellen Sie eine Unterklasse zur Aufnahme der abgespaltenen Verantwortlichkeiten.

  Benennen Sie die ursprüngliche Basisklasse um, wenn die

Verantwortlichkeiten nicht mehr zu ihrem Namen passen.

- Erstellen Sie bei der Instanziierung der ursprünglichen Klasse eine Instanz der Unterklasse und fügen Sie eine Verknüpfung von der Basisklasse zur Unterklasse hinzu.
- Wenden Sie *Feld verschieben* (Abschnitt 8.2) auf diejenigen Felder an, die Sie verschieben möchten. Führen Sie nach jeder Verschiebung Tests durch.
- Verwenden Sie *Funktion verschieben* (Abschnitt 8.1), um Methoden in die Unterklasse zu verschieben. Verschieben Sie zuerst Low-Level-Methoden (diejenigen, die aufgerufen werden, statt selbst Methoden aufzurufen). Führen Sie nach jeder Verschiebung Tests durch.
- Überprüfen Sie die Schnittstellen der beiden Klassen, entfernen Sie überflüssige Methoden und ändern Sie die Namen, sodass sie den neuen Gegebenheiten besser Rechnung tragen.
- Treffen Sie die Entscheidung, ob die neue Unterklasse öffentlich bereitgestellt wird. Wenn das der Fall ist, sollten Sie in Betracht ziehen, *Referenz durch Wert ersetzen* (Abschnitt 9.4) auf die neue Unterklasse anzuwenden.

7.5.3 BEISPIEL

Ich verwende wieder eine einfache `Person`-Klasse:

```
class Person...
get name() {return this._name;}
set name(arg) {this._name = arg;}
get telephoneNumber() {
  return `(${this.officeAreaCode})
${this.officeNumber}`;
}
get officeAreaCode() {return this._officeAreaCode;}
set officeAreaCode(arg) {this._officeAreaCode = arg;}
get officeNumber() {return this._officeNumber;}
```

```
set officeNumber(arg) {this._officeNumber = arg;}
```

Hier kann ich das Verhalten der Telefonnummer in eine eigene Klasse auslagern. Zunächst definiere ich eine leere Klasse:

```
class TelephoneNumber {
}
```

Das war ein Kinderspiel! Jetzt erstelle ich im Konstruktor der Klasse `Person` eine Instanz der Telefonnummer:

```
class Person...
constructor() {
  this._telephoneNumber = new TelephoneNumber();
}

class TelephoneNumber...
get officeAreaCode() {return this._officeAreaCode;}
set officeAreaCode(arg) {this._officeAreaCode = arg;}
```

Anschließend wende ich *Feld verschieben* (Abschnitt 8.2) auf eins der Felder an:

```
class Person...
get officeAreaCode() {
  return this._telephoneNumber.officeAreaCode;
}
set officeAreaCode(arg) {
```

```
  this._telephoneNumber.officeAreaCode = arg;
}
```

Ich teste und verschiebe dann das nächste Feld:

```
class TelephoneNumber...
get officeNumber() {return this._officeNumber;}
set officeNumber(arg) {this._officeNumber = arg;}

class Person...
get officeNumber() {
  return this._telephoneNumber.officeNumber;
}
set officeNumber(arg) {
  this._telephoneNumber.officeNumber = arg;
}
```

Ich teste erneut und verschiebe die Methode `telephoneNumber`:

```
class TelephoneNumber...
get telephoneNumber() {
  return `(${this.officeAreaCode})
${this.officeNumber}`;
}
class Person...
get telephoneNumber() {
  return this._telephoneNumber.telephoneNumber;
}
```

Jetzt sollte ich aufräumen. Dass der Code für die Telefonnummer »office« enthält, ergibt keinen Sinn,

deshalb entferne ich den Begriff:

```
class TelephoneNumber...
get areaCode() {return this._areaCode;}
set areaCode(arg) {this._areaCode = arg;}
get number() {return this._number;}
set number(arg) {this._number = arg;}

class Person...
get officeAreaCode() {return
this._telephoneNumber.areaCode;}
set officeAreaCode(arg) {
  this._telephoneNumber.areaCode = arg;
}
get officeNumber() {return
this._telephoneNumber.number;}
set officeNumber(arg) {this._telephoneNumber.number =
arg;}
```

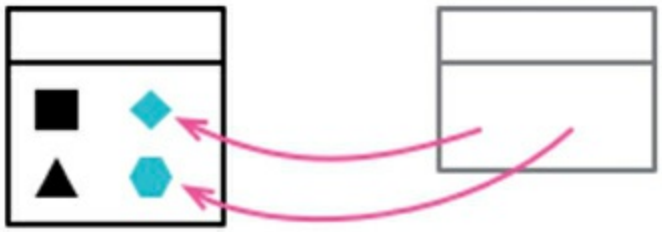
Die Methode `telephoneNumber` der Klasse `TelephoneNumber` ist auch nicht besonders sinnvoll, deshalb wende ich *Funktionsdeklaration ändern* (Abschnitt 6.5) an:

```
class TelephoneNumber...
toString() {return `(${this.areaCode})
${this.number}`;}

class Person...
get telephoneNumber() {
  return this._telephoneNumber.toString();
}
```

Telefonnummern sind grundsätzlich nützlich, deshalb werde ich das neue Objekt den Clients bereitstellen. Ich kann die ehemaligen »office«-Methoden durch Zugriffsfunktionen für die Telefonnummer ersetzen. Das funktioniert aber besser, wenn ich dafür ein Value Object (`https://martinfowler.com/bliki/ValueObject.html`) verwende, sodass ich zuerst *Referenz durch Wert ersetzen* (Abschnitt 9.4) anwende (das Beispiel dort zeigt, wie ich das für die Telefonnummer tun würde).

7.6 Klasse inline platzieren (Inline Class)

Umkehrung von: *Klasse extrahieren* (Extract Class, Abschnitt 7.5)



```
class Person {
  get officeAreaCode() {
    return this._telephoneNumber.areaCode;
  }
  get officeNumber() {return
this._telephoneNumber.number;}
}

class TelephoneNumber {
  get areaCode() {return this._areaCode;}
```

```
  get number() {return this._number;}
}
```

```
class Person {
  get officeAreaCode() {return this._officeAreaCode;}
  get officeNumber() {return this._officeNumber;}
}
```

7.6.1 MOTIVATION

Klasse inline platzieren ist die Umkehrung von *Klasse extrahieren* (Abschnitt 7.5). Ich verwende *Klasse inline platzieren*, wenn eine Klasse keinen nennenswerten Beitrag mehr leistet und deswegen auch nicht mehr vorhanden sein sollte. Oftmals ergibt sich aus einem Refactoring, dass Verantwortlichkeiten aus der Klasse verlagert werden, sodass nur noch wenig verbleibt. In diesem Fall verschiebe ich die Klasse in eine andere, die am häufigsten Gebrauch von der verbliebenen Funktionalität macht.

Ein weiterer Grund, *Klasse inline platzieren* anzuwenden, ist gegeben, wenn zwei Klassen vorhanden sind, deren Funktionalität ich anders aufteilen möchte. Möglicherweise ist es einfacher, zunächst *Klasse inline platzieren* anzuwenden, um beide in einer Klasse zu vereinen und anschließend mit *Klasse extrahieren*

(Abschnitt 7.5) eine neue Unterteilung vorzunehmen. Hierbei handelt es sich um einen allgemeinen Ansatz für eine Neuorganisation: Manchmal ist es einfacher, die Bestandteile einzeln von einem Kontext in einen anderen zu verschieben. Gelegentlich ist es jedoch besser, die beiden Kontexte zunächst durch ein Inline-Refactoring zu vereinen und anschließend ein Extraktions-Refactoring anzuwenden, um die Neuaufteilung vorzunehmen.

7.6.2 VORGEHEN

- Erstellen Sie in der Zielklasse Funktionen für alle öffentlichen Funktionen der Ursprungsklasse. Diese Funktionen sollten den Aufruf lediglich an die ursprüngliche Klasse weiterleiten.
- Ändern Sie alle Verweise auf die Methoden der Ursprungsklasse so, dass sie die weiterleitenden Funktionen der Zielklasse verwenden. Führen Sie nach jeder Änderung Tests durch.
- Verschieben Sie alle Funktionen und Daten von der ursprünglichen Klasse in die Zielklasse und führen Sie nach jeder Verschiebung Tests durch, bis die Ursprungsklasse leer ist.
- Löschen Sie die Ursprungsklasse und halten Sie eine kurze, schlichte Trauerfeier ab.

7.6.3 BEISPIEL

Die folgende Klasse enthält verschiedene Informationen zum Warenversand:

```
class TrackingInformation {
  get shippingCompany() {return
this._shippingCompany;}
```

```
  set shippingCompany(arg) {this._shippingCompany =
arg;}
  get trackingNumber() {return this._trackingNumber;}
  set trackingNumber(arg) {this._trackingNumber =
arg;}
  get display() {
    return `${this.shippingCompany}:
${this.trackingNumber}`;
  }
}
```

Sie wird als Teil einer `Shipment`-Klasse verwendet:

```
class Shipment...
get trackingInfo() {
return this._trackingInformation.display;
}
get trackingInformation() {return
this._trackingInformation;}
set trackingInformation(aTrackingInformation) {
  this._trackingInformation = aTrackingInformation;
}
```

Sie mag früher von Nutzen gewesen sein, leistet aber keinen wesentlichen Beitrag mehr, deshalb möchte ich sie in der `Shipment`-Klasse inline platzieren.

Als Erstes suche ich nach Stellen, an denen die Methoden von `TrackingInformation` aufgerufen werden.

```
Aufrufer...
```

```
aShipment.trackingInformation.shippingCompany =
                                  request.vendor;
```

Ich verschiebe alle Funktionen dieser Art in die `Shipment`-Klasse, gehe bei der Anwendung von *Funktion verschieben* (Abschnitt 8.1) allerdings ein wenig anders vor als sonst. In diesem Fall erstelle ich eine weiterleitende Methode in der `Shipment`-Klasse und passe den Client an, sodass er diese Methode aufruft:

```
class Shipment...
set shippingCompany(arg) {
  this._trackingInformation.shippingCompany = arg;
}
Aufrufer...
aShipment.trackingInformation.shippingCompany =
                                  request.vendor;
```

Auf diese Weise verfahre ich mit allen Elementen von `TrackingInformation`, die von Clients verwendet werden. Wenn das erledigt ist, kann ich alle Elemente von `TrackingInformation` in die `Shipment`-Klasse verschieben.

Zunächst wende ich *Funktion inline platzieren* (Abschnitt 6.2) auf die `display`-Methode an:

```
class Shipment...
get trackingInfo() {
```

```
  return `${this.shippingCompany}:
${this.trackingNumber}`;
}
```

Anschließend verschiebe ich das Feld `shippingCompany`:

```
get shippingCompany() {
  return this._trackingInformation._shippingCompany;
}
set shippingCompany(arg) {
  this._trackingInformation._shippingCompany = arg;
}
```

Ich verwende hier nicht die vollständige Vorgehensweise von *Feld verschieben* (Abschnitt 8.2), weil ich in diesem Fall einen Verweis auf `shippingCompany` nur innerhalb der `Shipment`-Klasse verwende, die das Ziel der Verschiebung ist. Deshalb entfällt der Schritt, einen Verweis dafür zu erstellen.

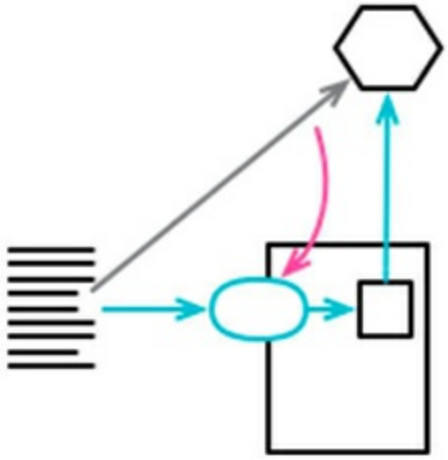
Ich fahre fort, bis alle Objekte verschoben sind. Anschließend kann ich die `TrackingInformation`-Klasse löschen:

```
class Shipment...
get trackingInfo() {
  return `${this.shippingCompany}:
${this.trackingNumber}`;
}
get shippingCompany() {return this._shippingCompany;}
```

```
set shippingCompany(arg) {this._shippingCompany =
arg;}
get trackingNumber() {return this._trackingNumber;}
set trackingNumber(arg) {this._trackingNumber = arg;}
```

7.7 Delegation verbergen (Hide Delegate)

Umkehrung von: *Vermittler entfernen* (Remove Middle Man, Abschnitt 7.8)



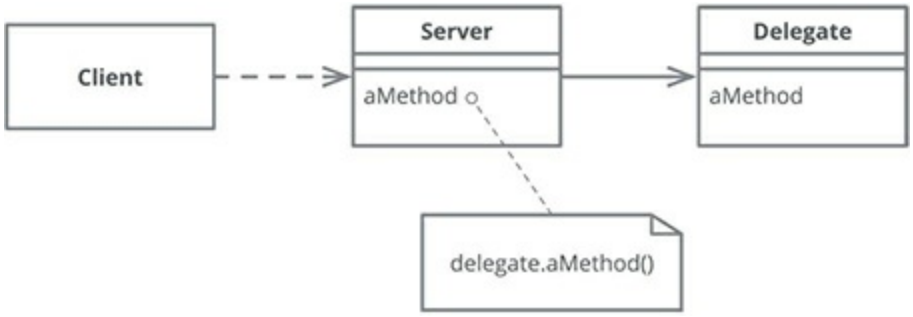
```
manager = aPerson.department.manager;
```

```
manager = aPerson.manager;
class Person {
  get manager() {return this.department.manager;}
```

7.7.1 MOTIVATION

Einer der Schlüssel – wenn nicht *der* Schlüssel zu einem guten modularen Design – ist die Kapselung. Kapselung bedeutet, dass Module weniger Kenntnisse über andere Bestandteile des Systems benötigen. Deshalb müssen bei Änderungen weniger Module darauf reagieren – und das vereinfacht es, Änderungen vorzunehmen.

Wenn wir zum ersten Mal mit Objektorientierung in Berührung kommen, lernen wir, dass Kapselung bedeutet, unsere Felder zu verbergen. Mit zunehmender Erfahrung stellen wir jedoch fest, dass sich noch mehr als das kapseln lässt.



Wenn der Client-Code eine Methode auf einem Objekt (hier Delegate) in einem Feld des Servers aufruft, muss dieses Objekt dem Client bekannt sein. Wenn sich die Schnittstelle des Objekts ändert, sind davon alle Clients des Servers betroffen, die das Objekt verwenden. Ich kann diese Abhängigkeit entfernen, indem ich eine einfache weiterleitende Methode für den Server erstelle, die das Objekt, an das der Aufruf delegiert wird (auch Delegate genannt), verbirgt. Dann betreffen jegliche

Änderungen, die ich am Delegate vornehme, nicht mehr die Clients, sondern nur den Server.

7.7.2 VORGEHEN

- Erstellen Sie für jede Methode des Delegates einfache weiterleitende Methoden in der Server-Klasse.
- Passen Sie den Client an, sodass er den Server aufruft. Führen Sie nach jeder Änderung Tests durch.
- Wenn kein Client mehr auf den Delegate zugreifen muss, können Sie die Zugriffsfunktion des Servers für den Delegate entfernen.
- Testen Sie.

7.7.3 BEISPIEL

Ich verwende hier eine Personen- und eine Abteilungs-Klasse (`Person` bzw. `Department`):

```
class Person...
constructor(name) {
  this._name = name;
}
get name() {return this._name;}
get department() {return this._department;}
set department(arg) {this._department = arg;}

class Department...
get chargeCode() {return this._chargeCode;}
set chargeCode(arg) {this._chargeCode = arg;}
get manager() {return this._manager;}
set manager(arg) {this._manager = arg;}
```

Der Client-Code soll den Abteilungsleiter (`manager`) einer Person ermitteln. Dafür muss zunächst einmal die Abteilung in Erfahrung gebracht werden:

```
Client-Code...
manager = aPerson.department.manager;
```

Dem Client wird dadurch offenbart, wie die `Department`-Klasse funktioniert und dass sie den Abteilungsleiter kennt. Ich kann diese Kopplung verringern, indem ich die `Department`-Klasse vor dem Client verberge. Zu diesem Zweck erstelle ich in der Klasse `Person` eine einfache weiterleitende Methode:

```
class Person...
get manager() {return this._department.manager;}
```

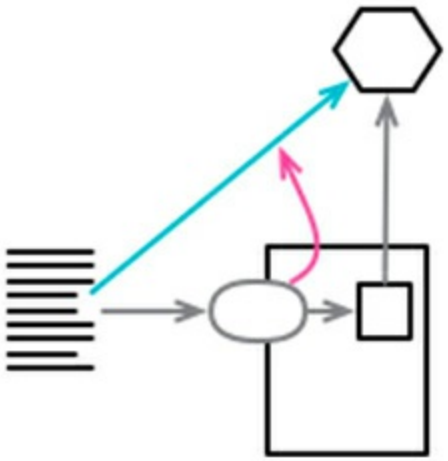
Nun muss ich alle Clients der `Person`-Klasse ändern, sodass sie die neue Methode verwenden:

```
Client-Code...
manager = aPerson.department.manager;
```

Nachdem ich die Änderung an allen Methoden der `Department`-Klasse und allen Clients der `Person`-Klasse vorgenommen habe, kann ich die `department`-Zugriffsfunktion in der `Person`-Klasse entfernen.

7.8 Vermittler entfernen (Remove Middle Man)

Umkehrung von: *Delegation verbergen* (Hide Delegate, Abschnitt 7.7)



```
manager = aPerson.manager;

class Person {
  get manager() {return this.department.manager;}
```

```
manager = aPerson.department.manager;
```

7.8.1 MOTIVATION

Im Abschnitt über die Motivation von *Delegation verbergen* (Abschnitt 7.7) habe ich über die Vorteile des

Kapselns von Objekten, an die etwas delegiert wird, gesprochen. Diese Vorteile haben jedoch ihren Preis. Jedes Mal, wenn der Client eine neue Funktionalität des Delegates verwenden möchte, muss ich dem Server eine einfache weiterleitende Methode hinzufügen. Nach dem Ergänzen weiterer Funktionalität beginnen die Weiterleitungen irritierend zu wirken. Die Server-Klasse ist nur noch ein »Vermittler« (siehe Abschnitt 3.18), und vielleicht ist es an der Zeit, dass der Client den Delegate direkt aufruft. (Dieser Code-Smell tritt meist auf, wenn jemand beim Befolgen des Gesetzes von Demeter – das ich lieber »Gelegentlich nützlicher Vorschlag von Demeter« nennen würde – übereifrig war.)

Es ist schwierig, hier das richtige Maß für das Verbergen zu finden. Dank *Delegation verbergen* (Abschnitt 7.7) und *Vermittler entfernen* wiegt das Ganze nicht so schwer. Ich kann meinen Code im Laufe der Zeit allmählich anpassen. Wenn sich ein System verändert, ändert sich auch die Grundlage für die Entscheidung, wie viel ich verberge. Eine Kapselung, die vor sechs Monaten sinnvoll war, könnte jetzt heikel sein. Refactoring bedeutet auch, dass ich mich niemals für Probleme entschuldigen muss – ich behebe sie einfach.

7.8.2 VORGEHEN

- Erstellen Sie einen Getter für den Delegate.
- Ersetzen Sie bei allen Clients, die weiterleitende Methoden aufrufen, den Aufruf durch die Zugriffsfunktion. Führen Sie nach

jeder Änderung Tests durch.

Wenn alle Aufrufe der weiterleitenden Methode ersetzt worden sind, kann diese gelöscht werden.

Bei automatisierten Refactorings können Sie *Variable kapseln* (Abschnitt 6.6) auf das Feld des Delegates anwenden. Wenden Sie anschließend *Funktion inline platzieren* (Abschnitt 6.2) auf alle Methoden an, die es verwenden.

7.8.3 BEISPIEL

Ich verwende eine `Person`-Klasse, die mit Hilfe eines `Department`-Objekts den dazugehörigen Abteilungsleiter ermittelt (wenn Sie das Buch der Reihe nach lesen, wird Ihnen dieses Bespiel möglicherweise seltsam vertraut erscheinen):

```
Client-Code...
manager = aPerson.manager;

class Person...
get manager() {return this._department.manager;}

class Department...
get manager() {return this._manager;}
```

Das ist leicht umzusetzen und kapselt die Abteilung. Wenn allerdings sehr viele Methoden so verfahren, führt es zu einer Unmenge einfacher Weiterleitungen in der `Person`-Klasse. In einem solchen Fall ist es sinnvoll, den Vermittler zu entfernen. Dazu erstelle ich als Erstes eine Zugriffsfunktion für den Delegate:

```
class Person...
get department() {return this._department;}
```

Jetzt ändere ich der Reihe nach die Clients, sodass sie direkt auf `department` zugreifen:

```
Client-Code...
manager = aPerson.department.manager;
```

Nachdem ich das bei allen Clients erledigt habe, kann ich die `manager`-Methode in der `Person`-Klasse löschen. Mit den übrigen einfachen Weiterleitungen in der `Person`-Klasse kann ich auf die gleiche Weise verfahren.

Ich kann hier auch einen Mittelweg einschlagen. Manche Weiterleitungen sind vielleicht so gebräuchlich, dass ich sie behalten möchte, um den Umgang mit Client-Code zu vereinfachen. Es gibt keinen Grund, unbedingt entweder eine Delegation zu verbergen oder einen Vermittler zu entfernen – die speziellen Umstände legen nahe, welcher Ansatz verfolgt werden sollte, und selbst bei intelligenten Leuten gehen die Meinungen darüber auseinander, was am besten funktioniert.

Wenn automatisierte Refactorings verfügbar sind, gibt es eine nützliche Variante dieser Schritte. Zunächst wende ich *Variable kapseln* (Abschnitt 6.6) auf `department` an. Der `manager`-Getter verwendet anschließend den

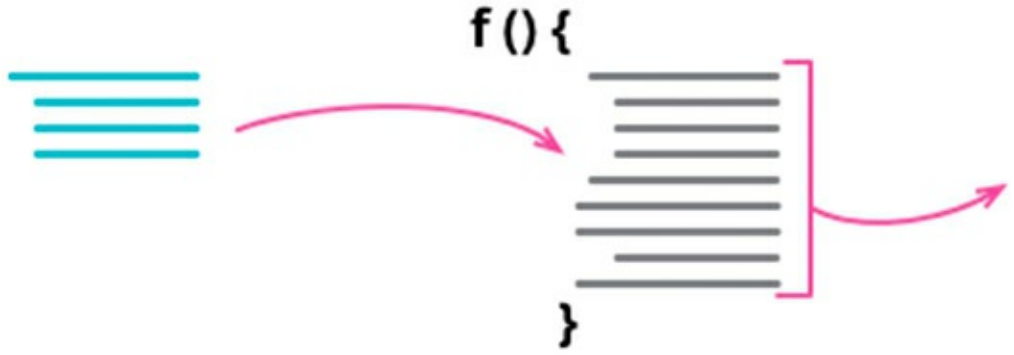
öffentlichen `department`-Getter:

```
class Person...
get manager() {return this.department.manager;}
```

In JavaScript ist diese Änderung etwas zu subtil, aber das Entfernen des Unterstrichs vor `department` *bewirkt, dass ich nicht direkt auf das Feld zugreife, sondern den neuen Getter verwende.*

Anschließend wende ich *Funktion inline platzieren* (Abschnitt 6.2) auf die `manager`-Methode an, um alle Aufrufer in einem einzigen Schritt zu ersetzen.

7.9 Algorithmus ersetzen (Substitute Algorithm)




```
function foundPerson(people) {
  for(let i = 0; i < people.length; i++) {
    if (people[i] === "Don") {
      return "Don";
```

```
    }
    if (people[i] === "John") {
      return "John";
    }
    if (people[i] === "Kent") {
      return "Kent";
    }
  }
return "";
}
```

```
function foundPerson(people) {
  const candidates = ["Don", "John", "Kent"];
  return people.find(p => candidates.includes(p)) ||
'';
}
```

7.9.1 MOTIVATION

Ich bin noch nie zu Fuß nach Rom gelaufen, aber es heißt, dass viele Wege dorthin führen. Ich bin mir sicher, dass manche Wege weniger beschwerlich sind als andere. So ist es auch bei Algorithmen. Wenn ich einen verständlicheren Weg finde, etwas zu tun, ersetze ich den komplizierten durch den verständlicheren Weg. Ein Refactoring kann eine komplexe Aufgabe in einfachere Teilaufgaben zerlegen, aber manchmal ist irgendwann der Punkt erreicht, an dem ich den gesamten Algorithmus entfernen und durch etwas Einfacheres

ersetzen muss. Das geschieht, wenn ich ein Problem besser verstehe und feststelle, dass es eine einfachere Möglichkeit gibt, es zu lösen, oder wenn ich von nun an eine Bibliothek einsetze, die Funktionalität meines Codes dupliziert.

Wenn ich einen Algorithmus verändern möchte, kann es einfacher sein, ihn zunächst durch etwas zu ersetzen, das die Durchführung meiner Änderungen vereinfacht.

Wenn ich gezwungen bin, diesen Schritt zu gehen, muss ich mich vergewissern, dass ich die betreffende Methode so weit wie möglich unterteilt habe. Einen umfangreichen, komplexen Algorithmus zu ersetzen ist äußerst schwierig. Erst durch diese Vereinfachung wird der Austausch überhaupt durchführbar.

7.9.2 VORGEHEN

- Ordnen Sie den zu ersetzenden Code so an, dass er eine vollständige Funktion bildet.
- Bereiten Sie Tests vor, die ausschließlich diese Funktion verwenden, um ihr Verhalten zu erfassen.
- Bereiten Sie den alternativen Algorithmus vor.
- Führen Sie statische Überprüfungen durch.
- Führen Sie Tests durch, um die Ergebnisse des alten und des neuen Algorithmus miteinander zu vergleichen. Wenn sie übereinstimmen, ist Ihre Arbeit erledigt. Verwenden Sie andernfalls beim Testen und Debuggen zum Vergleich den alten Algorithmus.

Kapitel 8

Verschiebungen

Bisher ging es bei den Refactorings um das Erstellen, Löschen und Umbenennen von Programmelementen. Ein weiterer wichtiger Bestandteil des Refactorings ist das Verschieben von Elementen zwischen verschiedenen Kontexten. Ich verwende *Funktion verschieben* (Abschnitt 8.1), um Funktionen in andere Klassen oder Module zu verschieben. Mit *Feld verschieben* (Abschnitt 8.2) können auch Felder verschoben werden.

Manchmal verschiebe ich auch einzelne Anweisungen. Ich verwende *Anweisungen in Funktion verschieben* (Abschnitt 8.3) und *Anweisungen in Aufrufer verschieben* (Abschnitt 8.4), um sie zwischen verschiedenen Funktionen zu verschieben, sowie *Anweisungen verschieben* (Abschnitt 8.6), um sie innerhalb einer Funktion zu verschieben. In manchen Fällen entsprechen einige Anweisungen einer bereits vorhandenen Funktion, und ich kann *Inline-Code durch Funktionsaufruf ersetzen* (Abschnitt 8.5) nutzen, um redundanten Code zu entfernen.

Für Schleifen verwende ich häufig die Refactorings *Schleife aufteilen* (Abschnitt 8.7), um zu gewährleisten,

dass eine Schleife nur eine einzige Aufgabe erledigt, sowie *Schleife durch Pipeline ersetzen* (Abschnitt 8.8), um eine Schleife vollständig loszuwerden.

Und dann gibt es auch noch das Lieblings-Refactoring so mancher talentierter Programmierer: *Toten Code entfernen* (Abschnitt 8.9). Nichts ist so zufriedenstellend wie die Anwendung des digitalen Flammenwerfers auf überflüssige Anweisungen.

8.1 Funktion verschieben (Move Function)

Ehemalige Bezeichnung: *Methode verschieben* (Move Method)

```
class Account {
  get overdraftCharge() {...}
```

```
class AccountType {
  get overdraftCharge() {...}
```

8.1.1 MOTIVATION

Das Herzstück eines guten Softwaredesigns ist dessen Modularität – also dass ich in der Lage bin, die meisten Änderungen an einem Programm vornehmen zu können, und dafür nur einen kleinen Teil des Programms verstehen muss. Um diese Modularität zu erzielen, muss ich gewährleisten, dass zusammengehörige Softwareelemente gemeinsam gruppiert sind und dass die Verbindungen zwischen ihnen leicht zu finden und zu verstehen sind. Meine Vorstellung davon, wie ich dabei verfahren sollte, ist jedoch nicht statisch – sobald ich die zu erledigende Aufgabe besser verstehe, lerne ich, wie ich die Softwareelemente am besten gruppiere. Um das zunehmende Verständnis einfließen zu lassen, muss ich die Elemente umherschieben.

Alle Funktionen gehören zu einem bestimmten Kontext. Dieser könnte global sein, aber meistens handelt es sich um eine Art Modul. In einem objektorientierten Programm ist eine Klasse der grundlegende modulare Kontext. Das Verschachteln einer Funktion in einer anderen erzeugt einen weiteren gemeinsamen Kontext. Die verschiedenen Programmiersprachen bieten unterschiedliche Formen von Modularität, die jeweils einen Kontext für eine Funktion schaffen, in dem diese leben kann.

Wenn eine Funktion häufiger auf Elemente anderer Kontexte zugreift als auf die im eigenen Umfeld

befindlichen, ist das einer der nächstliegenden Gründe, die Funktion zu verschieben. Die Funktion mit diesen Elementen zu vereinen, verbessert oft die Kapselung, sodass andere Teile der Software weniger stark von den Details dieses Moduls abhängen.

Ich könnte eine Funktion auch an eine bestimmte Stelle verschieben wollen, weil ihre Aufrufer sich dort befinden, oder weil ich sie dort in meinem nächsten Optimierungsschritt aufrufen möchte. Eine Hilfsfunktion, die innerhalb einer Funktion definiert ist, könnte auch als eigenständige Funktion von Nutzen sein, sodass es sich lohnt, sie in einen größeren Gültigkeitsbereich zu verschieben. Eine Methode einer Klasse könnte für mich einfacher zu verwenden sein, wenn sie zu einer anderen Klasse gehören würde.

Die Entscheidung, eine Funktion zu verschieben, fällt nur selten leicht. Zwecks Entscheidungsfindung untersuche ich den aktuellen und den potenziellen Kontext der Funktion. Ich muss berücksichtigen, von welchen Funktionen sie aufgerufen wird, welche sie selbst aufruft und welche Daten sie verwendet. Häufig stelle ich fest, dass für eine Gruppe von Funktionen ein neuer Kontext notwendig ist, den ich dann mit *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) oder *Klasse extrahieren* (Abschnitt 7.5) erstelle. Es kann schwierig sein, zu entscheiden, welcher Ort für eine Funktion am besten geeignet ist. Je schwerer die Entscheidung fällt, desto weniger wichtig ist sie häufig.

Ich halte es für sinnvoll, zu versuchen, mit Funktionen in einem einzelnen Kontext zu arbeiten, weil ich auf diese Art erkennen kann, wie gut sie zusammenpassen. Und wenn sie nicht passen, kann ich sie später wieder verschieben.

8.1.2 VORGEHEN

- Untersuchen Sie alle Programmelemente, die von der ausgewählten Funktion in ihrem aktuellen Kontext verwendet werden. Überlegen Sie, ob diese ebenfalls verschoben werden sollten.

  Wenn ich auf eine aufgerufene Funktion stoße, die ebenfalls verschoben werden sollte, verschiebe ich sie für gewöhnlich zuerst. Auf diese Weise kommt beim Verschieben von Funktionsgruppen zuerst diejenige Funktion an die Reihe, deren Abhängigkeit von den anderen Funktionen der Gruppe am geringsten ist.

  Wenn eine High-Level-Funktion der einzige Aufrufer von Unterfunktionen ist, können Sie diese in der High-Level-Methode inline platzieren, verschieben und am Zielort erneut extrahieren.

- Überprüfen Sie, ob die ausgewählte Funktion eine polymorphe Methode ist.

  In objektorientierten Programmiersprachen muss ich die Deklaration von Basis- und Unterklassen berücksichtigen.

- Kopieren Sie die Funktion in ihren Zielkontext und passen Sie sie an ihr neues Zuhause an.

  Wenn im Funktionskörper Programmelemente aus dem ursprünglichen Kontext verwendet werden, muss ich diese als Parameter übergeben oder einen Verweis auf den ursprünglichen Kontext bereitstellen.

  Beim Verschieben einer Funktion muss ich mir meistens einen anderen Namen für sie überlegen, der für das neue Umfeld besser geeignet ist.

- Führen Sie eine statische Analyse durch.
- Finden Sie heraus, wie Sie vom ursprünglichen Kontext aus auf die neue Funktion zugreifen können.
- Machen Sie aus der ursprünglichen Funktion eine delegierende Funktion.
- Testen Sie.
- Ziehen Sie in Betracht, *Funktion inline platzieren* (Abschnitt 6.2) auf die ursprüngliche Funktion anzuwenden.

  Die ursprüngliche Funktion kann auf unbestimmte Zeit als weiterleitende Funktion vorhanden bleiben. Wenn jedoch die Aufrufer ihr Ziel auch einfach direkt aufrufen können, ist es besser, den Vermittler zu entfernen.

8.1.3 BEISPIEL: VERSCHIEBEN EINER VERSCHACHTELTEN FUNKTION IN DIE OBERSTE EBENE

Ich verwende hier eine Funktion, die anhand von GPS-Daten die zurückgelegte Wegstrecke berechnet.

```
function trackSummary(points) {
  const totalTime = calculateTime();
  const totalDistance = calculateDistance();
  const pace = totalTime / 60 / totalDistance ;
  return {
    time: totalTime,
    distance: totalDistance,
    pace: pace
  };

  function calculateDistance() {
    let result = 0;
    for (let i = 1; i < points.length; i++) {
      result += distance(points[i-1], points[i]);
```

```
    }
    return result;
  }

  function distance(p1,p2) { ... }
  function radians(degrees) { ... }
  function calculateTime() { ... }
}
```

Ich möchte `calculateDistance` auf die oberste Ebene verschieben, damit ich Wegstrecken ohne all die anderen Bestandteile der Streckenzusammenfassung berechnen kann.

Als Erstes kopiere ich die Funktion in die oberste Ebene.

```
function trackSummary(points) {
  const totalTime = calculateTime();
  const totalDistance = calculateDistance();
  const pace = totalTime / 60 / totalDistance ;
  return {
    time: totalTime,
    distance: totalDistance,
    pace: pace
  };

  function calculateDistance() {
    let result = 0;
    for (let i = 1; i < points.length; i++) {
      result += distance(points[i-1], points[i]);
    }
    return result;
  }

  function distance(p1,p2) { ... }
```

```
  function radians(degrees) { ... }
  function calculateTime() { ... }
}

function calculateDistance() {
  let result = 0;
  for (let i = 1; i < points.length; i++) {
    result += distance(points[i-1], points[i]);
  }
  return result;
}
```

Wenn ich eine Funktion auf diese Art kopiere, ändere ich den Namen, damit ich die beiden Funktionen sowohl im Code als auch in meinem Kopf auseinanderhalten kann. Ich möchte zu diesem Zeitpunkt nicht über eine geeignete Bezeichnung nachdenken, deshalb verwende ich einen temporären Namen.

Das Programm funktioniert noch, aber meine statische Analyse beklagt sich zu recht, denn die neue Funktion enthält zwei undefinierte Symbole: `distance` und `points`. Es ist naheliegend, `points` als Parameter zu übergeben:

```
function top_calculateDistance(points) {
  let result = 0;
  for (let i = 1; i < points.length; i++) {
    result += distance(points[i-1], points[i]);
  }
  return result;
}
```

Ich könnte mit `distance` ebenso verfahren, aber es ist vielleicht sinnvoller, den Wert zusammen mit `calculateDistance` zu verschieben. Hier ist der relevante Codeabschnitt:

```
function trackSummary...
function distance(p1,p2) {
  // Haversine-Formula siehe
  // http://www.movable-type.co.uk/scripts/latlong.html
  const EARTH_RADIUS = 3959; // in Meilen
  const dLat = radians(p2.lat) - radians(p1.lat);
  const dLon = radians(p2.lon) - radians(p1.lon);
  const a = Math.pow(Math.sin(dLat / 2),2)
          + Math.cos(radians(p2.lat))
          * Math.cos(radians(p1.lat))
          * Math.pow(Math.sin(dLon / 2), 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-
a));
  return EARTH_RADIUS * c;
}
function radians(degrees) {
  return degrees * Math.PI / 180;
}
```

Ich stelle fest, dass `distance` nur `radians` verwendet, `radians` wiederum verwendet im aktuellen Umfeld selbst gar nichts. Anstatt die Funktionen zu übergeben, könnte ich sie vielleicht ebenfalls verschieben. Ich unternehme einen kleinen Schritt in diese Richtung, indem ich sie aus ihrem aktuellen Kontext entferne und sie in der verschachtelten Funktion `calculateDistance` unterbringe.

```
function trackSummary(points) {
  const totalTime = calculateTime();
  const totalDistance = calculateDistance();
  const pace = totalTime / 60 / totalDistance ;
  return {
    time: totalTime,
    distance: totalDistance,
    pace: pace
  };

  function calculateDistance() {
    let result = 0;
    for (let i = 1; i < points.length; i++) {
      result += distance(points[i-1], points[i]);
    }
    return result;

  function distance(p1,p2) { ... }
  function radians(degrees) { ... }
}
```

Auf diese Weise kann ich sowohl die statische Analyse als auch Tests verwenden, um herauszufinden, ob es irgendwelche Komplikationen gibt. In diesem Fall ist alles in Ordnung, also kopiere ich die Funktionen und füge sie in `top_calculateDistance` ein.

```
function top_calculateDistance(points) {
  let result = 0;
  for (let i = 1; i < points.length; i++) {
    result += distance(points[i-1], points[i]);
  }
  return result;
```

```
  function distance(p1,p2) { ... }
  function radians(degrees) { ... }
}
```

Auch hier ändert die Kopie nichts an der Funktionsweise des Programms, verschafft mir jedoch die Gelegenheit für weitere statische Analysen. Hätte ich nicht bemerkt, dass `radians` von `distance` aufgerufen wird, wäre das dem Linter bei diesem Schritt aufgefallen.

Nun ist alles vorbereitet, und es ist an der Zeit, die wesentliche Änderung durchzuführen – die ursprüngliche `calculateDistance`-Funktion soll jetzt `top_calculateDistance` aufrufen.

```
function trackSummary(points) {
  const totalTime = calculateTime();
  const totalDistance = calculateDistance();
  const pace = totalTime / 60 / totalDistance ;
  return {
    time: totalTime,
    distance: totalDistance,
    pace: pace
  };

  function calculateDistance() {
    return top_calculateDistance(points);
  }
```

Jetzt ist der entscheidende Moment gekommen, Tests auszuführen, um ausführlich zu prüfen, ob die

verschobene Funktion wohlbehalten in ihrem neuen Zuhause angekommen ist.

Das nun Folgende hat eine gewisse Ähnlichkeit mit dem Auspacken der Kartons nach einem Umzug. Zunächst muss entschieden werden, ob die ursprüngliche Funktion, die jetzt nur noch eine Delegation vornimmt, erhalten bleibt oder nicht. In diesem Fall gibt es nur wenige Aufrufer, die, wie es bei verschachtelten Funktionen üblich ist, alle lokal angesiedelt sind. Ich bin also froh, dass ich sie loswerden kann.

```
function trackSummary(points) {
  const totalTime = calculateTime();
  const totalDistance = top_calculateDistance(points);
  const pace = totalTime / 60 / totalDistance ;
  return {
    time: totalTime,
    distance: totalDistance,
    pace: pace
  };
```

Jetzt ist auch ein guter Zeitpunkt, um über den Namen der Funktion nachzudenken. Da Funktionen auf oberster Ebene praktisch überall sichtbar sind, möchte ich den bestmöglichen Namen vergeben. `totalDistance` scheint mir eine gute Wahl zu sein. Allerdings kann ich diese Bezeichnung nicht sofort verwenden, weil es innerhalb von `trackSummary` bereits eine Variable dieses Namens gibt. Ich wüsste jedoch nicht, warum ich

diese Variable beibehalten sollte, also wende ich *Variable inline platzieren* (Abschnitt 6.4) auf sie an.

```
function trackSummary(points) {
  const totalTime = calculateTime();
  const pace = totalTime / 60 / totalDistance(points);
  return {
    time: totalTime,
    distance: totalDistance(points),
    pace: pace
  };

function totalDistance(points) {
  let result = 0;
  for (let i = 1; i < points.length; i++) {
    result += distance(points[i-1], points[i]);
  }
  return result;
}
```

Wenn ich diese Variable aus irgendeinem Grund erhalten müsste, würde ich sie in `totalDistanceCache` oder `distance` umbenennen.

Die Funktionen `distance` und `radians` sind nicht von irgendwelchen Werten innerhalb von `totalDistance` abhängig. Deshalb ziehe ich es vor, sie ebenfalls auf die oberste Ebene zu verschieben, sodass sich nun alle vier Funktionen dort befinden.

```
function trackSummary(points) { ... }
function totalDistance(points) { ... }
```

```
function distance(p1,p2) { ... }
function radians(degrees) { ... }
```

Manche Entwickler würden es vorziehen, `distance` und `radians` innerhalb von `totalDistance` zu belassen, um ihre Sichtbarkeit zu begrenzen. In manchen Programmiersprachen wäre das eine Überlegung wert, andererseits verfügt JavaScript seit ES 2015 über einen ausgezeichneten Modulmechanismus, der zur Kontrolle der Sichtbarkeit von Funktionen bestens geeignet ist. Im Allgemeinen bin ich bei der Verwendung verschachtelter Funktionen vorsichtig, weil es dabei allzu leicht zu verborgenen wechselseitigen Beziehungen zwischen den Daten kommen kann, die sich nur schwer nachverfolgen lassen.

8.1.4 BEISPIEL: VERSCHIEBUNGEN ZWISCHEN KLASSEN

Zur Veranschaulichung dieser Variante von *Funktion verschieben* verwende ich eine Konto-Klasse (`Account`) und eine Methode für Überziehungsgebühren (`overdraftCharge`):

```
class Account...
get bankCharge() {
  let result = 4.5;
  if (this._daysOverdrawn > 0)
    result += this.overdraftCharge;
  return result;
}
```

```
get overdraftCharge() {
  if (this.type.isPremium) {
    const baseCharge = 10;
    if (this.daysOverdrawn <= 7)
      return baseCharge;
    else
      return baseCharge + (this.daysOverdrawn - 7) *
0.85;
  }
  else
    return this.daysOverdrawn * 1.75;
}
```

Die folgenden Änderungen führen zu verschiedenen Kontotypen, für die unterschiedliche Algorithmen zur Berechnung der Überziehungsgebühren verwendet werden. Es erscheint also angebracht, `overdraftCharge` in die Klasse `AccountType` zu verschieben.

Der erste Schritt ist hier, zu prüfen, welche Funktionalität in der `overdraftCharge`-Methode verwendet wird, und zu erwägen, ob es sinnvoll wäre, eine Reihe von Methoden gemeinsam zu verschieben. In diesem Fall muss die `daysOverdrawn`-Methode in der `Account`-Klasse verbleiben, weil sich die Anzahl der Tage einer Überziehung von Konto zu Konto unterscheiden wird.

Als Nächstes kopiere ich den Inhalt der Methode in die Klasse `AccountType` und passe ihn entsprechend ein:

```
class AccountType...
overdraftCharge(daysOverdrawn) {
  if (this.isPremium) {
    const baseCharge = 10;
    if (daysOverdrawn <= 7)
      return baseCharge;
    else
      return baseCharge + (daysOverdrawn - 7) * 0.85;
  } else
      return daysOverdrawn * 1.75;
}
```

Um die Methode an ihr neues Umfeld anzupassen, muss ich zwei Aufrufe bzw. deren Empfänger an den Gültigkeitsbereich anpassen. `isPremium` ist jetzt ein einfacher Aufruf von `this`. Bei `daysOverdrawn` muss ich entscheiden, ob ich den Wert oder das Konto übergebe. Für den Moment übergebe ich den einfachen Wert, aber das könnte sich zukünftig noch ändern, wenn ich weitere Informationen über das Konto benötige – insbesondere, wenn diese vom Kontotyp abhängig sind.

Jetzt ersetze ich den ursprünglichen Methodenrumpf durch eine Delegation:

```
class Account...
get bankCharge() {
  let result = 4.5;
  if (this._daysOverdrawn > 0)
    result += this.overdraftCharge;
  return result;
}
```

```
get overdraftCharge() {
  return
this.type.overdraftCharge(this.daysOverdrawn);
}
```

Nun steht die Entscheidung an, ob diese Weiterleitung an Ort und Stelle verbleibt oder ob `overdraftCharge` inline platziert wird. Das Inline-Platzieren ergibt Folgendes:

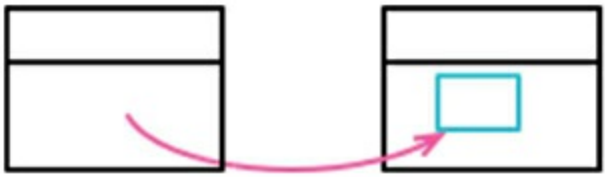
```
class Account...
get bankCharge() {
  let result = 4.5;
  if (this._daysOverdrawn > 0)
    result +=
this.type.overdraftCharge(this.daysOverdrawn);
  return result;
}
```

In den vorangegangenen Schritten habe ich `daysOverdrawn` als Parameter übergeben – wenn jedoch viele weitere Daten des Kontos übergeben werden müssen, könnte es vorteilhafter sein, das Konto selbst zu übergeben:

```
class Account...
get bankCharge() {
  let result = 4.5;
  if (this._daysOverdrawn > 0)
    result += this.overdraftCharge;
```

```
  return result;
}

get overdraftCharge() {
  return this.type.overdraftCharge(this);
}

class AccountType...
overdraftCharge(account) {
  if (this.isPremium) {
    const baseCharge = 10;
    if (account.daysOverdrawn <= 7)
      return baseCharge;
    else
      return baseCharge + (account.daysOverdrawn - 7)
* 0.85;
  }
  else
    return account.daysOverdrawn * 1.75;
}
```

8.2 Feld verschieben (Move Field)



```
class Customer {
  get plan() {return this._plan;}
  get discountRate() {return this._discountRate;}
```

```
class Customer {
  get plan() {return this._plan;}
  get discountRate() {return this.plan.discountRate;}
```

8.2.1 MOTIVATION

Beim Programmieren wird viel Code geschrieben, der Verhalten implementiert – die eigentliche Stärke eines Programms beruht jedoch auf den Datenstrukturen. Wenn geeignete Datenstrukturen vorhanden sind, die zur Aufgabe passen, ist auch der Code für das Verhalten einfach und verständlich. Ungeeignete Datenstrukturen hingegen führen zu einer Menge Code, dessen einzige Aufgabe es ist, die mangelhaft aufbereiteten Daten zu handhaben. Chaotischer, schwer verständlicher Code ist aber noch nicht alles: Solche Datenstrukturen verschleiern zudem, wie das Programm tatsächlich arbeitet.

Datenstrukturen sind also wirklich wichtig – aber wie bei so vielen anderen Dingen in der Programmierung ist es gar nicht so einfach, dabei keine Fehler zu machen. Ich führe anfangs eine Analyse durch, um die am besten geeigneten Datenstrukturen zu finden, und ich habe festgestellt, dass Erfahrung und Verfahren wie Domain-driven Design meine Fähigkeiten auf diesem Gebiet

verbessert haben. Doch trotz all meiner Kenntnisse und Erfahrungen stelle ich auch immer wieder fest, dass ich bei meinem anfänglichen Design häufig Fehler mache. Während des Programmierens lerne ich mehr über die Problemdomäne und meine Datenstrukturen. Das heißt, eine Designentscheidung, die in der einen Woche sinnvoll und richtig erscheint, kann sich schon in der nächsten als falsch erweisen.

Sobald ich merke, dass eine Datenstruktur nicht mehr passend ist, sollte sie umgehend geändert werden. Wenn ich derartige Schwachstellen in meinen Datenstrukturen belasse, werden sie mir das Denken erschweren und meinen Code bis weit in die Zukunft verkomplizieren.

Manchmal möchte ich Daten verschieben, weil ich festgestellt habe, dass ich stets ein bestimmtes Feld eines Datensatzes mit übergeben muss, wenn ich einer Funktion einen anderen Datensatz übergebe. Daten, die immer gemeinsam an eine Funktion übergeben werden, sollten am besten in einem einzelnen Datensatz gespeichert werden, um ihre Beziehung zueinander zu verdeutlichen. Änderungen spielen hier ebenfalls eine Rolle: Wenn die Änderung eines Datensatzes die Änderung eines Feldes in einem anderen Datensatz verursacht, ist das ein Anzeichen dafür, dass sich das Feld an der falschen Stelle befindet. Wenn ich das gleiche Feld in mehreren Datenstrukturen aktualisieren muss, ist das ein Hinweis darauf, dass es an eine andere Stelle verschoben werden sollte, sodass es nur einmal

aktualisiert werden muss.

Ich verwende *Feld verschieben* meist im Rahmen umfangreicherer Änderungen. Nachdem ich ein Feld verschoben habe, stelle ich oft fest, dass es für viele Aufrufer des Feldes günstiger ist, über das Zielobjekt auf diese Daten zuzugreifen als über die ursprüngliche Quelle. Die entsprechenden Änderungen führe ich dann mit nachträglichen Refactorings durch. Hin und wieder muss ich feststellen, dass ich *Feld verschieben* aufgrund der Art und Weise, wie die Daten genutzt werden, gar nicht anwenden kann. Ich muss zunächst ein Refactoring einiger Verwendungsformen durchführen, bevor ich die Verschiebung vornehmen kann.

Ich habe mich in meiner Beschreibung bislang immer auf Datensätze bezogen, aber die Überlegungen treffen auch auf Klassen und Objekte zu. Eine Klasse ist ein Datensatztyp mit zugehörigen Funktionen – und diesen Funktionen muss die gleiche Beachtung geschenkt werden wie allen anderen Daten auch. Die angehängten Funktionen erleichtern das Verschieben von Daten, denn die Daten sind durch Zugriffsfunktionen gekapselt. Ich kann die Daten verschieben und die Zugriffsfunktionen ändern – die Clients der Zugriffsfunktionen funktionieren weiterhin. Dieses Refactoring lässt sich also leichter mit Klassen durchführen, deshalb gehe ich in der nachfolgenden Beschreibung davon aus, dass Sie Klassen verwenden. Wenn ich schlichte Datensätze verwende, die keine Kapselung unterstützen, kann ich

derartige Änderungen dennoch vornehmen, die Sache ist jedoch kniffliger.

8.2.2 VORGEHEN

- Vergewissern Sie sich, dass das Feld der Quelle gekapselt ist.
- Testen Sie.
- Erstellen Sie im Zielobjekt ein Feld (und Zugriffsfunktionen).
- Führen Sie statische Überprüfungen durch.
- Vergewissern Sie sich, dass das Quellobjekt über einen Verweis auf das Zielobjekt verfügt.
- Ein bestehendes Feld oder eine bestehende Methode kann Ihnen die Referenz auf das Zielobjekt liefern. Wenn das nicht der Fall ist, überprüfen Sie, ob Sie problemlos eine entsprechende Methode erstellen können. Sollte das nicht funktionieren, müssen Sie möglicherweise im Quellobjekt ein neues Feld für den Verweis auf das Zielobjekt erstellen. Hierbei kann es sich um eine dauerhafte Änderung handeln oder auch um eine Übergangslösung, bis genügend Refactorings des weiteren Kontexts erfolgt sind.
- Passen Sie die Zugriffsfunktionen an, sodass sie das Feld des Zielobjekts verwenden.

  Wenn das Zielobjekt von mehreren Quellobjekten gemeinsam genutzt wird, sollten Sie in Betracht ziehen, zuerst den Setter zu aktualisieren, sodass er Quell- und Zielfeld ändert, und nachfolgend *Assertion einführen* (Abschnitt 10.6) verwenden, um inkonsistente Aktualisierungen zu erkennen. Sobald alles funktioniert, können Sie die Zugriffsfunktionen anpassen, sodass sie das Zielfeld verwenden.
- Testen Sie.
- Löschen Sie das Quellfeld.
- Testen Sie.

8.2.3 BEISPIEL

Hier beginne ich mit einem Kunden (`Customer`) und einem dazugehörigen Vertrag (`CustomerContract`):

```
class Customer...
constructor(name, discountRate) {
  this._name = name;
  this._discountRate = discountRate;
  this._contract = new CustomerContract(dateToday());
}
get discountRate() {return this._discountRate;}
becomePreferred() {
  this._discountRate += 0.03;
  // weitere Annehmlichkeiten
}
applyDiscount(amount) {
return
amount.subtract(amount.multiply(this._discountRate));
}

class CustomerContract...
constructor(startDate) {
  this._startDate = startDate;
}
```

Ich möchte das Feld für den Rabattsatz (`discountRate`) von der Klasse `Customer` in die Klasse `CustomerContract` verschieben.

Als Erstes muss ich *Variable kapseln* (Abschnitt 6.6) anwenden, um den Zugriff auf das Feld `discountRate` zu kapseln.

```
class Customer...
```

```
constructor(name, discountRate) {
  this._name = name;
  this._setDiscountRate(discountRate);
  this._contract = new CustomerContract(dateToday());
}
get discountRate() {return this._discountRate;}
_setDiscountRate(aNumber) {this._discountRate =
aNumber;}
becomePreferred() {
  this._setDiscountRate(this.discountRate + 0.03);
  // weitere Annehmlichkeiten
}
applyDiscount(amount) {
  return
amount.subtract(amount.multiply(this.discountRate));
}
```

Für die Aktualisierung des Rabattsatzes verwende ich eine private Methode anstelle eines Setters, denn ich möchte keinen öffentlichen Setter für den Rabattsatz bereitstellen.

Ich füge der `CustomerContract`-Klasse ein Feld und Zugriffsfunktionen hinzu.

```
class CustomerContract...
constructor(startDate, discountRate) {
  this._startDate = startDate;
  this._discountRate = discountRate;
}
get discountRate() {return this._discountRate;}
set discountRate(arg) {this._discountRate = arg;}
```

Jetzt ändere ich die Zugriffsfunktionen der `Customer`-Klasse, sodass sie das neue Feld verwenden. Daraufhin erhalte ich eine Fehlermeldung: »Cannot set property 'discountRate' of undefined« Der Grund dafür ist, dass `_setDiscountRate` aufgerufen wird, bevor im Konstruktor das `contract`-Objekt erstellt wird. Ich kehre also zum vorherigen Zustand zurück und verwende *Anweisungen verschieben* (Abschnitt 8.6), um `_setDiscountRate` hinter die Erstellung des `contract`-Objekts zu verschieben.

```
class Customer...
constructor(name, discountRate) {
  this._name = name;
  this._setDiscountRate(discountRate);
  this._contract = new CustomerContract(dateToday());
}
```

Ich teste und ändere anschließend die Zugriffsfunktionen erneut.

```
class Customer...
get discountRate() {return
this._contract.discountRate;}
_setDiscountRate(aNumber) {
  this._contract.discountRate = aNumber;
}
```

Da ich JavaScript verwende, gibt es keine Deklaration des Quellfeldes, also muss ich auch weiter nichts

entfernen.

Ändern eines einfachen Datensatzes

Dieses Refactoring ist generell leichter auf Objekte anwendbar, weil die Kapselung naturgemäß die Möglichkeit bietet, den Zugriff auf die Daten in Methoden zu verpacken. Wenn viele Funktionen auf einen einfachen Datensatz zugreifen, ist dieses Refactoring zwar immer noch sinnvoll, aber erheblich vertrackter.

Ich kann alle Lese- und Schreibvorgänge durch Zugriffsfunktionen ersetzen. Wenn das zu verschiebende Feld unveränderlich ist, kann ich bei der Wertzuweisung sowohl das Quell- als auch das Zielfeld aktualisieren und Lesevorgänge allmählich migrieren. Dennoch würde ich, sofern möglich, als Erstes *Datensatz kapseln* (Abschnitt 7.1) anwenden, um aus dem Datensatz eine Klasse zu machen, sodass ich die Änderung leichter durchführen kann.

8.2.4 BEISPIEL: VERSCHIEBUNG IN EIN GEMEINSAM GENUTZTES OBJEKT

Betrachten wir einen anderen Fall. Hier handelt es sich um ein Konto `Account` mit einem Zinssatz `interestRate`.

```
class Account...
constructor(number, type, interestRate) {
```

```
  this._number = number;
  this._type = type;
  this._interestRate = interestRate;
}
get interestRate() {return this._interestRate;}

class AccountType...
constructor(nameString) {
  this._name = nameString;
}
```

Ich möchte erreichen, dass der Zinssatz eines Kontos durch den Kontotyp festgelegt wird.

Der Zugriff auf den Zinssatz ist bereits gekapselt, also füge ich der Klasse `AccountType` nur das Feld und eine entsprechende Zugriffsfunktion hinzu.

```
class AccountType...
constructor(nameString, interestRate) {
  this._name = nameString;
  this._interestRate = interestRate;
}
get interestRate() {return this._interestRate;}
```

Wenn ich in der `Account`-Klasse die Zugriffsfunktionen aktualisiere, gibt es ein potenzielles Problem. Vor diesem Refactoring besaß jedes Konto einen eigenen Zinssatz. Jetzt sollen alle Konten den Zinssatz ihres Kontotyps verwenden. Wenn allen Konten des gleichen Typs bereits der gleiche Zinssatz zugewiesen ist, gibt es keine

sichtbare Änderung des Verhaltens, also ist das Refactoring in Ordnung. Wenn einem der Konten jedoch ein anderer Zinssatz zugewiesen ist, handelt es sich nicht mehr um ein Refactoring. Wenn die Kontodaten in einer Datenbank gespeichert werden, sollte ich die Datenbank überprüfen, um zu gewährleisten, dass allen Konten der ihrem Kontotyp entsprechende Zinssatz zugewiesen ist. Darüber hinaus kann ich in der `Account`-Klasse *Assertion einführen* (Abschnitt 10.6) anwenden.

```
class Account...
constructor(number, type, interestRate) {
  this._number = number;
  this._type = type;
  assert(interestRate === this._type.interestRate);
  this._interestRate = interestRate;
}
get interestRate() {return this._interestRate;}
```

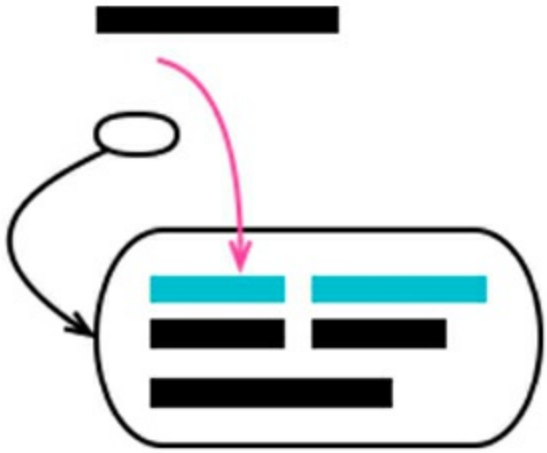
Ich könnte das System eine Weile mit aktivierter Assertion betreiben, um zu überprüfen, ob es zu Fehlermeldungen kommt. Statt die Assertion hinzuzufügen, könnte ich den Fehler auch protokollieren. Sobald ich mir sicher bin, dass ich keine Änderung des beobachtbaren Verhaltens verursache, kann ich den Zugriff modifizieren und die Aktualisierung komplett aus der `Account`-Klasse entfernen.

```
class Account...
constructor(number, type) {
```

```
  this._number = number;
  this._type = type;
}
get interestRate() {return this._type.interestRate;}
```

8.3 Anweisungen in Funktion verschieben (Move Statements into Function)

Umkehrung von: *Anweisungen in Aufrufer verschieben* (Move Statements to Caller, Abschnitt 8.4)



```
result.push(`<p>title: ${person.photo.title}</p>`);
result.concat(photoData(person.photo));

function photoData(aPhoto) {
  return [
    `<p>location: ${aPhoto.location}</p>`,
    `<p>date: ${aPhoto.date.toDateString()}</p>`,
  ];
}
```

```
result.concat(photoData(person.photo));

function photoData(aPhoto) {
  return [
    `<p>title: ${aPhoto.title}</p>`,
    `<p>location: ${aPhoto.location}</p>`,
    `<p>date: ${aPhoto.date.toDateString()}</p>`,
  ];
}
```

8.3.1 MOTIVATION

Redundanten Code zu entfernen ist ein ausgezeichnetes Vorgehen, um wartbaren Code zu erhalten. Wenn ich feststelle, dass beim Aufruf einer bestimmten Funktion der gleiche Code an verschiedenen Stellen ausgeführt wird, versuche ich, diesen sich wiederholenden Code in die Funktion selbst zu integrieren. Auf diese Weise müssen zukünftige Änderungen an dem Code nur an einer Stelle vorgenommen werden und stehen allen Aufrufern zur Verfügung. Sollten sich Änderungen am Code ergeben, kann ich ihn (oder Teile davon) mit *Anweisungen in Aufrufer verschieben* (Abschnitt 8.4) leicht wieder entfernen.

Ich verschiebe Anweisungen in eine Funktion, falls es mir leichter fällt, diese Anweisungen zu verstehen, wenn sie Teil der aufgerufenen Funktion sind. Sollte es keinen

Sinn ergeben, dass sie Bestandteil der aufgerufenen Funktion sind, wende ich einfach *Funktion extrahieren* (Abschnitt 6.1) auf die Anweisungen und die aufgerufene Funktion an. Hierbei handelt es sich im Wesentlichen um den gleichen Vorgang, den ich nachfolgend beschreibe, allerdings ohne die Schritte Inline-Platzierung und Umbenennung. Es ist nicht ungewöhnlich, so vorzugehen und diese Schritte später nachzuholen.

8.3.2 VORGEHEN

- Wenn sich der redundante Code nicht in der Nähe des Aufrufs der Zielfunktion befindet, wenden Sie *Anweisungen verschieben* (Abschnitt 8.6) auf ihn an.
- Wenn die Zielfunktion ausschließlich von der Quellfunktion aufgerufen wird, brauchen Sie den Code der Quelle einfach nur auszuschneiden, ihn im Ziel einzufügen und Tests durchzuführen. Ignorieren Sie in diesem Fall den verbleibenden Teil des Vorgehens.
- Wenn es mehrere Aufrufer gibt, können Sie *Funktion extrahieren* (Abschnitt 6.1) auf einen der Aufrufer anwenden, um sowohl den Aufruf der Zielfunktion als auch die Anweisungen, die Sie dorthin verschieben möchten, zu extrahieren. Vergeben Sie einen vorübergehenden Namen, der leicht greifbar ist.
- Modifizieren Sie die übrigen Aufrufe, sodass die neue Funktion verwendet wird. Testen Sie nach jeder Anpassung.
- Wenn alle ursprünglichen Aufrufer die neue Funktion verwenden, können Sie *Funktion inline platzieren* (Abschnitt 6.2) verwenden, um die ursprüngliche Funktion komplett in die neue Funktion zu verschieben, und die ursprüngliche Funktion löschen.
- Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um der neuen Funktion den Namen der ursprünglichen zu geben.

Vergeben Sie alternativ einen besseren Namen, falls Ihnen einer einfällt.

8.3.3 BEISPIEL

Der folgende Code gibt HTML zur Anzeige der Eigenschaften eines Fotos aus.

```
function renderPerson(outStream, person) {
  const result = [];
  result.push(`<p>${person.name}</p>`);
  result.push(renderPhoto(person.photo));
  result.push(`<p>title: ${person.photo.title}</p>`);
  result.push(emitPhotoData(person.photo));
  return result.join("\n"); }

function photoDiv(p) {
  return [
    "<div>",
    `<p>title: ${p.title}</p>`,
    emitPhotoData(p),
    "</div>",
  ].join("\n");
}

function emitPhotoData(aPhoto) {
  const result = [];
  result.push(`<p>location: ${aPhoto.location}</p>`);
  result.push(`<p>date: ${aPhoto.date.toDateString()}
</p>`);
  return result.join("\n");
}
```

Der Code enthält zwei Aufrufe von `emitPhotoData`, denen jeweils eine semantisch äquivalente Codezeile

vorausgeht. Ich möchte diese Redundanz entfernen, indem ich die Ausgabe des Titels in die Funktion `emitPhotoData` verschiebe. Gäbe es nur einen Aufrufer, würde ich den Code einfach verschieben, aber je mehr Aufrufer es gibt, desto eher bin ich gewillt, ein besseres Verfahren zu verwenden.

Ich wende zunächst *Funktion extrahieren* (Abschnitt 6.1) auf einen der Aufrufer an. Ich extrahiere die Anweisungen, die ich in die Funktion `emitPhotoData` verschieben möchte, sowie den Aufruf von `emitPhotoData` selbst.

```
function photoDiv(p) {
  return [
  "<div>",
  zznew(p),
  "</div>",
  ].join("\n");
}

function zznew(p) {
  return [
    `<p>title: ${p.title}</p>`,
    emitPhotoData(p),
  ].join("\n");
}
```

Ich sehe mir nun der Reihe nach die anderen Aufrufer von `emitPhotoData` an, ersetze die Aufrufe und die vorhergehende Anweisung durch Aufrufe der neuen Funktion.

```
function renderPerson(outStream, person) {
  const result = [];
  result.push(`<p>${person.name}</p>`);
  result.push(renderPhoto(person.photo));
  result.push(zznew(person.photo));
  return result.join("\n");
}
```

Nachdem ich das für alle Aufrufer erledigt habe, wende ich *Funktion inline platzieren* (Abschnitt 6.2) auf `emitPhotoData` an.

```
function zznew(p) {
  return [
    `<p>title: ${p.title}</p>`,
    `<p>location: ${p.location}</p>`,
    `<p>date: ${p.date.toDateString()}</p>`,
  ].join("\n");
}
```

Zum Abschluss verwende ich *Funktionsdeklaration ändern* (Abschnitt 6.5).

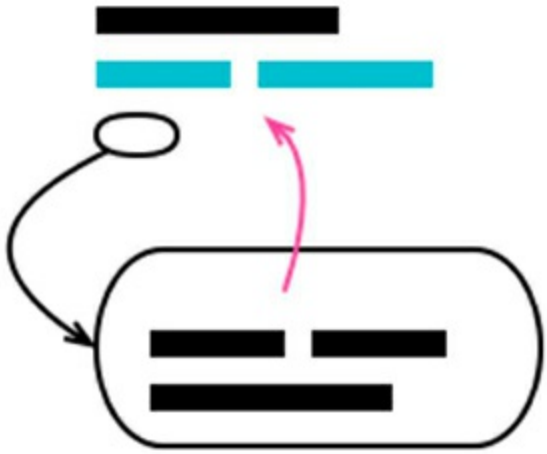
```
function renderPerson(outStream, person) {
  const result = [];
  result.push(`<p>${person.name}</p>`);
  result.push(renderPhoto(person.photo));
  result.push(emitPhotoData(person.photo));
  return result.join("\n");
}

function photoDiv(aPhoto) {
```

```
  return [
    "<div>",
    emitPhotoData(aPhoto),
    "</div>",
    ].join("\n");
}

function emitPhotoData(aPhoto) {
  return [
    `<p>title: ${aPhoto.title}</p>`,
    `<p>location: ${aPhoto.location}</p>`,
    `<p>date: ${aPhoto.date.toDateString()}</p>`,
  ].join("\n");
}
```

Bei dieser Gelegenheit ändere ich auch die Parameterbezeichnungen, sodass sie meiner Konvention entsprechen.

8.4 Anweisungen in Aufrufer verschieben (Move Statements to Caller)

Umkehrung von: *Anweisungen in Funktion verschieben* (Move Statements into Function, Abschnitt 8.3)



```
emitPhotoData(outStream, person.photo);

function emitPhotoData(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>location: ${photo.location}
</p>\n`);
}
```

```
emitPhotoData(outStream, person.photo);
outStream.write(
             `<p>location: ${person.photo.location}
</p>\n`);
function emitPhotoData(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
}
```

8.4.1 MOTIVATION

Funktionen sind die Grundbausteine der Abstraktionen,

die wir als Programmierer entwickeln, und wie auch bei jeder anderen Abstraktion ist es nicht immer ganz einfach, die Grenzen richtig einzuschätzen. Wenn sich die Funktionalität der Codebasis ändert, wie es bei Software häufig der Fall ist, verschieben sich häufig auch die Grenzen unserer Abstraktionen. Das bedeutet für Funktionen, dass aus einer ehemals in sich geschlossenen atomaren Verhaltenseinheit eine Mischung von zwei oder mehr unterschiedlichen Aufgaben wird.

Ein Auslöser dafür ist, dass das an mehreren Stellen verwendete Verhalten bei einigen Aufrufen variieren muss. Dann müssen wir das abweichende Verhalten aus der Funktion in die Aufrufer verlagern. In diesem Fall verwende ich *Anweisungen verschieben* (Abschnitt 8.6), um das abweichende Verhalten am Anfang oder am Ende der Funktion zu vereinen und anschließend *Anweisungen in Aufrufer verschieben*. Sobald sich der abweichende Code im Aufrufer befindet, kann ich ihn bei Bedarf ändern.

Anweisungen in Aufrufer verschieben funktioniert bei kleinen Änderungen gut, aber manchmal muss die Abgrenzung zwischen Aufrufer und aufgerufener Funktion vollständig überarbeitet werden. In diesem Fall sind die beste Wahl die Verwendung von *Funktion inline platzieren* (Abschnitt 6.2), das Verschieben von Anweisungen und die Extraktion neuer Funktionen, um so eine bessere Abgrenzung zu erzielen.

8.4.2 VORGEHEN

- In einfachen Fällen, in denen es nur ein oder zwei Aufrufer und eine einfache aufzurufende Funktion gibt, brauchen Sie lediglich den Code der Funktion auszuschneiden und in den Aufrufern einzufügen (und gegebenenfalls anzupassen). Testen Sie, und Sie sind fertig.
- Wenden Sie ansonsten *Funktion extrahieren* (Abschnitt 6.1) auf alle Anweisungen an, die Sie *nicht* verschieben möchten; vergeben Sie einen temporären Namen, nach dem sich leicht suchen lässt.

  Wenn es sich bei der Funktion um eine Methode handelt, die von Unterklassen überschrieben wird, sollten Sie die überschreibenden Methoden alle extrahieren, sodass die verbleibende Methode in allen Klassen identisch ist. Anschließend können Sie die Methoden der Unterklassen löschen.
- Wenden Sie *Funktion inline platzieren* (Abschnitt 6.2) auf die ursprüngliche Funktion an.
- Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um der extrahierten Funktion den Namen der ursprünglichen Funktion zu geben,

  oder vergeben Sie einen besseren, wenn Ihnen einer einfällt.

8.4.3 BEISPIEL

Hier ist ein einfacher Fall – eine Funktion mit zwei Aufrufern:

```
function renderPerson(outStream, person) {
  outStream.write(`<p>${person.name}</p>\n`);
  renderPhoto(outStream, person.photo);
  emitPhotoData(outStream, person.photo);
}

function listRecentPhotos(outStream, photos) {
```

```
  photos
    .filter(p => p.date > recentDateCutoff())
    .forEach(p => {
      outStream.write("<div>\n");
      emitPhotoData(outStream, p);
      outStream.write("</div>\n");
    });
}

function emitPhotoData(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>date:
${photo.date.toDateString()}</p>\n`);
  outStream.write(`<p>location: ${photo.location}
</p>\n`);
}
```

Ich muss die Software so ändern, dass `listRecentPhotos` die Ortsinformationen anders aufbereitet, während `renderPerson` unverändert bleibt. Ich wende *Anweisungen in Aufrufer verschieben* auf die letzte Zeile an, um die Änderung zu vereinfachen.

In einem so einfachen Fall wie diesem schneide ich für gewöhnlich einfach die letzte Zeile von `renderPerson` aus und füge sie nach den beiden Aufrufen ein. Da ich aber erklären möchte, wie man kniffligere Fälle handhabt, zeige ich das etwas umständlichere, aber zuverlässige Verfahren.

Als Erstes wende ich *Funktion extrahieren* (Abschnitt 6.1) auf den Code an, der in `emitPhotoData` verbleiben soll:

```
function renderPerson(outStream, person) {
  outStream.write(`<p>${person.name}</p>\n`);
  renderPhoto(outStream, person.photo);
  emitPhotoData(outStream, person.photo);
}

function listRecentPhotos(outStream, photos) {
  photos
    .filter(p => p.date > recentDateCutoff())
    .forEach(p => {
      outStream.write("<div>\n");
      emitPhotoData(outStream, p);
      outStream.write("</div>\n");
    });
}

function emitPhotoData(outStream, photo) {
  zztmp(outStream, photo);
  outStream.write(`<p>location: ${photo.location}
</p>\n`);
}

function zztmp(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>date:
${photo.date.toDateString()}</p>\n`);
}
```

Meistens ist der Name der extrahierten Funktion nur vorübergehend notwendig, deshalb mache ich mir keine Gedanken über eine sinnvolle Bezeichnung. Dennoch ist es hilfreich, wenn sie sich leicht wiederfinden lässt. An dieser Stelle kann ich testen, ob sich der Funktionsaufruf wie gewünscht verhält.

Jetzt wende ich der Reihe nach *Funktion inline platzieren* (Abschnitt 6.2) auf die Aufrufe an. Ich beginne mit `renderPerson`:

```
function renderPerson(outStream, person) {
  outStream.write(`<p>${person.name}</p>\n`);
  renderPhoto(outStream, person.photo);
  zztmp(outStream, person.photo);
  outStream.write(`<p>location:
${person.photo.location}</p>\n`);
}

function listRecentPhotos(outStream, photos) {
  photos
    .filter(p => p.date > recentDateCutoff())
    .forEach(p => {
      outStream.write("<div>\n");
      emitPhotoData(outStream, p);
      outStream.write("</div>\n");
    });
}

function emitPhotoData(outStream, photo) {
  zztmp(outStream, photo);
  outStream.write(`<p>location: ${photo.location}
</p>\n`);
}

function zztmp(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>date:
${photo.date.toDateString()}</p>\n`);
}
```

Ich teste wieder, ob der Aufruf ordnungsgemäß

funktioniert, und wende mich anschließend dem nächsten Aufruf zu:

```
function renderPerson(outStream, person) {
  outStream.write(`<p>${person.name}</p>\n`);
  renderPhoto(outStream, person.photo);
  zztmp(outStream, person.photo);
  outStream.write(`<p>location:
${person.photo.location}</p>\n`);
}

function listRecentPhotos(outStream, photos) {
  photos
    .filter(p => p.date > recentDateCutoff())
    .forEach(p => {
      outStream.write("<div>\n");
      zztmp(outStream, p);
      outStream.write(`<p>location: ${p.location}
</p>\n`);
      outStream.write("</div>\n");
    });
}

function emitPhotoData(outStream, photo) {
  zztmp(outStream, photo);
  outStream.write(`<p>location: ${photo.location}
</p>\n`);
}

function zztmp(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>date:
${photo.date.toDateString()}</p>\n`);
}
```

Jetzt kann ich die äußere Funktion löschen und damit

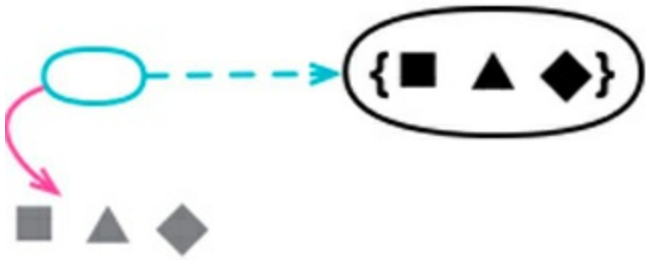
Funktion inline platzieren (Abschnitt 6.2) abschließen:

```
function renderPerson(outStream, person) {
  outStream.write(`<p>${person.name}</p>\n`);
  renderPhoto(outStream, person.photo);
  zztmp(outStream, person.photo);
  outStream.write(`<p>location:
${person.photo.location}</p>\n`);
}

function listRecentPhotos(outStream, photos) {
  photos
    .filter(p => p.date > recentDateCutoff())
    .forEach(p => {
      outStream.write("<div>\n");
      zztmp(outStream, p);
      outStream.write(`<p>location: ${p.location}
</p>\n`);
      outStream.write("</div>\n");
    });
}

function emitPhotoData(outStream, photo) {
  zztmp(outStream, photo);
  outStream.write(`<p>location: ${photo.location}
</p>\n`);
}

function zztmp(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>date:
${photo.date.toDateString()}</p>\n`);
}
```

Jetzt vergebe ich für `zztmp` wieder den ursprünglichen Namen:

```
function renderPerson(outStream, person) {
  outStream.write(`<p>${person.name}</p>\n`);
  renderPhoto(outStream, person.photo);
  emitPhotoData(outStream, person.photo);
  outStream.write(`<p>location:
${person.photo.location}</p>\n`);
}

function listRecentPhotos(outStream, photos) {
  photos
    .filter(p => p.date > recentDateCutoff())
    .forEach(p => {
      outStream.write("<div>\n");
      emitPhotoData(outStream, p);
      outStream.write(`<p>location: ${p.location}
</p>\n`);
      outStream.write("</div>\n");
    });
}

function emitPhotoData(outStream, photo) {
  outStream.write(`<p>title: ${photo.title}</p>\n`);
  outStream.write(`<p>date:
${photo.date.toDateString()}</p>\n`);
}
```

8.5 Inline-Code durch Funktionsaufruf ersetzen (Replace Inline Code with Function Call)



```
let appliesToMass = false;
for(const s of states) {
  if (s === "MA") appliesToMass = true;
}
```

```
appliesToMass = states.includes("MA");
```

8.5.1 MOTIVATION

Funktionen ermöglichen es mir, Verhaltensweisen zu bündeln. Für das Verständnis ist es nützlich, dass der Name einer Funktion deren Zweck beschreibt, nicht ihre Funktionsweise. Außerdem ist es sinnvoll, redundanten Code zu entfernen: Anstatt den gleichen Code zweimal zu schreiben, rufe ich einfach die Funktion auf. Wenn ich dann später die Implementierung der Funktion ändern möchte, muss ich nicht nach ähnlichem Code suchen, um alle Änderungen vorzunehmen. (Eventuell muss ich mir die Aufrufer ansehen, um zu überprüfen, ob alle den neuen Code verwenden sollten, aber das kommt nicht so

häufig vor und ist zudem viel einfacher, als doppelten Code zu warten.)

Wenn ich auf Inline-Code stoße, der die gleiche Aufgabe erledigt wie eine vorhandene Funktion, würde ich ihn in der Regel durch einen Funktionsaufruf ersetzen. Es gibt eine Ausnahme, nämlich wenn ich die Ähnlichkeit für zufällig halte, sodass ich bei einer Änderung der Funktion nicht erwarte, dass sich das Verhalten des Inline-Codes ändert. Der Name der Funktion gibt hier oft einen Hinweis: Ein guter Name sollte als Ersatz für den vorhandenen Inline-Code Sinn ergeben. Ist das nicht der Fall, könnte das an einem schlecht gewählten Namen liegen (den ich mit *Funktionsdeklaration ändern*, Abschnitt 6.5, ändern kann), oder daran, dass der Zweck der Funktion sich von dem unterscheidet, was in diesem Fall erforderlich ist – dann sollte ich sie nicht aufrufen.

Besonders zufriedenstellend ist dieses Vorgehen bei Aufrufen von Bibliotheksfunktionen – ich muss die Funktion dann gar nicht selbst schreiben.

8.5.2 VORGEHEN

- Ersetzen Sie den Inline-Code durch den Aufruf einer vorhandenen Funktion.
- Testen Sie.

8.6 Anweisungen verschieben (Slide

Statements)

Ehemalige Bezeichnung: *Redundante Bedingungsteile konsolidieren* (Consolidate Duplicate Conditional Fragment)

```
const pricingPlan = retrievePricingPlan();
const order = retreiveOrder();
let charge;
const chargePerUnit = pricingPlan.unit;
```

```
const pricingPlan = retrievePricingPlan();
const chargePerUnit = pricingPlan.unit;
const order = retreiveOrder();
let charge;
```

8.6.1 MOTIVATION

Code ist leichter zu verstehen, wenn sich Elemente, die in einer Beziehung zueinander stehen, am gleichen Ort befinden. Wenn mehrere Codezeilen auf dieselbe Datenstruktur zugreifen, gehören sie zusammen und

sollten nicht mit Code vermengt werden, der auf andere Datenstrukturen zugreift. Im einfachsten Fall verwende ich *Anweisungen verschieben*, um derartigen Code zusammenzuhalten. Die Deklaration und Verwendung von Variablen sind typische Beispiele hierfür. Manche Entwickler deklarieren sämtliche Variablen am Anfang einer Funktion. Ich ziehe es hingegen vor, eine Variable erst unmittelbar vor ihrer ersten Verwendung zu deklarieren.

Das Verschieben von zusammengehörigem Code ist häufig ein vorbereitender Schritt für ein anderes Refactoring, nicht selten für *Funktion extrahieren* (Abschnitt 6.1). Zusammengehörigen Code in deutlich voneinander abgegrenzte Funktionen zu verschieben, ist eine bessere Aufteilung, als nur einige Codezeilen zu vereinen, aber ich kann *Funktion extrahieren* (Abschnitt 6.1) erst anwenden, wenn der entsprechende Code im Vorfeld vereint wurde.

8.6.2 VORGEHEN

- Identifizieren Sie die Stelle, an die der Codeabschnitt verschoben werden soll. Untersuchen Sie die Anweisungen an der Quelle und am Ziel und prüfen Sie, ob der Abschnitt beeinträchtigt wird. Wenn das der Fall ist, müssen Sie das Refactoring abbrechen.

  Ein Codeabschnitt darf nicht vor die Deklaration eines Elements, auf das er verweist, verschoben werden.

  Ein Codeabschnitt darf nicht hinter ein Element verschoben werden, das darauf verweist.

  Ein Codeabschnitt darf nicht hinter eine Anweisung verschoben

werden, die ein Element ändert, auf das er verweist.

Ein Codeabschnitt, der ein Element ändert, darf nicht hinter ein anderes Element verschoben werden, das auf das veränderte Element verweist.

- Schneiden Sie den Codeabschnitt an der Quelle aus und fügen Sie ihn an der Zielposition ein.
- Testen Sie.

Wenn der Test fehlschlägt, versuchen Sie, die Verschiebung in kleineren Schritten vorzunehmen. Verschieben Sie den Codeabschnitt nicht so weit oder verringern Sie den Umfang des zu verschiebenden Abschnitts.

8.6.3 BEISPIEL

Beim Verschieben von Codeabschnitten sind zwei Entscheidungen zu treffen, nämlich, welcher Code verschoben werden soll und ob die Verschiebung überhaupt möglich ist. Die erste Entscheidung ist sehr kontextabhängig. Im einfachsten Fall möchte ich die Deklaration von Elementen dort vornehmen, wo ich sie auch verwende. Deshalb verschiebe ich häufig eine Deklaration nach hinten, damit sie sich dort befindet, wo sie verwendet wird. Der Grund für das Verschieben ist aber fast immer, dass ich ein anderes Refactoring vornehmen möchte – beispielsweise muss ich Codeabschnitte zusammenlegen, um *Funktion extrahieren* (Abschnitt 6.1) anwenden zu können.

Sobald ich herausgefunden habe, wohin ich den Code verschieben möchte, muss ich entscheiden, ob die Verschiebung tatsächlich möglich ist. Zu diesem Zweck muss ich mir den Code, den ich verschiebe, und den Code, den ich dabei überspringe, genau ansehen: Beeinträchtigen sich die Codeabschnitte in irgendeiner Form, die das sichtbare Verhalten des Programms verändern könnte?

Betrachten Sie folgenden Codeabschnitt:

```
 1 const pricingPlan = retrievePricingPlan();
 2 const order = retreiveOrder();
 3 const baseCharge = pricingPlan.base;
 4 let charge;
 5 const chargePerUnit = pricingPlan.unit;
 6 const units = order.units;
 7 let discount;
 8 charge = baseCharge + units * chargePerUnit;
 9 let discountableUnits = Math.max(units -
pricingPlan.discountThreshold, 0);
10 discount = discountableUnits *
pricingPlan.discountFactor;
11 if (order.isRepeat) discount += 20;
12 charge = charge - discount;
13 chargeOrder(charge);
```

Die ersten sieben Zeilen sind Deklarationen, die sich vergleichsweise einfach verschieben lassen. Wenn ich beispielsweise den Code, der mit Rabatten (`discount`) zu tun hat, zusammenlegen möchte, müsste ich Zeile 7 (`let discount`) über Zeile 10 (`discount = ...`)

platzieren. Eine Deklaration hat keine Seiteneffekte und verweist nicht auf andere Variablen, deshalb kann ich Zeile 7 problemlos so weit nach hinten verschieben, bis die erste Zeile auftaucht, die auf `discount` verweist. Solche Verschiebungen finden häufig statt – wenn ich etwa *Funktion extrahieren* (Abschnitt 6.1) auf die Programmlogik für Rabatte anwenden wollte, müsste ich vorher die Deklaration nach hinten verschieben.

Auf ähnliche Weise analysiere ich allen Code, der keine Seiteneffekte hat. Ich könnte also Zeile 2 (`const order = ...`) problemlos über Zeile 6 (`const units = ...`) platzieren.

In diesem Fall hilft mir auch die Tatsache weiter, dass der Code, den ich überspringe, ebenfalls keine Seiteneffekte hat. Tatsächlich kann ich Code ohne Seiteneffekte nach Herzenslust verschieben – das ist einer der Gründe dafür, dass erfahrene Programmierer bevorzugt Code mit möglichst wenig Seiteneffekten verwenden.

Hier gibt es jedoch einen Haken. Wie kann ich wissen, dass Zeile 2 keine Seiteneffekte verursacht? Wenn ich mir wirklich sicher sein will, komme ich nicht umhin, mir `retrieveOrder` anzusehen (und alle anderen Funktionen, die `retrieveOrder` aufruft, sowie die darin aufgerufenen Funktionen usw.). Wenn ich in der Praxis an meinem eigenen Code arbeite, weiß ich, dass ich mich grundsätzlich an die Trennung von Befehlen

und Abfragen halte (https://martinfowler.com/bliki/CommandQuerySeparation.html), sodass alle Funktionen, die einen Wert zurückgeben, keine Seiteneffekte verursachen. Ich kann mir dessen aber nur sicher sein, weil mir die Codebasis bekannt ist. Wenn ich eine unbekannte Codebasis bearbeiten würde, müsste ich vorsichtiger sein. Ich versuche aber, mich bei meinem eigenen Code strikt an die Trennung von Befehlen und Abfragen (Command-Query-Separation) zu halten, denn es ist wirklich gut zu wissen, dass der Code keine Seiteneffekte verursacht.

Wenn der verschobene oder der übersprungene Code nicht frei von Seiteneffekten ist, muss ich sehr viel vorsichtiger sein und überprüfen, ob die beiden Codeabschnitte sich gegenseitig beeinflussen. Nehmen wir an, ich möchte Zeile 11 (`if (order.isRepeat) = ...`) ans Ende verschieben. Zeile 12 verhindert das, weil sie einen Verweis auf die Variable enthält, deren Zustand ich in Zeile 11 ändere. Ich darf auch Zeile 13 (`chargeOrder(Charge)`) nicht nach vorne verschieben, weil Zeile 12 einen Zustand modifiziert, auf den Zeile 13 verweist. Allerdings kann Zeile 8 (`charge = baseCharge + ...`) die Zeilen 9 bis 11 überspringen, weil diese den Zustand, auf den Zeile 8 verweist, nicht ändern.

Die einfachste zu befolgende Regel lautet, dass ich einen Codeabschnitt nicht verschieben darf, wenn der Code

selbst und der übersprungene Code auf Daten verweisen, die von beiden Codeabschnitten geändert werden. Diese Regel ist jedoch nicht allgemeingültig – so kann ich beispielsweise jede der beiden folgenden Codezeilen nach Belieben verschieben:

```
a = a + 10;
a = a + 5;
```

Um beurteilen zu können, ob ich eine Verschiebung vornehmen darf, muss ich die beteiligten Operationen und deren Zusammensetzung gründlich verstanden haben.

Da ich mir so viele Gedanken über Aktualisierungen des Zustands machen muss, versuche ich, diese so weit wie möglich zu entfernen. Beim vorliegenden Code würde ich also *Variable aufteilen* (Abschnitt 9.1) auf `charge` anwenden, bevor ich den Versuch unternehme, irgendwelche Teile dieses Codes zu verschieben.

Die Analyse ist hier vergleichsweise einfach, weil ich meist nur lokale Variablen modifiziere. Bei komplexeren Datenstrukturen ist es erheblich schwieriger, festzustellen, ob es zu gegenseitigen Beeinflussungen der Codeabschnitte kommt. Deshalb spielen Tests eine bedeutende Rolle: Verschieben Sie einen Codeabschnitt, führen Sie Tests durch und prüfen Sie, ob noch alles funktioniert. Wenn meine Testabdeckung hinreichend

gut ist, kann ich mit diesem Refactoring zufrieden sein. Sollten die Tests jedoch nicht zuverlässig sein, muss ich auf der Hut sein – oder, was wahrscheinlicher ist, die Tests für den Code, an dem ich arbeite, verbessern.

Wenn nach einer Verschiebung ein Test fehschlägt, ist die wichtigste Konsequenz, die daraus zu ziehen ist, die Verwendung kleinerer Schritte. Anstatt zehn Zeilen zu überspringen, verschiebe ich den Code nur um fünf, oder ich verschiebe den Code nur bis zu einer Stelle, an der ich mit dem Auftreten von Problemen rechne. Manchmal bedeutet ein fehlgeschlagener Test auch, dass die Verschiebung zum jetzigen Zeitpunkt noch nicht sinnvoll ist und dass ich erst andere Aufgaben erledigen muss.

8.6.4 BEISPIEL: VERSCHIEBUNGEN UND BEDINGTE ANWEISUNGEN

Verschiebungen sind auch bei bedingten Anweisungen möglich. Die Richtung, in die der Code verschoben wird, legt fest, ob ich dabei redundante Programmlogik entferne oder hinzufüge.

Hier enthalten die beiden Zweige einer Bedingung die gleichen Anweisungen:

```
let result;
if (availableResources.length === 0) {
  result = createResource();
  allocatedResources.push(result);
} else {
```

```
  result = availableResources.pop();
  allocatedResources.push(result);
}
return result;
```

Wenn ich sie aus der Bedingung entferne, ergibt sich eine einzelne Anweisung außerhalb der Bedingung:

```
let result;
if (availableResources.length === 0) {
  result = createResource();
} else {
  result = availableResources.pop();
}
allocatedResources.push(result);
return result;
```

Im umgekehrten Fall, also dem Verschieben eines Codeabschnitts in eine bedingte Anweisung, wird dieser in jedem Zweig der Bedingung wiederholt.

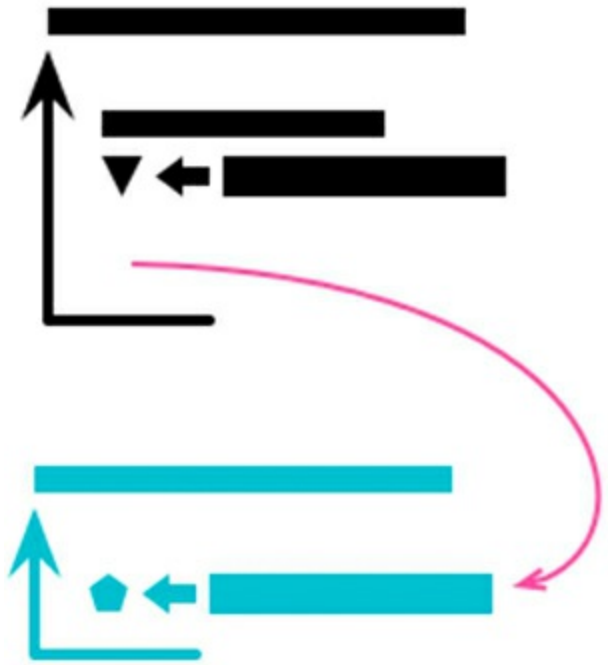
8.6.5 LITERATURHINWEIS

Ich bin einem nahezu identischen Refactoring begegnet, das den Namen *Anweisungen vertauschen* (Swap Statements, siehe `https://www.industriallogic.com/blog/swap-statement-refactoring/`) trägt. *Anweisungen vertauschen* verschiebt aufeinanderfolgende Codeabschnitte, die allerdings nur aus einer Anweisung bestehen dürfen. Sie können sich das wie eine Version von *Anweisungen*

verschieben vorstellen, bei der verschobener und übersprungener Code jeweils aus einer einzelnen Anweisung bestehen. Dieses Refactoring gefällt mir – schließlich reite ich ja stets darauf herum, kleine Schritte zu unternehmen, die Neulingen beim Refactoring manchmal geradezu lächerlich winzig erscheinen.

Ich habe für die Beschreibung dieses Refactorings größere Codeabschnitte verwendet, weil dies der Praxis entspricht. Ich verschiebe Anweisungen nur dann einzeln, wenn mir größere Verschiebungen Schwierigkeiten bereiten, und das kommt nur selten vor. Bei chaotischerem Code ist es allerdings einfacher, in kleineren Schritten zu verschieben.

8.7 Schleife aufteilen (Split Loop)



```
let averageAge = 0;
let totalSalary = 0;
for (const p of people) {
  averageAge += p.age;
  totalSalary += p.salary;
}
averageAge = averageAge / people.length;
```

```
let totalSalary = 0;
for (const p of people) {
  totalSalary += p.salary;
}
let averageAge = 0;
for (const p of people) {
  averageAge += p.age;
}
averageAge = averageAge / people.length;
```

8.7.1 MOTIVATION

Man sieht nicht selten Schleifen, die zwei verschiedene Dinge gleichzeitig ausführen, nur weil es möglich ist, sie mit einem Schleifendurchlauf zu erledigen. Aber wenn Sie zwei verschiedene Aufgaben in derselben Schleife erledigen, müssen Sie bei Änderungen an der Schleife auch beide Aufgaben verstehen. Durch das Aufteilen der Schleife können Sie gewährleisten, dass Sie nur das Verhalten verstehen müssen, das Sie ändern möchten.

Das Aufteilen einer Schleife kann zudem ihre Verwendung vereinfachen. Eine Funktion mit einer Schleife, die einen einzelnen Wert berechnet, kann diesen einen Wert zurückgeben. Funktionen mit Schleifen, die mehrere Aufgaben erledigen, müssen Datenstrukturen zurückliefern oder lokalen Variablen Werte zuweisen. Ich verwende häufig eine Abfolge von *Schleife aufteilen* und *Funktion extrahieren* (Abschnitt 6.1).

Viele Programmierer beschleicht bei diesem Refactoring ein unbehagliches Gefühl, weil sie gezwungen sind, die Schleife zweimal auszuführen. Ich empfehle dann wie üblich, das Refactoring von der Optimierung zu trennen (siehe *Refactoring und Performance*, Abschnitt 2.8). Sobald ich meinen Code aufgeräumt habe, optimiere ich ihn, und sollte sich das Durchlaufen der Schleife tatsächlich als Engpass herausstellen, können die beiden Schleifen leicht wieder vereint werden. Jedoch stellt das eigentliche Durchlaufen selbst großer Listen nur selten einen Engpass dar. Darüber hinaus ermöglicht die Aufteilung der Schleife oft weitere leistungsstarke Optimierungen.

8.7.2 VORGEHEN

- Kopieren Sie die Schleife.
- Suchen und entfernen Sie doppelten Code mit Seiteneffekten.
- Testen Sie.

Wenn das erledigt ist, sollten Sie in Betracht ziehen, *Funktion extrahieren* (Abschnitt 6.1) auf die Schleifen anzuwenden.

8.7.3 BEISPIEL

Ich beginne mit einem kleinen Codeabschnitt, der das Gehalt (`totalSalary`) und das Alter des jüngsten Mitarbeiters (`youngestAge`) berechnet:

```
let youngest = people[0] ? people[0].age : Infinity;
let totalSalary = 0;
for (const p of people) {
  if (p.age < youngest) youngest = p.age;
  totalSalary += p.salary;
}
return `youngestAge: ${youngest},
        totalSalary: ${totalSalary}`;
```

Die Schleife ist sehr einfach, führt jedoch zwei verschiedene Berechnungen durch. Ich kopiere die Schleife zunächst, um die Aufteilung vornehmen zu können:

```
let youngest = people[0] ? people[0].age : Infinity;
let totalSalary = 0;
for (const p of people) {
  if (p.age < youngest) youngest = p.age;
  totalSalary += p.salary;
}

for (const p of people) {
```

```
  if (p.age < youngest) youngest = p.age;
  totalSalary += p.salary;
}

return `youngestAge: ${youngest},
        totalSalary: ${totalSalary}`;
```

Nach dem Kopieren der Schleife muss ich den doppelten Code entfernen, der zu falschen Ergebnissen führen würde. Wenn Teile des Codes in der Schleife keine Seiteneffekte verursachen, kann er fürs Erste dort verbleiben, in diesem Beispiel ist das jedoch nicht der Fall:

```
let youngest = people[0] ? people[0].age : Infinity;
let totalSalary = 0;
for (const p of people) {
  if (p.age < youngest) youngest = p.age;
  totalSalary += p.salary;
}

for (const p of people) {
  if (p.age < youngest) youngest = p.age;
  totalSalary += p.salary;
}

return `youngestAge: ${youngest},
        totalSalary: ${totalSalary}`;
```

Offiziell ist das Refactoring *Schleife aufteilen* damit abgeschlossen. Der eigentliche Sinn des Refactorings ist jedoch gar nicht das, was es selbst leistet, sondern dass

es den Weg für den nächsten Schritt ebnet – ich versuche nämlich im Anschluss, durch Extraktion aus den Schleifen eigene Funktionen zu machen. Aber zunächst verwende ich *Anweisungen verschieben* (Abschnitt 8.6), um den Code etwas anders anzuordnen:

```
let totalSalary = 0;
for (const p of people) {
  totalSalary += p.salary;
}

let youngest = people[0] ? people[0].age : Infinity;
for (const p of people) {
  if (p.age < youngest) youngest = p.age;
}

return `youngestAge: ${youngest},
        totalSalary: ${totalSalary}`;
```

Jetzt wende ich mehrmals *Funktion extrahieren* (Abschnitt 6.1) an:

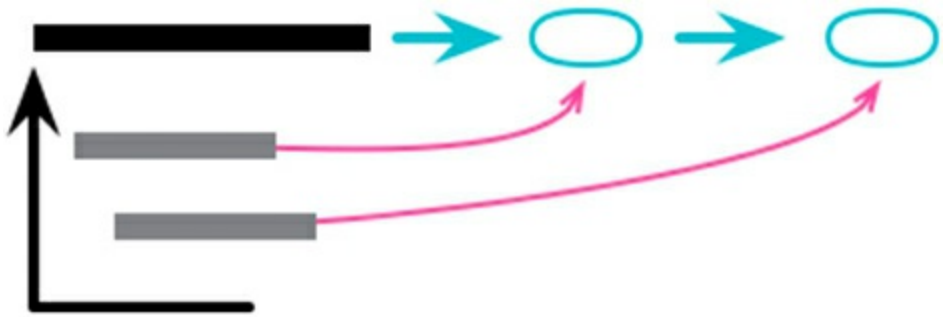
```
return `youngestAge: ${youngestAge()},
        totalSalary: ${totalSalary()}`;

function totalSalary() {
  let totalSalary = 0;
  for (const p of people) {
    totalSalary += p.salary;
  }
  return totalSalary;
}
```

```
function youngestAge() {
  let youngest = people[0] ? people[0].age : Infinity;
  for (const p of people) {
    if (p.age < youngest) youngest = p.age;
  }
  return youngest;
}
```

Ich kann nicht widerstehen, *Schleife durch Pipeline ersetzen* (Abschnitt 8.8) auf `totalSalary` anzuwenden und mit *Algorithmus ersetzen* (Abschnitt 7.9) die Berechnung von `youngestAge` durch einen auf der Hand liegenden Algorithmus zu verbessern:

```
return `youngestAge: ${youngestAge()},
        totalSalary: ${totalSalary()}`;

function totalSalary() {
  return people.reduce((total,p) => total + p.salary,
0);
}
function youngestAge() {
  return Math.min(...people.map(p => p.age));
}
```

8.8 Schleife durch Pipeline ersetzen (Replace Loop with Pipeline)



```
const names = [];
for (const i of input) {
  if (i.job === "programmer")
    names.push(i.name);
}
```

```
const names = input
  .filter(i => i.job === "programmer")
  .map(i => i.name)
;
```

8.8.1 MOTIVATION

Wie die meisten Programmierer habe ich gelernt, Schleifen zu verwenden, um über eine Objektsammlung, eine sogenannte *Collection*, zu iterieren. Verschiedene Sprachumgebungen bieten inzwischen jedoch ein besseres Konstrukt: die Collection-Pipeline. Collection-Pipelines (https://martinfowler.com/articles/collection-

`pipeline/`) ermöglichen mir, die Verarbeitung als eine Abfolge von Operationen zu beschreiben, die jeweils eine Objektsammlung entgegennehmen und zurückgeben. Die gebräuchlichsten dieser Operationen sind *map*, die eine Funktion auf alle Elemente der Eingabe anwendet, und *filter*, die eine Funktion verwendet, um eine Teilmenge der Eingabe für nachfolgende Schritte der Pipeline auszuwählen. Ich kann der Programmlogik viel besser folgen, wenn sie in Form einer Pipeline abläuft – ich muss nur von oben nach unten lesen, um erkennen zu können, wie die Objekte die Pipeline durchlaufen.

8.8.2 VORGEHEN

- Legen Sie eine neue Variable für die Objektsammlung der Schleife an.

  Hierbei kann es sich um eine einfache Kopie einer vorhandenen Variable handeln.

- Gehen Sie das Verhalten der Schleife schrittweise von oben nach unten durch und ersetzen Sie es jeweils durch eine geeignete Operation der Collection-Pipeline, die Sie auf der Collection ausführen. Testen Sie nach jeder Änderung.

- Nachdem Sie das gesamte Verhalten aus der Schleife entfernt haben, löschen Sie die Schleife.

  Wenn die Schleife Zuweisungen an eine akkumulierende Variable vornimmt, sollten Sie die Ausgabe der Pipeline dieser akkumulierenden Variable zuweisen.

8.8.3 BEISPIEL

Ich verwende hier eine CSV-Datei, die durch Kommata

getrennte Informationen über die Niederlassungen unserer Firma enthält:

```
office, country, telephone
Chicago, USA, +1 312 373 1000
Beijing, China, +86 4008 900 505
Bangalore, India, +91 80 4064 9570
Porto Alegre, Brazil, +55 51 3079 3550
Chennai, India, +91 44 660 44766
... (weitere Daten)
```

Die folgende Funktion wählt die Niederlassungen in Indien aus und gibt den Ort und die Telefonnummer zurück:

```
function acquireData(input) {
  const lines = input.split("\n");
  let firstLine = true;
  const result = [];
  for (const line of lines) {
    if (firstLine) {
      firstLine = false;
      continue;
    }
    if (line.trim() === "") continue;
    const record = line.split(",");
    if (record[1].trim() === "India") {
      result.push({city: record[0].trim(),
                   phone: record[2].trim()});
    }
  }
  return result;
}
```

Ich möchte diese Schleife durch eine Collection-Pipeline ersetzen. Als Erstes erstelle ich eine eigene Schleifenvariable:

```
function acquireData(input) {
  const lines = input.split("\n");
  let firstLine = true;
  const result = [];
  const loopItems = lines
  for (const line of loopItems) {
    if (firstLine) {
      firstLine = false;
      continue;
    }
    if (line.trim() === "") continue;
    const record = line.split(",");
    if (record[1].trim() === "India") {
      result.push({city: record[0].trim(),
                   phone: record[2].trim()});
    }
  }
  return result;
}
```

Im ersten Teil der Schleife geht es lediglich darum, die erste Zeile der CSV-Datei zu überspringen. Dafür ist eine `slice`-Operation bestens geeignet, deshalb entferne ich diesen Teil der Schleife und füge eine solche Operation hinzu:

```
function acquireData(input) {
  const lines = input.split("\n");
  let firstLine = true;
```

```
  const result = [];
  const loopItems = lines
        .slice(1);
  for (const line of loopItems) {
    if (firstLine) {
      firstLine = false;
      continue;
    }
    if (line.trim() === "") continue;
    const record = line.split(",");
    if (record[1].trim() === "India") {
      result.push({city: record[0].trim(),
                   phone: record[2].trim()});
    }
  }
  return result;
}
```

Als zusätzlichen Bonus kann ich noch `firstLine` entfernen – Steuerungsvariablen lösche ich besonders gern.

Der nächste Teil des Verhaltens entfernt leere Zeilen. Diesen Teil kann ich durch eine `filter`-Operation ersetzen:

```
function acquireData(input) {
  const lines = input.split("\n");
  let firstLine = true;
  const result = [];
  const loopItems = lines
        .slice(1)
        .filter(line => line.trim() !== "")
        ;
```

```
  for (const line of loopItems) {
    if (line.trim() === "") continue;
    const record = line.split(",");
    if (record[1].trim() === "India") {
      result.push({city: record[0].trim(),
                   phone: record[2].trim()});
    }
  }
  return result;
}
```

Beim Schreiben einer Pipeline halte ich es für das Beste, das abschließende Semikolon in einer eigenen Zeile zu platzieren.

Zum Umwandeln der Zeilen in ein Array aus Strings, das in der ursprünglichen Funktion irreführenderweise als `record` bezeichnet wird, verwende ich die `map`-Operation. Es ist fürs Erste jedoch sicherer, den Namen beizubehalten und eine Umbenennung erst später vorzunehmen:

```
function acquireData(input) {
  const lines = input.split("\n");
  const result = [];
  const loopItems = lines
        .slice(1)
        .filter(line => line.trim() !== "")
        .map(line => line.split(","))
        ;
  for (const line of loopItems) {
    const record = line;.split(",");
    if (record[1].trim() === "India") {
      result.push({city: record[0].trim(),
```

```
                    phone: record[2].trim()});
    }
  }
  return result;
}
```

Ein weiterer Filter wählt die Einträge für Indien aus:

```
function acquireData(input) {
  const lines = input.split("\n");
  const result = [];
  const loopItems = lines
        .slice(1)
        .filter(line => line.trim() !== "")
        .map(line => line.split(","))
        .filter(record => record[1].trim() ===
"India")
        ;
  for (const line of loopItems) {
    const record = line;
    if (record[1].trim() === "India") {
      result.push({city: record[0].trim(),
                   phone: record[2].trim()});
    }
  }
  return result;
}
```

Eine weitere `map`-Operation nimmt die Ausgabe vor:

```
function acquireData(input) {
  const lines = input.split("\n");
  const result = [];
  const loopItems = lines
```

```
        .slice(1)
        .filter(line => line.trim() !== "")
        .map(line => line.split(","))
        .filter(record => record[1].trim() ===
"India")
        .map(record => ({city: record[0].trim(),
                         phone: record[2].trim()}))
        ;
  for (const line of loopItems) {
    const record = line;
    result.push(line);
  }
  return result;
}
```

Jetzt finden in der Schleife nur noch Zuweisungen zur akkumulierenden Variable statt. Ich kann die Schleife also löschen und das Ergebnis der Pipeline der akkumulierenden Variable zuweisen.

```
function acquireData(input) {
  const lines = input.split("\n");
  const result =  lines
        .slice(1)
        .filter(line => line.trim() !== "")
        .map(line => line.split(","))
        .filter(record => record[1].trim() ===
"India")
        .map(record => ({city: record[0].trim(),
                         phone: record[2].trim()}))
        ;
  for (const line of loopItems) {
    const record = line;
    result.push(line);
  }
```

```
  return result;
}
```

Damit ist dieses Refactoring eigentlich abgeschlossen. Ich möchte allerdings noch ein wenig aufräumen. Dafür platziere ich `result` inline, benenne einige Lambda-Variablen um und ordne den Code so an, dass er eher einer Tabelle ähnelt:

```
function acquireData(input) {
  const lines = input.split("\n");
  return lines
         .slice  (1)
         .filter (line   => line.trim() !== "")
         .map    (line   => line.split(","))
         .filter (record => record[1].trim() ===
"India")
         .map    (record => ({city: record[0].trim(),
                             phone:
record[2].trim()}))
         ;
}
```

Ich habe erwogen, `lines` ebenfalls inline zu platzieren, mir dann aber gedacht, dass die Zeile erklärt, was hier getan wird.

8.8.4 LITERATURHINWEIS

Weitere Beispiele für das Umwandeln von Schleifen in Pipelines finden Sie in meinem Artikel *Refactoring with Loops and Collection Pipelines*

(https://martinfowler.com/articles/refactoring-pipelines.html).

8.9 Toten Code entfernen (Remove Dead Code)

```
if(false) {
  doSomethingThatUsedToMatter();
}
```

8.9.1 MOTIVATION

Wenn wir unseren Quelltext ausliefern, werden wir nicht nach Volumen vergütet. Ein paar Zeilen ungenutzter Code verlangsamen unsere Systeme nicht und belegen auch keinen nennenswerten Speicherplatz – tatsächlich werden sie von guten Compilern sogar automatisch

ausgelassen. Dennoch stellt ungenutzter Code für das Verständnis der Funktionsweise einer Software eine erhebliche Behinderung dar. Es gibt keine Warnhinweise, die den Programmierern mitteilen, dass sie eine Funktion nicht zu berücksichtigen brauchen, weil diese sowieso nicht mehr aufgerufen wird. Deshalb müssen sie weiterhin Zeit investieren, um zu verstehen, welche Aufgabe die Funktion erledigt und warum vorgenommene Änderungen nicht das erwartete Ergebnis liefern.

Sobald Code nicht mehr verwendet wird, sollten wir ihn löschen. Ich mache mir keine Sorgen darüber, dass ich ihn vielleicht demnächst wieder benötige. Sollte das der Fall sein, kann ich jederzeit auf meine Versionsverwaltung zurückgreifen und ihn wieder hervorkramen. Wenn ich denke, dass es sich um Code handelt, den ich eines Tages wieder verwenden möchte, erstelle ich einen Kommentar, der den Code beschreibt und in welcher Version er entfernt wurde. Offen gestanden kann ich mich aber nicht daran erinnern, wann ich das zum letzten Mal gemacht habe oder bereut hätte, dass ich es nicht getan habe.

Früher war es üblich, toten Code auszukommentieren. Das war zu einer Zeit, in der Versionsverwaltungen noch nicht so weit verbreitet und so komfortabel waren wie heute, auch durchaus sinnvoll. Heutzutage kann ich auch die kleinste Codebasis in eine Versionsverwaltung aufnehmen, deshalb ist das Auskommentieren nicht

mehr notwendig.

8.9.2 VORGEHEN

- Falls auf den toten Code von außerhalb zugegriffen werden kann, beispielsweise wenn es sich um eine vollständige Funktion handelt, sollten Sie überprüfen, ob es nicht doch noch Aufrufer gibt.
- Löschen Sie den toten Code.
- Testen Sie.

Kapitel 9

Daten organisieren

Datenstrukturen spielen in unseren Programmen eine wichtige Rolle, daher dürfte es kaum eine Überraschung sein, dass sich eine ganze Reihe von Refactorings darauf konzentriert. Eine Variable, die für unterschiedliche Aufgaben verwendet wird, ist Nährboden für Verwirrung und für Bugs, deshalb verwende ich *Variable aufteilen* (Abschnitt 9.1), um die verschiedenen Verwendungsweisen voneinander zu trennen. Wie auch bei allen anderen Programmelementen ist es knifflig und von großer Bedeutung, einen geeigneten Namen für eine Variable zu finden, also verwende ich häufig *Variable umbenennen* (Abschnitt 6.7), um den Namen zu ändern. In manchen Fällen ist es jedoch das Beste, sich einer Variable komplett zu entledigen – was ich mit *Abgeleitete Variable durch Abfrage ersetzen* (Abschnitt 9.3) erreichen kann.

Häufig lassen sich in einer Codebasis Probleme durch die Verwechslung von Referenzen und Werten finden. Deshalb verwende ich *Referenz durch Wert ersetzen* (Abschnitt 9.4) oder *Wert durch Referenz ersetzen* (Abschnitt 9.5), um zwischen Wert-basiertem Stil und Referenz-basiertem Stil zu wechseln.

9.1 Variable aufteilen (Split Variable)

Ehemalige Bezeichnung: *Zuweisungen zu Parametern entfernen* (Remove Assignments to Parameters)

Ehemalige Bezeichnung: *Temporäre Variable zerlegen* (Split Temp)

```
let temp = 2 * (height + width); console.log(temp);
temp = height * width;
console.log(temp);
```

```
const perimeter = 2 * (height + width);
console.log(perimeter);
const area = height * width;
console.log(area);
```

9.1.1 MOTIVATION

Für Variablen gibt es vielfältige Einsatzmöglichkeiten. In manchen Fällen führt das naturgemäß zu mehreren

Zuweisungen zu einer einzelnen Variable. Schleifenvariablen ändern bei jedem Durchlauf ihren Wert (wie etwa `i` in der Schleife `for(let i=0; i<10; i++)`). Zählvariablen wiederum speichern einen Wert, der während der Ausführung einer Methode ermittelt wird.

Viele Variablen werden zum Speichern der Ergebnisse komplizierter Ausdrücke verwendet, damit man später leicht auf diese verweisen kann. Diesen Variablen sollte nur einmalig ein Wert zugewiesen werden. Wenn mehr als eine Zuweisung erfolgt, ist das ein Hinweis darauf, dass sie innerhalb der Methode mehr als eine Aufgabe erfüllen. In diesem Fall sollte die Variable durch mehrere ersetzt werden, nämlich durch jeweils eine für jede Aufgabe. Für den Betrachter ist es äußerst irreführend, wenn eine Variable für zwei verschiedene Aufgaben verwendet wird.

9.1.2 VORGEHEN

- Ändern Sie den Namen der Variable in der Deklaration und bei der ersten Zuweisung.

  Wenn spätere Zuweisungen von der Form `i = i + irgendetwas` sind, handelt es sich um eine Zählvariable, nehmen Sie in diesem Fall also keine Aufteilung vor. Zählvariablen werden häufig für die Berechnung von Summen, für das Verketten von Strings, für das Schreiben in einen Datenstrom oder zum Hinzufügen von Objekten zu einer Objektsammlung verwendet.

- Deklarieren Sie die Variable als unveränderlich, falls möglich.

- Passen Sie alle Verweise auf die Variable, die sich vor der zweiten

Zuweisung befinden, an.

- Testen Sie.
- Wiederholen Sie den Vorgang stufenweise. Ändern Sie jeweils den Namen in der Deklaration und die Verweise bis zur nächsten Zuweisung, bis Sie die letzte Zuweisung erreicht haben.

9.1.3 BEISPIEL

In diesem Beispiel berechne ich die zurückgelegte Wegstrecke von Haggis (schottische Spezialität; besteht aus dem Magen eines Schafes). Anfänglich aus dem Stand heraus erfährt ein Haggis eine Kraft, die ihn in Bewegung versetzt. Nach einer Verzögerung erfährt er eine zweite Kraft, die ihn weiter beschleunigt. Entsprechend den Bewegungsgesetzen der Physik kann ich den zurückgelegten Weg wie folgt berechnen:

```
function distanceTravelled (scenario, time) {
  let result;
  let acc = scenario.primaryForce / scenario.mass;
  let primaryTime = Math.min(time, scenario.delay);
  result = 0.5 * acc * primaryTime * primaryTime;
  let secondaryTime = time - scenario.delay;
  if (secondaryTime > 0) {
    let primaryVelocity = acc * scenario.delay;
    acc = (scenario.primaryForce +
scenario.secondaryForce)
          / scenario.mass;
    result += primaryVelocity * secondaryTime + 0.5 *
acc
              * secondaryTime * secondaryTime;
  }
  return result;
}
```

Für das Beispiel ist hier von Interesse, dass der Variable `acc` zweimal ein Wert zugewiesen wird. Sie hat zwei Aufgaben: Zunächst speichert sie die durch die erste Kraft verursachte Beschleunigung, später die durch beide Kräfte verursachte. Diese Variable möchte ich aufteilen.

Beim Versuch, die Verwendung einer Variable zu verstehen, erweist es sich als äußerst nützlich, dass mein Editor alle Vorkommen eines Ausdrucks in einer Funktion oder Datei hervorheben kann. Die meisten modernen Editoren bieten diese Funktionalität.

Ich ändere zunächst den Variablennamen und deklariere sie als `const`. Anschließend ändere ich alle Referenzen auf die Variable, die bis zur nächsten Zuweisung vorkommen. Damit erhalte ich Folgendes:

```
function distanceTravelled (scenario, time) {
  let result;
  const primaryAcceleration = scenario.primaryForce
                              / scenario.mass;
  let primaryTime = Math.min(time, scenario.delay);
  result = 0.5 * primaryAcceleration * primaryTime
           * primaryTime;
  let secondaryTime = time - scenario.delay;
  if (secondaryTime > 0) {
    let primaryVelocity =
                        primaryAcceleration *
scenario.delay;
    let acc =
```

```
            (scenario.primaryForce +
scenario.secondaryForce)
            / scenario.mass;
    result += primaryVelocity * secondaryTime + 0.5 *
acc
              * secondaryTime * secondaryTime;
  }
  return result;
}
```

Ich wähle den neuen Namen so, dass er nur die erste Verwendung der Variable repräsentiert. Ich habe sie als `const` deklariert, um zu gewährleisten, dass ihr nur einmalig ein Wert zugewiesen wird. Anschließend kann ich bei der zweiten Zuweisung die ursprüngliche Variable deklarieren. Jetzt kann ich kompilieren und testen, und alles sollte funktionieren.

Ich fahre mit der zweiten Wertzuweisung zur Variable fort. Der ursprüngliche Name wird komplett entfernt und durch einen Variablennamen ersetzt, der entsprechend der zweiten Verwendung gewählt wird:

```
function distanceTravelled (scenario, time) {
  let result;
  const primaryAcceleration = scenario.primaryForce
                              / scenario.mass;
  let primaryTime = Math.min(time, scenario.delay);
  result = 0.5 * primaryAcceleration * primaryTime
           * primaryTime;
  let secondaryTime = time - scenario.delay;
  if (secondaryTime > 0) {
    let primaryVelocity =
```

```
                        primaryAcceleration *
scenario.delay;
    const secondaryAcceleration =
            (scenario.primaryForce +
scenario.secondaryForce)
            / scenario.mass;
    result += primaryVelocity * secondaryTime + 0.5
              * secondaryAcceleration * secondaryTime
              * secondaryTime;
  }
  return result;
}
```

Ich bin mir sicher, dass Ihnen viele weitere Refactorings einfallen, die hier anwendbar sind. Genießen Sie es. (Das ist zweifelsohne besser, als den Haggis zu essen – wissen Sie eigentlich, was da alles drin ist?)

9.1.4 BEISPIEL: ZUWEISUNG ZU EINEM EINGABEPARAMETER

Auch wenn eine Variable als Eingabeparameter deklariert ist, bietet sich das Refactoring *Variable aufteilen* an. Betrachten Sie folgenden Code:

```
function discount (inputValue, quantity) {
  if (inputValue > 50) inputValue = inputValue - 2;
  if (quantity > 100) inputValue = inputValue - 1;
  return inputValue;
}
```

Hier wird `inputValue` sowohl als Eingabe für die

Funktion als auch für das an den Aufrufer zurückgegebene Ergebnis verwendet. (Weil JavaScript hier Call-by-Value-Parameter verwendet, bekommt der Aufrufer eine Änderung von `inputValue` nicht mit.)

Ich würde die Variable hier wie folgt aufteilen.

```
function discount (originalInputValue, quantity) {
  let inputValue = originalInputValue;
  if (inputValue > 50) inputValue = inputValue - 2;
  if (quantity > 100) inputValue = inputValue - 1;
  return inputValue;
}
```

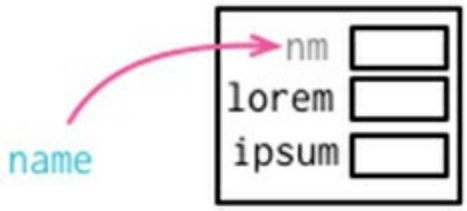
Anschließend wende ich zweimal *Variable umbenennen* (Abschnitt 6.7) an, um bessere Bezeichnungen zu erhalten.

```
function discount (inputValue, quantity) {
  let result = inputValue;
  if (inputValue > 50) result = result - 2;
  if (quantity > 100) result = result - 1;
  return result;
}
```

Sie werden bemerken, dass ich in der zweiten Zeile jetzt `inputValue` als Datenquelle verwende. Der Wert ist zwar derselbe, aber ich denke, dass es in dieser Zeile vor allem darum geht, die Änderung am Wert des Ergebnisses anhand des ursprünglichen Eingabewerts

vorzunehmen, nicht anhand des (wenn auch gleichen) Werts der akkumulierenden Variablen `result`.

9.2 Feld umbenennen (Rename Field)




```
class Organization {
  get name() {...}
}
```

```
class Organization {
  get title() {...}
}
```

9.2.1 MOTIVATION

Namen sind wichtig, und Bezeichnungen von Feldern in Datensatzstrukturen sind besonders wichtig, wenn sie in einem Programm auf breiter Front angewendet werden. Datenstrukturen spielen eine bedeutende Rolle für das Verständnis. Fred Brooks hat vor vielen Jahren einmal

gesagt: »Zeig mir deine Flussdiagramme und verbirg deine Tabellen, und ich werde weiter im Dunkeln tappen. Zeig mir deine Tabellen, und ich brauche deine Flussdiagramme meist gar nicht, denn sie sind offensichtlich.« Heutzutage zeichnet kaum noch jemand Flussdiagramme, dennoch bleibt die eigentliche Aussage immer noch richtig. Datenstrukturen sind von entscheidender Bedeutung, um zu verstehen, wie sich ein Programm verhält.

Und weil diese Datenstrukturen von so großer Bedeutung sind, ist es unverzichtbar, sie klar und deutlich zu gestalten. Je länger ich an der Software arbeite, desto besser verstehe ich die Daten, deshalb ist es wichtig, dass dieses verbesserte Verständnis in das Programm einfließt.

Ebenso wie Sie für ein Feld in einer Datensatzstruktur einen neuen Namen vergeben, ist dieses Refactoring auf Klassen anwendbar. Getter- und Setter-Methoden stellen dem Client einer Klasse eine effektive Form von Feldern zur Verfügung. Ihre Umbenennung ist genauso wichtig wie diejenige einfacher Datensatzstrukturen.

9.2.2 VORGEHEN

- Wenn der Gültigkeitsbereich des Datensatzes begrenzt ist, brauchen Sie nur alle Zugriffe darauf zu ändern und Tests durchzuführen. Die weiteren Schritte können Sie sich sparen.
- Sofern der Datensatz noch nicht gekapselt ist, wenden Sie *Datensatz kapseln* (Abschnitt 7.1) an.

- Benennen Sie das private Feld innerhalb des Objekts um und passen Sie die internen Methoden dementsprechend an.
- Testen Sie.
- Wenn der Konstruktor den Namen verwendet, können Sie ihn mit *Funktionsdeklaration ändern* (Abschnitt 6.5) umbenennen.
- Wenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5) auf die Zugriffsfunktionen an.

9.2.3 BEISPIEL: UMBENENNEN EINES FELDES

Als Erstes benenne ich eine Konstante um:

```
const organization = {name: "Acme Gooseberries",
                      country: "GB"};
```

Ich möchte `name` in `title` umbenennen. Das Objekt wird fast überall in der Codebasis verwendet, und `title` wird auch aktualisiert, deshalb wende ich zunächst einmal *Datensatz kapseln* (Abschnitt 7.1) an:

```
class Organization {
  constructor(data) {
    this._name = data.name;
    this._country = data.country;
  }
  get name() {return this._name;}
  set name(aString) {this._name = aString;}
  get country() {return this._country;}
  set country(aCountryCode) {this._country =
aCountryCode;}
}
```

```
const organization = new Organization(
                  {name: "Acme Gooseberries", country:
"GB"});
```

Nachdem die Datensatzstruktur in der Klasse gekapselt ist, muss ich an vier Stellen Umbenennungen vornehmen, nämlich: in der Getter- und der Setter-Funktion, im Konstruktor und in der internen Datenstruktur. Das hört sich zunächst so an, als ob meine Arbeitslast dadurch ansteigen würde, tatsächlich wird mir die Arbeit jedoch erleichtert, weil ich Änderungen nicht mehr gleichzeitig ausführen muss, sondern unabhängig voneinander in kleineren Schritten vornehmen kann. Kleinere Schritte bedeuten, dass weniger schief gehen kann, und das wiederum bedeutet weniger Arbeit. Würden mir keine Fehler unterlaufen, wäre mir meine Arbeit somit auch nicht erleichtert, aber den Traum, keine Fehler mehr zu begehen, habe ich ohnehin schon lange aufgegeben.

Da ich die Eingabedatenstruktur in die interne Datenstruktur übernommen habe, muss ich die Datenstrukturen auseinanderhalten, damit ich sie unabhängig voneinander bearbeiten kann. Zu diesem Zweck definiere ich ein separates Feld und sorge dafür, dass es von Konstruktor und Zugriffsfunktionen verwendet wird:

```
class Organization...
class Organization {
  constructor(data) {
    this._title = data.name;
    this._country = data.country;
  }
  get name() {return this._title;}
  set name(aString) {this._title = aString;}
  get country() {return this._country;}
  set country(aCountryCode) {this._country =
aCountryCode;}
}
```

Als Nächstes passe ich den Konstruktor an:

```
class Organization...
class Organization {
  constructor(data) {
    this._title = (data.title !== undefined) ?
data.title : data.name;
    this._country = data.country;
  }
  get name() {return this._title;}
  set name(aString) {this._title = aString;}
  get country() {return this._country;}
  set country(aCountryCode) {this._country =
aCountryCode;}
}
```

Aufrufer des Konstruktors können jetzt entweder `name` oder `title` verwenden (wobei `title` Priorität hat). Anschließend kann ich mir der Reihe nach alle Aufrufer des Konstruktors ansehen und den Namen ändern:

```
const organization = new Organization({title: "Acme
Gooseberries", country: "GB"});
```

Wenn das erledigt ist, kann ich die Unterstützung für den alten Namen entfernen:

```
class Organization...
class Organization {
  constructor(data) {
    this._title = data.title;
    this._country = data.country;
  }
  get name() {return this._title;}
  set name(aString) {this._title = aString;}
  get country() {return this._country;}
  set country(aCountryCode) {this._country =
aCountryCode;}
}
```

Jetzt verwenden der Konstruktor und die Daten den neuen Namen, und ich kann die Zugriffsfunktionen ändern. Dazu muss ich lediglich *Funktionsdeklaration ändern* (Abschnitt 6.5) auf sie anwenden:

```
class Organization...
class Organization {
  constructor(data) {
    this._title = data.title
    this._country = data.country;
  }
  get title() {return this._title;}
  set title(aString) {this._title = aString;}
```

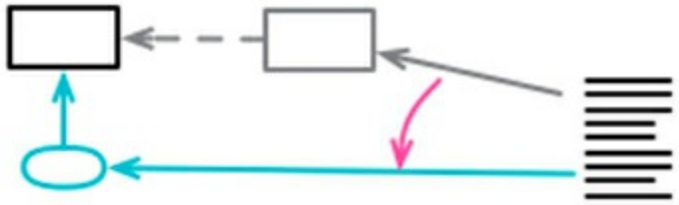
```
  get country() {return this._country;}
  set country(aCountryCode) {this._country =
aCountryCode;}
}
```

Ich habe dieses Verfahren hier in seiner umfangreichsten Form dargestellt, die für eine weitreichend verwendete Datenstruktur erforderlich ist. Wenn sie nur lokal verwendet wird, etwa in einer einzelnen Funktion, kann ich einfach die verschiedenen Eigenschaften in einem Rutsch umbenennen und auf die Kapselung verzichten. Ob das umfassende Verfahren angewendet werden sollte, ist eine Ermessensfrage – wenn jedoch einer meiner Tests fehlschlägt, ist das ein übliches Zeichen dafür, dass ich beim Refactoring in kleineren Schritten vorgehen sollte.

Einige Programmiersprachen ermöglichen mir, unveränderliche Datenstrukturen zu verwenden. In diesem Fall kann ich den Wert, anstatt ihn zu kapseln, dem neuen Namen zuweisen, nach und nach die Clients anpassen und den alten Namen schließlich entfernen. Bei der Verwendung veränderlicher Datenstrukturen kann das Duplizieren von Daten zu einer Katastrophe führen; unveränderliche Daten sind so verbreitet, weil sie derartige Katastrophen gekonnt verhindern.

9.3 Abgeleitete Variable durch Abfrage ersetzen (Replace Derived Variable with

Query)



```
get discountedTotal() {return this._discountedTotal;}
set discount(aNumber) {
  const old = this._discount;
  this._discount = aNumber;
  this._discountedTotal += old - aNumber;
}
```

```
get discountedTotal() {
  return this._baseTotal - this._discount;
}
set discount(aNumber) {this._discount = aNumber;}
```

9.3.1 MOTIVATION

Eine der größten Problemquellen von Software sind veränderliche Daten. Verschiedene Teile des Codes sind oft auf folgenschwere Weise miteinander gekoppelt, sodass veränderte Daten in einem der Teile zu Dominoeffekten führen können, die sich nur schwer nachvollziehen lassen. In vielen Situationen ist der

vollständige Verzicht auf veränderliche Daten unrealistisch. Ich plädiere jedoch dafür, den Gültigkeitsbereich veränderlicher Daten so weit wie möglich zu minimieren.

Eine ganze Menge kann ich schon dadurch erreichen, dass ich Variablen entferne, deren Wert ich auch berechnen kann. Eine Berechnung verdeutlicht oft, wie die Daten eigentlich zustande kommen, und schützt vor falschen Ergebnissen, wenn Sie übersehen, die Variable bei veränderten Ausgangsdaten zu aktualisieren.

Eine gerechtfertigte Ausnahme liegt vor, wenn die zur Berechnung herangezogenen Daten unveränderlich sind und wir erzwingen können, dass auch das Ergebnis unveränderlich ist. Transformationen dieser Art, die neue Datenstrukturen erzeugen, sind also durchaus vertretbar, selbst wenn sie durch Berechnungen ersetzbar wären. Tatsächlich kommen hier insbesondere zwei Konstrukte zum Einsatz: zum einen Objekte, die eine Datenstruktur mit einer Reihe berechneter Eigenschaften umgeben, und zum anderen Funktionen, die eine Datenstruktur in eine andere transformieren. Objekte sind deutlich besser geeignet, wenn sich die Ausgangsdaten verändern und Sie die Lebensdauer der abgeleiteten Datenstrukturen handhaben müssen. Sind die Ausgangsdaten hingegen unveränderlich oder die abgeleiteten Daten ausgesprochen kurzlebig, dann sind beide Ansätze zielführend.

9.3.2 VORGEHEN

- Suchen Sie nach allen Aktualisierungen der Variable. Verwenden Sie gegebenenfalls *Variable aufteilen* (Abschnitt 9.1), um die jeweiligen Zuweisungen voneinander zu trennen.
- Erstellen Sie eine Funktion, die den Wert der Variable berechnet.
- Verwenden Sie *Assertion einführen* (Abschnitt 10.6), um zu gewährleisten, dass der Wert der Variable und der Wert der Berechnung stets übereinstimmen, wenn die Variable verwendet wird.

  Falls erforderlich, können Sie *Variable kapseln* (Abschnitt 6.6) nutzen, um einen Ort für die Assertion bereitzustellen.
- Testen Sie.
- Ersetzen Sie alle Lesezugriffe auf die Variable durch Aufrufe der neuen Funktion.
- Testen Sie.
- Wenden Sie *Toten Code entfernen* (Abschnitt 8.9) auf die Deklaration und auf Zuweisungen zur Variable an.

9.3.3 BEISPIEL

Hier ist ein kleines, aber perfekt ausgeprägtes Beispiel für das Gegenteil von Schönheit:

```
class ProductionPlan...
get production() {return this._production;}
applyAdjustment(anAdjustment) {
  this._adjustments.push(anAdjustment);
  this._production += anAdjustment.amount;
}
```

Schönheit liegt im Auge des Betrachters. Ich erkenne

hier das Gegenteil in Form einer Duplizierung – nicht der übliche redundante Code, sondern redundante Daten. Bei einer Anpassung des Produktionsplans speichere ich diese nicht nur, sondern verwende sie auch, um eine Zählvariable zu verändern. Ich kann den Wert berechnen, ohne ihn aktualisieren zu müssen.

Ich bin jedoch ein vorsichtiger Zeitgenosse. Dass ich den Wert einfach berechnen kann, ist nur eine Hypothese, die ich aber durch Anwendung von *Assertion einführen* (Abschnitt 10.6) überprüfen kann:

```
class ProductionPlan...
get production() {
  assert(this._production ===
this.calculatedProduction);
  return this._production;
}

get calculatedProduction() {
  return this._adjustments
    .reduce((sum, a) => sum + a.amount, 0);
}
```

Nach der Einrichtung der Assertion führe ich meinen Test durch. Wenn er nicht fehlschlägt, kann ich die Rückgabe des Feldes durch die Rückgabe des berechneten Werts ersetzen:

```
class ProductionPlan...
get production() {
```

```
  assert(this._production ===
this.calculatedProduction);
  return this.calculatedProduction;
}
```

Anschließend wende ich *Funktion inline platzieren* (Abschnitt 6.2) an.

```
class ProductionPlan...
get production() {
  return this._adjustments
    .reduce((sum, a) => sum + a.amount, 0);
}
```

Referenzen auf die alte Variable lösche ich mit *Toten Code entfernen* (Abschnitt 8.9):

```
class ProductionPlan...
applyAdjustment(anAdjustment) {
  this._adjustments.push(anAdjustment);
  this._production += anAdjustment.amount;
}
```

9.3.4 BEISPIEL: MEHR ALS EINE QUELLE

Das obige Beispiel ist schön und einfach, weil es eindeutig nur eine einzige Quelle für den Wert von `production` gibt. Manchmal werden in der Zählvariable jedoch mehrere Elemente miteinander kombiniert:

```
class ProductionPlan...
constructor (production) {
  this._production = production;
  this._adjustments = [];
}
get production() {return this._production;}
applyAdjustment(anAdjustment) {
  this._adjustments.push(anAdjustment);
  this._production += anAdjustment.amount;
}
```

Wenn ich wie oben wieder *Assertion einführen* (Abschnitt 10.6) anwende, wird sie fehlschlagen, wenn der Anfangswert von `production` nicht null ist.

Ich kann aber noch immer die abgeleiteten Daten ersetzen. Der einzige Unterschied besteht darin, dass ich vorher *Variable aufteilen* (Abschnitt 9.1) anwenden muss.

```
constructor (production) {
  this._initialProduction = production;
  this._productionAccumulator = 0;
  this._adjustments = [];
}
get production() {
  return this._initialProduction +
         this._productionAccumulator;
}
```

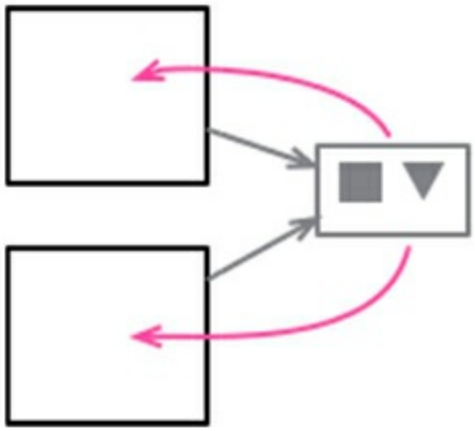
Jetzt kann ich *Assertion einführen* (Abschnitt 10.6) verwenden:

```
class ProductionPlan...
get production() {
  assert(this._productionAccumulator ===
         this.calculatedProductionAccumulator);
  return this._initialProduction +
         this._productionAccumulator;
}

get calculatedProductionAccumulator() {
  return this._adjustments
    .reduce((sum, a) => sum + a.amount, 0);
}
```

Jetzt könnte ich wie zuvor fortfahren. Ich würde allerdings `calculatedProductionAccumulator` als eigenständige Eigenschaft beibehalten und nicht inline platzieren.

9.4 Referenz durch Wert ersetzen (Change Reference to Value)

Umkehrung von: *Wert durch Referenz ersetzen* (Change Value to Reference, Abschnitt 9.5)



```
class Product {
applyDiscount(arg) {this._price.amount -= arg;}
```

```
class Product {
  applyDiscount(arg) {
    this._price = new Money(this._price.amount - arg,
                            this._price.currency);
  }
```

9.4.1 MOTIVATION

Wenn ein Objekt (oder eine Datenstruktur) in einer anderen Datenstruktur verschachtelt ist, kann ich das innere Objekt entweder als Referenz oder als Wert behandeln. Der Unterschied ist am deutlichsten daran erkennbar, wie ich Aktualisierungen der Eigenschaften des verschachtelten Objekts handhabe. Wenn ich es als

Referenz behandele, wird lediglich eine Eigenschaft des inneren Objekts geändert. Dabei bleibt das Objekt an sich erhalten. Wenn ich es als Wert behandele, wird hingegen das gesamte innere Objekt durch ein neues Objekt mit der gewünschten Eigenschaft ersetzt.

Wenn ich ein Feld als Wert behandele, kann ich die Klasse des inneren Objekts ändern, um daraus ein Value Object zu machen (`https://martinfowler.com/bliki/ValueObject.html`). Es ist generell einfacher, Überlegungen über Value Objects anzustellen, vor allem, weil sie unveränderlich sind. Unveränderliche Datenstrukturen lassen sich im Allgemeinen leichter handhaben. Unveränderliche Daten kann ich an andere Teile des Programms übergeben, ohne mir darüber Sorgen machen zu müssen, dass ein Wert verändert werden könnte, ohne dass das umgebende Objekt diese Änderung bemerkt. Ich kann Kopien der Werte in meinem Programm herumreichen und muss mir keine Gedanken um die Verwaltung von Speicherverknüpfungen machen. Value Objects erweisen sich insbesondere in verteilten oder nebenläufigen Systemen als äußerst nützlich.

Das ist auch ein Hinweis darauf, wann ich dieses Refactoring besser nicht durchführen sollte. Wenn ich ein Objekt zwischen mehreren anderen Objekten teilen möchte, sodass alle Änderungen am gemeinsam genutzten Objekt für alle anderen Objekte sichtbar sind,

muss ich eine Referenz verwenden.

9.4.2 VORGEHEN

- Prüfen Sie, ob die fragliche Klasse unveränderlich ist oder zu einer unveränderlichen Klasse gemacht werden kann.
- Wenden Sie *Setter entfernen* (Abschnitt 11.7) auf jeden Setter an.
- Stellen Sie eine auf Werten beruhende Methode zum Vergleich bereit, die auf die Felder des Value Objects zurückgreift.

  Zu diesem Zweck bieten die meisten Programmiersprachen eine überschreibbare Vergleichsfunktion. Für gewöhnlich müssen Sie zudem eine Funktion zum Erzeugen eines Hashwerts überschreiben.

9.4.3 BEISPIEL

Betrachten Sie ein `Person`-Objekt, das eine simple Telefonnummer enthält:

```
class Person...
constructor() {
  this._telephoneNumber = new TelephoneNumber();
}

get officeAreaCode() {return
this._telephoneNumber.areaCode;}
set officeAreaCode(arg) {
  this._telephoneNumber.areaCode = arg;
}
get officeNumber() {return
this._telephoneNumber.number;}
set officeNumber(arg) {this._telephoneNumber.number =
arg;}
```

```
class TelephoneNumber...
get areaCode() {return this._areaCode;}
set areaCode(arg) {this._areaCode = arg;}
get number() {return this._number;}
set number(arg) {this._number = arg;}
```

Der Code ist das Ergebnis der Anwendung von *Klasse extrahieren* (Abschnitt 7.5). Die alte Klasse verfügt noch immer über Methoden zum Aktualisieren des neuen Objekts. Hier bietet sich eine gute Gelegenheit, *Referenz durch Wert ersetzen* anzuwenden, weil es nur einen einzigen Verweis auf die neue Klasse gibt.

Als Erstes muss ich dafür sorgen, dass die Telefonnummer unveränderlich ist. Zu diesem Zweck wende ich *Setter entfernen* (Abschnitt 11.7) auf die Felder an. Der erste Schritt von *Setter entfernen* ist die Verwendung von *Funktionsdeklaration ändern* (Abschnitt 6.5), um die beiden Felder dem Konstruktor hinzuzufügen und ihn um Aufrufe der Setter zu ergänzen:

```
class TelephoneNumber...
constructor(areaCode, number) {
  this._areaCode = areaCode;
  this._number = number;
}
```

Jetzt sehe ich mir die Aufrufer der Setter an. Ich muss dort jeweils eine Neuzuweisung vornehmen. Ich beginne

mit der Vorwahl (`areaCode`):

```
class Person...
get officeAreaCode() {return
this._telephoneNumber.areaCode;}
set officeAreaCode(arg) {
  this._telephoneNumber =
                 new TelephoneNumber(arg,
this.officeNumber);
}
get officeNumber() {return
this._telephoneNumber.number;}
set officeNumber(arg) {this._telephoneNumber.number =
arg;}
```

Diesen Schritt wiederhole ich mit dem verbleibenden Feld:

```
class Person...
get officeAreaCode() {return
this._telephoneNumber.areaCode;}
set officeAreaCode(arg) {
  this._telephoneNumber =
                 new TelephoneNumber(arg,
this.officeNumber);
}
get officeNumber() {return
this._telephoneNumber.number;}
set officeNumber(arg) {
  this._telephoneNumber =
               new
TelephoneNumber(this.officeAreaCode, arg);
}
```

Nun ist die Telefonnummer unveränderlich und bereit, zu einem tatsächlichen Value Object zu werden. Ein echtes Value Object verwendet für Vergleiche die tatsächlichen Werte. In diesem Bereich gerät JavaScript ins Hintertreffen, denn weder die Sprache selbst noch die Standardbibliotheken bieten eine Möglichkeit, einen auf Referenzen beruhenden Vergleich durch einen auf Werten beruhenden zu ersetzen. Ich muss also selbst eine `equals`-Methode erstellen:

```
class TelephoneNumber...
equals(other) {
  if (!(other instanceof TelephoneNumber)) return
false;
  return this.areaCode === other.areaCode &&
         this.number === other.number;
}
```

Hierfür ist es wichtig, einen Test wie den folgenden durchzuführen:

```
it('telephone equals', function() {
  assert(        new TelephoneNumber("312", "555-
0142")
         .equals(new TelephoneNumber("312", "555-
0142")));
});
```

Die hier verwendete ungewöhnliche Formatierung soll verdeutlichen, dass es sich um Aufrufe desselben

Konstruktors handelt.

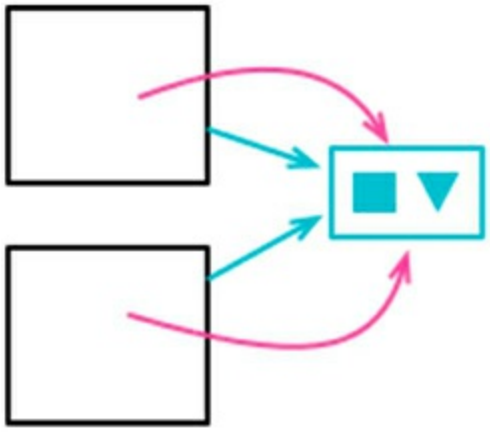
Bei diesem Test ist das Entscheidende, dass zwei unabhängig voneinander erstellte Objekte als gleich beurteilt werden.

In den meisten objektorientierten Programmiersprachen ist eine Vergleichsfunktion integriert, die überschrieben werden muss, wenn der Vergleich auf Werten beruhen soll. In Ruby kann ich den == Operator überschreiben; in Java überschreibe ich die Methode `Object.equals()`. Wenn ich eine Vergleichsmethode überschreibe, muss ich in der Regel auch eine Methode zur Erzeugung von Hashwerten (wie `Object.hashCode()` in Java) überschreiben, um zu gewährleisten, dass mein Wert in Collections, die Hashing verwenden, ordnungsgemäß funktioniert.

Wenn die Telefonnummer von mehr als einem Client verwendet wird, ändert sich an diesem Verfahren nichts. Bei der Anwendung von *Setter entfernen* (Abschnitt 11.7) muss ich dann nicht nur einen, sondern mehrere Clients berücksichtigen. Tests für die Ungleichheit von Telefonnummern sowie Vergleiche mit Werten, die keine Telefonnummern sind, oder mit `null`-Werten wären ebenfalls sinnvoll.

9.5 Wert durch Referenz ersetzen (Change Value to Reference)

Umkehrung von: *Referenz durch Wert ersetzen* (Change Reference to Value, Abschnitt 9.4)



```
let customer = new Customer(customerData);
```

```
let customer =
customerRepository.get(customerData.id);
```

9.5.1 MOTIVATION

Angenommen es existieren mehrere Datensätze, die mit derselben logischen Datenstruktur verknüpft sind. Ich könnte beispielsweise eine Liste von Bestellungen einlesen, von denen einige vom selben Kunden stammen. Ich kann den Kundendatensatz entweder als Wert oder als Referenz behandeln. Wenn ich ihn als Wert

behandele, werden die Kundendaten kopiert und in jeder Bestellung gespeichert. Behandele ich ihn als Referenz, gibt es nur eine einzige Datenstruktur, auf die mehrere Bestellungen verweisen.

Falls die Kundendaten niemals aktualisiert werden müssen, sind beide Ansätze geeignet. Es ist vielleicht ein wenig verwirrend, dass mehrere Kopien identischer Daten vorhanden sind. Das ist jedoch üblich und stellt kein Problem dar. In manchen Fällen kann es aufgrund redundanter Daten zu Speicherplatzengpässen kommen, aber das kommt, wie alle leistungsbezogenen Probleme, nur relativ selten vor.

Die größten Schwierigkeiten bereitet das Vorhandensein mehrerer physischer Kopien der gleichen logischen Daten, wenn ich die gemeinsam genutzten Daten aktualisieren muss. Ich bin dann gezwungen, sämtliche Kopien zu finden und zu aktualisieren. Sollte ich dabei eine übersehen, kommt es zu einer möglicherweise folgenschweren Inkonsistenz in den Daten. In diesem Fall ist es oft sinnvoll, die redundanten Daten zu einer einzelnen Referenz zusammenzufassen. Auf diese Weise sind Änderungen in allen Bestellungsdatensätzen sichtbar.

Einen Wert durch eine Referenz zu ersetzen bedeutet, dass eine Datenstruktur durch ein einzelnes Objekt repräsentiert wird. Für gewöhnlich benötige ich dafür irgendeine Art von Aufbewahrungsort, also ein

Repository, an dem ich auf die Objekte zugreifen kann. Ich erstelle dann pro Datenstruktur jeweils nur einmalig ein Objekt, auf das ich über das Repository von überall aus zugreifen kann.

9.5.2 VORGEHEN

- Erstellen Sie ein Repository für Instanzen des zugehörigen Objekts (sofern noch keins vorhanden ist).
- Vergewissern Sie sich, dass der Konstruktor in der Lage ist, die richtige Instanz des zugehörigen Objekts zu finden.
- Ändern Sie die Konstruktoren des umgebenden Objekts, sodass es das Repository verwendet, um an das zugehörige Objekt zu gelangen. Führen Sie nach jeder Änderung Tests durch.

9.5.3 BEISPIEL

Ich betrachte zunächst eine Klasse `Order`, die Bestellungen repräsentiert und die ich beispielsweise anhand eines JSON-Dokuments erzeuge. Die Bestelldaten enthalten eine Kunden-ID, die ich zum Erstellen eines Kunden-Objekts (`Customer`) verwende:

```
class Order...
constructor(data) {
  this._number = data.number;
  this._customer = new Customer(data.customer);
  // weitere Daten einlesen
}
get customer() {return this._customer;}

class Customer...
constructor(id) {
```

```
  this._id = id;
}
get id() {return this._id;}
```

Das auf diese Weise erstellte Kunden-Objekt ist ein Wert. Wenn fünf Bestellungen vorliegen, die auf die Kunden-ID 123 verweisen, sind fünf verschiedene Kunden-Objekte vorhanden. An einem dieser Objekte vorgenommene Änderungen haben keine Auswirkung auf die übrigen Objekte. Falls ich eine Ergänzung des Kunden-Objekts vornehmen möchte, beispielsweise durch Daten, die der Kundendienst in Erfahrung gebracht hat, dann muss ich alle fünf Kunden-Objekte mit identischen Daten aktualisieren. Mehrfach vorhandene Objekte wie diese machen mich immer nervös – es ist einfach verwirrend, wenn mehrere Objekte ein und dasselbe repräsentieren, wie hier einen Kunden. Dieses Problem ist besonders heikel, wenn das Kunden-Objekt veränderlich ist, weil dies zu Inkonsistenzen in den Kunden-Objekten führen kann.

Wenn ich immer dasselbe Objekt verwenden möchte, benötige ich dafür einen Speicherort. Wo genau das Abspeichern erfolgt, wird sich von Programm zu Programm unterscheiden, aber in einfachen Fällen wie diesem verwende ich ein Repository-Objekt (`https://martinfowler.com/eaaCatalog/repository.html`):

```
let _repositoryData;

export function initialize() {
  _repositoryData = {};
  _repositoryData.customers = new Map();
}

export function registerCustomer(id) {
  if (! _repositoryData.customers.has(id))
    _repositoryData.customers.set(id, new
Customer(id));
  return findCustomer(id);
}

export function findCustomer(id) {
  return _repositoryData.customers.get(id);
}
```

Das Repository ermöglicht mir, Kunden-Objekte mit einer ID anzulegen, und gewährleistet, dass ich nur ein Kunden-Objekt pro ID erstelle. Nachdem es eingerichtet ist, kann ich dafür sorgen, dass der Konstruktor der `Order`-Klasse das Repository verwendet.

Wenn ich dieses Refactoring in der Praxis vornehme, ist das Repository oft schon vorhanden, sodass ich es direkt verwenden kann.

Als Nächstes muss ich herausfinden, wie der Konstruktor der `Order`-Klasse an das richtige Kunden-Objekt gelangen kann. Das ist in diesem Fall ganz einfach, denn die Kunden-ID ist im Eingabedatenstrom bereits

enthalten:

```
class Order...
constructor(data) {
  this._number = data.number;
  this._customer = registerCustomer(data.customer);
  // weitere Daten einlesen
}
get customer() {return this._customer;}
```

Jetzt sind alle Änderungen, die ich am Kunden einer Bestellung vornehme, in allen anderen Bestellungen dieses Kunden sichtbar.

In diesem Beispiel habe ich bei der ersten Bestellung, die auf einen Kunden verwies, ein Kunden-Objekt erstellt. Ein anderer gängiger Ansatz ist die Erstellung einer Kundenliste, die im Repository hinterlegt wird, um beim Einlesen der Bestellungen darauf zu verweisen. In diesem Fall wäre eine Bestellung, die eine Kunden-ID enthält, die im Repository nicht vorhanden ist, ein Hinweis auf einen Fehler.

Bei diesem Code ist problematisch, dass der Konstruktor an das globale Repository gekoppelt ist. Bei globalen Daten sollte man immer Vorsicht walten lassen – wie bei einem starken Medikament, das in kleinen Dosen zwar heilsam ist, in zu großer Menge aber giftig sein kann. Wenn ich mir darüber Sorgen mache, kann ich das Repository dem Konstruktor als Parameter übergeben.

Kapitel 10

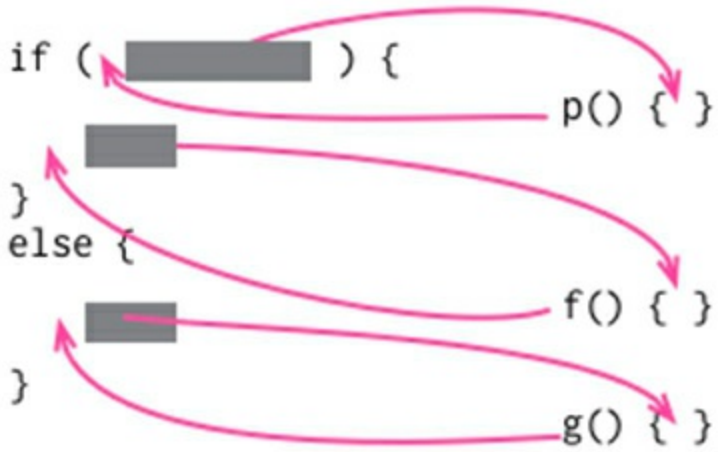
Bedingungen vereinfachen

Ein großer Teil der Leistungsfähigkeit von Programmen beruht auf ihrer Fähigkeit, bedingte Programmlogik zu implementieren – aber leider beruht auch ein beträchtlicher Teil der Komplexität eines Programms auf dieser bedingten Logik. Ich verwende Refactorings häufig, um solche Codeabschnitte verständlicher zu machen. So wende ich etwa regelmäßig *Bedingung zerlegen* (Abschnitt 10.1) auf komplizierte Bedingungen an, und ich verwende *Bedingten Ausdruck zusammenfassen* (Abschnitt 10.2), um logische Kombinationen zu verdeutlichen. Wenn ich vor der eigentlichen Verarbeitung einige Vorabüberprüfungen vornehmen möchte, verwende ich *Verschachtelte Bedingung durch Wächterbedingung ersetzen* (Abschnitt 10.3). Und wenn ich auf Fallunterscheidungen stoße, die mehrfach die gleiche Programmlogik verwenden, ist der richtige Zeitpunkt gekommen, um *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) anzuwenden.

Viele Bedingungen dienen dazu, Sonderfällen Rechnung zu tragen, wie etwa `null`-Werte. Wenn sich die Abfragelogik stark ähnelt, können Sie mit *Sonderfall*

einführen (Abschnitt 10.5) viel redundanten Code entfernen, und obwohl ich Bedingungen eigentlich lieber entferne, halte ich *Assertion einführen* (Abschnitt 10.6) für eine nützliche Ergänzung, wenn ich den Zustand eines Programms zum Ausdruck bringen (und überprüfen) möchte.

10.1 Bedingung zerlegen (Decompose Conditional)




```
if (!aDate.isBefore(plan.summerStart) &&
    !aDate.isAfter(plan.summerEnd))
  charge = quantity * plan.summerRate;
else
  charge = quantity * plan.regularRate +
           plan.regularServiceCharge;
```

```
if (summer())
  charge = summerCharge();
else
  charge = regularCharge();
```

10.1.1 MOTIVATION

Eine der häufigsten Ursachen für die Komplexität von Programmen ist komplizierte bedingte Logik. Wenn ich Code schreibe, der unterschiedliche Aufgaben in Abhängigkeit von verschiedenen Bedingungen erledigen soll, endet das schnell mit einer ziemlich langen Funktion. Die Länge einer Funktion ist selbst schon ein Faktor, der das Lesen erschwert, aber Bedingungen erhöhen den Schwierigkeitsgrad weiter. Das Problem wird meist dadurch verursacht, dass ich anhand der Bedingungen und Anweisungen zwar erkennen kann, *was* geschieht, diese aber leicht verschleiern, *warum* es geschieht.

Wie bei jedem größeren Codeblock kann ich meine Intention deutlicher zum Ausdruck bringen, indem ich ihn zerlege und bestimmte Abschnitte durch einen Funktionsaufruf ersetze, dessen Bezeichnung sich an seiner Aufgabe orientiert. Bei Bedingungen gefällt es mir besonders, diese Ersetzung für den Fall, dass die Bedingung erfüllt ist, und für die möglichen Alternativen vorzunehmen. Auf diese Weise hebe ich die Bedingung hervor und verdeutliche, was in welchem Fall ausgeführt wird.

Im Grunde genommen handelt es sich hierbei um einen besonderen Fall der Anwendung von *Funktion extrahieren* (Abschnitt 6.1) auf meinen Code. Ich möchte jedoch betonen, dass sich dieser Fall in Anbetracht des geringen Aufwands oft als äußerst lohnenswert herausgestellt hat.

10.1.2 VORGEHEN

- Wenden Sie *Funktion extrahieren* (Abschnitt 6.1) auf die Bedingung und alle Zweige der Bedingung an.

10.1.3 BEISPIEL

Nehmen wir an, ich berechne die Kosten (`charge`) für etwas, das im Sommer und im Winter unterschiedlich teuer ist:

```
if (!aDate.isBefore(plan.summerStart) &&
    !aDate.isAfter(plan.summerEnd))
  charge = quantity * plan.summerRate;
else
  charge = quantity * plan.regularRate +
           plan.regularServiceCharge;
```

Ich extrahiere die Bedingung und erstelle eine eigene Funktion:

```
if (summer())
  charge = quantity * plan.summerRate;
else
```

```
  charge = quantity * plan.regularRate +
           plan.regularServiceCharge;

function summer() {
return !aDate.isBefore(plan.summerStart) &&
       !aDate.isAfter(plan.summerEnd);
}
```

Jetzt ist die `then`-Verzweigung an der Reihe:

```
if (summer())
  charge = summerCharge();
else
  charge = quantity * plan.regularRate +
           plan.regularServiceCharge;

function summer() {
return !aDate.isBefore(plan.summerStart) &&
       !aDate.isAfter(plan.summerEnd);
}
function summerCharge() {
  return quantity * plan.summerRate;
}
```

Und schließlich die `else`-Verzweigung:

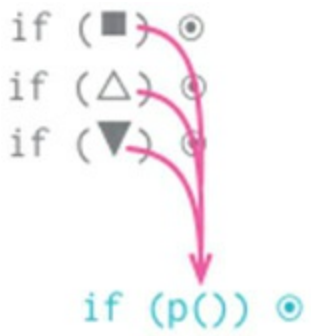
```
if (summer())
  charge = summerCharge();
else
  charge = regularCharge();

function summer() {
return !aDate.isBefore(plan.summerStart) &&
       !aDate.isAfter(plan.summerEnd);
```

```
}
function summerCharge() {
  return quantity * plan.summerRate;
}
function regularCharge() {
  return quantity * plan.regularRate +
         plan.regularServiceCharge;
}
```

Nachdem das erledigt ist, möchte ich für die bedingte Anweisung den ternären Operator verwenden:

```
charge = summer() ? summerCharge() : regularCharge();

function summer() {
return !aDate.isBefore(plan.summerStart) &&
       !aDate.isAfter(plan.summerEnd);
}
function summerCharge() {
  return quantity * plan.summerRate;
}
function regularCharge() {
  return quantity * plan.regularRate +
         plan.regularServiceCharge;
}
```

10.2 Bedingten Ausdruck zusammenfassen (Consolidate Conditional Expression)




```
if (anEmployee.seniority < 2) return 0;
if (anEmployee.monthsDisabled > 12) return 0;
if (anEmployee.isPartTime) return 0;
```

```
if (isNotEligableForDisability()) return 0;

function isNotEligableForDisability() {
  return ((anEmployee.seniority < 2)
          || (anEmployee.monthsDisabled > 12)
          || (anEmployee.isPartTime));
}
```

10.2.1 MOTIVATION

Gelegentlich stoße ich auf eine Reihe von Bedingungen, die sich zwar unterscheiden, deren resultierende Anweisung aber identisch ist. Ich verwende in diesem Fall die Operatoren `and` und `or`, um diese Bedingungen zu einer einzelnen Bedingung mit nur einer resultierenden Anweisung zusammenzufassen.

Das Zusammenfassen von bedingtem Code ist aus zwei Gründen von Bedeutung. Erstens wird dadurch klarer, dass ich tatsächlich nur eine einzige Überprüfung vornehme, die andere Überprüfungen kombiniert. Die Abfolge von Bedingungen bewirkt zwar dasselbe, erweckt aber den Eindruck, als ob ich eine Folge verschiedener Überprüfungen vornehmen würde, die sich nur zufällig nahe beisammen befinden. Der zweite Grund ist, dass hierdurch oft der Weg für *Funktion extrahieren* (Abschnitt 6.1) geebnet wird. Die Extraktion einer Bedingung ist eines der nützlichsten Refactorings, die mir zur Verfügung stehen, um die Verständlichkeit meines Codes zu verbessern. Sie ersetzt eine Anweisung, die besagt, *was* geschieht, durch eine Anweisung, die erklärt, *warum* es geschieht.

Neben den Gründen, die für das Zusammenfassen der Bedingungen sprechen, gibt es auch solche, die dagegen sprechen. Wenn ich es für Überprüfungen halte, die tatsächlich voneinander unabhängig sind und somit nicht als eine einzige Überprüfung betrachtet werden sollten, nehme ich dieses Refactoring nicht vor.

10.2.2 VORGEHEN

- Vergewissern Sie sich, dass keine der bedingten Anweisungen irgendwelche Nebeneffekte verursacht.

  Wenden Sie ansonsten zuerst *Abfrage von Veränderung trennen* (Abschnitt 11.1) auf die fraglichen Bedingungen an.

- Kombinieren Sie die Bedingungen zweier bedingter Anweisungen

durch einen Logikoperator.

Abfolgen von Bedingungen können mit `or`, verschachtelte `if`-Anweisungen durch `and` miteinander kombiniert werden.

- Testen Sie.
- Wiederholen Sie das Kombinieren der bedingten Anweisungen, bis sie alle zu einer einzigen Bedingung zusammengefasst sind.
- Ziehen Sie in Betracht, auf die sich ergebende Bedingung *Funktion extrahieren* (Abschnitt 6.1) anzuwenden.

10.2.3 BEISPIEL

Beim Durchstöbern von Code stoße ich auf Folgendes:

```
function disabilityAmount(anEmployee) {
  if (anEmployee.seniority < 2) return 0;
  if (anEmployee.monthsDisabled > 12) return 0;
  if (anEmployee.isPartTime) return 0;
  // Berechnung des Betrags
```

Hierbei handelt es sich um eine Abfolge von Bedingungen, die alle zum gleichen Ergebnis führen. Deshalb sollte ich sie zu einer einzigen Bedingung zusammenfassen. Bei einer Abfolge wie dieser verwende ich dafür den `or`-Operator:

```
function disabilityAmount(anEmployee) {
  if ((anEmployee.seniority < 2)
     || (anEmployee.monthsDisabled > 12)) return 0;
  if (anEmployee.isPartTime) return 0;
  // Berechnung des Betrags
```

Ich teste und binde anschließend die verbleibende Bedingung ein:

```
function disabilityAmount(anEmployee) {
  if ((anEmployee.seniority < 2)
     || (anEmployee.monthsDisabled > 12)
     || (anEmployee.isPartTime)) return 0;
  // Berechnung des Betrags
```

Jetzt, da es nur noch eine Bedingung gibt, wende ich *Funktion extrahieren* (Abschnitt 6.1) an:

```
function disabilityAmount(anEmployee) {
  if (isNotEligableForDisability()) return 0;
  // Berechnung des Betrags

function isNotEligableForDisability() {
  return ((anEmployee.seniority < 2)
          || (anEmployee.monthsDisabled > 12)
          || (anEmployee.isPartTime));
}
```

10.2.4 BEISPIEL: VERWENDUNG VON »AND«

Das vorherige Beispiel zeigte, wie Anweisungen durch den `or`-Operator kombiniert werden. Es gibt aber auch Fälle, in denen der `and`-Operator notwendig ist, wenn beispielsweise verschachtelte `if`-Anweisungen verwendet werden:

```
if (anEmployee.onVacation)
  if (anEmployee.seniority > 10)
   return 1;
return 0.5;
```

Ich kombiniere die Bedingungen durch einen `and`-Operator:

```
if ((anEmployee.onVacation)
    && (anEmployee.seniority > 10)) return 1;
return 0.5;
```

Wenn beide Fälle auftreten, kann ich die Verwendung von `and`- und `or`-Operatoren nach Bedarf miteinander kombinieren. In diesem Fall wird es schnell chaotisch, deshalb mache ich ausgiebigen Gebrauch von *Funktion extrahieren* (Abschnitt 6.1), damit das Ganze verständlich bleibt.

10.3 Verschachtelte Bedingung durch Wächterbedingung ersetzen (Replace Nested Conditional with Guard Clauses)

```
function getPayAmount() {
  let result;
  if (isDead)
    result = deadAmount();
  else {
    if (isSeparated)
      result = separatedAmount();
    else {
      if (isRetired)
        result = retiredAmount();
      else
        result = normalPayAmount();
     }
  }
  return result;
}
```

⇓

```
function getPayAmount() {
  if (isDead) return deadAmount();
  if (isSeparated) return separatedAmount();
```

```
  if (isRetired) return retiredAmount();
  return normalPayAmount();
}
```

10.3.1 MOTIVATION

Bedingte Ausdrücke treten meist in zwei Stilen auf. Beim ersten Stil sind beide Zweige der Bedingung Teil des normalen Verhaltens, beim zweiten Stil hingegen gehört ein Zweig zum normalen Verhalten, und der andere kennzeichnet einen Sonderfall.

Derartigen Bedingungen liegen unterschiedliche Absichten zugrunde – und diese Absichten sollten im Code zum Ausdruck gebracht werden. Wenn beide Zweige zum normalen Verhalten gehören, verwende ich bedingte Anweisungen mit einem `if`- und einem `else`-Zweig. Wenn die Bedingung das Vorliegen eines Sonderfalls überprüft und dieser eintrifft, breche ich ab. Eine solche Art der Prüfung wird oftmals als *Wächterbedingung* (*guard clause*) oder kurz *Wächter* bezeichnet.

Der zentrale Aspekt von *Verschachtelte Bedingung durch Wächterbedingung ersetzen* ist die Betonung. Wenn ich ein `if-then-else`-Konstrukt verwende, verleihe ich beiden Zweigen das gleiche Gewicht. Dem Betrachter wird dadurch signalisiert, dass beide Zweige gleich wahrscheinlich und wichtig sind. In der Wächterbedingung heißt es stattdessen: »Dieser Fall

betrifft nicht den eigentlichen Kern der Funktion; falls dieser Fall eintritt, dann tue etwas und steig aus.«

Ich habe festgestellt, dass ich *Verschachtelte Bedingung durch Wächterbedingung ersetzen* häufig verwende, wenn ich mit einem Programmierer zusammenarbeite, der gelernt hat, dass eine Methode nur einen Einstiegspunkt und nur einen Ausstiegspunkt besitzen sollte. Moderne Programmiersprachen erzwingen einen einzigen Einstiegspunkt. Aber nur einen Ausstiegspunkt zu verwenden, ist eigentlich keine besonders nützliche Regel. Das Schlüsselprinzip heißt Klarheit: Wenn die Methode durch einen einzigen Ausstiegspunkt verständlicher wird, verwenden Sie auch nur einen – und sonst nicht.

10.3.2 VORGEHEN

- Wählen Sie die äußerste Bedingung aus, die ersetzt werden soll, und wandeln Sie diese in eine Wächterbedingung um.
- Testen Sie.
- Wiederholen Sie das Ganze nach Bedarf.
- Wenn alle Wächterbedingungen dasselbe Ergebnis liefern, sollten Sie *Bedingten Ausdruck zusammenfassen* (Abschnitt 10.2) anwenden.

10.3.3 BEISPIEL

Der folgende Code berechnet einen Geldbetrag, der an einen Angestellten ausgezahlt wird. Er wird jedoch nur dann berücksichtigt, wenn der Angestellte noch in der

Firma tätig ist, deshalb müssen die beiden anderen möglichen Fälle überprüft werden:

```
function payAmount(employee) {
  let result;
  if(employee.isSeparated) {
    result = {amount: 0, reasonCode: "SEP"};
  }
  else {
    if (employee.isRetired) {
      result = {amount: 0, reasonCode: "RET"};
    }
    else {
      // Logik zur Berechnung des Betrags
      lorem.ipsum(dolor.sitAmet);
      consectetur(adipiscing).elit();
      sed.do.eiusmod = tempor.incididunt.ut(labore) &&
                       dolore(magna.aliqua);
      ut.enim.ad(minim.veniam);
      result = someFinalComputation();
    }
  }
  return result;
}
```

Die Verschachtelung der Bedingungen verschleiert die wahre Bedeutung dessen, was hier eigentlich vor sich geht. Der entscheidende Teil des Codes kommt nur dann zur Ausführung, wenn die Bedingungen nicht erfüllt sind. In diesem Fall wird die Intention des Codes durch Wächterbedingungen sehr viel deutlicher.

Ich unternehme wie bei jedem Refactoring nur kleine

Schritte. Zunächst befasse ich mich mit der allerersten Bedingung:

```
function payAmount(employee) {
  let result;
  if (employee.isSeparated) return {amount: 0,
                                    reasonCode:
"SEP"};
  if (employee.isRetired) {
    result = {amount: 0, reasonCode: "RET"};
  }
  else {
    // Logik zur Berechnung des Betrags
    lorem.ipsum(dolor.sitAmet);
    consectetur(adipiscing).elit();
    sed.do.eiusmod = tempor.incididunt.ut(labore) &&
                     dolore(magna.aliqua);
    ut.enim.ad(minim.veniam);
    result = someFinalComputation();
  }
  return result;
}
```

Ich teste diese Änderung und wende mich der nächsten zu:

```
function payAmount(employee) {
  let result;
  if (employee.isSeparated) return {amount: 0,
                                    reasonCode:
"SEP"};
  if (employee.isRetired) return {amount: 0,
                                  reasonCode: "RET"};
  // Logik zur Berechnung des Betrags
```

```
  lorem.ipsum(dolor.sitAmet);
  consectetur(adipiscing).elit();
  sed.do.eiusmod = tempor.incididunt.ut(labore) &&
                   dolore(magna.aliqua);
  ut.enim.ad(minim.veniam);
  result = someFinalComputation();
  return result;
}
```

Die Variable `result` ist jetzt überflüssig geworden, also lösche ich sie:

```
function payAmount(employee) {
  let result;
  if (employee.isSeparated) return {amount: 0,
                                    reasonCode:
"SEP"};
  if (employee.isRetired) return {amount: 0,
                                  reasonCode: "RET"};
  // Logik zur Berechnung des Betrags
  lorem.ipsum(dolor.sitAmet);
  consectetur(adipiscing).elit();
  sed.do.eiusmod = tempor.incididunt.ut(labore) &&
                   dolore(magna.aliqua);
  ut.enim.ad(minim.veniam);
  return = someFinalComputation();
}
```

Die Regel lautet: Für jede entfernte veränderliche Variable gibt es ein Fleißsternchen.

10.3.4 BEISPIEL: UMKEHRUNG DER BEDINGUNGEN

Bei der Durchsicht des Manuskripts der ersten Ausgabe dieses Buchs wies Joshua Kerievsky darauf hin, dass wir häufig *Verschachtelte Bedingung durch Wächterbedingung ersetzen* anwenden, indem wir die Bedingungen der Ausdrücke umkehren. Er hat mir sogar ein Beispiel zur Verfügung gestellt, sodass ich meine Vorstellungskraft nicht weiter bemühen muss.

```
function adjustedCapital(anInstrument) {
  let result = 0;
  if (anInstrument.capital > 0) {
    if (anInstrument.interestRate > 0 &&
        anInstrument.duration > 0) {
      result = (anInstrument.income /
anInstrument.duration)
                * anInstrument.adjustmentFactor;
    }
  }
  return result;
}
```

Ich nehme die Ersetzungen wieder der Reihe nach vor, allerdings kehre ich jetzt die Bedingung für den Wächter um:

```
function adjustedCapital(anInstrument) {
  let result = 0;
  if (anInstrument.capital <= 0) return result;
  if (anInstrument.interestRate > 0 &&
      anInstrument.duration > 0) {
    result = (anInstrument.income /
anInstrument.duration)
```

```
              * anInstrument.adjustmentFactor;
  }
  return result;
}
```

Die nächste Bedingung ist etwas komplizierter, deshalb nehme ich die Änderung in zwei Schritten vor. Als Erstes füge ich einfach einen `not`-Operator hinzu:

```
function adjustedCapital(anInstrument) {
  let result = 0;
  if (anInstrument.capital <= 0) return result;
  if (!(anInstrument.interestRate > 0 &&
        anInstrument.duration > 0)) return result;
  result = (anInstrument.income /
anInstrument.duration)
            * anInstrument.adjustmentFactor;
  return result;
}
```

`not`-Operatoren wie diesen in einer Bedingung zu belassen, belastet meine Gehirnwindungen in einem Ausmaß, das schmerzhaft ist, deshalb nehme ich eine Vereinfachung vor:

```
function adjustedCapital(anInstrument) {
  let result = 0;
  if (anInstrument.capital <= 0) return result;
  if (anInstrument.interestRate <= 0 ||
      anInstrument.duration <= 0) return result;
  result = (anInstrument.income /
anInstrument.duration)
```

```
            * anInstrument.adjustmentFactor;
  return result;
}
```

Beide Bedingungen geben dasselbe Ergebnis zurück, also wende ich *Bedingten Ausdruck zusammenfassen* (Abschnitt 10.2) an:

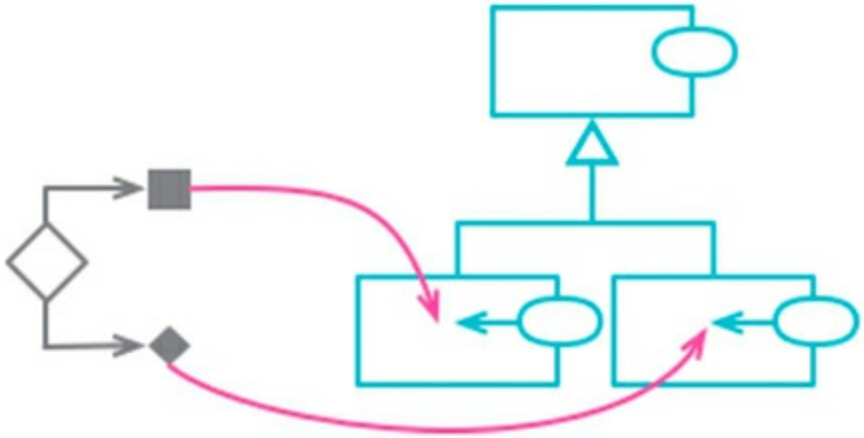
```
function adjustedCapital(anInstrument) {
  let result = 0;
  if (   anInstrument.capital      <= 0
      || anInstrument.interestRate <= 0
      || anInstrument.duration     <= 0) return
result;
  result = (anInstrument.income /
anInstrument.duration)
            * anInstrument.adjustmentFactor;
  return result;
}
```

Die Variable `result` erfüllt hier zwei Aufgaben. Zunächst wird ihr Wert auf null gesetzt. Damit ist festgelegt, welchen Wert die Funktion zurückgibt, wenn eine der Wächterbedingungen erfüllt ist. Der zweite Wert, den sie annimmt, ist das Ergebnis der abschließenden Berechnung. Ich kann sie loswerden, was nicht nur die doppelte Verwendung beseitigt, sondern mir auch ein Fleißsternchen einbringt:

```
function adjustedCapital(anInstrument) {
  if (   anInstrument.capital      <= 0
```

```
      || anInstrument.interestRate <= 0
      || anInstrument.duration     <= 0) return 0;
  return (anInstrument.income / anInstrument.duration)
          * anInstrument.adjustmentFactor;
}
```

10.4 Bedingung durch Polymorphie ersetzen (Replace Conditional with Polymorphism)



```
switch (bird.type) {
  case 'EuropeanSwallow':
    return "average";
  case 'AfricanSwallow':
    return (bird.numberOfCoconuts > 2) ? "tired" :
"average";
  case 'NorwegianBlueParrot':
    return (bird.voltage > 100) ? "scorched" :
"beautiful";
  default:
    return "unknown";
```

```
class EuropeanSwallow {
  get plumage() {
    return "average";
  }
class AfricanSwallow {
  get plumage() {
    return (this.numberOfCoconuts > 2) ? "tired" :
"average";
  }
class NorwegianBlueParrot {
  get plumage() {
    return (this.voltage > 100) ? "scorched" :
"beautiful";
  }
```

10.4.1 MOTIVATION

Komplexe bedingte Logik gehört zu den schwierigsten Themen, über die man beim Programmieren nachdenken muss, deshalb suche ich immer nach Möglichkeiten, sie zu strukturieren. Häufig stelle ich fest, dass ich die Logik verschiedenen Szenarien zuordnen kann – übergeordneten Fällen –, um die Bedingungen zu unterteilen. Manchmal reicht es schon aus, die Aufteilung in der Struktur einer Bedingung selbst widerzuspiegeln, aber die Verwendung von Klassen und Polymorphie kann diese Aufteilung viel expliziter verdeutlichen.

Ein typisches Beispiel hierfür ist, dass ich eine Reihe verschiedener Typen bilden kann, deren bedingte Logik unterschiedlich gehandhabt wird. Ich könnte beispielsweise feststellen, dass Bücher, Musik und Lebensmittel aufgrund ihres Typs anders gehandhabt werden. Das wird besonders offensichtlich, wenn mehrere Funktionen vorhanden sind, die `switch`-Anweisungen zur Fallunterscheidung anhand eines Typenschlüssels verwenden. In diesem Fall entferne ich die redundante `switch`-Logik, indem ich für jeden Typ eine Klasse erstelle und Polymorphie verwende, um das typenspezifische Verhalten hervorzuheben.

Es gibt auch die Situation, dass ich mir die Logik als einen Basisfall mit verschiedenen Varianten vorstellen kann. Der Basisfall tritt am häufigsten auf und ist der unkomplizierteste. Die dazugehörige Logik bringe ich in einer Basisklasse unter, über die ich mir Gedanken machen kann, ohne die Varianten berücksichtigen zu müssen. Für die Varianten erstelle ich anschließend Unterklassen, deren Code den Unterschied zum Basisfall hervorhebt.

Polymorphie ist ein essentielles Prinzip der objektorientierten Programmierung – und wie jedes nützliche Prinzip ist auch dieses anfällig dafür, überstrapaziert zu werden. Ich bin schon Entwicklern begegnet, die den Standpunkt vertreten, dass sämtliche bedingte Logik durch Polymorphie ersetzt werden sollte. Ich vertrete eine andere Ansicht. Der Großteil meiner

bedingten Programmlogik verwendet die elementaren Anweisungen `if/else` und `switch/case`. Aber wenn sich komplexe bedingte Logik auf die beschriebene Weise verbessern lässt, erweist sich Polymorphie als leistungsstarkes Instrument.

10.4.2 VORGEHEN

- Falls noch keine Klassen vorhanden sind, die Polymorphie ermöglichen, können Sie diese zusammen mit einer Fabrikfunktion zur Rückgabe der richtigen Instanz erstellen.
- Verwenden Sie im aufrufenden Code die Fabrikfunktion.
- Verschieben Sie die bedingte Funktion in die Basisklasse.

  Wenn die bedingte Logik nicht in einer eigenständigen Funktion enthalten ist, können Sie *Funktion extrahieren* (Abschnitt 6.1) verwenden, um das zu ändern.
- Wählen Sie eine der Unterklassen aus und erstellen Sie eine Unterklassenmethode, die Ihre Methode mit den bedingten Anweisungen überschreibt. Kopieren Sie den Inhalt eines Zweigs der bedingten Anweisung in die Unterklassenmethode und passen Sie ihn an.
- Wiederholen Sie diesen Vorgang für sämtliche Zweige der bedingten Anweisung.
- In der Basisklasse sollten Sie den `default`-Fall berücksichtigen. Sollte die Basisklasse abstrakt sein, können Sie eine abstrakte Methode deklarieren oder eine Fehlermeldung auslösen, um zu verdeutlichen, dass es sich um die Verantwortung einer Unterklasse handelt.

10.4.3 BEISPIEL

Ein Freund von mir besitzt verschiedene Vögel und

möchte wissen, wie schnell sie fliegen können (`speeds`) und wie ihr Gefieder (`plumages`) aussieht. Wir verwenden einige kleine Programme, um diese Informationen in Erfahrung zu bringen:

```
function plumages(birds) {
  return new Map(birds.map(b => [b.name,
plumage(b)]));
}
function speeds(birds) {
  return new Map(birds.map(b => [b.name,

airSpeedVelocity(b)]));
}

function plumage(bird) {
  switch (bird.type) {
  case 'EuropeanSwallow':
    return "average";
  case 'AfricanSwallow':
    return (bird.numberOfCoconuts > 2) ? "tired" :
"average";
  case 'NorwegianBlueParrot':
    return (bird.voltage > 100) ? "scorched" :
"beautiful";
  default:
    return "unknown";
   }
}

function airSpeedVelocity(bird) {
  switch (bird.type) {
  case 'EuropeanSwallow':
    return 35;
  case 'AfricanSwallow':
    return 40 - 2 * bird.numberOfCoconuts;
```

```
  case 'NorwegianBlueParrot':
   return (bird.isNailed) ? 0 : 10 + bird.voltage /
10;
  default:
    return null;
  }
}
```

Die verschiedenen Operationen variieren je nachdem, von welchem Typ ein Vogel ist. Deshalb ist es sinnvoll, Klassen zu erstellen und Polymorphie für typspezifisches Verhalten zu verwenden.

Zunächst wende ich *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) auf `airSpeedVelocity` und `plumage` an:

```
function plumage(bird) {
  return new Bird(bird).plumage;
}
function airSpeedVelocity(bird) {
  return new Bird(bird).airSpeedVelocity;
}

class Bird {
  constructor(birdObject) {
    Object.assign(this, birdObject);
  }
  get plumage() {
    switch (this.type) {
    case 'EuropeanSwallow':
      return "average";
    case 'AfricanSwallow':
      return (this.numberOfCoconuts > 2)
```

```
                                          ? "tired" :
"average";
    case 'NorwegianBlueParrot':
      return (this.voltage > 100) ? "scorched" :
"beautiful";
    default:
      return "unknown";
    }
  }
  get airSpeedVelocity() {
    switch (this.type) {
    case 'EuropeanSwallow':
      return 35;
    case 'AfricanSwallow':
      return 40 - 2 * this.numberOfCoconuts;
    case 'NorwegianBlueParrot':
      return (this.isNailed) ? 0 : 10 + this.voltage /
10;
    default:
      return null;
    }
  }
}
```

Jetzt füge ich Unterklassen für die verschiedenen Vogelarten sowie Fabrikfunktionen zum Instanziieren der entsprechenden Unterklassen hinzu:

```
function plumage(bird) {
  return createBird(bird).plumage;
}
function airSpeedVelocity(bird) {
  return createBird(bird).airSpeedVelocity;
}

function createBird(bird) {
```

```
  switch (bird.type) {
  case 'EuropeanSwallow':
    return new EuropeanSwallow(bird);
  case 'AfricanSwallow':
    return new AfricanSwallow(bird);
  case 'NorwegianBlueParrot':
    return new NorwegianBlueParrot(bird);
  default:
   return new Bird(bird);
  }
}

class EuropeanSwallow extends Bird {
}
class AfricanSwallow extends Bird {
}
class NorwegianBlueParrot extends Bird {
}
```

Jetzt ist die erforderliche Klassenstruktur vorhanden, und ich kann die Methoden mit den Bedingungen erstellen, zunächst für das Gefieder (`plumage`). Ich überschreibe einen Zweig der `switch`-Anweisung durch eine entsprechende Unterklasse:

```
class EuropeanSwallow...
get plumage() {
  return "average";
}

class Bird...
get plumage() {
  switch (this.type) {
  case 'EuropeanSwallow':
    throw "oops";
```

```
  case 'AfricanSwallow':
    return (this.numberOfCoconuts > 2) ? "tired" :
"average";
  case 'NorwegianBlueParrot':
    return (this.voltage > 100) ? "scorched" :
"beautiful";
  default:
    return "unknown";
  }
}
```

Ich habe hier noch eine `throw`*-Anweisung hinzugefügt, weil ich paranoid bin.*

Jetzt kann ich kompilieren und testen. Wenn alles in Ordnung ist, wende ich mich dem nächsten Zweig zu:

```
class AfricanSwallow...
get plumage() {
  return (this.numberOfCoconuts > 2) ? "tired" :
"average";
}
```

Anschließend ist der norwegische Blauling an der Reihe:

```
class NorwegianBlueParrot...
get plumage() {
  return (this.voltage > 100) ? "scorched" :
"beautiful";
}
```

Den `default`-Fall überlasse ich der Basisklasse:

```
class Bird...
get plumage() {
  return "unknown";
}
```

Ich wiederhole das Ganze mit `airSpeedVelocity`. Sobald dieser Schritt erledigt ist, sieht mein Code folgendermaßen aus (ich habe zudem die Funktionen `airSpeedVelocity` und `plumage` auf oberster Ebene inline platziert):

```
function plumages(birds) {
  return new Map(birds
                 .map(b => createBird(b))
                 .map(bird => [bird.name,
bird.plumage]));
}
function speeds(birds) {
  return new Map(birds
                 .map(b => createBird(b))
                 .map(bird => [bird.name,
                      bird.airSpeedVelocity]));
}

function createBird(bird) {
  switch (bird.type) {
  case 'EuropeanSwallow':
    return new EuropeanSwallow(bird);
  case 'AfricanSwallow':
    return new AfricanSwallow(bird);
  case 'NorwegianBlueParrot':
```

```
    return new NorwegianBlueParrot(bird);
  default:
    return new Bird(bird);
  }
}

class Bird {
  constructor(birdObject) {
    Object.assign(this, birdObject);
  }
  get plumage() {
    return "unknown";
  }
  get airSpeedVelocity() {
    return null;
  }
}
class EuropeanSwallow extends Bird {
  get plumage() {
    return "average";
  }
  get airSpeedVelocity() {
    return 35;
  }
}
class AfricanSwallow extends Bird {
  get plumage() {
    return (this.numberOfCoconuts > 2) ? "tired" :
"average";
  }
  get airSpeedVelocity() {
    return 40 - 2 * this.numberOfCoconuts;
  }
}
class NorwegianBlueParrot extends Bird {
  get plumage() {
    return (this.voltage > 100) ? "scorched" :
"beautiful";
  }
```

```
  get airSpeedVelocity() {
    return (this.isNailed) ? 0 : 10 + this.voltage /
10;
  }
}
```

Wenn ich den endgültigen Code betrachte, erkenne ich, dass die Basisklasse `Bird` gar nicht unbedingt notwendig ist. In JavaScript ist für Polymorphie keine Typenhierarchie erforderlich – sofern meine Objekte die entsprechend benannten Methoden implementieren, funktioniert alles tadellos. In diesem Fall würde ich die unnötige Basisklasse allerdings behalten, denn sie hilft zu verstehen, in welcher Beziehung die Klassen zueinander stehen.

10.4.4 BEISPIEL: POLYMORPHIE FÜR VARIATIONEN VERWENDEN

Beim Vogel-Beispiel verwende ich eine übersichtliche Generalisierungshierarchie. Auf diese Weise werden die Unterklassenbildung und Polymorphie in den meisten Lehrbüchern (wie auch in diesem) dargestellt – dabei handelt es sich jedoch nicht um die einzige Art und Weise, wie Vererbung in der Praxis verwendet wird. Tatsächlich ist es sogar nicht einmal die gebräuchlichste oder beste Art und Weise. Ich kann Vererbung auch einsetzen, wenn ich darauf hinweisen möchte, dass ein Objekt einem anderen Objekt sehr ähnlich ist – von kleinen Abweichungen abgesehen.

Betrachten Sie als Beispiel hierfür den Code, den eine Ratingagentur verwendet, um eine Investitionsbewertung für Reisen auf Segelschiffen zu ermitteln. Die Agentur vergibt die Bewertungen »A« und »B«, die von verschiedenen Faktoren abhängen, die mit dem Risiko und dem potenziellen Gewinn verbunden sind. Das Risiko ist durch die Art und Länge der Reise sowie durch den Verlauf der vorherigen Seereisen eines bestimmten Kapitäns gegeben:

```
function rating(voyage, history) {
  const vpf = voyageProfitFactor(voyage, history);
  const vr = voyageRisk(voyage);
  const chr = captainHistoryRisk(voyage, history);
  if (vpf * 3 > (vr + chr * 2)) return "A";
  else return "B";
}

function voyageRisk(voyage) {
  let result = 1;
  if (voyage.length > 4) result += 2;
  if (voyage.length > 8) result += voyage.length - 8;
  if (["china", "east-indies"].includes(voyage.zone))
    result += 4;
  return Math.max(result, 0);
}

function captainHistoryRisk(voyage, history) {
  let result = 1;
  if (history.length < 5) result += 4;
  result += history.filter(v => v.profit < 0).length;
  if (voyage.zone === "china" && hasChina(history))
    result -= 2;
  return Math.max(result, 0);
}
```

```
function hasChina(history) {
  return history.some(v => "china" === v.zone);
}

function voyageProfitFactor(voyage, history) {
  let result = 2;
  if (voyage.zone === "china") result += 1;
  if (voyage.zone === "east-indies") result += 1;
  if (voyage.zone === "china" && hasChina(history)) {
    result += 3;
    if (history.length > 10) result += 1;
    if (voyage.length > 12) result += 1;
    if (voyage.length > 18) result -= 1;
  }
  else {
    if (history.length > 8) result += 1;
    if (voyage.length > 14) result -= 1;
  }
  return result;
}
```

Die Funktionen `voyageRisk` und `captainHistoryRisk` tragen Punkte zum Risiko bei, `voyageProfitFactor` sammelt Punkte zur Beurteilung des potenziellen Gewinns, und `rating` errechnet aus diesen Werten die Gesamtbeurteilung der Reise.

Der aufrufende Code könnte etwa so aussehen:

```
const voyage = {zone: "west-indies", length: 10};
const history = [
  {zone: "east-indies", profit: 5},
```

```
  {zone: "west-indies", profit: 15},
  {zone: "china",       profit: -2},
  {zone: "west-africa", profit: 7},
];

const myRating = rating(voyage, history);
```

Ich möchte mich hier auf einige Stellen im Code konzentrieren, die bedingte Logik verwenden, um den Sonderfall einer Reise nach China zu handhaben, für den Fall, dass der Kapitän schon einmal dort war:

```
function rating(voyage, history) {
  const vpf = voyageProfitFactor(voyage, history);
  const vr = voyageRisk(voyage);
  const chr = captainHistoryRisk(voyage, history);
  if (vpf * 3 > (vr + chr * 2)) return "A";
  else return "B";
}

function voyageRisk(voyage) {
  let result = 1;
  if (voyage.length > 4) result += 2;
  if (voyage.length > 8) result += voyage.length - 8;
  if (["china", "east-indies"].includes(voyage.zone))
    result += 4;
  return Math.max(result, 0);
}

function captainHistoryRisk(voyage, history) {
  let result = 1;
  if (history.length < 5) result += 4;
  result += history.filter(v => v.profit < 0).length;
  if (voyage.zone === "china" && hasChina(history))
    result -= 2;
```

```
  return Math.max(result, 0);
}

function hasChina(history) {
  return history.some(v => "china" === v.zone);
}

function voyageProfitFactor(voyage, history) {
  let result = 2;
  if (voyage.zone === "china") result += 1;
  if (voyage.zone === "east-indies") result += 1;
  if (voyage.zone === "china" && hasChina(history)) {
    result += 3;
    if (history.length > 10) result += 1;
    if (voyage.length > 12) result += 1;
    if (voyage.length > 18) result -= 1;
  }
  else {
    if (history.length > 8) result += 1;
    if (voyage.length > 14) result -= 1;
  }
  return result;
}
```

Ich verwende Vererbung und Polymorphie, um die Logik zur Handhabung dieses Sonderfalls von der Basislogik zu trennen. Dieses Refactoring erweist sich als besonders nützlich, wenn ich im Begriff bin, weitere spezielle Bedingungen für diesen Sonderfall einzurichten – und die Logik für die wiederholten Chinareisen kann es erschweren, den Basisfall zu verstehen.

Ich beginne zunächst mit einem Satz an Funktionen. Für die Polymorphie ist eine Klassenstruktur erforderlich,

deshalb wende ich *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) an. Das führt zu folgendem Code:

```
function rating(voyage, history) {
  return new Rating(voyage, history).value;
}

class Rating {
  constructor(voyage, history) {
    this.voyage = voyage;
    this.history = history;
  }
  get value() {
  const vpf = this.voyageProfitFactor;
  const vr = this.voyageRisk;
  const chr = this.captainHistoryRisk;
  if (vpf * 3 > (vr + chr * 2)) return "A";
  else return "B";
}
get voyageRisk() {
  let result = 1;
  if (this.voyage.length > 4) result += 2;
  if (this.voyage.length > 8)
    result += this.voyage.length - 8;
  if (["china", "east-
indies"].includes(this.voyage.zone))
    result += 4;
  return Math.max(result, 0);
}
get captainHistoryRisk() {
  let result = 1;
  if (this.history.length < 5) result += 4;
  result += this.history.filter(v => v.profit <
0).length;
  if (this.voyage.zone === "china" &&
this.hasChinaHistory)
    result -= 2;
```

```
  return Math.max(result, 0);
}
get voyageProfitFactor() {
  let result = 2;
  if (this.voyage.zone === "china") result += 1;
  if (this.voyage.zone === "east-indies") result += 1;
  if (this.voyage.zone === "china" &&
this.hasChinaHistory) {
    result += 3;
    if (this.history.length > 10) result += 1;
    if (this.voyage.length > 12) result += 1;
    if (this.voyage.length > 18) result -= 1;
  }
  else {
    if (this.history.length > 8) result += 1;
    if (this.voyage.length > 14) result -= 1;
  }
  return result;
}
get hasChinaHistory() {
  return this.history.some(v => "china" === v.zone);
  }
}
```

Das ist die Klasse für den Basisfall. Jetzt muss ich eine leere Unterklasse erstellen, die das alternative Verhalten implementiert:

```
class ExperiencedChinaRating extends Rating {
}
```

Anschließend erstelle ich eine Fabrikfunktion, die im Sonderfall die Variante der Klasse zurückgibt:

```
function createRating(voyage, history) {
  if (voyage.zone === "china" &&
                      history.some(v => "china" ===
v.zone))
    return new ExperiencedChinaRating(voyage,
history);
  else return new Rating(voyage, history);
}
```

Ich muss alle Aufrufer ändern, sodass sie die Fabrikfunktion verwenden, statt den Konstruktor direkt aufzurufen. In diesem Fall ist das nur die `rating`-Funktion:

```
function rating(voyage, history) {
  return createRating(voyage, history).value;
}
```

Ich muss zwei Verhaltensweisen in eine Unterklasse verlagern. Als Erstes erledige ich das für die Programmlogik in `captainHistoryRisk`:

```
class Rating...
get captainHistoryRisk() {
  let result = 1;
  if (this.history.length < 5) result += 4;
  result += this.history.filter(v => v.profit <
0).length;
  if (this.voyage.zone === "china" &&
this.hasChinaHistory)
    result -= 2;
```

```
  return Math.max(result, 0);
}
```

In der Unterklasse überschreibe ich die Methode:

```
class ExperiencedChinaRating
get captainHistoryRisk() {
  const result = super.captainHistoryRisk - 2;
  return Math.max(result, 0);
}

class Rating...
get captainHistoryRisk() {
  let result = 1;
  if (this.history.length < 5) result += 4;
  result += this.history.filter(v => v.profit <
0).length;
  if (this.voyage.zone === "china" &&
this.hasChinaHistory)
    result -= 2;
  return Math.max(result, 0);
}
```

Das unterschiedliche Verhalten in `voyageProfitFactor` zu separieren ist etwas umständlicher. Ich kann das alternative Verhalten nicht einfach entfernen und die Methode der Basisklasse aufrufen, weil es hier noch eine weitere Fallunterscheidung gibt. Ich möchte aber auch nicht die gesamte Methode aus der Basisklasse in die Unterklasse kopieren:

```
class Rating...
get voyageProfitFactor() {
  let result = 2;
  if (this.voyage.zone === "china") result += 1;
  if (this.voyage.zone === "east-indies") result += 1;
  if (this.voyage.zone === "china" &&
this.hasChinaHistory) {
    result += 3;
    if (this.history.length > 10) result += 1;
    if (this.voyage.length > 12) result += 1;
    if (this.voyage.length > 18) result -= 1;
  }
  else {
    if (this.history.length > 8) result += 1;
    if (this.voyage.length > 14) result -= 1;
  }
  return result;
}
```

Also wende ich *Funktion extrahieren* (Abschnitt 6.1) auf die gesamte Bedingung an:

```
class Rating...
get voyageProfitFactor() {
  let result = 2;
  if (this.voyage.zone === "china") result += 1;
  if (this.voyage.zone === "east-indies") result += 1;
  result += this.voyageAndHistoryLengthFactor;
  return result;
}
get voyageAndHistoryLengthFactor() {
  let result = 0;
  if (this.voyage.zone === "china" &&
this.hasChinaHistory) {
    result += 3;
      if (this.history.length > 10) result += 1;
```

```
      if (this.voyage.length > 12) result += 1;
      if (this.voyage.length > 18) result -= 1;
  }
  else {
    if (this.history.length > 8) result += 1;
    if (this.voyage.length > 14) result -= 1;
  }
  return result;
}
```

Ein Funktionsname, der »And« enthält, ist ein ziemlich übler Code-Smell, aber ich belasse es zunächst dabei und verlagere den entsprechenden Teil der Methode in die Unterklasse:

```
class Rating...
get voyageAndHistoryLengthFactor() {
  let result = 0;
  if (this.history.length > 8) result += 1;
  if (this.voyage.length > 14) result -= 1;
  return result;
}

class ExperiencedChinaRating...
get voyageAndHistoryLengthFactor() {
  let result = 0;
  result += 3;
  if (this.history.length > 10) result += 1;
  if (this.voyage.length > 12) result += 1;
  if (this.voyage.length > 18) result -= 1;
  return result;
}
```

Damit ist dieses Refactoring formal gesehen

abgeschlossen – ich habe das alternative Verhalten in eine Unterklasse verlagert. Die Logik der Basisklasse ist leichter verständlich und einfacher nutzbar, und ich muss mich mit dem Sonderfall nur befassen, wenn ich den Code der Unterklasse bearbeite, dessen einzige Aufgabe darin besteht, die Unterschiede zur Basisklasse zu verdeutlichen.

Ich habe allerdings den Eindruck, dass ich zumindest kurz umreißen sollte, wie mit der ungewöhnlichen neuen Methode zu verfahren ist. Eine Methode nur zu erstellen, damit sie von einer Unterklasse überschrieben werden kann, ist durchaus üblich, wenn man bei der Vererbung nach Basisfall-und-Varianten-Schema vorgeht. Aber eine so plumpe Methode wie diese verschleiert eher, was vor sich geht, anstatt es zu verdeutlichen.

Das »And« im Funktionsnamen verrät, dass eigentlich zwei unterschiedliche Modifikationen stattfinden – und ich halte es für sinnvoll, sie voneinander zu trennen. Ich wende also sowohl in der Basisklasse als auch in der Unterklasse *Funktion extrahieren* (Abschnitt 6.1) auf die Berechnung von `historyLengthFactor` an. Zuerst erledige ich das in der Basisklasse:

```
class Rating...
get voyageAndHistoryLengthFactor() {
  let result = 0;
  result += this.historyLengthFactor;
  if (this.voyage.length > 14) result -= 1;
```

```
  return result;
}
get historyLengthFactor() {
  return (this.history.length > 8) ? 1 : 0;
}
```

Das Gleiche wiederhole ich in der Unterklasse:

```
class ExperiencedChinaRating...
get voyageAndHistoryLengthFactor() {
  let result = 0;
  result += 3;
  result += this.historyLengthFactor;
  if (this.voyage.length > 12) result += 1;
  if (this.voyage.length > 18) result -= 1;
  return result;
}
get historyLengthFactor() {
  return (this.history.length > 10) ? 1 : 0;
}
```

Auf die Basisklasse kann ich anschließend *Anweisungen in Aufrufer verschieben* (Abschnitt 8.4) anwenden:

```
class Rating...
get voyageProfitFactor() {
  let result = 2;
  if (this.voyage.zone === "china") result += 1;
  if (this.voyage.zone === "east-indies") result += 1;
  result += this.historyLengthFactor;
  result += this.voyageAndHistoryLengthFactor;
  return result;
}
```

```
get voyageAndHistoryLengthFactor() {
  let result = 0;
  result += this.historyLengthFactor;
  if (this.voyage.length > 14) result -= 1; return
result;
}

class ExperiencedChinaRating...
get voyageAndHistoryLengthFactor() {
  let result = 0;
  result += 3;
  result += this.historyLengthFactor;
  if (this.voyage.length > 12) result += 1;
  if (this.voyage.length > 18) result -= 1;
  return result;
}
```

Anschließend verwende ich *Funktion umbenennen* (Abschnitt 6.5).

```
class Rating...
get voyageProfitFactor() {
  let result = 2;
  if (this.voyage.zone === "china") result += 1;
  if (this.voyage.zone === "east-indies") result += 1;
  result += this.historyLengthFactor;
  result += this.voyageLengthFactor;
  return result;
}

get voyageLengthFactor() {
  return (this.voyage.length > 14) ? - 1: 0;
}
```

Zur Vereinfachung tausche ich `voyageLengthFactor` *durch einen ternären Operator aus.*

```
class ExperiencedChinaRating...
  get voyageLengthFactor() {
  let result = 0;
  result += 3;
  if (this.voyage.length > 12) result += 1;
  if (this.voyage.length > 18) result -= 1;
  return result;
}
```

Eine letzte Anmerkung: Ich halte es nicht für sinnvoll, bei der Berechnung des Faktors für die Länge der Reise (`voyageLengthFactor`) drei Punkte zu addieren – sie sollten besser zum Gesamtergebnis hinzuaddiert werden:

```
class ExperiencedChinaRating...
get voyageProfitFactor() {
  return super.voyageProfitFactor + 3;
}
get voyageLengthFactor() {
  let result = 0;
  result += 3;
  if (this.voyage.length > 12) result += 1;
  if (this.voyage.length > 18) result -= 1;
  return result;
}
```

Nach Abschluss des Refactorings ergibt sich der nachstehende Code. Am Anfang steht die `Rating`-Klasse

für den Basisfall, die keine der Komplikationen berücksichtigen muss, die sich im Fall von Kapitänen mit Chinaerfahrung ergeben:

```
class Rating {
  constructor(voyage, history) {
    this.voyage = voyage;
    this.history = history;
  }
  get value() {
    const vpf = this.voyageProfitFactor;
    const vr = this.voyageRisk;
    const chr = this.captainHistoryRisk;
    if (vpf * 3 > (vr + chr * 2)) return "A";
    else return "B";
  }
  get voyageRisk() {
    let result = 1;
    if (this.voyage.length > 4) result += 2;
    if (this.voyage.length > 8)
      result += this.voyage.length - 8;
    if (["china", "east-
indies"].includes(this.voyage.zone))
      result += 4;
    return Math.max(result, 0);
  }
  get captainHistoryRisk() {
  let result = 1;
  if (this.history.length < 5) result += 4;
  result += this.history.filter(v => v.profit <
0).length;
  return Math.max(result, 0);
  }
  get voyageProfitFactor() {
    let result = 2;
    if (this.voyage.zone === "china") result += 1;
    if (this.voyage.zone === "east-indies") result +=
```

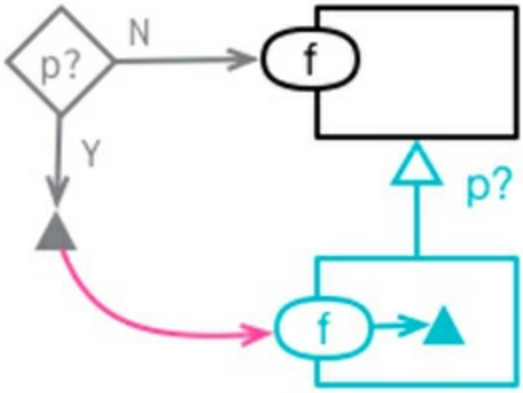
```
1;
    result += this.historyLengthFactor;
    result += this.voyageLengthFactor;
    return result;
  }
  get voyageLengthFactor() {
    return (this.voyage.length > 14) ? - 1: 0;
  }
  get historyLengthFactor() {
    return (this.history.length > 8) ? 1 : 0;
  }
}
```

Der Code für den Fall von Kapitänen mit Chinaerfahrung liest sich wie eine Aufstellung der vom Basisfall abweichenden Varianten:

```
class ExperiencedChinaRating extends Rating {
  get captainHistoryRisk() {
    const result = super.captainHistoryRisk - 2;
    return Math.max(result, 0);
  }
  get voyageLengthFactor() {
  let result = 0;
  if (this.voyage.length > 12) result += 1;
  if (this.voyage.length > 18) result -= 1;
  return result;
  }
  get historyLengthFactor() {
    return (this.history.length > 10) ? 1 : 0;
  }
  get voyageProfitFactor() {
    return super.voyageProfitFactor + 3;
  }
}
```

10.5 Sonderfall einführen (Introduce Special Case)

Ehemalige Bezeichnung: *Null-Objekt einführen* (Introduce Null Object)




```
if (aCustomer === "unknown") customerName =
"occupant";
```

```
class UnknownCustomer {
get name() {return "occupant";}
```

10.5.1 MOTIVATION

Redundanter Code entsteht häufig dann, wenn viele Clients auf einen bestimmten Wert einer Datenstruktur zugreifen und ihn anschließend für dieselbe Aufgabe verwenden. Wenn ich feststelle, dass viele Teile der

Codebasis auf einen bestimmten Wert mit derselben Aktion reagieren, möchte ich diese Reaktion an nur einer Stelle unterbringen.

Das Pattern *Special Case* (Sonderfall) ist hierfür ein gut geeigneter Mechanismus. Ich erstelle ein Sonderfall-Element zur Erfassung des gesamten gemeinsamen Verhaltens, das mir ermöglicht, die meisten Sonderfallprüfungen durch einfache Aufrufe zu ersetzen.

Ein Sonderfall kann sich auf verschiedene Weise zeigen. Wenn ein Objekt nur zum Lesen von Daten verwendet wird, kann ich ein einfaches Objekt bereitstellen, das die benötigten Werte als Literale vorhält. Wenn ich nicht nur einfache Werte, sondern auch Verhalten benötige, kann ich ein spezielles Objekt mit Methoden für das gängige Verhalten erstellen. Dieses Sonderfall-Objekt kann von einer kapselnden Klasse zurückgegeben oder mithilfe einer Transformation einer Datenstruktur hinzugefügt werden.

Ein typischer Wert, der eine Behandlung als Sonderfall erfordert, ist `null`. Aus diesem Grund wird das Pattern mitunter auch als Nullobjekt-Muster (Null Object pattern) bezeichnet. Der Ansatz ist aber bei allen Sonderfällen derselbe. Ich betone an dieser Stelle gern, dass das Nullobjekt-Muster ein Sonderfall des Sonderfall-Musters ist.

10.5.2 VORGEHEN

Zunächst einmal benötigen wir eine Container-Datenstruktur (oder eine Klasse), die eine Eigenschaft enthält, die Gegenstand dieses Refactorings ist. Die Clients des Containers vergleichen diese Eigenschaft mit einem speziellen Wert. Wir möchten diesen speziellen Wert durch eine Sonderfall-Klasse oder Datenstruktur ersetzen:

- Fügen Sie der Klasse eine Eigenschaft für die Sonderfallprüfung hinzu, die `false` zurückgibt.
- Erstellen Sie ein Sonderfall-Objekt, das nur diejenige Eigenschaft für die Sonderfallprüfung besitzt, die `true` zurückgibt.
- Wenden Sie *Funktion extrahieren* (Abschnitt 6.1) auf die Prüflogik für den Sonderfall an. Vergewissern Sie sich, dass alle Clients die neue Funktion verwenden und keinen direkten Vergleich anstellen.
- Verwenden Sie das Sonderfall-Objekt, indem Sie es entweder bei einem Funktionsaufruf zurückgeben oder eine Transformationsfunktion anwenden.
- Sorgen Sie dafür, dass die Funktion zum Vergleich mit dem Sonderfall die Eigenschaft für die Sonderfallprüfung verwendet.
- Testen Sie.
- Verwenden Sie *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) oder *Funktionen zu einer Transformation vereinen* (Abschnitt 6.10), um das gesamte Verhalten für den Sonderfall in das neue Element zu verlagern.

  Die Sonderfall-Klasse gibt in den meisten Fällen auf einfache Anfragen feste Werte zurück. In diesem Fall könnte der Sonderfall ein Datensatz sein, der Literale enthält.
- Wenden Sie dort, wo Sie auf die Funktion zum Vergleich mit dem Sonderfall nicht verzichten können, *Funktion inline platzieren* (Abschnitt 6.2) an.

10.5.3 BEISPIEL

Ein Versorgungsbetrieb bietet seine Dienste an verschiedenen Standorten (`Site`) an:

```
class Site...
get customer() {return this._customer;}
```

Die Kunden-Klasse besitzt verschiedene Eigenschaften, von denen ich drei betrachten werde:

```
class Customer...
get name() {...}
get billingPlan() {...}
set billingPlan(arg) {...}
get paymentHistory() {...}
```

Meistens ist einem Standort ein Kunde zugeordnet, manchmal aber auch nicht. Vielleicht ist jemand weggezogen, und ich weiß noch nicht, ob schon wieder jemand eingezogen ist. In diesem Fall wird dem Feld für den Kunden der String `unknown` (unbekannt) zugewiesen. Clients der Klasse `Site` müssen deshalb in der Lage sein, mit einem unbekannten Kunden zurechtzukommen. Hier sind einige Codeabschnitte als Beispiel:

```
Client 1...
const aCustomer = site.customer;
```

```
// ... jede Menge weiterer Code ...
let customerName;
if (aCustomer === "unknown") customerName =
"occupant";
else customerName = aCustomer.name;

Client 2...
const plan = (aCustomer === "unknown") ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
if (aCustomer !== "unknown") aCustomer.billingPlan =
newPlan;

Client 4...
const weeksDelinquent = (aCustomer === "unknown") ?
      0
      :
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Beim Betrachten der Codebasis fällt mir auf, dass viele Clients des `Site`-Objekts einen unbekannten Kunden handhaben müssen. Die meisten verhalten sich identisch: Sie vergeben den Namen `occupant` (Bewohner), weisen einen Basistarif als Abrechnungsmethode (`billingPlan`) zu und stufen den Kunden als pünktlichen Zahler ein, der die Zahlungsfrist um 0 Wochen (`weeksDelinquent`) überschreitet. Diese weitverbreiteten Tests auf einen Sonderfall und die gleichartige Reaktion darauf signalisieren mir, dass hier das Refactoring *Sonderfall einführen* angebracht ist.

Als Erstes füge ich der Kunden-Klasse eine Methode hinzu, die angibt, ob es sich um einen unbekannten Kunden handelt:

```
class Customer...
get isUnknown() {return false;}
```

Anschließend füge ich die Klasse `UnknownCustomer` hinzu:

```
class UnknownCustomer {
  get isUnknown() {return true;}
}
```

Beachten Sie, dass ich `UnknownCustomer` nicht zu einer Unterklasse von `Customer` mache. In anderen Programmiersprachen, insbesondere in statisch typisierten, würde ich so vorgehen, aber aufgrund der Regeln für die Unterklassenbildung in JavaScript und der dynamischen Typisierung ist es besser, hier darauf zu verzichten.

Jetzt kommt der hakelige Teil. Wenn ein unbekannter Kunde zu erwarten ist, muss ich das neue Sonderfall-Objekt zurückgeben und dafür sorgen, dass alle Tests auf einen unbekannten Kunden die neue `isUnknown`-Methode verwenden. Im Allgemeinen möchte ich die Dinge so umstrukturieren, dass ich nacheinander kleine

Änderungen vornehmen und anschließend testen kann. Wenn ich jedoch die Klasse `Customer` so ändere, dass sie anstatt `unknown` ein `UnknownCustomer`-Objekt zurückgibt, muss ich jeden Client, der auf `unknown` testet, dazu bringen, stattdessen `isUnknown` aufrufen – und das muss ich alles in einem Arbeitsschritt erledigen. Ich halte das für so reizvoll wie rohe Leber zu essen (also so überhaupt nicht).

Es gibt allerdings eine gängige Technik, die ich anwenden kann, wenn ich zu einem solchen Vorgehen gezwungen bin. Ich wende *Funktion extrahieren* (Abschnitt 6.1) auf den Code an, den ich an vielen Stellen ändern muss – in diesem Fall also auf den Code, der den Vergleich mit dem Sonderfall anstellt:

```
function isUnknown(arg) {
if (!((arg instanceof Customer) || (arg ===
"unknown")))
  throw new Error(`Ungültiger Wert: <${arg}>`);
return (arg === "unknown");
}
```

An dieser Stelle fange ich unerwartete Werte ab. Das hilft mir dabei, Fehler oder seltsames Verhalten aufzuspüren, wenn ich das Refactoring durchführe.

Jetzt kann ich diese Funktion verwenden, wann immer ich auf einen unbekannten Kunden prüfen möchte. Die Aufrufe der Funktion kann ich der Reihe nach anpassen

und jeweils Tests durchführen:

```
Client 1...
let customerName;
if (isUnknown(aCustomer)) customerName = "occupant";
else customerName = aCustomer.name;
```

Nach einer Weile habe ich sie alle ersetzt:

```
Client 2...
const plan = (isUnknown(aCustomer)) ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
if (!isUnknown(aCustomer)) aCustomer.billingPlan =
newPlan;

Client 4...
const weeksDelinquent = isUnknown(aCustomer) ?
      0
      :
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Nachdem ich dafür Sorge getragen habe, dass alle Aufrufer `isUnknown` verwenden, kann ich die Klasse `Site` so ändern, dass sie ein `UnknownCustomer`-Objekt zurückgibt:

```
class Site...
get customer() {
```

```
  return (this._customer === "unknown") ?
          new UnknownCustomer() : this._customer;
}
```

Dass der String `unknown` nicht mehr verwendet wird, kann ich überprüfen, indem ich die Funktion `isUnknown` dahingehend ändere, dass der tatsächliche boolsche Wert von `unknown` verwendet wird:

```
Client 1...
function isUnknown(arg) {
  if (!(arg instanceof Customer ||
        arg instanceof UnknownCustomer))
    throw new Error(`Ungültiger Wert: <${arg}>`);
  return arg.isUnknown;
}
```

Ich teste, um zu gewährleisten, dass alles funktioniert.

Jetzt beginnt der eigentliche Spaß. Ich kann *Funktionen zu einer Klasse vereinen* (Abschnitt 6.9) verwenden, um mir die Sonderfallprüfungen der Clients anzusehen und zu untersuchen, ob ich sie durch einen üblicherweise erwarteten Wert ersetzen kann. Momentan verwenden mehrere Clients für den Namen eines unbekannten Kunden »occupant«, so wie hier:

```
Client 1...
let customerName;
if (isUnknown(aCustomer)) customerName = "occupant";
```

```
else customerName = aCustomer.name;
```

Ich füge der Klasse `UnknownCustomer` eine entsprechende Methode hinzu:

```
class UnknownCustomer...
get name() {return "occupant";}
```

Jetzt kann ich den bedingten Code entfernen:

```
Client 1...
const customerName = aCustomer.name;
```

Nachdem ich getestet und sichergestellt habe, dass alles funktioniert, könnte ich wahrscheinlich auch noch *Variable inline platzieren* (Abschnitt 6.4) auf die Variable anwenden.

Als Nächstes ist die Eigenschaft für die Abrechnungsmethode (`billingPlan`) dran:

```
Client 2...
const plan = (isUnknown(aCustomer)) ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
if (!isUnknown(aCustomer)) aCustomer.billingPlan =
newPlan;
```

Beim Leseverhalten verfahre ich wie mit dem Namen – ich gebe die herkömmliche Antwort zurück. Beim Schreibverhalten ist es derzeit so, dass der Code bei einem unbekannten Kunden den Setter nicht aufruft. Wenn der Sonderfall eintritt, rufe ich ihn zwar auf, er tut aber nichts:

```
class UnknownCustomer...
get billingPlan() {return
registry.billingPlans.basic;}
set billingPlan(arg) { /* keine Funktion */ }

Lesender Client...
const plan = aCustomer.billingPlan;

Schreibender Client...
aCustomer.billingPlan = newPlan;
```

Sonderfall-Objekte sind Value Objects und sollten daher immer unveränderlich sein, selbst wenn die Objekte, die sie ersetzen, es nicht sind.

Der letzte Fall ist etwas komplizierter, weil im Sonderfall ein anderes Objekt zurückgegeben werden muss, das seine eigenen Eigenschaften besitzt:

```
Client...
const weeksDelinquent = isUnknown(aCustomer) ?
      0
      :
```

```
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Die allgemeine Regel für ein Sonderfall-Objekt lautet: Wenn es zugehörige Objekte zurückgeben muss, sind diese für gewöhnlich ihrerseits Sonderfälle. Hier muss ich also eine Null-Zahlungs-Historie (`NullPaymentHistory`) zurückgeben, die für die Wochen säumiger Zahlungen null zurückliefert:

```
class UnknownCustomer...
get paymentHistory() {return new
NullPaymentHistory();}

class NullPaymentHistory...
get weeksDelinquentInLastYear() {return 0;}

Client...
const weeksDelinquent =

aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Ich fahre fort, mir alle Clients anzusehen und zu prüfen, ob ich sie durch polymorphes Verhalten ersetzen kann. Es gibt aber auch Ausnahmen, nämlich Clients, die im Sonderfall etwas anderes bewirken wollen. Beispielsweise könnten 23 Clients für einen unbekannten Kunden den Namen »occupant« verwenden, aber es gibt immer auch einen, der etwas anderes benötigt.

```
Client...
```

```
const name = ! isUnknown(aCustomer) ?
               aCustomer.name : "unknown occupant";
```

In diesem Fall muss ich eine Sonderfallprüfung beibehalten. Ich werde sie ändern, sodass sie die `Customer`-Methode verwendet, indem ich im Wesentlichen *Funktion inline platzieren* (Abschnitt 6.2) auf `isUnknown` anwende:

```
Client...
const name = aCustomer.isUnknown ?
             "unknown occupant" : aCustomer.name;
```

Nachdem ich dies für alle Clients geprüft habe, sollte es mir möglich sein, *Toten Code entfernen* (Abschnitt 8.9) auf die globale `isPresent`-Funktion anzuwenden, weil sie nicht mehr aufgerufen werden sollte.

10.5.4 BEISPIEL: VERWENDUNG EINES OBJEKTS MIT LITERALEN

Es bedeutet ziemlich viel Arbeit, eine Klasse auf diese Weise zu erstellen, schließlich handelt es sich ja nur um einen einfachen Wert. Aber im Beispiel musste ich eine Klasse erstellen, weil die Kundendaten aktualisiert werden könnten. Wenn ich jedoch nur lesend auf die Datenstruktur zugreife, kann ich ein Objekt verwenden, das Literale speichert.

Hier ist noch einmal der eingangs betrachtete Fall, allerdings existiert jetzt kein Client mehr, der die Kundendaten aktualisiert:

```
class Site...
get customer() {return this._customer;}

class Customer...
get name() {...}
get billingPlan() {...}
set billingPlan(arg) {...}
get paymentHistory() {...}

Client 1...
const aCustomer = site.customer;
// ... jede Menge weiterer Code ...
let customerName;
if (aCustomer === "unknown") customerName =
"occupant";
else customerName = aCustomer.name;

Client 2...
const plan = (aCustomer === "unknown") ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
const weeksDelinquent = (aCustomer === "unknown") ?
      0
      :
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Wie im vorhergehenden Fall füge ich als Erstes der `Customer`-Klasse eine Eigenschaft `isUnknown` hinzu und erstelle ein Sonderfall-Objekt mit diesem Feld. Der

Unterschied besteht darin, dass der Sonderfall in diesem Fall ein Literal ist:

```
class Customer...
get isUnknown() {return false;}

Oberste Ebene...
function createUnknownCustomer() {
  return {
    isUnknown: true,
  };
}
```

Ich wende *Funktion extrahieren* (Abschnitt 6.1) auf die Bedingung zur Prüfung des Sonderfalls an:

```
function isUnknown(arg) {
  return (arg === "unknown");
}

Client 1...
let customerName;
if (isUnknown(aCustomer)) customerName = "occupant";
else customerName = aCustomer.name;

Client 2...
const plan = isUnknown(aCustomer) ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
const weeksDelinquent = isUnknown(aCustomer) ?
      0
      :
```

```
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Ich ändere die `Site`-Klasse und die zu prüfende Bedingung, sodass beide den Sonderfall verwenden:

```
class Site...
get customer() {
  return (this._customer === "unknown") ?
          createUnknownCustomer() : this._customer;
}

Oberste Ebene...
function isUnknown(arg) {
  return arg.isUnknown;
}
```

Jetzt ersetze ich die Standardrückgabe durch das entsprechende Literal. Ich fange mit dem Namen an:

```
function createUnknownCustomer() {
  return {
    isUnknown: true,
    name: "occupant",
  };
}

Client 1...
const customerName = aCustomer.name;
```

Nun kommt die Abrechnungsmethode:

```
function createUnknownCustomer() {
  return {
    isUnknown: true,
    name: "occupant",
    billingPlan: registry.billingPlans.basic,
  };
}

Client 2...
const plan = aCustomer.billingPlan;
```

Auf ähnliche Weise kann ich das Literal verwenden, um dem Wert der Wochen säumiger Zahlungen null zuzuweisen:

```
function createUnknownCustomer() {
  return {
    isUnknown: true,
    name: "occupant",
    billingPlan: registry.billingPlans.basic,
    paymentHistory: {
      weeksDelinquentInLastYear: 0,
    },
  };
}

Client 3...
const weeksDelinquent =

aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Wenn ich Literale auf diese Weise verwende, sollte ich sie unveränderlich machen, beispielsweise durch

`freeze`. Meistens verwende ich aber lieber eine Klasse.

10.5.5 BEISPIEL: VERWENDEN EINER TRANSFORMATION

In den beiden vorhergehenden Fällen wurden Klassen verwendet. Das Verfahren ist jedoch auch auf Datensätze anwendbar, wenn man eine Transformation vornimmt.

Nehmen wir an, die Eingabe ist eine einfache Datensatzstruktur, die folgendermaßen aussieht:

```
{
  name: "Acme Boston",
  location: "Malden MA",
  // weitere Standortdaten
  customer: {
    name: "Acme Industries",
    billingPlan: "plan-451",
    paymentHistory: {
      weeksDelinquentInLastYear: 7
      // weitere Einträge
    },
    // weitere Einträge
  }
}
```

In manchen Fällen ist der Kunde unbekannt. Die Beschreibung erfolgt auf die gleiche Weise:

```
{
  name: "Warehouse Unit 15",
```

```
  location: "Malden MA",
  // weitere Standortdaten
  customer: "unknown",
}
```

Der Client-Code prüft auf ähnliche Weise, ob der Kunde unbekannt ist:

```
Client 1...
const site = acquireSiteData();
const aCustomer = site.customer;
// ... jede Menge weiterer Code ...
let customerName;
if (aCustomer === "unknown") customerName =
"occupant";
else customerName = aCustomer.name;

Client 2...
const plan = (aCustomer === "unknown") ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
const weeksDelinquent = (aCustomer === "unknown") ?
      0
      :
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Als Erstes unterziehe ich die `rawSite`-Datenstruktur einer Transformation, die aktuell lediglich eine tiefe Kopie erstellt:

```
Client 1...
```

```
const rawSite = acquireSiteData();
const site = enrichSite(rawSite);
const aCustomer = site.customer;
// ... jede Menge weiterer Code ...
let customerName;
if (aCustomer === "unknown") customerName =
"occupant";
else customerName = aCustomer.name;

function enrichSite(inputSite) {
  return _.cloneDeep(inputSite);
}
```

Anschließend wende ich *Funktion extrahieren* (Abschnitt 6.1) an, um auf einen unbekannten Kunden zu prüfen:

```
function isUnknown(aCustomer) {
  return aCustomer === "unknown";
}

Client 1...
const rawSite = acquireSiteData();
const site = enrichSite(rawSite);
const aCustomer = site.customer;
// ... jede Menge weiterer Code ...
let customerName;
if (isUnknown(aCustomer)) customerName = "occupant";
else customerName = aCustomer.name;

Client 2...
const plan = (isUnknown(aCustomer)) ?
      registry.billingPlans.basic
      : aCustomer.billingPlan;

Client 3...
```

```
const weeksDelinquent = (isUnknown(aCustomer)) ?
      0
      :
aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

Ich beginne die Erweiterung (`enrichSite`) damit, dass ich der `Customer`-Klasse eine `isUnknown`-Eigenschaft hinzufüge:

```
function enrichSite(aSite) {
  const result = _.cloneDeep(aSite);
  const unknownCustomer = {
    isUnknown: true,
  };

  if (isUnknown(result.customer)) result.customer =

unknownCustomer;
  else result.customer.isUnknown = false;
  return result;
}
```

Anschließend kann ich dann die Prüfung für den Sonderfall um die Überprüfung der neuen Eigenschaft erweitern. Ich behalte aber auch die ursprüngliche Prüfung bei, sodass die Überprüfung sowohl mit den ursprünglichen als auch mit den erweiterten Standortdaten funktioniert.

```
function isUnknown(aCustomer) {
  if (aCustomer === "unknown") return true;
```

```
  else return aCustomer.isUnknown;
}
```

Ich teste, um zu gewährleisten, dass alles funktioniert, und wende dann *Funktionen zu einer Transformation vereinen* (Abschnitt 6.10) auf den Sonderfall an. Als Erstes verschiebe ich die Namenszuweisung in die Ergänzungsfunktion:

```
function enrichSite(aSite) {
  const result = _.cloneDeep(aSite);
  const unknownCustomer = {
    isUnknown: true,
    name: "occupant",
  };

  if (isUnknown(result.customer)) result.customer =

unknownCustomer;
  else result.customer.isUnknown = false;
  return result;
}

Client 1...
const rawSite = acquireSiteData();
const site = enrichSite(rawSite);
const aCustomer = site.customer;
// ... jede Menge weiterer Code ...
const customerName = aCustomer.name;
```

Ich teste und kümmere mich anschließend um die Abrechnungsmethode:

```
function enrichSite(aSite) {
  const result = _.cloneDeep(aSite);
  const unknownCustomer = {
    isUnknown: true,
    name: "occupant",
    billingPlan: registry.billingPlans.basic,
  };

  if (isUnknown(result.customer)) result.customer =

unknownCustomer;
  else result.customer.isUnknown = false;
  return result;
}

Client 2...
const plan = aCustomer.billingPlan;
```

Ich teste erneut und wende mich anschließend dem letzten Client zu:

```
function enrichSite(aSite) {
  const result = _.cloneDeep(aSite);
  const unknownCustomer = {
    isUnknown: true,
    name: "occupant",
    billingPlan: registry.billingPlans.basic,
    paymentHistory: {
      weeksDelinquentInLastYear: 0,
    }
  };

  if (isUnknown(result.customer)) result.customer =

unknownCustomer;
  else result.customer.isUnknown = false;
```

```
  return result;
}

Client 3...
const weeksDelinquent =

aCustomer.paymentHistory.weeksDelinquentInLastYear;
```

10.6 Assertion einführen (Introduce Assertion)

```
if (this.discountRate)
  base = base - (this.discountRate * base);
```

```
assert(this.discountRate >= 0);
if (this.discountRate)
  base = base - (this.discountRate * base);
```

10.6.1 MOTIVATION

Häufig funktionieren Teile des Codes nur dann, wenn

bestimmte Bedingungen erfüllt sind. Ein einfaches Beispiel hierfür wäre die Berechnung einer Quadratwurzel, die nur mit einem positiven Eingangswert funktioniert. Bei einem Objekt könnte es erforderlich sein, dass in einer Reihe von Feldern mindestens eins einen Wert enthält.

Solche Annahmen werden meistens nicht dokumentiert, sondern lassen sich nur herleiten, indem man den Ablauf eines Algorithmus nachverfolgt. Manchmal werden die Annahmen in einem Kommentar vermerkt. Eine bessere Technik, die Annahmen explizit zu machen, besteht darin, eine Assertion einzuführen.

Eine Assertion ist eine Bedingung, die immer als wahr angenommen wird. Das Fehlschlagen einer Assertion deutet auf einen Programmierfehler hin. Assertions sollten keinesfalls von anderen Teilen des Systems geprüft werden. Sie sollten so eingesetzt werden, dass das Programm auch dann gleichermaßen korrekt funktioniert, wenn sie nicht vorhanden sind. Tatsächlich bieten manche Programmiersprachen sogar Assertions, die sich durch ein Compiler-Flag deaktivieren lassen.

Ich höre oft, dass man Assertions verwenden sollte, um Fehler zu finden. Das ist zweifelsohne eine gute Idee, sollte aber nicht der einzige Grund für ihre Verwendung sein. Für mich stellen Assertions eine hilfreiche Form der Kommunikation dar – sie vermitteln dem Leser Informationen über den angenommenen Zustand, in

dem sich das Programm zum Zeitpunkt der Ausführung befinden sollte. Außerdem sind Assertions praktisch beim Debugging, und aufgrund ihrer Aussagekraft neige ich dazu, sie im Code zu belassen, nachdem ich den gesuchten Fehler behoben habe. Selbsttestender Code begrenzt ihren Nutzen beim Debugging wiederum, weil zunehmend eingrenzende Unittests meist zielführender sind, aber zu Kommunikationszwecken sind Assertions wie gesagt dennoch gut geeignet.

10.6.2 VORGEHEN

- Führen Sie eine Assertion ein, wenn Sie feststellen, dass eine Bedingung als wahr angenommen wird, um diese Annahme zu dokumentieren.

Assertions sollten keinen Einfluss auf den Ablauf eines Programms nehmen. Das Hinzufügen einer Assertion sollte also nichts am sichtbaren Verhalten ändern.

10.6.3 BEISPIEL

In diesem Beispiel geht es um Rabatte. Einem Kunden kann ein Preisnachlass gewährt werden, der auf alle seine Einkäufe angewendet wird:

```
class Customer...
applyDiscount(aNumber) {
  return (this.discountRate)
    ? aNumber - (this.discountRate * aNumber)
    : aNumber;
}
```

Hier wird angenommen, dass der Rabattsatz (`discountRate`) eine positive Zahl ist. Ich kann diese Voraussetzung explizit machen, indem ich eine Assertion verwende. In einem ternären Ausdruck kann ich allerdings nicht ohne Weiteres eine Assertion einfügen, deshalb formuliere ich ihn zunächst als `if-then`-Anweisung um:

```
class Customer...
applyDiscount(aNumber) {
  if (!this.discountRate) return aNumber;
  else return aNumber - (this.discountRate * aNumber);
}
```

Jetzt kann ich die Assertion problemlos hinzufügen:

```
class Customer...
applyDiscount(aNumber) {
  if (!this.discountRate) return aNumber;
  else {
    assert(this.discountRate >= 0);
    return aNumber - (this.discountRate * aNumber);
  }
}
```

In diesem Fall würde ich die Assertion lieber in der Setter-Methode platzieren. Wenn die Assertion in `applyDiscount` fehlschlägt, frage ich mich als Erstes,

wie der Rabattsatz überhaupt einen negativen Wert annehmen konnte:

```
class Customer...
set discountRate(aNumber) {
  assert(null === aNumber || aNumber >= 0);
  this._discountRate = aNumber;
}
```

Eine solche Assertion kann besonders dann wertvoll sein, wenn es schwierig ist, die Fehlerquelle aufzuspüren – dabei könnte es sich um ein fehlerhaftes Minuszeichen in den Eingabedaten oder irgendwo im Code handeln.

Es besteht allerdings auch die reale Gefahr der übermäßigen Verwendung von Assertions. Ich verwende keine Assertions, um jedwede Bedingungen zu überprüfen, von denen ich denke, dass sie wahr sein sollten, sondern nur, um Bedingungen zu prüfen, die wahr sein *müssen*. Redundanzen sind an dieser Stelle besonders hinderlich, weil an Bedingungen dieser Art häufig herumgedoktert wird. Deshalb halte ich es für wichtig, jede Redundanz in diesen Bedingungen zu entfernen, für gewöhnlich durch ausgiebigen Gebrauch von *Funktion extrahieren* (Abschnitt 6.1).

Ich verwende Assertions grundsätzlich nur in Situationen, die auf Programmiererfehler zurückzuführen sind. Beim Einlesen von Daten, die einer externen Quelle entstammen, sollte jede Überprüfung

der Werte primärer Bestandteil des eigentlichen Programms sein und nicht durch Assertions erfolgen – es sei denn, die externe Quelle ist wirklich vertrauenswürdig. Assertions sind eine »Ultima Ratio«, um Bugs dingfest zu machen – obwohl ich sie paradoxerweise nur dann verwende, wenn ich davon ausgehe, dass sie niemals fehlschlagen werden.

Kapitel 11

Refactoring von APIs

Module und ihre Funktionen sind die Bausteine unserer Software. APIs sind die Verbindungsstellen, die wir verwenden, um sie zu zusammenzufügen. Leicht verständliche und anwendbare APIs zu erstellen ist deshalb wichtig, aber auch nicht ganz einfach. Während ich lerne, wie ich sie verbessern kann, muss ich Refactorings vornehmen.

Eine gute API trennt alle Funktionen, die Daten aktualisieren, klar von denen, die nur Daten lesen. Wenn das nicht der Fall ist, verwende ich *Abfrage von Veränderung trennen* (Abschnitt 11.1), um beide Aspekte voneinander zu trennen. Funktionen, die sich nur hinsichtlich eines Wertes unterscheiden, kann ich mit *Funktion parametrisieren* (Abschnitt 11.2) vereinheitlichen. Manche Parameter sind allerdings eher ein Kennzeichen für ein vollkommen anderes Verhalten und werden am besten mit *Steuerungs-Flag entfernen* (Abschnitt 11.3) eliminiert.

Bei der Übergabe an Funktionen werden Datenstrukturen häufig unnötigerweise in ihre Bestandteile zerlegt; ich ziehe es vor, sie mit Hilfe von

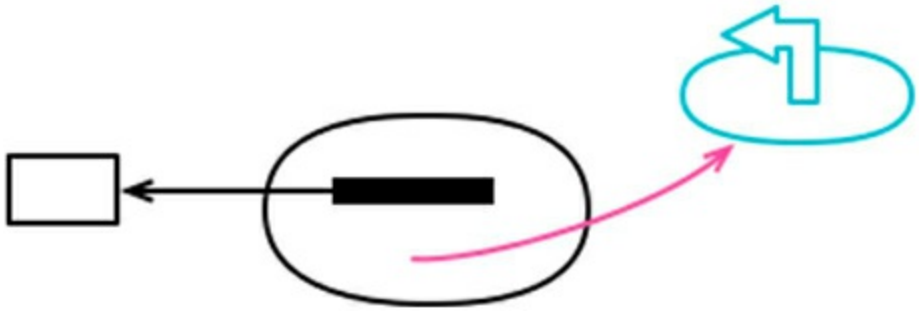
Vollständiges Objekt erhalten (Abschnitt 11.4) zusammenzuhalten. Die Entscheidungen, welche Werte als Parameter übergeben und welche Werte besser von der aufgerufenen Funktion selbst ermittelt werden, muss ich regelmäßig überdenken. Dabei verwende ich häufig *Parameter durch Abfrage ersetzen* (Abschnitt 11.5) und *Abfrage durch Parameter ersetzen* (Abschnitt 11.6).

Eine Klasse ist eine Form von Modul. Ich verwende bevorzugt Objekte, die möglichst unveränderlich sind, deshalb nutze ich bei jeder sich bietenden Gelegenheit *Setter entfernen* (Abschnitt 11.7). Wenn ein Aufrufer ein neues Objekt erwartet, bietet ein einfacher Konstruktor oft nicht genug Flexibilität. Ich kann diese aber durch Verwendung von *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8) erzielen.

Die letzten beiden Refactorings befassen sich mit der Schwierigkeit, eine besonders komplexe Funktion, die viele Daten weitergibt, zu unterteilen. Mit *Funktion durch Befehl ersetzen* (Abschnitt 11.9) kann ich eine solche Funktion in ein Objekt umwandeln, was es erleichtert, *Funktion extrahieren* (Abschnitt 6.1) auf den Rumpf der Funktion anzuwenden. Wenn ich die Funktion später vereinfache und das Befehlsobjekt nicht mehr benötige, wandle ich es mit *Befehl durch Funktion ersetzen* (Abschnitt 11.10) wieder in eine Funktion um.

11.1 Abfrage von Veränderung trennen

(Separate Query from Modifier)



```
function getTotalOutstandingAndSendBill() {
  const result = customer.invoices.reduce((total,
each) =>
                 each.amount + total, 0);
  sendBill();
  return result;
}
```

```
function totalOutstanding() {
  return customer.invoices.reduce((total, each) =>
         each.amount + total, 0);
}
function sendBill() {
  emailGateway.send(formatBill(customer));
}
```

11.1.1 MOTIVATION

Wenn eine Funktion einen Wert liefert und keine

sichtbaren Seiteneffekte hat, ist das schon eine Menge wert. Ich kann die Funktion nach Belieben aufrufen, ihren Aufruf in eine aufrufende Funktion verschieben, und sie lässt sich leichter testen. Kurz gefasst: Ich muss mir um einiges weniger Gedanken machen.

Es ist wichtig, deutlich auf den Unterschied zwischen Funktionen mit und ohne Seiteneffekten hinzuweisen. Als Faustregel gilt, dass eine Funktion, die einen Wert zurückgibt, keine sichtbaren Seiteneffekte haben sollte – die Trennung von Befehl und Abfrage (Command-Query-Separation) (`https://martinfowler.com/bliki/CommandQuerySeparation.html`). Für manche Entwickler ist das eine eiserne Regel. Ich vertrete hier keinen so puristischen Standpunkt (wie bei allem), versuche aber im Allgemeinen, mich daran zu halten, und bin damit gut gefahren.

Wenn ich auf eine Methode stoße, die einen Wert zurückgibt, aber sichtbare Seiteneffekte hat, versuche ich stets, die Abfrage von der Veränderung zu trennen.

Beachten Sie, dass ich den Ausdruck *sichtbare* Seiteneffekte verwende. Eine gängige Optimierung sieht vor, das Ergebnis einer Abfrage in einem Feld zwischenzuspeichern, sodass wiederholte Abfragen schneller beantwortet werden können. Dadurch wird zwar der Zustand des Objekts, das den Wert zwischenspeichert, geändert, die Änderung ist aber nicht

sichtbar. Eine Reihe von Abfragen liefert stets die gleichen Ergebnisse.

11.1.2 VORGEHEN

- Kopieren Sie die Funktion und vergeben Sie einen Namen, der sie als Abfrage kenntlich macht.

  Sehen Sie in der Funktion nach, was sie zurückgibt. Wenn die Abfrage für die Zuweisung zu einer Variable verwendet wird, sollte der Variablenname einen deutlichen Hinweis darauf geben.

- Entfernen Sie jegliche Seiteneffekte aus der neuen Abfragefunktion.
- Führen Sie statische Überprüfungen durch.
- Suchen Sie nach allen Aufrufen der ursprünglichen Methode. Wenn der Aufruf den Rückgabewert verwendet, ersetzen Sie den ursprünglichen Aufruf durch einen Aufruf der Abfrage und fügen Sie darunter einen Aufruf der ursprünglichen Methode ein. Testen Sie nach jeder Änderung.
- Entfernen Sie die Rückgabewerte aus der ursprünglichen Funktion.
- Testen Sie.

Nachdem diese Schritte erledigt sind, wird es in der Abfrage und der ursprünglichen Methode oftmals Redundanzen geben, die aufgeräumt werden sollten.

11.1.3 BEISPIEL

Hier folgt eine Funktion, die eine Liste von Namen nach Schurken durchsucht. Wird einer entdeckt, gibt sie den Namen des Schurken zurück und löst Alarm aus. Die Funktion verfährt nur beim ersten gefundenen Übeltäter auf diese Weise (ich schätze, einer ist genug).

```
function alertForMiscreant (people) {
  for (const p of people) {
    if (p === "Don") {
      setOffAlarms();
      return "Don";
    }
    if (p === "John") {
      setOffAlarms();
      return "John";
    }
  }
  return "";
}
```

Als Erstes kopiere ich die Funktion und benenne sie entsprechend der Art der Abfrage:

```
function findMiscreant (people) {
  for (const p of people) {
    if (p === "Don") {
      setOffAlarms();
      return "Don";
    }
    if (p === "John") {
      setOffAlarms();
      return "John";
    }
  }
  return "";
}
```

Anschließend entferne ich die Seiteneffekte dieser Abfrage:

```
function findMiscreant (people) {
  for (const p of people) {
    if (p === "Don") {
      setOffAlarms();
      return "Don";
    }
    if (p === "John") {
      setOffAlarms();
      return "John";
    }
  }
  return "";
}
```

Jetzt ersetze ich alle Aufrufe durch einen Aufruf der Abfrage, gefolgt von einem Aufruf der Funktion, die Veränderungen vornimmt.

Aus

```
const found = alertForMiscreant(people);
```

wird also

```
const found = findMiscreant(people);
alertForMiscreant(people);
```

Dann entferne ich die Rückgabewerte aus der verändernden Funktion:

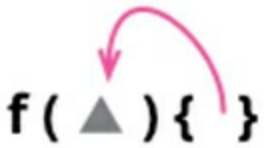
```
function alertForMiscreant (people) {
  for (const p of people) {
    if (p === "Don") {
      setOffAlarms();
      return;
    }
    if (p === "John") {
      setOffAlarms();
      return;
    }
  }
  return;
}
```

Jetzt gibt es in der ursprünglichen Funktion und der neuen Abfrage Redundanzen, also kann ich *Algorithmus ersetzen* (Abschnitt 7.9) verwenden, sodass die Funktion die neue Abfrage verwendet:

```
function alertForMiscreant (people) {
  if (findMiscreant(people) !== "") setOffAlarms();
}
```

11.2 Funktion parametrisieren (Parameterize Function)

Ehemalige Bezeichnung: *Methode parametrisieren* (Parameterize Method)




```
function tenPercentRaise(aPerson) {
  aPerson.salary = aPerson.salary.multiply(1.1);
}
function fivePercentRaise(aPerson) {
  aPerson.salary = aPerson.salary.multiply(1.05);
}
```

```
function raise(aPerson, factor) {
  aPerson.salary = aPerson.salary.multiply(1 +
factor);
}
```

11.2.1 MOTIVATION

Wenn zwei Funktionen eine sehr ähnliche Programmlogik auf verschiedene literale Werte anwenden, kann ich Redundanzen durch eine einzelne Funktion ersetzen, die unterschiedliche Werte als Parameter entgegennimmt. Die Funktion wird dadurch sogar noch nützlicher, weil ich sie auch an anderer Stelle mit verschiedenen Werten aufrufen kann.

11.2.2 VORGEHEN

- Wählen Sie eine der einander ähnlichen Methoden aus.
- Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um Parameter für alle Literale hinzuzufügen, die zu Parametern werden sollen.
- Fügen Sie den Aufrufern der Funktion den literalen Wert hinzu.
- Testen Sie.
- Sorgen Sie dafür, dass die Funktion die neuen Parameter verwendet. Testen Sie nach jeder Änderung.
- Ersetzen Sie alle Aufrufe der ähnlichen Funktionen durch Aufrufe der parametrisierten Funktion. Führen Sie nach jeder Änderung Tests durch.

  Wenn die ursprüngliche parametrisierte Funktion im Kontext einer der ähnlichen Funktionen nicht funktioniert, passen Sie sie entsprechend an, bevor Sie zur nächsten übergehen.

11.2.3 BEISPIEL

Hier zeige ich ein auf der Hand liegendes Beispiel:

```
function tenPercentRaise(aPerson) {
  aPerson.salary = aPerson.salary.multiply(1.1);
}
function fivePercentRaise(aPerson) {
  aPerson.salary = aPerson.salary.multiply(1.05);
}
```

Es ist hoffentlich offensichtlich, dass ich diesen Code durch den folgenden ersetzen kann:

```
function raise(aPerson, factor) {
  aPerson.salary = aPerson.salary.multiply(1 +
```

```
factor);
}
```

Es kann allerdings auch etwas komplizierter werden. Betrachten Sie folgenden Code:

```
function baseCharge(usage) {
  if (usage < 0) return usd(0);
  const amount =
        bottomBand(usage) * 0.03
        + middleBand(usage) * 0.05
        + topBand(usage) * 0.07;
  return usd(amount);
}

function bottomBand(usage) {
  return Math.min(usage, 100);
}
function middleBand(usage) {
  return usage > 100 ? Math.min(usage, 200) - 100 : 0;
}
function topBand(usage) {
return usage > 200 ? usage - 200 : 0;
}
```

Hier ist die Logik ziemlich ähnlich – aber ist sie hinreichend ähnlich, um eine parametrisierte Methode zu erlauben? Ist sie – allerdings ist es weniger offensichtlich als in dem obigen trivialen Beispiel.

Wenn ich einander ähnliche Funktionen parametrisieren möchte, verfolge ich den Ansatz, einer der Funktionen Parameter hinzuzufügen und dabei die anderen Fälle im

Auge zu behalten. Wenn sich die Elemente dabei wie in diesem Beispiel bestimmten Bereichen zuordnen lassen, ist es meist günstig, mit dem mittleren Bereich anzufangen. Ich sorge also dafür, dass `middleBand` Parameter verwendet, und passe die anderen Aufrufer entsprechend an.

Die Funktion `middleBand` verwendet zwei Literale: `100` und `200`. Sie repräsentieren die untere und die obere Grenze des mittleren Bands. Ich verwende zunächst *Funktionsdeklaration ändern* (Abschnitt 6.5), um sie dem Aufruf hinzuzufügen. Bei dieser Gelegenheit vergebe ich außerdem einen neuen Funktionsnamen, der für die Parametrisierung sinnvoll ist:

```
function withinBand(usage, bottom, top) {
  return usage > 100 ? Math.min(usage, 200) - 100 : 0;
}

function baseCharge(usage) {
  if (usage < 0) return usd(0);
  const amount =
        bottomBand(usage) * 0.03
        + withinBand(usage, 100, 200) * 0.05
        + topBand(usage) * 0.07;
  return usd(amount);
}
```

Ich ersetze die Literale durch einen Verweis auf den jeweiligen Parameter:

```
function withinBand(usage, bottom, top) {
  return usage > bottom ? Math.min(usage, 200) -
bottom : 0;
}
```

Anschließend:

```
function withinBand(usage, bottom, top) {
  return usage > bottom ? Math.min(usage, top) -
bottom : 0;
}
```

Ich ersetze den Aufruf von `bottomBand` durch einen Aufruf der frisch parametrisierten Funktion:

```
function baseCharge(usage) {
  if (usage < 0) return usd(0);
  const amount =
        withinBand(usage, 0, 100) * 0.03
        + withinBand(usage, 100, 200) * 0.05
        + topBand(usage) * 0.07;
  return usd(amount);
}

function bottomBand(usage) {
  return Math.min(usage, 100);
}
```

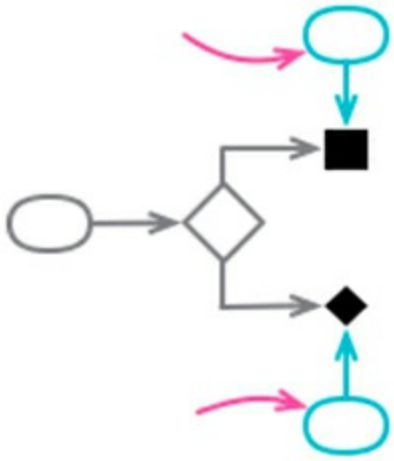
Bei der Ersetzung des Aufrufs von `topBand` muss ich von `Infinity` Gebrauch machen:

```
function baseCharge(usage) {
  if (usage < 0) return usd(0);
  const amount =
        withinBand(usage, 0, 100) * 0.03
        + withinBand(usage, 100, 200) * 0.05
        + withinBand(usage, 200, Infinity) * 0.07;
  return usd(amount);
}

function topBand(usage) {
  return usage > 200 ? usage - 200 : 0;
}
```

So wie die Programmlogik jetzt funktioniert, könnte ich auf die anfängliche Wächterbedingung nun verzichten. Sie ist zwar hinsichtlich der Logik unnötig, allerdings behalte ich sie bei, weil sie dokumentiert, wie dieser Fall zu handhaben ist.

11.3 Steuerungs-Flag entfernen (Remove Flag Argument)

Ehemalige Bezeichnung: *Parameter durch explizite Methoden ersetzen* (Replace Parameter with Explicit Methods)



```
function setDimension(name, value) {
  if (name === "height") {
    this._height = value;
    return;
  }
  if (name === "width") {
    this._width = value;
    return;
  }
}
```

```
function setHeight(value) {this._height = value;}
function setWidth (value) {this._width = value;}
```

11.3.1 MOTIVATION

Ein Steuerungs-Flag ist ein Funktionsargument, mit dem der Aufrufer steuert, welche Programmlogik die

aufgerufene Funktion ausführen soll. Ich könnte beispielsweise eine Funktion wie diese aufrufen:

```
function bookConcert(aCustomer, isPremium) {
  if (isPremium) {
    // Programmlogik für Premiumkunden
  } else {
    // Programmlogik für reguläre Kunden
  }
}
```

Wenn ich eine Buchung für einen Premiumkunden vornehmen möchte, rufe ich die Funktion wie folgt auf:

```
bookConcert(aCustomer, true);
```

Für Steuerungs-Flags können auch `enums` verwendet werden:

```
bookConcert(aCustomer, CustomerType.PREMIUM);
```

Auch Strings (oder Symbole, wenn die Programmiersprache welche verwendet) sind einsetzbar:

```
bookConcert(aCustomer, "premium");
```

Ich lehne Steuerungs-Flags ab, weil sie es erschweren, zu

verstehen, welche Funktionen zu welchem Zweck aufgerufen werden können und wie die Aufrufe erfolgen. Wenn ich mich in eine API einarbeite, sehe ich mir normalerweise als Erstes die Liste der verfügbaren Funktionen an. Steuerungs-Flags verbergen die Unterschiede, die in den verfügbaren Funktionsaufrufen vorhanden sind. Wenn ich eine Funktion verwenden möchte, muss ich erst herausfinden, welche Werte die Steuerungs-Flags annehmen können. Boolesche Flags sind noch schlimmer, weil sie für den Betrachter keine Aussagekraft besitzen – an einem Funktionsaufruf kann ich nicht erkennen, welche Bedeutung `true` hat. Es ist viel verständlicher, einen expliziten Funktionsaufruf für die zu erledigende Aufgabe bereitzustellen:

```
premiumBookConcert(aCustomer);
```

Argumente dieser Art sind nicht immer unbedingt Steuerungs-Flags. Damit es sich um ein Steuerungs-Flag handelt, muss der Aufrufer als booleschen Wert ein Literal übergeben. Zudem muss die implementierende Funktion das Argument zur Steuerung des Kontrollflusses verwenden – nicht als Information, die an andere Funktionen weitergereicht wird.

Durch das Entfernen von Steuerungs-Flags wird der Code nicht nur verständlicher, auch die Verwendung zusätzlicher Werkzeuge wird dadurch erleichtert. Auf

diese Art ist für Werkzeuge zur Code-Analyse der Unterschied zwischen dem Aufruf der Logik für Premiumkunden und dem für reguläre Kunden leichter erkennbar.

Steuerungs-Flags können sinnvoll sein, wenn eine Funktion gleich mehrere verwendet, weil sonst für alle möglichen Kombinationen der Werte eigene Funktionen erforderlich wären. Allerdings wäre das auch ein Hinweis darauf, dass die Funktion zu viele Aufgaben erledigt und ich nach einer Möglichkeit suchen sollte, einfachere Funktionen zu erstellen, die sich dieser Aufgaben annehmen.

11.3.2 VORGEHEN

- Erstellen Sie für alle Werte des Steuerungs-Flags eine eigene Funktion.

  Wenn die Hauptfunktion eine eindeutige Bedingung zur Aufgabenverteilung enthält, können Sie *Bedingung zerlegen* (Abschnitt 10.1) verwenden, um eigene Funktionen zu erstellen. Erstellen Sie anderenfalls Wrapper-Funktionen.

- Ersetzen Sie jeden Aufruf, der Literale als Parameter verwendet, durch einen Aufruf der expliziten Funktion.

11.3.3 BEISPIEL

Beim Durchsehen von Code fallen mir Aufrufe zur Berechnung des Lieferdatums einer Warensendung ins Auge. Die Aufrufe sehen folgendermaßen aus:

```
aShipment.deliveryDate = deliveryDate(anOrder, true);
```

Andere hingegen zeigen sich so:

```
aShipment.deliveryDate = deliveryDate(anOrder, false);
```

Angesichts dieses Codes frage ich mich sofort, welche Bedeutung der boolesche Wert hat. Welche Aufgabe hat er wohl?

Die Funktion `deliveryDate` ist folgendermaßen aufgebaut:

```
function deliveryDate(anOrder, isRush) {
  if (isRush) {
    let deliveryTime;
    if (["MA", "CT"] .includes(anOrder.deliveryState))
         deliveryTime = 1;
    else if (["NY",
"NH"].includes(anOrder.deliveryState))
         deliveryTime = 2;
    else deliveryTime = 3;
    return anOrder.placedOn.plusDays(1 +
deliveryTime);
  }
  else {
    let deliveryTime;
    if (["MA", "CT",
"NY"].includes(anOrder.deliveryState))
         deliveryTime = 2;
    else if (["ME", "NH"]
.includes(anOrder.deliveryState))
```

```
        deliveryTime = 3;
    else deliveryTime = 4;
    return anOrder.placedOn.plusDays(2 +
deliveryTime);
  }
}
```

Der Aufrufer verwendet hier ein boolesches Literal, um festzulegen, welcher Code ausgeführt werden soll – ein klassisches Steuerungs-Flag. Der eigentliche Sinn, eine Funktion zu verwenden, ist jedoch, die Anweisungen des Aufrufers nachzuverfolgen, deshalb ist es besser, die Intention des Aufrufers durch explizite Funktionen zu verdeutlichen.

In diesem Fall kann ich *Bedingung zerlegen* (Abschnitt 10.1) anwenden und erhalte Folgendes:

```
function deliveryDate(anOrder, isRush) {
  if (isRush) return rushDeliveryDate(anOrder);
  else return regularDeliveryDate(anOrder);
}

function rushDeliveryDate(anOrder) {
  let deliveryTime;
  if (["MA", "CT"] .includes(anOrder.deliveryState))
       deliveryTime = 1;
  else if (["NY",
"NH"].includes(anOrder.deliveryState))
       deliveryTime = 2;
  else deliveryTime = 3;
  return anOrder.placedOn.plusDays(1 + deliveryTime);
}
```

```
function regularDeliveryDate(anOrder) {
  let deliveryTime;
  if (["MA", "CT",
"NY"].includes(anOrder.deliveryState))
       deliveryTime = 2;
  else if (["ME", "NH"]
.includes(anOrder.deliveryState))
       deliveryTime = 3;
  else deliveryTime = 4;
  return anOrder.placedOn.plusDays(2 + deliveryTime);
}
```

Die beiden neuen Funktionen machen die Absicht des Aufrufs besser kenntlich, also ersetze ich jeden Aufruf von

```
aShipment.deliveryDate = deliveryDate(anOrder, true);
```

durch

```
aShipment.deliveryDate = rushDeliveryDate(anOrder);
```

und verfahre mit dem anderen Fall auf ähnliche Weise.

Nachdem ich alle Aufrufer ersetzt habe, entferne ich `deliveryDate`.

Ein Steuerungs-Flag ist mehr als nur das Vorhandensein eines booleschen Werts; es bedeutet auch, dass der boolesche Wert in Form eines Literals anstelle von Daten

übergeben wird. Nehmen wir an, alle Aufrufer von `deliveryDate` würden wie folgt aussehen:

```
const isRush = determineIfRush(anOrder);
aShipment.deliveryDate = deliveryDate(anOrder,
isRush);
```

Dann wäre die Signatur der Funktion `deliveryDate` unproblematisch (allerdings würde ich dennoch *Bedingung zerlegen* (Abschnitt 10.1) anwenden wollen).

Nun könnte es sein, dass einige Aufrufer das Argument als Steuerungs-Flag verwenden, indem sie ein Literal zuweisen, während andere Aufrufer Daten zuweisen. In diesem Fall würde ich *Steuerungs-Flag entfernen* ebenfalls anwenden, aber die Aufrufe, die Daten verwenden, nicht ändern und die Funktion `deliveryDate` nicht löschen. Auf diese Weise unterstütze ich beide Schnittstellen für unterschiedliche Anwendungsfälle.

Die Bedingung wie hier zu zerlegen ist eine gut geeignete Art und Weise, dieses Refactoring vorzunehmen. Es funktioniert jedoch nur, wenn die parameterbasierte Entscheidungsfindung außerhalb der Funktion stattfindet (oder ich sie mit Hilfe eines Refactorings leicht dahingehend umbauen kann). Es ist ebenfalls möglich, dass der Parameter umständlicher verwendet wird, so wie in dieser alternativen Version von

`deliveryDate`:

```
function deliveryDate(anOrder, isRush) {
  let result;
  let deliveryTime;
  if (anOrder.deliveryState === "MA"
                           || anOrder.deliveryState ===
"CT")
    deliveryTime = isRush? 1 : 2;
  else if (anOrder.deliveryState === "NY"
                          || anOrder.deliveryState ===
"NH") {
    deliveryTime = 2;
    if (anOrder.deliveryState === "NH" && !isRush)
      deliveryTime = 3;
  }
  else if (isRush)
    deliveryTime = 3;
  else if (anOrder.deliveryState === "ME")
    deliveryTime = 3;
  else
    deliveryTime = 4;
  result = anOrder.placedOn.plusDays(2 +
deliveryTime);
  if (isRush) result = result.minusDays(1);
  return result;
}
```

In diesem Fall macht es vermutlich mehr Arbeit, als mir lieb ist, `isRush` in eine Bedingung auf übergeordneter Ebene zu verlagern. Stattdessen führe ich auf der Ebene oberhalb von `deliveryDate` folgende Funktionen ein:

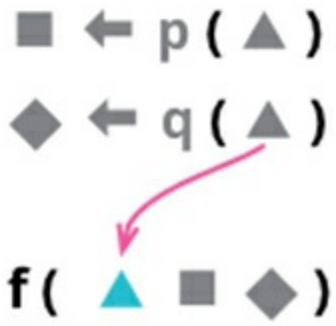
```
function rushDeliveryDate (anOrder) {
```

```
  return deliveryDate(anOrder, true);
}
function regularDeliveryDate(anOrder) {
  return deliveryDate(anOrder, false);
}
```

Diese Wrapper-Funktionen sind im Wesentlichen partielle Lösungen von `deliveryDate`, wenngleich sie eher in Form von Programmtext definiert werden und nicht durch Erstellen von Funktionen.

Anschließend kann ich weitere Aufrufer ersetzen, wie schon zuvor bei der zerlegten Bedingung. Wenn es keine Aufrufer gibt, die den Parameter mittels Daten verwenden, schränke ich die Sichtbarkeit der Funktion ein oder benenne sie so um, dass der Name erkennen lässt, dass sie nicht direkt verwendet werden sollte (wie etwa `deliveryDateHelperOnly`).

11.4 Vollständiges Objekt erhalten (Preserve Whole Object)



```
const low = aRoom.daysTempRange.low;
const high = aRoom.daysTempRange.high;
if (aPlan.withinRange(low, high))
```

```
if (aPlan.withinRange(aRoom.daysTempRange))
```

11.4.1 MOTIVATION

Wenn ich auf Code stoße, der Werte anhand eines Datensatzes berechnet und diese Werte anschließend an eine Funktion übergibt, ersetze ich diese Werte durch den gesamten Datensatz und sorge dafür, dass die Funktion die erforderlichen Werte selbst berechnet.

Falls die Funktion zukünftig weitere Werte benötigt, lassen sich Änderungen leichter handhaben, wenn der gesamte Datensatz übergeben wird – diese Änderung erfordert keine Anpassung der Parameterliste. Zudem wird die Parameterliste kürzer und damit in der Regel der Funktionsaufruf besser verständlich. Wenn mehrere Funktionen mit einzelnen Werten aufgerufen werden, führt das häufig zu redundanter Logik für die Verarbeitung der Werte – Logik, die häufig dem vollständigen Datensatz zugeordnet werden kann.

Der wichtigste Grund, auf die Übergabe des

vollständigen Datensatzes zu verzichten, ist die Vermeidung einer Abhängigkeit der aufgerufenen Funktion vom gesamten Datensatz. Dieser Fall tritt typischerweise auf, wenn die Funktion zu einem anderen Modul gehört.

Ein Objekt nach mehreren Werten zu fragen, um anschließend Programmlogik auf diese anzuwenden, ist an sich schon ein Code-Smell (*Feature-Neid*, siehe Abschnitt 3.9) und generell ein Hinweis darauf, dass die Programmlogik in das Objekt selbst verlagert werden sollte. *Vollständiges Objekt erhalten* kommt besonders häufig zum Einsatz, nachdem ich *Parameterobjekt einführen* (Abschnitt 6.8) angewendet habe und nach Vorkommen des ursprünglichen Datenblocks suche, um sie durch das neue Objekt zu ersetzen.

Wenn mehrere Codeabschnitte nur dieselbe Teilmenge der Eigenschaften eines Objekts verwenden, bietet sich womöglich eine gute Gelegenheit, *Klasse extrahieren* (Abschnitt 7.5) anzuwenden.

Häufig wird der Fall übersehen, dass ein Objekt ein anderes Objekt mit mehreren seiner eigenen Datenwerte aufruft. Ich ersetze diese Werte dann durch einen Verweis auf das Objekt selbst (`this` in JavaScript).

11.4.2 VORGEHEN

- Erstellen Sie eine leere Funktion mit den gewünschten Parametern.

Vergeben Sie einen Namen, nach dem Sie leicht suchen können, damit er zum Schluss ersetzt werden kann.

- Rufen Sie in der neuen Funktion die alte Funktion auf. In dem Aufruf müssen die neuen Parameter den alten zugeordnet sein.
- Führen Sie statische Überprüfungen durch.
- Sorgen Sie dafür, dass alle Aufrufer die neue Funktion verwenden.

  Unter Umständen kann das bedeuten, dass ein Teil des Codes zur Berechnung des Parameters nicht mehr benötigt wird und mit *Toten Code entfernen* (Abschnitt 8.9) gelöscht werden kann.
- Nachdem alle ursprünglichen Aufrufer geändert worden sind, können Sie *Funktion inline platzieren* (Abschnitt 6.2) auf die ursprüngliche Funktion anwenden.
- Ändern Sie die Namen der neuen Funktion und alle Aufrufe.

11.4.3 BEISPIEL

Ein Temperaturkontrollsystem vergleicht den täglich gemessenen Temperaturbereich mit der Vorgabe eines Heizplans:

```
Aufrufer...
const low = aRoom.daysTempRange.low;
const high = aRoom.daysTempRange.high;
if (!aPlan.withinRange(low, high))
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");

class HeatingPlan...
withinRange(bottom, top) {
  return (bottom >= this._temperatureRange.low) &&
            (top <= this._temperatureRange.high);
}
```

Anstatt die Informationen über den Temperaturbereich in Teilen zu übergeben, übergebe ich das gesamte Objekt.

Als Erstes lege ich die gewünschte Schnittstelle fest, indem ich eine leere Funktion erstelle:

```
class HeatingPlan...
xxNEWwithinRange(aNumberRange) {
}
```

Da ich vorhabe, die vorhandene Funktion `withinRange` zu ersetzen, verwende ich den gleichen Namen mit einem Präfix, das sich leicht entfernen lässt.

Anschließend füge ich den Funktionsrumpf hinzu, der die vorhandene Funktion `withinRange` aufruft. Der Aufruf ordnet die neuen Parameter den vorhandenen zu:

```
class HeatingPlan...
xxNEWwithinRange(aNumberRange) {
  return this.withinRange(aNumberRange.low,
                          aNumberRange.high);
}
```

Jetzt kann ich mit der eigentlichen Arbeit beginnen, indem ich dafür sorge, dass alle vorhandenen Funktionsaufrufe die neue Funktion aufrufen:

```
Aufrufer...
const low = aRoom.daysTempRange.low;
const high = aRoom.daysTempRange.high;
if (!aPlan.xxNEWwithinRange(aRoom.daysTempRange))
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");
```

Nachdem ich die Aufrufe angepasst habe, stelle ich möglicherweise fest, dass ein Teil des vorherigen Codes nicht mehr benötigt wird, und wende *Toten Code entfernen* (Abschnitt 8.9) an.

```
Aufrufer...
const low = aRoom.daysTempRange.low;
const high = aRoom.daysTempRange.high;
if (!aPlan.xxNEWwithinRange(aRoom.daysTempRange))
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");
```

Ich führe die Ersetzungen der Reihe nach durch und teste nach jeder Änderung. Sobald ich alle ersetzt habe, wende ich *Funktion inline platzieren* (Abschnitt 6.2) auf die ursprüngliche Funktion an:

```
class HeatingPlan...
xxNEWwithinRange(aNumberRange) {
  return (aNumberRange.low >=
this._temperatureRange.low) &&
         (aNumberRange.high <=
this._temperatureRange.high);
}
```

Zum Schluss entferne ich noch das unschöne Präfix von der neuen Funktion und all ihren Aufrufen. Dank des Präfixes kann ich eine einfache globale Ersetzung vornehmen, selbst wenn mein Editor keine verlässliche Funktion zum Umbenennen besitzt:

```
class HeatingPlan...
withinRange(aNumberRange) {
  return (aNumberRange.low >=
this._temperatureRange.low) &&
         (aNumberRange.high <=
this._temperatureRange.high);
}

Aufrufer...
if (!aPlan.withinRange(aRoom.daysTempRange))
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");
```

11.4.4 BEISPIEL: EINE VARIANTE ZUM ERSTELLEN DER NEUEN FUNKTION

Beim letzten Beispiel habe ich den Code für die neue Funktion direkt geschrieben. In den meisten Fällen ist das am einfachsten. Es gibt jedoch eine Variante, die sich gelegentlich als nützlich erweist, bei der sich die neue Funktion vollständig durch andere Refactorings ergibt.

Ich beginne mit einem Aufrufer der bestehenden Funktion:

```
Aufrufer...
const low = aRoom.daysTempRange.low;
const high = aRoom.daysTempRange.high;
if (!aPlan.withinRange(low, high))
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");
```

Ich möchte den Code so umstrukturieren, dass ich die neue Funktion erstellen kann, indem ich *Funktion extrahieren* (Abschnitt 6.1) auf schon vorhandenen Code anwende. Der Code des Aufrufers ist dafür noch nicht geeignet, aber ich kann ihn darauf vorbereiten, indem ich einige Male *Variable extrahieren* (Abschnitt 6.3) anwende. Als Erstes entkopple ich den Aufruf der alten Funktion von der Bedingung:

```
Aufrufer...
const low = aRoom.daysTempRange.low;
const high = aRoom.daysTempRange.high;
const isWithinRange = aPlan.withinRange(low, high);
if (!isWithinRange)
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");
```

Dann extrahiere ich den Eingabeparameter:

```
Aufrufer...
const tempRange = aRoom.daysTempRange;
const low = tempRange.low;
const high = tempRange.high;
const isWithinRange = aPlan.withinRange(low, high);
```

```
if (!isWithinRange)
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");
```

Jetzt kann ich *Funktion extrahieren* (Abschnitt 6.1) anwenden, um die neue Funktion zu erstellen:

```
Aufrufer...
const tempRange = aRoom.daysTempRange;
const isWithinRange = xxNEWwithinRange(aPlan,
tempRange);
if (!isWithinRange)
  alerts.push("Zimmertemperatur außerhalb der
Vorgabe");

Oberste Ebene...
function xxNEWwithinRange(aPlan, tempRange) {
  const low = tempRange.low;
  const high = tempRange.high;
  const isWithinRange = aPlan.withinRange(low, high);
  return isWithinRange;
}
```

Da die ursprüngliche Funktion zu einem anderen Kontext gehört (zur Klasse `HeatingPlan`), muss ich *Funktion verschieben* (Abschnitt 8.1) anwenden.

```
Aufrufer...
const tempRange = aRoom.daysTempRange;
const isWithinRange =
aPlan.xxNEWwithinRange(tempRange);
if (!isWithinRange)
  alerts.push("Zimmertemperatur außerhalb der
```

```
Vorgabe");

class HeatingPlan...
xxNEWwithinRange(tempRange) {
  const low = tempRange.low;
  const high = tempRange.high;
  const isWithinRange = this.withinRange(low, high);
  return isWithinRange;
}
```

Ich fahre wie zuvor fort, ersetze die übrigen Aufrufe und platziere die alte Funktion inline in der neuen. Ich würde außerdem die extrahierten Variablen inline platzieren, um eine geeignete Trennung für die anschließende Extraktion der neuen Funktion zu erreichen.

Da sich diese Variante ausschließlich aus Refactorings ergibt, ist sie besonders praktisch, wenn mir ein Refactoring-Tool zur Verfügung steht, das Extraktionen und Inline-Platzierungen zuverlässig automatisiert ausführt.

11.5 Parameter durch Abfrage ersetzen (Replace Parameter with Query)

Ehemalige Bezeichnung: *Parameter durch Methode ersetzen* (Replace Parameter with Method)

Umkehrung von: *Abfrage durch Parameter ersetzen* (Replace Query with Parameter, Abschnitt 11.6)

```
availableVaction(anEmployee, anEmployee.grade);

function availableVacation(anEmployee, grade) {
  // Verfügbare Urlaubstage berechnen ...
```

```
availableVacation(anEmployee)

function availableVacation(anEmployee) {
  const grade = anEmployee.grade;
  // Verfügbare Urlaubstage berechnen ...
```

11.5.1 MOTIVATION

Die Parameterliste einer Funktion sollte Aspekte ihrer Variabilität verdeutlichen und Hinweise auf die verschiedenen primären Verhaltensweisen der Funktion geben. Wie bei allen Anweisungen im Code ist es sinnvoll, Redundanzen zu vermeiden – zudem sind Funktionen mit kurzen Parameterlisten leichter verständlich.

Wenn ein Aufruf einen Wert übergibt, der von der Funktion auch genauso leicht selbst ermittelt werden

könnte, handelt es sich um eine Form der Redundanz – eine, die den Aufruf unnötig verkompliziert, weil der Aufrufer nun den Wert des Parameters selbst herausfinden muss, obwohl er von dieser Aufgabe befreit werden könnte.

Entscheidend ist hier der Ausdruck »genauso leicht«. Durch das Entfernen des Parameters verlagere ich die Verantwortlichkeit für die Bestimmung des Wertes des Parameters. Ist der Parameter vorhanden, liegt die Verantwortung dafür, den Wert zu ermitteln, beim Aufrufer, anderenfalls muss die Funktion diese Aufgabe erledigen. Ich habe mir zur Gewohnheit gemacht, Aufrufer zu entlasten, was bedeutet, dass ich die Verantwortlichkeit in die Funktion verlagere – sofern es angemessen ist.

Der häufigste Grund, *Parameter durch Abfrage ersetzen* nicht anzuwenden, ist gegeben, wenn durch das Entfernen des Parameters eine unerwünschte Abhängigkeit der Funktion entsteht, die sie zwingt, auf ein Programmelement zuzugreifen, von dem sie besser keine Kenntnis haben sollte. Dabei könnte es sich um eine neue Abhängigkeit handeln, oder um eine vorhandene, die ich entfernen möchte. Dieser Fall tritt für gewöhnlich auf, wenn ich der Funktion einen problematischen Aufruf hinzufügen oder auf einen Wert in einem Objekt zugreifen muss, das ich später lieber wieder entfernen würde.

Die Anwendung von *Parameter durch Abfrage ersetzen* ist am sichersten, wenn sich der Wert des zu entfernenden Parameters einfach durch Abfrage eines anderen Parameters in der Liste ermitteln lässt. Es ist schließlich nicht besonders sinnvoll, zwei Parameter zu übergeben, wenn der eine anhand des anderen bestimmt werden kann.

Zu beachten ist außerdem, ob die fragliche Funktion referenzielle Transparenz besitzt. Damit ist gemeint, ob ich mir sicher sein kann, dass sie sich stets auf die gleiche Weise verhält, wenn ich sie mit denselben Parameterwerten aufrufe. Solche Funktionen lassen sich viel besser verstehen und testen, und ich möchte sie nicht ändern, wenn diese Eigenschaft dabei verloren geht. So würde ich beispielsweise einen Parameter nicht durch einen Zugriff auf eine veränderliche globale Variable ersetzen.

11.5.2 VORGEHEN

- Wenden Sie, falls nötig, *Funktion extrahieren* (Abschnitt 6.1) auf die Berechnung des Parameters an.
- Ersetzen Sie in der Funktion enthaltene Verweise auf den Parameter durch Verweise auf den Ausdruck, der den Parameter liefert. Führen Sie nach jeder Änderung Tests durch.
- Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um den Parameter zu entfernen.

11.5.3 BEISPIEL

Ich verwende *Parameter durch Abfrage ersetzen* am häufigsten, nachdem ich andere Refactorings durchgeführt habe, die einen Parameter überflüssig machen. Betrachten Sie den folgenden Code:

```
class Order...
get finalPrice() {
  const basePrice = this.quantity * this.itemPrice;
  let discountLevel;
  if (this.quantity > 100) discountLevel = 2;
  else discountLevel = 1;
  return this.discountedPrice(basePrice,
discountLevel);
}

discountedPrice(basePrice, discountLevel) {
  switch (discountLevel) {
    case 1: return basePrice * 0.95;
    case 2: return basePrice * 0.9; }
}
```

Beim Vereinfachen einer Funktion bin ich bestrebt, *Temporäre Variable durch Abfrage ersetzen* (Abschnitt 7.4) anzuwenden. Damit ergibt sich Folgendes:

```
class Order...
get finalPrice() {
  const basePrice = this.quantity * this.itemPrice;
  return this.discountedPrice(basePrice,
this.discountLevel);
}

get discountLevel() {
```

```
  return (this.quantity > 100) ? 2 : 1;
}
```

Nachdem ich diese Änderung vorgenommen habe, ist es nicht mehr erforderlich, das Ergebnis von `discountLevel` an `discountedPrice` zu übergeben – die Funktion kann den Aufruf auch einfach selbst vornehmen.

Ich ersetze also die Verweise auf den Parameter durch Aufrufe der Methode:

```
class Order...
discountedPrice(basePrice, discountLevel) {
  switch (this.discountLevel) {
    case 1: return basePrice * 0.95;
    case 2: return basePrice * 0.9; }
}
```

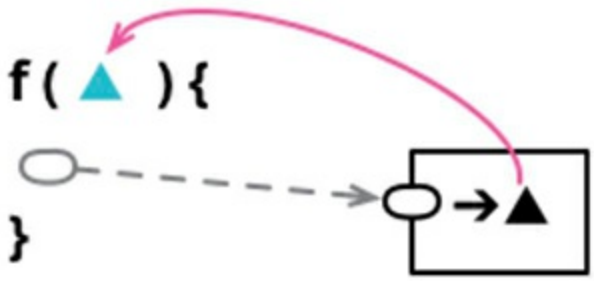
Jetzt kann ich *Funktionsdeklaration ändern* (Abschnitt 6.5) verwenden, um den Parameter zu entfernen:

```
class Order...
get finalPrice() {
  const basePrice = this.quantity * this.itemPrice;
  return this.discountedPrice(basePrice, this.discountLevel);
}

discountedPrice(basePrice, discountLevel) {
  switch (this.discountLevel) {
```

```
    case 1: return basePrice * 0.95;
    case 2: return basePrice * 0.9;
  }
}
```

11.6 Abfrage durch Parameter ersetzen (Replace Query with Parameter)

Umkehrung von: *Parameter durch Abfrage ersetzen* (Replace Parameter with Query, Abschnitt 11.5)




```
targetTemperature(aPlan)

function targetTemperature(aPlan) {
  currentTemperature = thermostat.currentTemperature;
  // verbleibender Teil der Funktion ...
```

```
targetTemperature(aPlan,
thermostat.currentTemperature)

function targetTemperature(aPlan, currentTemperature)
```

```
{
  // verbleibender Teil der Funktion ...
```

11.6.1 MOTIVATION

Beim Betrachten einer Funktion springen mir mitunter Verweise auf Objekte im Gültigkeitsbereich der Funktion ins Auge, die mir nicht gefallen. Das könnte ein Verweis auf eine globale Variable sein oder ein Verweis auf ein zum selben Modul gehörendes Objekt, das ich verschieben möchte. Um hier Abhilfe zu schaffen, muss ich die interne Referenz durch einen Parameter ersetzen, wodurch die Verantwortung für die Auflösung der Referenz auf den Aufrufer der Funktion übertragen wird.

Die meisten dieser Fälle kommen durch meinen Wunsch zustande, die Abhängigkeitsverhältnisse im Code zu ändern – die Zielfunktion soll nicht von dem Element abhängig sein, das ich parametrisieren möchte. Hier besteht ein Spannungsfeld zwischen der Umwandlung hin zu Parametern, was nicht nur zu langen Parameterlisten führt, sondern auch zu einem großen gemeinsamen Gültigkeitsbereich, der eine starke Kopplung von Funktionen zur Folge haben kann. Hierbei handelt es sich wie bei den meisten schwierigen Entscheidungen um ein Problem, für das ich nicht auf Anhieb eine richtige Lösung finden kann. Deshalb ist es so wichtig, dass ich zuverlässig Änderungen vornehmen kann, damit das Programm von meinem allmählich zunehmenden Verständnis profitiert.

Es fällt leichter, über Funktionen nachzudenken, die stets dieselben Ergebnisse liefern, wenn sie mit denselben Parameterwerten aufgerufen werden, die also die vorhin erwähnte referenzielle Transparenz besitzen. Wenn eine Funktion auf ein Element in ihrem Gültigkeitsbereich zugreift, das keine referenzielle Transparenz besitzt, ist sie selbst auch nicht referenziell transparent. Das kann ich beheben, indem ich dieses Element in einen Parameter umwandle. Dadurch wird zwar die Verantwortung dem Aufrufer aufgebürdet, aber Module mit referenzieller Transparenz zu erstellen bietet viele Vorteile. Typischerweise bestehen solche Module aus reinen Funktionen, die z.B. in Programmlogik zur I/O-Handhabung oder in anderen Programmelementen eingebettet sind. Ich kann *Abfrage durch Parameter ersetzen* verwenden, um Teile des Programms zu bereinigen, die sich dann besser testen und leichter verstehen lassen.

Allerdings hat *Abfrage durch Parameter ersetzen* nicht nur Vorteile. Wenn ich die Abfrage durch einen Parameter ersetze, zwinge ich den Aufrufer dazu, dessen Wert bereitzustellen, was das Aufrufen der Funktion verkompliziert. Ich bevorzuge es jedoch, Schnittstellen zu entwerfen, die das Aufrufen so einfach wie möglich gestalten. Letzen Endes geht es um die Verteilung von Verantwortungen innerhalb des Programms, und das ist eine Entscheidung, die weder leicht noch unabänderlich ist. Aus diesem Grund sollte ich mit diesem Refactoring

(und dessen Umkehrung) besonders gut vertraut sein.

11.6.2 VORGEHEN

- Wenden Sie *Variable extrahieren* (Abschnitt 6.3) auf den Abfragecode an, um ihn von der übrigen Funktion zu trennen.
- Wenden Sie auf den verbleibenden Code, der nicht zur Abfrage gehört, *Funktion extrahieren* (Abschnitt 6.1) an.

  Vergeben Sie einen Namen, nach dem Sie später beim Umbenennen leicht suchen können.
- Verwenden Sie *Variable inline platzieren* (Abschnitt 6.4), um die soeben erstellte Variable wieder loszuwerden.
- Wenden Sie *Funktion inline platzieren* (Abschnitt 6.2) auf die ursprüngliche Funktion an.
- Benennen Sie die neue Funktion entsprechend der ursprünglichen.

11.6.3 BEISPIEL

Es geht wieder um ein einfaches, aber dennoch lästiges Temperaturkontrollsystem. Es ermöglicht dem Anwender, an einem Thermostat eine Temperatur auszuwählen – die allerdings innerhalb des vom Heizplan vorgegebenen Bereichs liegen muss:

```
class HeatingPlan...
get targetTemperature() {
  if (thermostat.selectedTemperature > this._max)
    return this._max;
  else if (thermostat.selectedTemperature < this._min)
    return this._min;
  else return thermostat.selectedTemperature;
}
```

```
Aufrufer...
if (thePlan.targetTemperature >
    thermostat.currentTemperature) setToHeat();
else if (thePlan.targetTemperature <
thermostat.currentTemperature) setToCool();
else setOff();
```

Als Anwender eines solchen Systems bin ich vielleicht verärgert, weil der Heizplan Vorrang vor meiner Einstellung hat. Als Programmierer hingegen sollte ich mir eher darüber Gedanken machen, dass die Funktion `targetTemperature` von einem globalen `thermostat`-Objekt abhängig ist. Ich kann diese Abhängigkeit auflösen, indem ich sie in einen Parameter verlagere.

Als Erstes wende ich *Variable extrahieren* (Abschnitt 6.3) auf den Parameter an, den ich in meiner Funktion verwenden möchte:

```
class HeatingPlan...
get targetTemperature() {
  const selectedTemperature =
thermostat.selectedTemperature;
  if      (selectedTemperature > this._max) return
this._max;
  else if (selectedTemperature < this._min) return
this._min;
  else return selectedTemperature;
}
```

Jetzt ist es ganz einfach, *Funktion extrahieren* (Abschnitt 6.1) auf den Funktionsrumpf ohne den Teil, der den Parameter ermittelt, anzuwenden:

```
class HeatingPlan...
get targetTemperature() {
  const selectedTemperature =
thermostat.selectedTemperature;
  return
this.xxNEWtargetTemperature(selectedTemperature);
}

xxNEWtargetTemperature(selectedTemperature) {
  if (selectedTemperature > this._max) return
this._max;
  else if (selectedTemperature < this._min) return
this._min;
  else return selectedTemperature;
}
```

Anschließend platziere ich die soeben extrahierte Variable inline. Die Funktion ist jetzt nur noch ein einfacher Aufruf:

```
class HeatingPlan...
get targetTemperature() {
  return this.xxNEWtargetTemperature(

thermostat.selectedTemperature);
}
```

Jetzt kann ich *Funktion inline platzieren* (Abschnitt 6.2)

auf diese Methode anwenden:

```
Aufrufer...
if (thePlan.xxNEWtargetTemperature(

thermostat.selectedTemperature) >

thermostat.currentTemperature)
  setToHeat();
else if (thePlan.xxNEWtargetTemperature(

thermostat.selectedTemperature) <

thermostat.currentTemperature)
  setToCool();
else
  setOff();
```

Ich mache mir den leicht auffindbaren Namen der neuen Funktion zunutze und benenne sie um, indem ich das Präfix entferne:

```
Aufrufer...
if (thePlan.targetTemperature(

thermostat.selectedTemperature) >

thermostat.currentTemperature)
  setToHeat();
else if (thePlan.targetTemperature(

thermostat.selectedTemperature) <

thermostat.currentTemperature)
```

```
  setToCool();
else
  setOff();

class HeatingPlan...
targetTemperature(selectedTemperature) {
  if (selectedTemperature > this._max) return
this._max;
  else if (selectedTemperature < this._min) return
this._min;
  else return selectedTemperature;
}
```

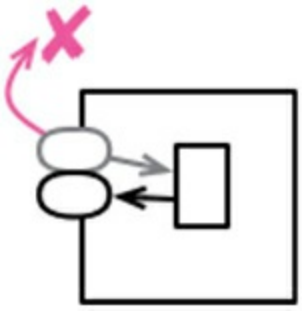
Bei diesem Refactoring ist es öfters der Fall, dass der aufrufende Code anschließend umständlicher aussieht als vorher. Das Entfernen einer Abhängigkeit aus einem Modul verlagert die Verantwortung für die Handhabung dieser Abhängigkeit zurück in den Aufrufer. Das ist der Kompromiss, der zugunsten der verringerten Kopplung eingegangen werden muss.

Das Entfernen der Kopplung an das `thermostat`-Objekt ist aber nicht der einzige Vorteil, den mir dieses Refactoring einbringt. Die Klasse `HeatingPlan` ist unveränderlich – den Feldern werden im Konstruktor ihre Werte zugewiesen, und es gibt keine Methoden, um sie zu ändern. (Ich erspare Ihnen die Mühe, sich die gesamte Klasse ansehen zu müssen; glauben Sie mir das einfach.) Durch das Entfernen des Verweises auf `thermostat` aus der Funktion habe ich `targetTemperature` referenziell transparent gemacht. Jedes Mal, wenn ich `targetTemperature` mit

demselben Argument aufrufe, werde ich stets dasselbe Ergebnis erhalten. Wenn alle Methoden der Klasse `HeatingPlan` referenziell transparent sind, lässt diese sich viel leichter testen und verstehen.

Das JavaScript Klassenmodell bringt das Problem mit sich, dass es nicht möglich ist, eine unveränderliche Klasse zu erzwingen – man kann immer irgendwie an die Daten eines Objekts gelangen. Aber meistens reicht es schon aus, eine Klasse zu schreiben, die auf die Unveränderlichkeit hinweist und diese fördert. Klassen mit diesen Eigenschaften zu erstellen ist oft eine sinnvolle Strategie, und *Abfrage durch Parameter ersetzen* ist dafür ein praktisches Werkzeug.

11.7 Setter entfernen (Remove Setting Method)



```
class Person {
  get name() {...}
  set name(aString) {...}
```

```
class Person {
  get name() {...}
```

11.7.1 MOTIVATION

Das Bereitstellen einer Setter-Methode ist ein Hinweis darauf, dass ein Feld geändert werden kann. Wenn ich nicht möchte, dass sich der Wert des Feldes nach dem Erzeugen des Objekts ändert, stelle ich auch keinen Setter bereit (und mache das Feld unveränderlich). Auf diese Weise wird dem Feld nur im Konstruktor ein Wert zugewiesen, meine Absicht, dass es nicht geändert werden soll, ist klar und ich beseitige jedwede Möglichkeit, das Feld zu ändern.

Es gibt eine Reihe typischer Fälle, für die dieses Refactoring infrage kommt, beispielsweise wenn zum Bearbeiten von Feldern immer eigene Zugriffsmethoden verwendet werden – sogar in Konstruktoren. Das führt dazu, dass der einzige Aufruf des Setters im Konstruktor erfolgt. Ich ziehe es vor, die Setter-Methode zu entfernen, um zu verdeutlichen, dass Aktualisierungen des Feldes nach dem Erstellen des Objekts keinen Sinn ergeben.

In anderen Fällen wird das Objekt von Clients erstellt,

die nicht einfach nur den Konstruktor aufrufen, sondern ein Erzeugungs-Skript ausführen. Ein solches Erzeugungs-Skript ruft zunächst den Konstruktor auf, anschließend folgt eine Reihe von Aufrufen der Setter-Methode zum Konfigurieren des Objekts. Nach Ablauf des Skripts ist eigentlich nicht zu erwarten, dass sich noch einige der Felder (oder sogar alle) ändern. Die Setter sollten stattdessen nur bei der ursprünglichen Erzeugung aufgerufen werden. In einem solchen Fall entferne ich sie, um meine Absicht zu verdeutlichen.

11.7.2 VORGEHEN

- Wenn der Wert, der zugewiesen werden soll, im Konstruktor nicht zur Verfügung steht, können Sie *Funktionsdeklaration ändern* (Abschnitt 6.5) verwenden, um ihn hinzuzufügen. Fügen Sie dem Konstruktor einen Aufruf der Setter-Methode hinzu.

  Wenn Sie mehrere Setter entfernen möchten, sollten Sie dem Konstruktor alle Werte gleichzeitig hinzufügen. Das vereinfacht die nachfolgenden Schritte.

- Entfernen Sie alle Aufrufe von Setter-Methoden außerhalb des Konstruktors und verwenden Sie stattdessen die dem Konstruktor hinzugefügten Werte. Führen Sie nach jeder Änderung Tests durch.

  Wenn Sie den Aufruf des Setters durch das Erstellen eines neuen Objekts nicht ersetzen können (weil Sie eine Zuweisung an ein gemeinsam genutztes Objekt vornehmen), sollten Sie dieses Refactoring unterlassen.

- Wenden Sie *Funktion inline platzieren* (Abschnitt 6.2) auf die Setter-Methode an. Machen Sie das Feld unveränderlich, falls möglich.

- Testen Sie.

11.7.3 BEISPIEL

Ich verwende eine einfache Klasse `Person`:

```
class Person...
get name() {return this._name;}
set name(arg) {this._name = arg;}
get id() {return this._id;}
set id(arg) {this._id = arg;}
```

Zum Erstellen eines neuen Objekts verwende ich Code wie diesen:

```
const martin = new Person();
martin.name = "martin";
martin.id = "1234";
```

Der Name einer Person kann sich nach dem Erstellen ändern, die ID hingegen nicht. Um das zu verdeutlichen, möchte ich die Setter-Methode für die ID entfernen.

Ich muss beim Erstellen nach wie vor eine ID zuweisen, also verwende ich *Funktionsdeklaration ändern* (Abschnitt 6.5), um sie dem Konstruktor hinzuzufügen:

```
class Person...
constructor(id) {
  this.id = id;
}
```

Anschließend passe ich das Erzeugungs-Skript an, sodass es die ID über den Konstruktor zuweist:

```
const martin = new Person("1234");
martin.name = "martin";
martin.id = "1234";
```

Ich nehme diese Anpassung an allen Stellen vor, an denen ich eine Person erzeuge, und führe nach jeder Änderung Tests durch. Wenn das erledigt ist, kann ich *Funktion inline platzieren* (Abschnitt 6.2) auf die Setter-Methode anwenden:

```
class Person...
constructor(id) {
  this._id = id;
}
get name() {return this._name;}
set name(arg) {this._name = arg;}
get id() {return this._id;}
set id(arg) {this._id = arg;}
```

11.8 Konstruktor durch Fabrikfunktion ersetzen (Replace Constructor with Factory Function)

Ehemalige Bezeichnung: *Konstruktor durch Fabrikmethode ersetzen* (Replace Constructor with Factory Method)

```
leadEngineer = new Employee(document.leadEngineer,
'E');
```

```
leadEngineer = createEngineer(document.leadEngineer);
```

11.8.1 MOTIVATION

Viele objektorientierte Programmiersprachen bieten eine spezielle Konstruktorfunktion, die zur Initialisierung eines Objekts aufgerufen wird. Clients rufen diesen Konstruktor typischerweise auf, wenn sie ein neues Objekt erzeugen möchten. Für diese Konstruktoren gelten aber oft ärgerliche Beschränkungen, denen allgemeinere Funktionen nicht unterliegen. In Java muss ein Konstruktor eine Instanz der Klasse zurückgeben, für die er aufgerufen wurde. Das bedeutet, dass je nach Umgebung oder Parametern weder eine Unterklasse noch ein Proxy-Objekt eingesetzt werden können. Der Name des Konstruktors ist fix, deshalb kann ich keinen

Namen verwenden, der aussagekräftiger als die Standardbezeichnung ist. Für den Aufruf des Konstruktors ist meist ein besonderer Operator vorgesehen (in vielen Sprachen »new«), wodurch es erschwert wird, Konstruktoren in einem Kontext zu verwenden, der normale Funktionen erwartet.

Eine Fabrikfunktion unterliegt diesen Einschränkungen nicht. Ihre Implementierung enthält in den meisten Fällen einen Aufruf des Konstruktors. Ich habe jedoch die freie Wahl, etwas anderes zu verwenden.

11.8.2 VORGEHEN

- Erstellen Sie eine Fabrikfunktion, die einen Aufruf des Konstruktors enthält.
- Ersetzen Sie alle Aufrufe des Konstruktors durch Aufrufe der Fabrikfunktion.
- Testen Sie nach jeder Änderung.
- Begrenzen Sie die Sichtbarkeit des Konstruktors so weit wie möglich.

11.8.3 BEISPIEL

Ein kurzes, aber eintöniges Beispiel verwendet verschiedene Arten von Angestellten. Betrachten Sie die folgende `Employee`-Klasse:

```
class Employee...
constructor (name, typeCode) {
  this._name = name;
```

```
  this._typeCode = typeCode;
}
get name() {return this._name;}
get type() {
  return Employee.legalTypeCodes[this._typeCode];
}
static get legalTypeCodes() {
  return {"E": "Engineer", "M": "Manager", "S":
"Salesman"};
}
```

Diese Klasse wird wie folgt verwendet:

```
Aufrufer...
candidate = new Employee(document.name,
document.empType);
```

Und so:

```
Aufrufer...
const leadEngineer = new
Employee(document.leadEngineer,'E');
```

Als Erstes erstelle ich eine Fabrikfunktion, die den Konstruktor aufruft:

```
Oberste Ebene...
function createEmployee(name, typeCode) {
  return new Employee(name, typeCode);
}
```

Anschließend suche ich nach den Aufrufern des Konstruktors und ändere sie der Reihe nach, sodass sie stattdessen die Fabrikfunktion verwenden.

Der erste Fall ist eindeutig:

```
Aufrufer...
candidate = createEmployee(document.name,
document.empType);
```

Beim zweiten Fall könnte ich die neue Fabrikfunktion wie folgt verwenden:

```
Aufrufer...
const leadEngineer =
                createEmployee(document.leadEngineer,
'E');
```

Ich möchte den Typenschlüssel hier allerdings nicht verwenden – einen Typenschlüssel als Stringliteral zu übergeben, ist im Allgemeinen ein Code-Smell. Also ziehe ich es vor, eine neue Fabrikfunktion zu erstellen, deren Name die Art von Angestelltem enthält, die erzeugt werden soll.

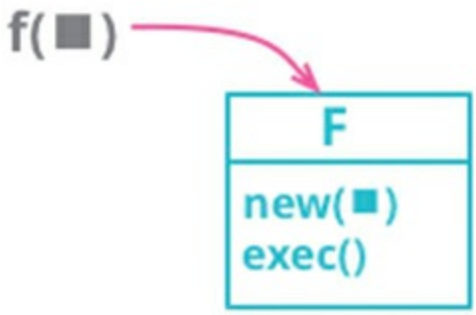
```
Aufrufer...
const leadEngineer =
createEngineer(document.leadEngineer);
```

```
Oberste Ebene...
function createEngineer(name) {
  return new Employee(name, 'E');
}
```

11.9 Funktion durch Befehl ersetzen (Replace Function with Command)

Ehemalige Bezeichnung: *Methode durch Methodenobjekt ersetzen* (Replace Method with Method Object)

Umkehrung von: *Befehl durch Funktion ersetzen* (Replace Command with Function, Abschnitt 11.10)




```
function score(candidate, medicalExam, scoringGuide) {
  let result = 0;
  let healthLevel = 0;
  // jede Menge Code
}
```

```
class Scorer {
  constructor(candidate, medicalExam, scoringGuide) {
    this._candidate = candidate;
    this._medicalExam = medicalExam;
    this._scoringGuide = scoringGuide;
  }

  execute() {
    this._result = 0;
    this._healthLevel = 0;
    // jede Menge Code
  }
}
```

11.9.1 MOTIVATION

Funktionen, entweder eigenständig oder als Methoden im Kontext von Objekten, sind fundamentale Bausteine der Programmierung. Manchmal erweist es sich jedoch als nützlich, eine Funktion in ihrem eigenen Objekt zu kapseln, das ich als »Befehlsobjekt« oder einfach nur *Befehl* bezeichne. Ein solches Objekt besteht im Wesentlichen aus einer einzelnen Methode, zu deren Aufruf und Ausführung das Objekt dient.

Ein Befehlsobjekt bietet bei der Steuerung und Formulierung einer Funktion größere Flexibilität als der herkömmliche Funktionsmechanismus. Befehlsobjekte können ergänzende Operationen besitzen, wie beispielsweise das Rückgängigmachen von Operationen. Ich kann Methoden zum Parametrisieren bereitstellen, um einen vielfältigen Verwendungszweck zu

unterstützen. Ich kann Anpassungen mit Hilfe von Vererbung und Delegation vornehmen. Wenn ich eine Programmiersprache benutze, die zwar Objekte, aber keine First-Class-Funktionen unterstützt, kann ich viele der Fähigkeiten durch die Verwendung von Befehlsobjekten bereitstellen. Zudem kann ich Methoden und Felder verwenden, um komplexe Funktionen aufzuteilen, selbst in Programmiersprachen, die keine verschachtelten Funktionen unterstützen, und ich kann diese Methoden beim Testen und Debuggen direkt aufrufen.

Das sind viele gute Gründe, die für Befehlsobjekte sprechen, und ich sollte bereit sein, bei Bedarf Refactorings anzuwenden, um aus einer Funktion ein Befehlsobjekt zu machen. Wir dürfen aber nicht vergessen, dass diese Flexibilität wie immer nicht umsonst zu haben ist, denn die Komplexität nimmt zu. Hätte ich die Wahl zwischen einer First-Class-Funktion und einem Befehlsobjekt, würde ich mich in 95% der Fälle für die Funktion entscheiden. Ich verwende Befehlsobjekte nur dann, wenn ich eine bestimmte Fähigkeit benötige, die einfachere Ansätze nicht bieten können.

Wie so viele Begriffe in der Softwareentwicklung, besitzt »Befehl« eine Vielzahl von Bedeutungen. In diesem Kontext ist damit ein Objekt gemeint, das eine Abfrage gemäß des Entwurfsmusters *Befehl* (Command) kapselt (Erich Gamma, Richard Helm, Ralph Johnson und John

Vlissides: *Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software*, mitp-Verlag 2015, ISBN 9783826697005). Wenn ich »Befehl« in diesem Sinne gebrauche, verwende ich zunächst den Ausdruck »Befehlsobjekt«, um auf den Zusammenhang hinzuweisen und anschließend »Befehl«. Der Ausdruck »Befehl« findet darüber hinaus auch noch im Prinzip der Trennung von Befehl und Abfrage (`https://martinfowler.com/bliki/CommandQuerySeparation.html`) Verwendung, bei dem ein Befehl eine Objektmethode ist, die den sichtbaren Zustand verändert. Ich habe versucht, zu vermeiden, den Begriff in diesem Sinne zu verwenden.

11.9.2 VORGEHEN

- Erstellen Sie eine leere Klasse für die Funktion. Vergeben Sie einen Namen, der auf der Aufgabe der Funktion beruht.
- Verwenden Sie *Funktion verschieben* (Abschnitt 8.1), um die Funktion in die leere Klasse zu verschieben.

  Erhalten Sie die ursprüngliche Funktion als Weiterleitungsfunktion mindestens bis zum Abschluss dieses Refactorings.

  Halten Sie sich an die Konventionen der Programmiersprache zur Benennung von Befehlen. Wenn es keine Konvention gibt, können Sie einen allgemeinen Namen für die Ausführungsmethode des Befehls wählen, wie beispielsweise »execute« oder »call«.
- Ziehen Sie in Betracht, für alle Argumente ein Feld anzulegen und die Argumente in den Konstruktor zu verschieben.

11.9.3 BEISPIEL

Die Programmiersprache JavaScript weist einige Mängel auf, aber die Entscheidung für First-Class-Funktionen war goldrichtig. Ich muss mir also nicht – wie in Sprachen, die First-Class-Funktionen nicht unterstützen – die Mühe machen, Befehle für gängige Aufgaben zu erstellen. Es kommt aber dennoch vor, dass ein Befehlsobjekt für eine bestimmte Aufgabe das richtige Werkzeug ist.

Das ist beispielsweise der Fall, wenn eine komplexe Funktion aufgeteilt wird, sodass ich sie besser verstehen und leichter ändern kann. Um die wahre Stärke dieses Refactorings demonstrieren zu können, benötige ich eine lange und komplizierte Funktion – es würde allerdings zu viel Zeit in Anspruch nehmen, sie zu schreiben oder sie nachzuvollziehen. Stattdessen verwende ich eine Funktion, die kurz genug ist, um nicht zu umständlich zu sein. Es geht um die Zuweisung von Punkten in der Anwendung einer Krankenversicherung:

```
Function score(candidate, medicalExam, scoringGuide) {
  let result = 0;
  let healthLevel = 0;
  let highMedicalRiskFlag = false;

  if (medicalExam.isSmoker) {
    healthLevel += 10;
    highMedicalRiskFlag = true;
  }
  let certificationGrade = "regular";
  if
(scoringGuide.stateWithLowCertification(candidate.orig
```

```
inState)) {
    certificationGrade = "low";
    result -= 5;
  }
  // weiterer ähnlicher Code
  result -= Math.max(healthLevel – 5, 0);
  return result;
}
```

Ich erstelle eine leere Klasse und verwende *Funktion verschieben* (Abschnitt 8.1), um die Funktion in die Klasse zu verschieben:

```
Function score(candidate, medicalExam, scoringGuide) {
  return new Scorer().execute(candidate,
                              medicalExam,
scoringGuide);
}

class Scorer {
  execute (candidate, medicalExam, scoringGuide) {
    let result = 0;
    let healthLevel = 0;
    let highMedicalRiskFlag = false;

    if (medicalExam.isSmoker) {
      healthLevel += 10;
      highMedicalRiskFlag = true;
    }
    let certificationGrade = "regular";
    if
(scoringGuide.stateWithLowCertification(candidate.orig
inState)) {
      certificationGrade = "low";
      result -= 5;
    }
```

```
    // weiterer ähnlicher Code
    result -= Math.max(healthLevel - 5, 0);
    return result;
  }
}
```

In den meisten Fällen übergebe ich die Argumente eines Befehls an den Konstruktor, die `execute`-Methode nimmt dann keine Parameter entgegen. Das spielt zwar bei einer einfachen Aufteilung wie dieser keine große Rolle, ist aber äußerst praktisch, wenn ich für den Befehl eine kompliziertere Parameterliste verwenden und größere Anpassungen vornehmen möchte. Unterschiedliche Befehlsklassen verwenden möglicherweise verschiedene Parameter, können aber gemischt werden, wenn sie zur Ausführung aneinandergereiht werden.

Ich bearbeite die Parameter der Reihe nach:

```
function score(candidate, medicalExam, scoringGuide) {
  return new Scorer(candidate).execute(candidate,
                                       medicalExam,
                                       scoringGuide);
}

class Scorer...
constructor(candidate){
  this._candidate = candidate;
}
execute (candidate, medicalExam, scoringGuide) {
  let result = 0;
  let healthLevel = 0;
```

```
  let highMedicalRiskFlag = false;

  if (medicalExam.isSmoker) {
    healthLevel += 10;
    highMedicalRiskFlag = true;
  }
  let certificationGrade = "regular";
  if
(scoringGuide.stateWithLowCertification(this._candidate
.originState)) {
    certificationGrade = "low";
    result -= 5;
  }
  // weiterer ähnlicher Code
  result -= Math.max(healthLevel - 5, 0);
  return result;
}
```

Ich fahre mit den verbleibenden Parametern fort.

```
function score(candidate, medicalExam, scoringGuide) {
  return new Scorer(candidate, medicalExam,
scoringGuide).execute();
}

class Scorer...
constructor(candidate, medicalExam, scoringGuide){
  this._candidate = candidate;
  this._medicalExam = medicalExam;
  this._scoringGuide = scoringGuide;
}
execute () {
  let result = 0;
  let healthLevel = 0;
  let highMedicalRiskFlag = false;
```

```
  if (this._medicalExam.isSmoker) {
    healthLevel += 10;
    highMedicalRiskFlag = true;
  }
  let certificationGrade = "regular";
  if
(this._scoringGuide.stateWithLowCertification(this._ca
ndidate.originState)) {
    certificationGrade = "low";
    result -= 5;
  }
  // weiterer ähnlicher Code
  result -= Math.max(healthLevel - 5, 0);
  return result;
}
```

Damit ist *Funktion durch Befehl ersetzen* abgeschlossen. Der eigentliche Sinn des Refactorings liegt jedoch darin, dass ich komplizierte Funktionen aufteilen kann. Deshalb zeige ich jetzt einige Schritte, mit denen sich das erreichen lässt. Im ersten Schritt werden aus allen lokalen Variablen Felder gemacht. Ich gehe wieder der Reihe nach vor:

```
class Scorer...
constructor(candidate, medicalExam, scoringGuide){
  this._candidate = candidate;
  this._medicalExam = medicalExam;
  this._scoringGuide = scoringGuide;
}

execute () {
  this._result = 0;
  let healthLevel = 0;
```

```
  let highMedicalRiskFlag = false;

  if (this._medicalExam.isSmoker) {
    healthLevel += 10;
    highMedicalRiskFlag = true;
  }
  let certificationGrade = "regular";
  if
(this._scoringGuide.stateWithLowCertification(this._ca
ndidate.originState)) {
    certificationGrade = "low";
    this._result -= 5;
  }
  // weiterer ähnlicher Code
  this._result -= Math.max(healthLevel - 5, 0);
  return this._result;
}
```

Ich gehe bei den übrigen lokalen Variablen auf die gleiche Weise vor (hierbei handelt es sich um eins der Refactorings, die mir so einfach erschienen, dass ich sie nicht in den Katalog aufgenommen habe; ich fühle mich deswegen ein wenig schuldig):

```
class Scorer...
constructor(candidate, medicalExam, scoringGuide){
  this._candidate = candidate;
  this._medicalExam = medicalExam;
  this._scoringGuide = scoringGuide;
}

execute () {
  this._result = 0;
  this._healthLevel = 0;
  this._highMedicalRiskFlag = false;
```

```
  if (this._medicalExam.isSmoker) {
    this._healthLevel += 10;
    this._highMedicalRiskFlag = true;
  }
  this._certificationGrade = "regular";
  if
(this._scoringGuide.stateWithLowCertification(this._ca
ndidate.originState)) {
    this._certificationGrade = "low";
    this._result -= 5;
  }
  // weiterer ähnlicher Code
  result -= Math.max(this._healthLevel - 5, 0);
  return this._result;
}
```

Jetzt habe ich alle Zustände der Funktion in das Befehlsobjekt verschoben und kann Refactorings wie *Funktion extrahieren* (Abschnitt 6.1) anwenden, ohne dass ich all die Variablen und ihre Gültigkeitsbereiche durcheinanderbringe:

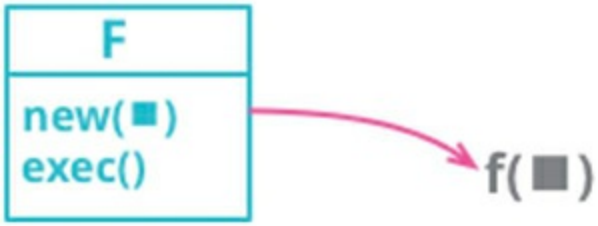
```
class Scorer...
constructor(candidate, medicalExam, scoringGuide){
  this._candidate = candidate;
  this._medicalExam = medicalExam;
  this._scoringGuide = scoringGuide;
}

execute () {
  this._result = 0;
  this._healthLevel = 0;
  this._highMedicalRiskFlag = false;
```

```
  this.scoreSmoking();
  this._certificationGrade = "regular";
  if
(this._scoringGuide.stateWithLowCertification(this._ca
ndidate.originState)) {
    this._certificationGrade = "low";
    this._result -= 5;
  }
  // weiterer ähnlicher Code
  result -= Math.max(this._healthLevel - 5, 0);
  return this._result;
}
scoreSmoking() {
if (this._medicalExam.isSmoker) {
  this._healthLevel += 10;
  this._highMedicalRiskFlag = true;
  }
}
```

Auf diese Weise kann ich mit dem Befehl ähnlich wie mit einer verschachtelten Funktion verfahren. Tatsächlich wären verschachtelte Funktionen in JavaScript eine sinnvolle Alternative zur Verwendung eines Befehls. Ich nutze hier dennoch einen Befehl, zum einen, weil mir Befehlsobjekte vertrauter sind, und zum anderen, weil ich Tests und Debugging der Unterfunktionen vornehmen kann.

11.10 Befehl durch Funktion ersetzen (Replace Command with Function)

Umkehrung von: *Funktion durch Befehl ersetzen* (Replace Function with Command, Abschnitt 11.9)




```
class ChargeCalculator {
  constructor (customer, usage){
    this._customer = customer;
    this._usage = usage;
  }
  execute() {
    return this._customer.rate * this._usage;
  }
}
```

```
function charge(customer, usage) {
  return customer.rate * usage;
}
```

11.10.1 MOTIVATION

Befehlsobjekte bieten einen leistungsfähigen Mechanismus zur Handhabung komplexer Berechnungen. Sie lassen sich leicht in einzelne Methoden aufteilen, die in Form von Feldern einen gemeinsamen Zustand verwenden; sie können auf unterschiedliche Weise für verschiedene Zwecke

aufgerufen werden; sie können ihre Daten stufenweise aufbauen. Diese Leistungsfähigkeit hat allerdings ihren Preis. In den meisten Fällen möchte ich eine Funktion einfach nur aufrufen, damit sie ihre Aufgabe erledigt. Wenn dem so ist und die Funktion nicht allzu komplex ist, lohnt sich der Aufwand für ein Befehlsobjekt nicht, und es sollte in eine normale Funktion umgewandelt werden.

11.10.2 VORGEHEN

- Wenden Sie *Funktion extrahieren* (Abschnitt 6.1) auf das Befehlsobjekt und den Aufruf der Methode zur Ausführung an.

  Hiermit wird die Funktion erstellt, die den Befehl zu gegebener Zeit ersetzt.

- Wenden Sie auf alle Funktionen, die von der Methode zum Ausführen des Befehls (z.B. »execute«) aufgerufen werden, *Funktion inline platzieren* (Abschnitt 6.2) an.

  Falls eine dieser Funktionen einen Wert zurückgibt, können Sie zuerst *Variable extrahieren* (Abschnitt 6.3) auf den Aufruf anwenden und anschließend *Funktion inline platzieren* (Abschnitt 6.2) einsetzen.

- Verwenden Sie *Funktionsdeklaration ändern* (Abschnitt 6.5), um die Parameter des Konstruktors in die Methode zum Ausführen des Befehls zu verlagern.

- Ersetzen Sie in der Methode zum Ausführen des Befehls Verweise auf Felder durch die entsprechenden Parameter. Führen Sie nach jeder Änderung Tests durch.

- Platzieren Sie den Aufruf des Konstruktors und den Aufruf der Methode zum Ausführen des Befehls inline in die ersetzende Funktion.

- Testen Sie.

- Wenden Sie *Toten Code entfernen* (Abschnitt 8.9) auf die Befehlsklasse an.

11.10.3 BEISPIEL

Ich verwende das folgende kleine Befehlsobjekt:

```
class ChargeCalculator {
  constructor (customer, usage, provider) {
    this._customer = customer;
    this._usage = usage;
    this._provider = provider;
  }
  get baseCharge() {
    return this._customer.baseRate * this._usage;
  }
  get charge() {
    return this.baseCharge +
this._provider.connectionCharge;
  }
}
```

Es wird von Code wie diesem genutzt:

```
Aufrufer...
monthCharge = new ChargeCalculator(customer, usage,
                                   provider).charge;
```

Die Befehlsklasse ist so kurz und einfach, dass eine Funktion besser geeignet ist. Ich wende zunächst einmal *Funktion extrahieren* (Abschnitt 6.1) an, um die Klassenerstellung und den Aufruf auszulagern:

```
Aufrufer...
monthCharge = charge(customer, usage, provider);

Oberste Ebene...
function charge(customer, usage, provider) {
  return new ChargeCalculator(customer, usage,
                              provider).charge;
}
```

Ich muss entscheiden, wie ich mit Hilfsfunktionen wie `baseCharge` verfahre. Bei einer Funktion, die einen Wert zurückgibt, verfolge ich für gewöhnlich den Ansatz, zuerst *Variable extrahieren* (Abschnitt 6.3) auf den fraglichen Wert anzuwenden:

```
class ChargeCalculator...
get baseCharge() {
  return this._customer.baseRate * this._usage;
}
get charge() {
  const baseCharge = this.baseCharge;
  return baseCharge + this._provider.connectionCharge;
}
```

Anschließend wende ich *Funktion inline platzieren* (Abschnitt 6.2) auf die Hilfsfunktion an:

```
class ChargeCalculator...
get charge() {
  const baseCharge = this._customer.baseRate *
this._usage;
```

```
  return baseCharge + this._provider.connectionCharge;
}
```

Nun finden alle Verarbeitungsschritte in einer einzelnen Funktion statt, also verschiebe ich die an den Konstruktor übergebenen Daten in die Hauptmethode. Zuerst wende ich *Funktionsdeklaration ändern* (Abschnitt 6.5) an, um die Parameter des Konstruktors der `charge`-Methode hinzuzufügen:

```
class ChargeCalculator...
constructor (customer, usage, provider) {
  this._customer = customer;
  this._usage = usage;
  this._provider = provider;
}

charge(customer, usage, provider) {
  const baseCharge = this._customer.baseRate *
this._usage;
  return baseCharge + this._provider.connectionCharge;
}

Oberste Ebene...
function charge(customer, usage, provider) {
  return new ChargeCalculator(customer, usage,
provider)
                      .charge(customer, usage,
provider);
}
```

Jetzt kann ich `charge` so anpassen, dass die übergebenen Parameter verwendet werden. Ich gehe

dabei wieder der Reihe nach vor:

```
class ChargeCalculator...
constructor (customer, usage, provider){
  this._customer = customer;
  this._usage = usage;
  this._provider = provider;
}

charge(customer, usage, provider) {
  const baseCharge = customer.baseRate * this._usage;
  return baseCharge + this._provider.connectionCharge;
}
```

Ich hätte die Zuweisung zu `this._customer` im Konstruktor eigentlich gar nicht löschen müssen, da sie keine Rolle mehr spielt. Ich entferne sie dennoch, weil es zum Fehlschlagen eines Tests führen könnte, wenn ich die Umstellung von der Verwendung des Felds auf die Verwendung des Parameters irgendwo vergesse. (Und wenn kein Test fehlschlägt, sollte ich in Betracht ziehen, einen weiteren Test hinzuzufügen.)

Ich wiederhole dieses Vorgehen mit den übrigen Parametern, womit sich Folgendes ergibt:

```
class ChargeCalculator...
charge(customer, usage, provider) {
  const baseCharge = customer.baseRate * usage;
  return baseCharge + provider.connectionCharge;
}
```

Nachdem alles erledigt ist, kann ich die `charge`-Funktion auf oberster Ebene inline platzieren. Hierbei handelt es sich um einen Spezialfall von *Funktion inline platzieren* (Abschnitt 6.2), denn Konstruktor und Methodenaufruf werden zusammen inline platziert:

```
Oberste Ebene...
function charge(customer, usage, provider) {
  const baseCharge = customer.baseRate * usage;
  return baseCharge + provider.connectionCharge;
}
```

Die Befehlsklasse ist jetzt toter Code, also verwende ich *Toten Code entfernen* (Abschnitt 8.9), damit sie in Würde von uns gehen kann.

Kapitel 12

Der Umgang mit Vererbung

In diesem letzten Kapitel befasse ich mich mit einem der bekanntesten Aspekte der objektorientierten Programmierung: der Vererbung. Wie jeder mächtige Mechanismus ist die Vererbung einerseits äußerst nützlich, lässt sich andererseits aber auch leicht missbrauchen. Zudem ist die unsachgemäße Verwendung häufig schwer zu erkennen, bis einen plötzlich die Folgen einholen.

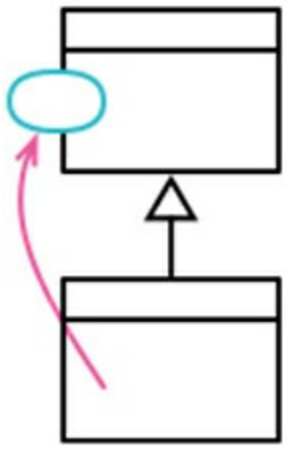
In vielen Fällen soll Funktionalität in der Vererbungshierarchie nach oben oder unten verschoben werden. Zu diesem Zweck stehen verschiedene Refactorings zur Verfügung: *Methode nach oben verschieben* (Abschnitt 12.1), *Feld nach oben verschieben* (Abschnitt 12.2), *Konstruktorrumpf nach oben verschieben* (Abschnitt 12.3), *Methode nach unten verschieben* (Abschnitt 12.4) und *Feld nach unten verschieben* (Abschnitt 12.5). Mit *Basisklasse extrahieren* (Abschnitt 12.8), *Unterklasse entfernen* (Abschnitt 12.7) und *Hierarchie abbauen* (Abschnitt 12.9) kann ich Klassen zu der Hierarchie hinzufügen oder entfernen. Wenn ich ein Feld, dessen Wert unterschiedliches Verhalten auslöst, durch eine

Unterklasse ersetzen möchte, kann ich dafür *Typenschlüssel durch Unterklassen ersetzen* (Abschnitt 12.6) verwenden.

Die Vererbung ist ein leistungsfähiges Werkzeug, das allerdings manchmal im falschen Umfeld eingesetzt wird – oder das Umfeld verändert sich und ist für den Einsatz nunmehr ungeeignet. Wenn das der Fall ist, verwende ich *Unterklasse durch Delegation ersetzen* (Abschnitt 12.10) oder *Basisklasse durch Delegation ersetzen* (Abschnitt 12.11), um die Vererbung in eine Delegation umzuwandeln.

12.1 Methode nach oben verschieben (Pull Up Method)

Umkehrung von: *Methode nach unten verschieben* (Push Down Method, Abschnitt 12.4)



```
class Employee {...}
```

```
class Salesman extends Employee {
  get name() {...}
}

class Engineer extends Employee {
  get name() {...}
}
```

```
class Employee {
  get name() {...}
}
class Salesman extends Employee {...}
class Engineer extends Employee {...}
```

12.1.1 MOTIVATION

Das Entfernen von redundantem Code ist wichtig. Dieser mag im gegenwärtigen Zustand zwar tadellos funktionieren, stellt jedoch eine Brutstätte für zukünftige Fehler dar. Wenn Redundanzen vorhanden sind, läuft man ständig Gefahr, dass die Änderung an einer Kopie bei einer anderen Kopie nicht vorgenommen wird. Für gewöhnlich ist es zudem schwierig, redundanten Code aufzuspüren.

Im einfachsten Anwendungsfall von *Methode nach oben verschieben* ist der Inhalt der Methoden identisch, was

dafür spricht, dass er durch simples Kopieren entstanden ist. Natürlich ist es nicht immer so offensichtlich. Ich könnte das Refactoring einfach vornehmen und prüfen, ob die Tests bestanden werden. Die Tests müssen dann allerdings sehr zuverlässig sein. Meistens ist es sinnvoll, nach Unterschieden in den Methoden Ausschau zu halten, denn häufig weisen diese auf Verhalten hin, das ich vergessen habe zu testen.

In vielen Fällen ist *Methode nach oben verschieben* erst ein nachfolgender Schritt. Wenn sich zwei Methoden in verschiedenen Klassen so parametrisieren lassen, dass sie letztendlich im Wesentlichen identisch sind, wende ich jeweils *Funktion parametrisieren* (Abschnitt 11.2) an und setze anschließend *Methode nach oben verschieben* ein.

Methode nach oben verschieben kann ziemlich kompliziert werden, wenn die Methode Verweise auf Eigenschaften enthält, die in der Unterklasse vorhanden sind, aber nicht in der Basisklasse. In diesem Fall muss ich zuerst *Feld nach oben verschieben* (Abschnitt 12.2) und kann erst danach *Methode nach oben verschieben* auf diese Elemente anwenden.

Wenn zwei Methoden sich im Großen und Ganzen ähneln, aber in Details unterscheiden, ziehe ich in Betracht, Form-Template-Method (`https://refactoring.com/catalog/formTemplateMethod.html`) anzuwenden.

12.1.2 VORGEHEN

- Vergewissern Sie sich, dass die Methoden identisch sind.

  Wenn die Funktionen die gleiche Aufgabe erledigen, aber nicht identisch sind, sollten Sie Refactorings vornehmen, bis sie identisch sind.

- Vergewissern Sie sich, dass alle Aufrufe und Verweise auf Felder innerhalb der Methode auch der Basisklasse zugänglich sind.
- Wenn sich die Signaturen der Methoden unterscheiden, können Sie *Funktionsdeklaration ändern* (Abschnitt 6.5) verwenden, um sie an diejenige anzupassen, die Sie in der Basisklasse verwenden möchten.
- Erstellen Sie in der Basisklasse eine neue Methode und kopieren Sie den Inhalt einer der Methoden in die neue Methode.
- Führen Sie statische Überprüfungen durch.
- Löschen Sie eine der Unterklassenmethoden.
- Testen Sie.
- Fahren Sie mit dem Löschen der Unterklassenmethoden fort, bis alle entfernt worden sind.

12.1.3 BEISPIEL

Diese beiden Unterklassenmethoden erledigen die gleiche Aufgabe:

```
class Employee extends Party...
get annualCost() {
  return this.monthlyCost * 12;
}

class Department extends Party...
get totalAnnualCost() {
```

```
  return this.monthlyCost * 12;
}
```

Wenn ich die beiden Klassen betrachte, stelle ich fest, dass sie die Eigenschaft `monthlyCost` verwenden, die nicht zur Basisklasse gehört, aber in beiden Unterklassen vorhanden ist. Da ich eine dynamische Sprache verwende, ist das in Ordnung. Bei einer statischen Sprache müsste ich an dieser Stelle eine abstrakte Methode in der Klasse `Party` definieren.

Die Methoden besitzen unterschiedliche Namen, also verwende ich *Funktionsdeklaration ändern* (Abschnitt 6.5), um sie anzugleichen:

```
class Department...
get annualCost() {
  return this.monthlyCost * 12;
}
```

Ich kopiere den Inhalt einer der Unterklassen und setze ihn in der Basisklasse ein:

```
class Party...
get annualCost() {
  return this.monthlyCost * 12;
}
```

Bei einer statischen Sprache würde ich jetzt kompilieren,

um zu gewährleisten, dass alle Verweise richtig funktionieren. Das würde mir hier nicht helfen, deshalb entferne ich `annualCost` zunächst aus `Employee` und führe Tests durch. Anschließend entferne ich `annualCost` auch aus `Department`.

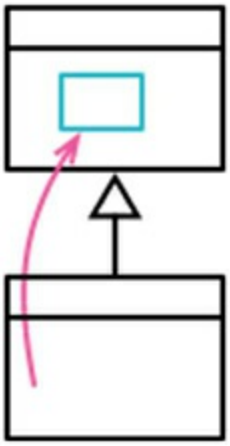
Das Refactoring ist damit abgeschlossen. Eine Frage ist jedoch noch offen. `annualCost` ruft `monthlyCost` auf, aber `annualCost` ist in der Klasse `Party` gar nicht vorhanden. Das Ganze funktioniert, weil JavaScript eine dynamische Sprache ist – aber es ist sinnvoll, zu verdeutlichen, dass Unterklassen von `Party` eine Implementierung von `monthlyCost` bereitstellen sollten – insbesondere dann, wenn später weitere Unterklassen hinzugefügt werden. Eine Methode wie die folgende ist gut dafür geeignet, auf diesen Umstand hinzuweisen:

```
class Party...
get monthlyCost() {
throw new SubclassResponsibilityError();
}
```

Ich verwende für solch einen Fehler die Bezeichnung `SubclassResponsibilityError` (der Fehler wird durch die fehlende Verantwortlichkeit in der Unterklasse ausgelöst), weil er in Smalltalk so hieß.

12.2 Feld nach oben verschieben (Pull Up Field)

Umkehrung von: *Feld nach unten verschieben* (Push Down Field, Abschnitt 12.5)



```
class Employee {...} // Java
class Salesman extends Employee {
  private String name;
}

class Engineer extends Employee {
  private String name;
}
```

```
class Employee {
  protected String name;
}
```

```
class Salesman extends Employee {...}
class Engineer extends Employee {...}
```

12.2.1 MOTIVATION

Wenn Unterklassen unabhängig voneinander entwickelt oder durch ein Refactoring kombiniert werden, muss ich oft feststellen, dass einige Merkmale redundant sind. Davon sind bestimmte Felder besonders betroffen. Solche Felder tragen oft ähnliche Bezeichnungen – aber nicht immer. Wenn ich den Tatsachen auf den Grund gehen möchte, habe ich keine andere Möglichkeit, als mir die Felder und ihre Verwendung anzusehen. Falls sie auf ähnliche Weise verwendet werden, kann ich sie in die Basisklasse verschieben.

Dadurch reduziere ich Redundanzen im Code auf zweierlei Weise. Ich entferne die mehrfach vorhandene Datendeklaration und kann darüber hinaus das Verhalten, das Verwendung vom Feld macht, von der Unterklasse in die Basisklasse verschieben.

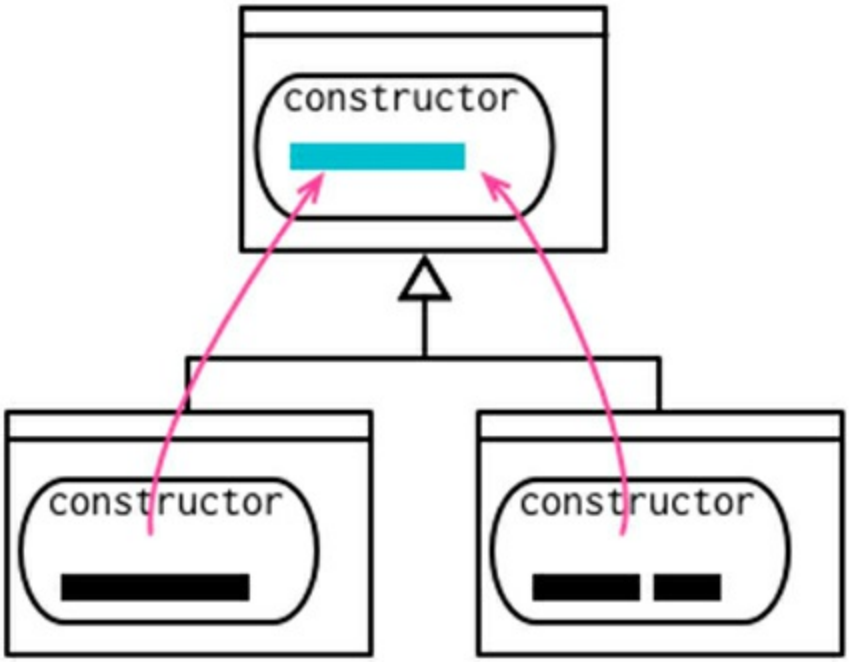
Bei vielen dynamischen Sprachen sind Felder kein Bestandteil der Klassendefinition – stattdessen tauchen sie auf, sobald ihnen erstmals ein Wert zugewiesen wird. In diesem Fall ist das Verschieben eines Feldes nach oben eine Folge des Refactorings *Konstruktorrumpf nach oben verschieben* (Abschnitt 12.3).

12.2.2 VORGEHEN

- Vergewissern Sie sich, dass die Verwendung des betreffenden Feldes überall auf die gleiche Weise erfolgt.
- Wenn sich die Namen der Felder unterscheiden, verwenden Sie zum Angleichen der Namen *Feld umbenennen* (Abschnitt 9.2).
- Erstellen Sie in der Basisklasse ein neues Feld.

  Das neue Feld muss für Unterklassen zugänglich sein (`protected` in gängigen Programmiersprachen).
- Löschen Sie die Felder der Unterklassen.
- Testen Sie.

12.3 Konstruktorrumpf nach oben verschieben (Pull Up Constructor Body)




```
class Party {...}

class Employee extends Party {
  constructor(name, id, monthlyCost) {
```

```
    super();
    this._id = id;
    this._name = name;
    this._monthlyCost = monthlyCost;
  }
}
```

```
class Party {
  constructor(name) {
    this._name = name;
  }
}

class Employee extends Party {
  constructor(name, id, monthlyCost) {
    super(name);
    this._id = id;
    this._monthlyCost = monthlyCost;
  }
}
```

12.3.1 MOTIVATION

Konstruktoren sind knifflig, denn es sind keine ganz normalen Methoden – deshalb sind meine Möglichkeiten innerhalb des Konstruktors eingeschränkt.

Wenn ich auf Unterklassenmethoden mit gemeinsamem Verhalten stoße, ist mein erster Gedanke, *Funktion extrahieren* (Abschnitt 6.1) mit nachfolgendem *Methode*

nach oben verschieben (Abschnitt 12.1) zu verwenden, um dieses Verhalten in die Basisklasse zu verlagern. Konstruktoren sind dabei hinderlich, weil für sie spezielle Regeln gelten, was in welcher Reihenfolge erfolgen muss, deshalb muss ich einen etwas anderen Ansatz verfolgen.

Wenn dieses Refactoring chaotisch zu werden droht, greife ich auf *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8) zurück.

12.3.2 VORGEHEN

- Definieren Sie in der Basisklasse einen Konstruktor, falls es noch keinen gibt. Vergewissern Sie sich, dass er von den Konstruktoren der Unterklassen aufgerufen wird.
- Verwenden Sie *Anweisungen verschieben* (Abschnitt 8.6), um die gemeinsamen Anweisungen direkt hinter den Aufruf von `super` zu verschieben.
- Entfernen Sie den gemeinsamen Code aus den Unterklassen und verlagern Sie ihn in die Basisklasse. Fügen Sie dem Aufruf von `super` die Konstruktorparameter hinzu, auf die der gemeinsame Code verweist.
- Testen Sie.
- Wenn es gemeinsamen Code gibt, der nicht an den Anfang des Konstruktors verschoben werden kann, verwenden Sie *Funktion extrahieren* (Abschnitt 6.1), gefolgt von *Methode nach oben verschieben* (Abschnitt 12.1).

12.3.3 BEISPIEL

Ausgangspunkt ist der folgende Code:

```
class Party {}

class Employee extends Party {
  constructor(name, id, monthlyCost) {
    super();
    this._id = id;
    this._name = name;
    this._monthlyCost = monthlyCost;
  }
  // übrige Klasse ...

class Department extends Party {
  constructor(name, staff){
    super();
    this._name = name;
    this._staff = staff;
  }
  // übrige Klasse ...
```

Der gemeinsame Code ist hier die Zuweisung des Namens. Ich verwende *Anweisungen verschieben* (Abschnitt 8.6), um die Zuweisung in `Employee` hinter den Aufruf von `super()` zu verschieben:

```
class Employee extends Party {
  constructor(name, id, monthlyCost) {
    super();
    this._name = name;
    this._id = id;
    this._monthlyCost = monthlyCost;
  }
  // übrige Klasse ...
```

Nachdem ich dies getestet habe, verschiebe ich den gemeinsamen Code in die Basisklasse. Da der Code einen Verweis auf ein Konstruktorargument enthält, übergebe ich diesen als Parameter:

```
class Party...
constructor(name){
  this._name = name;
}

class Employee...
constructor(name, id, monthlyCost) {
  super(name);
  this._id = id;
  this._monthlyCost = monthlyCost;
}

class Department...
  constructor(name, staff){
  super(name);
  this._staff = staff;
}
```

Ich führe Tests durch, und damit ist die Aufgabe erledigt.

In den meisten Fällen läuft das Verhalten des Konstruktors folgendermaßen ab: Erledigen Sie zuerst den gemeinsamen Teil (durch Aufruf von `super`) und anschließend die Besonderheiten, die für die jeweilige Unterklasse erforderlich sind. Gelegentlich kommt es auch vor, dass später weiteres gemeinsames Verhalten auftritt.

Betrachten Sie das folgende Beispiel:

```
class Employee...
constructor (name) {...}
get isPrivileged() {...}
assignCar() {...}

class Manager extends Employee...
constructor(name, grade) {
  super(name);
  this._grade = grade;
  if (this.isPrivileged) this.assignCar();
    // wird in allen Unterklassen ausgeführt
}

get isPrivileged() {
  return this._grade > 4;
}
```

Der Haken dabei ist, dass der Aufruf von `isPriviliged` erst erfolgen kann, nachdem dem Feld `grade` etwas zugewiesen wurde. Diese Zuweisung wiederum kann nur in der Unterklasse erfolgen.

In diesem Fall wende ich *Funktion extrahieren* (Abschnitt 6.1) auf den gemeinsamen Code an:

```
class Manager...
constructor(name, grade) {
  super(name);
  this._grade = grade;
  this.finishConstruction();
}
```

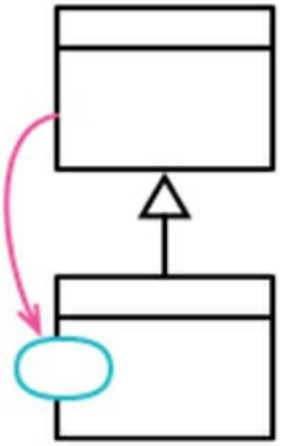
```
finishConstruction() {
  if (this.isPrivileged) this.assignCar();
}
```

Anschließend verwende ich *Methode nach oben verschieben* (Abschnitt 12.1) zum Verschieben in die Basisklasse:

```
class Employee...
finishConstruction() {
  if (this.isPrivileged) this.assignCar();
}
```

12.4 Methode nach unten verschieben (Push Down Method)

Umkehrung von: *Methode nach oben verschieben* (Pull Up Method, Abschnitt 12.1)



```
class Employee {
  get quota {...}
}
class Engineer extends Employee {...}
class Salesman extends Employee {...}
```

```
class Employee {...}
class Engineer extends Employee {...}
class Salesman extends Employee {
  get quota {...}
}
```

12.4.1 MOTIVATION

Sollte eine Methode nur für eine Unterklasse (oder einen geringen Anteil der Unterklassen) von Bedeutung sein, ist dies besser ersichtlich, wenn sie aus der Basisklasse entfernt und in die Unterklasse(n) verlagert wird. Ich kann dieses Refactoring nur dann vornehmen, wenn dem Aufrufer bekannt ist, dass er eine bestimmte Unterklasse verwendet – andernfalls sollte ich *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) anwenden und in der Basisklasse ein Placebo-Verhalten implementieren.

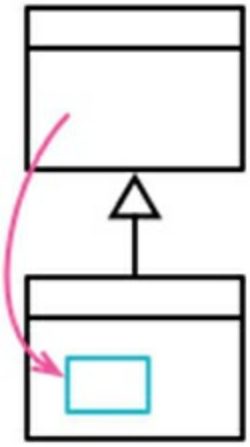
12.4.2 VORGEHEN

- Kopieren Sie die Methode in alle Unterklassen, die sie benötigen.

- Entfernen Sie die Methode aus der Basisklasse.
- Testen Sie.
- Entfernen Sie die Methode aus allen Basisklassen, die sie nicht benötigen.
- Testen Sie.

12.5 Feld nach unten verschieben (Push Down Field)

Umkehrung von: *Feld nach oben verschieben* (Pull Up Field, Abschnitt 12.2)



```
class Employee { // Java
  private String quota;
}
class Engineer extends Employee {...}
class Salesman extends Employee {...}
```

```
class Employee {...}
class Engineer extends Employee {...}
class Salesman extends Employee {
  protected String quota;
}
```

12.5.1 MOTIVATION

Sollte ein Feld nur von einer Unterklasse (oder einem geringen Anteil der Unterklassen) verwendet werden, verschiebe ich es in diese Unterklasse(n).

12.5.2 VORGEHEN

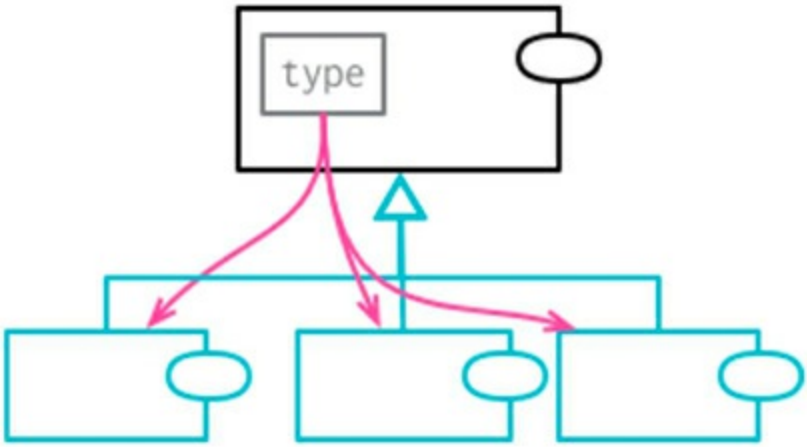
- Deklarieren Sie das Feld in allen Unterklassen, die es benötigen.
- Entfernen Sie das Feld aus der Basisklasse.
- Testen Sie.
- Entfernen sie das Feld aus allen Unterklassen, die es nicht benötigen.
- Testen Sie.

12.6 Typenschlüssel durch Unterklassen ersetzen (Replace Type Code with Subclasses)

Ersetzt: *Typenschlüssel durch Zustand/Strategie ersetzen* (Replace Type Code with State/Strategy)

Ersetzt: *Unterklasse extrahieren* (Extract Subclass)

Umkehrung von: *Unterklasse entfernen* (Remove Subclass, Abschnitt 12.7)




```
function createEmployee(name, type) {
  return new Employee(name, type);
}
```

```
function createEmployee(name, type) {
  switch (type) {
    case "engineer": return new Engineer(name);
    case "salesman": return new Salesman(name);
    case "manager": return new Manager (name);
}
```

12.6.1 MOTIVATION

Softwaresysteme müssen häufig verschiedene Arten derselben Sache repräsentieren. Ich könnte

beispielsweise Angestellte anhand ihres Tätigkeitsbereichs (Entwickler, Manager, Handlungsreisender) oder Bestellungen anhand ihrer Priorität (eilig, normal) klassifizieren. Hierfür verwende ich einen Typenschlüssel – je nach Programmiersprache könnte es sich dabei um eine Enumeration, ein Symbol, einen String oder eine Zahl handeln. Häufig wird der Typenschlüssel von einem externen Dienst festgelegt, der mir die Daten liefert, die ich verarbeite.

In den meisten Fällen ist ein solcher Typenschlüssel schon ausreichend. Hin und wieder dürfte es aber auch etwas mehr sein, und dieses »etwas mehr« sind Unterklassen. Diese sind aus zwei Gründen besonders reizvoll. Erstens ermöglichen sie mir die Verwendung von Polymorphie zur Handhabung bedingter Programmlogik. Das erweist sich als besonders nützlich, wenn mehrere Funktionen in Abhängigkeit vom Wert des Typenschlüssels unterschiedliches Verhalten aufweisen. Wenn Unterklassen verfügbar sind, kann ich *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) auf diese Funktionen anwenden.

Der zweite Grund hat mit Feldern oder Methoden zu tun, die nur für bestimmte Werte eines Typenschlüssels gültig sind, wie etwa eine Verkaufsquote, die nur auf den Typenschlüssel »Verkäufer« anwendbar ist. Ich kann dann eine Unterklasse erstellen und *Feld nach unten verschieben* (Abschnitt 12.5) anwenden. Obwohl ich auch eine Logik zur Überprüfung hinzufügen könnte, die

gewährleistet, dass ein Feld nur dann verwendet wird, wenn der Typenschlüssel den richtigen Wert besitzt, wird diese Abhängigkeit durch die Verwendung einer Unterklasse wesentlich expliziter.

Beim Einsatz von *Typenschlüssel durch Unterklassen ersetzen* muss ich erwägen, ob ich das Refactoring direkt auf die fragliche Klasse oder auf den Typenschlüssel selbst anwende. Sollte ich `Engineer` zu einem Subtyp von `Employee` machen oder besser `Employee` eine Typeigenschaft hinzufügen, die Werte wie `Engineer` und `Manager` annehmen kann? Direkte Unterklassenbildung ist einfacher, ich kann sie aber nicht für den Typ des Beschäftigungsverhältnisses verwenden, wenn ich sie für etwas anderes dringender benötige. Zudem kann ich keine Unterklasse verwenden, wenn der Typ veränderlich ist. Wenn ich alternativ zu einer Unterklassenbildung der `Employee`-Typeigenschaft übergehe, kann ich *Elementaren Wert durch Objekt ersetzen* (Abschnitt 7.3) auf den Typenschlüssel anwenden, um eine Klasse vom Typ `Employee` zu erstellen, und anschließend *Typenschlüssel durch Unterklassen ersetzen* auf die neue Klasse anwenden.

12.6.2 VORGEHEN

- Nehmen Sie eine Selbstkapselung des Feldes für den Typenschlüssel vor.
- Wählen Sie einen Typenschlüssel aus und erstellen Sie dafür eine

Unterklasse. Überschreiben Sie den Getter des Typenschlüssels, sodass er den literalen Wert zurückgibt.

- Erstellen Sie die Auswahllogik, die den Typenschlüssel der neuen Unterklasse zuordnet.

  Bei direkter Vererbung sollten Sie *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8) verwenden und die Auswahllogik in die Fabrikfunktion verlagern. Bei indirekter Vererbung kann die Auswahllogik im Konstruktor verbleiben.

- Testen Sie.

- Wiederholen Sie das Erstellen der Unterklasse und der Auswahllogik für alle Typenschlüssel. Führen Sie nach jeder Änderung Tests durch.

- Entfernen Sie das Feld für den Typenschlüssel.

- Testen Sie.

- Wenden Sie *Methode nach unten verschieben* (Abschnitt 12.4) und *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) auf alle Methoden an, die Zugriffsfunktionen für den Typenschlüssel verwenden. Wenn Sie alle ersetzt haben, können Sie die Zugriffsfunktionen für den Typenschlüssel löschen.

12.6.3 BEISPIEL

Ich verwende wieder das `Employee`-Beispiel:

```
class Employee...
constructor(name, type){
  this.validateType(type);
  this._name = name;
  this._type = type;
}
validateType(arg) {
if (!["engineer", "manager",
"salesman"].includes(arg))
```

```
  throw new Error(`Employee darf nicht vom Typ ${arg}
sein`);
}
toString() {return `${this._name} (${this._type})`;}
```

Als Erstes verwende ich *Variable kapseln* (Abschnitt 6.6), um die Selbstkapselung des Typenschlüssels vorzunehmen:

```
class Employee...
get type() {return this._type;}
toString() {return `${this._name} (${this.type})`;}
```

Beachten Sie, dass `toString` *den neuen Getter (ohne Unterstrich) verwendet.*

Ich wähle den ersten Typenschlüssel (`engineer`) aus. Dabei verwende ich direkte Vererbung, indem ich eine Unterklasse der `Employee`-Klasse selbst erstelle. Diese Unterklasse ist schlicht und einfach – sie überschreibt lediglich den Getter für den Typenschlüssel, sodass er den entsprechenden literalen Wert zurückgibt.

```
class Engineer extends Employee {
  get type() {return "engineer";}
}
```

In JavaScript können Konstruktoren zwar andere Objekte zurückgeben, aber es würde ein ziemliches

Durcheinander geben, wenn ich die Auswahllogik dort platzierte, weil sie dann mit der Initialisierung der Felder verwoben wäre. Deshalb verwende ich *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8), um einen geeigneten Platz dafür zu schaffen.

```
function createEmployee(name, type) {
  return new Employee(name, type);
}
```

In der Fabrikfunktion füge ich die Auswahllogik für die neue Unterklasse hinzu:

```
function createEmployee(name, type) {
  switch (type) {
    case "engineer": return new Engineer(name, type);
  }
  return new Employee(name, type);
}
```

Ich führe einen Test durch, um zu gewährleisten, dass alles richtig funktioniert. Aber ich bin paranoid, deshalb ändere ich den Rückgabewert der Unterklasse `Engineer`, um zu überprüfen, ob der Test dann fehlschlägt. Auf diese Weise kann ich mir sicher sein, dass die Unterklasse auch tatsächlich verwendet wird. Ich korrigiere den Rückgabewert wieder und fahre mit den anderen Fällen fort. Das kann ich der Reihe nach erledigen und nach jeder Änderung Tests durchführen:

```
class Salesman extends Employee {
  get type() {return "salesman";}
}

class Manager extends Employee {
  get type() {return "manager";}
}

function createEmployee(name, type) {
  switch (type) {
    case "engineer": return new Engineer(name, type);
    case "salesman": return new Salesman(name, type);
    case "manager": return new Manager (name, type);
  }
  return new Employee(name, type);
}
```

Nachdem dies erledigt ist, kann ich das Feld für den Typenschlüssel und die Getter-Methode der Basisklasse entfernen (die Getter der Unterklassen verbleiben).

```
class Employee...
constructor(name, type){
  this.validateType(type);
  this._name = name;
  ~~this._type = type;~~
}

~~get type() {return this._type;}~~
toString() {return `${this._name} (${this.type})`;}
```

Nachdem ich getestet und sichergestellt habe, dass noch immer alles funktioniert, kann ich die Prüflogik

entfernen, denn die `switch`-Anweisung leistet faktisch das Gleiche:

```
class Employee...
constructor(name, type){
  this.validateType(type);
  this._name = name;
}

function createEmployee(name, type) {
  switch (type) {
    case "engineer": return new Engineer(name, type);
    case "salesman": return new Salesman(name, type);
    case "manager": return new Manager (name, type);
    default: throw new Error(
             `Employee darf nicht vom Typ ${type}
sein`);
  }
  return new Employee(name, type);
}
```

Das Argument `type` des Konstruktors ist jetzt nutzlos, deshalb fällt es *Funktionsdeklaration ändern* (Abschnitt 6.5) zum Opfer:

```
class Employee...
constructor(name, type){
  this._name = name;
}

function createEmployee(name, type) {
  switch (type) {
    case "engineer": return new Engineer(name, type);
    case "salesman": return new Salesman(name, type);
```

```
    case "manager": return new Manager (name, type);
    default: throw new Error(
              `Employee darf nicht vom Typ ${type}
sein`);
  }
}
```

In den Unterklassen sind noch immer die Zugriffsfunktionen für den Typenschlüssel (`get type`) vorhanden. Ich würde sie normalerweise ebenfalls entfernen, aber ich muss noch warten, weil andere Methoden davon abhängig sind. Für diese Abhängigkeiten würde ich *Bedingung durch Polymorphie ersetzen* (Abschnitt 10.4) und *Methode nach unten verschieben* (Abschnitt 12.4) verwenden. Irgendwann gibt es keinen Code mehr, der diese Getter verwendet, also kann ich sie der milden Gnade von *Toten Code entfernen* (Abschnitt 8.9) überlassen.

12.6.4 BEISPIEL: INDIREKTE VERERBUNG

Wir kommen auf den eingangs verwendeten Fall zurück. Allerdings sind in diesem Fall schon Unterklassen für Teilzeit- und Vollzeitangestellte vorhanden, daher kann ich keine Unterklassen von `Employee` für die Typenschlüssel verwenden. Die Möglichkeit zu erhalten, den Typ eines Angestellten zu ändern, ist ein weiterer Grund dafür, keine direkte Vererbung zu verwenden:

```
class Employee...
```

```
constructor(name, type){
  this.validateType(type);
  this._name = name;
  this._type = type;
}
validateType(arg) {
  if (!["engineer", "manager",
"salesman"].includes(arg))
    throw new Error(
               `Employee darf nicht vom Typ ${arg}
sein`);
}
get type() {return this._type;}
set type(arg) {this._type = arg;}

get capitalizedType() {
  return this._type.charAt(0).toUpperCase() +
         this._type.substr(1).toLowerCase();
}

toString() {
  return `${this._name} (${this.capitalizedType})`;
}
```

Hier ist `toString` etwas komplizierter, um Ihnen kurz etwas zu demonstrieren.

Im ersten Schritt wende ich *Elementaren Wert durch Objekt ersetzen* (Abschnitt 7.3) auf den Typenschlüssel an:

```
class EmployeeType {
  constructor(aString) {
    this._value = aString;
  }
```

```
  toString() {return this._value;}
}

class Employee...
constructor(name, type){
  this.validateType(type);
  this._name = name;
  this._type = type;
}
validateType(arg) {
  if (!["engineer", "manager",
"salesman"].includes(arg))
    throw new Error(
               `Employee darf nicht vom Typ ${arg}
sein`);
}

get typeString() {return this._type.toString();}
get type()       {return this._type;}
set type(arg)    {this._type = new EmployeeType(arg);}

get capitalizedType() {
  return this.typeString.charAt(0).toUpperCase() +
         this.typeString.substr(1).toLowerCase();
}
toString() {
  return `${this._name} (${this.capitalizedType})`;
}
```

Anschließend wende ich die klassische Vorgehensweise von *Typenschlüssel durch Unterklassen ersetzen* (Abschnitt 12.6) auf den Typ `Employee` an:

```
class Employee...
set type(arg) {
  this._type = Employee.createEmployeeType(arg);
```

```
}

static createEmployeeType(aString) {
  switch(aString) {
    case "engineer": return new Engineer();
    case "manager": return new Manager ();
    case "salesman": return new Salesman();
    default: throw new Error(
              `Employee darf nicht vom Typ ${aString}
sein`);
  }
}

class EmployeeType {
}
class Engineer extends EmployeeType {
toString() {return "engineer";}
}
class Manager extends EmployeeType {
  toString() {return "manager";}
}
class Salesman extends EmployeeType {
  toString() {return "salesman";}
}
```

Wenn ich es dabei belassen würde, könnte ich die leere Klasse `EmployeeType` entfernen. Ich behalte sie jedoch, weil sie die Beziehung zwischen den verschiedenen Unterklassen verdeutlicht. Außerdem erweist sie sich als praktischer Ort, um weiteres Verhalten dorthin zu verlagern, wie etwa die Großschreibungslogik, die ich in diesem Beispiel speziell dafür verwende, diesen Punkt zu veranschaulichen:

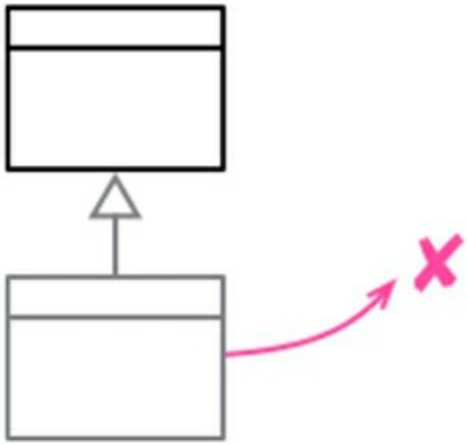
```
class Employee...
toString() {
  return `${this._name}
(${this.type.capitalizedName})`;
}

class EmployeeType...
get capitalizedName() {
  return this.toString().charAt(0).toUpperCase() +
         this.toString().substr(1).toLowerCase();
}
```

Falls Sie die erste Ausgabe dieses Buchs kennen: Dieses Beispiel ersetzt im Wesentlichen das Refactoring *Typenschlüssel durch Zustand/Strategie ersetzen* (Replace Type Code with State/Strategy). Ich betrachte das Refactoring jetzt wie *Typenschlüssel durch Unterklassen ersetzen* mit indirekter Vererbung, deshalb hielt und halte ich es nicht für sinnvoll, einen eigenen Eintrag im Katalog vorzunehmen. (Der Name gefiel mir sowieso noch nie.)

12.7 Unterklasse entfernen (Remove Subclass)

Ehemalige Bezeichnung: *Unterklasse durch Feld ersetzen* (Replace Subclass with Fields)

Umkehrung von: *Typenschlüssel durch Unterklassen ersetzen* (Replace Type Code with Subclasses, Abschnitt 12.6)



```
class Person {
  get genderCode() {return "X";}
}
class Male extends Person {
  get genderCode() {return "M";}
}
class Female extends Person {
  get genderCode() {return "F";}
}
```

```
class Person {
  get genderCode() {return this._genderCode;}
}
```

12.7.1 MOTIVATION

Unterklassen sind nützlich. Sie unterstützen Variationen der Datenstruktur sowie polymorphes Verhalten und

bieten damit eine gute Möglichkeit, sich bei der Programmierung von Unterschieden leiten zu lassen. Während sich jedoch ein Softwaresystem weiterentwickelt, können Unterklassen ihren Sinn verlieren, wenn die von ihnen unterstützten Varianten verschoben oder sogar ganz entfernt werden. Manchmal werden Unterklassen in Erwartung zukünftiger Features erstellt, die letzten Endes aber nie entwickelt werden oder aber deren Implementierung am Ende ohne die Unterklassen auskommt.

Eine Unterklasse, die zu wenig leistet, lohnt sich nicht, da sie im Verhältnis zu ihrem Nutzen das Verständnis zu sehr erschwert. Wenn es so weit kommt, ist es am besten, die Unterklasse durch ein Feld ihrer Basisklasse zu ersetzen.

12.7.2 VORGEHEN

- Wenden Sie *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8) auf den Konstruktor der Unterklasse an.

  Wenn die Clients des Konstruktors ein Feld verwenden, um zu ermitteln, welche Unterklasse erstellt werden soll, können Sie diese Programmlogik in eine Fabrikfunktion der Basisklasse verlagern.

- Wenn irgendwelcher Code den Typ der Unterklasse überprüft, können Sie *Funktion extrahieren* (Abschnitt 6.1) darauf anwenden und ihn mit *Funktion verschieben* (Abschnitt 8.1) in die Basisklasse verlagern. Führen Sie nach jeder Änderung Tests durch.

- Erstellen Sie ein Feld, das den Typ der Unterklasse repräsentiert.

- Ändern Sie die Methoden, die Verweise auf die Unterklasse enthalten, so, dass sie das neue Feld verwenden.

- Löschen Sie die Unterklasse.
- Testen Sie.

Dieses Refactoring wird häufig gleichzeitig auf eine Gruppe von Unterklassen angewendet. In diesem Fall sollten Sie zuerst Schritte unternehmen, um diese zu kapseln (Fabrikfunktion hinzufügen, Typprüfungen verschieben) und dann der Reihe nach in der Basisklasse aufnehmen.

12.7.3 BEISPIEL

Ich verwende eine Reihe rudimentärer Unterklassen:

```
class Person...
constructor(name) {
  this._name = name;
}
get name() {return this._name;}
get genderCode() {return "X";}
// weiterer Code ...

class Male extends Person {
get genderCode() {return "M";}
}
class Female extends Person {
  get genderCode() {return "F";}
}
```

Wenn das alles ist, was eine Unterklasse leistet, lohnt es sich eigentlich nicht, sie anzulegen. Aber bevor ich sie lösche, ist es für gewöhnlich sinnvoll, zu überprüfen, ob

die Clients irgendein von den Unterklassen abhängiges Verhalten zeigen, das verlagert werden müsste. In diesem Fall kann ich nichts entdecken, was dafür spricht, die Unterklassen zu behalten:

```
Client...
const numberOfMales =
             people.filter(p => p instanceof
Male).length;
```

Wenn ich ändern möchte, wie etwas repräsentiert wird, versuche ich als Erstes, die aktuelle Repräsentation zu kapseln, um die Auswirkungen auf den Client-Code zu minimieren. Beim Erstellen von Unterklassen sollte *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8) verwendet werden, um die Kapselung vorzunehmen In diesem Fall gibt es verschiedene Möglichkeiten, um die Fabrikfunktion zu erstellen.

Die direkteste Variante ist das Erstellen einer Fabrikfunktion für jeden Konstruktor:

```
function createPerson(name) {
  return new Person(name);
}
function createMale(name) {
  return new Male(name);
}
function createFemale(name) {
  return new Female(name);
}
```

Obwohl dies die direkteste Möglichkeit darstellt, werden Objekte wie diese häufig von einer Quelle geladen, die unmittelbar auf die Geschlechtskennzeichnung zugreift:

```
function loadFromInput(data) {
  const result = [];
  data.forEach(aRecord => {
    let p;
    switch (aRecord.gender) {
      case 'M': p = new Male(aRecord.name); break;
      case 'F': p = new Female(aRecord.name); break;
      default: p = new Person(aRecord.name);
    }
    result.push(p);
  });
  return result;
}
```

In diesem Fall halte ich es für besser, *Funktion extrahieren* (Abschnitt 6.1) auf die Auswahllogik für die Unterklasse anzuwenden und daraus die Fabrikfunktion zu machen:

```
function createPerson(aRecord) {
  let p;
  switch (aRecord.gender) {
    case 'M': p = new Male(aRecord.name); break;
    case 'F': p = new Female(aRecord.name); break;
    default: p = new Person(aRecord.name);
  }
  return p;
}
```

```
function loadFromInput(data) {
  const result = [];
  data.forEach(aRecord => {
    result.push(createPerson(aRecord));
  });
  return result;
}
```

Bei der Gelegenheit räume ich diese beiden Funktionen auf. Ich wende *Variable inline platzieren* (Abschnitt 6.4) auf `createPerson` an:

```
function createPerson(aRecord) {
  switch (aRecord.gender) {
    case 'M': return new Male (aRecord.name);
    case 'F': return new Female(aRecord.name);
    default: return new Person(aRecord.name);
  }
}
```

Anschließend wende ich *Schleife durch Pipeline ersetzen* (Abschnitt 8.8) auf `loadFromInput` an:

```
function loadFromInput(data) {
  return data.map(aRecord => createPerson(aRecord));
}
```

Die Fabrikfunktion kapselt das Erstellen der Unterklassen, aber es existiert ja auch noch der Aufruf von `instanceof` – ein stets übelriechender Code-Smell.

Ich wende also *Funktion extrahieren* (Abschnitt 6.1) auf die Typüberprüfung an:

```
Client...
const numberOfMales = people.filter(p =>
isMale(p)).length;

function isMale(aPerson) {return aPerson instanceof
Male;}
```

Jetzt verwende ich *Funktion verschieben* (Abschnitt 8.1), um sie in die Klasse `Person` zu bewegen:

```
class Person...
get isMale() {return this instanceof Male;}

Client...
const numberOfMales = people.filter(p =>
p.isMale).length;
```

Nachdem dieses Refactoring erledigt ist, sind die Informationen über die Unterklassen nun sicher in der Basisklasse und der Fabrikfunktion verwahrt. (Normalerweise hüte ich mich vor einer Basisklasse, die auf eine Unterklasse verweist, aber dieser Code wird nicht länger überdauern als bis zu meiner nächsten Tasse Tee, also mache ich mir an dieser Stelle keine weiteren Gedanken.)

Jetzt füge ich ein Feld hinzu, das die Unterschiede

zwischen den Unterklassen repräsentieren soll. Da ich ohnehin eine Kennzeichnung aus externer Quelle geladen habe, kann ich sie auch verwenden:

```
class Person...
constructor(name, genderCode) {
  this._name = name;
  this._genderCode = genderCode || "X";
}

get genderCode() {return this._genderCode;}
```

Bei der Initialisierung weise ich den `default`-Fall zu. (Nebenbei bemerkt können zwar die meisten Personen als männlich oder weiblich klassifiziert werden, einige aber nicht. Bei der Modellbildung wird häufig der Fehler begangen, das zu übersehen.)

Jetzt verlagere ich die Logik für den Fall »männlich« in die Basisklasse. Die Fabrikfunktion wird dahingehend angepasst, dass eine Person zurückgeben wird und alle Überprüfungen von `instanceof` das Feld für die Geschlechtskennzeichnung verwenden:

```
function createPerson(aRecord) {
  switch (aRecord.gender) {
    case 'M': return new Person(aRecord.name, "M");
    case 'F': return new Female(aRecord.name);
    default: return new Person(aRecord.name);
  }
}
```

```
class Person...
get isMale() {return "M" === this._genderCode;}
```

Ich teste, entferne die Unterklasse für den Fall »männlich«, teste erneut und wiederhole das für den Fall »weiblich«:

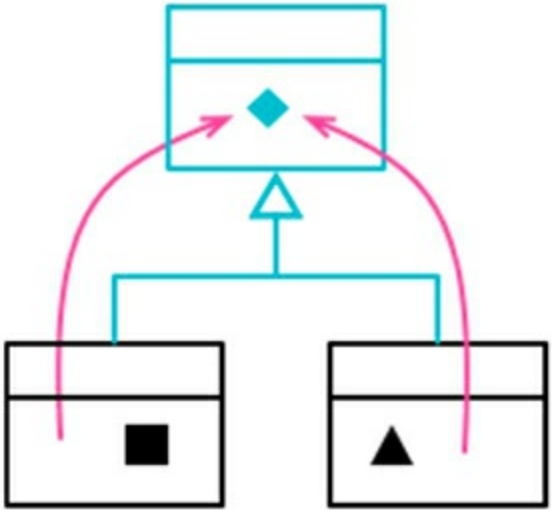
```
function createPerson(aRecord) {
  switch (aRecord.gender) {
    case 'M': return new Person(aRecord.name, "M");
    case 'F': return new Person(aRecord.name, "F");
    default: return new Person(aRecord.name);
  }
}
```

Ich empfinde die fehlende Symmetrie bei der Geschlechtskennzeichnung als störend. Ein zukünftiger Leser wird sich stets darüber wundern. Also ändere ich den Code, sodass er symmetrisch ist – sofern ich damit keine weitere Komplexität einführe, was hier nicht der Fall ist:

```
function createPerson(aRecord) {
  switch (aRecord.gender) {
    case 'M': return new Person(aRecord.name, "M");
    case 'F': return new Person(aRecord.name, "F");
    default:  return new Person(aRecord.name, "X");
  }
}

class Person...
```

```
constructor(name, genderCode) {
  this._name = name;
  this._genderCode = genderCode || "X";
}
```

12.8 Basisklasse extrahieren (Extract Superclass)



```
class Department {
  get totalAnnualCost() {...}
  get name() {...}
  get headCount() {...}
}

class Employee {
  get annualCost() {...}
  get name() {...}
  get id() {...}
}
```

```
class Party {
  get name() {...}
  get annualCost() {...}
}

class Department extends Party {
  get annualCost() {...}
  get headCount() {...}
}

class Employee extends Party {
  get annualCost() {...}
  get id() {...}
}
```

12.8.1 MOTIVATION

Wenn zwei Klassen ähnliche Aufgaben erledigen, kann ich mir den Vererbungsmechanismus zunutze machen, um die Gemeinsamkeiten in einer Basisklasse zu vereinen. Ich kann *Feld nach oben verschieben* (Abschnitt 12.2) verwenden, um gemeinsame Daten in die Basisklasse zu verschieben und mit *Methode nach oben verschieben* (Abschnitt 12.1) gemeinsames Verhalten auslagern.

Viele Autoren zum Thema Objektorientierung betrachten die Vererbung als etwas, das ausgehend von einer Klassifikationsstruktur aus der »realen Welt« von langer

Hand sorgfältig geplant werden sollte. Solche Klassifikationsstrukturen können durchaus ein Hinweis darauf sein, Vererbung zu verwenden – aber genauso häufig bemerke ich bei der fortlaufenden Weiterentwicklung eines Programms, dass an einer Stelle Vererbung angebracht ist, wenn ich auf gemeinsame Elemente stoße, die ich vereinen möchte.

Als Alternative zu *Basisklasse extrahieren* bietet sich *Klasse extrahieren* (Abschnitt 7.5) an. Hier haben Sie im Wesentlichen die Wahl, Vererbung oder Delegation zu verwenden, um redundantes Verhalten zu vereinheitlichen. Meistens ist *Basisklasse extrahieren* der einfachere Ansatz, deshalb gehe ich zuerst auf diese Weise vor, denn ich weiß, dass ich später bei Bedarf *Basisklasse durch Delegation ersetzen* (Abschnitt 12.11) verwenden kann.

12.8.2 VORGEHEN

- Erstellen Sie eine leere Basisklasse, von der Sie die ursprünglichen Klassen erben lassen.

  Wenden Sie bei Bedarf *Funktionsdeklaration ändern* (Abschnitt 6.5) auf die Konstruktoren an.

- Testen Sie.

- Wenden Sie der Reihe nach *Konstruktorrumpf nach oben verschieben* (Abschnitt 12.3), *Methode nach oben verschieben* (Abschnitt 12.1) und *Feld nach oben verschieben* (Abschnitt 12.2) an, um gemeinsame Elemente in die Basisklasse zu verlagern.

- Nehmen Sie die verbliebenen Methoden der Unterklassen in Augenschein und prüfen Sie, ob gemeinsame Bestandteile

vorhanden sind. Verwenden Sie gegebenenfalls *Funktion extrahieren* (Abschnitt 6.1), gefolgt von *Methode nach oben verschieben* (Abschnitt 12.1).

- Überprüfen Sie die Clients der ursprünglichen Klassen. Ziehen Sie in Betracht, sie anzupassen, sodass sie die Schnittstelle der Basisklasse verwenden.

12.8.3 BEISPIEL

Ich betrachte die beiden folgenden Klassen, die einige Gemeinsamkeiten aufweisen – die Bezeichnungen und die Verwendung jährlicher und monatlicher Kosten:

```
class Employee {
  constructor(name, id, monthlyCost) {
    this._id = id;
    this._name = name;
    this._monthlyCost = monthlyCost;
  }
  get monthlyCost() {return this._monthlyCost;}
  get name() {return this._name;}
  get id() {return this._id;}

  get annualCost() {
    return this.monthlyCost * 12;
  }
}

class Department {
  constructor(name, staff){
    this._name = name;
    this._staff = staff;
  }
  get staff() {return this._staff.slice();}
  get name() {return this._name;}
```

```
  get totalMonthlyCost() {
    return this.staff
      .map(e => e.monthlyCost)
      .reduce((sum, cost) => sum + cost);
  }
  get headCount() {
    return this.staff.length;
  }
  get totalAnnualCost() {
    return this.totalMonthlyCost * 12;
  }
}
```

Ich kann diese Gemeinsamkeiten hervorheben, indem ich sie extrahiere und in einer gemeinsamen Basisklasse vereine.

Zunächst erstelle ich eine leere Basisklasse, deren Unterklassen die ursprünglichen Klassen sind:

```
class Party {}

class Employee extends Party {
  constructor(name, id, monthlyCost) {
    super();
    this._id = id;
    this._name = name;
    this._monthlyCost = monthlyCost;
  }
  // übrige Klasse ...

class Department extends Party {
  constructor(name, staff){
    super();
    this._name = name;
```

```
    this._staff = staff;
  }
  // übrige Klasse ...
```

Bei der Anwendung von *Basisklasse extrahieren* kümmere ich mich zuerst um die Daten. In JavaScript bedeutet das, den Konstruktor zu bearbeiten. Ich führe also als Erstes *Feld nach oben verschieben* (Abschnitt 12.2) aus, um das Feld `name` zu übernehmen:

```
class Party...
constructor(name){
  this._name = name;
}

class Employee...
constructor(name, id, monthlyCost) {
  super(name);
  this._id = id;
  this._monthlyCost = monthlyCost;
}

class Department...
constructor(name, staff){
  super(name);
  this._staff = staff;
}
```

Beim Bewegen der Daten in die Basisklasse kann ich *Methode nach oben verschieben* (Abschnitt 12.1) auf die zugehörigen Methoden anwenden. Zuerst der Name:

```
class Party...
get name() {return this._name;}

class Employee...
~~get name() {return this._name;}~~

class Department...
~~get name() {return this._name;}~~
```

Zwei Methoden sind sich sehr ähnlich:

```
class Employee...
get annualCost() {
  return this.monthlyCost * 12;
}

class Department...
get totalAnnualCost() {
  return this.totalMonthlyCost * 12;
}
```

Die Methoden `monthlyCost` und `totalMonthlyCost`, die von ihnen aufgerufen werden, tragen verschiedene Namen und besitzen unterschiedlichen Inhalt – aber verfolgen sie dieselbe Absicht? Wenn dem so ist, sollte ich *Funktionsdeklaration ändern* (Abschnitt 6.5) verwenden, um die Namen zu vereinheitlichen:

```
class Department...
get totalAnnualCost() {
  return this.monthlyCost * 12;
```

```
}
get monthlyCost() { ... }
```

Bei den jährlichen Kosten (`annualCost`) nehme ich eine ähnliche Umbenennung vor:

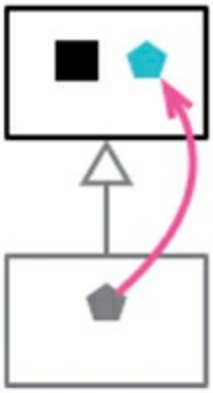
```
class Department...
get annualCost() {
  return this.monthlyCost * 12;
}
```

Jetzt kann ich *Methode nach oben verschieben* (Abschnitt 12.1) auf die `annualCost`-Methoden anwenden:

```
class Party...
get annualCost() {
  return this.monthlyCost * 12;
}

class Employee...
~~get annualCost() {~~
  ~~return this.monthlyCost * 12;~~
~~}~~

class Department...
~~get annualCost() {~~
  ~~return this.monthlyCost * 12;~~
~~}~~
```

12.9 Hierarchie abbauen (Collapse

Hierarchy)



```
class Employee {...}
class Salesman extends Employee {...}
```

```
class Employee {...}
```

12.9.1 MOTIVATION

Wenn ich ein Refactoring einer Klassenhierarchie vornehme, verschiebe ich häufig Merkmale nach oben oder unten. Bei der sich entwickelnden Hierarchie kann ich dann manchmal feststellen, dass sich eine Unterklasse und ihre Basisklasse nicht mehr so sehr unterscheiden, um eine Aufteilung zu rechtfertigen. In diesem Fall vereine ich sie miteinander.

12.9.2 VORGEHEN

- Wählen Sie aus, welche Klasse Sie entfernen möchten.

  Meine Auswahl beruht darauf, welcher Name zukünftig am sinnvollsten ist. Wenn keiner der Namen geeignet ist, treffe ich eine beliebige Auswahl.

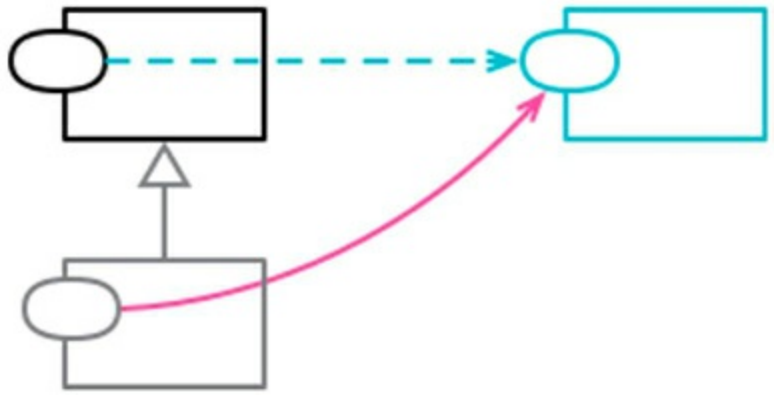
- Verwenden Sie *Feld nach oben verschieben* (Abschnitt 12.2), *Feld nach unten verschieben* (Abschnitt 12.5), *Methode nach oben verschieben* (Abschnitt 12.1) und *Methode nach unten verschieben* (Abschnitt 12.4), um alle Elemente in eine einzelne Klasse zu verschieben.

- Ändern Sie alle Verweise auf die zu entfernende Klasse, sodass sie auf die verbleibende Klasse verweisen.

- Löschen Sie die leere Klasse.

- Testen Sie.

12.10 Unterklasse durch Delegation ersetzen (Replace Subclass with Delegate)



```
class Order {
  get daysToShip() {
```

```
    return this._warehouse.daysToShip;
  }
}

class PriorityOrder extends Order {
  get daysToShip() {
    return this._priorityPlan.daysToShip;
  }
}
```

```
class Order {
  get daysToShip() {
    return (this._priorityDelegate)
      ? this._priorityDelegate.daysToShip
      : this._warehouse.daysToShip;
  }
}

class PriorityOrderDelegate {
  get daysToShip() {
    return this._priorityPlan.daysToShip
  }
}
```

12.10.1 MOTIVATION

Wenn das Verhalten verschiedener Objekte in Abhängigkeit von ihrer Kategorie variiert, bietet sich als naheliegender Mechanismus, um das zu verdeutlichen, die Vererbung an. Ich verlagere die gemeinsamen Daten

und das gemeinsame Verhalten in die Basisklasse und ergänze oder überschreibe nach Bedarf einzelne Merkmale in den Unterklassen. In objektorientierten Programmiersprachen lässt sich das leicht implementieren und wird entsprechend häufig genutzt.

Vererbung hat aber auch Nachteile. Sie ist offensichtlich eine Karte, die nur einmal ausgespielt werden kann. Wenn es mehr als einen Grund für eine Variation gibt, bin ich bei der Vererbung an eine einzige variable Achse gebunden. Das heißt, wenn ich das Verhalten einer Person entsprechend ihrer Altersgruppe oder ihres Einkommensniveaus variieren möchte, kann ich entweder Unterklassen für Jung und Alt oder für Wohlhabende und Arme verwenden – beides gleichzeitig funktioniert nicht.

Dass die Vererbung zu einer sehr engen Kopplung zwischen Klassen führt, ist ein weiteres Problem. Jede Änderung an der Basisklasse kann dazu führen, dass eine der Unterklassen nicht mehr funktioniert, deshalb muss ich sorgfältig vorgehen und genau nachvollziehen, wie die Unterklassen sich von der Basisklasse ableiten. Dieses Problem wird noch verschärft, wenn sich die Programmlogik zweier Klassen in verschiedenen Modulen befindet, die von unterschiedlichen Teams betreut werden.

Durch Delegation lassen sich beide Probleme in den Griff bekommen. Ich kann aus den unterschiedlichsten

Gründen Aufgaben an verschiedene Klassen delegieren. Delegation ist eine eindeutig geregelte Beziehung zwischen Objekten – ich kann also auf eine klar definierte Schnittstelle zurückgreifen, was eine sehr viel geringere Kopplung im Vergleich zur Unterklassenbildung bedeutet. Es kommt also häufiger vor, dass Probleme bei der Unterklassenbildung auftreten, die sich mit *Unterklasse durch Delegation ersetzen* lösen lassen.

Ein bekanntes Prinzip besagt, dass die Komposition der Vererbung vorzuziehen sei (wobei die Komposition faktisch der Delegation entspricht). Diese Regel wird oft oft als »Vererbung ist schädlich« interpretiert, und es wird behauptet, dass wir Vererbung überhaupt nicht verwenden sollten. Ich verwende Vererbung häufig, zum Teil wohl auch deswegen, weil ich weiß, dass ich später bei Bedarf *Basisklasse durch Delegation ersetzen* (Abschnitt 12.11) verwenden kann. Vererbung ist ein wertvoller Mechanismus, der seine Aufgabe in den meisten Fällen problemlos erledigt. Ich verwende also zunächst Vererbung und wechsle zur Delegation, falls es zu Schwierigkeiten kommt. Diese Art der Verwendung steht sogar im Einklang mit dem Prinzip, das dem Buch der Gang of Four (Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides: *Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software*, mitp-Verlag 2015, ISBN 9783826697005) entstammt und erklärt, wie Vererbung

und die Komposition von Objekten zusammenwirken. Das Aufstellen dieser Regel war eine Reaktion auf den übertriebenen Einsatz von Vererbung.

Wenn Ihnen das Buch der Gang of Four vertraut ist, könnte es hilfreich sein, sich dieses Refactoring als das Ersetzen von Unterklassen mit Hilfe der Muster *Zustand* (State) oder *Strategie* (Strategy) vorzustellen. Die beiden Muster gleichen sich strukturell und beruhen auf der Delegation an eine separate Hierarchie. Die Anwendung von *Unterklassen durch Delegation ersetzen* geht nicht in allen Fällen mit einer Vererbungshierarchie auf Seiten des Delegates einher (wie das erste nachfolgende Beispiel veranschaulicht). Das Einführen einer Hierarchie für Zustände oder Strategien erweist sich aber nicht selten als nützlich.

12.10.2 VORGEHEN

- Wenn es viele Aufrufer des Konstruktors gibt, sollten Sie *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8) anwenden.
- Erstellen Sie eine leere Klasse, an die Sie Funktionalität delegieren wollen (Delegate). Der Konstruktor sollte klassenspezifische Daten sowie i. d. R. einen Verweis zurück auf die Basisklasse entgegennehmen.
- Fügen Sie der Basisklasse ein Feld für den Delegate hinzu.
- Ändern Sie den Erstellungsprozess der Unterklasse, sodass das Feld für den Delegate mit einer Instanz initialisiert wird.

  Das kann in der Fabrikfunktion geschehen oder im Konstruktor, sofern der Konstruktor zuverlässig den richtigen Delegate erzeugen

kann.

- Wählen Sie die Unterklassenmethode aus, die in die Delegate-Klasse verschoben werden soll.
- Verwenden Sie *Funktion verschieben* (Abschnitt 8.1), um sie in die Delegate-Klasse zu verschieben. Den Code zur Weiterleitung in der Quellfunktion dürfen Sie nicht entfernen.

  Falls die Methode Elemente benötigt, die in den Delegate verschoben werden müssen, sollten Sie das tun. Wenn Elemente benötigt werden, die in der Basisklasse verbleiben sollen, können Sie dem Delegate ein Feld hinzufügen, das auf die Basisklasse verweist.
- Wenn die ursprüngliche Methode von Aufrufern außerhalb der Klasse verwendet wird, sollten Sie die delegierende Methode von der Unterklasse in die Basisklasse verschieben und sicherheitshalber eine Wächterbedingung für das Vorhandensein des Delegates hinzufügen. Wenden Sie ansonsten *Toten Code entfernen* (Abschnitt 8.9) an.

  Wenn mehrere Unterklassen vorhanden sind und Sie beginnen, Code zu duplizieren, können Sie *Basisklasse extrahieren* (Abschnitt 12.8) anwenden. In diesem Fall benötigen die delegierenden Methoden in der ursprünglichen Basisklasse keine Wächterbedingung mehr, sofern das Standardverhalten in die Basisklasse des Delegates verschoben wird.
- Testen Sie.
- Wiederholen Sie das Ganze, bis alle Methoden der Unterklasse verschoben worden sind.
- Suchen Sie nach den Aufrufern des Konstruktors der Unterklasse und ändern Sie diese, sodass sie den Konstruktor der Basisklasse verwenden.
- Testen Sie.
- Wenden Sie *Toten Code entfernen* (Abschnitt 8.9) auf die Unterklasse an.

12.10.3 BEISPIEL

Ich verwende eine Klasse `Booking`, die Buchungen für eine Show vornimmt.

```
class Booking...
constructor(show, date) {
  this._show = show;
  this._date = date;
}
```

Die Unterklasse `PremiumBooking` berücksichtigt die verschiedenen verfügbaren Extras:

```
class PremiumBooking extends Booking...
constructor(show, date, extras) {
  super(show, date);
  this._extras = extras;
}
```

Die Klasse `PremiumBooking` weist im Vergleich zu ihrer Basisklasse `Booking` eine ganze Reihe von Änderungen auf. Wenn man sich bei der Programmierung von Unterschieden leiten lässt, ist es typisch, dass die Unterklasse in einigen Fällen Methoden der Basisklasse überschreibt oder in anderen Fällen neue Methoden ergänzt, die nur für die Unterklasse relevant sind. Ich werde nicht auf alle eingehen, aber einige interessante Fälle auswählen.

Als Erstes geht es um ein einfaches Überschreiben. Normale Buchungen bieten die Möglichkeit, nach der Show an einer Gesprächsrunde teilzunehmen, allerdings nicht an Tagen mit hohem Besucheraufkommen:

```
class Booking...
get hasTalkback() {
  return this._show.hasOwnProperty('talkback') &&
!this.isPeakDay;
}
```

Die Klasse `PremiumBooking` überschreibt dieses Verhalten und bietet an allen Tagen Gesprächsrunden an:

```
class PremiumBooking...
get hasTalkback() {
  return this._show.hasOwnProperty('talkback');
}
```

Die Preisermittlung wird ebenfalls überschrieben, allerdings kommt hier noch der Kniff dazu, dass die Methode in der `PremiumBooking`-Klasse die Methode der Basisklasse aufruft:

```
class Booking...
get basePrice() {
  let result = this._show.price;
  if (this.isPeakDay) result += Math.round(result *
0.15);
```

```
  return result;
}

class PremiumBooking...
get basePrice() {
  return Math.round(super.basePrice +
                    this._extras.premiumFee);
}
```

Das letzte Beispiel zeigt ein Verhalten der `PremiumBooking`-Klasse, das in der Basisklasse nicht vorhanden ist:

```
class PremiumBooking...
get hasDinner() {
  return this._extras.hasOwnProperty('dinner') &&
                                  !this.isPeakDay;
}
```

In diesem Beispiel funktioniert die Vererbung gut. Ich kann die Basisklasse verstehen, ohne die Unterklasse verstehen zu müssen. Die Unterklasse ist dadurch definiert, dass festgelegt wird, wie sie sich vom Standardfall unterscheidet – das verringert Redundanzen und vermittelt klar und deutlich, welche Unterschiede bestehen.

Tatsächlich ist das Ganze nicht ganz so perfekt, wie der letzte Absatz es vermittelt. Die Struktur der Basisklasse enthält Elemente, die nur aufgrund der Unterklasse einen Sinn ergeben – wie beispielsweise die Methoden,

die so ausgelegt sind, dass das Überschreiben des entsprechenden Verhaltens vereinfacht wird. Ich kann die Basisklasse zwar fast immer modifizieren, ohne die Unterklassen verstehen zu müssen, aber hin und wieder wird dieses bewusste Ignorieren der Unterklassen dazu führen, dass eine Änderung an der Basisklasse dafür verantwortlich ist, dass eine Unterklasse nicht mehr funktioniert. Falls so etwas jedoch nicht allzu oft vorkommt, macht sich die Vererbung dennoch bezahlt – vorausgesetzt, mir stehen gute Tests zur Verfügung, die ein Fehlverhalten einer Unterklasse erkennen.

Warum aber sollte ich in einer solch glücklichen Situation *Unterklasse durch Delegation ersetzen* überhaupt anwenden? Vererbung ist ein Werkzeug, das nur einmalig nutzbar ist – wenn ich also einen anderen Grund habe, Vererbung einzusetzen, und denke, dass sie dann nützlicher für mich ist als die `PremiumBooking`-Unterklasse, dann muss ich bei den Premiumbuchungen auf andere Weise vorgehen. Möglicherweise muss ich auch dynamisch von der Klasse `Booking` zur Klasse `PremiumBooking` wechseln können – also eine Methode wie `aBooking.bePremium()`unterstützen. In manchen Fällen kann ich das umgehen, indem ich ein komplett neues Objekt erstelle (ein typisches Beispiel hierfür ist eine HTTP-Anfrage, die neue Daten vom Server lädt). Manchmal muss ich aber auch eine vorhandene Datenstruktur modifizieren und nicht von Grund auf neu erstellen. Zudem ist es schwierig, eine

einzelne Buchung zu ersetzen, auf die an vielen verschiedenen Stellen verwiesen wird. In solchen Fällen könnte es nützlich sein, wenn eine Buchung zwischen »normal« und »premium« hin und her wechseln könnte.

Wenn solche Anforderungen auftauchen, muss ich *Unterklasse durch Delegation ersetzen* anwenden. Die Clients rufen die Konstruktoren der beiden Klassen auf, um Buchungen vorzunehmen:

```
Booking-Client
aBooking = new Booking(show,date);

PremiumBooking-Client
aBooking = new PremiumBooking(show, date, extras);
```

Durch das Entfernen der Unterklassen wird sich all dies ändern, deshalb kapsele ich die Konstruktoraufrufe mit *Konstruktor durch Fabrikfunktion ersetzen* (Abschnitt 11.8).

```
Oberste Ebene...
function createBooking(show, date) {
  return new Booking(show, date);
}
function createPremiumBooking(show, date, extras) {
  return new PremiumBooking (show, date, extras);
}

Booking-Client
aBooking = createBooking(show, date);
```

```
PremiumBooking-Client
aBooking = createPremiumBooking(show, date, extras);
```

Jetzt erstelle ich die neue Delegate-Klasse. Die Parameter des Konstruktors sind diejenigen, die nur in den Unterklassen verwendet werden, sowie ein Rückverweis auf das `Booking`-Objekt. Dieses benötige ich, weil mehrere Unterklassenmethoden auf gespeicherte Daten in der Basisklasse zugreifen können müssen. Bei Vererbung ist das leicht machbar, aber bei einem Delegate benötige ich einen Rückverweis:

```
class PremiumBookingDelegate...
constructor(hostBooking, extras) {
  this._host = hostBooking;
  this._extras = extras;
}
```

Jetzt verknüpfe ich den Delegate mit dem `Booking`-Objekt, indem ich die Fabrikfunktion für `PremiumBooking`-Objekte ändere:

```
Oberste Ebene...
function createPremiumBooking(show, date, extras) {
  const result = new PremiumBooking (show, date,
extras);
  result._bePremium(extras);
  return result;
}

class Booking...
```

```
_bePremium(extras) {
  this._premiumDelegate = new
PremiumBookingDelegate(this, extras);
}
```

Ich verwende einen führenden Unterstrich für `_bePremium`, um darauf hinzuweisen, dass die Methode kein Bestandteil der öffentlichen Schnittstelle der `Booking`-Klasse sein sollte. Wenn der Sinn dieses Refactorings allerdings darin besteht, zu ermöglichen, dass aus einer normalen Buchung eine Premiumbuchung wird, kann die Schnittstelle öffentlich sein.

Alternativ könnte ich die Verknüpfungen im Konstruktor von `Booking` erstellen. Um das zu tun, muss ich dem Konstruktor signalisieren können, dass es sich um eine Premiumbuchung handelt. Dazu könnte ich einen separaten Parameter übergeben oder einfach den Paramter `extras` nutzen, wenn ich mir sicher sein kann, dass er bei einer Premiumbuchung immer vorhanden ist. Hier ziehe ich es der Deutlichkeit halber vor, dafür die Fabrikfunktion zu verwenden.

Jetzt sind die Grundstrukturen soweit hergestellt, und es ist an der Zeit, das Verhalten zu verschieben. Als Erstes betrachte ich das einfache Überschreiben von `hasTalkback`. Hier ist der vorhandene Code:

```
class Booking...
get hasTalkback() {
```

```
  return this._show.hasOwnProperty('talkback') &&
!this.isPeakDay;
}
class PremiumBooking...
get hasTalkback() {
return this._show.hasOwnProperty('talkback');
}
```

Ich verwende *Funktion verschieben* (Abschnitt 8.1), um die Unterklassenmethode in den Delegate zu verschieben. Zwecks Anpassung an das neue Umfeld leite ich Zugriffe auf Daten der Basisklasse durch einen Aufruf von `_host` um:

```
Class PremiumBookingDelegate...
get hasTalkback() {
  return this._host._show.hasOwnProper‘y('talkb’ck');
}

class PremiumBooking...
get hasTalkback() {
  return this._premiumDelegate.hasTalkback;
}
```

Ich führe Tests durch, um zu gewährleisten, dass alles funktioniert, und lösche anschließend die Unterklassenmethode:

```
class PremiumBooking...
~~get hasTalkback() {~~
~~return this._premiumDelegate.hasTalkback;~~
~~}~~
```

Erneut führe ich Tests durch und erwarte, dass einige davon fehlschlagen werden.

Ich vervollständige das Verschieben, indem ich Aufrufe der Basisklassenmethode an den Delegate weiterleite, sofern dieser vorhanden ist:

```
class Booking...
get hasTalkback() {
  return (this._premiumDelegate)
    ? this._premiumDelegate.hasTalkback
    : this._show.hasOwnProperty('talkback') &&
!this.isPeakDay;
}
```

Als Nächstes betrachte ich den Grundpreis `basePrice`:

```
class Booking...
get basePrice() {
  let result = this._show.price;
  if (this.isPeakDay) result += Math.round(result *
0.15);
  return result;
}

class PremiumBooking...
get basePrice() {
  return Math.round(super.basePrice +
this._extras.premiumFee);
}
```

Das ist fast das Gleiche, allerdings gibt es hier noch den unschönen Aufruf von `super` (der bei Unterklassenerweiterungen dieser Art ziemlich häufig vorkommt). Wenn ich den Code der Unterklasse in den Delegate verschiebe, muss ich die Basisklasse aufrufen, kann aber nicht einfach `this._host.basePrice` verwenden, ohne in einer endlosen Rekursion zu enden.

Ich habe hier mehrere Möglichkeiten. Ich könnte *Funktion extrahieren* (Abschnitt 6.1) auf die Berechnung des Grundpreises anwenden, um das Weiterleiten des Aufrufs von der Preisberechnung zu trennen (das Übrige läuft ab wie zuvor):

```
class Booking...
get basePrice() {
  return (this._premiumDelegate)
    ? this._premiumDelegate.basePrice
    : this._privateBasePrice;
}

get _privateBasePrice() {
  let result = this._show.price;
  if (this.isPeakDay) result += Math.round(result *
0.15);
  return result;
}

class PremiumBookingDelegate...
get basePrice() {
  return Math.round(this._host._privateBasePrice +
this._extras.premiumFee);
}
```

Alternativ könnte ich die Methode des Delegates zu einer Erweiterung der Methode in der Basisklasse machen:

```
class Booking...
get basePrice() {
  let result = this._show.price;
  if (this.isPeakDay) result += Math.round(result *
0.15);
  return (this._premiumDelegate)
    ? this._premiumDelegate.extendBasePrice(result)
    : result; }

class PremiumBookingDelegate...
extendBasePrice(base) {
  return Math.round(base + this._extras.premiumFee);
}
```

Beides funktioniert gleichermaßen gut, aber ich neige zu Letzterem, weil es etwas kürzer ist.

Der letzte Fall ist eine Methode, die nur in der Unterklasse vorhanden ist:

```
class PremiumBooking...
get hasDinner() {
return this._extras.hasOwnProperty('dinner') &&
                                !this.isPeakDay;
}
```

Ich verschiebe sie von der Unterklasse in den Delegate:

```
class PremiumBookingDelegate...
get hasDinner() {
return this._extras.hasOwnProperty('dinner') &&
                           !this._host.isPeakDay;
}
```

Anschließend füge ich der Klasse `Booking` die Weiterleitungslogik hinzu:

```
class Booking...
get hasDinner() {
  return (this._premiumDelegate)
    ? this._premiumDelegate.hasDinner
    : undefined;
}
```

Der Zugriff auf eine nicht definierte Eigenschaft eines Objekts liefert in JavaScript `undefined` zurück, und so verfahre ich auch hier. (Obwohl ich spontan dazu neigen würde, wie in den anderen objektorientierten dynamischen Sprachen, die ich verwende, eine Fehlermeldung auszulösen.)

Wenn das gesamte Verhalten erst einmal aus der Unterklasse verschoben ist, kann ich die Fabrikmethode ändern, sodass sie die Basisklasse zurückgibt. Nach dem Ausführen der Tests, um zu gewährleisten, dass alles funktioniert, kann ich die Unterklasse schließlich löschen:

```
Oberste Ebene...
function createPremiumBooking(show, date, extras) {
  const result = new ~~Premium~~Booking (show, date,
extras);
  result._bePremium(extras);
  return result;
}
~~class PremiumBooking extends Booking ...~~
```

Hier habe ich den Eindruck, dass das Refactoring allein den Code nicht verbessert – wobei sich die Vererbung als lohnenswert erweist, während bei der Delegation das Hinzufügen von Weiterleitungslogik und bidirektionalen Verweisen erforderlich ist, was zusätzliche Komplexität bedeutet. Dennoch kann sich das Refactoring insgesamt lohnen, wenn die Vorteile der Statusänderung einer Buchung oder anderweitiger Bedarf für Vererbung schwerer wiegen als die Nachteile des Verzichts auf Vererbung.

12.10.4 BEISPIEL: ERSETZEN EINER HIERARCHIE

Das letzte Beispiel zeigte, wie *Unterklasse durch Delegation ersetzen* auf eine einzelne Unterklasse angewendet wird. Mit einer ganzen Hierarchie kann ich jedoch ebenso verfahren:

```
function createBird(data) {
  switch (data.type) {
    case 'EuropeanSwallow':
```

```
      return new EuropeanSwallow(data);
    case 'AfricanSwallow':
      return new AfricanSwallow(data);
    case 'NorwegianBlueParrot':
      return new NorwegianBlueParrot(data);
    default:
      return new Bird(data);
  }
}

class Bird {
  constructor(data) {
    this._name = data.name;
    this._plumage = data.plumage;
  }
  get name() {return this._name;}
  get plumage() {
    return this._plumage || "average";
  }
  get airSpeedVelocity() {return null;}
}

class EuropeanSwallow extends Bird {
  get airSpeedVelocity() {return 35;}
}

class AfricanSwallow extends Bird {
  constructor(data) {
    super (data);
    this._numberOfCoconuts = data.numberOfCoconuts;
  }
  get airSpeedVelocity() {
    return 40 - 2 * this._numberOfCoconuts;
  }
}

class NorwegianBlueParrot extends Bird {
  constructor(data) {
    super (data);
```

```
    this._voltage = data.voltage;
    this._isNailed = data.isNailed;
  }
  get plumage() {
    if (this._voltage > 100) return "scorched";
    else return this._plumage || "beautiful";
  }
  get airSpeedVelocity() {
    return (this._isNailed) ? 0 : 10 + this._voltage /
10;
  }
}
```

Das System soll demnächst zwischen Vögeln in freier Wildbahn und Vögeln in Gefangenschaft unterscheiden. Dieser Unterschied könnte durch zwei Unterklassen von `Bird` erfasst werden: `WildBird` und `CaptiveBird`. Allerdings kann ich die Vererbung nur einmalig einsetzen. Wenn ich also Unterklassen zur Unterscheidung von wilden und in Gefangenschaft gehaltenen Vögeln verwenden möchte, muss ich die Unterklassen für die Vogelart entfernen.

Wenn mehrere Unterklassen beteiligt sind, bearbeite ich sie der Reihe nach und beginne mit einer einfachen – in diesem Fall mit `EuropeanSwallow`. Ich erstelle eine leere Delegate-Klasse:

```
class EuropeanSwallowDelegate {
}
```

Zunächst verwende ich noch keine Daten oder Parameter für Rückverweise. In diesem Beispiel füge ich sie erst bei Bedarf hinzu.

Ich muss entscheiden, wo ich die Initialisierung des Delegate-Feldes vornehme. Ich entscheide mich hier für den Konstruktor, denn alle erforderlichen Informationen sind in dem einzigen Datenparameter des Konstruktors enthalten. Da ich mehrere Delegates hinzufügen könnte, erstelle ich eine Funktion, die den richtigen Delegate anhand des Typenschlüssels im Datendokument auswählt:

```
class Bird...
constructor(data) {
  this._name = data.name;
  this._plumage = data.plumage;
  this._speciesDelegate =
this.selectSpeciesDelegate(data);
}

selectSpeciesDelegate(data) {
  switch(data.type) {
    case 'EuropeanSwallow':
      return new EuropeanSwallowDelegate();
    default: return null;
  }
}
```

Jetzt ist die Struktur eingerichtet, und ich kann *Funktion verschieben* (Abschnitt 8.1) auf die Funktion `airSpeedVelocity` anwenden:

```
class EuropeanSwallowDelegate...
get airSpeedVelocity() {return 35;}

class EuropeanSwallow...
get airSpeedVelocity() {
  return this._speciesDelegate.airSpeedVelocity;
}
```

Ich ändere die Funktion `airSpeedVelocity` in der Basisklasse, sodass sie einen Delegate aufruft, falls vorhanden:

```
class Bird...
get airSpeedVelocity() {
  return this._speciesDelegate
         ? this._speciesDelegate.airSpeedVelocity :
null;
}
```

Dann entferne ich die Unterklasse:

```
class EuropeanSwallow extends Bird {
  get airSpeedVelocity() {
    return this._speciesDelegate.airSpeedVelocity;
  }
}

Oberste Ebene...
function createBird(data) {
  switch (data.type) {
    case 'EuropeanSwallow':
      return new EuropeanSwallow(data);
```

```
    case 'AfricanSwallow':
      return new AfricanSwallow(data);
    case 'NorwegianBlueParrot':
      return new NorwegianBlueParrot(data);
    default:
      return new Bird(data);
  }
}
```

Jetzt kommt `AfricanSwallow` an die Reihe, ich erstelle eine Klasse. In diesem Fall benötigt der Konstruktor das Datendokument:

```
class AfricanSwallowDelegate...
constructor(data) {
  this._numberOfCoconuts = data.numberOfCoconuts;
}

class Bird...
selectSpeciesDelegate(data) {
  switch(data.type) {
    case 'EuropeanSwallow':
      return new EuropeanSwallowDelegate();
    case 'AfricanSwallow':
      return new AfricanSwallowDelegate(data);
    default:
      return null;
  }
}
```

Ich wende *Funktion verschieben* (Abschnitt 8.1) auf `airSpeedVelocity` an:

```
class AfricanSwallowDelegate...
get airSpeedVelocity() {
  return 40 - 2 * this._numberOfCoconuts;
}

class AfricanSwallow...
get airSpeedVelocity() {
  return this._speciesDelegate.airSpeedVelocity;
}
```

Jetzt kann ich die Unterklasse `AfricanSwallow` entfernen:

```
class AfricanSwallow extends Bird {
  // sämtlicher Code ...
}

function createBird(data) {
  switch (data.type) {
    case 'AfricanSwallow':
      return new AfricanSwallow(data);
    case 'NorwegianBlueParrot':
      return new NorwegianBlueParrot(data);
    default:
      return new Bird(data);
  }
}
```

Und nun zum norwegischen Blauling. Das Erstellen der Klasse und das Verschieben der Funktion `airSpeedVelocity` verlaufen auf die gleiche Weise wie zuvor, deshalb zeige ich nur das Ergebnis:

```
class Bird...
selectSpeciesDelegate(data) {
  switch(data.type) {
    case 'EuropeanSwallow':
      return new EuropeanSwallowDelegate();
    case 'AfricanSwallow':
      return new AfricanSwallowDelegate(data);
    case 'NorwegianBlueParrot':
      return new NorwegianBlueParrotDelegate(data);
    default:
      return null;
  }
}

class NorwegianBlueParrotDelegate...
constructor(data) {
  this._voltage = data.voltage;
  this._isNailed = data.isNailed;
}
get airSpeedVelocity() {
  return (this._isNailed) ? 0 : 10 + this._voltage /
10;
}
```

So weit, so gut, allerdings überschreibt `NorwegianBlueParrot` die Eigenschaft `plumage`, um die ich mich in den anderen Fällen nicht kümmern musste. Das anfängliche *Funktion verschieben* (Abschnitt 8.1) ist problemlos machbar, wenngleich es notwendig ist, den Konstruktor zu ändern, sodass er einen Rückverweis auf `bird` erhält:

```
class NorwegianBlueParrot...
get plumage() {
```

```
  return this._speciesDelegate.plumage;
}

class NorwegianBlueParrotDelegate...
get plumage() {
  if (this._voltage > 100) return "scorched";
  else return this._bird._plumage || "beautiful";
}

constructor(data, bird) {
  this._bird = bird;
  this._voltage = data.voltage;
  this._isNailed = data.isNailed;
}

class Bird...
selectSpeciesDelegate(data) {
  switch(data.type) {
    case 'EuropeanSwallow':
      return new EuropeanSwallowDelegate();
    case 'AfricanSwallow':
      return new AfricanSwallowDelegate(data);
    case 'NorwegianBlueParrot':
      return new NorwegianBlueParrotDelegate(data,
this);
    default:
      return null;
  }
}
```

Das Knifflige ist hier das Entfernen der Unterklassenmethode für `plumage`. Ich versuche Folgendes:

```
class Bird...
get plumage() {
```

```
  if (this._speciesDelegate)
    return this._speciesDelegate.plumage;
  else
    return this._plumage || "average";
}
```

Nun erhalte ich einen Haufen Fehlermeldungen, weil die Delegate-Klassen der anderen Vogelarten keine `plumage`-Eigenschaft besitzen.

Ich könnte eine präzisere Bedingung verwenden:

```
class Bird...
get plumage() {
  if (this._speciesDelegate
           instanceof NorwegianBlueParrotDelegate)
    return this._speciesDelegate.plumage;
  else
    return this._plumage || "average";
}
```

Aber ich hoffe, das riecht für Sie genauso nach »verwesendem Papagei« (Code-Smell) wie für mich. Es ist fast nie eine gute Idee, eine Klasse so wie in diesem Beispiel explizit zu überprüfen.

Eine weitere Option wäre, den Standardfall in den anderen Delegates zu implementieren:

```
class Bird...
get plumage() {
```

```
  if (this._speciesDelegate)
    return this._speciesDelegate.plumage;
  else
    return this._plumage || "average";
}

class EuropeanSwallowDelegate...
get plumage() {
  return this._bird._plumage || "average";
}

class AfricanSwallowDelegate...
get plumage() {
  return this._bird._plumage || "average";
}
```

Dadurch wird allerdings Code der Standardmethode für `plumage` dupliziert, und als ob das nicht allein schon schlimm genug wäre, enthalten auch die Konstruktoren bei der Zuweisung der Rückverweise eine zusätzliche Redundanz.

Diese Redundanz lässt sich selbstverständlich durch Vererbung verhindern. Ich wende *Basisklasse extrahieren* (Abschnitt 12.8) auf die Delegates der Vogelarten an:

```
class SpeciesDelegate {
  constructor(data, bird) {
    this._bird = bird;
  }
  get plumage() {
    return this._bird._plumage || "average";
  }
```

```
class EuropeanSwallowDelegate extends SpeciesDelegate
{

class AfricanSwallowDelegate extends SpeciesDelegate {
  constructor(data, bird) {
    super(data, bird);
    this._numberOfCoconuts = data.numberOfCoconuts;
  }

class NorwegianBlueParrotDelegate extends
SpeciesDelegate {
  constructor(data, bird) {
    super(data, bird);
    this._voltage = data.voltage;
    this._isNailed = data.isNailed;
  }
```

Jetzt verfüge ich über eine Basisklasse und kann beliebiges Standardverhalten von `Bird` nach `SpeciesDelegate` verschieben, indem ich gewährleiste, dass das Feld `speciesDelegate` immer einen Wert enthält:

```
class Bird...
selectSpeciesDelegate(data) {
  switch(data.type) {
    case 'EuropeanSwallow':
      return new EuropeanSwallowDelegate(data, this);
    case 'AfricanSwallow':
      return new AfricanSwallowDelegate(data, this);
    case 'NorwegianBlueParrot':
      return new NorwegianBlueParrotDelegate(data,
this);
    default: return new SpeciesDelegate(data, this);
```

```
  }
}
// weiterer Code ...

get plumage() {return this._speciesDelegate.plumage;}

get airSpeedVelocity() {
  return this._speciesDelegate.airSpeedVelocity;
}

class SpeciesDelegate...
get airSpeedVelocity() {return null;}
```

Mir gefällt das, denn die delegierenden Methoden der Klasse `Bird` werden dadurch vereinfacht. Ich kann leicht erkennen, welches Verhalten delegiert wird und welches verbleibt.

Hier sehen Sie den endgültigen Zustand dieser Klassen:

```
function createBird(data) {
  return new Bird(data);
}

class Bird {
  constructor(data) {
    this._name = data.name;
    this._plumage = data.plumage;
    this._speciesDelegate =
this.selectSpeciesDelegate(data);
  }
  get name() {return this._name;}
  get plumage() {return
this._speciesDelegate.plumage;}
  get airSpeedVelocity() {
```

```
    return this._speciesDelegate.airSpeedVelocity;
  }
  selectSpeciesDelegate(data) {
    switch(data.type) {
      case 'EuropeanSwallow':
        return new EuropeanSwallowDelegate(data,
this);
      case 'AfricanSwallow':
        return new AfricanSwallowDelegate(data, this);
      case 'NorwegianBlueParrot':
        return new NorwegianBlueParrotDelegate(data,
this);
      default: return new SpeciesDelegate(data, this);
    }
  }
  // weiterer Code ...
}

class SpeciesDelegate {
  constructor(data, bird) {
    this._bird = bird;
  }
  get plumage() {
    return this._bird._plumage || "average";
  }
  get airSpeedVelocity() {return null;}
}

class EuropeanSwallowDelegate extends SpeciesDelegate
{
  get airSpeedVelocity() {return 35;}
}

class AfricanSwallowDelegate extends SpeciesDelegate {
  constructor(data, bird) {
    super(data,bird);
    this._numberOfCoconuts = data.numberOfCoconuts;
  }
  get airSpeedVelocity() {
```

```
  return 40 - 2 * this._numberOfCoconuts;
  }
}

class NorwegianBlueParrotDelegate extends
SpeciesDelegate {
  constructor(data, bird) {
    super(data, bird);
    this._voltage = data.voltage;
    this._isNailed = data.isNailed;
  }
  get airSpeedVelocity() {
    return (this._isNailed) ? 0 : 10 + this._voltage /
10;
  }
  get plumage() {
    if (this._voltage > 100) return "scorched";
    else return this._bird._plumage || "beautiful";
  }
}
```

Im Beispiel werden die ursprünglichen Unterklassen durch einen Delegate ersetzt. Die Klasse `SpeciesDelegate` weist aber eine sehr ähnliche Vererbungsstruktur auf. Habe ich durch dieses Refactoring, abgesehen vom Aufheben der Vererbung in der Klasse `Bird`, irgendetwas gewonnen? Die Vererbung ist nun auf einen kleineren Bereich beschränkt und umfasst nur noch die Daten und Funktionen, die in Abhängigkeit der Vogelart variieren. Jeglicher Code, der allen Vogelarten gemein ist, verbleibt in der Klasse `Bird` und in zukünftigen Unterklassen.

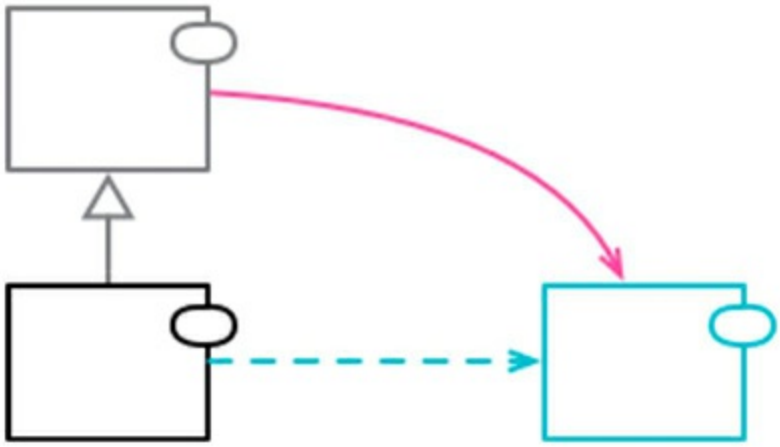
Das Erstellen einer Basisklasse für Delegates könnte ich

auch im Beispiel der Buchungen anwenden. Dieser Schritt würde es mir ermöglichen, die Methoden von `Booking` mit Weiterleitungslogik durch einfache Aufrufe des Delegates zu ersetzen, und für die Weiterleitung wäre die Vererbung verantwortlich. Aber es ist fast Essenszeit, deshalb überlasse ich es dem Leser, das als Übung durchzuführen.

Die Beispiele veranschaulichen, dass die Regel »Bevorzuge Komposition anstelle von Vererbung«, besser lauten sollte: »Bevorzugen Sie eine besonnene Mischung aus Komposition und Vererbung, statt eines von beidem zu bevorzugen« – allerdings befürchte ich, dass dies nicht so griffig ist.

12.11 Basisklasse durch Delegation ersetzen (Replace Superclass with Delegate)

Ehemalige Bezeichnung: *Vererbung durch Delegation ersetzen* (Replace Inheritance with Delegation)



```
class List {...}
class Stack extends List {...}
```

```
class Stack {
  constructor() {
    this._storage = new List();
  }
}
class List {...}
```

12.11.1 MOTIVATION

Bei objektorientierten Programmen bietet die Vererbung eine einfache und leistungsfähige Möglichkeit, vorhandene Funktionalität wiederzuverwenden. Ich lasse von einer vorhandenen Klasse erben und überschreibe Funktionen oder füge weitere Funktionalität hinzu. Die

Unterklassenbildung kann aber auch für Verwirrung sorgen und zu Komplikationen führen.

Einen Stack zu einer Unterklasse einer Liste zu machen, ist ein klassisches Beispiel für die unbedachte Nutzung der Vererbung aus der Frühzeit der objektorientierten Programmierung. Diesem Vorgehen lag die Idee zugrunde, die Speicherfunktionalität und die Bearbeitungsmöglichkeiten einer Liste wiederzuverwenden. Die Wiederverwendung ist durchaus sinnvoll, aber die Vererbung bereitet Probleme: Alle Listenoperationen sind Bestandteil der Schnittstelle des Stacks, obwohl die meisten dieser Operationen auf einen Stack gar nicht anwendbar sind. Ein besserer Ansatz ist es, die Liste zu einem Feld des Stacks zu machen und die erforderlichen Operationen an dieses Feld zu delegieren.

Das ist einer der Gründe dafür, *Basisklasse durch Delegation ersetzen* zu verwenden – wenn die Funktionen der Basisklasse für die Unterklasse keinen Sinn ergeben, ist das ein Hinweis darauf, dass ich keine Vererbung verwenden sollte, um die Funktionalität der Basisklasse zu nutzen.

Die Unterklasse sollte nicht nur alle Funktionen der Basisklasse verwenden, sondern jede Instanz der Unterklasse sollte auch eine Instanz der Basisklasse sein und in allen Situationen eingesetzt werden können, in denen die Basisklasse verwendet wird. Bei einer Klasse

für Modellautos, die Eigenschaften wie Modellbezeichnung und Hubraum besitzt, könnte ich auf den Gedanken kommen, sie zur Repräsentation eines richtigen Fahrzeugs wiederzuverwenden, und Funktionen für die Fahrgestellnummer und das Herstellungsdatum hinzufügen. Hierbei handelt es sich um einen geläufigen und oft subtilen Fehler bei der Modellbildung, den ich als »Typ-Instanz-Homonym« (`https://martinfowler.com/bliki/TypeInstanceHomonym.html`) bezeichne.

Diese Probleme, die für Verwirrung sorgen und zu Fehlern führen, lassen sich leicht umgehen, indem Vererbung durch Delegation an ein separates Objekt ersetzt wird. Die Delegation verdeutlicht, dass es sich um eine andere Sache handelt – eine, auf die nur einige der Funktionen übertragbar sind.

Ich verwende *Basisklasse durch Delegation ersetzen* auch in Fällen, in den die Unterklassenbildung durchaus sinnvoll ist, weil Unterklasse und Basisklasse stark gekoppelt sind und die Unterklasse schnell nicht mehr funktioniert, wenn die Basisklasse geändert wird. Das hat zwar den Nachteil, dass ich für alle Funktionen, die in der Host-Klasse und im Delegate übereinstimmen, Weiterleitungsfunktionen implementieren muss, aber diese sind so einfach, dass ich dabei glücklicherweise nicht viel verkehrt machen kann – auch wenn es langweilig ist, sie zu schreiben.

Aus all diesen Gründen wird manchmal dazu geraten, Vererbung vollständig zu vermeiden, aber das sehe ich anders. Sofern die semantischen Bedingungen erfüllt sind (alle Methoden der Basisklasse sind auf die Unterklasse anwendbar und alle Instanzen der Unterklasse sind Instanzen der Basisklasse), ist Vererbung ein einfacher und effektiver Mechanismus. Sollte sich die Situation ändern, sodass Vererbung nicht mehr die beste Option darstellt, kann ich jederzeit *Basisklasse durch Delegation ersetzen* verwenden. Ich rate also dazu, (meistens) zuerst Vererbung zu verwenden und auf *Basisklasse durch Delegation ersetzen* zurückzugreifen, sobald die Vererbung zum Problem wird.

12.11.2 VORGEHEN

- Erstellen Sie in der Unterklasse ein Feld, das auf das Objekt der Basisklasse verweist. Initialisieren Sie diesen Verweis auf den Delegate mit einer neuen Instanz.
- Erstellen Sie für alle Elemente der Basisklasse in der Unterklasse eine Weiterleitungsfunktion, die den Aufruf an den Delegate weiterreicht. Testen Sie jeweils, nachdem Sie für eine Gruppe zusammengehöriger Elemente Weiterleitungen eingerichtet haben.

  In den meisten Fällen können Sie nach jeder weitergeleiteten Funktion Tests durchführen, aber `get/set`-Paare können beispielsweise erst getestet werden, nachdem beide Funktionen weitergeleitet wurden.
- Wenn alle Elemente der Basisklasse durch Weiterleitungen überschrieben worden sind, können Sie die Vererbung entfernen.

12.11.3 BEISPIEL

Ich habe kürzlich eine Bibliothek beraten, die antike Schriftrollen besitzt. Die Details der Schriftrollen sind in einem Katalog erfasst. Jede Rolle besitzt eine ID, unter der ihr Titel und eine Liste von Schlagwörtern abgelegt werden:

```
class CatalogItem...
constructor(id, title, tags) {
  this._id = id;
  this._title = title;
  this._tags = tags;
}

get id() {return this._id;}
get title() {return this._title;}
hasTag(arg) {return this._tags.includes(arg);}
```

Die Schriftrollen müssen regelmäßig gereinigt werden. Der Code dafür erweitert die Klasse für einen Katalogeintrag (`CatalogItem`) um die Daten, die zur Reinigung notwendig sind:

```
class Scroll extends CatalogItem...
constructor(id, title, tags, dateLastCleaned) {
  super(id, title, tags);
  this._lastCleaned = dateLastCleaned;
}

needsCleaning(targetDate) {
  const threshold = this.hasTag("revered") ? 700 :
1500;
```

```
  return this.daysSinceLastCleaning(targetDate) >
threshold;
}
daysSinceLastCleaning(targetDate) {
  return
this._lastCleaned.until(targetDate,ChronoUnit.DAYS);
}
```

Hier handelt es sich um einen typischen Fehler bei der Modellbildung. Zwischen einer physischen Schriftrolle und einem Katalogeintrag besteht ein Unterschied. Von der Schriftrolle, die beispielsweise die Grauschuppen-Krankheit beschreibt, könnten mehrere Exemplare vorhanden sein, es gibt jedoch nur einen einzigen Katalogeintrag dafür.

In vielen Fällen kann ich mit einem Modellfehler wie diesem davonkommen. Ich könnte weitere Einträge mit Kopien des Titels und der Schlagwörter im Katalog erstellen. Sollten sich die Daten nie ändern, funktioniert diese Art der Repräsentation sogar. Sollten sie sich jedoch ändern, muss ich gewährleisten können, dass alle Kopien eines Katalogeintrags aktualisiert werden.

Auch wenn es dieses Problem nicht gäbe, würde ich die Beziehungen zwischen den Daten dennoch ändern wollen. Einen Katalogeintrag als Basisklasse für Schriftrollen zu verwenden, dürfte künftige Programmierer verwirren. Deshalb ist dieser Modellansatz ungeeignet.

Als Erstes erstelle ich in der Klasse `Scroll` eine Eigenschaft, die auf einen Katalogeintrag verweist, und initialisiere sie mit einer neuen Instanz:

```
class Scroll extends CatalogItem...
constructor(id, title, tags, dateLastCleaned) {
  super(id, title, tags);
  this._catalogItem = new CatalogItem(id, title,
tags);
  this._lastCleaned = dateLastCleaned;
}
```

Dann erstelle ich Methoden zur Weiterleitung für die Elemente der Basisklasse, die ich in der Unterklasse verwende:

```
class Scroll...
get id() {return this._catalogItem.id;}
get title() {return this._catalogItem.title;}
hasTag(aString) {return
this._catalogItem.hasTag(aString);}
```

Anschließend entferne ich die Vererbung:

```
class Scroll extends CatalogItem {
  constructor(id, title, tags, dateLastCleaned) {
    super(id, title, tags);
    this._catalogItem = new CatalogItem(id, title,
tags);
    this._lastCleaned = dateLastCleaned;
  }
```

Mit dem Entfernen der Vererbung ist das grundlegende Refactoring *Basisklasse durch Delegation ersetzen* abgeschlossen, aber in diesem Fall muss ich noch mehr erledigen.

Durch das Refactoring wird der Katalogeintrag zu einer Komponente der Klasse `Scroll`; alle Instanzen von `Scroll` enthalten eine eigene `CatalogItem`-Instanz. In den meisten Fällen ist das schon ausreichend. Hier wäre es jedoch besser, den Katalogeintrag für die Grauschuppen-Krankheit mit denjenigen sechs Rollen in der Bibliothek zu verknüpfen, die Kopien dieser Schriftrolle sind. Dabei handelt es sich im Wesentlichen um eine Anwendung von *Wert durch Referenz ersetzen* (Abschnitt 9.5).

Bevor ich allerdings *Wert durch Referenz ersetzen* anwenden kann, muss ich ein Problem beheben. In der ursprünglichen Vererbungsstruktur verwendet die Klasse `Scroll` das ID-Feld der Klasse `CatalogItem` zum Speichern der ID. Wenn ich jedoch `CatalogItem` als Verweis behandele, muss das ID-Feld für die ID des Katalogeintrags verwendet werden und nicht für die ID der Schriftrolle. Das bedeutet, dass ich in der Klasse `Scroll` ein ID-Feld anlegen und es anstelle des ID-Feldes in der Klasse `CatalogItem` verwenden muss. Ich muss also eine Art Mischung von Verschieben und Aufteilen vornehmen:

```
class Scroll...
constructor(id, title, tags, dateLastCleaned) {
  this._id = id;
  this._catalogItem = new CatalogItem(null, title,
tags);
  this._lastCleaned = dateLastCleaned;
}
get id() {return this._id;}
```

Das Anlegen einer `null`-ID wäre für gewöhnlich ein Warnhinweis und sollte die Alarmglocken klingeln lassen, aber ich verwende sie nur übergangsweise, bis alles in Form gebracht ist. Wenn ich damit fertig bin, werden die `Scroll`-Instanzen einen gemeinsam genutzten Katalogeintrag mit der korrekten ID verwenden.

Aktuell werden die Schriftrollen in einer Einleseroutine initialisiert:

```
Einleseroutine...
const scrolls = aDocument
      .map(record => new Scroll(record.id,

record.catalogData.title,

record.catalogData.tags,

LocalDate.parse(record.lastCleaned)));
```

Der erste Schritt des Refactorings *Wert durch Referenz*

ersetzen (Abschnitt 9.5) besteht darin, ein Repository zu finden oder zu erstellen. Mir steht eins zur Verfügung, das ich in der Einleseroutine problemlos importieren kann. Es enthält Katalogeinträge, die nach IDs indiziert sind. Nun muss ich noch eine Möglichkeit finden, die ID dem Konstruktor der Klasse `Scroll` zu übergeben. Erfreulicherweise ist die ID in den Eingabedaten enthalten, wurde jedoch bislang nicht beachtet, weil sie bei der Verwendung von Vererbung keine Rolle spielt. Nachdem das geklärt ist, kann ich *Funktionsdeklaration ändern* (Abschnitt 6.5) anwenden, um den Parametern des Konstruktors den Katalog und die ID des Katalogeintrags hinzuzufügen:

```
Einleseroutine...
const scrolls = aDocument
      .map(record => new Scroll(record.id,

record.catalogData.title,

record.catalogData.tags,

LocalDate.parse(record.lastCleaned,
                                  record.catalogData.id,
                                  catalog));

class Scroll...
constructor(id, title, tags, dateLastCleaned,
catalogID, catalog) {
  this._id = id;
  this._catalogItem = new CatalogItem(null, title,
tags);
  this._lastCleaned = dateLastCleaned;
}
```

Jetzt ändere ich den Konstruktor, sodass die ID des Katalogeintrags verwendet wird, um einen Eintrag nachzuschlagen, anstatt einen neuen zu erstellen:

```
class Scroll...
constructor(id, title, tags, dateLastCleaned,
catalogID, catalog) {
  this._id = id;
  this._catalogItem = catalog.get(catalogID);
  this._lastCleaned = dateLastCleaned;
}
```

Den Titel und die Schlagwörter, die an den Konstruktor übergeben werden, benötige ich nicht mehr, deshalb verwende ich *Funktionsdeklaration ändern* (Abschnitt 6.5), um sie zu entfernen:

```
Einleseroutine...
const scrolls = aDocument
      .map(record => new Scroll(record.id,

record.catalogData.title,

record.catalogData.tags,

LocalDate.parse(record.lastCleaned,
                                 record.catalogData.id,
                                 catalog));

class Scroll...
constructor(id, title, tags, dateLastCleaned,
```

```
catalogID, catalog) {
  this._id = id;
  this._catalogItem = new CatalogItem(null, title,
tags);
  this._lastCleaned = dateLastCleaned;
}
```

Anhang A

Bibliografie

Eine Online-Version dieser Bibliografie finden Sie unter https://martinfowler.com/books/refactoring-bibliography.html. Viele der Einträge verweisen auf das »Bliki« – einen Bereich der Website martinfowler.com, der kompakte Beschreibungen verschiedener Begriffe enthält, die in der Softwareentwicklung gebräuchlich sind. Beim Schreiben des Buchs habe ich mich entschlossen, die Leser auf diese Erklärungen zu verweisen, statt sie in den Text des Buchs aufzunehmen.

Kapitel 1

Babel. https://babeljs.io.

Kapitel 2

Maudite. https://en.wikipedia.org/wiki/Unibroue.

Martin Fowler. *Bliki: DesignStaminaHypothesis*. https://martinfowler.com/bliki/DesignStaminaHypothesis.html.

Martin Fowler. *Bliki: BranchByAbstraction*. https://martinfowler.com/bliki/BranchByAbstraction.h

tml.

Nicole Forsgren, Jez Humble und Gene Kim. Accelerate: *The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018. ISBN 1942788339.

Martin Fowler. *Bliki: SelfTestingCode*. https://martinfowler.com/bliki/SelfTestingCode.html.

Jay Bazuzi. *Safely Extract a Method in Any C++ Code*. http://jay.bazuzi.com/Safely-extract-a-method-in-any-C++-code/.

Michael Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004. ISBN 0131177052. Deutsche Ausgabe: mitp-Verlag 2010. ISBN 9783826690211.

Pramod Sadalageuand Martin Fowler. *Evolutionary Database Design*. https://martinfowler.com/articles/evodb.html.

Scott W. Ambler und Pramod J. Sadalage. *Refactoring Databases*. Addison-Wesley, 2006. ISBN 0321293533.

Danilo Sato. *Bliki: ParallelChange*. https://martinfowler.com/bliki/ParallelChange.html.

Martin Fowler. *Bliki: Yagni*.

https://martinfowler.com/bliki/Yagni.html.

Neal Ford, Rebecca Parsons und Patrick Kua. *Building Evolutionary Architectures*. O'Reilly, 2017. ISBN 1491986360.

Martin Fowler. *Bliki: ExtremeProgramming*. https://martinfowler.com/bliki/ExtremeProgramming.html.

Martin Fowler. *The New Methodology*. https://martinfowler.com/articles/newMethodology.html.

Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides. Design Patterns: *Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 0201634988. Deutsche Ausgabe: mitp-Verlag 2015. ISBN 9783826697005

William F. Opdyke. *Refactoring Object-Oriented Frameworks*. Doktorarbeit. University of Illinois at Urbana-Champaign, 1992. http://www.laputan.org/pub/papers/opdyke-thesis.pdf.

IntelliJ. https://www.jetbrains.com/idea/.

Eclipse. http://www.eclipse.org.

Language Server. https://langserver.org.

William C. Wake. *Refactoring Workbook*. Addison-Wesley, 2003. ISBN 0321109295.

Joshua Kerievsky. *Refactoring to Patterns*. Addison-Wesley, 2004. ISBN 0321213351.

Elliotte Rusty Harold. *Refactoring HTML*. Addison-Wesley, 2008. ISBN 0321503635.

Jay Fields, Shane Harvie und Martin Fowler. *Refactoring Ruby Edition*. Addison- Wesley, 2009. ISBN 0321603508.

Refactoring, Website zum Buch. `https://refactoring.com`.

Kapitel 3

Martin Fowler. *Bliki: TwoHardThings*. `https://martinfowler.com/bliki/TwoHardThings.html`.

Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides. Design Patterns: *Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 0201634988. Deutsche Ausgabe: mitp-Verlag 2015. ISBN 9783826697005

Kent Beck. *Smalltalk Best Practice Patterns*. Addison-Wesley, 1997. ISBN 013476904X.

Kapitel 4

Martin Fowler. *Bliki: Xunit*. https://martinfowler.com/bliki/Xunit.html.

Martin Fowler. *Bliki: TestDrivenDevelopment*. https://martinfowler.com/bliki/TestDrivenDevelopment.html.

Mocha. https://mochajs.org.

Chai. http://chaijs.com.

Martin Fowler. *Bliki: TestCoverage*. https://martinfowler.com/bliki/TestCoverage.html.

Kapitel 6

Martin Fowler. *Bliki: ClockWrapper*. https://martinfowler.com/bliki/ClockWrapper.html.

Martin Fowler. *Bliki: OverloadedGetterSetter*. https://martinfowler.com/bliki/OverloadedGetterSetter.html.

Martin Fowler. *Bliki: ValueObject*. https://martinfowler.com/bliki/ValueObject.html.

Martin Fowler. *Range*. https://martinfowler.com/eaaDev/Range.html

Martin Fowler. *Bliki: FunctionAsObject*.

https://martinfowler.com/bliki/FunctionAsObject.html.

Martin Fowler. *Bliki: UniformAccessPrinciple*. https://martinfowler.com/bliki/UniformAccessPrinciple.html.

Kapitel 7

D. L. Parnas. *On the Criteria to Be Used in Decomposing Systems into Modules*. In: Communications of the ACM, Band 15, Ausgabe 12, Seiten 1053–1058. Dez. 1972.

Martin Fowler. *Bliki: ListAndHash*. https://martinfowler.com/bliki/ListAndHash.html.

Martin Fowler. *Refactoring Code to Load a Document*. https://martinfowler.com/articles/refactoring-document-load.html.

Martin Fowler. *Collection Pipeline*. https://martinfowler.com/articles/collection-pipeline/.

Martin Fowler. *Bliki: ValueObject*. https://martinfowler.com/bliki/ValueObject.html.

Kapitel 8

Martin Fowler. *Bliki: CommandQuerySeparation*. https://martinfowler.com/bliki/CommandQuerySeparation.html.

Martin Fowler. *Refactoring with Loops and Collection Pipelines*. https://martinfowler.com/articles/refactoring-pipelines.html.

Bill Wake. *The Swap Statement Refactoring*. https://www.industriallogic.com/blog/swap-statement-refactoring/.

Kapitel 9

Martin Fowler. *Repository*. https://martinfowler.com/eaaCatalog/repository.html.

Kapitel 12

Martin Fowler. *Form Template Method*. https://refactoring.com/catalog/formTemplateMethod.html.

Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides. Design Patterns: *Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 0201634988. Deutsche Ausgabe: mitp-Verlag 2015. ISBN 9783826697005

Martin Fowler. *Bliki: TypeInstanceHomonym*. https://martinfowler.com/bliki/TypeInstanceHomonym.html.

Anhang B

Liste der Refactorings

Refactoring	Englische Bezeichnung	Abschnitt
Abfrage durch Parameter ersetzen	Replace Query with Parameter	11.6
Abfrage von Veränderung trennen	Separate Query from Modifier	11.1
Abgeleitete Variable durch Abfrage ersetzen	Replace Derived Variable with Query	9.3
Algorithmus ersetzen	Substitute Algorithm	7.9
Anweisungen in Aufrufer verschieben	Move Statements to Callers	8.4
Anweisungen in Funktion verschieben	Move Statements into Function	8.3
Anweisungen verschieben	Slide Statements	8.6
Assertion einführen	Introduce Assertion	10.6

Basisklasse durch Delegation ersetzen	Replace Superclass with Delegate	12.11
Basisklasse extrahieren	Extract Superclass	12.8
Bedingte Anweisung zerlegen	Decompose Conditional	10.1
Bedingten Ausdruck zusammenfassen	Consolidate Conditional Expression	10.2
Bedingung durch Polymorphie ersetzen	Replace Conditional with Polymorphism	10.4
Befehl durch Funktion ersetzen	Replace Command with Function	11.10
Collection kapseln	Encapsulate Collection	7.2
Datensatz kapseln	Encapsulate Record	7.1
Delegation verbergen	Hide Delegate	7.7
Elementaren Wert durch Objekt ersetzen	Replace Primitive with Object	7.3
Feld nach unten verschieben	Push Down Field	12.5
Feld umbenennen	Rename Field	9.2

Feld verschieben	Move Field	8.2
Funktion durch Befehl ersetzen	Replace Function with Command	11.9
Funktion extrahieren	Extract Function	6.1
Funktion inline platzieren	Inline Function	6.2
Funktion parametrisieren	Parameterize Function	11.2
Funktion verschieben	Move Function	8.1
Funktionen zu einer Klasse vereinen	Combine Functions into Class	6.9
Funktionen zu einer Transformation vereinen	Combine Functions into Transform	6.10
Funktionsdeklaration ändern	Change Function Declaration	6.5
Hierarchie abbauen	Collapse Hierarchy	12.9
Inline-Code durch Funktionsaufruf ersetzen	Replace Inline Code with Function Call	8.5
Klasse extrahieren	Extract Class	7.5
Klasse inline platzieren	Inline Class	7.6

Konstruktor durch Fabrikfunktion ersetzen	Replace Constructor with Factory Function	11.8
Konstruktorrumpf nach oben verschieben	Pull Up Constructor Body	12.3
Methode nach oben verschieben	Pull Up Method	12.1
Methode nach unten verschieben	Push Down Method	12.4
Parameter durch Abfrage ersetzen	Replace Parameter with Query	11.5
Parameterobjekt einführen	Introduce Parameter Object	6.8
Phase aufteilen	Split Phase	6.11
Referenz durch Wert ersetzen	Change Reference to Value	9.4
Schleife aufteilen	Split Loop	8.7
Schleife durch Pipeline ersetzen	Replace Loop with Pipeline	8.8
Setter entfernen	Remove Setting Method	11.7
Sonderfall einführen	Introduce Special Case	10.5

Steuerungs-Flag entfernen	Remove Flag Argument	11.3
Temporäre Variable durch Abfrage ersetzen	Replace Temp with Query	7.4
Toten Code entfernen	Remove Dead Code	8.9
Typenschlüssel durch Unterklassen ersetzen	Replace Type Code with Subclasses	12.6
Unterklasse durch Delegation ersetzen	Replace Subclass with Delegate	12.10
Unterklasse entfernen	Remove Subclass	12.7
Variable aufteilen	Split Variable	9.1
Variable extrahieren	Extract Variable	6.3
Variable inline platzieren	Inline Variable	6.4
Variable kapseln	Encapsulate Variable	6.6
Variable umbenennen	Rename Variable	6.7
Vermittler entfernen	Remove Middle Man	7.8
Verschachtelte Bedingungen durch Wächterbedingung ersetzen	Replace Nested Conditional with Guard Clauses	10.3

Vollständiges Objekt erhalten	Preserve Whole Object	11.4
Wert durch Referenz ersetzen	Change Value to Reference	9.5

Anhang C

Code-Smells

Code-Smell	Gängige Refactorings
Alternative Klassen mit unterschiedlichen Schnittstell	Funktionsdeklaration ändern (Change Function Declaration, Abschnitt 6.5), Funktion verschieben (Move Function, Abschnitt 8.1), Basisklasse extrahieren (Extract Superclass, Abschnitt 12.8)

en	
Ausgeschlagenes Erbe	Methode nach unten verschieben (Push Down Method, Abschnitt 12.4), Feld nach unten verschieben (Push Down Field, Abschnitt 12.5), Unterklasse durch Delegation ersetzen (Replace Subclass with Delegate, Abschnitt 12.10), Basisklasse durch Delegation ersetzen (Replace Superclass with Delegate, Abschnitt 12.11)
Datenklasse	Datensatz kapseln (Encapsulate Record, Abschnitt 7.1), Setter entfernen (Remove Setting Method, Abschnitt 11.7), Funktion verschieben (Move Function, Abschnitt 8.1), Funktion extrahieren (Extract Function, Abschnitt 6.1), Phase aufteilen (Split Phase, Abschnitt 6.11)
Datenklumpen	Klasse extrahieren (Extract Class, Abschnitt 7.5), Parameterobjekt einführen (Introduce Parameter Object, Abschnitt 6.8), Vollständiges Objekt erhalten (Preserve Whole Object, Abschnitt 11.4)
Divergierende Änderungen	Phase aufteilen (Split Phase, Abschnitt 6.11), Funktion verschieben (Move Function, Abschnitt 8.1), Funktion extrahieren (Extract Function, Abschnitt 6.1), Klasse extrahieren (Extract Class, Abschnitt 7.5)

Chirurgie mit der Schrotflinte	Feld verschieben (Move Field, Abschnitt 8.2), Funktionen zu einer Klasse vereinen (Combine Functions into Class, Abschnitt 6.9), Funktionen zu einer Transformation vereinen (Combine Functions into Transform, Abschnitt 6.10), Phase aufteilen (Split Phase, Abschnitt 6.11), Funktion inline platzieren (Inline Function, Abschnitt 6.2), Klasse inline platzieren (Inline Class, Abschnitt 7.6)
Feature-Neid	Funktion verschieben (Move Function, Abschnitt 8.1), Funktion extrahieren (Extract Function, Abschnitt 6.1)
Globale Daten	Variable kapseln (Encapsulate Variable, Abschnitt 6.6)
Insiderhandel	Funktion verschieben (Move Function, Abschnitt 8.1), Feld verschieben (Move Field, Abschnitt 8.2), Delegation verbergen (Hide Delegate, Abschnitt 7.7), Unterklasse durch Delegation ersetzen (Replace Subclass with Delegate, Abschnitt 12.10), Basisklasse durch Delegation ersetzen (Replace Superclass with Delegate, Abschnitt 12.11)
Kommentar	Funktion extrahieren (Extract Function, Abschnitt 6.1), Funktionsdeklaration ändern (Change Function Declaration, Abschnitt 6.5), Assertion einführen (Introduce Assertion, Abschnitt 10.6)

re	
Lange Funktion	Funktion extrahieren (Extract Function, Abschnitt 6.1), Temporäre Variable durch Abfrage ersetzen (Replace Temp with Query, Abschnitt 7.4), Parameterobjekt einführen (Introduce Parameter Object, Abschnitt 6.8), Vollständiges Objekt erhalten (Preserve Whole Object, Abschnitt 11.4), Funktion durch Befehl ersetzen (Replace Function with Command, Abschnitt 11.9), Bedingung zerlegen (Decompose Conditional, Abschnitt 10.1), Bedingung durch Polymorphie ersetzen (Replace Conditional with Polymorphism, Abschnitt 10.4), Schleife aufteilen (Split Loop, Abschnitt 8.7)
Lange Parameterliste	Parameter durch Abfrage ersetzen (Replace Parameter with Query, Abschnitt 11.5), Vollständiges Objekt erhalten (Preserve Whole Object, Abschnitt 11.4), Parameterobjekt einführen (Introduce Parameter Object, Abschnitt 6.8), Steuerungs-Flag entfernen (Remove Flag Argument, Abschnitt 11.3), Funktionen zu einer Klasse vereinen (Combine Functions into Class, Abschnitt 6.9)
Mitteilungsketten	Delegation verbergen (Hide Delegate, Abschnitt 7.7), Funktion extrahieren (Extract Function, Abschnitt 6.1), Funktion verschieben (Move Function, Abschnitt 8.1)
Obsession für elme	Elementaren Wert durch Objekt ersetzen (Replace Primitive with Object, Abschnitt 7.3), Typenschlüssel durch Unterklassen ersetzen (Replace Type Code with Subclasses, Abschnitt 12.6), Bedingung durch Polymorphie ersetzen (Replace Conditional with Polymorphism, Abschnitt 10.4), Klasse extrahieren (Extract Class, Abschnitt 7.5), Parameterobjekt einführen (Introduce Parameter Object, Abschnitt 6.8)

ntare Datentypen	
Rätselhafte Bezeichnung	Funktionsdeklaration ändern (Change Function Declaration, Abschnitt 6.5), Variable umbenennen (Rename Variable, Abschnitt 6.7), Feld umbenennen (Rename Field, Abschnitt 9.2)
Redundanter Code	Funktion extrahieren (Extract Function, Abschnitt 6.1), Anweisungen verschieben (Slide Statements, Abschnitt 8.6), Methode nach oben verschieben (Pull Up Method, Abschnitt 12.1)
Schleifen	Schleife durch Pipeline ersetzen (Replace Loop with Pipeline, Abschnitt 8.8)
Spekulative Ge	Hierarchie abbauen (Collapse Hierarchy, Abschnitt 12.9), Funktion inline platzieren (Inline Function, Abschnitt 6.2), Klasse inline platzieren (Inline Class, Abschnitt 7.6), Funktionsdeklaration ändern (Change Function Declaration, Abschnitt 6.5), Toten Code entfernen (Remove Dead Code, Abschnitt 8.9)

neralisierung	
Temporäres Feld	Klasse extrahieren (Extract Class, Abschnitt 7.5), Funktion verschieben (Move Function, Abschnitt 8.1), Sonderfall einführen (Introduce Special Case, Abschnitt 10.5)
Träges Element	Funktion inline platzieren (Inline Function, Abschnitt 6.2), Klasse inline platzieren (Inline Class, Abschnitt 7.6), Hierarchie abbauen (Collapse Hierarchy, Abschnitt 12.9)
Umfangreiche Klasse	Klasse extrahieren (Extract Class, Abschnitt 7.5), Basisklasse extrahieren (Extract Superclass, Abschnitt 12.8), Typenschlüssel durch Unterklassen ersetzen (Replace Type Code with Subclasses, Abschnitt 12.6)
Veränderlich	Variable kapseln (Encapsulate Variable, Abschnitt 6.6), Variable aufteilen (Split Variable, Abschnitt 9.1), Anweisungen verschieben (Slide Statements, Abschnitt 8.6), Funktion extrahieren (Extract Function, Abschnitt 6.1), Abfrage von Veränderung trennen (Separate Query from Modifier, Abschnitt 11.1), Setter entfernen

e Da ten	(Remove Setting Method, Abschnitt 11.7), Abgeleitete Variable durch Abfrage ersetzen (Replace Derived Variable with Query, Abschnitt 9.3), Funktionen zu einer Klasse vereinen (Combine Functions into Class, Abschnitt 6.9), Funktionen zu einer Transformation vereinen (Combine Functions into Transform, Abschnitt 6.10), Referenz durch Wert ersetzen (Change Reference to Value, Abschnitt 9.4)
Ve rm ittl er	Vermittler entfernen (Remove Middle Man, Abschnitt 7.8), Funktion inline platzieren (Inline Function, Abschnitt 6.2), Basisklasse durch Delegation ersetzen (Replace Superclass with Delegate, Abschnitt 12.11), Unterklasse durch Delegation ersetzen (Replace Subclass with Delegate, Abschnitt 12.10)
Wi ed er hol te swi tch - An we isu ng en	Bedingung durch Polymorphie ersetzen (Replace Conditional with Polymorphism, Abschnitt 10.4)